© 2026 Sreenidhi Rajakrishnan · All rights reserved.
For personal use. Not to be redistributed.

THE 2026 EDITION

The Ultimate Q&A Interview Kit Playwright TypeScript 2026 (Expanded)

Concept · Interview Strategy · Code

Built for the 2026 Indian SDET market.

EXPANDED EDITION

TypeScript, Architecture, Locators, Auto-waiting, POM, Network, API, CI/CD, Docker, AI, MCP and Agents

325 questions · 28 sections · 140 + 30 quiz MCQs

by Sreenidhi Rajakrishnan
Automation Architect · QA and SDET Educator

Edition 1.0 · July 2026

Table of Contents

325 questions across 28 sections, organised by tier so you can work the kit in your own order.

FOUNDATIONAL · the bread and butter

Section 01 TypeScript Fundamentals for SDETs (18 Q)
Section 02 Playwright Architecture and Setup (14 Q)
Section 03 Locators and DOM Strategies (22 Q)
Section 04 Auto-waiting and Synchronisation (17 Q)
Section 05 Page Object Model in Playwright (10 Q)
Section 08 Assertions and the Expect API (14 Q)

CORE · the day-to-day senior topics

Section 06 Network Interception and Mocking (16 Q)
Section 07 API Testing with Playwright (18 Q)
Section 09 Parallel Execution and Sharding (10 Q)
Section 10 CI/CD Integration (14 Q)
Section 11 Debugging and Tracing (11 Q)
Section 12 Reporting (8 Q)
Section 13 Configuration and Fixtures (15 Q)
Section 14 Advanced Playwright Patterns (11 Q)

STRATEGIC, ADVANCED, AND ROLE-LEVEL

Section 15 AI for SDET Engineers (10 Q)
Section 16 Test Design Techniques (8 Q)
Section 17 QA Fundamentals and Test Strategy (8 Q)
Section 18 Mobile Testing with Playwright (8 Q)
Section 19 Microservices and API Architecture (8 Q)
Section 20 Security and Performance Testing (9 Q)
Section 22 Advanced TypeScript for SDETs (7 Q)
Section 23 Docker and Containerised Testing (7 Q)
Section 24 Database and SQL Testing (7 Q)
Section 25 Git and Version Control (5 Q)
Section 26 Playwright vs Other Frameworks (7 Q)

BEHAVIOURAL AND 2026 FRONTIER

Section 21 Behavioural and Soft Skills (8 Q)
Section 27 AI, MCP and Agentic Testing (10 Q)

Section 28 Real-Time Interview Scenarios (25 Q)

Each section opens with an intro and a short bullet outline, then 5-28 questions in three-pill format (Concept · Interview Strategy · Code). A 5-question MCQ quiz closes every section. The kit ends with a 30-question Final Mock Quiz drawing from every section.

HOW TO USE THIS KIT

Read it like an interviewer would

This is not a cheat sheet to skim the night before. Every answer is built so you can **say it out loud** in a real interview and sound like someone who has shipped Playwright suites, not someone who memorised bullet points. Here is how the pieces fit together.

The three-pill card

- **Concept**: the clean, correct mental model. This is the answer you give first.
- **Interview Strategy**: what the interviewer is really probing, the follow-up they will ask, and how to steer the conversation to your strengths.
- **Code**: the TypeScript that proves you have actually done it. Read it, type it, run it.

The level badge

Each question is tagged with the seniority it usually shows up at: **J** Junior, **JM** Junior to Mid, **M** Mid, **MS** Mid to Senior, **S** Senior, **SA** Senior to Architect, **A** Architect. Use it to calibrate: if you are targeting a Senior role, you should be fluent through S and at least comfortable with SA and A.

Quizzes, answer key, and cross-references

- Every section closes with a **5-question MCQ quiz**. Attempt it before peeking: the **answer key** directly underneath explains why each option is right or wrong.
- Answer keys and questions use a **"See Q"** mechanic: when a point connects to another card, it points you to that question number so you can build the full picture.
- The kit ends with a **30-question Final Mock Quiz** drawn from every section: treat it as a timed dry run.

Suggested order

Work the tiers in the order the Table of Contents lists them: **Foundational** to lock in the bread-and-butter, then **Core** for the day-to-day senior topics, then **Strategic / Advanced / Role-level**, and finally **Behavioural and 2026 Frontier**. If you are interviewing this week, skim every Interview Strategy pill first, then go deep where you feel shaky.

SECTION 01

TypeScript Fundamentals for SDETs

Almost every Playwright role expects TypeScript, not plain JavaScript, and interviewers open here to see whether you actually understand the language or just copy snippets. These eighteen questions cover the type-system fundamentals that show up in real frameworks, interfaces, generics, enums, strict mode, async/await, plus the tooling (ESLint, tsc, path aliases) that keeps a large suite maintainable. Answer these well and you signal that your tests are engineered, not scripted.

In this section

- Why TypeScript is the default for Playwright in 2026, and what it buys you.
- Interfaces vs types, and the Omit/Partial/Pick patterns that keep contracts honest.
- async/await and why a single missing await is the most common Playwright bug.
- Generics for typed API clients and reusable test-data factories.
- Enums, utility types, and Record maps for exhaustive role/credential handling.
- Strict mode, strictNullChecks, and catching undefined before it reaches CI.
- Path aliases, a centralised types directory, and tsc --noEmit as a CI gate.

Q 1 LEVEL · Junior

Why is TypeScript the preferred language for Playwright automation?

CONCEPT

TypeScript extends JavaScript with **static typing**, interfaces, generics, enums, and compile-time error detection. In an automation framework that translates into four concrete wins. Type errors surface at compile time, before a single browser launches. Playwright ships full type definitions, so the entire API surface autocompletes in the editor, you see the exact signature of **page.route** as you type it. Large suites stay maintainable because contracts are explicit: change an interface and the compiler shows every call site that breaks. And the code is **self-documenting**, which cuts onboarding time. The 2026 reality is that almost every Playwright job advert assumes TypeScript; plain JavaScript is now the exception, not the norm.

INTERVIEW STRATEGY

This looks like a softball, but the interviewer is checking whether you can connect a language choice to engineering outcomes rather than reciting features. Structure it as benefits, not a feature list: tie each property of TypeScript to a concrete testing payoff, autocomplete to speed, compile checks to CI cost, types to maintainability. The failure mode is answering 'because it is type-safe' and stopping; that reads junior. The follow-up is almost always 'where do you catch those type errors?', so plant tsc --noEmit and no-floating-promises early and you answer it before it is asked.

How to phrase it

> *TypeScript is effectively the default for Playwright now, and I prefer it for reasons that show up in a real framework, not just on paper. The first is autocomplete. Playwright ships its own type definitions, so when I type page.route I see the exact signature and return type without a documentation lookup, and on a large codebase that easily saves twenty to thirty percent of the time I would lose hunting through docs. The second is compile-time safety, and this is where I would pre-empt the obvious follow-up about where errors get caught: I run tsc --noEmit as the very first step in CI, and it flags type errors in about three seconds, before a single browser starts. That is shift-left applied to type safety: the cheapest possible stage. The third is maintainability. When the backend changes a response shape, the compiler shows me every test that touches the removed field instead of letting it fail silently at runtime hours later. I also enforce the no-floating-promises ESLint rule, which catches missing await keywords, the single most common Playwright bug. So for me TypeScript is not a preference, it is the baseline for any framework I expect to maintain in 2026.*

Key points to hit

(1) Frame it as outcomes: autocomplete -> speed, types -> maintainability. **(2)** Autocomplete from Playwright's type defs cuts real development time. **(3)** tsc --noEmit as the first CI step catches type errors in seconds. **(4)** Contract changes surface at compile time, not as silent runtime failures. **(5)** Trap to avoid: stopping at 'it is type-safe', too junior. **(6)** Follow-up to pre-empt: where you catch errors, name tsc and no-floating-promises.

CODE

```
// JavaScript: wrong types accepted silently
async function login(page, email, password) {
await page.locator('#email').fill(email);
}
login(page, 12345, true); // no error; bug ships

// TypeScript: caught at compile time
import { Page } from '@playwright/test';
async function login(page: Page, email: string, password: string): Promise<void> {
await page.getByLabel('Email').fill(email);
}
login(page, 12345, true); // error: number not assignable to string
```

Q 2 LEVEL · Junior to Mid

What is the difference between interfaces and types, and when do you use each?

CONCEPT

Both describe the shape of data, and for plain object shapes they are largely interchangeable. The differences that matter in a framework: **interfaces** support extends and implements and can merge declarations, which makes them the natural fit for page-object contracts, service layers, and fixture types. **Type aliases** are more flexible for unions, intersections, and mapped transformations, string unions like an environment list, or utility-type derivations. The high-value habit is deriving types from a single source of truth: define one response interface, then use Omit and **Partial** to derive the create and update request shapes. Change the source interface once and every derived type updates with it, so a backend contract change cannot drift silently across the suite.

INTERVIEW STRATEGY

They want to know if you have a working rule or are just reciting a docs table. Give the rule fast, interfaces for things classes extend or implement, types for unions and derivations, then immediately move past trivia to the part that signals framework experience: deriving request shapes from one response interface with Omit and Partial. The classic error is getting pulled into obscure declaration-merging details, which wastes the chance to show impact. Expect the follow-up ‘what happens when the API adds a required field?’, answer it with the one-edit, compiler-fans-out story.

How to phrase it

> *In practice I use interfaces for anything a class extends or implements, page objects, service layers, fixture types, because interfaces support extends and implements cleanly. I use type aliases for string unions and utility-type derivations. But the part that actually matters in a framework is deriving from one source of truth, so let me show that rather than list differences. I define the UserResponse interface once, matching the API schema exactly. From it I derive the create-request type by omitting id and createdAt, and the update-request type as a Partial of that. Now to the follow-up that always comes: what happens when the backend adds a new required field. Because everything is derived, I change the one interface and TypeScript immediately flags every test that now has to send that field. Without that, a contract change is invisible during development and only shows up as scattered runtime failures across a dozen files, which is exactly the kind of bug that survives to staging. So my rule is simple: model the domain once, derive everything else, and let the compiler keep the tests honest when the API moves underneath you.*

Key points to hit

(1) Rule: interfaces for extends/implements; types for unions and derivations. **(2)** Define a response shape once; derive create/update with Omit and Partial. **(3)** One interface change flags every affected test at compile time. **(4)** Prevents silent drift when an API contract changes. **(5)** The common miss: getting lost in declaration-merging trivia instead of showing impact. **(6)** Asked next: a new required field, demonstrate the one-edit compiler fan-out.

CODE

```
interface BasePage {
navigate(url: string): Promise<void>;
waitForLoad(): Promise<void>;
}
interface LoginPage extends BasePage {
login(credentials: UserCredentials): Promise<void>;
}

type Environment = 'dev' | 'staging' | 'production';

interface UserResponse {
id: number; name: string; email: string; createdAt: string;
}
type CreateUserRequest = Omit<UserResponse, 'id' | 'createdAt'>;
type UpdateUserRequest = Partial<CreateUserRequest>;
```

Q 3 LEVEL · Junior

How do async/await work, and why does a missing await break a Playwright test?

CONCEPT

Every Playwright browser interaction is asynchronous and returns a **Promise**. The async/await syntax lets you write those Promise chains as readable, sequential steps while the runtime still works asynchronously underneath. Forgetting an await is the most common Playwright bug: the

operation fires but the test does not wait for it to finish, so the next line runs against an unsettled DOM and you get intermittent, hard-to-reproduce failures. A related advanced pattern is **Promise.all** for capturing a network response that a UI action triggers, you must start **waitForResponse** before the click, because Playwright can only capture a response it was already listening for. The 2026 guard rail is the no-floating-promises ESLint rule, which flags a missing await at lint time, long before CI runs.

INTERVIEW STRATEGY

This is a flakiness question in disguise; the interviewer is testing whether you understand why tests fail intermittently, not whether you know what await does. Lead with the missing-await failure mode and say plainly it is the most common Playwright bug, then show you prevent it rather than chase it by naming the ESLint rule. To separate yourself, escalate to the Promise.all-before-click pattern. The failure mode is describing await mechanically; the follow-up is usually 'how do you capture the API call a button triggers?', which the Promise.all answer covers directly.

How to phrase it

> *Every Playwright action returns a Promise, and async/await just lets me write them as clean sequential steps while the runtime stays asynchronous underneath. The reason this question matters is flakiness. The classic bug is a missing await: the action fires but the test does not wait, so the next line runs against a DOM that has not settled, and you get a failure that reproduces maybe one run in twenty and is miserable to debug. It is the first thing I look for in code review. But I would rather prevent it than hunt it, so I turn on the no-floating-promises ESLint rule, which flags a missing await at lint time, before CI even runs. Then, to answer the follow-up that always comes, how do you capture the API call a button triggers, I use Promise.all with waitForResponse. The non-obvious part is ordering: I start waiting for the response before the click, because Playwright can only capture a response it was already listening for. That single pattern lets me assert that clicking a button fires the right request with the right method and status. So the headline is: await everything, let the linter enforce it, and know the Promise.all ordering cold.*

Key points to hit

(1) Every Playwright action returns a Promise; await sequences them. **(2)** Missing await runs the next line against an unsettled DOM, flakiness. **(3)** Prevent, do not chase: no-floating-promises catches it at lint time. **(4)** Promise.all: start waitForResponse BEFORE the click that triggers it. **(5)** You can only capture a response you were already listening for. **(6)** Follow-up: capturing a triggered API call, the Promise.all pattern answers it.

CODE

```
// Broken: nothing is awaited
test('broken', async ({ page }) => {
page.goto('/login'); // fires, not awaited
page.fill('#email', 'admin'); // race condition
});

// Correct: capture the API call the click triggers
const [response] = await Promise.all([
page.waitForResponse('**/api/login'), // listen first
page.getByRole('button', { name: 'Sign in' }).click(),
]);
expect(response.status()).toBe(200);
```

Q 4 LEVEL · Mid

What are TypeScript generics and how do you use them in a Playwright framework?

CONCEPT

Generics let a function or class work over a **type supplied by the caller** while keeping full type safety. In a framework the highest-value use is a **typed API client**: one function serves every endpoint, and the caller declares the response type so the result is fully typed downstream. The second common use is a **test-data factory** that takes a defaults object and an optional partial override, returning the same type, so each test specifies only the fields that matter to it. The payoff is that when a response shape changes, every typed call site lights up at compile time instead of failing as a vague runtime error. Generics are what let you stay DRY without throwing away type safety.

INTERVIEW STRATEGY

The question digs at whether you can apply a language feature to a real framework problem, not define it. Do not explain what a generic is in the abstract, anchor immediately on the typed API client, and make the trade-off explicit by naming the alternatives you are avoiding: a function per response type, or casting to any and losing safety. That comparison is what separates junior from senior thinking. The classic error is a textbook ‘a generic is a type parameter’ definition. Expect the follow-up ‘where else do you use them?’, have the test-data factory ready as your second example.

How to phrase it

> *Generics let me write one function that works across types while keeping everything fully typed, and I would rather show that than define it abstractly. The piece I rely on most is a generic API client, a single apiGet function that takes a response type parameter, so calling it for a user gives me a typed user back and calling it for an order gives me a typed order. The reason this is the right design is the alternative: without generics I would need either a separate function per response type, which is a maintenance nightmare, or I cast everything to any and throw away the type safety that justified using TypeScript in the first place. Because the responses stay typed, when the backend removes a field the compiler shows me every test that read it. For the follow-up about where else I use generics, my second example is the test-data factory: it takes a defaults object and an optional Partial override, so a test specifies only the fields it actually cares about and the rest fall back to sensible defaults. That makes the test intent obvious: you read the override and you know exactly what the test exercises. So generics are how I stay DRY without giving up the safety that makes the whole framework worth typing.*

Key points to hit

(1) Open on the typed API client, not an abstract definition. **(2)** Name the alternatives avoided: function-per-type, or casting to any. **(3)** One typed client serves every endpoint with a typed response. **(4)** Response-shape changes light up every typed call site at compile time. **(5)** Second example for the follow-up: a generic factory with Partial overrides. **(6)** Avoid: reciting 'a generic is a type parameter' with no application.

CODE

```
import { APIRequestContext, expect } from '@playwright/test';

async function apiGet<TResponse>(
request: APIRequestContext,
endpoint: string,
): Promise<TResponse> {
const response = await request.get(endpoint);
expect(response.ok()).toBeTruthy();
return (await response.json()) as TResponse;
}

interface User { id: number; name: string; email: string; }
const user = await apiGet<User>(request, '/users/1');
console.log(user.name); // typed: TS knows .name exists

function createTestData<T>(defaults: T, overrides?: Partial<T>): T {
return { ...defaults, ...overrides };
}
```

Q 5 LEVEL · Junior to Mid

What are TypeScript enums and utility types, and how do they help in automation?

CONCEPT

Enums replace magic strings with named constants, which kills the ‘admin’ versus ‘Admin’ class of typo bugs. Utility types transform existing interfaces so you do not duplicate shapes: Partial makes every field optional, **Omit** removes named fields, Pick selects named fields, and Record builds a typed key-value map. The standout pattern for test data is a Record keyed by an enum, for example a credentials map keyed by user role. It enforces **exhaustiveness**: add a new role to the enum and TypeScript immediately reports that the credentials map is missing an entry. That is impossible to get from a plain object, and it is exactly the safety you want when roles drive a large test matrix.

INTERVIEW STRATEGY

The point of this one is to see you reach for the type system to prevent test-data bugs, not just to annotate variables. Pair the two ideas tightly, enums kill magic-string typos, utility types kill duplication, then land on the detail that separates seniors: a Record keyed by an enum gives compile-time exhaustiveness. The trap is treating enums as cosmetic; the value is the bug class they remove. The follow-up is usually ‘how do you keep credentials or test data in sync as the app grows?’, and the exhaustive Record is precisely that answer.

How to phrase it

> *Enums and utility types are how I keep test data both readable and impossible to leave half-updated. Enums let me name the values that matter, user roles, order statuses, instead of scattering string literals, and that removed a whole class of bugs in our framework where someone wrote admin with a capital A in one place and lower-case in another. Utility types stop me duplicating shapes: Partial, Omit, Pick and Record all derive from an interface I have already defined. My favourite pattern, and the one I would use to answer the inevitable follow-up about keeping test data in sync, is a Record keyed by an enum for credentials. Because the key is the role enum, TypeScript enforces exhaustiveness: the moment I add a new role to the enum, the compiler tells me the credentials map is missing that role. A plain object would just silently lack the entry and I would discover it at runtime when a test for the new role fails for the wrong reason. I call this exhaustiveness checking for test data, it turns adding a role from a risky, easy-to-forget change into a safe one the compiler walks me through. So enums plus utility types are a small investment that pays off every time the domain grows.*

Key points to hit

(1) Enums replace magic strings: no more 'admin' vs 'Admin' typos. **(2)** Partial / Omit / Pick / Record derive shapes instead of duplicating. **(3)** Record keyed by an enum enforces exhaustiveness on test data. **(4)** Add a role to the enum and the compiler flags the missing map entry. **(5)** The easy mistake: treating enums as cosmetic rather than as bug-class removal. **(6)** Expect next: keeping test data in sync as the app grows, the exhaustive Record.

CODE

```
enum UserRole { Admin = 'admin', User = 'user', Viewer = 'viewer' }

interface User {
id: number; name: string; email: string; role: UserRole;
}
type CreateUser = Omit<User, 'id'>;
type UserSummary = Pick<User, 'id' | 'name' | 'email'>;

// Record keyed by the enum: must cover every role
const credentials: Record<UserRole, { email: string; password: string }> = {
[UserRole.Admin]: { email: 'admin@test.com', password: 'Admin@123' },
[UserRole.User]: { email: 'user@test.com', password: 'User@123' },
[UserRole.Viewer]: { email: 'viewer@test.com', password: 'Viewer@123' },
};
```

Q 6 LEVEL · Mid

What is TypeScript strict mode and why is it essential for automation projects?

CONCEPT

Setting strict: true in tsconfig.json turns on a family of checks, **strictNullChecks**, **noImplicitAny**, **strictFunctionTypes**, and strictPropertyInitialization. The one that pays off most in test code is strictNullChecks: it forces you to handle null and undefined explicitly at compile time, so a lookup that can return undefined cannot be dereferenced without a guard. In fixtures this is especially valuable for environment variables, which are typed as possibly-undefined, strict mode makes you validate them and throw a clear error rather than letting an undefined value flow into a test and crash cryptically later. Strict mode is non-negotiable on any framework I expect to maintain; turning it off trades a few minutes now for confusing failures later.

INTERVIEW STRATEGY

Anyone can list the strict flags; the interviewer wants to know if you have felt the pain that strict mode prevents. Define it as a bundle of checks in one line, then spend your time on strictNullChecks with a real incident, a war story beats a flag list every time. The classic error is reciting flag names with no consequence attached. Expect the follow-up 'isn't strict mode annoying / slowing you down?', so close on the framing that it converts cryptic runtime crashes into actionable compile errors, which is a net time saving.

How to phrase it

> *Strict mode is a set of compiler checks you switch on with strict true, strictNullChecks, noImplicitAny and a few others, but rather than list them I would explain the one that earns its keep in test code. strictNullChecks forces me to handle undefined explicitly, and it actually caught a production-class issue for us. We had an optional environment variable that silently defaulted to undefined when it was not set. Tests passed locally because we had it set, then failed in CI with a cryptic undefined-property error deep inside a test, and it took an afternoon to trace. With strictNullChecks, TypeScript flagged every place we read that variable without a null check. We added explicit validation, so now if it is missing the run aborts immediately with a message naming the variable. That is the whole point, and it answers the follow-up about strict mode being annoying: it moves the failure from a mysterious runtime crash to an actionable compile error, which saves far more time than the few extra guards cost. I treat strict mode as mandatory on any framework I have to maintain, turning it off just defers the bug to a worse moment.*

Key points to hit

(1) strict: true bundles strictNullChecks, noImplicitAny and more. **(2)** strictNullChecks forces explicit null/undefined handling at compile time. **(3)** Env vars are typed as possibly-undefined: must be validated. **(4)** Anchor on a real incident, not a flag list. **(5)** Trap: naming flags with no consequence attached. **(6)** They usually ask next: 'isn't it annoying?', cryptic crash becomes actionable compile error.

CODE

```
// Without strictNullChecks: crashes if undefined
const user = findUser(id); // User | undefined
console.log(user.name); // runtime crash

// With strictNullChecks: guard required
const found = findUser(id);
if (!found) throw new Error(`User ${id} not found`);
console.log(found.name); // safe: narrowed to User

// Fixtures: validate env vars explicitly
export const test = base.extend<{ adminEmail: string }>({
adminEmail: async ({}, use) => {
const email = process.env.ADMIN_EMAIL; // string | undefined
if (!email) throw new Error('ADMIN_EMAIL is required');
await use(email);
},
});
```

Q 7 LEVEL · Junior to Mid

How do you configure path aliases and project structure for a large Playwright framework?

CONCEPT

Path aliases in tsconfig.json map a short prefix to a real directory, so imports become **location-independent**. Instead of fragile relative paths that climb several levels and break the

moment you move a file, you import from a stable alias like a pages or data prefix. Aliases do two things: they survive file restructuring, and they communicate intent, a pages prefix always means page objects, a data prefix always means factories. New engineers know where to put a new file without asking. Pair aliases with a **centralised types directory** so one interface change propagates everywhere with the compiler's help. Note that the test runner needs the alias resolved at runtime too, a small resolver such as tsconfig-paths or an equivalent config keeps editor and run-time in agreement.

INTERVIEW STRATEGY

This question is really about whether you have scaled a framework past a handful of files. Sell aliases as refactor safety plus self-documenting structure, not cosmetic tidiness. The detail that marks real experience is that the runner must resolve the alias at run time, not just the editor, plenty of people set the tsconfig path and are then baffled when tests fail to import. The thing to avoid is describing folder layout for its own sake. The likely follow-up is 'how do you keep imports clean as the framework grows?', which the central types directory plus aliases answers.

How to phrase it

> *Path aliases are one of the first things I configure on a new project, because they are really about surviving scale. They map a short prefix to a real folder, so instead of importing through three levels of dot-dot-slash I import from a stable pages or data prefix. The first benefit is refactor safety: moving a file does not break its imports because the alias does not change. The second is that the structure communicates intent, the pages prefix is always page objects, the data prefix is always factories, so a new engineer knows exactly where to add a file without asking, which cuts onboarding friction. I pair that with a central types directory so one interface change propagates everywhere with the compiler's help, and that is also my answer to how I keep imports clean as the framework grows. The detail people miss, and the one I would call out to show I have actually shipped this, is that the test runner has to resolve the alias at run time, not just the editor; if you only set the tsconfig path you get green squiggles in the IDE and a failing import at runtime. So I make sure the runtime resolver and tsconfig agree, and after that the imports stay clean and stable no matter how big the suite gets.*

Key points to hit

(1) Aliases map a prefix to a folder: imports become location-independent. **(2)** Refactor-safe: moving files does not break imports. **(3)** Self-documenting: a prefix always means one kind of file. **(4)** Centralise interfaces/enums in a types directory. **(5)** Senior detail: resolve the alias at run time too, not just in the editor. **(6)** Where this goes wrong: describing folder layout without the refactor-safety reason.

CODE

```
// tsconfig.json
{
"compilerOptions": {
"strict": true,
"baseUrl": ".",
"paths": {
"@pages/*": ["src/pages/*"],
"@fixtures/*": ["src/fixtures/*"],
"@data/*": ["src/test-data/*"],
"@types/*": ["src/types/*"]
}
}
}

// fragile: import { LoginPage } from '../../../src/pages/auth/LoginPage';
import { LoginPage } from '@pages/auth/LoginPage'; // stable
```

Q 8 LEVEL · Mid

How do you configure ESLint for a Playwright TypeScript project?

CONCEPT

ESLint is the second guard rail after the type checker. The **eslint-plugin-playwright** package adds rules that catch framework-specific mistakes, and the typescript-eslint rules add type-aware checks. The two rules that earn their place immediately are **no-floating-promises**, which flags a missing await, and prefer-web-first-assertions, which steers you off one-shot state checks toward retrying web-first assertions. Add **no-focused-tests** to stop a stray test.only reaching the main branch, and treat warnings as build failures in CI with a zero-warning threshold. The 2026 reality is the flat config format (eslint.config.js / .ts); older projects on the legacy .eslintrc still work, but new setups should use flat config.

INTERVIEW STRATEGY

A config dump is forgettable; the interviewer wants to know which rules catch real bugs and why. Name two or three and explain what each prevents, the prefer-web-first-assertions explanation, one-shot isVisible versus retrying toBeVisible, is the bit that shows depth. The risk is reciting an .eslintrc verbatim with no rationale. Mention flat config as the 2026 default so you sound current, and be ready for the follow-up ‘how do you stop people ignoring warnings?’, the answer is failing the build on any warning.

How to phrase it

> *I run ESLint alongside the type checker as a second safety net, with the Playwright plugin and the typescript-eslint rules, but the part worth talking about is which rules actually catch bugs. Two earn their keep on day one. no-floating-promises flags a missing await before CI ever runs, and that is the most common Playwright bug caught at the cheapest stage. The other is prefer-web-first-assertions, and it is the one most people do not know they need. isVisible checks state once and returns immediately; toBeVisible retries until timeout. Most of the time you want the retrying form, but people reach for isVisible because it reads more naturally, and that quietly reintroduces flakiness. The ESLint rule makes the retrying behaviour the default, and after we turned it on we caught several genuine bugs in the first week. I also add no-focused-tests so a stray test.only cannot reach main. For the follow-up about people ignoring warnings, my answer is that I fail the build on any warning at all, a warning that does not block is a warning everyone learns to scroll past. One current note: new projects should use the flat config format, not the legacy .eslintrc. These rules are cheap and they remove whole categories of mistake before review.*

Key points to hit

(1) eslint-plugin-playwright + typescript-eslint = framework + type-aware rules. **(2)** no-floating-promises flags missing await before CI runs. **(3)** prefer-web-first-assertions: isVisible is one-shot, toBeVisible retries. **(4)** no-focused-tests stops a stray test.only reaching main. **(5)** The next question: stop ignored warnings by failing the build on any warning. **(6)** 2026 default is flat config; trap is reciting a config with no rationale.

CODE

```
// eslint.config.ts (flat config, 2026 default)
import playwright from 'eslint-plugin-playwright';
import tseslint from 'typescript-eslint';

export default tseslint.config(
...tseslint.configs.recommendedTypeChecked,
{
files: ['**/*.ts'],
plugins: { playwright },
rules: {
'@typescript-eslint/no-floating-promises': 'error',
'playwright/no-wait-for-timeout': 'warn',
'playwright/no-focused-tests': 'error',
'playwright/prefer-web-first-assertions': 'error',
},
},
);
```

Q 9 LEVEL · Mid

How do you structure TypeScript types for a large framework?

CONCEPT

Centralise all interfaces and enums in a **types directory**, split into domain files, user types, API types, order types, with a **barrel export** (an index file that re-exports each module) so tests import

everything from one clean path. Generic shapes such as a standard API response wrapper or a paginated response live here too and are reused everywhere. The benefit is leverage: when the API team changes a shared shape, you edit one file and the compiler shows every test that must adapt. Without a central home for types, breaking changes are invisible during development and only appear as scattered runtime failures across many files. A barrel export also keeps imports short, which matters once dozens of test files depend on the same handful of core types.

INTERVIEW STRATEGY

What they are checking is for maintainability instincts at scale, so describe the directory plus barrel-export pattern and then justify it with the payoff: one edit, compiler-guided fan-out across the suite. A short real example, a new required field rippling through tests, lands far better than describing folder layout. The pitfall is making it about tidiness rather than change-safety. The follow-up is usually 'how do you handle a breaking API change across many tests?', which this structure answers directly.

How to phrase it

> *I keep every interface and enum in a central types directory, split by domain, user types in one file, API types in another, order types in a third, with a barrel export, an index file that re-exports each module, so a test imports everything from one path. Shared generic shapes live there too: a standard response wrapper, a paginated response. This is the single biggest contributor to maintainability at scale, and the way I would prove it is with the follow-up everyone asks, how do you handle a breaking API change across many tests. When our API team added a required phoneNumber field to the user profile, I changed the one interface and TypeScript showed me every test that needed to handle it. Without that structure, the breaking change would have been invisible during development and surfaced as runtime failures scattered across a dozen files, probably in CI on someone else's branch. The barrel export also keeps imports short and consistent, which matters a lot once many files depend on the same core types. So my approach is one home for types, derive the rest, and let the compiler drive the fan-out whenever the contract moves, it turns a scary cross-suite change into a guided one.*

Key points to hit

(1) Central types directory, split by domain, with a barrel export. **(2)** Shared generic shapes (API response, pagination) live there too. **(3)** One edit + compiler shows every test that must adapt. **(4)** Without it, breaking changes surface only as scattered runtime failures. **(5)** What loses the point: framing it as tidiness instead of change-safety. **(6)** Asked next: a breaking API change across tests, the central types answer.

CODE

```
// src/types/user.types.ts
export enum UserRole { Admin = 'admin', User = 'user', Viewer = 'viewer' }
export interface UserCredentials { email: string; password: string; }
export interface UserProfile extends UserCredentials {
id: number; firstName: string; lastName: string; role: UserRole;
}

// src/types/api.types.ts
export interface APIResponse<T> { data: T; status: number; message: string; }
export interface PaginatedResponse<T> extends APIResponse<T[]> {
total: number; page: number; pageSize: number;
}

// src/types/index.ts: barrel export
export * from './user.types';
export * from './api.types';
```

Q 10 LEVEL · Mid

How do you handle TypeScript compilation in CI pipelines?

CONCEPT

Run **tsc --noEmit** as a pre-test step. It type-checks the whole project without producing output files and **fails fast**, usually in a few seconds, before any browser is installed or launched. The recommended ordering is typecheck, then lint, then install browsers, then run tests, so the cheapest checks gate the expensive ones. Wire it through package scripts (a typecheck script, a lint script, and a combined ci script) and call those from the workflow so local and CI behave identically. This is shift-left applied to type safety: catch the error at the cheapest stage. The contrast is stark: a type error caught by tsc costs seconds, the same error caught after browser startup and test init can cost minutes per run, multiplied across every shard.

INTERVIEW STRATEGY

This is a pipeline-design question; the interviewer wants to see you think about cost and ordering, not just that you run tsc. Lead with the ordering principle, cheapest checks gate the expensive ones, and quantify it: seconds for a type error versus minutes per shard if it slips through. Framing it as shift-left for type safety is what separates junior from senior thinking. The mistake is just saying 'I run tsc in CI' with no ordering rationale. Follow-up: 'how do you keep local and CI consistent?', answer with package scripts the workflow calls.

How to phrase it

> *I run tsc --noEmit as the first step in CI. It type-checks the whole project without emitting files and fails in a few seconds, before a single browser is installed, and the reason I order it first is deliberate: I want the cheapest checks to gate the expensive ones. My sequence is typecheck, then lint, then install browsers, then run the tests. The cost difference is the whole argument: a type error caught by tsc costs about three seconds, while the same error caught after two minutes of browser startup and test setup costs minutes per run, and that multiplies across every shard in the matrix, so it could be ten or fifteen wasted minutes for one typo. I describe this as shift-left applied to type safety: find the bug at the cheapest possible stage. For the follow-up about keeping local and CI consistent, I put these behind package scripts, a typecheck script, a lint script and a combined ci script, and the workflow just calls those, so running it on my machine and running it in CI are identical, and nobody hits a green local run that fails in CI. The net effect is that a type error becomes a build failure before the test stage even begins.*

Key points to hit

(1) tsc --noEmit type-checks without output and fails in seconds. **(2)** Order: typecheck -> lint -> install browsers -> run tests. **(3)** Quantify: seconds via tsc vs minutes-per-shard if it slips through. **(4)** Shift-left for type safety: catch errors at the cheapest stage. **(5)** Follow-up: package scripts the workflow calls keep local and CI identical. **(6)** The common miss: 'I run tsc in CI' with no ordering or cost rationale.

CODE

```
// package.json
{
"scripts": {
"typecheck": "tsc --noEmit",
"lint": "eslint . --max-warnings 0",
"test": "playwright test",
"ci": "npm run typecheck && npm run lint && npm run test"
}
}

# .github/workflows: cheapest checks gate the expensive ones
- run: npm run typecheck # fails fast, no browser needed
- run: npm run lint
- run: npx playwright install --with-deps chromium
- run: npm run test -- --shard=${{ matrix.shard }}/${{ matrix.total }}
```

Q 11 LEVEL · Mid to Senior

What is the satisfies operator, and when do you use it instead of `as` or a type annotation?

CONCEPT

satisfies (TypeScript 4.9+) validates that an expression matches a type without widening the inferred literal types, the best of both worlds between a type annotation and `as`. With a type annotation, TypeScript widens to the broader declared type and you lose literal information. With **as**, you bypass the type system entirely, dangerous, because `as` silences mistakes the compiler

would otherwise catch. satisfies verifies the constraint but keeps the narrow inferred types. For Playwright: use satisfies on configuration objects, test data dictionaries, and tool-argument shapes so the compiler validates the structure but you can still write **config.projects[0].name** and have TypeScript infer the literal string. In interviews, knowing satisfies signals you're tracking modern TypeScript, most 2024 code uses `as`, which is the wrong default in 2026.

INTERVIEW STRATEGY

You are being asked to show modern TypeScript fluency. Lead with the three-way comparison: type annotation widens, `as` silences, satisfies validates without widening. The example that lands: a config object where satisfies preserves the literal project names. Where this goes wrong is using `as` because it compiles, it silences real errors. They usually follow with 'when is `as` still appropriate?', answer narrowly, for external untyped data after a type guard, never as a default.

How to phrase it

> *satisfies is TypeScript four point nine and onwards, and it fills a real gap. It validates that an expression matches a type without widening the inferred literal types, which is the best of both worlds. The way I explain it in interviews is a three-way comparison. With a type annotation like const config colon Config, TypeScript widens the value to the declared type and I lose literal information, config.projects zero name becomes string, not the literal chromium I actually wrote. With as Config, I bypass the type system entirely, which is dangerous because as silences mistakes the compiler would otherwise catch, if I mistype a property, as happily lets it through. With satisfies Config, the compiler verifies the constraint but keeps the narrow inferred types, so config.projects zero name stays the literal chromium and I can use it in switch statements that exhaustively check the project. For Playwright I use satisfies on configuration objects, test data dictionaries, and MCP tool argument shapes so the compiler validates the structure but I keep the literal types for narrowing later. The follow-up about when as is still appropriate is narrow: for external untyped data like a JSON.parse return value, after running a type guard to validate the shape, you can use as to assert the validated type. Never as a default for type assertions, and never to silence errors. The pitfall is using as because it compiles: it silences real errors that satisfies would have caught. Knowing satisfies signals 2026 TypeScript fluency in interviews; most 2024 code is still on as.*

Key points to hit

(1) satisfies validates structure WITHOUT widening literal types. **(2)** Type annotation widens; `as` silences; satisfies preserves narrow types. **(3)** Use on config objects, test data, tool-argument shapes, keeps literal info. **(4)** as is appropriate ONLY for external untyped data after a type guard. **(5)** Expect next: as for narrowed JSON.parse output is fine; never as default. **(6)** Avoid: using `as` because it compiles, it silences real errors.

CODE

```
// Three-way comparison
type ProjectConfig = { name: string; viewport: { width: number; height: number } };

// 1. Type annotation: widens, loses literal
const a: ProjectConfig = { name: 'chromium', viewport: { width: 1280, height: 720 } };
// a.name is `string`, not `'chromium'`: can't use in literal switch

// 2. `as`, dangerous, silences errors
const b = { name: 'chromium', viewprt: { width: 1280, height: 720 } } as ProjectConfig;
// typo `viewprt` compiles: production bug

// 3. `satisfies`: validates structure, keeps narrow types
const c = { name: 'chromium', viewport: { width: 1280, height: 720 } } satisfies
ProjectConfig;
// c.name is the literal 'chromium': can drive exhaustive switches
// typo `viewprt` would fail compilation: real safety
```

Q 12 LEVEL · Mid to Senior

What are discriminated unions, and how do you use them to model test scenarios cleanly?

CONCEPT

A **discriminated union** is a union of object types each with a shared literal discriminator field (often called kind, type, status) that lets TypeScript narrow the union based on that field's value. In a test framework, they're the right way to model anything with mutually-exclusive states: a TestResult that's either passed, failed, or skipped (different fields available in each case); an HTTP response that's success (with data) or error (with code and message); a Page state that's loading, loaded (with content), or error (with reason). The advantage over a flat shape with optional fields: TypeScript guarantees correctness, you can only access **result.error** when you've already checked **result.status === 'failed'**. The compiler enforces the state machine. Without discriminated unions, every field is optional and developers either crash on undefined or pepper the code with non-null assertions.

INTERVIEW STRATEGY

What scores here is showing you model state correctly. Lead with the shared-discriminator-field structure and the real example: TestResult with passed/failed/skipped variants each having different available fields. The compiler-enforced-state-machine framing is what separates junior from senior thinking. The trap is flat shapes with optional fields everywhere. The next question is normally ‘what does the discriminator buy you?’, answer narrowing: in the failed branch, the compiler knows error and stack exist; in the passed branch, it knows duration exists, and there's no ambiguity.

How to phrase it

> *A discriminated union is a union of object types each with a shared literal discriminator field, often kind, type, or status, that lets TypeScript narrow the union based on that field's value. The way I describe them in interviews, because it lands as the senior framing, is that they make the compiler enforce a state machine. In a test framework I use them constantly. A TestResult that's either status passed with a duration field, status failed with error and stack fields, or status skipped with a reason field, different fields available in each case. An HTTP response that's success with data or error with code and message. A Page state that's loading, loaded with content, or error with reason. The advantage over a flat shape with optional fields everywhere is huge, and this is my answer to the follow-up about what the discriminator buys you. TypeScript narrows the union when I check the discriminator. In the failed branch the compiler knows error and stack exist; in the passed branch it knows duration exists. There is no ambiguity, no accessing fields that might be undefined, no non-null assertions sprinkled defensively through the code. The compiler refuses to let me access result.error without checking result.status equals failed first, which means I cannot ship a runtime bug where I try to log the error of a passed test. The failure mode is flat shapes with optional fields like result.duration question mark and result.error question mark on the same type. Every field is potentially undefined, developers either crash on undefined access or pepper the code with non-null assertions to silence the compiler, and the state machine isn't actually enforced anywhere. Discriminated unions make impossible states impossible.*

Key points to hit

(1) Union of object types sharing a literal discriminator field. **(2)** Compiler narrows the union when you check the discriminator. **(3)** Each variant has DIFFERENT fields available: not all optional. **(4)** Compiler enforces a state machine: impossible states become impossible. **(5)** They usually ask next: in the 'failed' branch, error/stack exist with no ambiguity. **(6)** The easy mistake: flat shape with optional fields, every access is undefined-prone.

CODE

```
// Discriminated union for test results
type TestResult =
| { status: 'passed'; duration: number }
| { status: 'failed'; error: string; stack: string }
| { status: 'skipped'; reason: string };

function report(result: TestResult): string {
switch (result.status) {
case 'passed':
return `✓ ${result.duration}ms`; // duration exists here
case 'failed':
return `✗ ${result.error}\n${result.stack}`; // error + stack exist here
case 'skipped':
return `- ${result.reason}`; // only reason exists here
}
// No need for `result.error?`: the compiler narrows automatically
}
```

Q 13

LEVEL · Mid

What are type guards and how do you use them to safely narrow `unknown` to a specific type?

CONCEPT

A **type guard** is a function that returns a **type predicate**, a return type of the form `obj is SomeType`, telling the compiler that when the function returns true, the argument is that type. Type guards are the safe way to narrow `unknown` (the type of any external data: API responses, JSON.parse output, postMessage data) into a known shape. The pattern: check `typeof obj === 'object'` and `obj !== null` first, cast to `Record<string, unknown>`, then validate every expected field's type. The discipline that matters: never use `as` on external data without a type guard first. `as` silences the compiler but doesn't change the runtime value, so a malformed API response with `as MyResponse` produces undefined-access crashes at runtime that TypeScript should have prevented. The type guard catches the shape mismatch where it matters, at the boundary between your code and the outside world.

INTERVIEW STRATEGY

This question is there to see safe narrowing, not `as` assertions. Lead with the type-predicate signature (obj is SomeType), then the validation pattern: typeof check, null check, then field-by-field type validation. The boundary-with-outside-world framing is the experience marker. The classic error is `as` on JSON.parse output. A common follow-up is 'where do you actually use type guards?', answer every place external data enters your code: API responses, JSON files, postMessage, localStorage reads.

How to phrase it

> *A type guard is a function that returns a type predicate, a return type of the form obj is SomeType, telling the compiler that when the function returns true, the argument is that type. They're the safe way to narrow unknown, which is the type of any external data, API responses, JSON.parse output, postMessage data, localStorage reads. The pattern I use is: check typeof obj equals object and obj is not null first, cast to Record string unknown so I can index into the object, then validate every expected field's type. If all checks pass, return true and TypeScript narrows the argument to the typed shape; if any fail, return false and the caller handles invalid input. The discipline that matters, and the framing that lands with interviewers, is this: never use as on external data without a type guard first. as silences the compiler but doesn't change the runtime value, so a malformed API response cast with as MyResponse produces undefined-access crashes at runtime that TypeScript should have prevented. The type guard catches the shape mismatch where it actually matters, which is at the boundary between my code and the outside world. On the natural follow-up about where I actually use type guards. Every place external data enters my code, including parsing API responses in tests, validating MCP tool-call results from an LLM, reading config from JSON files, accepting messages from a worker or iframe. Internal data I control with types from creation; external data I validate at the boundary. The classic error is sprinkling as everywhere because it makes errors go away, it makes errors compile cleanly, which is worse than not compiling because they then crash at runtime.*

Key points to hit

(1) Type predicate `obj is SomeType`: compiler narrows on true return. **(2)** Pattern: typeof check -> null check -> cast to Record -> validate each field. **(3)** Narrows unknown safely; the only correct way to handle external data. **(4)** NEVER use `as` on external data without a type guard first. **(5)** The next question: use everywhere external data enters, APIs, JSON, postMessage. **(6)** Trap: `as` silences the compiler but not the runtime, errors crash later.

CODE

```
interface ApiUser { id: number; email: string; isAdmin: boolean; }

function isApiUser(obj: unknown): obj is ApiUser {
if (typeof obj !== 'object' || obj === null) return false;
const u = obj as Record<string, unknown>;
return typeof u.id === 'number'
&& typeof u.email === 'string'
&& typeof u.isAdmin === 'boolean';
}

// At the boundary: API response
const raw: unknown = await response.json();
if (!isApiUser(raw)) throw new Error(`Malformed user response: ${JSON.stringify(raw)}`);
// raw is now narrowed to ApiUser: every field safely typed
console.log(raw.email.toLowerCase()); // safe
```

Q 14 LEVEL · Senior

What are branded (nominal) types and how do they prevent ID-mixup bugs?

CONCEPT

TypeScript is **structurally typed**: if two types have the same shape, they're interchangeable. That's a problem for domain IDs: UserId and OrderId are both string, so the compiler happily lets you call **deleteOrder(userId)** if the function takes a string. **Branded types** add a phantom discriminator that exists only at compile time: type UserId = string & { __brand: 'UserId' }. Now UserId and OrderId are distinct types even though both are still strings at runtime, and the compiler refuses to pass a UserId where an OrderId is expected. The zero-runtime-cost insight: branded types add no runtime overhead because the brand is a compile-time-only intersection. In a Playwright framework: brand every domain ID (UserId, OrderId, ProductId, TenantId) plus session tokens and tenant scopes, prevents the bug category where the wrong ID is passed to the wrong API and the test silently does the wrong thing.

INTERVIEW STRATEGY

They want to see you push past structural typing's limits. Lead with the problem, structural typing makes UserId and OrderId interchangeable, then introduce branded types as the fix.
Zero-runtime-cost is the senior signal. Where this goes wrong is letting the compiler think string is enough. The follow-up is often ‘what's the runtime cost?’, answer zero, the brand exists only at compile time as a phantom intersection.

How to phrase it

> *TypeScript is structurally typed, which means if two types have the same shape, they're interchangeable. That's a problem for domain IDs: UserId and OrderId are both string, so the compiler happily lets me call deleteOrder with a userId if the function signature takes a string. That's a real bug category: in a Playwright test that sets up a user, then accidentally passes the userId where an orderId was meant, the test silently does the wrong thing and the production bug it would have caught ships. Branded types fix this. The pattern is type UserId equals string and an intersection with an object containing a double-underscore brand field of literal value UserId. The intersection adds a phantom discriminator that exists only at the type level. Now UserId and OrderId are distinct types even though both are still strings at runtime, and the compiler refuses to pass a UserId where an OrderId is expected. The detail that lands in interviews, and my answer to the follow-up about runtime cost, is zero. The brand exists only at compile time as a phantom intersection, there's no runtime tag, no wrapper object, no field added to the actual string. Once compiled, a UserId is literally just a string. So I get nominal typing safety with no runtime overhead. In a Playwright framework I brand every domain ID, UserId, OrderId, ProductId, TenantId, plus session tokens and tenant scopes. The factory function that creates a user returns UserId; the assertion that deletes an order takes OrderId; the compiler enforces correctness across the test. The risk is leaving every ID as raw string: structurally typed, compiler can't help, ID-mixup bugs ship.*

Key points to hit

(1) TypeScript is structural: UserId and OrderId as `string` are interchangeable. **(2)** Branded type: `type UserId = string & { __brand: 'UserId' }`. **(3)** Phantom intersection: distinct types at compile time. **(4)** ZERO runtime cost: brand exists only at the type level. **(5)** Asked next: at runtime it's still a string; the safety is compile-time-only. **(6)** Where this goes wrong: leaving domain IDs as raw `string`, ID-mixup bugs ship undetected.

CODE

```
// Brand each domain ID: zero runtime cost
type UserId = string & { readonly __brand: 'UserId' };
type OrderId = string & { readonly __brand: 'OrderId' };
type TenantId = string & { readonly __brand: 'TenantId' };

// Constructor functions create the branded type from raw strings
const UserId = (s: string) => s as UserId;
const OrderId = (s: string) => s as OrderId;

// Domain API takes the right type
async function deleteOrder(id: OrderId) { /* ... */ }

const user = UserId('u_123');
const order = OrderId('o_789');

await deleteOrder(order); // OK
await deleteOrder(user); // ✗ Error: UserId is not OrderId
// compiler catches the bug at the call site
```

Q 15

LEVEL · Senior

What are mapped types, and how do you use key remapping to derive types from existing ones?

CONCEPT

A **mapped type** creates a new type by iterating over the keys of an existing one and transforming each key, value, or both, the mechanism Pick, Omit, Partial, Required, and Readonly are built on. The general form: { [K in keyof T]: F<T[K]> } where K is each key in turn and F is some transformation of the original value type. **Key remapping** (TypeScript 4.1+) adds as inside the brackets to rename keys: { [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K] } derives a getter interface from a data interface. In a Playwright framework: derive AsyncPageObject from a sync API, derive a builder interface from a domain interface, derive mock function types from a service interface. The principle that lands: **derive types, don't duplicate them**. When the source interface changes, every derived type updates automatically, one source of truth, multiple shapes.

INTERVIEW STRATEGY

Your job here is to show derived types, not duplicated ones. Lead with the general form and the relationship to Pick/Omit/Partial. Key remapping with template literals is the advanced detail. The framing 'derive, don't duplicate' is what separates junior from senior thinking. The pitfall is manually-duplicated interfaces that drift over time. Expect the follow-up ‘what's a real use case in your framework?’, answer deriving the page-object getter interface from the domain interface, or builder types from entity types.

How to phrase it

> *A mapped type creates a new type by iterating over the keys of an existing type and transforming each one, and it's the mechanism Pick, Omit, Partial, Required and Readonly are built on. The general form is open brace K in keyof T colon F of T at K close brace, where K is each key in turn and F is some transformation of the original value type. So Partial of T is keyof T mapped where each value type becomes T at K with a question mark, making every field optional. Once you understand that mechanism, you can build your own mapped types instead of duplicating interfaces. The newer feature is key remapping, TypeScript 4.1 and onwards, which adds an as clause inside the mapping brackets to rename keys, often using template literal types. So I can write open brace K in keyof T as backtick get capital K backtick colon function returning T at K, and that derives a getter interface from a data interface automatically. The framing I would lead with, because it lands as the senior principle, is: derive types, don't duplicate them. When the source interface changes, a new field added, an old one renamed, every derived type updates automatically. One source of truth, multiple shapes. The follow-up usually probes this, and the answer is about a real use case in the framework. I derive the page-object getter interface from the domain interface so adding a field to User automatically adds getName to the user page object. I derive builder types from entity types so a new User field gets a withFieldName method in the builder. I derive mock function types from service interfaces for typed test doubles. The trap is manually-duplicated interfaces that drift over time, the source changes, the duplicates don't, and tests start to silently lie about reality.*

Key points to hit

(1) Mapped type: { [K in keyof T]: F<T[K]> }, transforms keys/values. **(2)** Pick, Omit, Partial, Required, Readonly are all built on this. **(3)** Key remapping (TS 4.1+): `as` inside brackets renames keys. **(4)** Combine with template literals: derive `getName` from `name`, etc. **(5)** Follow-up: derive POM getters, builder types, mock types from one source. **(6)** What loses the point: manually-duplicated interfaces drift; derived types update automatically.

CODE

```
// Derive a getter interface from a data interface via key remapping
interface User { id: string; name: string; email: string; isAdmin: boolean; }

type Getters<T> = {
[K in keyof T as `get${Capitalize<string & K>}`]: () => Promise<T[K]>
};

type UserGetters = Getters<User>;
// equivalent to:
// interface UserGetters {
// getId(): Promise<string>;
// getName(): Promise<string>;
// getEmail(): Promise<string>;
// getIsAdmin(): Promise<boolean>;
// }

// Add a User field -> UserGetters updates automatically, single source of truth

// Make every field optional (Partial)
type PartialUser = { [K in keyof User]?: User[K] };

// Make every field readonly (Readonly)
type ReadonlyUser = { readonly [K in keyof User]: User[K] };
```

Q 16 LEVEL · Senior

What are template literal types, and how can they make selectors and routes type-safe?

CONCEPT

Template literal types (TypeScript 4.1+) let you build string types from other string types using the same backtick syntax as runtime template literals, type Greeting<T extends string> = `Hello ${T}`. Combined with intrinsic helpers like Uppercase, Lowercase, Capitalize, and Uncapitalize, they let you encode rules about valid strings in the type system. In a Playwright framework: type test-ID selectors as `[data-test="${string}"]` so any selector violating the pattern is a compile error; type route patterns as **`/users/${number}/orders`** so the compiler catches a typo; derive event-handler types from event names with `on${Capitalize<Event>}`. The depth signal that lands in interviews: template literal types encode **protocol** in the type system, the compiler enforces the same conventions you used to enforce with naming rules in CONTRIBUTING.md.

INTERVIEW STRATEGY

Show depth on advanced TypeScript. Lead with the backtick syntax mirroring runtime template literals, then the concrete framework use, typed test-ID selectors and route patterns. The 'encode protocol in the type system' framing is the senior signal. The classic error is enforcing string conventions only with comments. They usually follow with ‘where do you actually use these?’, answer typed selectors, typed routes, typed event-handler signatures derived from event names.

How to phrase it

> *Template literal types are TypeScript 4.1 onwards and they let me build string types from other string types using the same backtick syntax as runtime template literals. So type Greeting of T extends string equals backtick Hello dollar T backtick, and Greeting of literal World becomes the literal type Hello World. Combined with intrinsic helpers, Uppercase, Lowercase, Capitalize, Uncapitalize, I can encode rules about valid strings in the type system itself. The framing I would lead with, because it lands as the depth signal interviewers respond to, is that template literal types let me encode protocol in the type system. The compiler enforces the same conventions I used to enforce only with comments and naming rules in CONTRIBUTING.md. On the natural follow-up about where I actually use them. Three concrete places. First, typed test-ID selectors: I declare type TestIdSelector equals backtick open bracket data dash test equals quote dollar string close brace close bracket backtick, and any selector that violates the pattern is a compile error, the compiler catches a typo before the test even runs. Second, typed route patterns: route constants like backtick slash users slash dollar number slash orders backtick describe the actual URL shape, and helper functions that build URLs take the matching parameters. The compiler catches calling buildUrl with the wrong shape. Third, derived event-handler types: from a union of event names I derive on-prefixed handler signatures using key remapping plus Capitalize. All three are protocol enforcement at compile time. The trap is enforcing string conventions only with comments and hoping reviewers catch violations, they don't, consistently, and the convention erodes.*

Key points to hit

(1) Backtick syntax mirrors runtime template literals, same intuition. **(2)** Combine with Uppercase/Lowercase/Capitalize for rules on strings. **(3)** Encode protocol in the type system: not just in CONTRIBUTING.md. **(4)** Use cases: typed test-ID selectors, route patterns, event-handler names. **(5)** Expect next: typed selectors catch typos before the test runs. **(6)** The common miss: enforcing string conventions with comments and hoping reviewers catch.

CODE

```
// Typed test-ID selectors
type TestIdSelector = `[data-test="${string}"]`;
function byTestId(s: TestIdSelector) { /* ... */ }

byTestId('[data-test="submit"]'); // OK
byTestId('#submit'); // ✗ not a TestIdSelector

// Typed route patterns (TS 4.5+ supports template literal pattern matching well)
type UserOrderRoute = `/users/${number}/orders`;
const valid: UserOrderRoute = `/users/${42}/orders`; // OK
// const wrong: UserOrderRoute = `/users/abc/orders`; // ✗ 'abc' is not number

// Derived event-handler names
type EventName = 'click' | 'submit' | 'change';
type Handlers = { [E in EventName as `on${Capitalize<E>}`]: () => void };
// -> { onClick: () => void; onSubmit: () => void; onChange: () => void }
```

Q 17 LEVEL · Senior

What is the `never` type, and how does it enforce exhaustive switch statements at compile time?

CONCEPT

never is the type of values that can never occur, the return type of a function that always throws, the type TypeScript narrows a union to when every case has been eliminated. The pattern that uses this for compile-time exhaustiveness checking: in the default branch of a switch on a discriminated union, assign the discriminator to a variable typed never. If every case is covered, the discriminator has been narrowed to never and the assignment compiles; if a new variant is added to the union and the switch isn't updated, the assignment fails compilation because the unhandled variant is still in the union. The result: the compiler enforces that every union variant is handled. Adding a third TestResult variant after writing the report function fails the build, forcing the developer to handle the new case. This is one of the most powerful patterns in TypeScript for keeping state machines correct as they evolve.

INTERVIEW STRATEGY

They are looking to see compile-time enforcement of exhaustiveness. Lead with the never-assignment-in-default pattern, that's the technique. The 'forces handling new variants when the union grows' framing is the senior signal because it shows you understand the maintenance advantage. The thing to avoid is omitting the default branch or having it throw at runtime instead of failing compilation. The next question is normally ‘why never specifically?’, answer never is the type a union narrows to when every variant has been eliminated; if an unhandled variant remains, it's not never anymore.

How to phrase it

> *never is the type of values that can never occur, the return type of a function that always throws, the type TypeScript narrows a union to when every case has been eliminated. On its own it sounds abstract, but the pattern that makes it powerful in real code is compile-time exhaustiveness checking on discriminated unions. In the default branch of a switch on the discriminator, I assign the discriminator to a variable typed never. If every case in the switch has been covered, by the time control reaches default the discriminator has been narrowed to never and the assignment compiles fine. But if I add a new variant to the union later, say a fourth TestResult status like blocked, and I don't update the switch, the discriminator in the default branch is still the literal blocked, not never, and the assignment fails compilation. The compiler refuses to build until I handle the new case. That's also my answer to the follow-up about why never specifically. never is the type a union narrows to when every variant has been eliminated; if an unhandled variant remains, the discriminator type is not never, and the assignment fails. It's the only type that captures 'there should be nothing left here'. The framing I would emphasise is that this is one of the most powerful patterns in TypeScript for keeping state machines correct as they evolve. The compiler enforces that every variant is handled, automatically, every time someone adds a new one. The failure mode is omitting the default branch or having it throw at runtime instead of failing compilation. Runtime errors find bugs in production; compile-time errors find them on the developer's machine before anything ships.*

Key points to hit

(1) never = the type of values that can never occur, narrows-to-empty. **(2)** In default branch of a switch, assign discriminator to a `never` variable. **(3)** All cases handled -> narrows to never -> compiles; missing case -> fails build. **(4)** Adding a new union variant later forces handling, compile-time enforced. **(5)** They usually ask next: never is the only type that captures 'nothing should be here'. **(6)** Avoid: runtime default that throws, finds bug in prod, not on dev's machine.

CODE

```
type TestResult =
| { status: 'passed' }
| { status: 'failed' }
| { status: 'skipped' };

function assertExhaustive(value: never): never {
throw new Error(`Unhandled variant: ${JSON.stringify(value)}`);
}

function report(r: TestResult): string {
switch (r.status) {
case 'passed': return '✓';
case 'failed': return '✗';
case 'skipped': return '-';
default: return assertExhaustive(r); // compiles only if all cases handled
}
}

// Adding `| { status: 'blocked' }` to TestResult, build FAILS until report() handles it
```

Q 18 LEVEL · Senior

What is declaration merging, and how does it let you extend Playwright's `test` and `expect` cleanly?

CONCEPT

Declaration merging lets two or more declarations with the same name in the same scope be merged by the compiler into a single combined declaration, most commonly used to extend interfaces or modules without modifying the original source. For Playwright: **test.extend** returns a new test object with custom fixtures, and TypeScript's inference handles fixture typing in the simple case. For deeper extensions, custom matchers via expect.extend, augmenting **PlaywrightTestConfig** with custom project options, adding properties to **TestInfo** via module augmentation, you use declaration merging via declare module '@playwright/test'. The pattern: redeclare the namespace, extend the interfaces you need with your new fields or matchers, and TypeScript merges them with the originals. The discipline that matters: put module augmentations in a single file like types/playwright.d.ts so the augmentation is discoverable and one place to update when the framework's types shift.

INTERVIEW STRATEGY

The point of this one is to see how custom Playwright extensions are typed. Lead with the simple case (test.extend handled by inference) then escalate to module augmentation for custom matchers and TestInfo fields. The single-file-for-augmentations discipline is the senior signal. Where this goes wrong is augmenting from multiple files, they all merge but it becomes impossible to find the source of a type. A common follow-up is ‘how do you add a custom matcher?’, answer expect.extend at runtime, module augmentation on the Matchers interface at the type level.

How to phrase it

> *Declaration merging lets two or more declarations with the same name in the same scope be merged by the compiler into a single combined declaration, and it's the mechanism for extending interfaces or modules without modifying the original source. For Playwright the simple case is handled by inference: test.extend returns a new test object with custom fixtures, and TypeScript infers the fixture types from the generic on test.extend. I don't need declaration merging for ordinary fixtures. The cases where I do need it, and this is the depth that lands in interviews, are three. First, custom matchers via expect.extend, where I need to tell TypeScript that expect now has a new method, say toHaveRowCount for a table. I'd pick the name carefully, because expect.extend will happily shadow a built-in matcher at runtime. toHaveAccessibleName, for instance, has shipped since one point four four, so redefining it would quietly replace the real one. Second, augmenting PlaywrightTestConfig with custom project options if I have project-specific config I want strongly typed. Third, adding properties to TestInfo via module augmentation, for example a custom annotation or metadata field. The pattern is: declare module quote at-playwright-slash-test quote, redeclare the namespace, extend the interface I need, typically Matchers for custom expect methods, with my new fields or methods, and TypeScript merges them with the originals. The discipline I would emphasise, because this is where frameworks get confusing fast, is putting all module augmentations in a single file like types-slash-playwright dot d dot ts. The augmentation works no matter where it lives, but if I scatter it across files, it becomes impossible to find the source of a type later. One file, well-named, discoverable. That's also my answer to the follow-up about adding a custom matcher: expect.extend at runtime to register the implementation, module augmentation on the Matchers interface at the type level so TypeScript knows the method exists. Both halves are required: runtime alone gives untyped access, type alone gives runtime errors. The risk is augmenting from multiple files and losing track of where extensions live.*

Key points to hit

(1) Declaration merging: multiple declarations with the same name combine. **(2)** test.extend uses inference: no manual merging needed for fixtures. **(3)** Use module augmentation for custom matchers, TestInfo fields, config. **(4)** Pattern: `declare module '@playwright/test'` + extend the interface. **(5)** Put all augmentations in ONE file (types/playwright.d.ts) for discoverability. **(6)** The next question: custom matcher needs BOTH expect.extend AND Matchers augmentation. **(7)** Matchers<R, T> returns R, not Promise<R>; and never shadow a built-in matcher name.

CODE

```
// types/playwright.d.ts: ONE place for all augmentations
export {};

declare global {
namespace PlaywrightTest {
// 1. Custom matcher. Note the return type is R, not Promise<R>
interface Matchers<R, T> {
toHaveRowCount(expected: number, options?: { timeout?: number }): R;
}
}
}
// 2. Custom TestInfo field
declare module '@playwright/test' {
interface TestInfo { jiraTicket?: string; }
}

// fixtures/expect.ts: register the implementation, export the wrapped expect
import { expect as baseExpect, Locator } from '@playwright/test';

export const expect = baseExpect.extend({
async toHaveRowCount(locator: Locator, expected: number,
options?: { timeout?: number }) {
let actual = 0;
try {
await baseExpect.poll(
async () => (actual = await locator.getByRole('row').count()),
{ timeout: options?.timeout ?? 5_000 },
).toBe(expected);
return { pass: true, name: 'toHaveRowCount', expected, actual,
message: () => `expected the table NOT to have ${expected} rows` };
} catch {
return { pass: false, name: 'toHaveRowCount', expected, actual,
message: () => `expected ${expected} rows, got ${actual}` };
}
},
});

// Use: fully typed, and it retries like a built-in web-first assertion
await expect(page.getByRole('table')).toHaveRowCount(12);

// WATCH OUT: never name a custom matcher after a built-in.
// toHaveAccessibleName has shipped since v1.44, so redefining it
// silently shadows the real one at runtime.
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Which command type-checks the whole project without producing output files?

A) tsc --noEmit
B) tsc --build
C) tsc --watch
D) tsc --strict

2. Which ESLint rule catches a missing await, the most common Playwright bug?

A) @typescript-eslint/no-explicit-any
B) @typescript-eslint/no-floating-promises
C) playwright/no-focused-tests
D) playwright/no-wait-for-timeout

3. You define UserResponse once and want a create-request type without id and createdAt. Which utility type?

A) Partial
B) Pick
C) Readonly
D) Omit

4. What does a Record keyed by an enum give you that a plain object does not?

A) Compile-time exhaustiveness over every enum member
B) Faster property access
C) Automatic JSON serialisation
D) Runtime validation of values

5. Why must waitForResponse be started before the click that triggers the request?

A) Because clicks are synchronous
B) Because the network is slower than the UI
C) Because Playwright can only capture a response it was already listening for
D) Because await reorders the calls automatically

Section 1 · Answer key

#	Answer	Why and where to revisit
1	A) tsc --noEmit	tsc --noEmit reports type errors without emitting JS, which is why it is the fast first CI gate. *Revisit Section 1.*
2	B) @typescript-eslint/no-floating-promises	no-floating-promises flags a Promise that is never awaited or handled, surfacing missing awaits at lint time. *Revisit Section 1.*
3	D) Omit	Omit removes the named keys, deriving the create-request shape from the single source-of-truth interface. *Revisit Section 1.*
4	A) Compile-time exhaustiveness over every enum member	A Record keyed by an enum forces an entry for every member, so adding a new role flags the missing entry at compile time. *Revisit Section 1.*
5	C) Because Playwright can only capture a response it was already listening for	Playwright captures only responses it is already subscribed to, so the listener must exist before the response arrives. *Revisit Section 1.*

SECTION 02

Playwright Architecture and Setup

Interviewers use architecture questions to separate people who run Playwright from people who understand it. The headline is the persistent WebSocket connection, it is the reason network interception, the trace viewer, and real-time events exist at all. These fourteen questions cover that model, the Browser/Context/Page hierarchy, a production playwright.config.ts, globalSetup, multi-environment handling, browser version management, and the difference between the test runner and the bare library.

In this section

- The persistent WebSocket model and why it beats Selenium's stateless HTTP.
- Browser vs BrowserContext vs Page: and the multi-user test it makes possible.
- A production playwright.config.ts and the three decisions worth defending.
- globalSetup for fail-fast health checks, and why auth belongs in a setup project.
- Managing multiple environments with dotenv without committing secrets.
- How Playwright pins browser versions so there is no driver management.
- @playwright/test (the framework) vs playwright (the bare library).

Q 19 LEVEL · Mid to Senior

Explain Playwright's internal architecture in depth.

CONCEPT

Playwright keeps a **persistent, bidirectional WebSocket connection** to each browser process. For Chromium and Edge it speaks the **Chrome DevTools Protocol (CDP)**; for Firefox it uses a Firefox protocol; for WebKit it uses the WebKit remote debugging protocol. Classic Selenium, by contrast, uses a stateless HTTP model, every command is a fresh request to a driver, with the round-trip cost that implies. The persistent channel does two things a pure request-response model cannot: it drops per-command latency dramatically, and it lets Playwright subscribe to browser events in real time, console messages, failed requests, dialogs, network traffic. That single architectural fact is why auto-waiting, network interception, and the trace viewer are even possible. Be accurate about Selenium here, because an interviewer who works with it will catch an overclaim: Selenium 4 does have an event channel now, first through CDP for Chromium, and since 4.18 through **WebDriver BiDi** over its own WebSocket, with real network interception. The honest difference is maturity and uniformity, not possibility: Playwright was built on a persistent connection from day one, so interception is one API across all three engines, while Selenium reached the same place later and still splits it between CDP and BiDi. The hierarchy on top of the connection is Browser then BrowserContext then Page.

INTERVIEW STRATEGY

This is the question that sorts memorised feature-lists from genuine understanding, so do not list features, explain the cause. Lead with WebSocket versus stateless HTTP, then make the senior move: frame the connection as the reason the feature set exists, because you can only intercept what you are continuously listening for. A vivid analogy locks it in. The classic error is saying ‘Playwright is faster and has interception’ without explaining why architecturally. The follow-up is almost always ‘so how does Selenium handle network requests?’, and the strong answer is the current one: classic WebDriver had no channel for it, Selenium 4 added CDP for Chromium, and Selenium 4.18 added WebDriver BiDi over a WebSocket, which does real interception. Saying Selenium ‘cannot’ intercept is out of date and will cost you credibility with anyone who has used BiDi.

How to phrase it

> *Playwright holds a persistent bidirectional WebSocket connection to the browser. For Chromium and Edge that is the Chrome DevTools Protocol; Firefox and WebKit use their own protocols. Selenium uses stateless HTTP, where every command is a brand-new request to a driver. The way I describe the difference is that Selenium sends letters, a fresh envelope and delivery for every command, whereas Playwright is a phone call, one open line where both sides talk in real time. That open line matters for two reasons. First, latency drops a lot because there is no per-command handshake; per-command time goes from tens of milliseconds to a handful. Second, and this is the important one, Playwright can subscribe to browser events as they happen, console logs, failed requests, dialogs, network traffic. That is the architectural reason network interception exists in Playwright. I'd be careful with the follow-up people ask about Selenium, though, because the usual answer is out of date. Classic WebDriver had nowhere to receive those events, but Selenium 4 added CDP access for Chromium, and Selenium 4.18 added WebDriver BiDi over its own WebSocket, with add intercept, fail request and continue with auth, and phases like before request sent and auth required. So Selenium can intercept today. The real difference is that Playwright had the persistent connection from the start, so interception is one API that behaves the same on Chromium, Firefox and WebKit, whereas Selenium got there later and still splits it between CDP and BiDi. The trace viewer and auto-waiting fall out of the same fact. On top of the connection sits the hierarchy: Browser, then an isolated BrowserContext, then Page. So the feature gap between the two tools is not luck or polish; it comes straight from the connection model.*

Key points to hit

(1) Persistent bidirectional WebSocket; CDP for Chromium/Edge. **(2)** Classic Selenium is stateless HTTP: a new request per command. **(3)** Persistent channel cuts latency and enables real-time event subscriptions. **(4)** Interception/trace/auto-wait exist because you can listen continuously. **(5)** Currency: Selenium 4 has CDP, and 4.18+ has WebDriver BiDi interception over a WebSocket. **(6)** So the honest gap is uniformity and maturity, not 'Selenium cannot intercept'. **(7)** What loses the point: listing features without the architectural cause.

CODE

```
// One persistent connection: subscribe to events in real time
page.on('console', msg => console.log('Browser log:', msg.text()));
page.on('requestfailed', req => console.log('Failed:', req.url()));
page.on('dialog', dialog => dialog.accept());

// Listen for traffic, then act: only possible on an open channel
page.on('response', res => {
if (res.url().includes('/api/') && !res.ok()) {
console.log('API error', res.status(), res.url());
}
});
```

Q 20 LEVEL · Mid

What is the difference between Browser, BrowserContext, and Page?

CONCEPT

Browser is the actual browser process: expensive to launch, so it is shared across tests inside a worker. **BrowserContext** is an isolated session with its own cookies, localStorage, service workers, and permissions, effectively an incognito window. Each test gets a fresh context by default, which is what makes tests independent. **Page** is a tab within a context and is where most interactions happen. The pattern that demonstrates real understanding is two contexts in one test: an admin context and a user context running side by side, so you can drive a two-actor workflow, user submits, admin approves, user sees the result, in a single test. That is impossible with one Selenium WebDriver instance, and it is fast because contexts are cheap relative to launching a browser.

INTERVIEW STRATEGY

What they are checking is whether you grasp isolation and cost, because that is what drives test independence and speed. Give the three definitions with their cost profile, Browser expensive and shared, Context cheap and isolated, Page the tab, then prove understanding with the multi-context multi-user test, which is the example that wins this question. The pitfall is reciting three definitions and stopping. The follow-up is reliably 'how did you synchronise the two actors?', so have Promise.all on the concurrent actions ready before they ask.

How to phrase it

> *Browser is the process itself, expensive to start, so it is shared across tests in a worker. BrowserContext is an isolated session with its own cookies, storage and permissions, basically an incognito window, and each test gets a fresh one by default, which is what keeps tests independent. Page is a tab inside a context, and that is where most of my interactions happen. Rather than stop at definitions, the scenario I use to show I have actually built with this is a multi-user test in a single test body. I create an admin context and a user context, give each its own page, and drive both at once: the user submits a request, the admin approves it, and the user's page updates to Approved. That is impossible with one Selenium WebDriver instance, you would need two driver instances and external coordination between them. Interviewers almost always follow up by asking how I synchronise the two actors, so I would pre-empt it: I use Promise.all on the concurrent actions so they happen together rather than my forcing an artificial order. The whole test is about fifteen lines, because contexts are cheap compared to launching a browser, and that combination of isolation and low cost is the real point of the hierarchy.*

Key points to hit

(1) Browser: the process, expensive, shared across a worker's tests. **(2)** Context: isolated session (cookies/storage/permissions), fresh per test. **(3)** Page: a tab inside a context where interactions happen. **(4)** Two contexts in one test drive a real two-actor workflow. **(5)** Pre-empt this: synchronise the actors with Promise.all. **(6)** The common miss: listing definitions without the multi-context use case.

CODE

```
test('admin approves a user request', async ({ browser }) => {
const adminCtx = await browser.newContext({ storageState: 'auth/admin.json' });
const userCtx = await browser.newContext({ storageState: 'auth/user.json' });
const adminPage = await adminCtx.newPage();
const userPage = await userCtx.newPage();

await userPage.goto('/requests/new');
await userPage.getByRole('button', { name: 'Submit Request' }).click();

await adminPage.goto('/admin/pending-requests');
await adminPage.getByRole('button', { name: 'Approve' }).click();

await expect(userPage.getByTestId('request-status')).toHaveText('Approved');
await Promise.all([adminCtx.close(), userCtx.close()]);
});
```

Q 21 LEVEL · Mid

Walk through a production-grade playwright.config.ts.

CONCEPT

The config is the brain of the framework: nearly every behaviour is set here. The essentials: **testDir**, **fullyParallel** for in-file parallelism, **forbidOnly** tied to CI so a stray test.only fails the pipeline, retries only in CI, and a workers count. The reporter array usually pairs the HTML report with a machine-readable one (JUnit) and a CI-native reporter. Under use, the high-value settings are trace: 'on-first-retry' (not 'on', which wastes storage on passing tests), screenshot: 'only-on-failure', sensible actionTimeout and navigationTimeout, and testIdAttribute. projects define the browser matrix including a mobile device, and the first project is a setup project that logs in once and saves storageState, with every browser project declaring dependencies: ['setup']. That is the current pattern for auth; globalSetup is still available, but it runs outside the runner so it gets no fixtures, no trace and no report line, which is why auth belongs in a setup project instead.

INTERVIEW STRATEGY

Do not narrate every field: the interviewer is testing judgement, not recall, and reciting the whole file is the trap. Pick three decisions and defend each with a reason: forbidOnly tied to CI, trace on-first-retry rather than on, and a deliberately tight actionTimeout. Explaining why each is set that way is the signal. The likely follow-up is ‘why not just set trace: on and a big timeout to be safe?’, answer with storage cost and hidden regressions, which shows you have run this at scale, not copied a starter config.

How to phrase it

> *I treat the config as the brain of the framework, but in an interview I focus on three decisions I can defend rather than reading every line, because reading the whole file proves nothing. First, forbidOnly tied to CI. If someone commits a test.only, the pipeline fails instead of silently running just that one test, and that has genuinely saved a release for us once. Second, trace on-first-retry, not trace on. This is also my answer to the follow-up about why not just turn everything on to be safe: storing traces for passing tests wastes five to ten times the storage for zero benefit, whereas on-first-retry gives me a full trace exactly when a test failed and is retrying, which is the only time I need it. Third, an explicit actionTimeout around fifteen seconds. A blanket large timeout feels safe but hides performance regressions, a page that doubled in load time still passes, so I keep it tight and let genuinely slow interactions fail loudly. Beyond those, I set fullyParallel, retries only in CI, and a projects matrix across Chromium, Firefox, WebKit and a mobile device, with a setup project in front of it that logs in once and saves the storage state. I'd call that choice out deliberately, because a lot of configs still put login in globalSetup. A setup project is a real test inside the runner, so when auth breaks I get a trace and a line in the report instead of a raw stack trace. And the 2026 default I rely on is loading environment-specific config through dotenv, so the same suite runs anywhere by changing one variable.*

Key points to hit

(1) Defend three decisions, don't narrate the whole file. **(2)** forbidOnly tied to CI fails the build on a stray test.only. **(3)** trace: 'on-first-retry', full trace when it matters, no storage waste. **(4)** Deliberate actionTimeout surfaces regressions instead of hiding them. **(5)** Auth belongs in a setup project with dependencies, not in globalSetup. **(6)** The next question: why not trace:on + big timeout?, storage cost, hidden regressions. **(7)** Avoid: reciting every field with no rationale.

CODE

```
import { defineConfig, devices } from '@playwright/test';
import dotenv from 'dotenv';
dotenv.config({ path: `.env.${process.env.TEST_ENV || 'staging'}` });

export default defineConfig({
testDir: './tests',
fullyParallel: true,
forbidOnly: !!process.env.CI,
retries: process.env.CI ? 2 : 0,
workers: process.env.CI ? 4 : undefined,
reporter: [['html', { open: 'never' }], ['junit', { outputFile: 'results/junit.xml' }],
['github']],
use: {
baseURL: process.env.BASE_URL,
trace: 'on-first-retry',
screenshot: 'only-on-failure',
actionTimeout: 15_000,
testIdAttribute: 'data-testid',
},
projects: [
// auth once, inside the runner, so a broken login gets a trace
{ name: 'setup', testMatch: /.*\.setup\.ts/ },
{ name: 'chromium', dependencies: ['setup'],
use: { ...devices['Desktop Chrome'], storageState: '.auth/user.json' } },
{ name: 'mobile', dependencies: ['setup'],
use: { ...devices['iPhone 14'], storageState: '.auth/user.json' } },
],
});
```

Q 22 LEVEL · Mid

How does globalSetup work and what should you put in it?

CONCEPT

globalSetup runs once before any workers start, and it runs outside the test runner: no fixtures, no trace, no line in the report. That single fact decides what belongs in it. Browser-less environment work belongs here: an environment health check, seeding a database, starting a mock server, setting env vars. The health-check pattern is to ping a /health endpoint first and abort the whole run in seconds if the environment is down, far better than watching every test fail for eight minutes against a dead environment. Authentication does not belong here any more. Login is the thing most likely to break, and in globalSetup a break gives you a raw stack trace and nothing else. Put login in a setup project instead: a real test file matched by **testMatch**, with the browser projects declaring dependencies: ['setup']. It still runs once, it still saves **storageState** so the 300 tests that follow start already authenticated, and when it fails you get a trace and a named failure. Say that distinction out loud, it is the current guidance and most candidates are still a version behind on it.

INTERVIEW STRATEGY

Start with the one fact that explains everything else: globalSetup runs outside the runner, so no fixtures, no trace, no report line. Then split the work by that fact. Browser-less environment work stays in globalSetup, and the fail-fast health check is the pattern to tell as a war story, the stronger you make the pain, the more it lands. Auth moves to a setup project, and saying so is the currency

signal, because a lot of candidates still recite the old login-in-globalSetup answer. The mistake is describing only login. The follow-up is usually ‘how do tests then use that auth?’, be ready to say the browser projects declare a dependency on the setup project and load the saved storageState.

How to phrase it

> *globalSetup runs once before any workers spin up, and the detail that decides everything else is that it runs outside the test runner. No fixtures, no trace, no line in the report. So I use it for browser-less environment work only. The pattern I always add, and it came from a genuinely painful experience, is an environment health check: three hundred tests ran for eight minutes and every single one failed because staging was down for maintenance, and we only found out after the full run. Now globalSetup pings the health endpoint first, and if it is not healthy the run aborts in about two seconds with a clear message naming the environment. Seeding a database or starting a mock server fits the same slot. What I deliberately do not put there any more is login, and I'd say that explicitly because it is where the guidance moved. Auth goes in a setup project instead: a real test file matched by testMatch, with the browser projects declaring dependencies setup. It still runs once and still saves storageState, so the tests that follow start already logged in rather than each repeating a five second login, which across a few hundred tests is a couple of thousand seconds of pure waste deleted in one place. The difference is that when login breaks, and login is the thing most likely to break, a setup project gives me a trace and a named failure in the report, while globalSetup gives me a raw stack trace. I write the per-role login as a loop over the roles rather than three copy-pasted blocks, so adding a role is one line. That also answers the follow-up about how tests use it: the browser projects depend on the setup project and load the saved storageState for their role.*

Key points to hit

(1) Runs once before any worker starts, and OUTSIDE the runner: no fixtures, no trace. **(2)** Keep it for browser-less env work: health check, DB seed, mock server, env vars. **(3)** Health-check the environment first; abort fast if it is down. **(4)** Auth moved OUT: login belongs in a setup project with dependencies: ['setup']. **(5)** Same storageState saving, but a broken login now gets a trace and a report line. **(6)** The easy mistake: reciting the old login-in-globalSetup answer, it reads a version behind.

CODE

```
// global.setup.ts, browser-less environment work only
import { FullConfig } from '@playwright/test';

export default async function globalSetup(config: FullConfig) {
const baseURL = config.projects[0].use.baseURL!;
const health = await fetch(`${baseURL}/health`);
if (!health.ok) throw new Error(`Environment ${baseURL} is not healthy.`);
}

// auth.setup.ts, a REAL test, so a broken login gets a trace
import { test as setup } from '@playwright/test';

for (const role of ['admin', 'user', 'viewer'] as const) {
setup(`authenticate as ${role}`, async ({ page }) => {
await page.goto('/login');
await page.getByLabel('Email').fill(process.env[`${role.toUpperCase()}_EMAIL`]!);
await page.getByLabel('Password').fill(process.env[`${role.toUpperCase()}_PASSWORD`]!);
await page.getByRole('button', { name: 'Sign in' }).click();
await page.waitForURL('**/dashboard');
await page.context().storageState({ path: `.auth/${role}.json` });
});
}

// playwright.config.ts
// { name: 'setup', testMatch: /.*\.setup\.ts/ },
// { name: 'chromium', dependencies: ['setup'],
// use: { storageState: '.auth/user.json' } },
```

Q 23 LEVEL · Junior to Mid

How do you manage multiple environments in Playwright?

CONCEPT

Use **dotenv** with environment-specific files, a staging file, a production file, and load the right one based on a **TEST_ENV** variable in playwright.config.ts. Commit the files that hold **URLs and configuration structure**, never credentials; passwords are CI secrets injected at runtime. Wire npm scripts per environment so switching target is one command. Add a guard at the start of globalSetup: if BASE_URL is not set, throw a clear error immediately rather than letting hundreds of tests fail with confusing messages. The result is one suite that runs against any environment by changing a single variable, with secrets kept out of the repository.

INTERVIEW STRATEGY

Beyond the dotenv mechanics, the interviewer is listening for security discipline. State the dotenv-plus-TEST_ENV pattern, then make the boundary explicit: structure and URLs in the repo, secrets injected at runtime. The early BASE_URL guard shows you have debugged a misconfigured run. The failure mode is implying credentials live in the .env files. The follow-up is commonly ‘how do you handle secrets in CI?’, answer with CI secret injection, and add that you restrict production runs to a smoke subset on one browser.

How to phrase it

> *I use dotenv with one file per environment and pick the file from a TEST_ENV variable in the config, defaulting to staging. But the part that actually matters, and the part interviewers are really probing, is the discipline about what goes in those files. I commit the URLs and the shape of the configuration, but never credentials. Passwords are CI secrets injected at runtime, so nothing sensitive ever lands in the repository, that is also my answer to the follow-up about handling secrets in CI. I add per-environment npm scripts so pointing the suite at staging or production is a single command, and I deliberately restrict production runs to a smoke subset on one browser rather than the full matrix, because I do not want a full regression hammering prod. The detail I always include is a guard at the very start of globalSetup: if BASE_URL is not set, throw a clear error straight away. Otherwise you get hundreds of tests failing with confusing messages when the real problem is one missing variable, and that is an afternoon lost to a non-issue. The net effect is that the same suite runs against any environment by changing one variable, and secrets stay out of source control entirely.*

Key points to hit

(1) dotenv + TEST_ENV selects the environment file. **(2)** Commit URLs and structure; never commit credentials. **(3)** Passwords are CI secrets injected at runtime. **(4)** Guard BASE_URL early with a clear error to avoid confusing mass failures. **(5)** Asked next: secrets in CI, injection; restrict prod to smoke on one browser. **(6)** Trap: implying credentials live in the committed .env files.

CODE

```
// playwright.config.ts
import dotenv from 'dotenv';
dotenv.config({ path: `.env.${process.env.TEST_ENV || 'staging'}` });

// package.json
{
"scripts": {
"test:staging": "TEST_ENV=staging playwright test",
"test:production": "TEST_ENV=production playwright test --project=chromium --grep
@smoke",
"test:smoke": "TEST_ENV=staging playwright test --grep @smoke"
}
}
```

Q 24 LEVEL · Junior

How does Playwright manage browser downloads and versions?

CONCEPT

Playwright downloads specific browser builds tested against each release, there is **no manual driver management**. Upgrading the Playwright version automatically pulls the compatible browser binaries, so the version skew that plagues driver-based tools simply does not happen. Install with **npx playwright install**, and in Linux CI add **--with-deps** to pull the OS-level libraries the browsers need. In CI, cache the browser binaries keyed on the lockfile so they only re-download when the Playwright version actually changes, saving a few minutes per run. The contrast with Selenium is concrete: a browser auto-update that breaks a mismatched driver is a recurring failure there, and

a non-issue here because the browser is pinned to the framework.

INTERVIEW STRATEGY

On the surface this is trivia; the interviewer is really checking whether you have felt CI pain. Frame zero driver management as an underrated operational win and make it tangible with the Selenium driver-mismatch story, that is what shows experience. The CI caching detail, keyed on the lockfile, signals you run this at scale. The failure mode is a flat 'Playwright downloads its own browsers' with no why-it-matters. The follow-up is often 'how do you speed up CI runs?', the browser-binary cache is a ready answer.

How to phrase it

> *Playwright pins browser builds to each release and downloads them for you, so there is no separate driver to manage, and when I upgrade the Playwright version it fetches the compatible browsers automatically. That sounds like trivia until you have lived the alternative, which is what makes this question worth answering with a story. In our Selenium setup, Chrome would auto-update and the ChromeDriver version would no longer match, and CI would start failing randomly every few weeks; the fix was a manual driver bump and a redeploy, and it always landed at a bad time. With Playwright the browser is tied to the framework version, so that whole class of flakiness simply disappears. Operationally, I install with npx playwright install, and in Linux CI I add --with-deps to pull the system libraries the browsers need. The detail I would offer for the follow-up about speeding up CI is caching the browser binaries keyed on the lockfile, so they only re-download when the Playwright version actually changes, that saves two to three minutes per run on every job. Zero driver management is honestly one of the better day-to-day wins of the tool, even if it is not the flashy one people mention first.*

Key points to hit

(1) Playwright pins and downloads browsers per release, no driver mgmt. **(2)** Upgrading Playwright pulls compatible browsers automatically. **(3)** npx playwright install; add --with-deps for Linux CI system libs. **(4)** Eliminates the Selenium driver-mismatch class of CI flakiness. **(5)** Follow-up: speed up CI, cache binaries keyed on the lockfile. **(6)** Where this goes wrong: stating the fact with no why-it-matters story.

CODE

```
# Install all browsers
npx playwright install

# CI on Linux: include OS-level dependencies
npx playwright install --with-deps chromium

# GitHub Actions: cache keyed on the lockfile
- uses: actions/cache@v4
with:
path: ~/.cache/ms-playwright
key: playwright-${{ hashFiles('package-lock.json') }}
```

Q 25

LEVEL · Junior

What is the difference between @playwright/test and the playwright library?

CONCEPT

@playwright/test is the full test framework: it includes fixtures, the expect assertion library with web-first retrying assertions, **reporters**, parallelism, and retry logic. The lower-level playwright package is browser automation **without a test runner**, you get Browser, Context, and Page, but none of the test scaffolding. Use the test package for any automation framework, because fixtures, retries, parallel execution and the expect API all come for free. Reach for the bare library only for utility work: screenshot-generation pipelines, PDF rendering, a scraper, or a serverless function that drives a browser. Knowing the split shows you understand the layering rather than treating Playwright as one monolithic thing.

INTERVIEW STRATEGY

Keep it crisp, one is the framework, one is the engine, but the interviewer is checking judgement, so show it by naming the narrow cases where the bare library is the right call. That contrast turns a definition into evidence of real usage. The pitfall is only defining the two and stopping. The follow-up is sometimes 'so when would you ever use the bare library?', have a concrete utility example ready, like a screenshot pipeline or a serverless browser task.

How to phrase it

> *They sit at different layers. The playwright package is the browser-automation engine, Browser, Context, Page, with no test runner around it. @playwright/test is the full framework built on top: fixtures, the expect assertion library with retrying web-first assertions, reporters, parallel execution and retries. For any automation framework I use @playwright/test, because all of that scaffolding comes for free and I would otherwise be rebuilding it badly. To show I have actually used both, and to answer the follow-up about when you would ever reach for the bare library, I use it only when I am writing a utility rather than a test suite, a pipeline that generates marketing screenshots at several viewport sizes, a PDF renderer, a small scraper, or a serverless function that needs to drive a browser without the test machinery. In those cases the test runner would just be dead weight. Knowing the distinction matters because it shows I have chosen between them deliberately and I understand how the toolset is layered, rather than treating Playwright as a single undifferentiated thing. So: framework for tests, engine for tooling, and a clear reason each time.*

Key points to hit

(1) @playwright/test: fixtures, expect, reporters, parallelism, retries. **(2)** playwright: the bare automation engine, no test runner. **(3)** Use the test package for any framework: scaffolding for free. **(4)** Use the library for scripts: screenshots, PDF, scraping, serverless. **(5)** Expect next: a concrete bare-library use case, e.g. a screenshot pipeline. **(6)** What loses the point: defining both and stopping, with no judgement shown.

Q 26 LEVEL · Senior

What exactly does BrowserContext isolate, and why is it the unit of test parallelism?

CONCEPT

A BrowserContext is Playwright's incognito-equivalent isolation boundary, and it's the unit of parallelism because everything that could cause cross-test contamination lives inside the context. A context isolates: cookies, **localStorage** and **sessionStorage**, **IndexedDB**, service workers, cache, HTTP authentication state, permissions grants, geolocation, color scheme, locale, timezone, viewport size, user agent, and network interception routes. What it does not isolate is anything OS-level, the file system, environment variables, downloaded files in /tmp, because those live outside the browser process. This is why Playwright creates a fresh context per test by default (the test fixture builds one and disposes it), and why each context is cheap (no new browser process needed). Multiple contexts share one browser binary; one browser hosts dozens of contexts running tests in parallel.

INTERVIEW STRATEGY

They are looking to see what context actually isolates and what it does not. List the in-browser state it isolates (cookies, storage, cache, perms, viewport, network routes), then explicitly call out what stays shared at OS level (filesystem, /tmp, env vars). The 'fresh context per test by default plus cheap' framing is the seniority marker because it explains why parallelism scales. The mistake is assuming context isolates downloads to disk. Expect the follow-up ‘what about files written by tests?’, answer not isolated; collisions happen if two tests write to the same path.

How to phrase it

> *A BrowserContext is Playwright's incognito-equivalent isolation boundary, and it's the unit of parallelism because everything that could cause cross-test contamination lives inside the context. The list of what a context isolates is long and worth knowing: cookies, localStorage and sessionStorage, IndexedDB, service workers, cache, HTTP authentication state, permissions grants like geolocation and notifications, color scheme, locale, timezone, viewport size, user agent, and the network interception routes I've configured. Two parallel tests in two contexts can simultaneously intercept the same URL with different mocks without interfering. What context does not isolate is anything OS-level, the file system, environment variables, downloaded files in slash tmp, because those live outside the browser process entirely. That's also my answer to the follow-up about files written by tests: not isolated. If two parallel tests both write to slash tmp slash report dot pdf, they collide, and the second one wins or both corrupt. The pattern is to namespace any disk paths by worker ID or test title. The framing I would lead with, because it explains why Playwright parallelism scales the way it does, is fresh context per test by default plus cheap. The test fixture builds a context and disposes it, each one is cheap because no new browser process is needed, multiple contexts share one browser binary, and one browser hosts dozens of contexts running tests in parallel. That's why context, not page or browser, is the right granularity for parallelism, it's small enough to be cheap to spin up but big enough to isolate everything that matters in the browser. The trap is assuming context isolates downloads to disk, which makes flaky file-related tests when parallelism is enabled.*

Key points to hit

(1) Context isolates browser state: cookies/storage/cache/perms/locale/routes. **(2)** Does NOT isolate OS state: filesystem, env vars, /tmp. **(3)** Fresh context per test by default (test fixture builds + disposes). **(4)** Multiple contexts share one browser binary: cheap to spin up. **(5)** They usually ask next: namespace disk paths by worker ID, OS isn't isolated. **(6)** The common miss: assuming downloads to /tmp are isolated, they collide in parallel.

CODE

```
// Default flow: fresh context per test, automatic
test('isolated test', async ({ page }) => {
// `page` came from a fresh context created by the test fixture
// Cookies, localStorage, cache, permissions: all empty
});

// Explicit context for multi-user scenarios
test('two users', async ({ browser }, testInfo) => {
const ctxA = await browser.newContext({ storageState: 'auth/user-a.json' });
const ctxB = await browser.newContext({ storageState: 'auth/user-b.json' });
// Two contexts inside one browser: cheap, fully isolated cookies/storage

// BUT: filesystem is NOT isolated. Namespace by worker:
const downloadPath = `tmp/worker-${testInfo.parallelIndex}/report.pdf`;
await Promise.all([ctxA.close(), ctxB.close()]);
});
```

Q 27 LEVEL · Mid to Senior

Explain the Page/Frame hierarchy and how navigation events flow through it.

CONCEPT

A Page contains a tree of Frame objects, every page has at least one (the main frame), and additional frames are added when iframes are loaded. **page.mainFrame()** returns the top-level frame; **page.frames()** returns the full flat list. Each Frame has its own URL, navigation state, and execution context, which is why frameLocator drills into one specifically. Navigation events fire per frame: framenavigated when a frame navigates, frameattached when one is added, framedetached when one is removed. Page-level events like **page.on('load')** and page.on('domcontentloaded') fire only for the main frame's navigation by design, because iframe loads would otherwise flood the test with events for content the test doesn't care about. The implication: when you need iframe-level navigation hooks, use page.on('framenavigated') and filter by frame; when you need main-page navigation, use the simpler page-level events.

INTERVIEW STRATEGY

The point of this one is to see hierarchy and event discipline. Lead with the Page-contains-Frame-tree structure, then split event ownership: page-level events fire for main frame only; frame events fire per frame. The 'why', to avoid flooding tests with iframe events, is the test of seniority on this. The classic error is expecting page.on('load') to fire for iframe loads. They usually follow with ‘how do you wait for an iframe to navigate?’, answer page.on framenavigated filtered by frame, or frameLocator's auto-waiting takes care of it implicitly.

How to phrase it

> *A Page contains a tree of Frame objects. Every page has at least one frame, the main frame, and additional frames are added to the tree when iframes load. The API surface: page.mainFrame returns the top-level frame, page.frames returns the flat list of all frames including the main one and any nested iframes transitively. Each Frame has its own URL, its own navigation state, and its own JavaScript execution context, which is why frameLocator drills into one specifically when I'm working inside iframes, the frame is the unit of scope. Navigation events fire per frame at the frame level: framenavigated when a frame navigates, frameattached when one is added to the tree, framedetached when one is removed. But page-level events are deliberately different, and this is the design choice worth understanding. page.on load and page.on domcontentloaded fire only for the main frame's navigation, not for iframe loads. The reason, and the framing that lands in interviews, is that iframe loads would otherwise flood the test with events for content the test doesn't care about, a single page with five analytics iframes would emit six load events, and any test code listening for load would have to filter every time. So Playwright makes the common case clean: page-level events for main-frame navigation. For the follow-up question about how to wait for an iframe specifically to navigate. Two options. page.on framenavigated and filter the callback's Frame argument by URL or name, useful when I genuinely need to react to an iframe load, a third-party widget completing init, say. Or, more often, frameLocator handles it implicitly: when I act through frameLocator on an element inside the iframe, auto-waiting waits for the frame to load and the element to be actionable in one. The failure mode is expecting page.on load to fire for iframe loads.*

Key points to hit

(1) Page contains a tree of Frames: mainFrame + iframes (transitively). **(2)** Each Frame has its own URL, navigation state, execution context. **(3)** Frame events fire per-frame: framenavigated, frameattached, framedetached. **(4)** Page events (load, domcontentloaded) fire ONLY for main frame. **(5)** The next question: page.on('framenavigated') with filter for iframe waits. **(6)** Avoid: expecting page.on('load') to fire for iframe loads, it doesn't.

CODE

```
// Page-level event: main frame only
page.on('load', () => console.log('main frame loaded'));

// Frame-level events: fire for every frame
page.on('framenavigated', frame => {
if (frame === page.mainFrame()) return; // skip main
console.log('iframe navigated:', frame.url());
});
page.on('frameattached', f => console.log('attached:', f.url()));
page.on('framedetached', f => console.log('detached:', f.url()));

// Inspect the tree
console.log('frame count:', page.frames().length);
console.log('main URL:', page.mainFrame().url());

// Implicit iframe wait: frameLocator auto-waits on the frame + element
await page.frameLocator('iframe[title="Payment"]')
.getByLabel('Card number')
.fill('4111 1111 1111 1111');
```

Q 28 LEVEL · Mid to Senior

Walk through the request/response lifecycle in Playwright and the four events it surfaces.

CONCEPT

Every network request a page makes flows through four events Playwright exposes on the page object: request fires when the request is initiated; response fires when the response headers arrive (before body); requestfinished fires after the response body has been received in full; requestfailed fires when the request fails (network error, aborted, blocked by route). These events let you observe and assert on the network layer without instrumenting the application. The **page.route()** interception sits between request and response, it can fulfill (return a mock), continue (forward with modifications), or abort (fail the request) before the network ever fires. **waitForRequest** and **waitForResponse** are convenient wrappers around the events for asserting that a specific request fired with specific characteristics during a test action.

INTERVIEW STRATEGY

You are being asked to show network-layer understanding. List the four events in order, request, response, requestfinished, requestfailed, then place page.route between request and response. The waitForRequest/Response wrappers are the practical convenience. Where this goes wrong is conflating response (headers arrived) with requestfinished (body complete). The next question is normally ‘when do you use which?’, answer response when checking headers or status; requestfinished when needing the full body; requestfailed for retry logic on aborted requests.

How to phrase it

> *Every network request a page makes flows through four events Playwright exposes on the page object, and knowing the order matters because each event marks a different moment in the lifecycle. First, request fires when the request is initiated by the browser, the URL, method, headers, body are all known at this point. Second, response fires when the response headers arrive from the server, before the body has been received, so I can read status code and headers but the body is still streaming. Third, requestfinished fires after the response body has been fully received, if I need the JSON body, this is the event to await on. Fourth, requestfailed fires when the request fails for any reason, a network error, an aborted request, blocked by a page.route abort. Page.route interception sits between request and response in this flow, it can fulfill with a mock response, continue with modifications to the outgoing request, or abort to fail it, all before the network ever fires. So the order is request, then optional route handling, then response, then requestfinished or requestfailed. The practical wrappers are waitForRequest and waitForResponse, which let me wait for a specific request or response matching a URL pattern during a test action, useful for asserting that the right API call fired with the right parameters. That's also my answer to the follow-up about when to use which event. Response when I'm checking status code or response headers. RequestFinished when I need the full body of the response for assertions or downstream logic. RequestFailed for retry logic on aborted or failed requests. The mistake is conflating response with requestfinished, listening on response and then trying to read the body before the body has actually arrived, which produces a race condition that flakes intermittently.*

Key points to hit

(1) Four events: request -> (route) -> response -> requestfinished/requestfailed. **(2)** request: initiated; response: headers arrived; finished: body complete; failed: any failure. **(3)** page.route() sits between request and response, fulfill/continue/abort. **(4)** waitForRequest/waitForResponse: convenient wrappers for specific URL patterns. **(5)** Asked next: response for status/headers; requestfinished for body; failed for retry. **(6)** The easy mistake: reading body on 'response' event, body may not have arrived yet.

CODE

```
// Listen to the four events in order
page.on('request', req => console.log('->', req.method(), req.url()));
page.on('response', res => console.log('<-', res.status(), res.url()));
page.on('requestfinished', req => console.log('✓', req.url(), 'body received'));
page.on('requestfailed', req => console.log('✗', req.url(), req.failure()?.errorText));

// page.route sits between request and response
await page.route('**/api/users/**', async route => {
if (route.request().method() === 'GET') {
await route.fulfill({ status: 200, body: JSON.stringify(mockUsers) });
} else {
await route.continue(); // forward unchanged
}
});

// Wait for specific request/response in a test action
const [response] = await Promise.all([
page.waitForResponse(r => r.url().includes('/api/orders') && r.status() === 200),
page.getByRole('button', { name: 'Place Order' }).click(),
]);
const body = await response.json(); // safe here, body has arrived
```

Q 29 LEVEL · Senior

What is the project model in Playwright, and how do project dependencies enable setup-then-test flows?

CONCEPT

A **project** in playwright.config.ts is a named configuration with its own browser, viewport, storageState, testMatch, and other settings. Projects are the unit of parallelism across configurations, you can have 'chromium', 'firefox', 'mobile-chrome' as projects and Playwright runs each against the same tests, in parallel by default. Project dependencies (Playwright 1.31+) let one project run **before another** via the dependencies array. The canonical use: a **'setup' project** that authenticates and writes storageState files, listed as a dependency of the actual test projects. The setup runs once across the suite (not per test), and the test projects all start with the auth state already prepared. This replaces the older globalSetup pattern for many cases and has the advantage that the setup is itself a test file with reporting, retries, and trace if it fails.

INTERVIEW STRATEGY

What scores here is showing project-level architecture. Lead with projects as a unit of parallelism across configurations, then make project dependencies the centerpiece because they enable the setup-as-a-test-project pattern. The 'setup-is-a-test-with-reporting-and-retries' framing marks senior judgement. The classic error is reaching for globalSetup for everything when project dependencies handle auth-setup more cleanly. A common follow-up is ‘when do you still use globalSetup?’, answer for things that aren't tests, environment health checks, fixture-data seeding via raw API calls, anything that doesn't need reporting.

How to phrase it

> *A project in playwright.config.ts is a named configuration with its own browser, viewport, storageState, testMatch, and other settings. Projects are the unit of parallelism across configurations, I can have chromium, firefox, mobile-chrome as separate projects and Playwright runs the same tests across each in parallel by default. So one test file is automatically run on however many browser-viewport combinations I configure, without duplicating test code. The newer and more interesting half, from a 1.31+ standpoint and the part senior interviewers probe on, is project dependencies. The dependencies array lets one project run before another. The canonical use is a setup project that authenticates users and writes storageState files to disk, listed as a dependency of the actual test projects, chromium depends on setup, firefox depends on setup, etc. The setup runs once across the suite, not per test, and the test projects all start with the auth state already prepared. This replaces the older globalSetup pattern for many cases and has real advantages: the setup is itself a test file, so it gets reporting, retries, and trace if it fails. With globalSetup, a failure in setup is a stack-trace in stderr; with a setup project, it's a properly-reported test with the same debugging story as any other test. That's also my answer to the follow-up about when I still use globalSetup. For things that aren't tests: verifying the environment is healthy before the suite starts, seeding fixture data through raw API calls, anything that doesn't need reporting or doesn't fit naturally as a test. But for auth setup and anything that benefits from being treated as a first-class test, project dependencies are the modern pattern. The thing to avoid is putting everything in globalSetup out of habit when project dependencies give better observability.*

Key points to hit

(1) Project = named config (browser, viewport, storageState, testMatch). **(2)** Projects are the unit of parallelism across configurations. **(3)** dependencies array (1.31+) makes one project run before another. **(4)** Setup-as-project: auth runs once; gets reporting/retries/trace like a test. **(5)** Follow-up: globalSetup for non-test work (env health check, data seed). **(6)** Trap: defaulting to globalSetup when project dependencies give better observability.

CODE

```
// playwright.config.ts: setup project as dependency
export default defineConfig({
projects: [
{ name: 'setup', testMatch: /.*\.setup\.ts/ },
{
name: 'chromium',
use: { ...devices['Desktop Chrome'], storageState: 'auth/admin.json' },
dependencies: ['setup'], // runs after setup
},
{
name: 'firefox',
use: { ...devices['Desktop Firefox'], storageState: 'auth/admin.json' },
dependencies: ['setup'],
},
],
});

// tests/auth.setup.ts: the setup is itself a test file
import { test as setup } from '@playwright/test';
setup('authenticate as admin', async ({ page }) => {
await page.goto('/login');
// fill credentials, sign in
await page.context().storageState({ path: 'auth/admin.json' });
// Failure here -> reported as a test, with trace, with retries
});
```

Q 30 LEVEL · Senior

What is the full test-runner lifecycle, and in what order do beforeAll, fixtures, beforeEach, the test, and teardown run?

CONCEPT

Knowing the exact ordering is what stops mysterious "why did my setup run twice" bugs. The lifecycle for one test in one worker, top to bottom: 1. Worker starts; 2. Worker fixtures set up (worker-scoped fixtures); **3. test.beforeAll** (runs once per file per worker); 4. Test fixtures set up (test-scoped fixtures, in dependency order); **5. test.beforeEach**; 6. The test body; **7. test.afterEach**; 8. Test fixtures torn down (reverse order); 9. test.afterAll (after the last test in the file); 10. Worker fixtures torn down when the worker shuts down. The non-obvious facts: beforeAll runs per worker, not globally, if 4 workers run the same file, beforeAll fires 4 times; fixtures resolve in dependency-graph topological order, not file order; test-scoped fixtures are torn down in reverse order to guarantee dependencies survive their consumers.

INTERVIEW STRATEGY

They want exact ordering. List the 10 steps top-to-bottom: worker, worker fixtures, beforeAll, test fixtures, beforeEach, test, afterEach, test fixtures down, afterAll, worker fixtures down. The two non-obvious points are beforeAll-runs-per-worker and reverse-order-teardown, mention both because they're where interviews probe. Where this goes wrong is thinking beforeAll runs once globally. The follow-up is often ‘what if I have 4 workers and one test file?’, answer beforeAll fires 4 times, once per worker.

How to phrase it

> *Knowing the exact ordering is what stops the mysterious why-did-my-setup-run-twice bugs. Let me walk it top to bottom for one test in one worker. Step one, the worker starts. Step two, worker-scoped fixtures set up: these are fixtures declared with scope worker, and they live for the entire worker lifetime across many tests. Step three, test dot beforeAll runs, and this is one of the non-obvious facts worth calling out: beforeAll runs per worker, not globally. If I have four workers running the same test file, beforeAll fires four times, not once. That answers the follow-up. Step four, test-scoped fixtures set up in dependency-graph topological order, not file order, if my authenticatedPage fixture depends on the page fixture, page resolves first regardless of how I declared them. Step five, test dot beforeEach runs. Step six, the test body executes. Step seven, test dot afterEach. Step eight, test-scoped fixtures are torn down in reverse order of setup. The reverse-order rule is the second non-obvious detail, fixtures dispose in the opposite order they were created so dependencies are guaranteed to survive their consumers. If page came up first and authenticatedPage came up second, authenticatedPage tears down first and page tears down last. Step nine, test dot afterAll runs after the last test in the file. Step ten, worker-scoped fixtures tear down when the worker itself shuts down. The framing I would lead with, because it lands as the senior insight, is that ordering is precise and mostly obvious, but the two facts that catch people out are beforeAll fires per worker not globally, and fixtures tear down in reverse order. Both are exam material for senior interviews.*

Key points to hit

(1) 10 steps: worker -> worker fixtures -> beforeAll -> test fixtures -> beforeEach -> test -> afterEach -> fixture teardown -> afterAll -> worker teardown. **(2)** beforeAll runs PER WORKER, not globally: 4 workers -> 4 invocations. **(3)** Fixtures resolve in dependency-graph topological order, not file order. **(4)** Fixtures tear down in REVERSE order: dependencies outlive consumers. **(5)** Expect next: 4 workers + 1 file -> beforeAll fires 4 times. **(6)** Where this goes wrong: assuming beforeAll runs once across the whole suite.

CODE

```
// Order of execution (1 test in 1 worker)
//
// 1. worker starts
// 2. worker fixtures up (scope: 'worker')
// 3. test.beforeAll (once per worker per file)
// 4. test fixtures up (test-scoped, topological order)
// 5. test.beforeEach
// 6. TEST BODY
// 7. test.afterEach
// 8. test fixtures down (REVERSE order)
// 9. test.afterAll (after last test in file)
// 10. worker fixtures down (when worker shuts down)
//
// 4 workers × 1 file -> beforeAll fires 4 times, NOT 1
```

Q 31

LEVEL · Senior

What does Playwright's trace viewer actually capture, and what's the performance cost of trace: 'on'?

CONCEPT

A trace is a **zip file** containing a complete recording of a test run. It captures: DOM snapshots at every action (before and after, so you can scrub forward and back); network requests and responses with full headers and bodies; console logs; action timing for every Playwright command; **screenshots** at key moments; the source code of the test for the call-site view. The cost is real, and worth stating honestly rather than with invented figures: trace: 'on' makes every action snapshot the DOM and capture network, so it slows the run and produces a trace zip for every test, passing ones included. How much depends entirely on the suite, so measure it on your own suite rather than quoting a number. The right config in CI is trace: 'on-first-retry', zero cost on passing tests, full trace on flaky ones, or 'retain-on-failure', which records every test but deletes the trace file after passing tests. The discipline: 'on' for local debugging, **'on-first-retry' for CI**.

INTERVIEW STRATEGY

This question is there to see trace architecture and cost awareness. List what's captured (DOM, network, logs, timing, screenshots, source), then be honest about the cost: it is real but suite-specific, so say you measure it rather than quoting a figure you cannot back up. An invented number is easy for an interviewer to challenge. The on-first-retry default for CI is the test of seniority on this because it shows operational discipline. The classic error is trace: 'on' in CI, paying for traces on every green run. Expect the follow-up ‘what mode do you use in CI?’, answer on-first-retry: zero cost on green, full trace on flake.

How to phrase it

> *A trace is a zip file containing a complete recording of a test run, and it's one of the most powerful debugging tools Playwright provides. What it captures is comprehensive: DOM snapshots at every action, both before and after each step, so I can scrub forward and back in the timeline to see exactly what the page looked like; network requests and responses with full headers and bodies; console logs from the page; action timing for every Playwright command, showing what took how long; screenshots at key moments; and the source code of the test for the call-site view, so I can jump from a trace entry to the line of code that made the call. The performance cost, and the part senior interviewers probe on, is real. Trace on makes every action take a DOM snapshot and capture network, so the run gets slower and you get a trace zip for every test, including the ones that passed. I would not quote a headline number for that, because it depends completely on the suite, and I'd rather say I measured it on ours than repeat a figure Playwright never published. What is certain is the shape of it: on is the wrong default for CI, because it costs time and artifact storage on every green run for data nobody opens. The right config in CI is trace on-first-retry, which is zero cost on passing tests and full trace on flaky ones, or retain-on-failure, which records every test but deletes the trace file after passing tests. That's also my answer to the follow-up about what mode I use in CI: on-first-retry. The discipline I would summarise is trace on for local debugging, trace on-first-retry for CI, same observability, fraction of the cost on green builds. The failure mode is setting trace on in CI because it sounds safer; you pay on every run for trace data that is unused almost all of the time, because most tests pass.*

Key points to hit

(1) Trace = zip with DOM snapshots, network, logs, timing, screenshots, source. **(2)** Cost is real but suite-specific: measure it, don't quote a figure. **(3)** trace: 'on' for local debugging only, too expensive in CI. **(4)** trace: 'on-first-retry' for CI, zero cost on green, full trace on flake. **(5)** They usually ask next: on-first-retry is the CI default, same observability, lower cost. **(6)** What loses the point: trace: 'on' in CI, paying on every green run for data nobody opens.

CODE

```
// playwright.config.ts: right trace mode per environment
export default defineConfig({
use: {
trace: process.env.CI ? 'on-first-retry' : 'on',
// CI: zero cost on passing tests, full trace on retries
// local: trace every test for debugging
},
});

// Inspect a trace
// 1. Download trace.zip from CI artifacts
// 2. https://trace.playwright.dev (drag-and-drop the zip)
// 3. Scrub timeline; see DOM + network at every action
//
// Alternative modes
// 'on' every test: expensive
// 'off' never
// 'on-first-retry' only when a test retries (CI default)
// 'retain-on-failure' record every test, delete on pass
```

Q 32 LEVEL · Senior

What is Playwright's reporter API, and how would you build a custom reporter?

CONCEPT

A reporter is a class implementing the Reporter interface that receives events from the test runner during execution: **onBegin** when the suite starts (with config and root suite), **onTestBegin** per test, **onTestEnd** per test (with result, status, duration), onStepBegin/onStepEnd per test.step, onError for global errors, onEnd when the suite finishes (with the overall result). Custom reporters are exported from a TypeScript module and listed in the reporter config option, where you can also pass options to the constructor. Real use cases: posting Slack notifications on suite failure, writing test results to an internal observability system, generating a custom HTML format the standard reporter doesn't support, feeding a requirements traceability dashboard. The principle: reporters are the integration layer between Playwright and the rest of the engineering stack, JIRA, Slack, internal dashboards, custom alert systems.

INTERVIEW STRATEGY

They want to see custom integration ability. Walk through the Reporter interface lifecycle: onBegin, onTestBegin/End, onStepBegin/End, onError, onEnd. Real use cases (Slack on failure, JIRA traceability) make it concrete. The 'integration layer between Playwright and engineering stack'

framing is the seniority marker. The pitfall is treating the built-in HTML reporter as the only option. They usually follow with 'what's a real reporter you've built?', answer specific: requirements traceability dashboard or Slack-on-failure with the first three failure messages.

How to phrase it

> *A reporter is a class implementing the Reporter interface that receives events from the test runner during execution, and it's the right tool whenever I need to plug Playwright into the rest of the engineering stack. The lifecycle is event-driven: onBegin when the suite starts, with the config and root suite passed in. OnTestBegin per test, onTestEnd per test with the result, status, and duration so I can react to pass or fail. OnStepBegin and onStepEnd per test.step if I want step-level granularity. OnError for global errors outside the per-test context. OnEnd when the suite finishes, with the overall result, this is where I'd post a summary somewhere. Custom reporters are exported from a TypeScript module and listed in the reporter config option, where I can also pass options to the constructor. The framing I would lead with, because it lands as the senior insight, is that reporters are the integration layer between Playwright and the rest of the engineering stack, JIRA, Slack, internal dashboards, custom alert systems. Real use cases I've seen and that's also my answer to the follow-up about reporters I've actually built: a Slack reporter that posts on suite failure with the first three failure messages and a link to the trace; a requirements traceability reporter that parses at-REQ tags from test annotations and writes a coverage dashboard to S3; a reporter that ships test results to an internal observability system so we can graph flakiness trends over time. The mistake is treating the built-in HTML reporter as the only option, it's excellent for local debugging but it lives in isolation. A custom reporter is how I make test results visible where my team actually looks: in Slack, in JIRA, on a dashboard. Not in a folder of HTML files nobody opens.*

Key points to hit

(1) Reporter interface: onBegin, onTestBegin/End, onStepBegin/End, onError, onEnd. **(2)** Class-based; exported and listed in the reporter config option. **(3)** Integration layer: Slack, JIRA, observability, traceability dashboards. **(4)** Receives full result objects: status, duration, error, retries. **(5)** The next question: real reporters, Slack-on-failure, REQ-traceability, flakiness trends. **(6)** The common miss: treating built-in HTML reporter as the only option, it lives in isolation.

CODE

```
// reporters/slack-on-failure.ts
import type { Reporter, TestCase, TestResult, FullResult } from
'@playwright/test/reporter';

export default class SlackReporter implements Reporter {
private failed: { test: TestCase; result: TestResult }[] = [];

onTestEnd(test: TestCase, result: TestResult): void {
if (result.status === 'failed' || result.status === 'timedOut') {
this.failed.push({ test, result });
}
}

async onEnd(result: FullResult): Promise<void> {
if (this.failed.length === 0) return;
const summary = this.failed.slice(0, 3).map(f =>
`• ${f.test.title}: ${f.result.error?.message?.slice(0, 200)}`
).join('\n');
await postToSlack(`✗ ${this.failed.length} tests failed\n${summary}`);
}
}

// playwright.config.ts
// reporter: [['list'], ['html'], ['./reporters/slack-on-failure.ts']]
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What kind of connection does Playwright maintain to the browser?

A) A new stateless HTTP request per command
B) A persistent bidirectional WebSocket
C) A polling REST endpoint
D) A shared-memory channel

2. Which object is the isolated session equivalent to an incognito window?

A) Browser
B) Page
C) BrowserContext
D) Worker

3. Why prefer trace: 'on-first-retry' over trace: 'on'?

A) It avoids storing traces for passing tests, saving storage
B) It produces higher-quality traces
C) It is required for parallel execution
D) It disables tracing in CI

4. What is the main purpose of saving storageState in a setup project?

A) To cache browser binaries
B) To configure reporters
C) To enable sharding
D) To let tests start already authenticated instead of logging in each time

5. Which package includes fixtures, the expect API, reporters and parallelism?

A) playwright
B) @playwright/browser
C) playwright-core
D) @playwright/test

Section 2 · Answer key

#	Answer	Why and where to revisit
1	B) A persistent bidirectional WebSocket	The persistent WebSocket is what enables real-time event subscriptions and therefore network interception and tracing. *Revisit Section 2.*
2	C) BrowserContext	BrowserContext has its own cookies, storage and permissions; each test gets a fresh one by default. *Revisit Section 2.*
3	A) It avoids storing traces for passing tests, saving storage	on-first-retry captures a trace only when a test failed and is retrying, avoiding wasteful traces for passing tests. *Revisit Section 2.*
4	D) To let tests start already authenticated instead of logging in each time	Saving an authenticated session once per role lets every later test reuse it, removing per-test login cost. A setup project is preferred over globalSetup because it runs inside the runner, so a broken login produces a trace and a report line. *Revisit Section 2.*
5	D) @playwright/test	@playwright/test is the full framework; the bare playwright library is automation without a test runner. *Revisit Section 2.*

SECTION 03

Locators and DOM Strategies

Locators are where most Playwright interviews spend real time, because how you find elements decides how stable your suite is. The big shift from Selenium is that a Playwright locator is a lazy, auto-retrying instruction, not a handle to a DOM node, so the stale-element error simply does not exist. These twenty-two questions cover the Locator API, the recommended locator priority, chaining and filtering, iframes, shadow DOM, dynamic elements, and dialogs, popups and downloads.

In this section

- Locators as lazy auto-retrying instructions: why stale elements vanish.
- The recommended locator priority and why getByRole leads.
- Chaining and filtering: filter({ hasText }) and filter({ has }) for tables.
- frameLocator for iframes: no switchTo state to manage.
- Shadow DOM piercing with plain CSS, no JavaScript traversal.
- Handling dynamic IDs by targeting stable, semantic characteristics.
- Dialogs, popups and downloads: listen before you trigger.

Q 33 LEVEL · Junior to Mid

What is the Locator API and how is it fundamentally different from Selenium's findElement?

CONCEPT

A Playwright **Locator** is **lazy**, it does not query the DOM until you perform an action; auto-retrying, it keeps trying until the element is actionable or the timeout is reached; and strict, it throws if more than one element matches, forcing you to be precise. Selenium's **findElement** queries the DOM immediately, throws if the element is not present at that instant, and returns a WebElement reference that can go stale if the DOM re-renders. The mental model that makes this click: a locator is an instruction, not a reference. You are telling Playwright what to find, not holding a handle to a node. Because every action re-queries with retries built in, the StaleElementReferenceException that plagues Selenium suites cannot occur.

INTERVIEW STRATEGY

You are being marked on whether you understand the source of Selenium flakiness, not whether you can list API names. Lead with the three properties, lazy, auto-retrying, strict, then deliver the instruction-not-a-reference mental model, which is the line that sticks and signals you teach this. The pitfall is reciting method differences without the why. The follow-up is usually 'what real Selenium pain does this remove?', answer with StaleElementReferenceException disappearing by construction, ideally with how often it used to bite you.

How to phrase it

> *A Playwright locator has three properties that set it apart, and I would frame the whole answer around them. It is lazy, so it does not touch the DOM until I act on it. It is auto-retrying, so it keeps trying until the element is actionable or it times out. And it is strict, so if my selector matches more than one element it throws and forces me to be precise. Selenium's findElement is the opposite on each count: it queries the DOM the instant you call it, throws immediately if the element is not there yet, and hands back a reference that goes stale the moment the page re-renders. The mental model I teach, and the one I would give the interviewer, is that a locator is an instruction, not a reference, you are describing what to find, not holding a handle to a node, so every click or fill re-queries with retries built in. That answers the follow-up about what real pain this removes: StaleElementReferenceException was our third most common Selenium failure type, and in Playwright it cannot happen at all. Eliminating one of your top false-positive failure modes by design, not by adding waits, is the kind of thing that made the migration worth it for us.*

Key points to hit

(1) Locators are lazy: no DOM query until an action runs. **(2)** Auto-retrying until actionable or timeout; strict on multiple matches. **(3)** Mental model: a locator is an instruction, not a reference. **(4)** Selenium findElement queries now and can go stale. **(5)** Asked next: stale elements cannot occur by construction, a top flake source gone. **(6)** Avoid: listing method differences without explaining the why.

CODE

```
// Selenium (Java): immediate query, can go stale
// WebElement b = driver.findElement(By.id("submit")); b.click();

// Playwright: lazy, auto-retrying
const button = page.locator('#submit'); // no query yet
await button.click(); // queries + retries

// Strict mode guards against ambiguity
// await page.locator('button').click();
// Error: strict mode violation, resolved to 6 elements
await page.getByRole('button', { name: 'Submit Order' }).click();
```

Q 34 LEVEL · Junior

What is the recommended locator priority, and why?

CONCEPT

Playwright recommends user-facing locators that mirror how a real person finds an element, ordered by resilience. **getByRole** first: semantic, accessibility-based, survives CSS refactors. **getByLabel** for form fields, tied to the visible label. **getByPlaceholder** as a fallback for unlabelled inputs. getByText for links and static content. getByTestId last, as a deliberate, stable contract with the frontend team via a data-testid attribute. The principle behind the order: target what the user perceives, role, label, text, rather than implementation details like CSS classes or DOM position. A bonus of getByRole is that it doubles as a free accessibility audit: if a role-based locator cannot find your control, a screen reader cannot either.

INTERVIEW STRATEGY

Anyone can recite the order; the interviewer wants to know whether you understand why it is ordered that way. Give the list quickly, then spend your words on getByRole and the accessibility-audit framing, which is the insight that separates a memorised answer from understanding. The mistake is rattling off five method names with no rationale. The likely follow-up is ‘when do you use getByTestId, then?’, answer that it is a deliberate last resort and a contract you push the frontend to honour, not a first reach.

How to phrase it

> *The recommended order, most resilient first, is getByRole, getByLabel, getByPlaceholder, getByText, then getByTestId, but the reason for the order is the part worth saying out loud. The top options target what the user actually perceives, the role of a control, its label, its text, rather than implementation details that change. getByRole is my default because it is tied to semantic meaning, not CSS: a button named Sign in is still found after a developer changes its class, moves it in the DOM, or reparents it. I also frame getByRole as a free accessibility audit, which usually gets a nod, if a role-based locator cannot find your button, a screen reader cannot find it either, so a failing locator is often a genuine accessibility bug. For the follow-up about getByTestId, I keep it as the last resort, and when I use it I treat it as a deliberate contract: I push the frontend team to add data-testid on critical interactive elements so we have a stable hook when role-based locators are not specific enough. The whole priority list exists for one reason: to keep locators surviving UI change instead of breaking every time someone touches the markup.*

Key points to hit

(1) Order: getByRole, getByLabel, getByPlaceholder, getByText, getByTestId. **(2)** Top options target user-perceived attributes, not implementation. **(3)** getByRole survives CSS/DOM refactors and is a free accessibility audit. **(4)** Follow-up: getByTestId is a deliberate last-resort contract with the frontend. **(5)** The order exists to keep locators surviving UI change. **(6)** The easy mistake: reciting five names with no rationale for the order.

CODE

```
await page.getByRole('button', { name: 'Submit' }).click();
await page.getByLabel('Email address').fill('user@test.com');
await page.getByPlaceholder('Search...').fill('laptops');
await page.getByText('Welcome back').waitFor();
await page.getByTestId('checkout-btn').click(); // stable contract
```

Q 35 LEVEL · Mid

How do you chain and filter locators for complex scenarios?

CONCEPT

Chaining scopes a locator inside a parent container, so you find an element relative to a known region rather than across the whole page. Filtering narrows a set of matches: **filter({ hasText })** keeps only matches containing given text, and **filter({ has })** keeps only matches that contain a given child locator. Together they read like a specification, find the table row that contains this order number, then click its View Details button, and they replace the brittle, position-based XPath that breaks when the DOM structure shifts. The chained-and-filtered version survives structural change because it targets content and roles, not absolute paths. This is the single most useful pattern for table and list interactions.

INTERVIEW STRATEGY

They are testing whether you have left XPath behind for something maintainable. Show that chaining plus filtering replaces complex XPath, and lean on the table-row example because it is the

one everyone has fought. Reading the locator aloud as a sentence is what demonstrates why it is more maintainable than a path expression. The trap is treating this as syntax trivia. The next question is normally 'how is this better than XPath?', answer that it targets content and roles, so it survives the structural changes that break position-based XPath.

How to phrase it

> *Chaining scopes a locator within a parent, and filtering narrows a set of matches by text or by the presence of a child. Together they replaced essentially all the complex XPath in our framework, and that is the story I would tell. The pattern I use constantly is filtering a table row by its content: find the row that has this order number, then within that row click View Details. It reads like a specification of what I want, and that is the point. To answer the follow-up about why this beats XPath, the equivalent XPath would be a long position-based expression that breaks the moment the table structure changes, when a column gets added or a wrapper div appears. The chained version survives those structural changes because it targets content and roles rather than absolute DOM paths. filter with has is the other half: I use it to select only the rows that contain, say, an Active status badge, then assert on their count. For tables and lists this is the most useful locator pattern in the whole API, and it is the thing I reach for long before I would even consider writing an XPath, in modern Playwright I almost never write one.*

Key points to hit

(1) Chaining scopes a locator inside a known parent container. **(2)** filter({ hasText }) narrows by text; filter({ has }) by a child locator. **(3)** Reads like a spec; replaces brittle position-based XPath. **(4)** Expect next: better than XPath because it targets content/roles, not paths. **(5)** Best pattern for tables and lists; survives structural change. **(6)** Trap: treating chaining/filtering as syntax trivia.

CODE

```
// Scope within a parent container
const loginForm = page.locator('form#login-form');
await loginForm.getByRole('button', { name: 'Sign in' }).click();

// filter({ hasText }): the table-row pattern
const targetRow = page.locator('tbody tr').filter({ hasText: 'ORDER-12345' });
await targetRow.getByRole('button', { name: 'View Details' }).click();

// filter({ has }): rows that contain an Active badge
const activeRows = page.locator('tbody tr').filter({
has: page.getByText('Active'),
});
await expect(activeRows).toHaveCount(3);
```

Q 36 LEVEL · Mid

How do you handle iframes with Playwright?

CONCEPT

frameLocator() returns a **FrameLocator** that scopes every subsequent locator to that iframe's DOM. You chain getByLabel, getByRole and the rest off the frame locator exactly as you would off

the page, and nesting works by chaining frameLocator calls. The contrast with Selenium is the important part: Selenium needs an explicit **switchTo().frame()** before interacting and switchTo().defaultContent() after, and if an exception fires in between you are stranded in the wrong frame context for the rest of the test. Playwright's frame locator carries no switching state, there is nothing to reset, and an error cannot leave you in a broken context. This matters most in payment testing, where Stripe, PayPal and Adyen all render fields inside iframes for PCI compliance.

INTERVIEW STRATEGY

What they want is to know if you have handled iframes in anger, not just read the API. Define frameLocator briefly, then sell it by contrast with Selenium's stateful switchTo and the stuck-in-the-wrong-frame failure mode, that contrast is the experience signal. Grounding it in payment iframes (Stripe, PayPal) shows real exposure. The mistake is just naming frameLocator. The follow-up is commonly 'where have you actually needed this?', have the PCI-compliant payment-field example ready.

How to phrase it

> *I use frameLocator, which returns a frame-scoped locator, and then I chain my normal getByLabel or getByRole calls off it as if it were the page; nested iframes just mean chaining frameLocator twice. But the reason this is so much better than Selenium, which is what I would emphasise, is statelessness. In Selenium you call switchTo frame before interacting and switchTo defaultContent after, and the trap is that if an exception fires in between, you never switch back and you are stranded in the wrong frame for the rest of the test, which produces baffling failures. Playwright's frame locator scopes automatically and carries no switching state, so there is nothing to reset and an error cannot strand you. For the follow-up about where I have actually needed this, it is payment testing. Stripe, PayPal and Adyen all render their card fields inside iframes for PCI reasons, so almost every checkout test touches a frame. With frameLocator I fill the card number, expiry and CVC inside the frame and click Pay, with no frame bookkeeping at all, and no risk of a half-completed test leaving the next assertion looking in the wrong document. That reliability is exactly what you want around money flows.*

Key points to hit

(1) frameLocator scopes all subsequent locators to that iframe; nest by chaining. **(2)** Selenium needs switchTo frame/defaultContent: stateful and fragile. **(3)** An error cannot strand you in the wrong frame in Playwright. **(4)** They usually ask next: real use case is PCI payment iframes (Stripe, PayPal, Adyen). **(5)** No frame bookkeeping: reliability that matters around money flows. **(6)** Where this goes wrong: naming frameLocator without the statefulness contrast.

CODE

```
// Payment iframe: scope once, interact normally
const payment = page.frameLocator('#stripe-payment-iframe');
await payment.getByLabel('Card Number').fill('4111111111111111');
await payment.getByLabel('Expiry date').fill('12/28');
await payment.getByLabel('Security code').fill('123');
await payment.getByRole('button', { name: 'Pay Now' }).click();

// Nested iframes
const inner = page.frameLocator('#outer').frameLocator('#inner');
await inner.getByRole('button', { name: 'Submit' }).click();
```

Q 37 LEVEL · Mid

How does Playwright handle shadow DOM elements?

CONCEPT

Playwright **pierces shadow DOM automatically** when you use CSS or the recommended user-facing locators, no special syntax, no manual traversal. A normal locator reaches an input inside a custom element's shadow root as if the boundary were not there. This is a sharp contrast with Selenium, which requires JavaScript **shadowRoot** traversal for every interaction. When you genuinely need the raw shadow content, for an assertion against internal markup, you can drop into **evaluate** and read shadowRoot directly, but that is the exception. The everyday case is that locators just work. This is one of the most concrete advantages to cite for applications built on Web Components, such as anything using Lit, where nearly every interactive element lives inside a shadow root.

INTERVIEW STRATEGY

The question is checking real exposure to component-based UIs. State plainly that it just works with normal locators, then contrast with Selenium's per-interaction JavaScript traversal, the contrast is what shows you have felt the difference. A Web Components / Lit war story makes it memorable. The failure mode is overcomplicating an answer that is genuinely simple. The follow-up is sometimes ‘is there ever a case it does not handle?’, be ready with the evaluate-plus-shadowRoot escape for raw-markup assertions.

How to phrase it

> *With Playwright, shadow DOM mostly just works, and I would say that plainly rather than overcomplicate it. When I use a CSS selector or one of the user-facing locators, Playwright pierces the shadow boundary automatically, so reaching an input inside a custom element's shadow root needs no special syntax at all. That is a big contrast with Selenium, where every shadow-DOM interaction meant writing JavaScript to walk shadowRoot by hand. The war story I would give: I worked on a product built entirely on Lit web components, where essentially every interactive element sat inside a shadow root. In Selenium we had written JavaScript traversal helpers for every single interaction, and they were brittle and a constant maintenance cost. In Playwright a plain locator on the custom element's input simply worked. For the follow-up about whether there is ever a case it does not handle, the one time I drop to evaluate and read shadowRoot directly is when I need to assert against the internal markup of a component, but that is rare. For any app using Web Components this is one of the most concrete technical reasons I give when I recommend Playwright over Selenium, it turns a whole category of helper code into nothing.*

Key points to hit

(1) Playwright pierces shadow DOM automatically with normal locators. **(2)** No special syntax for everyday interactions; Selenium needs JS traversal. **(3)** War story: a Lit Web Components app where Selenium needed helpers everywhere. **(4)** The next question: evaluate + shadowRoot only for raw-markup assertions. **(5)** Major reason to pick Playwright for component-based UIs. **(6)** What loses the point: overcomplicating an answer that is genuinely simple.

CODE

```
// Shadow DOM is pierced transparently
await page.locator('my-custom-component input[type="text"]').fill('value');
await expect(page.locator('custom-button #inner-btn')).toBeVisible();

// Only when you need the raw shadow content
const html = await page.locator('my-component').evaluate(
el => el.shadowRoot?.innerHTML ?? '',
);
```

Q 38 LEVEL · Junior to Mid

How do you handle dynamic elements with changing attributes or IDs?

CONCEPT

Dynamic elements change their attributes, IDs, classes, positions, on each render, so any locator tied to those values breaks on the next run. The fix is to target **stable characteristics**: prefer **semantic locators** (role, text) that follow what the user sees, and where you must use attributes, use **partial matchers** on the stable portion, a prefix match on an id stem, or a substring match on a class. The firm rule: never anchor on an id that embeds a timestamp, a random number, or anything that changes between loads. The recommended locator priority exists precisely to keep you off these volatile hooks. When auditing a framework, locators with numbers baked into them are the first thing to refactor.

INTERVIEW STRATEGY

This question is really about flaky-suite hygiene. Give the rule as a rule, never anchor on a value that changes between loads, then show the semantic and partial-match alternatives. Framing it as the first thing you refactor in an audit signals you have cleaned up real suites. The failure mode is suggesting waits or retries to paper over a bad locator. A common follow-up is 'what do you do when there is no stable attribute at all?', answer with content-based filtering on the row or item the user would recognise.

How to phrase it

> *Dynamic elements change their IDs, classes or positions on every render, so a locator pinned to those values breaks the next run, and my rule is blunt: never use an id that contains a timestamp, a random number, or any value that changes between page loads. Instead I target stable characteristics. The first choice is a semantic locator, getByRole or getByText, because those follow what the user sees rather than internal DOM identifiers, so they are immune to id churn. When I genuinely must use an attribute, I use a partial match on the stable part, a prefix match on the fixed stem of an id, or a substring match on a class, usually combined with a hasText filter to disambiguate. That is also my answer to the follow-up about what to do when there is no stable attribute at all: I filter by the content the user would recognise, like the row containing a specific email, and act relative to that. What I will not do is paper over a bad locator with waits or retries; that hides the real problem. Whenever I audit a framework and see locators with numbers baked into them, that is the very first thing I refactor, because every one of them is a flaky failure waiting to happen on the next deploy.*

Key points to hit

(1) Rule: never anchor on a value that changes between loads. **(2)** Prefer semantic locators (role, text) immune to id churn. **(3)** If you must use attributes, partial-match the stable portion + hasText. **(4)** Asked next: no stable attribute? Filter by recognisable content. **(5)** Locators with embedded numbers are the first audit refactor. **(6)** The common miss: papering over a bad locator with waits or retries.

CODE

```
// Breaks every run: do not do this
// await page.locator('#user-card-1679234857233').click();

// Partial attribute match on the stable stem
await page.locator('[id^="user-card-"]').first().click();

// Semantic: immune to id changes
await page.getByRole('listitem').filter({ hasText: 'Alice Johnson' }).click();

// Dynamic table row: filter by content
const row = page.locator('tbody tr').filter({ hasText: 'john@test.com' });
await row.getByRole('button', { name: 'Edit' }).click();
```

Q 39 LEVEL · Mid

How do you handle popups, browser dialogs, and new windows?

CONCEPT

Native browser dialogs, alert, confirm, prompt, need a listener registered **before** the action that triggers them. Use **page.once('dialog', ...)** rather than page.on, because once removes the listener after the first use and stops it bleeding into later tests in the same context. New windows and tabs use the popup event in a **Promise.all** pattern, start waiting for the popup before the click, because if the window opens before the listener is attached you miss it entirely. Downloads follow the same shape with the download event. The unifying principle across all three is: listen first, then trigger. Getting the order wrong is the classic source of intermittent failures here.

INTERVIEW STRATEGY

What is being probed is whether you understand event ordering, which is the root of flakiness here. Make the unifying rule explicit, listen before you trigger, and call out once versus on for dialogs, because that test-pollution detail is exactly what a senior interviewer fishes for. The risk is registering the listener after the click and then wondering why it is flaky. The follow-up is often 'why once and not on?', answer that once detaches after the first dialog so it cannot swallow a later test's dialog in the same context.

How to phrase it

> *The single principle for all of these is: attach the listener before you trigger the thing, and I would state that up front because it is the root of the flakiness people hit here. For native dialogs, alert, confirm, prompt, I register a handler before the action that pops them, and I deliberately use page.once rather than page.on. That is also my answer to the follow-up about why once and not on: once removes the listener after the first dialog, so it cannot affect later tests in the same context, whereas using on for dialogs is a common source of test pollution where one test's handler quietly swallows another's dialog and you get a confusing cross-test failure. For new windows or tabs I use the Promise.all pattern with the popup event: I start waiting for the popup, then click, in one Promise.all, because if the window opens before the listener is attached I miss it completely. Downloads are the exact same shape with the download event, wait for download, then click. So across dialogs, popups and downloads it is one rule applied three ways, and getting the ordering wrong, registering the listener after the trigger, is the classic cause of intermittent failures in this whole area.*

Key points to hit

(1) Unifying rule: listen first, then trigger. **(2)** Register dialog handlers before the triggering action. **(3)** New windows/downloads: Promise.all with the popup/download event. **(4)** Follow-up: why once not on, once detaches, avoiding cross-test pollution. **(5)** Wrong ordering (listen after trigger) is the classic flakiness cause. **(6)** Avoid: attaching the listener after the click.

CODE

```
// Dialog: register BEFORE the trigger; once() avoids pollution
page.once('dialog', async dialog => {
expect(dialog.type()).toBe('confirm');
await dialog.accept();
});
await page.getByRole('button', { name: 'Delete Account' }).click();

// New tab: listen first
const [popup] = await Promise.all([
page.waitForEvent('popup'),
page.getByRole('link', { name: 'Open Report' }).click(),
]);
await expect(popup).toHaveURL(/report/);

// Download: same shape
const [download] = await Promise.all([
page.waitForEvent('download'),
page.getByRole('button', { name: 'Export CSV' }).click(),
]);
expect(await download.path()).toBeTruthy();
```

Q 40 LEVEL · Mid

What are the key options for getByRole, and how do they let you select precisely without ambiguity?

CONCEPT

getByRole's power isn't the role alone: it's the option object that narrows the match with accessibility semantics the way a screen reader would. The options that matter: **name** (the accessible name, from aria-label, label, or text content), accepting a string or RegExp; **level** 1-6 for headings (h1 through h6); checked: true/false for checkboxes and switches; pressed: true/false for toggle buttons (aria-pressed); expanded: true/false for collapsibles; exact: true forces name to match exactly rather than substring; **includeHidden**: true to include elements with aria-hidden. The combination lets you say getByRole('button', { name: /save/i, pressed: false }), "the unpressed Save button", a precise selector that's stable across redesigns because it's anchored on accessibility, not CSS structure. Knowing these options well is one of the fastest ways to demonstrate Playwright depth in interviews.

INTERVIEW STRATEGY

Your job here is to show precision via options, not just the role name. Walk through the high-leverage options: name (string or RegExp), level for headings, checked/pressed/expanded for stateful elements, exact for tight matching. The accessibility-stable framing is the senior signal because it lands the why. The pitfall is using getByRole alone and then chaining .filter() when an option would have worked. Expect the follow-up ‘when do you use exact: true?’, answer when a substring match would also catch unintended buttons, ‘Save’ also matches ‘Save Draft’.

How to phrase it

> *getByRole's power isn't the role alone, it's the option object that narrows the match with accessibility semantics the way a screen reader would. The options that matter, and where I would demonstrate depth in an interview, are these. Name is the accessible name, drawn from aria-label, label, or text content, and it accepts either a string for substring matching or a RegExp for case-insensitive or pattern-based matching, I use a RegExp by default for case insensitivity. Level one through six for headings, so I can target h1 through h6 specifically with role heading level one. Checked true or false for checkboxes and switches. Pressed true or false for toggle buttons that use aria-pressed. Expanded true or false for collapsibles like accordions. Exact true forces the name to match exactly rather than substring, and that's also my answer to the follow-up about when to use exact. It's when a substring match would catch unintended elements, if I write getByRole button name Save without exact and the page also has a Save Draft button, my locator matches both and I get a strict-mode violation. With exact true, only the literal Save matches. IncludeHidden true to include elements with aria-hidden, which you sometimes need for dialog content before it animates in. The combination lets me write getByRole button with name slash save slash i and pressed false, the unpressed Save button, a precise selector that's stable across redesigns because it's anchored on accessibility, not CSS. The trap is reaching for filter chains when an option would have worked; the options are the first place to look.*

Key points to hit

(1) Options narrow getByRole via accessibility semantics, not CSS. **(2)** name (string or RegExp), level (1-6 for headings), checked/pressed/expanded. **(3)** exact: true forces literal match; default is substring. **(4)** Combining options builds precise stable selectors (e.g. unpressed Save button). **(5)** Expect next: exact: true when substring would match unintended elements. **(6)** The easy mistake: filter chains when an option would have done the same job.

CODE

```
// Common patterns
await page.getByRole('heading', { level: 1, name: 'Dashboard' }).click();
await page.getByRole('checkbox', { name: 'Remember me', checked: false }).check();
await page.getByRole('button', { name: 'Submit', pressed: false }).click();
await page.getByRole('button', { name: /save/i }).click();

// `exact` prevents 'Save' from also matching 'Save Draft'
await page.getByRole('button', { name: 'Save', exact: true }).click();

// Combined: the unpressed Save toggle
await page.getByRole('button', { name: /save/i, pressed: false }).click();
```

Q 41 LEVEL · Mid

What are the four filter() options, and when do you use each?

CONCEPT

filter() narrows a locator's matches in place, the locator starts as a set of candidates and filter keeps the ones matching the condition. Four options worth knowing: **hasText** keeps elements

whose text content contains the given string or matches the RegExp, the most common filter, used to pick the right row in a table; **hasNotText** excludes elements with matching text, the inverse, used to skip a known row; has keeps elements that contain a child matching the given locator, used to pick the row containing a specific button or link; hasNot excludes elements that contain the given child, used to pick the row that doesn't have a checkbox checked, for example. The combination of these lets you write expressive table-row pickers like rows.filter({ hasText: 'Acme Corp' }).filter({ has: page.getByRole('button', { name: 'Delete' }) }), the Acme row that has a delete button, without writing CSS selectors at all.

INTERVIEW STRATEGY

Show expressive locator composition. Walk through the four filter options with the table-row picker as the worked example, it's the case most commonly asked. The has and hasNot options are the depth signal because most candidates only know hasText. The mistake is reaching for CSS selectors when filter chains would compose cleanly. They usually follow with 'when do you use has versus hasText?', answer hasText when filtering on text content, has when filtering on the presence of a structural element.

How to phrase it

> *filter narrows a locator's matches in place: the locator starts as a set of candidates and filter keeps the ones matching the condition. There are four options I would walk through. hasText keeps elements whose text content contains the given string or matches the RegExp, the most common one, used to pick the right row in a table. hasNotText is the inverse, excluding elements with matching text, which I use to skip a known header row when iterating. has keeps elements that contain a child matching the given locator, so I can say the row that has a delete button, expressed as a locator condition not a CSS query. hasNot excludes elements containing the given child, used for example to pick the row without a checked checkbox. That's also my answer to the follow-up about when to use has versus hasText. hasText is when I'm filtering on text content; has is when I'm filtering on the presence of a structural element like a button or an icon, a different semantic. The reason I would emphasise the four together, because most candidates only know hasText, is that they compose. I can write rows dot filter hasText Acme Corp dot filter has the delete button locator, the Acme row that has a delete button, without writing a single CSS selector. That's expressive, stable across CSS changes, and reads like the test scenario rather than the HTML structure. The trap is reaching for CSS selectors when filter chains would compose cleanly, a CSS selector locks the test to the markup; a filter chain locks it to the behaviour.*

Key points to hit

(1) filter narrows in place: hasText, hasNotText, has, hasNot. **(2)** hasText: filter on text content (most common). **(3)** has: filter on presence of a child locator (depth signal). **(4)** Compose: rows.filter({ hasText }).filter({ has }) for table-row pickers. **(5)** They usually ask next: hasText for content; has for structural presence. **(6)** Trap: CSS selectors when filter chains compose more stably.

CODE

```
const rows = page.getByRole('row');

// Pick the Acme Corp row that contains a delete button
const acmeWithDelete = rows
.filter({ hasText: 'Acme Corp' })
.filter({ has: page.getByRole('button', { name: 'Delete' }) });
await acmeWithDelete.getByRole('button', { name: 'Delete' }).click();

// Pick rows without a checked checkbox
const unchecked = rows.filter({ hasNot: page.getByRole('checkbox', { checked: true })
});

// Skip the header row by excluding its text
const dataRows = rows.filter({ hasNotText: 'Customer Name' });
```

Q 42 LEVEL · Mid to Senior

What do locator.and() and locator.or() do, and when are they the right tool?

CONCEPT

locator.and() returns a locator that matches elements matching both locators (intersection); **locator.or()** returns a locator that matches elements matching either (union). **or()** arrived in Playwright 1.33 and and() in 1.34, and they solve compositional locator problems that previously required evaluate or workarounds. and() is useful when two independent locator concepts must both hold, a button that's also visible (button.and(page.locator(':visible'))), a heading that's also the current page in a sidebar. or() is the right tool for waiting on one of multiple possible outcomes, after submitting a form, await the success toast or the error alert (toast.or(error)); after a flaky load, await the data table or the empty state. The or() pattern replaces fragile if-then-else logic and Promise.race workarounds with a single wait. The principle: compose locators where you previously composed control flow.

INTERVIEW STRATEGY

They are looking to see 2024+ Playwright API knowledge. Lead with and (intersection) and or (union), then make or the centerpiece because waiting on ‘toast or error’ is a common interview gotcha. The 'compose locators where you previously composed control flow' framing signals depth here. Where this goes wrong is Promise.race or if-then-else awaits where or() would be one line. The next question is normally ‘what's a real use case for or?’, answer await on toast.or(errorAlert) after a form submit, deterministic, no flake.

How to phrase it

> *locator dot and returns a locator that matches elements matching both input locators, an intersection. locator dot or returns a locator that matches elements matching either, a union. or arrived in Playwright one point three three and and in one point three four, and they solve compositional locator problems that previously required page.evaluate or Promise.race workarounds. and is useful when two independent locator concepts must both hold. The example I would give is a button that's also visible, button dot and a visibility locator, or a heading that's also marked current page in a sidebar. or is the more interesting one for testing, and this is my answer to the follow-up about a real use case. After a form submit there are typically two possible outcomes: the success toast appears or the error alert appears. Before or, the only way to wait deterministically was Promise.race or branching control flow, both of which are clumsy and prone to subtle bugs. With or, I write toast dot or error alert dot waitFor, and the locator resolves as soon as either appears, deterministic, no flake, one line. Same pattern for await on the data table or the empty state after a slow query, whichever arrives first satisfies the wait. The framing I would lead with, because it lands as the senior insight, is to compose locators where I previously composed control flow. The composition stays at the locator layer rather than leaking into branching code, which keeps tests readable. The trap is Promise.race awaiting on selectors or if page dot is visible then else branching, both work, both are now obsolete since or exists.*

Key points to hit

(1) and() = intersection of two locators (both must match). **(2)** or() = union of two locators (either matches). **(3)** Added Playwright 1.34+; replaces Promise.race and if-then-else awaits. **(4)** or() classic use: await success-toast OR error-alert after form submit. **(5)** The next question: or() for outcomes-could-go-either-way, one line, no flake. **(6)** Where this goes wrong: Promise.race or if/else branching where or() composes cleanly.

CODE

```
// or(), await on either outcome after form submit
await page.getByRole('button', { name: 'Save' }).click();
const toast = page.locator('.toast.success');
const error = page.locator('.alert.error');
await toast.or(error).waitFor(); // resolves on whichever appears
if (await toast.isVisible()) { /* success path */ }
else { /* error path */ }

// and(): the visible Save button (when invisible duplicates exist)
const visibleSave = page.getByRole('button', { name: 'Save' })
.and(page.locator(':visible'));
await visibleSave.click();
```

Q 43 LEVEL · Mid to Senior

What is the order-of-evaluation trap with first(), last(), nth(), and filter()?

CONCEPT

The trap: filter() applies to the set, first()/last()/nth() picks one from the set, the order matters. **rows.filter({ hasText: 'Acme' }).first()** filters the rows to just those containing Acme, then picks the first matching row, "the first Acme row". **rows.first().filter({ hasText: 'Acme' })** picks the very first row in the table, then tests whether that row contains Acme, "the first row, if it contains Acme". The first yields an Acme row; the second yields either an empty match (if row 1 is the header) or row 1 of the data, and the test fails or selects the wrong row depending on the table. The principle: **narrow the set first, then pick**. nth() has the same trap: nth(2) before filter picks the third element regardless of content; nth(2) after filter picks the third element matching the filter. Reach for filter().first() by default; first().filter() is almost never what you want.

INTERVIEW STRATEGY

The point of this one is to see the order-of-evaluation intuition. Lead with the contrast: filter-first-then-pick versus pick-first-then-filter, same locators, different results. The 'narrow the set first, then pick' rule demonstrates calibration. The failure mode is the exact reversal that produces silent wrong selections. A common follow-up is ‘how do you debug it when the wrong element is picked?’, answer trace viewer + highlight() show the element ranges; reading the locator chain right to left helps too.

How to phrase it

> *The trap, and the one that catches even experienced candidates in interviews, is that filter applies to the set while first, last, and nth pick one from the set, so the order matters and the same components in different order produce different results. The worked example I would give: rows dot filter hasText Acme dot first filters the rows to just those containing Acme, then picks the first matching row, which reads as the first Acme row. Rows dot first dot filter hasText Acme picks the very first row in the table, then tests whether that row contains Acme, which reads as the first row, only if it contains Acme. The first one yields an Acme row. The second yields either an empty match if row one is the header, or row one of the data, and the test silently fails or selects the wrong row depending on the table. Same for nth: nth two before filter picks the third element regardless of content; nth two after filter picks the third element matching the filter. The principle I would lead with, because it's the rule that prevents the trap, is narrow the set first, then pick. Reach for filter dot first by default; first dot filter is almost never what you want, but it compiles and runs, so the bug is silent until it picks the wrong element. If they push on this in the follow-up, my answer is about debugging when the wrong element is picked. Trace viewer and highlight on the locator show the ranges, and reading the locator chain right to left, from the final action backward, reveals what set was actually narrowed. The trap costs hours when it hits production, because the test passes against the wrong element.*

Key points to hit

(1) filter() narrows the set; first/last/nth picks one, order matters. **(2)** rows.filter(hasText Acme).first() = the first Acme row. **(3)** rows.first().filter(hasText Acme) = row 1, only if it has Acme. **(4)** Rule: narrow the set first, then pick, filter().first() is the default. **(5)** Asked next: highlight() + reading right-to-left helps debug. **(6)** What loses the point: first().filter() silently picks wrong element; compiles fine.

CODE

```
const rows = page.getByRole('row');

// CORRECT: narrow first, then pick
const firstAcme = rows.filter({ hasText: 'Acme' }).first();
await firstAcme.getByRole('button', { name: 'Edit' }).click();

// WRONG: silent bug. Picks row 1; fails if row 1 doesn't have Acme.
const wrong = rows.first().filter({ hasText: 'Acme' });

// Same trap with nth
const thirdAcme = rows.filter({ hasText: 'Acme' }).nth(2); // ✓ the 3rd Acme row
const wrongNth = rows.nth(2).filter({ hasText: 'Acme' }); // ✗ row 3, only if Acme
```

Q 44 LEVEL · Mid to Senior

How do you handle nested iframes with frameLocator chains?

CONCEPT

Nested iframes require **chained frameLocator calls**, one frameLocator per iframe layer. **page.frameLocator('#outer').frameLocator('#inner').getByRole('button')** drills through two iframe layers and selects within the innermost. Each frameLocator returns a FrameLocator object whose getBy methods operate inside that frame, and chaining them mirrors the DOM structure. The common production case: a payment iframe (Stripe) inside an OAuth iframe (an identity provider) inside the main page. The discipline that matters: locate the iframe by its accessibility name or stable attribute, not by its CSS class or its index in the DOM, iframes often have auto-generated IDs that change every load. Use frame title or a data-testid on the iframe element itself. Inside the frame, the standard getByRole/getByLabel priority applies as if you'd narrowed scope. Auto-waiting works inside the frame exactly as on the page, you don't need to wait for the iframe to load before using frameLocator.

INTERVIEW STRATEGY

You are being asked to show frame chaining and stable iframe selection. Lead with chained frameLocator calls, one per layer, then make iframe selection by accessibility name or data-testid the senior detail, auto-generated IDs are the iframe equivalent of CSS class selectors. Auto-waiting inside the frame is the practical reassurance. The classic error is selecting iframes by index or auto-generated ID. The follow-up is often ‘how do you wait for an iframe to load?’, answer you don't, auto-waiting on the frame's locators handles it.

How to phrase it

> *Nested iframes require chained frameLocator calls, one frameLocator per iframe layer. So page dot frameLocator hash outer dot frameLocator hash inner dot getByRole button drills through two iframe layers and selects within the innermost. Each frameLocator returns a FrameLocator object whose getBy methods operate inside that frame, and chaining them mirrors the DOM structure. The common production case I would describe is a payment iframe like Stripe inside an OAuth iframe from an identity provider inside the main page, three frame layers, three frameLocator calls. The discipline that matters, and the senior detail I would emphasise, is locating the iframe by its accessibility name or a stable attribute like data-testid, not by its CSS class or its index in the DOM. Iframes often have auto-generated IDs that change every load, especially in modern frameworks, so an iframe selector by hash iframe dash one two three four is the iframe equivalent of a brittle CSS class selector, it works in dev and breaks in production. I prefer the frame's title attribute or a data-testid added specifically for testing. Inside the frame the standard getByRole and getByLabel priority applies as if I'd narrowed scope to that frame, so the techniques I use for normal locators transfer directly. If they push on this in the follow-up, my answer is about how to wait for an iframe to load: I don't explicitly wait, auto-waiting works inside the frame exactly as on the page. When I call dot click on a locator inside a frameLocator chain, Playwright waits for the iframe to load and the element to be actionable, all in one. The pitfall is selecting iframes by index, like frameLocator open paren iframe close paren, which works until the page adds or removes another iframe and your test silently targets the wrong one.*

Key points to hit

(1) One frameLocator per iframe layer: chain them through nested iframes. **(2)** frameLocator returns FrameLocator; getBy methods work inside the frame. **(3)** Select iframes by title or data-testid, NOT auto-generated IDs. **(4)** Inside the frame, getByRole priority applies as normal. **(5)** Follow-up: don't explicitly wait, auto-waiting works inside the frame. **(6)** The common miss: selecting iframes by index, breaks when iframe order changes.

CODE

```
// Nested iframes: payment iframe inside OAuth iframe inside main page
const paymentInOauth = page
.frameLocator('iframe[title="Sign in with Acme ID"]') // outer (stable title)
.frameLocator('iframe[title="Card payment"]'); // inner (stable title)

await paymentInOauth.getByLabel('Card number').fill('4111 1111 1111 1111');
await paymentInOauth.getByLabel('Expiry').fill('12 / 30');
await paymentInOauth.getByLabel('CVC').fill('123');
// Auto-waiting works inside the frame: no explicit iframe wait needed

// AVOID: brittle, breaks when iframe order changes
// page.frameLocator('iframe').nth(1).frameLocator('iframe').first()
```

Q 45 LEVEL · Senior

How does Playwright handle open vs closed Shadow DOM, and what are the limits?

CONCEPT

Playwright **pierces open Shadow DOM automatically**, getByRole, getByLabel, getByText, and locator() all see through open shadow roots as if they were normal DOM. This is the common case (most design systems and web components attach shadow with mode: 'open'). Closed Shadow DOM is deliberately inaccessible from JavaScript, including Playwright, it's a privacy mechanism, not a security one, but the consequence is the same: the host element exists in the DOM, but its shadow root is not reachable. The escape hatch when you must test inside closed shadow: page.evaluate with reflection on the host's shadowRoot internal slot, which works only because evaluate runs in the page context where the closed shadow was created. For interviews, the right answer covers all three: most shadow DOM is open and Playwright handles it transparently; **closed shadow is intentionally inaccessible**; page.evaluate is the escape hatch but should be a last resort because it's fragile and bypasses Playwright's safety net.

INTERVIEW STRATEGY

What scores here is showing you know both the common case (transparent piercing) and the edge case (closed shadow). Lead with open as the default and getByRole working out of the box, then closed as the intentional limit, then page.evaluate as the last-resort escape. The 'fragile, bypasses Playwright's safety net' framing on evaluate is the seniority marker. The risk is reaching for evaluate-based hacks before checking if the shadow is actually open. Expect the follow-up ‘how do you know if a shadow root is closed?’, answer page.evaluate to inspect element.shadowRoot, null on closed, the root on open.

How to phrase it

> *Playwright pierces open Shadow DOM automatically. getByRole, getByLabel, getByText, and locator all see through open shadow roots as if they were normal DOM, and that's the common case I would lead with, most design systems and web components attach shadow with mode open, so the average developer never notices they're crossing a shadow boundary. The framing that lands in interviews is: in 2026, you do not need a special selector syntax for open shadow DOM; standard Playwright locators just work. Closed Shadow DOM is the edge case. It's deliberately inaccessible from JavaScript, including Playwright, because closed mode is the host's way of saying these internals are not yours. It's a privacy mechanism, not a security one, with enough work you can still poke at it, but the consequence is real: the host element exists in the DOM, but its shadow root is null when you try to reach it. The escape hatch, when I genuinely must test inside a closed shadow root, is page.evaluate running in the page context where the closed shadow was created, sometimes reading through reflection or by intercepting the attachShadow call before the component runs. But I treat that as a last resort because it's fragile and bypasses Playwright's actionability checks, auto-waiting, and trace viewer, which means I lose the safety net that makes Playwright reliable. That's also my answer to the follow-up about how I know if a shadow root is closed: page.evaluate to inspect element dot shadowRoot, null on closed, the root object on open. The mistake is reaching for evaluate-based hacks before checking whether the shadow is actually open, because nine times out of ten it is, and a normal locator works.*

Key points to hit

(1) Open Shadow DOM: Playwright pierces transparently, most common case. **(2)** getByRole, getByLabel, locator() all see through open shadow roots. **(3)** Closed Shadow DOM: intentionally

inaccessible, host.shadowRoot is null. **(4)** Last resort: page.evaluate, fragile, bypasses actionability checks. **(5)** Expect next: page.evaluate(el => el.shadowRoot) returns null if closed. **(6)** Avoid: reaching for evaluate hacks before checking if shadow is open.

CODE

```
// Open shadow DOM: Playwright pierces automatically. No special syntax.
await page.getByRole('button', { name: 'Add to Cart' }).click();
// Works even if Add-to-Cart is inside <my-product-card>'s open shadow root

// Check whether a shadow root is open or closed
const isClosed = await page.locator('my-secret-widget').evaluate(
el => el.shadowRoot === null // null -> closed, object -> open
);

// Last-resort access into closed shadow: fragile, bypasses auto-waiting
await page.locator('my-secret-widget').evaluate((host) => {
// Only works if the component leaked a reference, or via reflection tricks.
// Lose Playwright's safety net. Avoid unless absolutely necessary.
});
```

Q 46 LEVEL · Mid

How do you configure a custom test-ID attribute (data-test, data-qa) instead of the default data-testid?

CONCEPT

Set **testIdAttribute** in playwright.config.ts under the use block: **use: { testIdAttribute: 'data-test' }**. From that point, getByTestId('submit') matches [data-test="submit"] rather than the default [data-testid="submit"]. You can also override per-test with **test.use({ testIdAttribute: 'data-qa' })** in a describe block. The reason this matters: teams have different established conventions, React Testing Library popularised data-testid, but many teams use data-test, data-qa, data-cy (Cypress holdover), or a company-specific prefix like data-app-test. Forcing tests onto a different attribute name than the codebase already uses is friction that gets removed by one config line. The discipline: standardise on one attribute name across the framework and document it in CONTRIBUTING.md, because mixing attribute names inside one suite is the kind of confusion that compounds over time.

INTERVIEW STRATEGY

This question is there to see configuration awareness, not just default usage. Lead with the testIdAttribute config under the use block, then mention test.use for override. The 'teams have different conventions' context shows you've worked across codebases. The trap is keeping the default when the codebase uses something else and forcing a mismatch. They usually follow with ‘what if different parts of the app use different attributes?’, answer standardise on one across the framework, even if it means adding the new attribute to legacy components.

How to phrase it

> *I set testIdAttribute in playwright.config.ts under the use block. So use colon open brace testIdAttribute colon quote data-test quote close brace, and from that point getByTestId of submit matches open bracket data dash test equals quote submit quote close bracket rather than the default data dash testid. I can also override per-test with test.use testIdAttribute data dash qa inside a describe block if I have an isolated suite that needs to target a different attribute, but I would do that rarely. The reason this matters, and the framing that lands with interviewers because it shows operational awareness, is that teams have different established conventions for test-ID attributes. React Testing Library popularised data-testid, but many teams use data-test because it's shorter, or data-qa as a deliberate namespace, or data-cy as a Cypress holdover from a previous framework, or a company-specific prefix like data-app-test. Forcing my tests onto a different attribute name than the codebase already uses is friction that gets removed by one config line. The discipline I would emphasise is to standardise on one attribute name across the framework and document it in CONTRIBUTING.md. Mixing attribute names inside one suite, some tests on data-testid, others on data-test, is the kind of confusion that compounds over time, because developers see inconsistent usage and start adding new attributes to whichever they happen to see first. That's also my answer to the follow-up about different parts of the app using different attributes: I standardise across the framework even if it means adding the chosen attribute to legacy components, because consistency in the tests is worth more than tolerating two conventions. The thing to avoid is keeping the default when the codebase uses something else and forcing a developer-experience mismatch that everyone works around with raw CSS selectors.*

Key points to hit

(1) use: { testIdAttribute: 'data-test' } in playwright.config.ts. **(2)** test.use({ testIdAttribute: 'data-qa' }) per describe-block override. **(3)** Match the codebase's convention: data-testid, data-test, data-qa, data-cy, … **(4)** Standardise on ONE attribute across the framework, documented. **(5)** They usually ask next: even add the chosen attribute to legacy components for consistency. **(6)** The easy mistake: default data-testid when codebase uses data-test, forced mismatch.

CODE

```
// playwright.config.ts: match your codebase's convention
export default defineConfig({
use: {
baseURL: process.env.BASE_URL,
testIdAttribute: 'data-test', // -> getByTestId targets [data-test=...]
},
});

// Now getByTestId resolves to your codebase's attribute
await page.getByTestId('submit-button').click(); // -> [data-test="submit-button"]

// Per-describe override when truly needed
test.describe('Legacy area on data-qa', () => {
test.use({ testIdAttribute: 'data-qa' });
test('legacy flow', async ({ page }) => { /* ... */ });
});
```

Q 47

LEVEL · Mid to Senior

What is locator strict mode, and what does it mean when an action throws on multiple matches?

CONCEPT

Locator actions and assertions are strict by default, if the locator matches more than one element when a click, fill, or assertion runs, Playwright throws a **strictModeViolation** error rather than silently picking one. This is a deliberate safety mechanism: ambiguous locators are the leading cause of silently-wrong tests (clicking the wrong Submit button in a page with multiple forms). The violation error names the count and provides the matched elements' descriptors so you can narrow the locator. The right fix is **narrow the locator** until it matches exactly one: add a name option to getByRole, add filter with hasText, scope to the right container via .getByRole('region', { name: … }).getBy…, or chain a .first()/.nth()/.last() when you intentionally want one of many. The wrong fix is **chaining .first() to silence violations** reflexively, that makes the test pass against an arbitrary element, often the wrong one, and the bug ships.

INTERVIEW STRATEGY

They want to see strict mode as safety, not annoyance. Lead with what strict mode does and why it exists, ambiguous locators are the leading cause of silently-wrong tests. Then make the fix landscape explicit: narrow the locator first, only first/nth/last if you genuinely want one of many. The 'reflexive .first() to silence' trap is what separates junior from senior thinking because it's the bug that ships. The next question is normally ‘when is .first() actually appropriate?’, answer when you deliberately want one of many, e.g. the first row in a table for a smoke check.

How to phrase it

> *Locator actions and assertions are strict by default. If the locator matches more than one element when a click, fill, or assertion runs, Playwright throws a strictModeViolation error rather than silently picking one. The reason that exists, and the framing I would lead with because it's the safety case, is that ambiguous locators are the leading cause of silently-wrong tests, clicking the wrong Submit button in a page that has multiple forms, asserting on the wrong row in a table. Strict mode makes that failure mode loud rather than silent, which is exactly what you want from a test framework. The violation error helpfully names the count and provides descriptors of the matched elements so I can see what to narrow on. The right fix is to narrow the locator until it matches exactly one. Add a name option to getByRole. Add a filter with hasText. Scope to the right container by chaining getByRole region with name then getBy inside it. Any of these is preferable to picking an arbitrary match. The wrong fix, and the trap I would call out explicitly because it's where tests silently lie, is chaining first reflexively to silence violations. That makes the test pass against an arbitrary element, often the wrong one, and the bug ships because the test reports green. That's also my answer to the follow-up about when first is actually appropriate. It's when I deliberately want one of many, the first row in a table for a smoke check that the table has data at all, or the first tab in a tablist to verify it's selected by default, cases where any element satisfying the selector is genuinely fine. If I'd care which specific element is picked, first is wrong, and the right tool is narrowing.*

Key points to hit

(1) Strict mode: actions/assertions throw on >1 match, not silent pick. **(2)** Why: ambiguous locators silently click wrong elements; strict catches it. **(3)** Right fix: narrow with name option / filter / parent

scope. **(4)** first()/nth()/last() ONLY when you genuinely want one of many. **(5)** The next question: first() appropriate for 'any row exists' smoke checks. **(6)** Trap: chaining .first() reflexively to silence, the bug ships.

CODE

```
// Two Submit buttons on the page: ambiguous locator
await page.getByRole('button', { name: 'Submit' }).click();
// -> Error: strict mode violation. Resolved to 2 elements:
// 1) <button>Submit</button> in the login form
// 2) <button>Submit</button> in the feedback form

// CORRECT: narrow by container
await page.getByRole('form', { name: 'Login' })
.getByRole('button', { name: 'Submit' })
.click();

// WRONG: silences the violation, picks an arbitrary form
// await page.getByRole('button', { name: 'Submit' }).first().click();
```

Q 48 LEVEL · Senior

How does Playwright resolve the accessible name for getByLabel and getByRole, and why does it matter?

CONCEPT

Accessible name resolution follows the W3C ARIA specification's name computation algorithm, which checks sources in a strict priority order, the same order a screen reader uses. **1. aria-labelledby** (highest priority): a space-separated list of IDs whose elements' text is concatenated; **2. aria-label**: an explicit string attribute; 3. label element: a <label> associated via for/id or wrapping the control; **4. placeholder** for form controls without other labels; 5. title attribute; 6. text content for elements like buttons. Understanding this priority explains surprising behaviour: a button with both visible text "Save" and aria-label="Submit form" matches getByRole('button', { name: 'Submit form' }), not 'Save', because aria-label wins. The implication for tests: when getByLabel finds the wrong control or finds nothing, check what the developer attached as aria-label or aria-labelledby, the visible text is often not the accessible name.

INTERVIEW STRATEGY

Your job here is to show ARIA-level understanding. Walk through the six-step priority, aria-labelledby, aria-label, label, placeholder, title, text content, because the order is the explanation for surprising behaviour. The visible-text-vs-accessible-name distinction demonstrates calibration. The pitfall is assuming the visible text is what getByRole's name matches against. A common follow-up is ‘why does getByLabel return nothing when I can see the label?’, answer the visible label may not be programmatically associated; check aria-label and the for-id pairing.

How to phrase it

> *Accessible name resolution follows the W3C ARIA specification's name computation algorithm, and it checks sources in a strict priority order, the same order a screen reader uses. The order I would walk through, because the order is the entire explanation for surprising behaviour, is six steps. First, aria-labelledby, the highest priority: a space-separated list of element IDs whose text content is concatenated to form the name. Second, aria-label, an explicit string attribute. Third, the label element associated via for-id or by wrapping the control. Fourth, placeholder for form controls that have no other label, which is sometimes controversial because placeholder is a poor accessibility pattern, but it is part of the algorithm. Fifth, the title attribute. Sixth, text content, which is what applies to most buttons. The implication for tests, and the senior signal in interviews, is that the visible text is often not the accessible name. A button with visible text Save and an aria-label of Submit form matches getByRole button with name Submit form, not Save, because aria-label has higher priority. If they push on this in the follow-up, my answer is about why getByLabel returns nothing when I can clearly see the label. The visible label may not be programmatically associated with the control, missing for-id pairing, missing aria-labelledby, missing wrapping label element. Visually it looks labelled to a sighted developer; programmatically the control has no accessible name and getByLabel can't find it. The fix is in the application, not the test: add the association. This is one of the places where the test reveals an accessibility bug the app had all along, which is a useful side effect of role-based locators. The thing to avoid is assuming the visible text is what matches against; always check the aria-label and the for-id pairing when locators behave unexpectedly.*

Key points to hit

(1) ARIA name computation: 6-step priority order. **(2)** Order: aria-labelledby -> aria-label -> label -> placeholder -> title -> text. **(3)** aria-label OVERRIDES visible text: source of surprising getByRole names. **(4)** Locator failures often reveal real accessibility bugs in the app. **(5)** Asked next: missing for-id pairing means visible label ≠ accessible name. **(6)** Where this goes wrong: assuming visible text is what name option matches, it isn't always.

CODE

```
// HTML:
// <button aria-label="Submit form">Save</button>
//
// Visible text is 'Save' but accessible name is 'Submit form' (aria-label wins)

await page.getByRole('button', { name: 'Submit form' }).click(); // ✓ matches
// await page.getByRole('button', { name: 'Save' }).click(); // ✗ no match

// Inspect what Playwright sees as the accessible name
const name = await page.getByRole('button').first().evaluate(el => {
return el.getAttribute('aria-label') ?? el.textContent;
});

// Common bug: visible <label> but no for-id pairing, getByLabel fails
// <span>Email</span> <- looks like a label, isn't programmatically one
// <input type="email"> <- has no associated label
// Fix: add <label for="email">Email</label> + id="email" on the input.
```

Q 49 LEVEL · Mid

What's the difference between toBeVisible, toBeAttached, and toBeInViewport?

CONCEPT

Three distinct visibility states, each useful for a different assertion. **toBeAttached**: the element is present in the DOM, it exists in the document tree but may be display:none, visibility:hidden, opacity:0, or fully off-screen. The lowest bar of presence. **toBeVisible**: the element is in the DOM and has non-zero size and is not display:none or visibility:hidden, the standard "the user can see this" assertion, and the default for most assertions. **toBeInViewport**: the element is within the visible viewport region, not scrolled off-screen, useful for testing lazy-loading triggers, scroll-into-view behaviour, and sticky elements. Attached and visible do form a ladder, visible implies attached. In viewport is a separate axis, not a stricter rung: toBeInViewport asks the Intersection Observer whether the element's box intersects the viewport, and it does *not* require the element to be visible. An element with visibility:hidden sitting in the middle of the screen passes toBeInViewport and fails toBeVisible. So pick by the question you are asking, not by strictness. For toast notifications: toBeVisible (the user sees it). For a hidden form that exists but isn't shown: toBeAttached. For testing that the scroll-spy highlighted the right section as the user scrolled: toBeInViewport.

INTERVIEW STRATEGY

Show precise visibility semantics. Walk through all three states, note that attached and visible stack but in viewport is a different axis entirely, then anchor each to a real use case: toBeVisible for toasts, toBeAttached for hidden forms, toBeInViewport for scroll-spy. The thing to avoid is using toBeVisible indiscriminately when toBeAttached or toBeInViewport is the right question. The follow-up is often ‘which one for a tooltip that's invisible until hover?’, answer depends on whether you want to check the tooltip element exists pre-hover (toBeAttached) or appears on hover (toBeVisible).

How to phrase it

> *Three distinct visibility states, each answering a different question. toBeAttached: the element is present in the DOM. It exists in the document tree but may be display none, visibility hidden, opacity zero, or fully off-screen. The lowest bar of presence. toBeVisible: the element is in the DOM and has non-zero size and is not display none or visibility hidden, the standard 'the user can see this' assertion, and the default people reach for. toBeInViewport: the element is within the visible viewport region, not scrolled off-screen, useful for testing lazy-loading triggers, scroll-into-view behaviour, sticky elements. The framing I would lead with, and this is where a lot of people go wrong, is that attached and visible stack, visible implies attached, but in viewport is a separate axis rather than a stricter rung. toBeInViewport asks the Intersection Observer whether the box intersects the viewport, and it does not require the element to be visible at all. So an element with visibility hidden sitting right in the middle of the screen passes toBeInViewport and fails toBeVisible. Once that is clear, the skill is picking the right one for the question being asked. For toast notifications I use toBeVisible: the user sees the toast appear, which is the behaviour I'm testing. For a hidden form that exists in the DOM but isn't yet shown, perhaps because it's mounted but collapsed, I use toBeAttached, the test verifies the form was registered even though it's not visible. For testing that the scroll-spy highlighted the right section as the user scrolled down, I use toBeInViewport, the question is whether the section actually became visible in the viewport, not just whether it's in the DOM. That's also my answer to the follow-up about a tooltip that's invisible until hover. It depends on what I'm testing. If I want to verify the tooltip element exists pre-hover, a hidden div ready to be shown, toBeAttached is the right question. If I want to verify the tooltip appears on hover, that's toBeVisible after performing the hover. The trap is reflexively reaching for toBeVisible and either missing assertions you should make or making ones that fail for the wrong reason.*

Key points to hit

(1) attached < visible is a real ladder; in viewport is a separate axis, not a rung. **(2)** toBeAttached: in DOM, may be hidden/off-screen. **(3)** toBeVisible: in DOM AND non-zero size AND not display:none/visibility:hidden. **(4)** toBeInViewport: intersects the viewport (Intersection Observer), visibility not required. **(5)** So visibility:hidden mid-screen PASSES toBeInViewport and FAILS toBeVisible. **(6)** Follow-up: tooltip pre-hover -> toBeAttached; tooltip on hover -> toBeVisible. **(7)** What loses the point: toBeVisible reflexively when the question is attached or in-viewport.

CODE

```
// toBeAttached, lowest bar: in DOM, may be hidden
await expect(page.locator('.hidden-form')).toBeAttached();

// toBeVisible, user can see it (size + not display:none/visibility:hidden)
await expect(page.locator('.toast.success')).toBeVisible();

// toBeInViewport: within the scrolled-into-view region
await page.locator('section#pricing').scrollIntoViewIfNeeded();
await expect(page.locator('section#pricing')).toBeInViewport();

// Tooltip example: different assertions for different questions
await expect(page.locator('.tooltip')).toBeAttached(); // exists before hover
await page.locator('button.info').hover();
await expect(page.locator('.tooltip')).toBeVisible(); // appears on hover
```

Q 50 LEVEL · Senior

How do you write Playwright tests that double as accessibility checks, and where do dedicated tools (axe-core, Lighthouse a11y) earn their place?

CONCEPT

Playwright's role-based locators are accessibility tests in disguise: every **getByRole**, **getByLabel**, **getByAltText**, getByPlaceholder call exercises an accessibility-tree node, so a button without an accessible name, an input without a label, or an image without alt text fails the test naturally. That's the baseline coverage: free, automatic, and growing with every locator you add. The explicit layer: @axe-core/playwright runs the full axe-core ruleset against the page in a test, returning structured violations (color-contrast, aria-required-attr, heading-order, landmark-one-main) with severity. The pattern: in a dedicated tests/a11y project, run an axe scan on every key page after the user reaches that page through the app. Lighthouse a11y via lighthouse-ci gives a higher-level score and is useful as a budget gate, but it's noisier and slower than axe for per-test enforcement. The rule worth leading with: locators give you free a11y coverage; axe adds the explicit ruleset; Lighthouse audits trend over time. The pitfall is treating accessibility as a separate concern handled only in audits, the bug ships, then is discovered in compliance review months later. Wiring accessibility into the regular test suite turns it into a precondition for merging, not a quarterly audit task.

INTERVIEW STRATEGY

What earns the tick is accessibility calibrated as part of the testing strategy, not bolted on. Lead with locators-as-implicit-a11y-tests, the framing most candidates miss, then axe-core for the explicit layer, then Lighthouse for trending. The 'baseline + explicit + audit' tiering marks senior judgement. The mistake is treating a11y as a separate audit concern. Expect the follow-up ‘what about keyboard navigation?’, answer test the tab order explicitly via page.keyboard, asserting focus moves through interactive elements in the expected sequence.

How to phrase it

> *Playwright's role-based locators are accessibility tests in disguise, and this is the framing most candidates miss when asked about accessibility in interviews. Every getByRole, getByLabel, getByAltText, getByPlaceholder call exercises an accessibility-tree node, so a button without an accessible name, an input without a label, or an image without alt text fails the test naturally. That's the baseline coverage: free, automatic, and growing with every locator I add. If the test suite is well-structured around role-based locators, it's already an accessibility regression suite. The explicit layer is at-axe-core slash playwright. It runs the full axe-core ruleset against the page in a test, returning structured violations like color-contrast, aria-required-attr, heading-order, landmark-one-main, with severity ratings. The pattern is in a dedicated tests slash a11y project, run an axe scan on every key page after the user reaches that page through the app, logged in, with their data loaded, in the state real users see. Scanning the static HTML misses violations that only appear after dynamic content loads. Lighthouse a11y via lighthouse-ci gives a higher-level score and is useful as a budget gate, fail the build if the a11y score drops below ninety, but it's noisier and slower than axe for per-test enforcement, so I use it for trending and budgets, not for finding specific violations. The rule worth leading with is locators give you free a11y coverage; axe adds the explicit ruleset; Lighthouse audits trend over time. Three tiers, three purposes, all complementary. For the follow-up about keyboard navigation, my answer is test the tab order explicitly via page dot keyboard, asserting focus moves through interactive elements in the expected sequence. Focus-trap testing matters for modals; skip-link testing matters for navigation. The pitfall is treating accessibility as a separate concern handled only in audits, which means the bug ships, gets discovered in compliance review months later, and the fix is more expensive than catching it in the original PR. Wiring accessibility into the regular test suite turns it into a precondition for merging, not a quarterly audit task. The bonus benefit is that good a11y structure makes the test suite more stable too, role-based locators don't break on CSS refactors the way CSS selectors do.*

Key points to hit

(1) getByRole/getByLabel/getByAltText: every locator is an implicit a11y check. **(2)** @axe-core/playwright: explicit ruleset, structured violations with severity. **(3)** Lighthouse a11y via lighthouse-ci: high-level score, useful as budget gate. **(4)** Tier: baseline (locators) + explicit (axe per-page) + audit (Lighthouse trend). **(5)** Keyboard nav: test tab order via page.keyboard, focus-trap on modals. **(6)** Pitfall: a11y as separate quarterly audit, bugs ship, fixed expensively.

CODE

```
import AxeBuilder from '@axe-core/playwright';

test.describe('Checkout page accessibility', () => {
test('axe-core scan: zero serious or critical violations', async ({ page }) => {
await page.goto('/checkout');
await expect(page.getByRole('heading', { name: 'Checkout' })).toBeVisible();

const results = await new AxeBuilder({ page })
.withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
.analyze();

const severe = results.violations.filter(
v => v.impact === 'serious' || v.impact === 'critical');
expect(severe, JSON.stringify(severe, null, 2)).toEqual([]);
});

test('keyboard tab order traverses checkout form correctly', async ({ page }) => {
await page.goto('/checkout');
await page.keyboard.press('Tab'); // first focusable
await expect(page.getByLabel('Card number')).toBeFocused();
await page.keyboard.press('Tab');
await expect(page.getByLabel('Expiry')).toBeFocused();
await page.keyboard.press('Tab');
await expect(page.getByLabel('CVC')).toBeFocused();
});
});

// Lighthouse a11y as a budget gate (lighthouse-ci in CI)
// assert: { 'categories:accessibility': ['error', { minScore: 0.9 }] }
```

Q 51 LEVEL · Mid

setInputFiles vs the filechooser event: which do you use for file uploads, and when is the filechooser event the only option?

CONCEPT

Both drive a file upload, but they attack it from different ends. **locator.setInputFiles()** sets the files directly on a native <input type="file"> element, you give it a path or a buffer and Playwright fills the input without any OS dialog ever appearing. It is the default, the simplest, and what you reach for in the vast majority of cases. The filechooser event is a listener-first pattern: you call **page.waitForEvent('filechooser')** before the click that opens the picker, then call **fileChooser.setFiles()** on the resulting object. The case where the event is the only option is when there is no real file input to target, a custom upload widget that opens the OS picker via a button or a JavaScript click rather than exposing a reachable <input>. There you cannot call setInputFiles on anything, so you intercept the chooser the browser raises instead. The unifying point: setInputFiles when an input exists, filechooser when it does not.

INTERVIEW STRATEGY

The aim is to know you reach for the simple tool by default and only escalate when the DOM forces you to. Lead with setInputFiles as the everyday choice and the reason it is nicer, it never opens an OS dialog, so there is no native window to wrestle with. Then name the one situation that flips it: a

custom widget with no targetable input, where the filechooser event is the only door in. The trap is presenting them as interchangeable or always using the event pattern. The follow-up is usually 'why not just always listen for the filechooser?', answer that the listener-first pattern is more ceremony and a flakiness risk if you attach it after the click, so you only pay that cost when you have to.

How to phrase it

> *By default I use setInputFiles, and I would say that plainly. It sets the files straight onto a native file input, I hand it a path or a buffer and Playwright fills the input, and no OS file dialog ever opens, so there is no native window to fight. That covers the large majority of uploads. The filechooser event is the pattern I keep for the case where there is no real input to target. Some apps build a custom upload widget: a styled button that opens the OS picker through a JavaScript click, with the actual input either absent or hidden in a way I cannot reach. There, setInputFiles has nothing to act on, so I listen for the filechooser the browser raises instead, I start waiting for the filechooser event, click the button in the same Promise.all, and then call setFiles on the chooser object. That is the answer to when the event is the only option: when the DOM gives me no targetable input. For the follow-up about why I do not just always listen for the chooser, it is because the listener-first pattern is more ceremony and a flakiness risk, if I attach the listener after the click, the chooser has already fired and I miss it. So I only take on that ordering discipline when the simpler setInputFiles cannot do the job. Reach for the input directly first; fall back to the event only when there is no input to reach.*

Key points to hit

(1) setInputFiles sets files directly on a native input, no OS dialog opens. **(2)** It is the default and simplest choice for the large majority of uploads. **(3)** filechooser event is listen-first: waitForEvent then fileChooser.setFiles(). **(4)** Only option when a custom widget exposes no targetable file input. **(5)** Expect next: don't always use the event, it's more ceremony and a flake risk. **(6)** The common miss: presenting the two as interchangeable.

CODE

```
// Default: set files straight on a native input, no OS dialog
await page.getByLabel('Upload CV').setInputFiles('files/cv.pdf');
await page.getByLabel('Attachments').setInputFiles([
'files/a.png', 'files/b.png',
]);
// In-memory buffer also works
await page.getByLabel('Upload CV').setInputFiles({
name: 'cv.pdf', mimeType: 'application/pdf', buffer: Buffer.from('...'),
});

// Only option: custom widget with no reachable input. Listen first.
const [fileChooser] = await Promise.all([
page.waitForEvent('filechooser'),
page.getByRole('button', { name: 'Choose file' }).click(),
]);
await fileChooser.setFiles('files/cv.pdf');
```

Q 52

LEVEL · Mid

Locator vs ElementHandle (page.$): why is the locator's lazy auto-waiting safer than an ElementHandle's eager snapshot?

CONCEPT

A Locator is a lazy, re-evaluated instruction: it holds a description of how to find an element and re-runs that query, with auto-waiting, every time you act on it. An **ElementHandle**, what **page.$()** returns, is an eager snapshot: it queries the DOM immediately and hands back a reference to that one node as it existed at that instant. The safety gap follows directly. If the framework re-renders, React replacing a node, a list re-sorting, a row being rebuilt, the locator simply re-finds the element on the next action, while the ElementHandle now points at a detached node and throws an "element is not attached to the DOM" error, which is the Playwright cousin of Selenium's stale-element problem. ElementHandle also does not auto-wait the way a locator does. The Playwright team's own guidance is blunt: **ElementHandle is discouraged**; use Locator. The handful of legitimate ElementHandle uses are narrow, passing a live node into evaluate for direct DOM work, and even those are increasingly covered by locator methods.

INTERVIEW STRATEGY

This is really the lazy-instruction-versus-eager-reference idea applied one level down, and the interviewer wants to hear that you know ElementHandle reintroduces the very staleness Playwright was built to kill. Contrast the two crisply: locator re-finds with auto-waiting, ElementHandle snapshots once and goes detached on re-render. Say outright that the Playwright team discourages ElementHandle, that is the signal you are current. The trap is treating page.$ as just an older spelling of locator. They usually follow with 'is there ever a reason to use ElementHandle?', have the narrow evaluate / live-node case ready, and note even that is shrinking.

How to phrase it

> *A locator is a lazy instruction: it stores how to find the element and re-runs that query with auto-waiting every time I act on it. An ElementHandle, which is what page.$ gives back, is an eager snapshot: it queries the DOM the instant I call it and hands me a reference to that one node as it was at that moment. That difference is the whole safety story. If the app re-renders, React swapping a node out, a list re-sorting, a table row rebuilt, the locator just re-finds the element on the next action and carries on. The ElementHandle now points at a detached node and throws element is not attached to the DOM, which is effectively Selenium's stale-element error sneaking back in through a different door. On top of that, ElementHandle does not auto-wait the way a locator does, so I also lose the actionability checks. That is why the Playwright team itself discourages ElementHandle and tells you to use Locator, and I would say that explicitly. For the follow-up about whether there is ever a legitimate reason, the honest answer is yes but narrow: when I need to pass a live DOM node into evaluate to do something directly in the page, an element handle is the right currency. But even those cases are increasingly covered by locator methods like evaluate on the locator itself, so in modern Playwright I almost never reach for page.$, the default is always Locator, and ElementHandle is the rare, deliberate exception.*

Key points to hit

(1) Locator = lazy re-evaluated instruction with auto-waiting. **(2)** ElementHandle (page.$) = eager one-time snapshot of a node. **(3)** On re-render the locator re-finds; the handle goes detached and

throws. **(4)** Detached-node error is Selenium's stale-element creeping back in. **(5)** Playwright team discourages ElementHandle: default to Locator. **(6)** They usually ask next: narrow handle use is passing a live node to evaluate; even that is shrinking.

CODE

```
// Discouraged: eager snapshot, no auto-wait, goes stale on re-render
// const handle = await page.$('#cart-total');
// ... app re-renders the cart ...
// await handle.click(); // Error: element is not attached to the DOM

// Preferred: lazy, re-found with auto-waiting on every action
const cartTotal = page.locator('#cart-total');
await cartTotal.click(); // re-queries here
await expect(cartTotal).toHaveText('₹4,000'); // re-queries again

// Rare legitimate handle use: passing a live node into evaluate
const handle = await cartTotal.elementHandle();
await page.evaluate(el => el?.scrollIntoView(), handle);
```

Q 53 LEVEL · Junior to Mid

getByText vs getByLabel vs getByPlaceholder vs getByAltText vs getByTitle, what does each target, and how do you choose between them?

CONCEPT

These are the content-and-attribute locators, each tied to a different thing the user perceives. **getByText** matches by an element's visible text content, ideal for links, buttons, headings and static copy. **getByLabel** matches a form control by its associated label (a <label>, aria-label, or aria-labelledby), the right choice for inputs, selects and checkboxes, because it is how a user reads the form. **getByPlaceholder** matches an input by its placeholder attribute, a fallback for inputs that have no proper label. getByAltText matches by an image's alt text, for <img> and any element exposing alt. getByTitle matches by the title attribute, often tooltips and icon-only controls. The way to choose is to follow the recommended priority: prefer the locator tied to the most stable, user-perceived attribute. For a labelled field use getByLabel, never getByPlaceholder; use getByPlaceholder only when no label exists; use getByAltText for images and getByTitle for tooltip-only controls. And remember getByRole usually sits above all of these, these fill the gaps role cannot address.

INTERVIEW STRATEGY

They are quietly checking that you can pick the right locator for the right element rather than defaulting to one for everything. Give a one-line target for each, text content, label, placeholder, alt, title, then make the selection rule explicit: prefer the most stable user-perceived attribute, and slot each one to the element type it suits. The senior move is calling out the common mistake of using getByPlaceholder for a field that actually has a label, since placeholders disappear on input and are not real labels. The next question is normally 'label versus placeholder, which and why?', answer label always when one exists, placeholder only as a fallback. Tie the whole set back under

getByRole as the first choice.

How to phrase it

> *These are the content-and-attribute locators, and each targets a different thing the user actually perceives, so I would map them out one by one. getByText matches an element by its visible text content, I use it for links, headings and static copy. getByLabel matches a form control by its associated label, whether that is a label element, an aria-label or aria-labelledby, that is my default for inputs, selects and checkboxes because it is exactly how a user reads the form. getByPlaceholder matches an input by its placeholder attribute, and I treat it strictly as a fallback for inputs that have no real label. getByAltText matches an image by its alt text, for img elements and anything exposing alt. getByTitle matches by the title attribute, which is mostly tooltips and icon-only controls that have no visible text. How I choose between them is the recommended priority: prefer the locator tied to the most stable, user-perceived attribute, and slot each one to the element it fits. That is also my answer to the common label-versus-placeholder follow-up, if a field has a label I always use getByLabel, never getByPlaceholder, because the placeholder disappears the moment the user types and was never a real label in the first place; placeholder is only for the unlabelled case. And I would close by tying the whole set back under getByRole, which usually sits above all of these, these five fill the gaps that a role-based locator on its own cannot address, like distinguishing two inputs by their label or finding an image by its alt text.*

Key points to hit

(1) getByText: visible text content, links, headings, static copy. **(2)** getByLabel: form control by its label, default for inputs/selects/checkboxes. **(3)** getByPlaceholder: placeholder attribute, fallback only when no label exists. **(4)** getByAltText: image alt text; getByTitle: title attribute (tooltips, icon controls). **(5)** Choose by the most stable user-perceived attribute; getByRole sits above all five. **(6)** The next question: label always over placeholder, placeholders vanish on input.

CODE

```
await page.getByText('Forgot your password?').click(); // visible text
await page.getByLabel('Email address').fill('a@test.com'); // labelled field
await page.getByPlaceholder('Search products').fill('laptop'); // no label -> fallback
await page.getByAltText('Company logo').click(); // image alt
await page.getByTitle('Close dialog').click(); // tooltip / icon-only

// Prefer label over placeholder when both exist
// BAD: await page.getByPlaceholder('Enter email').fill(...);
// GOOD: await page.getByLabel('Email address').fill(...);
```

Q 54 LEVEL · Mid to Senior

frameLocator vs page.frame(): when do you use the retrying frameLocator and when the older frame handle?

CONCEPT

Both reach into an iframe, but they belong to different generations of the API. **page.frameLocator()** returns a **FrameLocator**, a lazy, auto-retrying scope. You chain getByRole, getByLabel and the rest off it, and every action re-resolves the frame and the element with auto-waiting, so it works even if the iframe has not finished loading yet or gets re-created. This is the recommended, default way to handle frames. **page.frame()** (and frame handles via page.frames()) returns an older Frame object, a reference you look up eagerly by name or URL. It returns null if the frame is not present yet, it does not retry, and a Frame object exposes the older page-style API rather than the locator API. You reach for page.frame() only when you need that frame-level object for something the FrameLocator does not surface, reading the frame's URL or name, calling frame-level navigation or evaluate, or iterating page.frames() to inspect every frame on the page. The rule: frameLocator for interacting with elements, page.frame() only when you genuinely need the frame object itself.

INTERVIEW STRATEGY

They are looking to see that you default to the modern, retrying API and understand why the old handle is a footgun for element work. Contrast them on the axis that matters: frameLocator is lazy and auto-retrying, page.frame() is an eager lookup that returns null and does not wait. Say plainly that frameLocator is the recommended default for interacting with elements. Then be precise about the narrow place page.frame() still earns its keep, when you need the frame object itself for its URL, name, navigation or to iterate page.frames(). The trap is using page.frame() for element interaction and then bolting on manual waits to cope with the null-when-not-loaded behaviour. A common follow-up is ‘why is frameLocator safer?’, answer that it re-resolves the frame and element with auto-waiting, so it cannot go stale or fire before the frame exists.

How to phrase it

> *Both reach into an iframe, but they are from different generations of the API and I default firmly to one. page.frameLocator returns a FrameLocator, which is a lazy, auto-retrying scope, I chain my normal getByRole or getByLabel off it, and every action re-resolves both the frame and the element with auto-waiting. So it works even if the iframe has not finished loading or gets re-created mid-test, and that is exactly why it is the recommended default for actually interacting with elements inside a frame. page.frame, and the frame handles you get from page.frames, return the older Frame object, an eager lookup by name or URL. The catch, and this is my answer to why frameLocator is safer, is that page.frame returns null if the frame is not present yet, it does not retry, and the Frame object exposes the older page-style API rather than the locator API. So if I use it for element work I end up writing manual waits and null checks just to cope with timing that frameLocator handles for me automatically. Where page.frame still earns its place is when I genuinely need the frame object itself rather than to click something inside it, reading the frame's URL or name, calling frame-level navigation or evaluate, or iterating page.frames to inspect every frame on the page, which is handy when I do not know the frame's selector up front. So the rule I would give is: frameLocator for interacting with elements, which is almost always; page.frame only when I actually need the frame-level object. The trap I would warn against is using page.frame for element interaction and then papering over its null-when-not-loaded behaviour with sleeps, which is the old fragile pattern frameLocator was designed to retire.*

Key points to hit

(1) frameLocator returns a lazy, auto-retrying FrameLocator, the recommended default. **(2)** Chain getByRole/getByLabel off it; it re-resolves frame + element with auto-waiting. **(3)** page.frame() is

an eager lookup: returns null if absent, no retry, old API. **(4)** Use page.frame() only for the frame object: URL/name, navigation, evaluate, page.frames(). **(5)** Rule: frameLocator to interact with elements; page.frame() only for the frame itself. **(6)** Avoid: page.frame() for element work, then bolting on manual waits for null timing.

CODE

```
// Default: retrying FrameLocator, auto-waits, chains normal locators
const checkout = page.frameLocator('iframe[title="Card payment"]');
await checkout.getByLabel('Card number').fill('4111 1111 1111 1111');
await checkout.getByRole('button', { name: 'Pay' }).click();

// page.frame(): only when you need the frame OBJECT itself
const frame = page.frame({ name: 'payment' });
if (frame) {
console.log(frame.url()); // read frame-level URL
await frame.evaluate(() => document.title);
}

// Iterate every frame on the page
for (const f of page.frames()) console.log(f.name(), f.url());
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Why can a Playwright Locator never throw StaleElementReferenceException?

A) It caches the element on creation
B) It runs in a separate thread
C) It disables DOM mutations
D) It is a lazy instruction that re-queries with retries on each action

2. Which locator is recommended first because it survives CSS refactors?

A) getByTestId
B) getByRole
C) getByText
D) locator with absolute XPath

3. Which filter selects table rows that contain a given child locator?

A) filter({ has })
B) filter({ hasText })
C) filter({ visible: true })
D) filter({ nth: 0 })

4. How does Playwright handle an input inside a custom element's shadow root?

A) Requires JavaScript shadowRoot traversal
B) Cannot interact with shadow DOM
C) Pierces the shadow boundary automatically with normal locators
D) Needs a special frameLocator call

5. Why use page.once('dialog', ...) rather than page.on for a confirm dialog?

A) once removes the listener after the first use, avoiding cross-test pollution
B) once is faster
C) on does not support dialogs
D) once retries automatically

Section 3 · Answer key

#	Answer	Why and where to revisit
1	D) It is a lazy instruction that re-queries with retries on each action	A locator holds no node reference; it re-queries the DOM each time you act, so there is nothing to go stale. *Revisit Section 3.*
2	B) getByRole	getByRole targets semantic role and accessible name, so it survives class and DOM-structure changes. *Revisit Section 3.*
3	A) filter({ has })	filter({ has }) narrows a set to elements that contain the supplied child locator; hasText narrows by text. *Revisit Section 3.*
4	C) Pierces the shadow boundary automatically with normal locators	Playwright pierces shadow DOM automatically for CSS and user-facing locators; no manual traversal is needed. *Revisit Section 3.*
5	A) once removes the listener after the first use, avoiding cross-test pollution	once fires a single time and detaches, so the handler cannot leak into later tests sharing the context. *Revisit Section 3.*

SECTION 04

Auto-waiting and Synchronisation

Auto-waiting is the feature interviewers most want you to understand properly, because it is the reason Playwright suites are less flaky than Selenium ones. The trick is to explain the actionability checks, then show you still know when an explicit wait is genuinely needed. These seventeen questions cover the internal polling model, the four timeout layers, explicit waits, observe-versus-control with waitForResponse and route, load states, the expect retry mechanism, and time control with page.clock.

In this section

- The actionability checks each action runs, and the retry-until-timeout loop.
- The four timeout layers and why a huge global timeout hides regressions.
- When explicit waits are still needed despite auto-waiting.
- waitForResponse (observe) vs route (control): two different goals.
- Load states: domcontentloaded vs load vs networkidle, and why networkidle is fragile.
- Web-first assertions retry; isVisible is a one-shot check for conditionals.
- page.clock for session timeouts, token expiry and deterministic dates.

Q 55 LEVEL · Mid

How does Playwright's auto-waiting mechanism work internally?

CONCEPT

Before every action Playwright runs a set of **actionability checks**, and the set is action-specific. A click checks that the element is visible, stable (not animating), enabled, and receiving pointer events (not covered); Playwright scrolls it into view before acting, but that scroll is not itself a check. A **fill** checks visible, enabled and editable, and nothing else. A hover checks visible, stable and receiving pointer events. Playwright keeps re-checking these conditions and retries until they all pass or the action timeout is reached, then performs the action. The pointer-events check is the one that kills the classic overlay bug. There is a **force: true** escape that bypasses every check, use it only when you have consciously verified the bypass is correct, never as a way to silence an auto-waiting failure.

INTERVIEW STRATEGY

This is a flakiness question at heart; the interviewer wants to know whether you understand why Playwright tests are stable, not just that they are. Name the checks, stress they are per-action, then tell the cookie-banner story, it is the single clearest demonstration of why the pointer-events check matters. The thing to avoid is reaching for force as a fix. The follow-up is reliably 'what does force: true do and when do you use it?', answer that it bypasses every check and is a conscious last resort, never a flakiness silencer.

How to phrase it

> *Before each action Playwright runs actionability checks, and they differ by action. A click waits for the element to be visible, stable, enabled, and receiving pointer events; a fill waits for visible, enabled and editable. It polls those conditions continuously and retries until they all pass or it times out, then it acts. The example I use to make this concrete is a cookie banner. Picture a button that is visible but covered by a consent overlay. Selenium's explicit wait sees visible equals true and happily clicks, except it clicks the overlay, not the button, and the test fails for a reason that looks nothing like the real cause. Playwright's pointer-events check detects that the button is obstructed and waits until the overlay is gone, so it refuses to click the wrong element. That single check removes a whole category of misclick flakiness. For the follow-up about force, which always comes: force true bypasses every actionability check, and I treat it as a conscious decision for a genuinely verified case, never as a way to mute a failing check. If a check is failing, that is usually the test telling me something real about the page, and silencing it just ships the bug.*

Key points to hit

(1) Actionability checks run before each action and are action-specific. **(2)** Click: visible, stable, enabled, pointer-events; retried until timeout. **(3)** Pointer-events check defeats the cookie-overlay misclick. **(4)** Asked next: force: true bypasses all checks, conscious last resort only. **(5)** A failing check is usually a real signal, not noise to silence. **(6)** The easy mistake: reaching for force to mask auto-waiting failures.

CODE

```
// All checks run automatically before this click
await page.getByRole('button', { name: 'Submit' }).click();
// internally: visible? stable? receives pointer events? enabled?
// t=0 visible? no -> retry
// t=100 visible yes, stable? no (animating) -> retry
// t=200 every check this action runs passes -> click

// force bypasses EVERY check: use consciously, not as a workaround
await page.locator('#hidden-trigger').click({ force: true });
```

Q 56 LEVEL · Mid

What are the timeout levels in Playwright and how should you configure them?

CONCEPT

There are four timeout layers, each overridable at different granularities. **Test timeout** (default 30s) bounds the whole test. **Action timeout** bounds a single interaction. Navigation timeout bounds a page load. Expect timeout (default 5s) bounds a retrying assertion. The principle that matters: a huge global timeout hides performance regressions. Keep the action timeout tight, around 15 seconds, so that if an interaction takes longer, the test fails loudly rather than waiting silently. For legitimately slow tests like report generation, mark them with **test.slow()** (which triples the test timeout) rather than raising the global baseline. Per-action and per-assertion overrides exist for the genuine exceptions.

INTERVIEW STRATEGY

This one tests judgement, not recall, anyone can name four timeouts. List them quickly, then make your real point: tight baseline, explicit exceptions, because a generous global timeout hides performance regressions. The risk is recommending a big global timeout to ‘reduce flakiness’, which actually masks slowdowns. The follow-up is usually ‘how do you handle a genuinely slow test?’, answer test.slow() on that test rather than raising the baseline, so the slow one is tagged and tracked.

How to phrase it

> *There are four timeout layers: the test timeout for the whole test, the action timeout for a single interaction, the navigation timeout for a page load, and the expect timeout for retrying assertions. But the part that usually wins this question is the configuration philosophy, not the numbers. Setting a huge global timeout feels safe and people do it to reduce flakiness, but it actually hides performance regressions, a page that used to load in two seconds and now takes twenty will still pass, and you never find out until a user complains. So I keep the action timeout tight, around fifteen seconds; if an interaction takes longer than that, something is genuinely wrong and I want the test to fail loudly. That sets up the follow-up about genuinely slow tests, like report generation: I mark those with test.slow, which triples just that test's timeout, rather than raising the baseline for everything. That way the baseline stays tight, the slow tests are explicitly flagged and tracked, and a creeping regression shows up as a failure instead of hiding inside a generous limit. Tight by default, loosened only where I have made a deliberate, visible exception.*

Key points to hit

(1) Four layers: test, action, navigation, expect (defaults vary). **(2)** A huge global timeout hides performance regressions. **(3)** Keep action timeout tight (~15s) so slow interactions fail loudly. **(4)** Follow-up: genuinely slow test -> test.slow(), not a bigger baseline. **(5)** Slow tests stay explicitly flagged and tracked. **(6)** Trap: recommending a big global timeout to 'reduce flakiness'.

CODE

```
export default defineConfig({
timeout: 60_000, // test
expect: { timeout: 10_000 }, // assertions
use: {
actionTimeout: 15_000, // per action, kept tight
navigationTimeout: 30_000, // navigation
},
});

test('report generation', async ({ page }) => {
test.slow(); // triples THIS test's timeout, not the global baseline
// ...
});

await expect(page.locator('#async-data')).toBeVisible({ timeout: 20_000 });
```

Q 57 LEVEL · Mid

When do you need explicit waits despite auto-waiting?

CONCEPT

Auto-waiting covers element actionability, it does not cover everything. Explicit waits are still needed for a few specific conditions. URL changes after a navigation, with **waitForURL**. Disappearance of an element such as a loading spinner or skeleton, with waitFor({ state: 'hidden' }). Network correlation, capturing a response a UI action triggers with **waitForResponse** inside a Promise.all. And custom JavaScript state in legacy apps, with **waitForFunction** polling a window

flag. The rule of thumb: let auto-waiting handle the element, and add an explicit wait only for the surrounding condition it cannot see, navigation, disappearance, network, or app-defined readiness.

INTERVIEW STRATEGY

They want to confirm you are not the candidate who thinks auto-waiting means never waiting, and equally not the one who sprays fixed sleeps everywhere. Group explicit waits into named scenarios, navigation, disappearance, network, custom state, so it sounds systematic. The thing to avoid is page.waitForTimeout, a fixed sleep, which is the answer that gets you marked down. The follow-up is often 'why not just add a sleep?', answer that a fixed sleep is either too short and flaky or too long and slow, while a condition wait is both fast and reliable.

How to phrase it

> *Auto-waiting handles element actionability, but it does not handle everything, so I keep a small mental list of when an explicit wait is still right, and just as importantly which wait. First, navigation: after clicking something that changes the page, I use waitForURL to confirm I have actually arrived. Second, disappearance: auto-waiting waits for things to appear, so for a loading spinner or skeleton I explicitly wait for the hidden state before asserting on the content behind it. Third, network correlation: when I need the response a UI action triggers, I use waitForResponse inside a Promise.all, and the order matters, I start the wait before the click, because if the response arrives before my listener is attached I miss it. Fourth, custom application state in legacy apps, where I poll a window readiness flag with waitForFunction. What I never reach for is a fixed sleep, which is the answer to the follow-up about why not just add a wait: a fixed sleep is either too short, so it is flaky, or too long, so it is slow on every single run, whereas a condition wait is both fast and reliable because it returns the instant the condition is true. So the rule is: let auto-waiting handle the element, and add a specific condition wait only for what it genuinely cannot see.*

Key points to hit

(1) Auto-waiting covers actionability, not navigation/network/custom state. **(2)** waitForURL for page changes; waitFor({ state: 'hidden' }) for spinners. **(3)** Promise.all + waitForResponse, listener attached before the click. **(4)** waitForFunction for custom window readiness flags in legacy apps. **(5)** Expect next: never a fixed sleep, flaky if short, slow if long. **(6)** Where this goes wrong: answering with page.waitForTimeout.

CODE

```
// URL change
await page.getByRole('button', { name: 'Checkout' }).click();
await page.waitForURL('/checkout', { timeout: 10_000 });

// Disappearance
await page.locator('#loading-spinner').waitFor({ state: 'hidden' });

// Network correlation: listen first
const [response] = await Promise.all([
page.waitForResponse(r => r.url().includes('/api/orders') && r.status() === 201),
page.getByRole('button', { name: 'Place Order' }).click(),
]);
expect((await response.json()).orderId).toBeTruthy();

// Custom app state
await page.waitForFunction(() => (window as any).__APP_READY__ === true);
```

Q 58 LEVEL · Mid

What is the difference between waitForResponse and page.route() for API interaction?

CONCEPT

The distinction is observe versus control. **waitForResponse** is passive and read-only, it watches the network and captures responses that match a filter, so you can validate what the real backend actually returned and what payload the UI sent. **page.route()** is active and write, it intercepts a request and decides what the app receives, so you can mock, modify, or fail it. Use waitForResponse for **integration-style** tests that assert the UI talks to the real backend correctly. Use route for unit-style frontend tests that isolate the UI from the backend entirely. They are not competitors; they serve different testing goals, and a mature suite uses both.

INTERVIEW STRATEGY

The point of this one is to see that you choose deliberately rather than mocking everything by reflex. Compress the answer into observe-versus-control, read-versus-write, then map each to a testing goal, integration vs isolated frontend. The pitfall is implying route is the default for all API testing, which produces tests that never touch the real contract. Expect the follow-up ‘when would you use one over the other?’, answer integration tests use waitForResponse against the real backend, isolated frontend tests use route to control the response.

How to phrase it

> *I frame it as observing versus controlling. waitForResponse is read-only: it watches the network and captures the response that matches my filter, so I can see what the real app sent and what the real backend returned. route is write: it intercepts the request and lets me decide what the app receives, so I can mock it, modify it, or make it fail. Because of that they serve different goals, and choosing between them is the real skill the question is testing. I use waitForResponse for integration-style tests, for example, asserting that clicking Place Order really does POST the right items to the real orders endpoint and gets a 201 back. I use route for unit-style frontend tests that isolate the UI from the backend entirely, like checking the UI shows the right error banner when the payments API returns a 402. That is also my answer to the follow-up about when I pick one over the other. The mistake I see, and the one I would warn against, is reaching for route to mock everything, which produces a suite that passes happily while never verifying the real contract, so a backend change can break production while every test stays green. Both patterns matter, and a mature suite uses each where it fits.*

Key points to hit

(1) waitForResponse: passive, read-only, observe real traffic. **(2)** route: active, write, control what the app receives. **(3)** waitForResponse for integration; route for isolated frontend tests. **(4)** They usually ask next: pick by goal, verify real contract vs isolate the UI. **(5)** Over-mocking with route can stay green while production breaks. **(6)** What loses the point: implying route is the default for all API testing.

CODE

```
// Observe: validate the real API call
const [res] = await Promise.all([
page.waitForResponse(r => r.url().includes('/api/orders') && r.request().method() ===
'POST'),
page.getByRole('button', { name: 'Place Order' }).click(),
]);
expect((await res.request().postDataJSON()).items).toHaveLength(2);
expect(res.status()).toBe(201);

// Control: force a payment failure the backend cannot reliably produce
await page.route('**/api/payments', route => route.fulfill({
status: 402, body: JSON.stringify({ error: 'Card declined' }),
}));
await page.getByRole('button', { name: 'Pay Now' }).click();
await expect(page.getByText('Card declined')).toBeVisible();
```

Q 59 LEVEL · Junior to Mid

How does page.waitForLoadState() work, and when do you use each option?

CONCEPT

There are three load states. **domcontentloaded** fires when the HTML is parsed, before sub-resources load, fastest. load fires when HTML and resources are loaded, the default for page.goto. **networkidle** fires after there are no network requests for about 500ms. networkidle

sounds ideal but is fragile for apps with background polling, analytics beacons, or WebSocket keepalives, because the quiet window may never arrive. The more deterministic pattern is to wait for domcontentloaded and then wait for a **specific element** that signals the page is ready, a stats panel visible, a skeleton hidden. That targets meaningful application state rather than a timing heuristic, which is what you want inside a page object's ready method.

INTERVIEW STRATEGY

They are checking whether you have an opinion grounded in experience, not just a definition of three states. Define them, then take a clear position: avoid networkidle for anything with background traffic, and wait on a meaningful element instead. The thing to avoid is recommending networkidle as the safe default, which hangs on modern apps that poll. They usually follow with 'why is networkidle discouraged?', answer that analytics, polling and WebSocket keepalives mean the network never goes quiet, so the wait times out or is unreliable.

How to phrase it

> *There are three load states. domcontentloaded fires when the HTML is parsed but before images and other resources finish; it is the fastest. load fires when the page and its resources are loaded, and it is the default for goto. networkidle fires once there have been no network requests for about half a second. I use networkidle sparingly, and I would say so directly because it is a place people go wrong: it sounds perfect but is fragile in real apps. Anything with background polling, analytics beacons, or WebSocket keepalives may never go quiet for that half second, so the wait either hangs or behaves unpredictably, which is exactly the answer to the follow-up about why networkidle is discouraged. My preferred pattern, especially inside a page object's ready-check method, is to wait for domcontentloaded and then wait for a specific element that actually means the page is ready: the stats panel visible, the loading skeleton hidden. That is more deterministic because it targets meaningful application state instead of a timing heuristic about network silence, and it is faster because it returns the moment the real signal appears. So I treat the load states as a coarse tool and lean on element-level readiness for the signal that actually matters.*

Key points to hit

(1) domcontentloaded: HTML parsed (fastest); load: HTML + resources (default). **(2)** networkidle: ~500ms of no requests, fragile with background traffic. **(3)** Prefer domcontentloaded + wait for a meaningful element. **(4)** The next question: networkidle discouraged, polling/analytics/WebSockets never go quiet. **(5)** Element readiness targets real app state, not a timing heuristic. **(6)** The common miss: recommending networkidle as the safe default.

CODE

```
await page.goto('/simple', { waitUntil: 'domcontentloaded' }); // fastest
await page.goto('/dashboard'); // load (default)

// Deterministic page-ready check in a page object
async waitForDashboardLoad(): Promise<void> {
await this.page.waitForLoadState('domcontentloaded');
await this.page.getByRole('region', { name: 'Statistics' }).waitFor();
await this.page.locator('#loading-skeleton').waitFor({ state: 'hidden' });
}
```

Q 60 LEVEL · Junior to Mid

How does Playwright's expect() retry mechanism work?

CONCEPT

Playwright's web-first assertions automatically retry, they poll until the assertion passes or the expect timeout is reached. **toHaveText**, **toBeVisible**, **toHaveValue** and the rest all behave this way. By contrast, the methods isVisible, isEnabled and isChecked check state once, immediately and return a boolean, they do not retry. That makes them suited to conditional logic, not assertions. The classic bug is calling isVisible and then asserting on the boolean: it captures a point-in-time snapshot and fails if the element appears a millisecond later. The fix is the retrying web-first assertion, and the prefer-web-first-assertions ESLint rule enforces it so the correct, retrying form is the default.

INTERVIEW STRATEGY

This is a knowledge checkpoint interviewers use to separate doc-readers from framework-builders, so be precise. Frame it as question versus assertion: isVisible asks 'is it visible now', toBeVisible asserts 'it should become visible'. Where this goes wrong is asserting on an isVisible boolean, which reintroduces the snapshot race. The follow-up is usually 'when would you ever use isVisible?', answer genuine conditional logic, like filling an optional field only if it is present, and close on the ESLint rule that enforces the retrying form.

How to phrase it

> *Playwright's web-first assertions retry automatically: toBeVisible, toHaveText, toHaveValue and so on poll until they pass or the expect timeout is hit. The methods that look similar but behave differently are isVisible, isEnabled and isChecked, those check state once, right now, and return a boolean without retrying. This is a knowledge checkpoint that separates people who have read the docs from people who have shipped a framework, so I am precise about it. The phrasing I use: isVisible is a question, is it visible right now; toBeVisible is an assertion, I expect it to become visible within the timeout. The classic bug, and the trap, is calling isVisible and asserting on the result, which captures a snapshot and fails if the element shows up a millisecond later, instant flakiness. So I use the retrying assertion for almost everything. For the follow-up about when I would ever use isVisible, the honest answer is genuine conditional logic: filling an optional phone-number field only if it happens to be present, for instance, where I actually do want a one-time check and a branch. And I turn on the prefer-web-first-assertions ESLint rule so the retrying form is the default and nobody on the team reintroduces the snapshot bug by writing isVisible because it reads more naturally.*

Key points to hit

(1) Web-first assertions poll/retry until pass or expect timeout. **(2)** isVisible/isEnabled/isChecked check once and return a boolean. **(3)** Phrasing: isVisible is a question, toBeVisible is an assertion. **(4)** Asked next: use isVisible only for genuine conditional logic. **(5)** prefer-web-first-assertions enforces the retrying form by default. **(6)** Avoid: asserting on an isVisible boolean, the snapshot race.

CODE

```
// Retrying web-first assertion
await expect(page.locator('#status')).toHaveText('Order Confirmed');

// Wrong: point-in-time snapshot
// const visible = await page.locator('#result').isVisible();
// expect(visible).toBe(true); // fails if it appears 1ms later

// Right use of isVisible: conditional logic
if (await page.locator('#phone-number').isVisible()) {
await page.locator('#phone-number').fill('+91-9876543210');
}
```

Q 61 LEVEL · Mid to Senior

How do you use page.clock to test time-dependent features?

CONCEPT

page.clock.install() replaces the browser's native time APIs, Date, setTimeout, setInterval, requestAnimationFrame, with a controllable fake clock, so you can simulate time passing instantly instead of actually waiting. It is the right tool for session timeouts, token expiry, scheduled updates, countdowns, and relative timestamps. **fastForward** jumps time forward and fires the timers that would have triggered in that span; **runFor** runs pending timers within a duration; setFixedTime pins the clock to a specific moment for deterministic date rendering. The key rule is to install the clock before navigating, so it captures timers set during page load. A 24-hour session-timeout test then completes in milliseconds rather than a day.

INTERVIEW STRATEGY

This is a chance to stand out; page.clock is underused, so knowing it well signals you keep current. Position it as the modern answer to time-dependent testing, then explain the two things that show real use: install before navigation, and fastForward fires timers instantly rather than waiting real time. The pitfall is the old workarounds, real waits, overriding Date by hand, or mocking the backend, which are all fragile. The next question is normally 'how would you test a session timeout?', answer the install-then-fastForward pattern that runs in milliseconds.

How to phrase it

> *page.clock is one of the most underused features and one of the best to bring up, because knowing it signals I keep current with the tool. Calling clock.install replaces every time API in the browser, Date, setTimeout, setInterval, requestAnimationFrame, with a fake clock I control, so I can simulate time passing instead of waiting. That matters because of what it replaces: before it existed, testing a 24-hour session timeout meant either waiting 24 hours, mocking the backend to return an expired token, or overriding Date with page.evaluate, all of them fragile. So for the follow-up about how I would test a session timeout, my answer is the clean one: I install the clock, and the important detail is to install it before navigating so it captures timers set during page load. Then fastForward jumps time forward, and the key insight is that it does not wait real time, it instantly fires all the timers that would have fired in that span, so a 24-hour timeout test runs in milliseconds. I use runFor to run pending timers within a window for token-refresh tests, and setFixedTime to pin a date for deterministic countdowns and relative-timestamp rendering, so a 'posted 2 days ago' label is testable without flake. It turns slow, flaky, time-based tests into fast deterministic ones, which is exactly the kind of upgrade that impresses in an interview.*

Key points to hit

(1) clock.install replaces Date/setTimeout/setInterval/rAF with a fake clock. **(2)** Install BEFORE navigation so it captures load-time timers. **(3)** fastForward fires due timers instantly: no real waiting. **(4)** runFor runs pending timers in a window; setFixedTime pins a date. **(5)** Follow-up: test a session timeout via install-then-fastForward in ms. **(6)** The easy mistake: old workarounds, real waits, hand-overriding Date, backend mocks.

CODE

```
test('logged out after 24h session timeout', async ({ page }) => {
await page.clock.install(); // before navigation
await page.goto('/dashboard');
await expect(page.getByTestId('welcome-banner')).toBeVisible();

await page.clock.fastForward(24 * 60 * 60 * 1000); // jump 24h instantly

await expect(page.getByTestId('session-expired-modal')).toBeVisible();
await expect(page).toHaveURL(/\/login/);
});

// Deterministic relative timestamp
await page.clock.setFixedTime(new Date('2026-04-03T10:00:00Z'));
await page.goto('/posts/123'); // created 2026-04-01
await expect(page.locator('.post-timestamp')).toHaveText('2 days ago');
```

Q 62 LEVEL · Mid to Senior

What are the five actionability checks Playwright runs, and which actions run which?

CONCEPT

Playwright runs actionability checks before an action and retries until they pass. The docs name five: **1. Visible**, a non-empty bounding box and not visibility:hidden; **2. Stable**, the element hasn't moved for at least two consecutive animation frames, so no CSS transition is in progress; 3. Receives events, the element is the hit target at the point of action, so no overlay is sitting on top of it; 4. Enabled, not disabled; 5. Editable, not readonly and not disabled. The part that earns the marks is that the set is per action, not one fixed list. click, dblclick, tap, check, uncheck and setChecked run visible, stable, receives events and enabled. hover and dragTo run visible, stable and receives events. fill and clear run visible, enabled and editable only, with no stable check and no receives-events check, which is exactly why a cookie banner blocks a click but not a fill. **selectOption** runs visible and enabled; selectText runs visible alone. And focus, blur, press, pressSequentially, dispatchEvent and setInputFiles run no actionability checks at all. Two things candidates wrongly add to the list: attached is an assertion (toBeAttached), not one of the five, and in viewport is not a check at all, Playwright simply scrolls the element into view before acting.

INTERVIEW STRATEGY

You are being asked to show precise knowledge of actionability, not just ‘Playwright waits’. Name the five with brief definitions, visible, stable, receives events, enabled, editable, then immediately make the point that the set is per action. That second half is where almost every candidate loses ground, because they recite one fixed list and then cannot explain why fill still works under a cookie banner. Say plainly that fill runs visible, enabled and editable only, and that focus and press run no checks at all. The 'error tells you which check failed' framing signals depth. Two traps to avoid: calling attached one of the five (it is an assertion), and calling in viewport a check (Playwright scrolls, it does not check). A common follow-up is ‘what does "stable" mean exactly?’, answer no movement for two consecutive animation frames.

How to phrase it

> *The docs name five actionability checks, and the second half of my answer is the one that matters, because the five are not a fixed list that runs before everything. First the five. One, visible: a non-empty bounding box, not visibility hidden. Two, stable, and this is where my answer to the follow-up sits: the element hasn't moved for at least two consecutive animation frames. That's how Playwright detects there's no CSS transition in progress. A button sliding into view isn't stable yet, and clicking it mid-transition would either miss or hit the wrong spot. Three, receives events: the element is the actual hit target at the point of action, so no modal or cookie overlay is sitting on top of it. Four, enabled: not disabled. Five, editable: not readonly and not disabled. Now the part that surprises people. Each action runs its own subset. Click, dblclick, tap, check and uncheck run visible, stable, receives events and enabled. Hover and dragTo run visible, stable and receives events, and skip enabled, because a disabled element can still be hovered. Fill and clear run only visible, enabled and editable. No stable check and no receives events check. That is a genuinely useful thing to know, because it explains something people find odd in practice: a cookie banner will block a click and will not block a fill. And at the far end, focus, blur, press, pressSequentially, dispatchEvent and setInputFiles run no actionability checks at all, which is why press is sometimes the pragmatic escape when an action is being blocked. Two corrections I'd make if the interviewer offers them. Attached is not one of the five, it is an assertion, expect locator toBeAttached. And in viewport is not a check either. Playwright scrolls the element into view before acting, and scrolling is not the same as checking. The framing I'd close on, because it makes auto-waiting feel diagnosable rather than magical, is that when an action times out the error names the failing check, element is not stable, element does not receive pointer events. The error tells me which check failed and therefore where the bug is.*

Key points to hit

(1) The five: visible, stable, receives events, enabled, editable. **(2)** The set is per action, not one fixed list run before everything. **(3)** click, dblclick, tap, check: visible + stable + receives events + enabled. **(4)** fill and clear: visible + enabled + editable ONLY, no stable, no receives events. **(5)** hover: visible + stable + receives events. focus, press, dispatchEvent: no checks. **(6)** Trap: 'attached' is an assertion, and 'in viewport' is not a check, Playwright scrolls.

CODE

```
// Each action runs its OWN subset. From the actionability docs:
//
// click, dblclick, tap visible + stable + receives events + enabled
// check, uncheck, setChecked same as click
// hover, dragTo visible + stable + receives events
// screenshot visible + stable
// fill, clear visible + enabled + editable (no stable!)
// selectOption visible + enabled
// selectText visible
// scrollIntoViewIfNeeded stable
// focus, blur, press,
// pressSequentially,
// dispatchEvent,
// setInputFiles no actionability checks at all
//
// Why it matters: a cookie banner blocks the click, not the fill
await page.getByLabel('Email').fill('a@b.com'); // works under the overlay
await page.getByRole('button', { name: 'Submit' }).click(); // waits for it to clear

// Failure mode: the error names the failing check
// Error: element is not stable - waiting...
// Error: element does not receive pointer events
// Error: element is not enabled
```

Q 63 LEVEL · Mid

What is the difference between isVisible() and toBeVisible(), and why does the distinction cause subtle test bugs?

CONCEPT

isVisible() returns a boolean at the moment it is called, a one-shot, point-in-time check, no waiting. **expect(...).toBeVisible()** is an assertion that retries until the element is visible or the timeout expires, the auto-retrying assertion. The two look interchangeable but produce dramatically different behaviour. Calling isVisible() right after triggering a load typically returns false because the element hasn't appeared yet, and the test moves on, silently passing a check that should have caught a real issue. toBeVisible() polls until the element actually appears or fails after the timeout. The same trap exists for every isXxx vs toBeXxx pair: **isEnabled vs toBeEnabled**, isChecked vs toBeChecked, isHidden vs toBeHidden. Rule: assertions use toBeXxx (retrying); boolean branches use isXxx (one-shot), because in a boolean branch you genuinely want the current value, not a retried one.

INTERVIEW STRATEGY

What scores here is showing the one-shot vs retrying distinction. Lead with the difference, isVisible is point-in-time, toBeVisible polls, then make the silent-pass framing the senior signal because that's where tests lie. The simple rule (toBeXxx for assertions, isXxx for boolean branches) lands. The trap is the assertion-shaped statement that's actually a one-shot check. The follow-up is often ‘when do you actually use isVisible?’, answer boolean branches: if-statements on locator state where the current value is what matters.

How to phrase it

> *isVisible returns a boolean at the moment it's called, it's a one-shot, point-in-time check with no waiting. Expect dot to be visible is an assertion that retries until the element is visible or the timeout expires, the auto-retrying assertion. The two look interchangeable but produce dramatically different behaviour, and the framing that lands in interviews, because it's where tests silently lie, is this. If I call isVisible right after triggering a load, it typically returns false because the element hasn't appeared yet, and the test moves on. If I then assert on that boolean, expect await locator dot isVisible toBe true, the assertion passes trivially because I'm asserting on a value I read before the element appeared. The test reports green; the bug ships. ToBeVisible polls until the element actually appears or fails after the timeout, which is what I actually wanted. The same trap exists for every isXxx versus toBeXxx pair, isEnabled versus toBeEnabled, isChecked versus toBeChecked, isHidden versus toBeHidden. The rule I would close on, because it's the rule that prevents the bug, is: assertions use toBeXxx because they need retrying; boolean branches use isXxx because they genuinely want the current value, not a retried one. On the natural follow-up about when I actually use isVisible. In if-statements on locator state, where I want to fork the test based on what's currently shown, if the cookie banner is visible accept it, otherwise skip, the current value is what matters. The failure mode is the assertion-shaped statement that's actually a one-shot check; once you know to look for it, it's everywhere in early-career Playwright code.*

Key points to hit

(1) isVisible(): one-shot boolean, no waiting, current value only. **(2)** toBeVisible(): retrying assertion, polls until visible or timeout. **(3)** expect(await locator.isVisible()).toBe(true) silently passes if not yet visible. **(4)** Same pattern: isEnabled/toBeEnabled, isChecked/toBeChecked, isHidden/toBeHidden. **(5)** Rule: assertions use toBeXxx; boolean branches use isXxx. **(6)** Where this goes wrong: assertion-shaped isVisible() that passes the test before the element renders.

CODE

```
// WRONG: silently passes if element hasn't rendered yet
await page.getByRole('button', { name: 'Save' }).click();
expect(await page.locator('.toast').isVisible()).toBe(true); // ✗ one-shot before paint

// CORRECT: polls until visible or timeout
await page.getByRole('button', { name: 'Save' }).click();
await expect(page.locator('.toast')).toBeVisible(); // ✓ retrying

// isVisible() is correct for boolean branching
if (await page.getByRole('dialog', { name: 'Cookie banner' }).isVisible()) {
await page.getByRole('button', { name: 'Accept' }).click();
}
// here we WANT the current value, not a retried one
```

Q 64 LEVEL · Mid to Senior

When and how do you use waitForFunction to poll arbitrary application state?

CONCEPT

page.waitForFunction evaluates a function in the **page context** repeatedly until it returns a truthy value, or the timeout expires. It's the right tool when the state you need to wait on isn't represented in the DOM, it lives in JavaScript memory: a Redux store value, a window-level flag set by an analytics SDK, a Chart.js instance having loaded data, a service worker having registered. For those cases, no DOM-based assertion exists; waitForFunction polls JavaScript directly. The default polling interval is **requestAnimationFrame** (every paint), which is right for most cases; polling: 'mutation' waits for DOM mutations instead; polling: 100 sets an explicit millisecond interval. The discipline: reach for waitForFunction only when no DOM signal exists; if the state surfaces in the DOM (even via a data attribute or a hidden field), a normal locator with toHaveAttribute or toBeVisible is more robust because it survives JS errors.

INTERVIEW STRATEGY

This question is there to see when JS-state polling is right. Lead with 'state not in DOM' as the only justification, Redux stores, window flags, Chart.js instances, service workers. The 'reach for it only when no DOM signal exists' discipline is the senior signal. The mistake is using waitForFunction when a locator would have worked. Expect the follow-up ‘what polling interval do you use?’, answer the default (requestAnimationFrame) for most; polling: 'mutation' for DOM-tied checks; explicit ms only when paint-rate is wasteful.

How to phrase it

> *page.waitForFunction evaluates a function in the page context repeatedly until it returns a truthy value, or the timeout expires, and it's the right tool when the state I need to wait on isn't represented in the DOM, it lives in JavaScript memory. The cases I'd give as examples: a Redux store value that affects behaviour but isn't rendered, a window-level flag set by an analytics SDK when it's loaded, a Chart.js instance having received its data, a service worker having registered. For those cases there's no DOM-based assertion I can write, because the state never crosses into the DOM. The default polling interval is requestAnimationFrame, which means it polls every paint, usually about every 16 milliseconds at 60fps, and that's right for most cases because it costs almost nothing on a modern browser. Polling mutation waits for DOM mutations instead, which is useful when the check itself reads DOM state. Polling number sets an explicit millisecond interval, which I'd use when paint-rate polling is wasteful, the function does real work like JSON parsing. That's also my answer to the follow-up about polling intervals. The discipline that lands as the senior framing, though, is to reach for waitForFunction only when no DOM signal exists. If the state surfaces in the DOM at all, a data attribute, a hidden field, a class change, a normal locator with toHaveAttribute or toBeVisible is more robust because it survives JavaScript errors. If the page errors out, waitForFunction hangs until timeout; a locator still gives me the trace viewer and a clear actionability failure. The failure mode is reaching for waitForFunction when a locator would have worked, loses observability and debuggability for no real gain.*

Key points to hit

(1) Polls a function in the page context until truthy or timeout. **(2)** Right for state NOT in DOM: Redux stores, window flags, JS instances. **(3)** Default polling: requestAnimationFrame (every paint); 'mutation' for DOM-tied. **(4)** Explicit ms interval when paint-rate polling is wasteful. **(5)** Discipline: only when no DOM signal exists, locators are more robust. **(6)** What loses the point: waitForFunction when a locator would have worked, loses observability.

CODE

```
// State not in DOM: wait on a window-level analytics flag
await page.waitForFunction(
() => (window as any).__analyticsReady === true,
null,
{ timeout: 10_000 }
);

// Wait for a Redux store value
await page.waitForFunction(
() => (window as any).store.getState().cart.items.length > 0
);

// Wait for a chart instance to have rendered data
await page.waitForFunction(
() => (window as any).myChart?.data?.datasets?.[0]?.data?.length > 0,
null,
{ polling: 100 } // explicit ms when paint-rate is overkill
);

// PREFER a locator when the state IS in the DOM
await expect(page.locator('[data-loaded="true"]')).toBeVisible();
```

Q 65

LEVEL · Mid

How does page.waitForURL work, and when is it the right tool?

CONCEPT

page.waitForURL waits until the page's URL matches the given pattern, a string for exact match, a glob, or a RegExp, then returns. It's the right tool to assert that a navigation completed to the expected URL, especially after a redirect chain (OAuth flows, password resets, post-checkout redirects) where the final URL isn't known until the chain resolves. Three matching modes: string with glob like '**/dashboard?**' for path-only matching with a query string; **RegExp** for arbitrary patterns; predicate function taking a URL object for complex logic. Pair it with **expect(page).toHaveURL** for assertions, they retry the same way and produce cleaner failure messages. The discipline: waitForURL for waiting, toHaveURL for assertions. Use waitForURL when subsequent test logic depends on the navigation having completed; use toHaveURL when the destination is the thing being verified.

INTERVIEW STRATEGY

They want to see URL waiting + asserting discipline. Lead with the three matching modes (glob string, RegExp, predicate), then the waitForURL-for-waiting vs toHaveURL-for-assertions rule. Redirect-chain examples (OAuth, password reset) make it concrete. The pitfall is using string equality after a redirect chain. They usually follow with ‘what about after a click that navigates?’, answer click already auto-waits for navigation; waitForURL is for asserting on the destination after that.

How to phrase it

> *page.waitForURL waits until the page's URL matches the given pattern, a string for glob matching, a RegExp for arbitrary patterns, or a predicate function taking a URL object for complex logic, then returns. It's the right tool to assert that a navigation completed to the expected URL, especially after a redirect chain where the final URL isn't known until the chain resolves. The examples I'd give to make it concrete are OAuth flows, where the user bounces through the identity provider and back to my app on a callback URL with tokens in the query string; password resets, which land on different URLs depending on token validity; post-checkout redirects, which go to a confirmation URL with the order ID. The three matching modes worth knowing. String with glob patterns like double-star slash dashboard question-mark double-star for path-only matching with any query string. RegExp for arbitrary patterns when I need precise control. Predicate function taking a URL object when the matching logic is complex, say, asserting the query string contains a non-empty session token. The discipline I would lead with, because it lands as the senior pattern, is waitForURL for waiting and toHaveURL for assertions. They use the same matchers and the same retry behaviour, but toHaveURL produces cleaner failure messages, expected such-and-such URL, got something else, and waitForURL is meant for the moments when subsequent test logic depends on the navigation having completed. That's also my answer to the follow-up about clicks that navigate. The click itself already auto-waits for navigation, it's one of the actions that waits on the navigation event, so I don't need waitForURL just to wait. I use waitForURL or toHaveURL after the click when I want to assert specifically what URL we landed on, separate from the click's own auto-wait. The pitfall is using string equality after a redirect chain, which is fragile to query string changes.*

Key points to hit

(1) Waits until URL matches: glob string, RegExp, or predicate function. **(2)** Right for redirect chains: OAuth, password reset, post-checkout flows. **(3)** Glob example: '**/dashboard?**', path match with any query string. **(4)** Pair with toHaveURL: waitForURL for waiting, toHaveURL for assertions. **(5)** Expect next: click already auto-waits for nav; waitForURL is for assertion after. **(6)** The common miss: string equality after a redirect chain, fragile to query changes.

CODE

```
// After OAuth callback, wait for the dashboard URL with any query string
await page.getByRole('button', { name: 'Sign in with SSO' }).click();
// SSO redirect chain happens here
await page.waitForURL('**/dashboard?**');

// Predicate for complex matching
await page.waitForURL(url => {
return url.pathname === '/checkout/confirmation'
&& url.searchParams.get('orderId')?.startsWith('ord_');
});

// Assert on the URL (cleaner failure message than waitForURL)
await expect(page).toHaveURL(/\/dashboard$/);

// waitForURL for waiting, toHaveURL for assertions, same matchers, different purpose
```

Q 66

LEVEL · Mid to Senior

Why must waitForEvent be inside Promise.all with the action that triggers the event?

CONCEPT

Events fire asynchronously, and a listener registered after the event has already fired will never see it. Naive code that clicks a download button and then awaits waitForEvent('download') has a race condition: if the download event fires before the await line executes, the test hangs until timeout. Promise.all wraps the listener and the trigger together so the listener is registered before the trigger runs, the event is guaranteed to be captured. The pattern applies to every event-triggering action: **downloads**, **popups** (waitForEvent('popup')), new pages (context.waitForEvent('page')), console messages (page.waitForEvent('console')), **WebSocket frames**, specific requests (waitForRequest/waitForResponse). The principle: register the listener before doing the thing. Promise.all is the convenient way to express that registration order in async code.

INTERVIEW STRATEGY

Your job here is to show the listener-before-action race understood. Lead with the failure mode, event fires before listener registered, test hangs, then make Promise.all the structural fix. The generalisation across downloads/popups/pages/console/WebSocket is what separates junior from senior thinking. The failure mode is registering after the click. The next question is normally 'is there an alternative to Promise.all?', answer yes, register the listener with .on before the action; Promise.all is the convenience when you want a one-shot.

How to phrase it

> *Events fire asynchronously, and a listener registered after the event has already fired will never see it. That's the race condition that catches most candidates and shows up everywhere in Playwright code. The naive version: click the download button, then on the next line await waitForEvent download. If the download event fires between the click completing and the await line executing, which happens often because downloads can be fast, the listener registers too late, the event is gone, and the test hangs until the timeout expires looking for an event that already passed. Promise.all wraps the listener and the trigger together in one expression, and the JavaScript runtime starts both expressions before awaiting either, which means the listener is registered before the trigger runs, the event is guaranteed to be captured. The pattern applies to every event-triggering action, and listing them is what shows depth in interviews. Downloads via waitForEvent download. Popups via waitForEvent popup. New pages opened in the same context via context dot waitForEvent page. Console messages via waitForEvent console. WebSocket frames via page-level waitForEvent websocket then ws-level events. Specific requests via waitForRequest and waitForResponse with URL patterns. The principle I would close on, because it generalises beyond Playwright, is register the listener before doing the thing. Promise.all is the convenient way to express that registration order in async code. When they push for the follow-up about alternatives. I can register the listener with page dot on before the action and read the captured event later, which is the right pattern when I want to capture multiple events; Promise.all is the convenience for one-shot capture. The classic error is registering after the click, it compiles, runs, hangs to timeout.*

Key points to hit

(1) Events fire asynchronously: a late listener never sees the event. **(2)** Promise.all wraps listener + trigger so listener registers first. **(3)** Pattern applies to: downloads, popups, new pages, console, WebSocket, requests. **(4)** Principle: register the listener BEFORE doing the thing. **(5)** They usually ask next: page.on() before action is the alternative for multi-event capture. **(6)** Avoid: click first, listen second, test hangs to timeout silently.

CODE

```
// CORRECT: listener registered before trigger via Promise.all
const [download] = await Promise.all([
page.waitForEvent('download'),
page.getByRole('button', { name: 'Export CSV' }).click(),
]);

// Same pattern for popups (target='_blank')
const [popup] = await Promise.all([
page.waitForEvent('popup'),
page.getByRole('link', { name: 'Open report' }).click(),
]);

// Same for waiting on a specific API call during the test action
const [response] = await Promise.all([
page.waitForResponse(r => r.url().includes('/api/orders') && r.status() === 201),
page.getByRole('button', { name: 'Place Order' }).click(),
]);

// WRONG: the event may fire between the click and the await
// await page.getByRole('button', { name: 'Export CSV' }).click();
// const download = await page.waitForEvent('download'); // hangs to timeout
```

Q 67 LEVEL · Mid to Senior

What does the force option do, and when (rarely) is it appropriate?

CONCEPT

force: true on actions like click, fill, check tells Playwright to skip the actionability checks and execute the action regardless of whether the element is visible, stable, enabled, or receives events. It's an escape hatch, and it should be the **last tool you reach for**, not a default. The cases where it's legitimately useful: testing the disabled state itself, asserting that clicking a disabled button doesn't submit the form (here the disabled state is the test target, not the obstacle); working around a known browser bug where actionability incorrectly reports failure; **performance optimisation** for actions where you've already independently verified the precondition. The trade-off is loss of safety: force-clicking an element that's covered by a modal silently misses the click (or hits the modal). The discipline: if you reach for force, write a comment explaining why, because anyone reading the code six months later will assume it's wrong otherwise.

INTERVIEW STRATEGY

Show judgement about when to skip safety. Lead with what force does, then the narrow legitimate cases (testing disabled state, browser bugs, perf opt), then the strong default discipline: comment when used. The 'last tool, not default' framing is the experience marker. The thing to avoid is

reaching for force to silence actionability failures. A common follow-up is ‘what bugs could it hide?’, answer force-clicking an element covered by a modal silently misses or hits the modal, the test passes but does nothing useful.

How to phrase it

> *Force true on actions like click, fill, check tells Playwright to skip the actionability checks and execute the action regardless of whether the element is visible, stable, enabled, or receives events. It's an escape hatch, and the framing I would lead with, because senior interviewers test for judgement here, is that it should be the last tool you reach for, not a default. The cases where force is legitimately useful are narrow. First, testing the disabled state itself: if I want to assert that clicking a disabled button doesn't submit the form, I need to actually click the disabled button, and force lets me do that. The disabled state is the test target here, not the obstacle. Second, working around a known browser bug where the actionability check incorrectly reports failure for a genuinely-clickable element, rare but it happens, and the right response is to file the bug and add a force with a comment linking the issue. Third, performance optimisation for actions where I've already independently verified the precondition, in tight loops where the actionability check is a measurable cost. Those are the only three legitimate cases. The trade-off, and my answer to the follow-up about what bugs force can hide, is loss of safety. Force-clicking an element that's covered by a modal overlay silently misses the click or hits the modal instead, the test passes but does nothing useful. Force-filling a field that's readonly silently fails to set the value. The discipline I would close on is: if you reach for force, write a comment explaining why. Anyone reading the code six months later will assume it's wrong otherwise, because the default assumption from a code reader is that force was added to silence a failure rather than for a legitimate reason. The thing to avoid is reaching for force when an actionability error reports a real issue, fixing the underlying problem is almost always the right move.*

Key points to hit

(1) force: true skips all actionability checks, last resort escape hatch. **(2)** Legitimate uses: testing disabled state, browser-bug workaround, perf opt. **(3)** Trade-off: force-click on a covered element silently misses or hits overlay. **(4)** Rule: always write a comment explaining why force was needed. **(5)** The next question: covered-by-modal clicks are the classic bug force hides. **(6)** The easy mistake: reaching for force to silence actionability errors that report real bugs.

CODE

```
// LEGITIMATE: testing the disabled state itself
// Reason: we want to verify that clicking the disabled button does NOT submit.
const submit = page.getByRole('button', { name: 'Submit' });
await expect(submit).toBeDisabled();
await submit.click({ force: true }); // force needed: clicking a disabled button
await expect(page).not.toHaveURL(/success/);

// LEGITIMATE: known browser bug, link the issue
// https://github.com/microsoft/playwright/issues/12345
// Actionability incorrectly reports failure on this SVG button.
await svgIcon.click({ force: true });

// WRONG: silencing a real failure
// await submitBtn.click({ force: true }); // because the test failed without it
```

Q 68 LEVEL · Mid

How does navigation auto-waiting work when a click triggers a page transition?

CONCEPT

When you click a link or button that triggers navigation, the click waits for the navigation it started before resolving. The docs put it plainly: actions that initiate navigations wait for those navigations to happen and for pages to start loading. That is why most tests need no explicit waitForNavigation or waitForURL after a click; the click itself waits, and adding a second wait is duplicate logic that only makes the test slower and noisier. Two precisions that separate a candidate who has read the API from one who has guessed at it. First, an action takes no lifecycle option: click's options are button, clickCount, delay, force, modifiers, noWaitAfter, position, timeout and trial. The four lifecycle milestones belong to **page.goto**, which the next question covers. Second, the opt-out flag noWaitAfter is deprecated and will default to true, so Playwright is deliberately stepping back from the post-click navigation wait. The durable habit is therefore to **assert on the destination** rather than lean on the wait. Reach for **page.waitForURL** when you need the destination URL itself, when one click can trigger several navigations, or when the click is an SPA route change with no page load at all.

INTERVIEW STRATEGY

They are looking to see automatic-navigation fluency. Lead with the click-already-waits behaviour, then the discipline that follows from it: do not add waitForNavigation after a click. That is what separates junior from senior thinking. The trap here is inventing a lifecycle option on the action, because a lot of candidates assume click takes the same settings as page.goto. It does not, and saying so out loud is a quiet credibility win. Mention that noWaitAfter is deprecated and will default to true, which shows you track where the library is heading. The follow-up is often ‘when do you need an explicit waitForURL?’, answer when asserting the destination, when one click can start several navigations, or when the click is an SPA route change with no page load at all.

How to phrase it

> *When I click a link or button that triggers navigation, the click waits for the navigation it started before it resolves. The way the docs put it is that actions which initiate navigations wait for those navigations to happen and for pages to start loading. The practical implication, and the framing that lands with interviewers because most candidates don't realise it, is that I don't need an explicit waitForNavigation or waitForURL after a click that navigates. The click already waits. Adding one is duplicate logic that makes the test slower and noisier without adding safety. I'd also be precise about one thing candidates often get wrong. An action does not take a lifecycle option. Click's options are button, clickCount, delay, force, modifiers, noWaitAfter, position, timeout and trial. The lifecycle milestones are a page dot goto concern, not a click concern, so anyone reaching for one on a click is writing code that will not compile. And the direction of travel matters here. The noWaitAfter flag, which is the opt out from that post click wait, is deprecated and will default to true, so Playwright is stepping back from waiting after the action. That's why my habit is to assert on the destination rather than depend on the wait. To the follow-up about when I need an explicit waitForURL, three cases. When I'm asserting on the destination URL itself, expect page toHaveURL slash dashboard. When one click can kick off more than one navigation, a redirect chain for example, where I want to name the URL I actually care about. And when the click is an SPA route change with no real page load, where the router just swaps content, so there is no navigation to wait for and I assert on the new URL or on an element that appears after the route change.*

Key points to hit

(1) A click that navigates already waits for that navigation: no explicit wait needed. **(2)** Discipline: don't add waitForNavigation after a click, it is duplicate logic. **(3)** click's options: button, clickCount, delay, force, modifiers, noWaitAfter, position, timeout, trial. **(4)** No lifecycle option on an action: those belong to page.goto. **(5)** noWaitAfter is deprecated and will default to true, so assert on the destination. **(6)** waitForURL for: asserting the destination, multiple navigations, SPA route changes.

CODE

```
// A click that navigates already waits: no extra wait needed
await page.getByRole('link', { name: 'Dashboard' }).click();
await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
// the click waited for the navigation to start loading before resolving

// WRONG: duplicate logic, the click already waited
// await page.getByRole('link', { name: 'Reports' }).click();
// await page.waitForNavigation();

// One click, several navigations (login then redirect): name the destination
await page.getByRole('button', { name: 'Sign in' }).click();
await page.waitForURL('**/dashboard');

// SPA route change (no page load): assert on the new URL or new content
await page.getByRole('button', { name: 'Open profile' }).click();
await expect(page).toHaveURL(/\/profile$/);
```

Q 69

LEVEL · Mid

In page.goto, what does waitUntil 'load' vs 'domcontentloaded' vs 'networkidle' each wait for, and why is networkidle discouraged?

CONCEPT

page.goto resolves once a chosen lifecycle milestone is reached, set by **waitUntil**. **'domcontentloaded'** resolves the moment the HTML is parsed and the DOM is built, before images, stylesheets or fonts finish; it is the fastest and right when the test only needs the markup present. 'load' (the default) waits for the window load event, meaning the DOM *and* all sub-resources have loaded; it is the safe middle ground for a normal page. 'networkidle' waits until there have been no network requests for about 500ms, it sounds like the strongest guarantee but is the weakest in practice. The contrast that matters: load and domcontentloaded key off deterministic browser events that always fire, whereas networkidle keys off a quiet window that may never arrive. Modern apps run background polling, analytics beacons and WebSocket keepalives, so the network never goes silent and the wait drags to timeout or behaves unpredictably. That is why networkidle is discouraged: prefer domcontentloaded plus a wait on a meaningful element that signals the page is genuinely ready.

INTERVIEW STRATEGY

This is a versus question, so the interviewer wants you to contrast all three crisply and then take a clear position on networkidle. Answer it as a ladder, domcontentloaded is fastest (DOM only), load is the default (DOM plus resources), networkidle is the tempting one to avoid, then explain the why: load and domcontentloaded fire on guaranteed browser events, networkidle waits for a quiet that modern apps never deliver. The trap is recommending networkidle as the 'safest' default. The follow-up is usually 'so what do you use instead?', answer domcontentloaded plus a wait on a meaningful element.

How to phrase it

> *All three are waitUntil milestones for page.goto, and I would contrast them as a ladder from fastest to most fragile. domcontentloaded resolves the instant the HTML is parsed and the DOM is built, before images, CSS or fonts arrive, it is the fastest, and it is right when my test only needs the markup to exist. load is the default; it waits for the window load event, so the DOM and all the sub-resources have finished, which is the sensible middle ground for an ordinary page. networkidle waits until there have been no network requests for roughly half a second, and this is the one I would flag, because it sounds like the strongest guarantee and is actually the weakest. The reason sits in what each one keys off. load and domcontentloaded hang on browser lifecycle events that always fire, so they are deterministic. networkidle hangs on a quiet window, and on a modern app that window may never come, background polling, analytics beacons, WebSocket keepalives all keep the network chattering, so the wait either drags to timeout or behaves differently run to run. That is exactly why it is discouraged. So for the follow-up about what I use instead, my answer is domcontentloaded plus a wait on a meaningful element, a stats panel visible, a skeleton hidden. That targets real application readiness rather than a timing heuristic about network silence, and it is both faster and more reliable because it returns the moment the signal I actually care about appears.*

Key points to hit

(1) domcontentloaded: DOM parsed, before resources, fastest. **(2)** load (default): DOM + all sub-resources loaded, safe middle ground. **(3)** networkidle: ~500ms of no requests, strongest-sounding, weakest in practice. **(4)** Why: load/domcontentloaded fire on guaranteed events; networkidle waits on a quiet that may never come. **(5)** Discouraged because polling/analytics/WebSockets keep the network busy. **(6)** Trap: networkidle as the 'safe' default, prefer domcontentloaded + a meaningful element.

CODE

```
// domcontentloaded: fastest, DOM only
await page.goto('/article', { waitUntil: 'domcontentloaded' });

// load: the default, DOM + images/CSS/fonts
await page.goto('/dashboard'); // waitUntil: 'load'

// networkidle, discouraged: a polling SPA never goes quiet
// await page.goto('/feed', { waitUntil: 'networkidle' }); // may hang to timeout

// Preferred: fast lifecycle + a meaningful readiness signal
await page.goto('/feed', { waitUntil: 'domcontentloaded' });
await expect(page.getByRole('region', { name: 'Latest posts' })).toBeVisible();
```

Q 70 LEVEL · Junior to Mid

What is the difference between headless and headed run mode, when do you run headed, and why can a test pass in one and fail in the other?

CONCEPT

Headless runs the browser without a visible window, the rendering engine still runs, but nothing is drawn to a screen; it is the default and what CI uses, because it is faster and needs no display. Headed launches a real visible browser window you can watch, set with headless: false or **--headed**; you run it when debugging locally, to see what the test sees, often paired with **--debug** or **slowMo** to step through. The contrast that actually matters is *why behaviour can differ*. Modern Chromium headless is very close to headed, but a few real differences remain: the default viewport and device-pixel-ratio can differ, so a responsive layout shows a different breakpoint and an element that is off-screen in one mode is in view in the other; fonts and font-rendering can differ on CI, shifting element sizes and visual snapshots; timing differs because headed is slower, which can mask or expose a race condition; and some sites behave differently when they detect automation or no display. So a test that quietly depends on a real human pace, a particular screen size, or a font passes headed on your laptop and fails headless on CI, the test was relying on the environment, not just the application.

INTERVIEW STRATEGY

Define both modes quickly, headless is no window and the CI default, headed is a visible window for local debugging, then spend your time on the part that earns the marks: why a test can pass in one and fail in the other. Group the reasons, viewport and DPR, fonts, timing, automation detection, so it sounds systematic rather than a shrug. The trap is treating a headless-only failure as flakiness to

retry away; it is usually the test leaning on the environment. Expect the follow-up 'how do you debug it?', answer run headed with --debug or trace on, and pin the viewport so both modes match.

How to phrase it

> *Headless runs the browser with no visible window, the engine still renders internally, but nothing is painted to a screen. It is the default and it is what CI uses, because it is faster and needs no display. Headed launches a real visible window I can watch, with headless false or the --headed flag, and I run it when I am debugging locally and want to see what the test sees, usually with --debug or a bit of slowMo so I can follow each step. The interesting half of the question is why a test can pass in one mode and fail in the other, because that is the situation people actually hit. Modern Chromium headless is very close to headed now, but a few differences remain and I keep a short list. First, the default viewport and device pixel ratio can differ, so a responsive layout hits a different breakpoint and an element that is below the fold in one mode is in view in the other. Second, fonts and font rendering differ on a CI box that lacks the fonts my laptop has, which shifts element sizes and breaks visual snapshots. Third, timing: headed runs slower, so a race condition that is hidden by the slower pace on my machine surfaces in the faster headless run on CI, or the reverse. Fourth, some sites behave differently when they detect automation or no display. So the pattern is almost always that the test was quietly depending on the environment, a human pace, a screen size, a font, rather than on the application, and headless on CI strips that prop away. That is also my answer to the follow-up about debugging it: I rerun headed with --debug or with trace on to compare, and I pin an explicit viewport in the config so both modes render the same layout. The thing I avoid is labelling a headless-only failure as flakiness and just retrying it, that buries a real environment dependency instead of fixing it.*

Key points to hit

(1) Headless: no visible window, faster, the CI default; headed: real window for local debugging. **(2)** Run headed with --headed / headless:false, often plus --debug or slowMo to step through. **(3)** Differences that bite: viewport/DPR, fonts, timing/races, automation-or-no-display detection. **(4)** A pass-headed/fail-headless test was leaning on the environment, not the app. **(5)** Asked next: debug by rerunning headed with --debug or trace, and pin an explicit viewport. **(6)** Where this goes wrong: treating a headless-only failure as flakiness to retry away.

CODE

```
// Run modes
// npx playwright test -> headless (default, CI)
// npx playwright test --headed -> visible window
// npx playwright test --debug -> headed + inspector, step through

// Pin the environment so headed and headless render identically
export default defineConfig({
use: {
viewport: { width: 1280, height: 720 }, // same breakpoint in both modes
headless: true, // default; flip per-run with --headed
trace: 'on-first-retry', // capture why a headless run failed
},
});

// Local debugging of a headless-only failure
// npx playwright test login.spec.ts --headed --debug
```

Q 71 LEVEL · Mid to Senior

For click(), what does the default action do versus the force, trial, and noWaitAfter options, what does each bypass, and what is the risk of each?

CONCEPT

A plain **click()** is the safe path: it runs the actionability checks click uses (visible, stable, receives events, enabled; note that Playwright *scrolls* the element into view rather than checking that it already is), performs the click, and then waits for any navigation the click triggers to start before resolving. The three options each switch off a different part of that, and they are easy to confuse. force: true bypasses the actionability checks, it clicks regardless of whether the element is obscured, disabled or unstable; the risk is a silent misclick, e.g. hitting a modal overlay instead of the button while the test stays green. trial: true runs all the actionability checks but does not actually perform the click, it is a dry run that tells you whether the element *would* be clickable; the risk is assuming it changed any state, because it does nothing but check. **noWaitAfter: true** performs the click but skips the post-action wait for a triggered navigation; the risk is a **race**, where the next line runs before the click's side effect has landed. The clean mental model: force skips the checks *before*, noWaitAfter skips the wait *after*, and trial keeps the checks but skips the click *itself*. One currency caveat, and say it out loud in an interview: the docs now mark noWaitAfter as deprecated, and it will default to true, so Playwright is stepping back from that post-action wait altogether. Keep it as the conceptual “skip the after-wait” slot in this model, but do not write it in new code. For a download or a popup the current idiom is to create the waiter first, then click, then await the waiter, with no noWaitAfter anywhere.

INTERVIEW STRATEGY

These four are constantly mixed up, so the interviewer is testing whether you can place each one precisely on the timeline of a click: checks before, the click itself, the wait after. Give that mental model first, force drops the before-checks, trial keeps checks but skips the click, noWaitAfter drops the after-wait, then attach the risk to each. The trap is conflating force and noWaitAfter, or thinking trial actually clicks. They usually follow with ‘which would you ever use in real tests?’, answer trial for probing clickability, noWaitAfter for self-managed flows, and force only as a commented last

resort.

How to phrase it

> *The cleanest way I keep these straight is to lay them on the timeline of a single click. A plain click does three things in order. it runs the actionability checks that click uses, visible, stable, receives events and enabled, then it performs the click, then it waits for any navigation the click triggered to start before resolving. Each option switches off one part of that. force true drops the checks at the front, so it clicks regardless of whether the element is obscured, disabled or mid-animation; the risk is a silent misclick, the classic one is clicking a modal overlay instead of the real button while the test stays green and proves nothing. trial true keeps all the checks but does not actually perform the click, it is a dry run that tells me whether the element would be clickable right now; the risk is assuming it changed state, when by design it changes nothing. noWaitAfter true does perform the click but skips the wait afterwards for navigation or load to settle; the risk is a race, where my next line runs before the click's side effect has landed. So the model is: force skips the checks before, noWaitAfter skips the wait after, and trial keeps the checks but skips the click itself, which is also why people confuse force and noWaitAfter, since they sit on opposite ends. For the follow-up about which I would actually use, trial is genuinely handy for probing whether something is clickable before I commit, and force only as a deliberate last resort with a comment explaining why, never to silence a real actionability failure. noWaitAfter I would name but not recommend, because it is deprecated and will default to true. For a download I set up the download promise first, click normally, then await the promise, and I never pass noWaitAfter at all.*

Key points to hit

(1) Default click(): its actionability checks -> click -> wait for a triggered navigation. **(2)** force: skips the checks BEFORE, risk is a silent misclick (hits an overlay, stays green). **(3)** trial: keeps checks but does NOT click, dry run; risk is assuming it changed state. **(4)** noWaitAfter: clicks but skips the wait AFTER, risk is a race with the click's side effect. **(5)** Model: force=before, noWaitAfter=after, trial=skips the click itself. **(6)** noWaitAfter is deprecated and will default to true: name it, never write it. **(7)** Follow-up: trial to probe, force only as a commented last resort; downloads use a waiter promise.

CODE

```
// Default: checks, click, then wait for the triggered navigation to settle
await page.getByRole('button', { name: 'Save' }).click();

// force, skip the checks BEFORE; risk: silent misclick on an overlay
await page.getByRole('button', { name: 'Save' }).click({ force: true });

// trial: run checks but DON'T click; a dry run probing clickability
await page.getByRole('button', { name: 'Save' }).click({ trial: true });
// nothing changed: it only verified the element would be clickable

// noWaitAfter is DEPRECATED and will default to true. Don't write it.
// The current idiom for a download: set the waiter up, click, then await it
const downloadPromise = page.waitForEvent('download');
await page.getByRole('button', { name: 'Export' }).click();
const download = await downloadPromise;
await download.saveAs('./tmp/report.csv');
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Which actionability check stops Playwright clicking a button hidden under a cookie overlay?

A) receives events
B) stable
C) enabled
D) editable

2. Why keep the action timeout tight (around 15s) rather than setting a large global timeout?

A) Tight timeouts run tests faster
B) A large global timeout hides performance regressions
C) Playwright requires a 15s action timeout
D) It reduces memory usage

3. You need a test to wait for a loading spinner to go away. Which is correct?

A) spinner.waitFor({ state: 'hidden' })
B) page.waitForTimeout(2000)
C) expect(spinner).toBeVisible()
D) spinner.isVisible()

4. What is the difference between waitForResponse and page.route()?

A) They are identical
B) route is read-only; waitForResponse mocks
C) waitForResponse observes real traffic; route controls what the app receives
D) Only route works in CI

5. Why install page.clock before calling page.goto()?

A) goto resets the clock
B) Otherwise fastForward waits real time
C) It is required for screenshots
D) So the fake clock captures timers set during page load

Section 4 · Answer key

#	Answer	Why and where to revisit
1	A) receives events	The receives-events check detects that another element would receive the click, so Playwright waits instead of misclicking. Note that fill() does NOT run this check, which is why a cookie banner blocks a click but not a fill. *Revisit Section 4.*
2	B) A large global timeout hides performance regressions	A generous global timeout lets a slowed-down page still pass; a tight action timeout makes regressions fail loudly. *Revisit Section 4.*
3	A) spinner.waitFor({ state: 'hidden' })	waitFor({ state: 'hidden' }) waits deterministically for disappearance; a fixed sleep is flaky and slow. *Revisit Section 4.*
4	C) waitForResponse observes real traffic; route controls what the app receives	waitForResponse passively observes and validates real traffic; route actively intercepts and controls it. *Revisit Section 4.*
5	D) So the fake clock captures timers set during page load	Installing before navigation ensures timers created during load use the controllable fake clock. *Revisit Section 4.*

SECTION 05

Page Object Model in Playwright

Every framework interview includes the Page Object Model, and the Playwright twist is what interviewers listen for: locators are lazy, so they live as readonly properties, not methods, and the new keyword never appears in a test. These ten questions cover a production page class, fixture-based dependency injection, an abstract BasePage, component-level objects for tables and widgets, and returning the next page object to model a user journey.

In this section

- Locators as readonly properties (lazy): no findElement-per-method habit.
- Action methods that hide the how; tests read at the intent level.
- Fixture-based DI so the new keyword never appears in a test file.
- An abstract BasePage for shared navigation and helpers.
- Component objects (DataTable) scoped to a container for reuse.
- Returning the next page object to model the user journey fluently.

Q 72 LEVEL · Mid

How do you implement production-grade POM in Playwright TypeScript?

CONCEPT

A page object is a TypeScript class whose **locators are readonly properties** and whose **action methods are typed** and contain no test assertions. The Playwright-specific decision is that locators are **properties, not methods**: because locators are lazy and auto-retry, a locator defined once in the constructor stays correct for the life of the page, so there is no need to re-find elements on every call. Action methods describe intent and hide the steps, a login method fills the fields, clicks, and waits for the URL change, then returns. The test calls login and knows that on the next line it is on the dashboard. Keeping assertions out of the page class keeps the class reusable and the test in charge of what to verify.

INTERVIEW STRATEGY

Everyone knows POM, so the interviewer is listening for the Playwright-specific decisions and whether you understand why. Lead with locators as readonly properties because they are lazy, and contrast that with the Selenium findElement-per-method habit, that contrast is the depth signal. Where this goes wrong is describing generic POM that could be any language. The follow-up is commonly 'why properties and not methods?', answer that lazy auto-retrying locators never go stale, so a constructor-defined property works forever, unlike a stale-prone Selenium reference.

How to phrase it

> *My page objects are TypeScript classes with locators as readonly properties and typed action methods, and I keep test assertions out of them. The two decisions I always explain are the Playwright-specific ones, because generic POM could be any language. First, locators are properties, not methods, which sets up the follow-up about why: in Selenium you called findElement inside every method because references went stale, but Playwright locators are lazy and auto-retrying, so a locator I define once in the constructor keeps working forever, there is no reason to re-find it on each call, and trying to is just Selenium habit carried over. Second, action methods hide the how. My login method fills the email and password, clicks Sign in, and waits for the dashboard URL, then returns, so the test just calls login and knows that on the next line it is authenticated and on the dashboard; it does not need to know the steps. I deliberately keep assertions out of the page class so the class stays reusable across tests and the test stays in charge of what to verify, mixing assertions into the page object is a common smell that makes pages single-purpose. That separation is what keeps the framework maintainable as it grows past a handful of pages.*

Key points to hit

(1) Locators are readonly properties because they are lazy and auto-retry. **(2)** No re-finding elements per method (the Selenium habit). **(3)** Action methods describe intent and hide the steps; login() returns void. **(4)** No assertions inside page classes: the test decides what to verify. **(5)** Expect next: why properties not methods, lazy locators never go stale. **(6)** What loses the point: describing generic POM that could be any language.

CODE

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
readonly emailInput: Locator;
readonly passwordInput: Locator;
readonly loginButton: Locator;
readonly errorAlert: Locator;

constructor(readonly page: Page) {
this.emailInput = page.getByLabel('Email address');
this.passwordInput = page.getByLabel('Password');
this.loginButton = page.getByRole('button', { name: 'Sign in' });
this.errorAlert = page.getByRole('alert');
}

async navigate(): Promise<void> { await this.page.goto('/login'); }

async login(email: string, password: string): Promise<void> {
await this.emailInput.fill(email);
await this.passwordInput.fill(password);
await this.loginButton.click();
await this.page.waitForURL('/dashboard');
}
}
```

Q 73 LEVEL · Mid

How do you use fixtures for page object dependency injection?

CONCEPT

Custom **fixtures inject page objects into tests**. You extend the base test object **once** in a fixture file, registering each page object as a fixture, and every test imports test and expect **from that fixture file** rather than from the framework directly. The payoff is that the new keyword never appears in a test: the test declares what it needs in its arguments, a login page, a dashboard page, and the fixture provides a ready instance. If a page object's constructor changes, you update the one fixture and every test that uses it is untouched. Re-exporting expect from the fixture file means each test has exactly one import line. This is dependency injection applied properly: the test states its needs, the fixture supplies them, the test does not care how.

INTERVIEW STRATEGY

The point of this one is to see whether your POM is wired by hand or by the framework. State the headline, the new keyword never appears in a test, then explain the maintainability win: a constructor change touches one fixture, not fifty tests. Mentioning the single-import-line convention shows you have structured a real suite. The trap is instantiating page objects inside each test or a beforeEach. The next question is normally ‘how is this better than newing it up in beforeEach?’, answer that fixtures compose, lazy-instantiate only what a test asks for, and centralise construction.

How to phrase it

> *I inject page objects through custom fixtures. I extend the base test object once in a fixture file, registering each page object as a fixture, and then every test imports test and expect from that fixture file instead of from the framework directly. The visible result, and the headline I lead with, is that the new keyword never appears in a test file in my frameworks, a test just declares what it needs in its arguments, a login page and a dashboard page, and the fixture hands it a ready instance. The maintainability win is the real point: if the LoginPage constructor changes, I update the one fixture and all fifty tests that use it are untouched. That sets up the follow-up about how this beats newing the page up in a beforeEach: fixtures compose, they only instantiate what a given test actually asks for rather than building everything every time, and construction lives in one place instead of being duplicated across files. I also re-export expect from the fixture file, so every test has exactly one import line, which is a small thing that keeps the suite tidy at scale. That is dependency injection done properly: the test declares its needs, the fixture provides them, and the test stays focused on behaviour rather than wiring.*

Key points to hit

(1) Extend the base test once; register each page object as a fixture. **(2)** The new keyword never appears in a test. **(3)** Constructor change -> update one fixture, every test untouched. **(4)** They usually ask next: better than beforeEach, fixtures compose and lazy-instantiate. **(5)** Re-export expect so each test has one import line. **(6)** The common miss: newing up page objects inside each test or beforeEach.

CODE

```
// src/fixtures/pages.fixture.ts
import { test as base, expect } from '@playwright/test';
import { LoginPage } from '@pages/LoginPage';
import { DashboardPage } from '@pages/DashboardPage';

type PageFixtures = { loginPage: LoginPage; dashboardPage: DashboardPage };

export const test = base.extend<PageFixtures>({
loginPage: ({ page }, use) => use(new LoginPage(page)),
dashboardPage: ({ page }, use) => use(new DashboardPage(page)),
});
export { expect };

// tests/login.spec.ts: zero new keyword, one import line
import { test, expect } from '@fixtures/pages.fixture';
test('valid login redirects', async ({ loginPage, dashboardPage }) => {
await loginPage.navigate();
await loginPage.login('admin@test.com', 'Admin@123');
await expect(dashboardPage.welcomeHeading).toBeVisible();
});
```

Q 74 LEVEL · Mid

How do you build an abstract BasePage class?

CONCEPT

A **BasePage** holds the functionality common to every page, navigation, shared helpers like a dropdown selector, and concrete page classes **extend** it. Marking the class **abstract** means it cannot be instantiated directly; you can only extend it, which enforces the pattern. Shared methods that pages use but tests should not call directly are marked protected. For simple locators, modern TypeScript class-field initialisers (readonly heading = this.page.getByRole(…)) read more cleanly than assigning in the constructor. The result is one place for cross-cutting page behaviour, inherited consistently, with the compiler preventing misuse, a small but real signal of framework-design maturity.

INTERVIEW STRATEGY

This one is checking whether you use the type system deliberately or just inherit by habit. Explain why abstract and protected are intentional: abstract enforces extend-only, protected hides shared helpers from tests. Mentioning class-field initialisers as the modern style shows current TypeScript fluency. The failure mode is putting too much in BasePage so it becomes a god-class. The follow-up is sometimes 'what belongs in BasePage versus a page?', answer truly cross-cutting behaviour only (navigation, common waits), not page-specific actions.

How to phrase it

> *BasePage is where I put what every page genuinely shares, a navigate method, common helpers like selecting from a dropdown, and each concrete page extends it. I mark the class abstract on purpose, and I would call that out: it means you cannot accidentally instantiate BasePage directly, you can only extend it, which enforces the pattern at compile time. Shared methods like selectOption that pages use internally but tests should never call directly I mark protected, so the compiler keeps tests from reaching into page internals. For simple locators I use class-field initialisers, readonly welcomeHeading equals this.page.getByRole and so on, because that reads cleaner than assigning every locator in the constructor, and it shows current TypeScript. The follow-up I am ready for is what belongs in BasePage versus a page: only truly cross-cutting behaviour, navigation and common waits, goes in the base, because the trap is letting BasePage swell into a god-class that every page drags around. Kept disciplined, the result is one consistent home for cross-cutting behaviour with the type system preventing misuse, and using abstract and protected deliberately is exactly the kind of detail that signals real framework-design maturity in an interview.*

Key points to hit

(1) BasePage holds shared navigation and helpers; pages extend it. **(2)** abstract prevents direct instantiation: enforces extend-only. **(3)** protected hides shared helpers from test code. **(4)** Class-field initialisers read cleaner and show current TypeScript. **(5)** The next question: only cross-cutting behaviour belongs in BasePage. **(6)** Avoid: a swollen BasePage god-class.

CODE

```
import { Page, Locator } from '@playwright/test';

export abstract class BasePage {
constructor(protected readonly page: Page) {}

async navigate(path: string): Promise<void> {
await this.page.goto(path);
await this.page.waitForLoadState('domcontentloaded');
}

protected async selectOption(dropdown: Locator, option: string): Promise<void> {
await dropdown.click();
await this.page.getByRole('option', { name: option }).click();
}
}

export class DashboardPage extends BasePage {
readonly welcomeHeading = this.page.getByRole('heading', { level: 1 });
}
```

Q 75 LEVEL · Mid to Senior

How do you implement component-level page objects?

CONCEPT

A component object takes a **container locator** in its constructor and scopes all its internal locators to that container, so the **same class works for every instance** of that component on any page. A DataTable component, for example, exposes methods like get-row-by-text, get-cell-value, get-row-count and click-action-in-row, all relative to whichever table you handed it. A page that has a users table and an audit table simply constructs two DataTable instances with different container locators, identical methods, different scope. This collapses hundreds of lines of duplicated table-interaction code into one reusable class, and when pagination or row behaviour changes for all tables, you update the component once.

INTERVIEW STRATEGY

This separates people who have scaled a framework from people who have a few page classes. Use the DataTable example throughout: it is the canonical component-POM answer. The line that lands is ‘same class, different container locator’, and the maintainability payoff: fix table behaviour once for every table in the app. Where this goes wrong is duplicating table logic across page classes. A common follow-up is ‘what else becomes a component?’, answer modals, nav bars, date pickers, any repeated widget.

How to phrase it

> *A component object is a class that takes a container locator and scopes everything inside it to that container, so one class handles every instance of that component anywhere in the app. My standard example, and the one I would walk through, is a DataTable. It takes the table's container locator and exposes methods like get row by text, get cell value, get row count, and click an action in a given row, all relative to that container. So if a page has a users table and an audit-log table, I construct two DataTable instances with different container locators; they have identical methods but different scopes, and neither knows about the other. That is the line that matters: same class, different container locator. It turns what used to be hundreds of lines of duplicated table-handling code, often copy-pasted between page classes, into one reusable class, and that duplication is exactly the trap I am avoiding. The real win shows up on change: when pagination behaviour or row selection changes for all tables, I update DataTable once and every table in the framework gets the fix. For the follow-up about what else becomes a component, my answer is anything that repeats, modals, navigation bars, date pickers, all get the same container-scoped treatment.*

Key points to hit

(1) Component takes a container locator; scopes all internals to it. **(2)** Same class serves every instance (e.g. every table): different scope. **(3)** Collapses duplicated interaction code into one reusable class. **(4)** Behaviour change -> fix the component once, all instances updated. **(5)** Asked next: modals, nav bars, date pickers, any repeated widget. **(6)** The easy mistake: duplicating table logic across multiple page classes.

CODE

```
import { Locator } from '@playwright/test';

export class DataTable {
constructor(private readonly container: Locator) {}

rowByText(text: string): Locator {
return this.container.locator('tbody tr').filter({ hasText: text });
}
async rowCount(): Promise<number> {
return this.container.locator('tbody tr').count();
}
async clickActionInRow(rowText: string, action: string): Promise<void> {
await this.rowByText(rowText).getByRole('button', { name: action }).click();
}
}

// Same component, two tables, different containers
class UsersPage extends BasePage {
readonly usersTable = new DataTable(this.page.locator('table#users'));
readonly auditTable = new DataTable(this.page.locator('table#audit-log'));
}
```

Q 76 LEVEL · Mid to Senior

How do you handle navigation flows between page objects?

CONCEPT

Make a page action method **return the next page object** when the action causes navigation. A login method that ends on the dashboard returns a DashboardPage; a dashboard method that opens the users screen returns a UsersPage. This builds a **fluent, type-safe navigation chain** that mirrors the user journey: the test reads as log in as admin, go to users, verify users exist, with no page-object instantiation and no URL assertions scattered through the test. The page object owns the waiting and the URL knowledge. When a destination URL changes, you update the one navigation method that produces that page and every test that travels through it stays correct.

INTERVIEW STRATEGY

They are probing for test readability and whether mechanics leak into tests. Sell the readability, tests that read like user stories, and the type-safe chain where each method returns the next page. 'URL changes touch one method' is the maintainability payoff. The risk is over-applying it so navigation methods always return a page even when there is no clear next page. The follow-up is sometimes 'when does a method not return a page?', answer when the action stays on the same page, where returning void or this is clearer.

How to phrase it

> *When a page action causes navigation, I have its method return the next page object, a login method that lands on the dashboard returns a DashboardPage, and a dashboard method that opens the users screen returns a UsersPage. That gives me a fluent, type-safe navigation chain that mirrors the actual user journey, and the readability is the selling point: the test ends up reading like a user story, log in as admin, navigate to users, verify users exist, with no page-object instantiation in the test and no URL assertions scattered around. The page object owns the waiting and the knowledge of which URL it lands on. It is also genuinely type-safe, because each method's return type tells the next call exactly what is available, so the editor guides me through the journey. The maintainability benefit is the same theme as the rest of POM: when the URL for the users page changes, I update the one navigation method and every test that travels through it stays correct. For the follow-up about when a method should not return a page, if the action stays on the same page, like applying a filter, I return void or this rather than inventing a page, because forcing a return type where there is no real next page is the trap that makes the pattern feel artificial. Tests describe behaviour; the page objects absorb the mechanics.*

Key points to hit

(1) Navigation methods return the next page object, a type-safe chain. **(2)** Tests read like user stories; no instantiation, no scattered URL asserts. **(3)** The page object owns waiting and URL knowledge. **(4)** URL change -> update one method, all journeys stay correct. **(5)** Follow-up: same-page actions return void/this, not a page. **(6)** Trap: forcing a page return where there is no real next page.

CODE

```
export class LoginPage extends BasePage {
async loginAs(email: string, password: string): Promise<DashboardPage> {
await this.page.getByLabel('Email').fill(email);
await this.page.getByLabel('Password').fill(password);
await this.page.getByRole('button', { name: 'Sign in' }).click();
await this.page.waitForURL('/dashboard');
return new DashboardPage(this.page);
}
}

// Test reads like a user journey
test('admin can view all users', async ({ loginPage }) => {
const dashboard = await loginPage.loginAs('admin@test.com', 'Admin@123');
const usersPage = await dashboard.navigateToUsers();
await expect(usersPage.usersTable.rowCount()).resolves.toBeGreaterThan(0);
});
```

Q 77 LEVEL · Mid to Senior

Do you store locators as class properties or chain them inline in methods, and what's the trade-off?

CONCEPT

Two patterns with real trade-offs. **Locators as properties**: declared once in the constructor or as private readonly fields, private readonly submit = this.page.getByRole('button', { name: 'Submit' }), and referenced by name in methods. Advantages: each locator is named, the page object reads like a vocabulary of the screen, refactoring is easy because the locator definition lives in one place, and locators are constructed once rather than rebuilt on every method call. **Inline chaining**: build the locator inside the method that uses it, this.page.getByRole('table').getByRole('row', { name: user }).getByRole('button', { name: 'Delete' }).click(), no field declaration. Advantages: the locator stays close to the code using it, dynamic locators that depend on method arguments work naturally, less boilerplate. The pragmatic rule: store as properties when the locator is reused or static; chain inline when it's single-use or argument-dependent. Many POMs do both.

INTERVIEW STRATEGY

You are being asked to show locator-organisation trade-offs. Lead with the two patterns and their real advantages, properties = named vocabulary + reuse + single definition; chain = dynamic + less boilerplate. The 'both have their place' pragmatic rule is what separates junior from senior thinking. The failure mode is religious adherence to one pattern. The follow-up is often 'which do you prefer?', answer properties for reused/static, chains for dynamic/single-use, not either-or.

How to phrase it

> *Two patterns with real trade-offs, and a senior POM usually does both depending on what each locator needs. Locators as properties: declared once in the constructor or as private readonly fields like private readonly submit equals this.page.getByRole button name Submit, and referenced by name in methods that act on them. Advantages: each locator is named, the page object reads like a vocabulary of the screen, submit, cancel, emailField, rememberMeCheckbox, refactoring is easy because the locator definition lives in one place, and locators are constructed once rather than rebuilt on every method call. The named-vocabulary aspect is the one I'd emphasise because it makes the POM discoverable: reading the class tells you what's on the screen. Inline chaining: build the locator inside the method that uses it, like this.page.getByRole table getByRole row name user getByRole button name Delete dot click. Advantages: the locator stays close to the code using it, dynamic locators that depend on method arguments work naturally, the user-parameter just goes into the name field of the getByRole row call, and there's less boilerplate when the locator is used in only one method. The pragmatic rule I would close on, because it's the heuristic that lands rather than a tribal preference, is store as properties when the locator is reused or static; chain inline when it's single-use or argument-dependent. Many POMs do both, and that's correct. The submit button on a login form is a property because it's static and gets used by multiple methods. The delete-button-for-user-X locator inside a users table is inline-chained because it's parameterised by user name and is built fresh each call. That's also my answer to the follow-up about which I prefer: not either-or. Properties for reused or static locators, chains for dynamic or single-use. The pitfall is religious adherence to one pattern; storing every locator as a property produces dead vocabulary for one-shot elements, and chaining everything inline loses the named-vocabulary benefit for the static cases.*

Key points to hit

(1) Properties: named vocabulary, reuse, single definition, single construction. **(2)** Inline chains: dynamic args work naturally, less boilerplate for one-shots. **(3)** Pragmatic rule: properties for reused/static, chains for dynamic/single-use. **(4)** Many POMs do both: not religious about one pattern. **(5)** Expect next: prefer the right tool per locator, not either-or. **(6)** Where this goes wrong: storing every locator as a property, dead vocabulary for one-shots.

CODE

```
export class UsersPage {
constructor(private readonly page: Page) {}

// Static, reused: stored as properties
readonly heading = this.page.getByRole('heading', { name: 'Users' });
readonly searchInput = this.page.getByLabel('Search users');
readonly newUserBtn = this.page.getByRole('button', { name: 'New user' });
readonly table = this.page.getByRole('table');

// Dynamic, argument-dependent: inline chain inside method
async deleteUser(email: string): Promise<void> {
await this.table
.getByRole('row', { name: email })
.getByRole('button', { name: 'Delete' })
.click();
await this.page.getByRole('button', { name: 'Confirm' }).click();
}
}
```

Q 78 LEVEL · Senior

Should assertions live inside POM methods, and what's the case for both sides?

CONCEPT

Long-running debate with reasonable arguments on each side. **The case for assertions inside POMs**: tests read cleaner (loginPage.expectErrorMessage('Invalid credentials') vs the test asserting on a loginPage.errorMessage locator), assertion logic is centralised so any change to error-display markup only touches the page object, and tests focus on flow rather than UI verification details. **The case against**: assertions belong in tests because tests are where the intent lives, POMs that assert become opaque and you can't tell from the test code what's actually being verified, and reusing a POM across tests that need to assert different things forces either many assertion methods or boolean returns that defeat the purpose. The pragmatic resolution that holds up in interviews: POMs return locators or data; tests do the assertions. The exception is page-state checks the POM owns as part of its API, **loginPage.expectLoaded()** verifying the page actually loaded, because page state is internal POM knowledge, not test intent.

INTERVIEW STRATEGY

What scores here is showing the debate understood, not just one side. Walk both cases briefly, then make the pragmatic resolution (POMs return locators/data, tests assert; exception for page-state checks) the senior signal. Where this goes wrong is religious adherence to either side. Expect the follow-up 'give an example of an exception?', answer expectLoaded as a page-state check the POM owns, because page loading is internal knowledge.

How to phrase it

> *Long-running debate with reasonable arguments on each side, and the senior answer engages with both rather than picking a tribe. The case for assertions inside POMs: tests read cleaner, loginPage dot expectErrorMessage Invalid credentials reads better than the test asserting on a loginPage dot errorMessage locator directly. Assertion logic is centralised, so any change to how error messages render only touches the page object. Tests focus on flow rather than UI verification details. Real benefits, particularly for teams whose tests describe user journeys rather than UI inspections. The case against: assertions belong in tests because tests are where the intent lives. POMs that assert become opaque: you can't tell from the test code what's actually being verified. And reusing a POM across tests that need to assert different things forces either an explosion of assertion methods (expectErrorIs, expectErrorContains, expectErrorVisible, expectNoError) or boolean returns that defeat the purpose. Also real benefits, particularly for tests that verify diverse properties of the same page. The pragmatic resolution that holds up in interviews, and the framing I would lead with, is POMs return locators or data; tests do the assertions. POMs expose state the test can verify, loginPage dot errorMessage returns the locator, loginPage dot getCurrentUser returns the displayed name as a string, and tests write the expect calls because that's where the intent is. The exception is page-state checks the POM owns as part of its API, which is my answer to the follow-up: loginPage dot expectLoaded verifying the page actually loaded before the test proceeds. That's not test intent, the test doesn't care that loading happened, only that subsequent operations succeed, it's internal POM knowledge that the page is in the right state. Same for expectError as a method only when the test wants to assert any error rather than a specific one; the POM knowing what an error looks like is internal. The failure mode is religious adherence to either side; the real answer is locators-and-data out, assertions in tests, with a narrow exception for page-state.*

Key points to hit

(1) For: cleaner tests, centralised assertion logic, focus on flow. **(2)** Against: assertions are intent (test territory); POMs that assert are opaque. **(3)** Pragmatic: POMs return locators/data; tests do the assertions. **(4)** Exception: page-state checks (expectLoaded) the POM owns as internal knowledge. **(5)** They usually ask next: expectLoaded() is the exception, page state, not test intent. **(6)** What loses the point: religious adherence, the real answer balances both with the exception.

CODE

```
export class LoginPage {
constructor(private readonly page: Page) {}

readonly emailInput = this.page.getByLabel('Email');
readonly errorBanner = this.page.getByRole('alert');

// EXCEPTION: page-state check the POM owns
async expectLoaded(): Promise<void> {
await expect(this.emailInput).toBeVisible();
await expect(this.page).toHaveURL(/\/login/);
}

// PREFERRED: expose locator/data, let the test assert
// getErrorMessage(): Locator { return this.errorBanner; } // returning Locator
}

// Test does the intent assertion explicitly
test('invalid credentials shows error', async ({ loginPage }) => {
await loginPage.expectLoaded(); // page-state
await loginPage.signIn('bad@user.com', 'wrong');
await expect(loginPage.errorBanner).toHaveText('Invalid credentials'); // test intent
});
```

Q 79 LEVEL · Senior

When do you use inheritance versus composition for sharing headers, footers, and navigation across POMs?

CONCEPT

Both work; both fail in characteristic ways when misapplied. **Inheritance** (every POM extends a BasePage that owns the header and footer): clean for uniform layouts where every page genuinely has the same shell. Failure mode: when pages diverge, a checkout flow with no header, a modal that's technically a page, the inheritance hierarchy becomes a workaround tree of overrides and exceptions, and the base class accretes page-specific logic. Composition (each POM instantiates Header, Footer, NavBar as component objects via constructor): explicit dependencies, each component testable in isolation, no class hierarchy to navigate. Failure mode: more boilerplate per page object, and missing the header instantiation in one POM silently breaks header-related test code. **The rule that holds up**: inheritance for genuine is-a relationships (every Page is-a thing with a header in this app); **composition for has-a relationships** (this page has a sidebar, that page doesn't). Most modern Playwright codebases lean composition because modern web apps have less uniform shells than they used to.

INTERVIEW STRATEGY

This question is there to see the trade-off, not a dogmatic answer. Lead with inheritance's right case (uniform layouts) and its failure (divergence creates override trees). Then composition's right case (mixed pages) and its failure (boilerplate). The is-a vs has-a rule is what separates junior from senior thinking. The modern lean toward composition is the closer. Where this goes wrong is defaulting to inheritance because it feels object-oriented. They usually follow with ‘which do you prefer for new projects?’, answer composition by default; modern apps have less uniform shells.

How to phrase it

> *Both work; both fail in characteristic ways when misapplied, and the senior answer understands those failure modes rather than picking sides. Inheritance, every POM extends a BasePage that owns the header, footer, navigation, is clean for uniform layouts where every page genuinely has the same shell. Admin dashboards, internal tools, traditional multi-page apps where every screen looks structurally similar. The failure mode, and where I've seen this go wrong in real codebases, is when pages diverge. A checkout flow with no header to keep the user focused. A modal that's technically a page in the test framework's eyes but doesn't have any of the shell. Once divergence starts, the inheritance hierarchy becomes a workaround tree of overrides and exceptions, method override to hide the header, conditional logic in the base class for the special cases, and the base accretes page-specific concerns. Composition, each POM instantiates Header, Footer, NavBar as component objects via constructor, is explicit. Each component is testable in isolation, no class hierarchy to navigate, and pages that genuinely lack a header just don't instantiate one. The failure mode is more boilerplate per page object, and missing the header instantiation in one POM silently breaks header-related test code in that page. The rule that holds up in interviews, and the framing I would lead with, is inheritance for genuine is-a relationships, composition for has-a relationships. Every Page in this app is-a thing with a header? Inheritance. This page has-a sidebar and that page doesn't? Composition. The closer that lands as the senior insight, and my answer to the follow-up about new projects, is most modern Playwright codebases lean composition by default because modern web apps have less uniform shells than they used to.*
> *Single-page apps with route-specific layouts, checkout flows with custom navigation, marketing pages with no chrome, the uniform-shell assumption that justified inheritance breaks earlier and more often. The thing to avoid is defaulting to inheritance because it feels object-oriented; in 2026 test code, composition is usually the right call.*

Key points to hit

(1) Inheritance: clean for uniform layouts; fails as pages diverge. **(2)** Composition: explicit deps, isolated testability; more boilerplate. **(3)** Rule: inheritance for is-a, composition for has-a. **(4)** Modern apps have less uniform shells: composition is the default lean. **(5)** The next question: composition by default in 2026, SPAs and route-specific layouts. **(6)** The common miss: defaulting to inheritance because it feels OO, divergence breaks the hierarchy.

CODE

```
// COMPOSITION: default for modern web apps
export class DashboardPage {
readonly header: Header;
readonly sidebar: Sidebar;
readonly footer: Footer;

constructor(public readonly page: Page) {
this.header = new Header(page);
this.sidebar = new Sidebar(page);
this.footer = new Footer(page);
}
}

// Pages that lack the shell simply don't instantiate it, no override gymnastics
export class CheckoutPage {
constructor(public readonly page: Page) {}
// no header, no sidebar: by design (focused flow)
readonly steps = this.page.getByRole('list', { name: 'Checkout steps' });
}

// INHERITANCE: right when every page genuinely has the same shell
// export abstract class BasePage { protected header = new Header(this.page); ... }
// export class UsersPage extends BasePage { ... }
```

Q 80 LEVEL · Mid to Senior

When should action methods return a POM (fluent API), and when should they return void?

CONCEPT

The fluent pattern, action methods return the POM they navigate to, lets tests read as a chain: **const dashboard = await loginPage.signIn(...)**, then **const users = await dashboard.openUsers()**. Strengths: test code reads as a user journey, the type system enforces that the next operation happens on the right page (calling dashboard methods on a users page is a compile error), and the navigation flow is documented in the POM signatures. **Void return**: the test instantiates each POM explicitly. Strengths: no implicit page transitions buried in methods, easier to mix and match POMs in complex flows, and no coupling between page objects that the fluent style creates. The rule: fluent for linear navigation flows that match the intended journey (login -> dashboard -> users); void or explicit returns for divergent flows where the next page depends on logic or where the action might not navigate at all. The trap: fluent everywhere produces tightly-coupled POMs where every page knows about every other page it could reach.

INTERVIEW STRATEGY

They want to see fluent-vs-void weighed deliberately. Lead with both strengths: fluent reads as journey + type-safe nav; void = no coupling. The 'fluent for linear, void for divergent' rule is the test of seniority on this. The thing to avoid is fluent everywhere creating coupling. The next question is normally ‘how do you handle actions that might or might not navigate?’, answer return void, let the test handle the next POM explicitly based on outcome.

How to phrase it

> *The fluent pattern, where action methods return the POM they navigate to, lets tests read as a chain: const dashboard equals await loginPage dot signIn, then const users equals await dashboard dot openUsers. The strengths are real and worth listing. Test code reads as a user journey, sign in, then go to dashboard, then open users, which is exactly what a senior interviewer wants tests to communicate. The type system enforces that the next operation happens on the right page, calling dashboard methods on a users page is a compile error, because users page doesn't have those methods. And the navigation flow is documented in the POM signatures: I can read DashboardPage and see what pages it navigates to. Void return, where the test instantiates each POM explicitly, also has real strengths. No implicit page transitions buried inside methods. Easier to mix and match POMs in complex flows that don't follow a single happy path. And no coupling between page objects that the fluent style creates, the dashboard POM doesn't have to import and depend on the users POM. The rule I would lead with, because it's the heuristic that lands, is fluent for linear navigation flows that match the intended journey, void or explicit returns for divergent flows. Login arrow dashboard arrow users is linear. fluent fits, the chained reads naturally. A search action that might land on a results page or a no-results page or an error page depending on input is divergent. return void, let the test instantiate the right next POM based on what actually happened. That's also my answer to the follow-up about actions that might or might not navigate. Submit a form that stays on the same page on validation error and navigates on success, I return void; the test checks for error or navigates to the next page itself. The trap is fluent everywhere, which produces tightly-coupled POMs where every page knows about every other page it could reach. The login page imports dashboard, the dashboard imports users, the users page imports user-detail, and changing any one POM ripples through the whole import graph. Fluent where the journey is linear and intentional, void where it isn't.*

Key points to hit

(1) Fluent return: chain reads as user journey; type-safe navigation enforced. **(2)** Void return: no implicit transitions; less coupling between POMs. **(3)** Rule: fluent for linear journey-matching flows; void for divergent flows. **(4)** Submit-that-might-not-navigate is the classic void case. **(5)** Asked next: void for actions where next page depends on outcome. **(6)** Avoid: fluent everywhere, tightly-coupled import graph across POMs.

CODE

```
// FLUENT: for linear journey flows
export class LoginPage {
constructor(private readonly page: Page) {}
async signIn(email: string, password: string): Promise<DashboardPage> {
await this.page.getByLabel('Email').fill(email);
await this.page.getByLabel('Password').fill(password);
await this.page.getByRole('button', { name: 'Sign in' }).click();
return new DashboardPage(this.page);
}
}

// VOID: for divergent or might-not-navigate actions
export class SearchPage {
constructor(private readonly page: Page) {}
async submitSearch(query: string): Promise<void> {
await this.page.getByRole('searchbox').fill(query);
await this.page.getByRole('button', { name: 'Search' }).click();
// Could land on results, no-results, or error, test decides which POM next
}
}

// Test:
// const dashboard = await loginPage.signIn(...); // fluent
// await searchPage.submitSearch('xyz');
// if (await page.getByText('No results').isVisible()) {
// /* handle empty */
// } else {
// const results = new ResultsPage(page);
// }
```

Q 81 LEVEL · Senior

What is the Screenplay pattern, and when is it a better fit than POM?

CONCEPT

The Screenplay pattern (popularised by Serenity BDD, now common in Playwright via SerenityJS) models tests as Actors performing Tasks with Abilities, asking Questions. An Actor ("Anna") has Abilities (browseTheWeb, callAnApi) and performs Tasks (LogIn.as(username), AddToCart.theItem(product)) asking Questions (TheBalance.of(account), TheItems.inTheCart()). Tests read as user behaviour at a higher level of abstraction than POMs. **When it fits better than POM**: BDD-heavy teams where business analysts read tests; tests that span multiple personas in one scenario (buyer-and-seller marketplace flows); applications where the same user task happens across many screens. **When POM is still right**: most Playwright projects with engineering-only audiences, because Screenplay adds a layer of abstraction that's only valuable if the audience uses it. The rule: POM is the right default; Screenplay is a deliberate choice for teams whose test-readability requirements justify the extra machinery. Knowing it exists is enough; knowing when to choose it is the seniority marker.

INTERVIEW STRATEGY

Your job here is to show breadth without becoming an evangelist. Define Screenplay (Actor + Ability + Task + Question), name the fit cases (BDD teams, multi-actor scenarios), then make 'POM is

default, Screenplay is deliberate' the senior signal. The pitfall is recommending Screenplay for projects that don't need it. A common follow-up is ‘have you actually used Screenplay?’, answer knowing when to choose it is more important than having shipped it; default to POM.

How to phrase it

> *The Screenplay pattern, popularised by Serenity BDD and now common in Playwright via libraries like SerenityJS, models tests as actors performing tasks with abilities, asking questions. An Actor named Anna has Abilities like browseTheWeb or callAnApi, performs Tasks like LogIn dot as username or AddToCart dot theItem product, and asks Questions like TheBalance dot of account or TheItems dot inTheCart. Tests read as user behaviour at a higher level of abstraction than POMs, it's not 'click this button' or 'fill this field', it's 'Anna logs in' and 'Anna adds the laptop to her cart'. The framing that lands in interviews because it shows breadth is knowing where it fits better than POM and where POM is still right. Screenplay fits better in three cases. First, BDD-heavy teams where business analysts read tests as living documentation, the actor-task-question vocabulary maps cleanly to Gherkin Given-When-Then. Second, tests that span multiple personas in one scenario like marketplace flows with buyer and seller, because actors are first-class in Screenplay and you can have two of them in one test. Third, applications where the same user task happens across many screens, LogIn dot as Anna is one task no matter which page hosts the login. POM is still right for most Playwright projects with engineering-only audiences, because Screenplay adds a layer of abstraction that's only valuable if the audience actually uses it. If business analysts aren't reading tests, the indirection costs more than it saves. The rule I would lead with, because it converts this from a trend-following question to a judgement one, is POM is the right default; Screenplay is a deliberate choice for teams whose test-readability requirements justify the extra machinery. That's also my answer to the follow-up about whether I've used it: knowing when to choose Screenplay is more important than having shipped it. Most teams don't need it; the ones that do benefit substantially. The mistake is recommending Screenplay for projects that don't need it; you've added an actor abstraction nobody reads and the test indirection slows down maintenance for no gain.*

Key points to hit

(1) Actor + Ability + Task + Question: higher abstraction than POM. **(2)** Tests read as user behaviour: 'Anna logs in' not 'click submit'. **(3)** Fits: BDD-heavy teams, multi-persona scenarios, cross-screen tasks. **(4)** POM still right for engineering-only audiences, indirection isn't valuable. **(5)** Rule: POM default; Screenplay deliberate for teams whose audience uses it. **(6)** The easy mistake: recommending Screenplay where it isn't needed, indirection without value.

CODE

```
// Screenplay style (via @serenity-js/playwright)
import { actorCalled } from '@serenity-js/core';
import { BrowseTheWeb, LogIn } from './abilities-and-tasks';

test('Anna can see her balance', async ({ page }) => {
const Anna = actorCalled('Anna').whoCan(BrowseTheWeb.using(page));
await Anna.attemptsTo(
LogIn.as('anna@bank.com', 'Secret!1'),
);
await Anna.attemptsTo(
Ensure.that(TheBalance.ofAccount('savings'), equals(1_234.56)),
);
});

// POM equivalent (more direct, less ceremony)
test('user sees savings balance', async ({ loginPage }) => {
const dashboard = await loginPage.signIn('anna@bank.com', 'Secret!1');
await expect(dashboard.balanceOf('savings')).toHaveText('1,234.56');
});
// Same test, different audience optimisations
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Why are Playwright locators stored as readonly properties rather than re-found in each method?

A) Properties are faster to type
B) Methods cannot be async
C) TypeScript forbids methods returning locators
D) Locators are lazy and auto-retry, so a once-defined locator stays correct

2. How does fixture-based DI remove the new keyword from test files?

A) The fixture constructs the page object and injects a ready instance
B) It bans the keyword via ESLint only
C) Page objects become global variables
D) Tests share a single page object

3. What does marking BasePage abstract achieve?

A) It makes methods faster
B) It prevents direct instantiation: BasePage can only be extended
C) It enables parallel execution
D) It auto-generates locators

4. How does a reusable DataTable component object work?

A) It hardcodes one table's selectors
B) It requires a new class per table
C) It takes a container locator and scopes all internal locators to it
D) It only works with getByTestId

5. What is the benefit of a navigation method returning the next page object?

A) It speeds up navigation
B) It removes the need for waits
C) It builds a type-safe chain so tests read like the user journey
D) It disables URL assertions globally

Section 5 · Answer key

#	Answer	Why and where to revisit
1	D) Locators are lazy and auto-retry, so a once-defined locator stays correct	Because locators are lazy instructions, a property defined once works forever, no Selenium-style re-finding. *Revisit Section 5.*
2	A) The fixture constructs the page object and injects a ready instance	The fixture builds the page object and provides it as a test argument, so tests never instantiate one directly. *Revisit Section 5.*
3	B) It prevents direct instantiation: BasePage can only be extended	An abstract class cannot be instantiated directly, which enforces that pages extend BasePage rather than use it raw. *Revisit Section 5.*
4	C) It takes a container locator and scopes all internal locators to it	Passing a container locator lets the same class serve every table on every page with different scope. *Revisit Section 5.*
5	C) It builds a type-safe chain so tests read like the user journey	Returning the next page object gives a fluent, type-safe chain and keeps URL knowledge inside the page objects. *Revisit Section 5.*

SECTION 06

Network Interception and Mocking

Network interception is one of Playwright's headline capabilities, and it exists because of the persistent connection from Section 2. Interviewers want to see that you understand the four route options, that you mock at the right layer, and that you use interception for the error states real users actually hit. These sixteen questions cover page.route in depth, suite-wide mocking via a fixture, error and edge-case testing, request-payload validation, and HAR-based replay.

In this section

- page.route and its four options: fulfill, continue, abort, fetch.
- The route.fetch pattern: modify a real response without backend changes.
- Suite-wide mocking in a fixture, plus blocking analytics on every test.
- Error states (500, 401, empty) that are impossible to trigger reliably otherwise.
- Validating the request payload the UI sends from inside route.
- HAR record-and-replay for deterministic third-party integrations.

Q 82 LEVEL · Mid

How does page.route() work in depth?

CONCEPT

page.route(pattern, handler) registers an interceptor for every request matching the URL pattern; the handler receives a **Route** object with four options. **fulfill** returns a mock response without hitting the server. continue passes the request through, optionally modifying headers or body first. abort simulates a network failure. fetch makes the real request, hands you the real response to modify, and you fulfill with the modified version. That last one is the most powerful and least known: it lets you test edge cases on real data, inject an extra item, null out a field, return 101 items instead of 100, without any backend change. It is how you reproduce a UI bug that only appears on a response shape the backend will not produce on demand.

INTERVIEW STRATEGY

They need to know whether you have used interception beyond simple mocking. List the four options crisply, then spend your time on route.fetch, because that is the one that signals depth, a concrete bug story (only happened past 100 items, reproduced by injecting a 101st into the real response) beats describing the API. The mistake is only knowing fulfill. The follow-up is often 'how do you test an edge case you cannot make the real backend produce?', answer with the fetch-modify-fulfill pattern.

How to phrase it

> *page.route registers an interceptor for every request matching a URL pattern, and the handler gets a Route object with four options. fulfill returns a mock response without touching the server. continue passes the request through, optionally after I modify headers or body. abort simulates a network failure. And fetch is the interesting one, the one I would dwell on, it makes the real call, gives me the real response, lets me modify it, then I fulfill with the modified version. fetch is the most powerful and least known of the four, and it answers the follow-up that usually comes about testing an edge case you cannot make the real backend produce. It lets me test edge cases on real data without any backend change: what if the API returns an extra item, what if a field comes back null, what if there are 101 results instead of 100. I actually used this to reproduce a UI bug that only appeared when the API returned more than a hundred items, I intercepted the real response and injected a hundred-and-first item, with no backend work and no test-database setup, and the bug reproduced instantly. So the four options cover mocking, modifying, failing and augmenting, and fetch is the one I reach for when I need genuine data with one controlled twist.*

Key points to hit

(1) route(pattern, handler) intercepts every matching request. **(2)** Four options: fulfill (mock), continue (pass/modify), abort (fail), fetch (real+modify). **(3)** fetch makes the real call and lets you modify the real response. **(4)** Follow-up: an edge case the backend won't produce, fetch-modify-fulfill. **(5)** Reproduces shape-dependent bugs (e.g. only past 100 items) with no backend change. **(6)** Trap: only knowing fulfill and missing fetch.

CODE

```
// Mock
await page.route('**/api/users', route => route.fulfill({
status: 200, contentType: 'application/json',
body: JSON.stringify([{ id: 1, name: 'Alice' }]),
}));

// Modify headers and pass through
await page.route('**/api/**', route =>
route.continue({ headers: { ...route.request().headers(), 'X-Test-Run': 'true' } }));

// Fetch real response, modify, fulfill
await page.route('**/api/products', async route => {
const response = await route.fetch();
const json = await response.json();
json.push({ id: 999, name: 'Injected', price: 0 });
await route.fulfill({ response, json });
});
```

Q 83 LEVEL · Mid

How do you mock API responses for an entire test suite?

CONCEPT

Define the route handlers in a **fixture** so they apply to every test that uses it, and keep the mock payloads in **version-controlled JSON files** for maintainability. Two patterns belong here. First, mocking the core endpoints once so every test starts from a known data state. Second, **blocking analytics and tracking**, analytics, session-replay, heatmap scripts, on every test, because test runs should never appear in production analytics dashboards and skew real user data. Doing this in a fixture means no per-test configuration; every test inherits the blocks and mocks automatically. Storing mock data as JSON alongside the tests means that when the API schema changes, the mock and the tests update together in the same commit.

INTERVIEW STRATEGY

What they are checking is whether you think at suite level, not just per test. Surface two patterns: suite-wide mocking via a fixture, and blocking analytics on day one. The analytics point is the one that shows production awareness, test traffic polluting real dashboards is a mistake juniors do not think about. Where this goes wrong is repeating the same route setup in every test. Expect the follow-up ‘how do you keep mocks from drifting from the real API?’, answer that JSON mocks live beside the tests so schema changes update both in one commit.

How to phrase it

> *I put the route handlers in a fixture so they apply to every test that uses it, and I keep the mock payloads in JSON files under version control. There are two patterns I rely on, and I lead with them because they show suite-level thinking rather than per-test fiddling. The first is mocking the core endpoints once, so every test starts from a known data state instead of whatever happens to be in the environment. The second is one I add to every project on day one: blocking analytics and tracking scripts. Test runs should never show up in production analytics dashboards, they skew the real user-behaviour data, so I abort analytics, session-replay and heatmap requests in the fixture, and that is the kind of production awareness interviewers notice. Because it is a fixture, every test inherits both the mocks and the analytics blocks automatically, with no per-test setup, so I am not copy-pasting route handlers everywhere. For the follow-up about mocks drifting from the real API, my answer is that keeping the mock data in JSON next to the tests means a schema change updates the mock data and the tests together in the same commit, so the mocks never silently fall out of sync with the contract. That combination keeps the suite deterministic and keeps test noise out of production analytics.*

Key points to hit

(1) Define route handlers in a fixture: applies to every test. **(2)** Block analytics/tracking on every test so runs do not skew dashboards. **(3)** No per-test config: tests inherit mocks and blocks automatically. **(4)** Keep mock payloads in version-controlled JSON files. **(5)** Expect next: JSON mocks beside tests so schema changes update both together. **(6)** Where this goes wrong: repeating the same route setup in every test.

CODE

```
// src/fixtures/mocks.fixture.ts
import { test as base } from '@playwright/test';
import usersData from '@data/mocks/users.json';

export const test = base.extend({
page: async ({ page }, use) => {
// Never pollute production analytics with test traffic
await page.route('**/analytics/**', route => route.abort());
await page.route('**/hotjar/**', route => route.abort());
// Known data state for every test
await page.route('**/api/users', route => route.fulfill({
status: 200, contentType: 'application/json',
body: JSON.stringify(usersData),
}));
await use(page);
},
});
```

Q 84 LEVEL · Mid

How do you test error states and edge cases using route()?

CONCEPT

Error states are the **highest-value use** of route, because they are hard to trigger reliably on real infrastructure but critical to test. A **500** needs the backend broken; a 401 session expiry needs token

manipulation; an **empty state** needs all data deleted from the database. With route you produce all three in milliseconds, with no backend dependency, and assert that the UI responds correctly, the error banner appears, the app redirects to login, the empty-state message shows. These are exactly the paths that catch the bugs users actually hit, because the happy path rarely fails in production; it is the error handling that is under-tested and breaks. Interception turns untestable failure modes into fast, deterministic tests.

INTERVIEW STRATEGY

They are probing whether you test failure paths or just the happy path. Make the case that error states are where route earns its keep: impossible to trigger reliably otherwise, and where real bugs live. Three quick examples, 500, 401, empty, each with why it is hard to produce, are more persuasive than one generic line. The trap is testing only success responses. They usually follow with 'which bugs does this actually catch?', answer that the happy path rarely fails in production; it is error handling that silently breaks after a refactor.

How to phrase it

> *Error-state testing is where route gives me the most value, and I would frame it that way because the question is really about whether I test failure paths or just the happy path. These states are hard to trigger reliably on real infrastructure but they are exactly where bugs hide. Think about what each one normally requires: a 500 needs the backend to actually be broken, a 401 session expiry needs me to manipulate session tokens, and an empty state needs every record deleted from the database, none of which I can do cleanly and repeatably in a normal run. With route I produce all three in milliseconds with zero backend dependency: fulfill a 500 and assert the error banner appears, fulfill a 401 and assert the app redirects to login, return an empty array and assert the no-data message shows. That answers the follow-up about which bugs this catches: the happy path rarely fails in production, because it is exercised constantly; it is the error handling that is under-tested and quietly breaks after a refactor, and these are precisely the failures users actually hit. So I treat error and edge-case interception as a core part of the suite, not an afterthought, and I would push back in an interview on any suite that only tests success responses.*

Key points to hit

(1) Error states are hard to trigger reliably but critical to test. **(2)** 500 needs a broken backend; 401 needs token manipulation; empty needs no data. **(3)** route produces all three in ms with no backend dependency. **(4)** Assert the UI response: error banner, redirect, empty-state message. **(5)** They usually ask next: happy path rarely fails, error handling silently breaks. **(6)** What loses the point: testing only success responses.

CODE

```
test('shows error banner on 500', async ({ page }) => {
await page.route('**/api/orders', route => route.fulfill({ status: 500 }));
await page.goto('/orders');
await expect(page.getByRole('alert')).toContainText('Something went wrong');
});

test('redirects to login on session expiry', async ({ page }) => {
await page.route('**/api/**', route => route.fulfill({ status: 401 }));
await page.goto('/dashboard');
await expect(page).toHaveURL('/login');
});

test('shows empty state when no data', async ({ page }) => {
await page.route('**/api/products', route => route.fulfill({ status: 200, body: '[]'
}));
await page.goto('/products');
await expect(page.getByText('No products found')).toBeVisible();
});
```

Q 85 LEVEL · Mid

How do you validate request payloads using route()?

CONCEPT

The route handler receives the **full request**, method, headers, and body, so you can assert on the payload the UI sent before fulfilling a response. Capture the request body with **postDataJSON()**, fulfill with a success response so the flow continues, then assert that the captured request had the right fields: the correct items, the correct payment method, a defined shipping address. This bridges UI testing and API contract testing, you verify that clicking a button serialises the form into exactly the request the backend expects, without needing a real backend. It is especially valuable for complex forms where the serialisation logic is non-trivial, and it catches bugs where a frontend refactor changed a field name and broke the contract silently.

INTERVIEW STRATEGY

Show that you can test what the UI sends, not only what it shows. Frame this as the bridge between UI and contract testing, then make it concrete with the wrong-field-name-after-refactor bug, which is the catch that makes the value obvious. The trap is only ever asserting on rendered output. The follow-up is sometimes ‘how is this different from a pure API test?’, answer that it verifies the UI's serialisation, the layer a backend-only test never exercises.

How to phrase it

> *Inside a route handler I get the full request, method, headers and body, so I can assert on the payload the UI actually sent before I fulfill a response, and I would frame this as testing what the UI sends rather than only what it shows. The pattern is: capture the request body with postDataJSON, fulfill with a success response so the flow continues normally, then assert on what I captured, that the order had the right items, the right payment method, a defined shipping address. This bridges UI testing and API contract testing. For the follow-up about how this differs from a pure API test, the answer is that an API test hits the endpoint directly and never exercises the UI's serialisation logic, whereas this verifies that clicking Place Order turns the form into exactly the request the backend expects, that whole front-end layer that a backend test skips. It is especially useful for complex forms where the serialisation is non-trivial. I have caught real bugs this way: a frontend refactor renamed a field, the screen still looked perfect, but the UI was sending the wrong field name to the API, and a payload assertion in route is what flagged it before it reached production. So it is one of my favourite ways to test the contract from the UI side.*

Key points to hit

(1) The route handler exposes method, headers and body of the request. **(2)** Capture the body with postDataJSON; fulfill success, then assert the payload. **(3)** Bridges UI testing and API contract testing without a real backend. **(4)** The next question: unlike an API test, it exercises the UI's serialisation layer. **(5)** Catches refactors that silently change a sent field name. **(6)** The common miss: only asserting on rendered output, never on what the UI sends.

CODE

```
test('order creation sends correct payload', async ({ page }) => {
let captured: any;
await page.route('**/api/orders', async route => {
captured = route.request().postDataJSON();
await route.fulfill({ status: 201, body: JSON.stringify({ orderId: 'ORD-001' }) });
});

await page.goto('/checkout');
await page.getByRole('button', { name: 'Place Order' }).click();

expect(captured.items).toHaveLength(2);
expect(captured.paymentMethod).toBe('credit_card');
expect(captured.shippingAddress).toBeDefined();
});
```

Q 86 LEVEL · Mid to Senior

How do you use HAR files for network mocking?

CONCEPT

A **HAR (HTTP Archive)** file records all network traffic from a real session. **page.routeFromHAR** can **record** a HAR (with update on) by driving the flow once against the real backend, and later replay it so every matching request is served from the archive with no real network calls. You can scope replay to a URL pattern and choose what happens for unmatched requests (fall through to the

network, or fail). This is the fastest way to mock a complex flow: record once, commit the HAR, and all later runs are fully offline and deterministic. It is especially valuable for third-party integrations, payment gateways, shipping and address-validation APIs, where you cannot control the backend. The HAR also documents the exact contract the UI depends on.

INTERVIEW STRATEGY

This one rewards real experience, so make it concrete: record once, replay offline, and stress the third-party use case, payment and shipping APIs you do not control. Mentioning that the HAR doubles as contract documentation is a nice senior flourish to close on. The trap is hand-writing dozens of individual route handlers for a complex flow. The next question is normally 'how do you keep a HAR from going stale?', answer that you re-record with update on when the real contract changes, and the diff in the committed HAR shows exactly what moved.

How to phrase it

> *A HAR file is an HTTP archive that records all the network traffic from a real session, and Playwright can both record and replay it with routeFromHAR. I record once by running the flow against the real backend with the update option on, which captures every request and response, and after that, replaying the HAR serves every matching request from the archive so the test makes no real network calls. I can scope the replay to a URL pattern and decide what happens for unmatched requests, fall through to the network or fail. This is the fastest way to mock a complex flow, and the contrast I would draw is with hand-writing dozens of individual route handlers, which is the trap: record once, commit the HAR, and every later run is fully offline and deterministic. Where it really pays off is third-party integrations, payment gateways, shipping APIs, address validation, where I cannot control the backend but still need stable, repeatable behaviour. For the follow-up about keeping a HAR from going stale, my answer is that when the real contract changes I re-record with update on, and because the HAR is committed, the diff shows exactly what moved, so the archive doubles as living documentation of the contract the UI depends on, which a reviewer can read to see precisely what the front end expects from those services.*

Key points to hit

(1) HAR records all network traffic; routeFromHAR records once and replays offline. **(2)** Scope replay by URL; choose fall-through or fail for unmatched requests. **(3)** Best for third-party integrations you cannot control. **(4)** Asked next: keep it fresh by re-recording with update; the diff shows changes. **(5)** The committed HAR documents the contract the UI depends on. **(6)** Avoid: hand-writing dozens of route handlers for one complex flow.

CODE

```
// Record once against the real backend
test('record HAR', async ({ page }) => {
await page.routeFromHAR('fixtures/checkout.har', { update: true });
await page.goto('/checkout');
// complete the flow: all requests are recorded
});

// Replay offline in tests
test('checkout flow from HAR', async ({ page }) => {
await page.routeFromHAR('fixtures/checkout.har', {
url: '**/api/**',
notFound: 'fallthrough',
});
await page.goto('/checkout');
await page.getByRole('button', { name: 'Place Order' }).click();
await expect(page.getByTestId('order-confirmation')).toBeVisible();
});
```

Q 87 LEVEL · Mid to Senior

In what order do multiple route() handlers run, and how do you compose them?

CONCEPT

When multiple route() handlers match the same request, Playwright runs them in reverse order of registration (LIFO), the most recently registered handler runs first. Each handler then ends in one of two ways, and this is the distinction people get wrong. It can end the chain by calling **fulfill** (answer it), **abort** (kill it) or continue (send it straight to the network), or it can defer to the next matching handler by calling fallback. Only fallback continues the chain. The docs are explicit that continue "will immediately send the request to the network, other matching handlers won't be invoked". This LIFO behaviour enables a clean composition pattern: register a default mock at suite setup that handles the broad URL pattern, then override per-test with a more specific handler that runs first because it's more recent. The per-test handler can call **route.fallback()** to let the suite-level handler take over for non-matching cases. The principle that lands in interviews: specific-handler-first plus fallback chain is the right pattern for layered mocking. The mistake is registering handlers in the wrong order and wondering why the default always wins.

INTERVIEW STRATEGY

They are looking to see route composition under the hood. Lead with the LIFO (reverse-of-registration) order, then make the layered-mocking pattern the centrepiece, suite default + per-test specific with route.fallback() chaining. The 'specific first, fallback chain' framing is the senior signal. The thing to avoid is registering in wrong order and the default winning. A common follow-up is ‘what if you don't call fallback?’, answer the chain stops; broader handlers don't run for that request.

How to phrase it

> *When multiple route handlers match the same request, Playwright runs them in reverse order of registration, LIFO, so the most recently registered handler runs first. Each handler then ends in one of two ways, and the distinction is worth being exact about because it is commonly muddled. Three of the four verbs end the chain. fulfill answers the request, abort kills it, and continue sends it straight to the network. The docs say it plainly, continue will immediately send the request to the network and other matching handlers will not be invoked. Only fallback defers to the next matching handler. So continue is not a pass through in the chain sense, it is an exit that happens to hit the real server. The LIFO behaviour is what enables a clean composition pattern that I'd lead with because it's the practical takeaway. At suite setup I register a default mock that handles a broad URL pattern like double-star slash api double-star, returning the standard happy-path response. Then in a specific test I override with a more specific handler, say, the failure case for double-star slash api slash payment double-star, and that handler runs first because it's more recently registered. The per-test handler calls route dot fallback when it doesn't want to take a request, letting the suite-level handler take over for non-matching cases. So I get layered mocking: suite-wide defaults, per-test overrides, predictable order. That's also my answer to the follow-up about what happens without fallback. If a handler matches and does anything other than fallback, the chain stops there, broader handlers below it in the stack don't run for that request. That's correct behaviour for a deliberately-final mock, but it's also where bugs come from: I register a test-specific handler, forget to call fallback, and other tests in the suite mysteriously stop seeing their default mock because my handler ate the request. The principle I would close on, because it's the rule that prevents the bug, is specific handler first plus fallback chain is the right pattern for layered mocking. The trap is registering handlers in the wrong order, expecting the broad one to be overridden, and wondering why the default always wins.*

Key points to hit

(1) LIFO order: most recently registered handler runs first. **(2)** fulfill, abort and continue all END the chain; only fallback defers. **(3)** continue sends the request straight to the network, other handlers are skipped. **(4)** Layered pattern: suite default + per-test specific + fallback chain. **(5)** fallback() lets broader handlers handle non-matching cases. **(6)** Follow-up: no fallback means the chain stops; broader handlers skipped. **(7)** The easy mistake: wrong registration order, default mock always wins instead of override.

CODE

```
// Suite-level default: registered first, so it runs LAST (broad handler)
test.beforeAll(async ({ context }) => {
await context.route('**/api/**', route =>
route.fulfill({ status: 200, body: JSON.stringify({ ok: true }) }));
});

// Per-test override: registered later, so it runs FIRST (specific handler)
test('payment service down', async ({ page }) => {
await page.route('**/api/payment', route =>
route.fulfill({ status: 503, body: JSON.stringify({ error: 'down' }) }));
// /api/payment -> 503 (specific runs first, returns 503)
// /api/orders -> 200 (specific doesn't match, default runs)
// /api/users -> 200 (specific doesn't match, default runs)
});

// Deferring to the broader handler: fallback() is the ONLY verb that does this
await page.route('**/api/**', route => {
if (!route.request().url().includes('/payment')) return route.fallback();
return route.fulfill({ status: 503, body: '{}' });
});

// route.continue() does NOT defer: it goes straight to the network
// and no other matching handler is invoked.
```

Q 88 LEVEL · Mid

What is the difference between route.fulfill, route.continue, route.fallback, and route.abort?

CONCEPT

Four flow-control verbs, each ending the handler with a different outcome. **route.fulfill(response)**: the handler responds directly with a synthetic response, the request never hits the network, the handler chain stops. **route.continue(overrides?)**: the request proceeds to the network, optionally with modifications to URL, method, headers, or post data; chain stops. **route.fallback(overrides?)**: passes the request to the next matching handler in the LIFO chain (or to the network if no more handlers), the right verb for layered mocking. route.abort(errorCode?): fails the request with a network error, simulates connection failures, DNS errors, blocked requests. The rule of thumb: fulfill for synthetic responses, continue when modifying outgoing requests with no intention to chain, fallback when passing to the next handler, abort for failure simulations. The most-mixed-up pair is continue vs fallback, continue ends the handler chain and goes to network; fallback continues the handler chain.

INTERVIEW STRATEGY

You need to give precise distinctions. Walk through all four: fulfill (synthetic), continue (to network with optional overrides), fallback (to next handler), abort (network error). The continue-vs-fallback distinction is the senior signal because they look similar but do fundamentally different things. The pitfall is using continue when fallback was meant. The follow-up is often ‘what's the difference between continue with overrides and fallback with overrides?’, answer continue overrides apply to the network request; fallback overrides apply to what the next handler receives.

How to phrase it

> *Four flow-control verbs, each ending the handler with a different outcome. route.fulfill with a response object: the handler responds directly with a synthetic response, the request never hits the network, the handler chain stops. This is the verb for pure mocks. route.continue with optional overrides: the request proceeds to the network, optionally with modifications to URL, method, headers, or post data, and the handler chain stops. So continue means I'm done handling, let the real network take it. route.fallback with optional overrides: passes the request to the next matching handler in the LIFO chain, or to the network if no more handlers remain. This is the right verb for layered mocking, because it preserves the chain so other handlers can still run. route.abort with an optional error code: fails the request with a network error, simulating connection failures, DNS errors, or blocked requests. The most-mixed-up pair is continue versus fallback, and the framing I would emphasise because this is where senior interviewers probe is that they look similar but do fundamentally different things. Continue ends the handler chain and goes to the actual network. Fallback continues the handler chain, passing the request to the next registered route handler. If I have a suite default and a per-test specific, calling continue in the specific bypasses the suite default and hits the network; calling fallback lets the suite default run. That's also my answer to the follow-up about overrides on each. Continue's overrides apply to the actual network request, modify URL, method, headers, post data before the request goes out. Fallback's overrides apply to what the next handler sees, modify the request as the next handler in the chain perceives it, for re-routing or rewriting without actually hitting the network yet. The trap is using continue when fallback was meant; looks fine in isolation, breaks the layered mocking pattern.*

Key points to hit

(1) fulfill: synthetic response, chain stops, no network call. **(2)** continue: request goes to network (with optional overrides), chain stops. **(3)** fallback: request goes to NEXT handler in chain (or network if none). **(4)** abort: request fails with network error (DNS, blocked, etc). **(5)** Expect next: continue overrides modify the network request; fallback overrides modify what the next handler sees. **(6)** Trap: continue used when fallback was meant, breaks layered mocking.

CODE

```
await page.route('**/api/**', async route => {
const req = route.request();

// 1. Synthetic response: never touches network
if (req.url().includes('/api/auth')) {
return route.fulfill({ status: 200, body: JSON.stringify({ token: 't_mock' }) });
}

// 2. Continue to network with modifications
if (req.url().includes('/api/legacy')) {
return route.continue({ url: req.url().replace('/legacy/', '/v2/') });
}

// 3. Fail the request: simulate network error
if (req.url().includes('/api/analytics')) {
return route.abort('blockedbyclient');
}

// 4. Pass to the next handler in the chain (layered mocking)
return route.fallback();
});
```

Q 89 LEVEL · Mid

How do you conditionally mock based on URL, method, headers, or post data?

CONCEPT

A single route() handler can dispatch on any property of the incoming Request object: **request.url()** for path-based logic, **request.method()** for GET-vs-POST splits, **request.headers()** for auth-token-dependent responses, request.postData() or request.postDataJSON() for body-based dispatch. The pattern lets you keep one handler for a broad URL pattern and branch inside it: GET returns the list, POST creates and returns 201, DELETE returns 204. Combined with fallback for unmatched cases, this gives you a full REST mock in one handler. The discipline that matters: handle exactly the cases your test needs and fallback everything else, a handler that tries to be a complete API mock becomes a second implementation to maintain. Mock the minimum, fallback the rest.

INTERVIEW STRATEGY

The point of this one is to see request-property dispatch. Lead with the four dispatch dimensions, url, method, headers, post data, then the REST-in-one-handler pattern. The mock-minimum-fallback-rest discipline is the senior signal because it prevents the second-implementation anti-pattern. The mistake is building a complete API mock by hand. Expect the follow-up ‘when do you mock vs fallback?’, answer mock the cases your test needs to control; fallback everything else.

How to phrase it

> *A single route handler can dispatch on any property of the incoming Request object. The four dimensions worth knowing: request.url for path-based logic, request.method for GET-versus-POST splits, request.headers when the response depends on auth tokens or content types, request.postData or postDataJSON for body-based dispatch, useful when the same endpoint takes different shapes depending on what's being created. The pattern lets me keep one handler for a broad URL pattern and branch inside it: GET on slash api slash users returns the list, POST creates and returns 201 with the new user, DELETE on slash api slash users slash id returns 204. Combined with fallback for unmatched cases, this gives me a full REST mock in one handler that's still readable. The point I would open on, because it prevents the failure mode that comes from building too much, is handle exactly the cases your test needs and fallback everything else. A handler that tries to be a complete API mock becomes a second implementation to maintain, the real API changes, my mock drifts, tests pass against a fictional API. So mock the minimum, fallback the rest. That's also my answer to the follow-up about when to mock versus when to fallback. I mock the specific cases my test needs to control, the failure response from the payment service, the empty state from the users endpoint, the slow response from search to test the loading indicator. Everything else falls back to either the next handler in the chain or to the real service. The failure mode is building a complete API mock by hand; it always lags reality, and the tests it supports become unreliable indicators of real behaviour.*

Key points to hit

(1) Dispatch on: request.url(), method(), headers(), postData()/postDataJSON(). **(2)** One handler for broad pattern; branch by request property inside. **(3)** REST in one handler: GET list, POST create, DELETE 204, fallback rest. **(4)** Discipline: mock the minimum for this test; fallback everything else. **(5)** They usually ask next: mock what the test controls; fallback what it doesn't care about. **(6)** Where this goes wrong: building a complete API mock, drifts from reality, tests lie.

CODE

```
await page.route('**/api/users/**', async route => {
const req = route.request();

// Dispatch by method
if (req.method() === 'GET') {
return route.fulfill({ status: 200, body: JSON.stringify([{ id: 1, name: 'Alice' }]) });
}
if (req.method() === 'POST') {
const body = req.postDataJSON(); // dispatch by post data shape
if (!body?.email) {
return route.fulfill({ status: 400, body: JSON.stringify({ error: 'email required' })
});
}
return route.fulfill({ status: 201, body: JSON.stringify({ id: 99, ...body }) });
}
if (req.method() === 'DELETE') {
return route.fulfill({ status: 204 });
}

// Auth-token-dependent: check headers
if (!req.headers()['authorization']) {
return route.fulfill({ status: 401 });
}

return route.fallback(); // anything else -> next handler
});
```

Q 90 LEVEL · Mid to Senior

How do you take a real API response, modify it, and return the modified version?

CONCEPT

The pattern is fetch the real response inside the handler, modify the JSON, and call fulfill with the modified body. Use **route.fetch()** (or **page.request.fetch()**) to perform the actual network call without ending the handler, parse the response, mutate it, then return the modified version via **route.fulfill**. This lets you keep the real API live for the data that doesn't need mocking, while injecting specific scenarios into the response, a real user list with one extra synthetic admin row, a real product catalogue with one product marked out-of-stock, a real order with the total bumped to test currency formatting at extreme values. The discipline: this is for surgical test-scenario injection, not for rebuilding the response from scratch. If you find yourself replacing most of the body, you're better off with a pure mock; if you're tweaking one or two fields, fetch-modify-fulfill is the right tool.

INTERVIEW STRATEGY

You are being asked to show hybrid mocking: real data with surgical modification. Lead with the fetch -> modify -> fulfill flow. The 'surgical injection, not rebuild' framing is the senior signal because it tells you when to stop and switch to a pure mock. The classic error is rebuilding most of the response and losing the realism. They usually follow with ‘what if the API is slow?’, answer route.fetch waits for the real response, so test runtime tracks the API; for slow APIs, consider pure mocking instead.

How to phrase it

> *The pattern is fetch the real response inside the handler, modify the JSON, and call fulfill with the modified body. Inside the route handler I use route.fetch, which performs the actual network call without ending the handler, to get the real response. I parse the JSON, mutate the specific fields I need to change, and call route.fulfill with the modified body and the original status and headers. This gives me a powerful capability: keep the real API live for all the data that doesn't need mocking, while injecting specific scenarios into the response. The examples I'd give are concrete because they land. A real user list with one extra synthetic admin row, to test admin-only UI without polluting the real database. A real product catalogue with one product marked out-of-stock, to test the out-of-stock UI on a known product. A real order with the total bumped to a million dollars, to test currency formatting at extreme values. The point I would open on, because it tells me when to switch tools, is this is for surgical test-scenario injection, not for rebuilding the response from scratch. If I find myself replacing most of the body, I'm better off with a pure mock that returns exactly what I designed. If I'm tweaking one or two fields, fetch-modify-fulfill is the right tool because it keeps the response realistic for everything else. That's also my answer to the follow-up about slow APIs. route.fetch waits for the real response to come back, so test runtime tracks the actual API speed. For slow APIs, I'd consider pure mocking instead because the real-data benefit isn't worth the test runtime cost. The pitfall is rebuilding most of the response and losing the realism that was the whole point.*

Key points to hit

(1) route.fetch() gets the real response without ending the handler. **(2)** Parse, mutate fields, fulfill with modified body + original headers. **(3)** Use for surgical scenario injection: one out-of-stock item, one admin row. **(4)** Switch to pure mock when replacing most of the response, not worth the realism. **(5)** The next question: route.fetch waits for real API, slow APIs cost test runtime. **(6)** What loses the point: rebuilding most of the body via this pattern, loses the realism.

CODE

```
// Surgical modification: real user list + one extra synthetic admin row
await page.route('**/api/users', async route => {
const response = await route.fetch(); // perform the real request
const json = await response.json() as { users: User[] };

json.users.unshift({ id: 999_999, email: 'test-admin@local', isAdmin: true });

await route.fulfill({
response, // preserve original status + headers
body: JSON.stringify(json),
});
});

// Real product catalogue with one product marked out-of-stock for the test
await page.route('**/api/products/sku-42', async route => {
const r = await route.fetch();
const product = await r.json();
product.stock = 0;
await route.fulfill({ response: r, body: JSON.stringify(product) });
});
```

Q 91 LEVEL · Mid to Senior

What is the difference between context.route and page.route, and when do you use each?

CONCEPT

page.route() intercepts requests on a single Page; **context.route()** intercepts requests across all pages in the BrowserContext, including any popups or pages opened by clicks. Use **context.route** when the mock should apply to the whole test, baseline mocks for environment stubs, third-party scripts, telemetry endpoints, anything that fires from every page; use page.route when the mock is specific to one page's behaviour, a particular endpoint that only the checkout page calls. Order of evaluation: page.route handlers run before context.route handlers for that page's requests, both still following LIFO within their scope. This means page-specific mocks override context-level defaults automatically. browser.route does not exist: the highest scope is context. The discipline: context.route in fixtures for cross-cutting concerns, page.route in tests for test-specific behaviour.

INTERVIEW STRATEGY

What scores here is showing scope discipline. Lead with the page-vs-context distinction, single page vs all pages in the context including popups, then make the cross-cutting-concerns vs test-specific split explicit. The page-runs-before-context order is the depth signal. The trap is putting environment stubs in page.route and missing popup pages. The next question is normally ‘what about new windows from clicks?’, answer context.route covers them; page.route does not.

How to phrase it

> *page.route intercepts requests on a single Page object; context.route intercepts requests across all pages in the BrowserContext, including any popups or new pages opened by clicks. So they're the same machinery at different scopes. The judgement on which to use, and the framing that lands in interviews, is cross-cutting concerns versus test-specific behaviour. I use context.route in fixtures or beforeAll for the cross-cutting cases: baseline mocks for the environment, third-party scripts I want to block, telemetry endpoints, anything that fires from every page. I use page.route inside a test for the specific behaviour I'm verifying, the particular API endpoint the checkout page calls, the error response I want to inject for this scenario. The order of evaluation matters too and is worth knowing. page.route handlers run before context.route handlers for that page's requests, both still following LIFO within their own scope. This means page-specific mocks automatically override context-level defaults without me having to think about it, which is the right ergonomics: specific wins over general. One detail worth stating because it surprises people: browser.route does not exist. The highest scope is context. If I want the same mocks across multiple browser contexts, I register them in each context's setup. That's also my answer to the follow-up about new windows from clicks. When the user clicks a link that opens a new window, that window is a new Page in the same context. context.route covers it automatically; page.route does not, because page.route is scoped to the page object I registered it on. If I have third-party telemetry firing from a popup, page.route on the main page misses it; context.route catches it. The thing to avoid is putting environment stubs in page.route and missing any popup-opened pages.*

Key points to hit

(1) page.route: single Page; context.route: all pages in context (incl. popups). **(2)** Use context.route for cross-cutting: env stubs, telemetry, third-party. **(3)** Use page.route for test-specific endpoint mocks. **(4)** page.route handlers run BEFORE context.route handlers for that page. **(5)** Asked next: popups are new pages in the context, context.route catches them. **(6)** The common miss: env stubs in page.route, misses popups and new windows.

CODE

```
// In a fixture or beforeAll: cross-cutting concerns at the context
test.beforeEach(async ({ context }) => {
// Block analytics across every page in the context
await context.route('**/analytics.example.com/**', route => route.abort());
// Stub a third-party script every page loads
await context.route('**/cdn.example.com/widget.js', route =>
route.fulfill({ body: '/* stubbed */', contentType: 'application/javascript' }));
});

// Inside a test: page-specific override
test('checkout failure', async ({ page }) => {
await page.route('**/api/payment', route =>
route.fulfill({ status: 503 })); // runs BEFORE context.route
// ... test continues
});

// Note: browser.route() does NOT exist, highest scope is context
```

Q 92

LEVEL · Mid

How do you simulate slow networks, offline mode, and connection failures?

CONCEPT

Three different simulations, each with the right tool. Offline: **context.setOffline(true)** drops all network for the context; setOffline(false) restores it. Use for PWA offline-mode testing and verifying graceful degradation. Slow networks: there's no built-in throttle API but two approaches, a route() handler that introduces a setTimeout before fulfilling (works per-request), or the CDP Network.emulateNetworkConditions via **context.newCDPSession()** for proper bandwidth and latency simulation across the context (chromium only). Connection failures: **route.abort()** with an error code, 'failed' for generic failure, 'aborted' for user-cancel simulation, 'addressunreachable', 'connectionreset', 'timedout'. Each tests a different recovery path: offline tests service-worker behaviour; slow tests timeout handling and loading states; aborted tests error UI. The trap is conflating these: setOffline is not the same as a timeout, and a timeout is not the same as a 500 response.

INTERVIEW STRATEGY

This question is there to see three distinct network conditions handled with the right tool. Lead with offline (setOffline), slow (CDP throttle or setTimeout in handler), failure (abort with error code). The 'each tests a different recovery path' framing shows you know they're not interchangeable. The thing to avoid is treating offline, slow, and failed as the same thing. A common follow-up is 'which is right for testing timeout handling?', answer slow network via setTimeout in a route handler that exceeds the app's timeout.

How to phrase it

> *Three different simulations, each with the right tool, because they test different recovery paths. Offline: I use context.setOffline true to drop all network for the context, and setOffline false to restore. This is the right tool for PWA offline-mode testing and for verifying graceful degradation when the user genuinely loses connectivity. Slow networks: there isn't a built-in throttle API at the route level but two approaches work. A route handler that introduces a setTimeout before fulfilling, which works per-request and is simple to control, set a five-second delay on a specific endpoint and watch the loading state behave. Or for proper bandwidth and latency simulation across the whole context, context.newCDPSession plus Network.emulateNetworkConditions for Chromium-only configuration of download bandwidth, upload bandwidth, latency. CDP throttle is right for testing on a simulated slow 3G network across an entire flow. Connection failures: route.abort with an error code, failed for generic failure, aborted for user-cancel simulation, addressunreachable, connectionreset, timedout. Each maps to a specific browser network error category. The framing that lands in interviews, and my answer to the follow-up about which tool is right for timeout handling, is each tests a different recovery path. Offline tests service-worker behaviour and cached content. Slow tests timeout handling and loading states, so for timeout handling specifically, setTimeout in a route handler that exceeds the app's timeout is the right tool. Abort tests error UI and retry logic. The trap, and where most candidates lose marks, is conflating these. setOffline is not the same as a timeout: it returns immediately with a network error, not after the app's timeout expired. A timeout is not the same as a 500 response: a 500 came back from the server, the app got a response, just an unhappy one. Right tool for the right recovery path.*

Key points to hit

(1) Offline: context.setOffline(true), PWA service-worker testing. **(2)** Slow: setTimeout in route handler (per-request) OR CDP throttle (full context). **(3)** Failure: route.abort('failed'|'aborted'|'connectionreset'|'timedout'). **(4)** Each tests a different recovery: cache (offline), timeout/loading (slow), error UI (abort). **(5)** Follow-up: setTimeout in route handler is right for timeout-handling tests. **(6)** Avoid: conflating offline / slow / failed, they exercise different code paths.

CODE

```
// Offline mode: PWA service-worker test
await context.setOffline(true);
// ... verify cached content renders, offline indicator shows
await context.setOffline(false);

// Slow network on a specific endpoint: test loading-state and timeout
await page.route('**/api/search', async route => {
await new Promise(r => setTimeout(r, 5_000)); // 5s artificial delay
await route.continue();
});

// Full-context throttle via CDP (chromium only)
const cdp = await context.newCDPSession(page);
await cdp.send('Network.emulateNetworkConditions', {
offline: false, latency: 400, downloadThroughput: 50 * 1024, uploadThroughput: 20 *
1024,
});

// Specific connection error: test the error UI
await page.route('**/api/payment', route => route.abort('connectionreset'));
```

Q 93 LEVEL · Mid to Senior

What is routeFromHAR, and when is it the right tool for offline testing?

CONCEPT

page.routeFromHAR(harPath, options) serves responses from a previously-recorded HAR file, letting you run a complete test offline against recorded network traffic. The recording phase: run the test with **page.routeFromHAR(harPath, { update: true })** to capture all requests as they happen. The playback phase: remove the update flag, the test now serves recorded responses without touching the network. This is different from a single mock because it captures the entire response set for a session in one file, including third-party requests, CDN assets, analytics calls. The right use case: regression testing against a frozen point-in-time backend, tests that must run in air-gapped CI environments, reproducing a specific historical state like a production incident's response shape. The discipline: **HAR drifts**, the recorded responses don't update as the API evolves, so HAR-based tests need periodic re-recording or they test fictional behaviour.

INTERVIEW STRATEGY

They want to see HAR-based replay understood. Lead with the record-then-replay flow, then make 'entire session captured in one file' the differentiator from single mocks. The right use cases

(regression, air-gapped CI, historical reproduction) plus the drift warning is the senior signal. The failure mode is using HAR for live development tests where the API changes. The follow-up is often 'how do you keep HARs current?', answer periodic re-recording on a schedule, or never if the goal is reproducing a frozen state.

How to phrase it

> *page.routeFromHAR serves responses from a previously-recorded HAR file, letting me run a complete test offline against recorded network traffic. The two-phase flow: recording phase, I run the test with routeFromHAR pointing at a HAR path with the update flag set to true, which captures all requests as they actually happen against the real backend. Playback phase, I remove the update flag, and the test now serves the recorded responses without touching the network. The differentiator from a single mock, and the framing worth emphasising, is that this captures the entire response set for a session in one file, including third-party requests, CDN assets, analytics calls, everything the page fetched. So it's not 'mock this one endpoint', it's 'replay this whole session'. The right use cases follow from that. Regression testing against a frozen point-in-time backend, where I want the test to verify the UI behaves the same against a specific state of the API. Tests that must run in air-gapped CI environments with no outbound network. Reproducing a specific historical state, like the response shape that caused a production incident, so I can write a regression test for the exact conditions. The discipline, and where this approach falls down if I'm not careful, is that HARs drift. The recorded responses don't update as the API evolves, so a HAR-based test from six months ago is testing fictional behaviour the API no longer exhibits. That's also my answer to the follow-up about keeping HARs current. Periodic re-recording on a schedule, like weekly or per-release, refreshes the HAR against the current API. Or never, if the goal is specifically to reproduce a frozen state, incident-reproduction tests should preserve the recorded responses indefinitely because the historical shape is the point. The trap is using HAR for live development tests where the API changes; the HAR drifts, tests lie, and you learn nothing real.*

Key points to hit

(1) page.routeFromHAR replays a recorded session offline. **(2)** Two phases: record with { update: true }, then replay. **(3)** Captures the entire session: APIs, CDN, third-party, analytics. **(4)** Use cases: frozen regression, air-gapped CI, historical incident reproduction. **(5)** Expect next: re-record on a schedule, or never (for frozen incident repros). **(6)** The easy mistake: HARs drift, stale HAR = tests against fictional behaviour.

CODE

```
// Record phase: run once against the real backend
await page.routeFromHAR('fixtures/checkout-flow.har', {
update: true, // capture this session into the HAR
updateMode: 'full',
updateContent: 'embed',
});
// ... run the test: all requests recorded into checkout-flow.har

// Playback phase: offline, deterministic
await page.routeFromHAR('fixtures/checkout-flow.har');
// All recorded responses served from disk; no network calls

// Combine: replay HAR by default, override specific request for a scenario
await page.routeFromHAR('fixtures/checkout-flow.har');
await page.route('**/api/payment', route => // overrides recorded response
route.fulfill({ status: 503 }));
```

Q 94 LEVEL · Senior

How do you mock GraphQL operations specifically, given they all hit the same endpoint?

CONCEPT

GraphQL is a single endpoint (typically /graphql) that dispatches based on the **operationName** and query in the POST body, so URL-based routing doesn't work, every operation looks identical to page.route at the URL level. The pattern: register one handler on the GraphQL endpoint and dispatch on **request.postDataJSON().operationName**, with a switch or lookup table per operation. Mock GetUsers, mock CreateOrder, fallback for anything else. For operations with variables (typed query parameters), inspect **postDataJSON().variables** too: return different responses based on the variables. The discipline that matters: match operationName explicitly, fallback the rest. A handler that tries to mock every GraphQL operation becomes a second GraphQL server you have to maintain. The advantage GraphQL has over REST for this: every operation is named, so the dispatch logic is explicit and discoverable in the test.

INTERVIEW STRATEGY

Your job here is to show GraphQL-specific routing. Lead with the single-endpoint dispatch problem, every operation hits /graphql, then the operationName-based switch as the solution. Variable-based dispatch is the depth signal. The discipline (match specific ops, fallback the rest) is the senior framing. The pitfall is trying to URL-route GraphQL. Expect the follow-up 'how do you handle GraphQL subscriptions?', answer they use WebSocket, not HTTP, different mocking story entirely.

How to phrase it

> *GraphQL is a single endpoint, typically slash graphql, that dispatches based on the operationName and the query in the POST body, so URL-based routing doesn't work for it, every operation looks identical to page.route at the URL level. The pattern that handles this cleanly, and the one I would lead with, is register one handler on the GraphQL endpoint and dispatch on request.postDataJSON dot operationName, with a switch statement or a lookup table per operation. So I mock GetUsers, I mock CreateOrder, I fallback for anything else. The depth signal that distinguishes a strong answer is operations with variables. GraphQL queries can take typed parameters, the variables object, and different variable values often warrant different responses. So I inspect postDataJSON dot variables too: GetUser with user ID one returns admin, GetUser with user ID two returns regular user. The discipline I would close on, because it's the same rule as REST mocking: match operationName explicitly, fallback the rest. A handler that tries to mock every GraphQL operation becomes a second GraphQL server I have to maintain, drifts from the real schema, and the tests become unreliable. The advantage GraphQL has over REST for this kind of mocking, which is worth stating because it's a real argument: every operation is named, so the dispatch logic is explicit and discoverable in the test. I can read a test and see exactly which operations are mocked, which isn't true of URL-based REST mocks. That's also my answer to the follow-up about GraphQL subscriptions. Subscriptions use WebSocket, not HTTP, so they don't flow through page.route at all, different mocking story, page.routeWebSocket or page.on websocket for frame interception. The mistake is trying to URL-route GraphQL the way you would REST; one URL doesn't tell you which operation, so you'd be matching everything as the same thing.*

Key points to hit

(1) Single endpoint /graphql: URL routing useless; dispatch on operationName. **(2)** Switch on postDataJSON().operationName: mock specific ops, fallback rest. **(3)** Variable-based dispatch via postDataJSON().variables for parametrised ops. **(4)** GraphQL's advantage: operations named explicitly, dispatch is discoverable. **(5)** They usually ask next: subscriptions use WebSocket, page.routeWebSocket, not page.route. **(6)** Trap: URL-routing GraphQL, every operation hits the same URL.

CODE

```
await page.route('**/graphql', async route => {
const body = route.request().postDataJSON() as {
operationName: string;
variables?: Record<string, unknown>;
};

switch (body.operationName) {
case 'GetUsers':
return route.fulfill({
status: 200,
body: JSON.stringify({ data: { users: [{ id: 1, name: 'Alice' }] } }),
});

case 'GetUser':
// Variable-based dispatch
const userId = body.variables?.id;
const user = userId === '1'
? { id: 1, name: 'Alice', isAdmin: true }
: { id: userId, name: 'User', isAdmin: false };
return route.fulfill({ status: 200, body: JSON.stringify({ data: { user } }) });

default:
return route.fallback(); // let real backend handle the rest
}
});
```

Q 95 LEVEL · Senior

What is the right strategy for what to mock and what not to mock?

CONCEPT

The rule that scales: mock external dependencies, never mock the system under test. External dependencies are services you don't control, third-party APIs (Stripe, Twilio, SendGrid), external auth providers, your CDN, anything your team didn't write. These get mocked because: rate limits and cost, deterministic test runtime, PCI and compliance, and the ability to test specific failure responses on demand. The system under test, your application's own APIs, your own database, your own services, stays live because that's the integration you're verifying. Mocking your own backend produces tests that pass while production breaks, the worst possible failure mode. The exceptions worth knowing: **cross-team service boundaries** within the same company can be mocked if the other team has their own test suite and contract; **slow internal services in unit-level tests** can be mocked for speed but should be live in integration tests; **destructive operations** (mass deletes, production-grade emails) can be mocked even when internal, for safety.

INTERVIEW STRATEGY

Show judgement about what to mock, not just how. Lead with the central rule, mock external, not the SUT, then list the external dependencies (third-party, CDN, auth) with reasons (rate limits, cost, PCI). Mocking-your-own-backend-produces-passing-tests-that-break-prod marks senior judgement. The three exceptions show calibration. The risk is extensive mocking that loses integration confidence. They usually follow with ‘when do you mock your own service?’, answer for speed at unit level, never at integration level.

How to phrase it

> *The rule that scales, and the one I would lead with because it's the heuristic senior interviewers want to hear, is mock external dependencies, never mock the system under test. External dependencies are services I don't control, third-party APIs like Stripe, Twilio, SendGrid, external auth providers, my CDN, anything my team didn't write. These get mocked for four reasons that compound: rate limits and cost, especially for paid services; deterministic test runtime, because third-party latency is unpredictable; PCI and compliance concerns for anything touching payment data; and the ability to test specific failure responses on demand, which is impossible against a real third party. The system under test, my application's own APIs, my own database, my own services, stays live because that's the integration I'm actually verifying. Mocking my own backend produces tests that pass while production breaks, which is the worst possible failure mode because everything looks green and the bug ships. The three exceptions worth knowing, because senior interviewers probe the edges, are cross-team service boundaries within the same company, if another team owns a service with their own test suite and a contract between us, I can mock at the boundary; slow internal services in unit-level tests can be mocked for speed, but they should be live in integration tests, which is my answer to the follow-up about when I mock my own service; and destructive operations like mass deletes or production-grade emails can be mocked even when internal, for safety reasons. The thing to avoid is extensive mocking that loses integration confidence, the tests pass, the components haven't actually been wired together, and the first user finds the integration bug. Mock where the risk-to-value ratio favours it; live where it doesn't.*

Key points to hit

(1) Rule: mock external dependencies, never the system under test. **(2)** External: third-party APIs, CDN, auth providers, rate limits, cost, PCI. **(3)** Mocking your own backend hides bugs that break prod. **(4)** Three exceptions: cross-team contracts, slow internal at unit level, destructive ops. **(5)** The next question: mock own service for speed at unit level, never at integration level. **(6)** Where this goes wrong: extensive mocking loses integration confidence, prod breaks anyway.

CODE

```
// CORRECT: mock external third-party (Stripe), keep our backend live
await context.route('**/api.stripe.com/**', route =>
route.fulfill({ status: 200, body: JSON.stringify({ id: 'pi_mock', status: 'succeeded'
}) }));

// Our own /api/checkout is LIVE: we're verifying the integration
// await page.route('**/api/checkout', ...) // ✗ do NOT mock the SUT

// EXCEPTION: destructive op, mock for safety even when internal
await context.route('**/api/admin/delete-all-users', route =>
route.fulfill({ status: 200, body: JSON.stringify({ deleted: 0 }) }));

// EXCEPTION: cross-team service with a contract
await context.route('**/billing-service.internal/**', route =>
route.fulfill({ status: 200, body: mockBillingContract }));
```

Q 96

LEVEL · Mid

page.route (intercept/mock) vs APIRequestContext (direct API call), when do you mock the network vs hit the API directly?

CONCEPT

These two tools sit on opposite sides of the browser. **page.route** intercepts the traffic the browser itself generates, the requests your front end fires as the user clicks around, and lets you mock, modify, or block them. It is a UI-side tool: the application is driving, and you are shaping what the network gives back to it. **APIRequestContext** (exposed as request in tests, or **page.request** / context.request) is the opposite, it makes a real HTTP call yourself, with no browser and no rendering, sharing the context's cookies and storage state. You use page.route when you want to control what the UI receives, mock a 500, inject an edge-case payload, block analytics. You use APIRequestContext when you want to talk to the backend directly, seed a user before a UI test, assert an API contract without a browser, or tear down test data afterwards. The two are complementary, not competing: a mature suite uses APIRequestContext to arrange state fast and page.route to control the network the UI sees during the act. The line that lands: mock when you are testing the front end's reaction to a response; call the API directly when the API is the thing you are testing, or when you are setting up and cleaning up around a UI test.

INTERVIEW STRATEGY

This is a contrast question, so the interviewer wants to hear that you know these are different layers, not two ways to do the same job. Answer crisply by naming the axis first, page.route shapes traffic the browser generates, APIRequestContext makes a real call with no browser, then give the one-line decision rule before any detail. Lead with 'mock when you are testing the UI's reaction to a response; call the API directly when the API itself is under test or when you are seeding and tearing down state'. The senior move is to say they are complementary and sketch the arrange-with-API, control-with-route pattern. The trap is treating them as rivals, or reaching for page.route to set up data when a direct call is faster and clearer. The next question is normally ‘how do you seed a logged-in user without clicking through login?’, answer that you POST to the auth endpoint via request and reuse the storage state.

How to phrase it

> *These two live on opposite sides of the browser, and I would frame it that way first because the question is really testing whether I see them as different layers. page.route intercepts the traffic the browser itself generates, the requests my front end fires as the user clicks around, and lets me mock, modify or block them. It is a UI-side tool: the application is driving, and I am shaping what the network hands back to it. APIRequestContext, which I get as request in a test or page.request, is the opposite: it makes a real HTTP call myself, with no browser and no rendering, sharing the context's cookies and storage state. So the decision rule I would give straight away is this, I mock with page.route when I am testing how the front end reacts to a response, and I call the API directly with request when the API itself is the thing under test, or when I am setting up and cleaning up around a UI test. Concrete examples land best. If I want to check that the UI shows an error banner on a 500, or that it handles an empty list, or that test traffic never reaches analytics, that is page.route, I am controlling what the UI receives. If I want to seed a user before the test, assert an API contract without spinning up a browser, or delete the test data afterwards, that is APIRequestContext, I am talking to the backend directly and fast. The senior point, and the one I would close on, is that these are complementary, not competing. In a real suite I use request to arrange state quickly, create the account, log in via the auth endpoint, capture the storage state, and then page.route to control the network the UI sees during the actual test. That answers the usual follow-up about seeding a logged-in user without clicking through the login form: I POST to the auth endpoint through request, save the storage state, and reuse it, so the UI test starts already authenticated. The trap is treating them as rivals, or reaching for page.route to set up data, setup belongs on the direct API, mocking belongs on the browser traffic, and a good suite uses both deliberately.*

Key points to hit

(1) page.route shapes traffic the browser generates; APIRequestContext makes a real call with no browser. **(2)** Mock with page.route when testing the UI's reaction to a response. **(3)** Use request/page.request when the API itself is under test, or to seed and tear down state. **(4)** They are complementary: arrange state via API, control the UI's network via route. **(5)** Asked next: seed a logged-in user by POSTing to auth via request and reusing storage state. **(6)** What loses the point: treating them as rivals, or using page.route for setup that a direct call does faster.

CODE

```
// page.route: control what the UI receives (testing the front end's reaction)
test('shows error banner on 500', async ({ page }) => {
await page.route('**/api/orders', route => route.fulfill({ status: 500 }));
await page.goto('/orders');
await expect(page.getByRole('alert')).toBeVisible();
});

// APIRequestContext: talk to the backend directly (no browser)
test('order API contract', async ({ request }) => {
const res = await request.post('/api/orders', { data: { sku: 'A-1', qty: 2 } });
expect(res.status()).toBe(201);
expect((await res.json()).orderId).toBeDefined();
});

// Combined: arrange state fast via API, then drive the UI
test('dashboard for a seeded user', async ({ page, request }) => {
// Seed + log in directly: no clicking through the login form
await request.post('/api/test/users', { data: { email: 'sdet@local', plan: 'pro' } });
const login = await request.post('/api/auth/login',
{ data: { email: 'sdet@local', password: 'pw' } });
expect(login.ok()).toBeTruthy();

// Control the UI's network during the act
await page.route('**/api/usage', route =>
route.fulfill({ status: 200, body: JSON.stringify({ used: 0, limit: 100 }) }));
await page.goto('/dashboard');
await expect(page.getByText('Pro plan')).toBeVisible();
});
```

Q 97 LEVEL · Mid

How do you block images, CSS, fonts, and third-party resources with page.route to speed up tests, and what are the risks?

CONCEPT

Most of a page's network cost is not the data your test cares about, it is images, fonts, stylesheets, and third-party widgets. You can abort these inside a route handler to cut load time, dispatching on **route.request().resourceType()**, which reports values like **image**, **stylesheet**, font, media and script. A common pattern is one handler at context level that aborts the heavy resource types and lets everything else through, so the whole suite loads only the HTML and the API calls that actually drive behaviour. The speed-up is real on image-heavy pages and on slow CI. But the risks are equally real and you must name them. Blocking CSS breaks any visual or layout assertion, screenshots, visual regression, and anything that depends on an element being visible can fail, because Playwright's visibility checks consider layout. Blocking images breaks tests that assert on image loading, alt-driven behaviour, or lazy-load triggers. Blocking third-party scripts can break the app itself if a widget you killed was actually wired into the page. And it makes the test environment less like production, so a bug that only appears with real rendering can slip through. The discipline: block only on tests where rendering is irrelevant, pure functional and API-driven flows, and never on visual, screenshot, or layout-sensitive tests. Blocking analytics and trackers is almost always safe; blocking CSS almost never is.

INTERVIEW STRATEGY

This is not a versus question, so the interviewer wants the technique plus honest judgement about its cost. Lead with the how, abort by resourceType in a context-level handler, then spend most of your answer on the risks, because volunteering the downsides is the senior signal here. State the speed-up plainly, then the sharp warnings: blocking CSS breaks visibility and layout assertions, blocking images breaks image-loading tests, blocking third-party scripts can break the app, and the whole thing drifts the environment away from production. Close on the rule: block on functional and API-driven tests, never on visual or screenshot tests; analytics is safe to block, CSS is not. The trap is blocking globally to save time and then chasing flaky visibility failures. A common follow-up is ‘why did your element-visible check start failing after you added this?’, answer that Playwright's visibility considers layout, so killing CSS can make a present element read as not visible.

How to phrase it

> *The starting point is that most of a page's network cost is not the data my test cares about, it is images, fonts, stylesheets and third-party widgets. So to speed tests up I abort those inside a route handler, dispatching on route.request().resourceType(), which tells me whether a request is an image, a stylesheet, a font, media or a script. The pattern I use is one handler at context level that aborts the heavy resource types and lets everything else through, so the whole suite loads only the HTML and the API calls that actually drive behaviour. On image-heavy pages and on slow CI that is a genuine speed-up. But I would not stop there, because the risks are the real test of whether I understand this, and I would volunteer them. Blocking CSS is the dangerous one: Playwright's visibility checks consider layout, so if I kill the stylesheet an element that is actually present can read as not visible, and any screenshot or visual-regression test is meaningless without styles. Blocking images breaks tests that assert on image loading, alt-driven behaviour or lazy-load triggers. Blocking third-party scripts can break the app itself if a widget I killed was wired into the page rather than just decorative. And more broadly, every resource I block makes the environment a little less like production, so a bug that only shows up with real rendering can slip through. That answers the follow-up I usually get, why an element-visible check started failing after I added the blocking, it is the layout point: no CSS, no layout, so visibility goes false. So my discipline is to block only where rendering is irrelevant: pure functional and API-driven flows, where I care about behaviour and not appearance. I never block on visual, screenshot or layout-sensitive tests. And the simple heuristic I would leave them with is that blocking analytics and trackers is almost always safe, I do it on every project, but blocking CSS almost never is. The trap is switching it on globally to shave seconds and then spending far longer chasing flaky visibility failures it caused.*

Key points to hit

(1) Abort heavy requests by route.request().resourceType(): image, stylesheet, font, media, script. **(2)** One context-level handler blocks the heavy types and lets HTML + API through. **(3)** Real speed-up on image-heavy pages and slow CI. **(4)** Risk: blocking CSS breaks visibility/layout/screenshot assertions; images break image-load tests. **(5)** Risk: killing a wired-in third-party script can break the app; environment drifts from prod. **(6)** Rule: block on functional/API tests only, never visual; analytics safe, CSS not.

CODE

```
// Context-level: block heavy resource types for the whole suite
const BLOCKED = new Set(['image', 'stylesheet', 'font', 'media']);

test.beforeEach(async ({ context }) => {
await context.route('**/*', route => {
const type = route.request().resourceType();
return BLOCKED.has(type) ? route.abort() : route.continue();
});
});

// Almost always safe: block analytics / trackers by URL
await context.route('**/{analytics,gtm,hotjar,segment}.**', route => route.abort());

// DANGER, do NOT block CSS on a visual / layout-sensitive test:
// no stylesheet -> no layout -> toBeVisible() can fail on a present element,
// and any screenshot assertion is meaningless. Keep blocking off here.
test('checkout layout @visual', async ({ page }) => {
// (no resource blocking in this test)
await page.goto('/checkout');
await expect(page).toHaveScreenshot('checkout.png');
});
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Which route option makes the real API call and lets you modify the real response?

A) fulfill
B) continue
C) fetch
D) abort

2. Why block analytics requests in a shared fixture?

A) Analytics slow tests down
B) It is required for sharding
C) Analytics break auto-waiting
D) Test traffic should not pollute production analytics dashboards

3. Why is route() the best tool for testing a 401 session-expiry redirect?

A) It is faster than real login
B) A 401 is hard to trigger reliably without manipulating session tokens
C) Playwright cannot test real 401s
D) It avoids using assertions

4. How do you assert the payload the UI sent inside a route handler?

A) route.response().json()
B) page.content()
C) route.request().postDataJSON()
D) route.fulfill().body()

5. What does page.routeFromHAR let you do after recording a session?

A) Replay all matching requests offline from the archive
B) Generate load tests
C) Convert HAR to JUnit
D) Record video of the run

Section 6 · Answer key

#	Answer	Why and where to revisit
1	C) fetch	route.fetch performs the real request so you can alter the genuine response before fulfilling it. *Revisit Section 6.*
2	D) Test traffic should not pollute production analytics dashboards	Aborting analytics keeps automated test runs from skewing real user-behaviour data in production dashboards. *Revisit Section 6.*
3	B) A 401 is hard to trigger reliably without manipulating session tokens	Forcing a 401 with route reproduces session expiry deterministically without token manipulation on the backend. *Revisit Section 6.*
4	C) route.request().postDataJSON()	route.request().postDataJSON() exposes the request body so you can assert the UI serialised the form correctly. *Revisit Section 6.*
5	A) Replay all matching requests offline from the archive	A recorded HAR is replayed to serve matching requests with no real network calls, giving deterministic flows. *Revisit Section 6.*

SECTION 07

API Testing with Playwright

Playwright ships a full HTTP client, so API and UI tests live in one framework, one runner, one report, and interviewers want to see you exploit that rather than bolt on Axios or Supertest. These eighteen questions cover the request fixture, authenticating API calls by reusing browser auth state, type-safe contract testing, the API-setup-plus-UI-verification pattern that makes integration tests fast, and testing GraphQL.

In this section

- The built-in request fixture: one expect API for both UI and API tests.
- Authenticating API calls by reusing storageState from the browser session.
- Contract testing with TypeScript interfaces, run as the fast first stage.
- API setup + UI verification + API cleanup: the fast integration pattern.
- Testing and mocking GraphQL by operation name, since the URL is shared.
- tests/api for contracts, tests/integration for hybrid, structure that scales.

Q 98 LEVEL · Mid

How do you use Playwright's APIRequestContext for API testing?

CONCEPT

Playwright's built-in **request** fixture is a full HTTP client with the same expect API as browser tests, no Axios or Supertest needed. You call **request.get**, **post**, put, delete with params, data, and headers, then assert on status and the parsed body. The strategic value is consolidation: API and UI tests share one framework, one runner, one report, and the same assertions you use on DOM elements work on API responses. A clean structure follows: tests/api for pure contract tests and tests/integration for tests that use the API to set up state and then verify it through the UI. That hybrid layout gives broad coverage with little duplication.

INTERVIEW STRATEGY

You are being marked on whether you treat Playwright as a UI-only tool or as a full testing platform. Lead with the consolidation argument, one framework, one runner, one report, the same expect API for both layers, because that is the insight that shows range. Where this goes wrong is saying you would reach for Supertest or Axios, which signals you have not used the request fixture. The follow-up is usually 'how do you organise API vs UI tests?', answer with the tests/api and tests/integration split.

How to phrase it

> *Playwright has a built-in request fixture that is a full HTTP client, and the first thing I would stress is that it uses the exact same expect API as my browser tests, so I do not pull in Axios or Supertest. I call request.get or request.post with params, data and headers, then assert on the status and the parsed body, exactly the way I assert on the DOM. The strategic value, and the reason I would lead with it, is consolidation: API and UI tests live in one framework, one runner and one report, so there is no second toolchain to maintain and no context-switch for the team. That sets up the follow-up about organising the two layers, where I split the suite into tests/api for pure contract tests that hit endpoints directly, and tests/integration for tests that use the API to set up state and then verify it through the UI. That hybrid layout gives me broad coverage with very little duplication, because each layer tests what it is best at. The 2026 reality is that teams expect one framework to cover both, and reaching for a separate API library now reads as legacy thinking rather than a deliberate choice.*

Key points to hit

(1) request fixture is a full HTTP client with the same expect API. **(2)** No Axios or Supertest needed: one framework, one runner, one report. **(3)** Same assertions work on API responses and DOM elements. **(4)** Follow-up: tests/api for contracts, tests/integration for hybrid flows. **(5)** 2026 reality: one framework for both layers is the expectation. **(6)** The common miss: saying you would add Supertest/Axios, signals you skipped the fixture.

CODE

```
import { test, expect } from '@playwright/test';

test('GET /api/users returns a paginated list', async ({ request }) => {
const response = await request.get('/api/users', { params: { page: 1, limit: 10 } });
expect(response.status()).toBe(200);
const body = await response.json();
expect(body.data).toHaveLength(10);
expect(body.page).toBe(1);
});

test('POST /api/users creates a user', async ({ request }) => {
const response = await request.post('/api/users', {
data: { name: 'Test User', email: 'test@example.com', role: 'user' },
});
expect(response.status()).toBe(201);
expect((await response.json()).id).toBeDefined();
});
```

Q 99 LEVEL · Mid

How do you authenticate API requests in Playwright?

CONCEPT

Create an authenticated **APIRequestContext** in a fixture, either by reusing the **storageState** from the browser session or by setting Authorization headers directly. The elegant approach reuses the same auth state generated in globalSetup for browser tests, one place produces the session, both UI and API tests consume it, and there is no separate token management. Build one context per role (an admin context, a user context), give each its baseURL and headers, and **dispose** it after the test to prevent session leakage between tests. A high-value side effect: the admin API context becomes your fast path for seeding test data before a UI test, which is far quicker than driving setup through the interface.

INTERVIEW STRATEGY

They are looking to see you avoid duplicate auth plumbing. Lead with reusing storageState from globalSetup, one auth source for both layers, because that is the elegant answer. Mention disposing the context to prevent session leakage; that detail signals you have debugged cross-test contamination. The mistake is hand-rolling token management per test. The follow-up is often ‘how do you use the authenticated API context beyond assertions?’, answer that the admin context seeds test data far faster than the UI.

How to phrase it

> *I create an authenticated APIRequestContext in a fixture, and the elegant way to do it, which is what I would lead with, is to reuse the storageState that globalSetup already generated for the browser tests. So the same auth source feeds both layers: one place logs in per role and saves the session, and both my UI and API tests consume it, with no separate token management to maintain or refresh. I build one context per role, an admin context and a user context, each with its baseURL and content-type headers, and I dispose each context after the test. That dispose matters, and it is the detail I would call out: without it you get session leakage between tests, where one test's context bleeds into another and you get confusing cross-test failures. For the follow-up about using the API context beyond assertions, the admin context is also my fast path for seeding data before a UI test. Creating a user through the API takes a couple of hundred milliseconds versus thirty seconds of filling forms in the UI, so it is roughly ten times faster and it makes the tests independent of each other. So the whole approach is one auth source, disposed cleanly, doubling as a fast setup tool.*

Key points to hit

(1) Build an APIRequestContext in a fixture; reuse storageState from globalSetup. **(2)** One auth source feeds both UI and API tests: no separate token mgmt. **(3)** Dispose the context after each test to prevent session leakage. **(4)** One context per role (admin, user) with baseURL and headers. **(5)** Expect next: the admin context seeds data ~10x faster than the UI. **(6)** Avoid: hand-rolling per-test token management.

CODE

```
// src/fixtures/api.fixture.ts
import { test as base, APIRequestContext } from '@playwright/test';

type APIFixtures = { adminAPI: APIRequestContext };

export const test = base.extend<APIFixtures>({
adminAPI: async ({ playwright }, use) => {
const context = await playwright.request.newContext({
baseURL: process.env.API_URL,
storageState: 'auth/admin-state.json',
extraHTTPHeaders: { 'Accept': 'application/json' },
});
await use(context);
await context.dispose(); // prevent session leakage
},
});
```

Q 100 LEVEL · Mid

How do you implement API contract testing with Playwright?

CONCEPT

Contract testing validates that API responses conform to the expected **schema**, field names, types, and business rules. Paired with **TypeScript interfaces** it becomes a type-safe contract test: type the parsed response, then assert on each field's type, allowed values, and format (for example, a

valid email pattern or a parseable date). These tests are the safety net between frontend and backend teams: when the backend renames or retypes a field, the contract test fails in CI before any UI test runs. Run them as the first test stage: they need no browser, so they are fast and catch breaking changes early. For complex APIs, a runtime schema validator such as **zod** generates TypeScript types from the schema and validates responses at runtime, giving both compile-time and runtime safety.

INTERVIEW STRATEGY

What they want is to know if you guard the frontend-backend boundary, not just test happy paths. Frame contract tests as the safety net between teams, and stress running them as the fast first stage so breaking changes fail before any browser starts. Mentioning zod for runtime validation signals current practice. The failure mode is asserting exact values for generated fields like IDs. Expect the follow-up 'how do you get both compile-time and runtime safety?', answer that TypeScript interfaces cover compile time and zod covers runtime.

How to phrase it

> *Contract testing validates that an API response matches the expected schema, the field names, the types, and the business rules, and paired with TypeScript interfaces it becomes type-safe. I type the parsed response against an interface, then assert on each field's type, its allowed values, and its format, like a valid email pattern or a parseable date, rather than asserting exact values for generated fields like IDs, which would make the test brittle. The framing I use is that these are the safety net between the frontend and backend teams: when the backend renames or retypes a field, the contract test fails in CI before any UI test even runs. That is why I run them as the first test stage, they need no browser, so they are fast and they catch breaking changes early, which is shift-left applied to the API contract. For the follow-up about getting both compile-time and runtime safety, my answer is to combine the two: TypeScript interfaces give me compile-time checking, and for complex APIs I add zod, which generates TypeScript types from the schema and validates the response at runtime, so a field that is the wrong type at runtime fails loudly rather than slipping through. That combination is what makes the contract genuinely enforced rather than just documented.*

Key points to hit

(1) Validate field names, types, allowed values and format, not exact values. **(2)** TypeScript interfaces make the contract test type-safe. **(3)** The safety net between frontend and backend teams. **(4)** Run as the fast first stage: no browser, catches breaking changes early. **(5)** They usually ask next: zod adds runtime schema validation on top of compile-time types. **(6)** The easy mistake: asserting exact values for generated fields like IDs.

CODE

```
interface UserResponse {
id: number; name: string; email: string;
role: 'admin' | 'user' | 'viewer'; createdAt: string;
}

test('GET /api/users/:id matches the contract', async ({ request }) => {
const response = await request.get('/api/users/1');
await expect(response).toBeOK();
const user: UserResponse = await response.json();
expect(typeof user.id).toBe('number');
expect(['admin', 'user', 'viewer']).toContain(user.role);
expect(new Date(user.createdAt).toString()).not.toBe('Invalid Date');
expect(user.email).toMatch(/^[^\s@]+@[^\s@]+\.[^\s@]+$/);
});
```

Q 101

LEVEL · Mid

How do you combine API setup with UI verification in tests?

CONCEPT

The most efficient integration pattern: use the **API to create test data**, **verify through the UI**, then use the **API to clean up**. Creating a user through the UI means filling forms, handling validation, and waiting for redirects, around 30 seconds; creating the same user via API takes roughly 200ms. The test is faster, more reliable, and tests exactly what it claims to: that the UI correctly displays data that exists in the system. The UI creation flow is not skipped: it is tested separately in its own dedicated test, where it is the thing under test rather than incidental setup. This separation is what keeps a large integration suite both fast and honest about what each test actually verifies.

INTERVIEW STRATEGY

What is being tested is whether you build fast, focused integration tests or slow end-to-end monsters. Lead with the pattern, API setup, UI verification, API cleanup, and quantify it (30 seconds via UI versus 200ms via API). The key nuance, and the senior signal, is that you still test the UI creation flow, just in its own dedicated test. Where this goes wrong is doing all setup through the UI, which makes tests slow and conflates setup with the assertion. They usually follow with ‘do you ever test creation through the UI?’, answer yes, once, in a focused test.

How to phrase it

> *This is the pattern I use for the large majority of our integration tests: create the test data through the API, verify it through the UI, then clean up through the API. The reason is partly speed and partly focus. Creating a user through the UI means filling forms, handling validation and waiting for redirects, about thirty seconds, whereas creating the same user through the API takes around two hundred milliseconds, so the test is dramatically faster and more reliable. But the focus point matters just as much: this test claims to verify that the UI correctly displays data that exists in the system, and doing the setup through the API means the test is actually testing that, not accidentally re-testing the creation form. That sets up the obvious follow-up about whether I ever test creation through the UI, and the answer is yes, the UI creation flow is tested once, in its own dedicated test, where it is the thing under test rather than incidental setup. So I am not skipping UI coverage, I am putting each concern in the test that is best at it. That separation is what keeps a big integration suite both fast and honest about what each test really verifies.*

Key points to hit

(1) Pattern: API create, UI verify, API clean up. **(2)** ~30s to create via UI vs ~200ms via API: faster and more reliable. **(3)** The test verifies the UI displays data, not the creation form. **(4)** The next question: the UI creation flow is tested once, in its own dedicated test. **(5)** Keeps a large integration suite fast and honest about what it verifies. **(6)** Trap: doing all setup through the UI, slow and conflates concerns.

CODE

```
test('created user appears in the admin list', async ({ request, page }) => {
// Setup via API: fast and reliable
const created = await request.post('/api/users', {
data: { name: 'Test User', email: 'testuser@example.com', role: 'user' },
});
const { id } = await created.json();

// Verify through the UI
await page.goto('/admin/users');
const row = page.locator('tbody tr').filter({ hasText: 'testuser@example.com' });
await expect(row).toBeVisible();
await expect(row.getByTestId('role-badge')).toHaveText('user');

// Cleanup via API: no UI teardown
await request.delete(`/api/users/${id}`);
});
```

Q 102 LEVEL · Mid to Senior

How do you test GraphQL APIs with Playwright?

CONCEPT

GraphQL uses **POST requests with a query body**, and Playwright's request fixture handles that natively, you post the query and variables and assert on the data. The interesting part is mocking: because all GraphQL requests go to the same endpoint, you cannot match by URL alone. Instead

you inspect the request body and match on the **operation name**, mocking specific queries while passing others through with continue. To keep mocks and queries in sync, extract operation names into **constants** so a rename surfaces as a TypeScript mismatch rather than a silently unmatched mock. This is what makes GraphQL testing maintainable rather than a tangle of body-string checks.

INTERVIEW STRATEGY

The aim is to know if you have hit the GraphQL single-endpoint problem. State that querying is native via the request fixture, then make the real point: mocking must match on operation name, not URL, because every query shares one endpoint. The constants-for-operation-names tip is the maintainability signal. The trap is trying to match GraphQL mocks by URL, which matches everything. The next question is normally 'how do you keep the mock in sync with the query?', answer with shared operation-name constants that fail typecheck on a rename.

How to phrase it

> *GraphQL uses POST requests with a query body, and Playwright's request fixture handles that natively, I post the query and variables and assert on the data that comes back, same as any other API test. The part worth talking about, and the thing that trips people up, is mocking. Because all GraphQL requests go to the same endpoint, I cannot match by URL the way I would with REST, a URL match would catch every query at once. So the pattern is to inspect the request body and match on the operation name, fulfilling the specific query I want to mock and calling continue for everything else. That answers the question of how you mock one query without breaking the rest. For the follow-up about keeping the mock in sync with the query, my answer is to extract operation names into shared constants that both the query and the mock reference, so if someone renames an operation, TypeScript flags the mismatch at compile time rather than leaving me with a mock that silently never matches and a test that quietly tests nothing. That discipline is what makes GraphQL testing maintainable instead of a tangle of fragile body-string checks, and it is the kind of detail that shows I have actually run GraphQL suites at scale.*

Key points to hit

(1) Query natively via the request fixture: post query + variables, assert on data. **(2)** All GraphQL hits one endpoint: mock by operation name, not URL. **(3)** Match the operation in the body; continue for everything else. **(4)** Asked next: shared operation-name constants fail typecheck on a rename. **(5)** Keeps mocks in sync and avoids a silently unmatched mock. **(6)** Where this goes wrong: trying to match GraphQL mocks by URL, matches everything.

CODE

```
// Query natively
test('fetch user via GraphQL', async ({ request }) => {
const response = await request.post('/graphql', {
data: { query: 'query GetUser($id: ID!) { user(id: $id) { id email } }', variables: {
id: '1' } },
});
await expect(response).toBeOK();
expect((await response.json()).data.user.email).toBeDefined();
});

// Mock by operation name (URL is shared)
await page.route('**/graphql', async route => {
const body = route.request().postDataJSON();
if (body.query.includes('GetProducts')) {
await route.fulfill({ status: 200, body: JSON.stringify({ data: { products: [] } }) });
} else {
await route.continue();
}
});
```

Q 103 LEVEL · Mid to Senior

What are the three ways to get an APIRequestContext, and when do you use each?

CONCEPT

Three sources, three use cases. **1. The request fixture**: async ({ request }) => ..., a standalone APIRequestContext for pure-API tests with no browser, the right choice for fast smoke tests, contract tests, and any test that doesn't need a UI. Fresh per test, isolated, fast. **2. page.request**: **page.request.get(...)**, a context attached to a Page that shares cookies and storage with the browser, the right choice when an API call must use the same authentication state as the browser session, especially for testing that the UI and API see the same data after a user action. 3. Custom via playwright.request.newContext(): an explicitly-created context with custom baseURL, extraHTTPHeaders, or storageState, the right choice for tests that need specific auth headers or want fine-grained control over shared state across many requests. The rule: request fixture for pure-API, page.request when sharing browser auth, newContext for custom configuration.

INTERVIEW STRATEGY

The point of this one is to see source-discipline. List the three contexts: request fixture (standalone), page.request (shares browser auth), newContext (custom config). The shares-cookies-with-browser distinction for page.request is what separates junior from senior thinking because it's why you pick it. The classic error is using request fixture when you needed the browser's auth state. A common follow-up is ‘when do you need page.request specifically?’, answer when verifying that the API sees the same auth state as the UI.

How to phrase it

> *Three sources, three different use cases, and choosing the right one is what shows API-testing fluency. First, the request fixture: async curly request returns a standalone APIRequestContext. It's fresh per test, completely isolated, and has no browser attached. This is the right choice for pure-API tests with no UI involvement, contract tests, smoke tests, anything where a browser would just be overhead. Second, page.request: a context attached to a Page that shares cookies and storage state with the browser. So a request made through page.request sends the same auth cookies the browser is using, which is the right choice when an API call must use the same authentication state as the browser session. The example I'd give is verifying that after the user performs an action in the UI, the API now returns the corresponding data, same session, same view of the world. Third, custom context via playwright.request.newContext: I create it explicitly with options like baseURL, extraHTTPHeaders for an API key, or storageState for pre-loaded auth. This is the right choice for tests that need specific configuration that doesn't match the default, testing as a different user, against a different environment, with specific headers like X-Test-Mode. The rule I would close on, because it's the heuristic that lands in interviews, is request fixture for pure-API, page.request when sharing browser auth, newContext for custom configuration. That's also my answer to the follow-up about when I specifically need page.request. When I'm verifying that the API sees exactly the same auth state the UI does, not a similar one with the same credentials, but the actual same session cookie. Useful for testing session-scoped endpoints, CSRF tokens, anything where the browser's cookie jar is the source of truth. The trap is using the request fixture when I needed the browser's auth state, the test runs against a different session than the UI, and integration questions about consistency become harder to answer.*

Key points to hit

(1) request fixture: standalone, fresh per test, no browser, pure-API tests. **(2)** page.request: shares cookies/storage with the browser, same session as UI. **(3)** playwright.request.newContext(): custom baseURL, headers, storageState. **(4)** Rule: fixture for pure-API; page.request for browser-auth; newContext for custom. **(5)** Follow-up: page.request when API must see the same session as the UI. **(6)** What loses the point: request fixture when you needed the browser's auth, different session.

CODE

```
// 1. request fixture: pure-API, no browser overhead
test('GET /users returns users', async ({ request }) => {
const res = await request.get('/api/users');
expect(res.status()).toBe(200);
});

// 2. page.request: shares browser auth
test('UI action reflects in API', async ({ page }) => {
await page.goto('/profile');
await page.getByLabel('Display name').fill('New Name');
await page.getByRole('button', { name: 'Save' }).click();
// Same session as the browser: sees the just-saved state
const res = await page.request.get('/api/me');
expect((await res.json()).displayName).toBe('New Name');
});

// 3. Custom newContext: specific headers/baseURL
const apiCtx = await playwright.request.newContext({
baseURL: process.env.API_URL,
extraHTTPHeaders: { 'X-API-Key': process.env.API_KEY!, 'X-Test-Mode': 'true' },
});
```

Q 104

LEVEL · Senior

How do you reuse authentication tokens across many tests without re-authenticating each time?

CONCEPT

Three strategies depending on the scope. **storageState for browser-scoped auth**: globalSetup or a setup project logs in once via the UI, saves cookies+localStorage to a JSON file, and every test loads via **use: { storageState: 'auth/admin.json' }**. Zero login overhead per test. extraHTTPHeaders for header-based auth (Bearer tokens, API keys): fetch the token once in setup, write it to a file, and tests load it into extraHTTPHeaders. Works for both UI tests (header sent on every request) and pure-API tests. **Worker-scoped fixture for short-lived tokens**: a worker fixture that authenticates once at worker startup and provides the token to every test in that worker, right for tokens that expire too quickly to share suite-wide but too expensively to fetch per test. The principle: match the auth scope to the token lifetime. Long-lived tokens go in storageState; medium-lived in a worker fixture; ephemeral tokens fetched per test.

INTERVIEW STRATEGY

You are being asked to show scope-matched auth reuse. List the three strategies, storageState, extraHTTPHeaders, worker fixture, each with the token lifetime that justifies it. The 'match scope to token lifetime' rule is the seniority marker. The trap is logging in per test. The follow-up is often ‘what about tokens that expire mid-suite?’, answer worker-scoped fixture refreshes on 401, with a single in-flight refresh to avoid thundering herd.

How to phrase it

> *Three strategies, and the rule that ties them together is match the auth scope to the token lifetime. First, storageState for browser-scoped auth. globalSetup or a setup project logs in once via the UI, saves cookies and localStorage to a JSON file, and every test loads via use storageState pointing at the file. Zero login overhead per test. This is right for long-lived sessions: a normal user logged in for the test suite duration. Second, extraHTTPHeaders for header-based auth like Bearer tokens or API keys. I fetch the token once in setup, write it to a file, and tests load it into extraHTTPHeaders on the request context. Works for both UI tests, where the header is sent on every request the browser makes through that context, and for pure-API tests via the request fixture or a custom newContext. Third, a worker-scoped fixture for short-lived tokens. The fixture authenticates once at worker startup and provides the token to every test in that worker, right for tokens that expire too quickly to share suite-wide but too expensively to fetch per test. The principle I would close on, because it's the heuristic that lands, is match the auth scope to the token lifetime. Long-lived tokens go in storageState. Medium-lived in a worker fixture. Ephemeral tokens fetched per test. That's also my answer to the follow-up about tokens that expire mid-suite. I use a worker-scoped fixture that refreshes the token on a 401 response, with a single in-flight refresh shared across the worker to avoid thundering herd, ten parallel tests in a worker all hitting 401 and all trying to refresh simultaneously is a real failure mode. One mutex on the refresh, all tests share the new token. The classic error is logging in per test, which five-to-eight-seconds overhead times five hundred tests is over an hour of pure auth time the suite doesn't need.*

Key points to hit

(1) storageState: long-lived browser auth via globalSetup/setup project. **(2)** extraHTTPHeaders: token loaded from setup file, UI + pure-API both. **(3)** Worker-scoped fixture: short-lived tokens, authenticated once per worker. **(4)** Rule: match auth scope to token lifetime (long/medium/ephemeral). **(5)** Expect next: refresh on 401 with single in-flight to avoid thundering herd. **(6)** The common miss: per-test login, 5-8s × 500 tests = +60min of pure auth overhead.

CODE

```
// Strategy 1: storageState (long-lived session)
// auth.setup.ts saves: page.context().storageState({ path: 'auth/admin.json' })
// playwright.config.ts, use: { storageState: 'auth/admin.json' }

// Strategy 2: Bearer token via extraHTTPHeaders
const token = JSON.parse(fs.readFileSync('auth/api-token.json', 'utf8')).token;
const apiCtx = await playwright.request.newContext({
baseURL: process.env.API_URL,
extraHTTPHeaders: { Authorization: `Bearer ${token}` },
});

// Strategy 3: worker-scoped fixture for short-lived tokens
export const test = base.extend<{}, { sharedToken: string }>({
sharedToken: [async ({}, use) => {
const token = await fetchTokenFromAuthService();
await use(token);
}, { scope: 'worker' }],
});
```

Q 105 LEVEL · Senior

How do you validate API responses against a JSON Schema or OpenAPI specification?

CONCEPT

JSON Schema validation catches the failure mode regular field-by-field assertions miss: the API silently changed shape. Use **ajv** or **zod** to compile a schema and validate every response in the test, ajv for working from JSON Schema directly (often generated from an OpenAPI spec), zod when defining schemas inline in TypeScript because the validator type and the inferred TypeScript type are the same source of truth. The pattern: **schemaForGetUsers** defined once, asserted on the response of every test that hits that endpoint. The discipline that matters: generate schemas from the OpenAPI spec rather than hand-writing them, so the test schema is the contract the API publishes. When the API changes its OpenAPI spec, the regenerated schema fails old assertions, and the test suite acts as a contract enforcer. Schema validation is what catches the case where the response has all the right field names but a wrong type, string where number was expected, missing a required field, extra unknown field.

INTERVIEW STRATEGY

What scores here is showing schema-based contract enforcement. Lead with the failure mode schema validation catches: silent shape changes. Then make ajv-vs-zod the source-of-truth distinction, ajv for JSON Schema (often from OpenAPI), zod for TypeScript-first. The 'generate from OpenAPI, don't hand-write' discipline is the test of seniority on this. Where this goes wrong is field-by-field assertions that pass when fields have the wrong type. Expect the follow-up ‘what if the team doesn't have an OpenAPI spec?’, answer use zod and let the test schemas be the contract.

How to phrase it

> *JSON Schema validation catches the failure mode regular field-by-field assertions miss, which is the API silently changed shape. I'd lead with that, because it's the case-for-schemas in one sentence. Field-by-field assertions like expect body dot email to be defined will pass if email is a number instead of a string, the field is there, just wrong type. Schema validation catches that. I use one of two libraries depending on the source of truth. ajv for working from JSON Schema directly, which is often generated from an OpenAPI spec the API team publishes. Or zod when defining schemas inline in TypeScript, because the validator type and the inferred TypeScript type are the same source of truth, z dot object gives me a runtime validator and a TypeScript type from one declaration. The pattern is: a schemaForGetUsers defined once and asserted on the response of every test that hits that endpoint, so every test is implicitly a contract test. The discipline I would emphasise, because it's where this stops being a manual exercise and starts scaling, is generate schemas from the OpenAPI spec rather than hand-writing them. The test schema is then the contract the API publishes, when the API team changes their spec, the regenerated schema fails old assertions, and the test suite acts as a contract enforcer. Schema validation catches the case where the response has all the right field names but a wrong type: string where number was expected, missing a required field, extra unknown field. Those are exactly the changes that ship to production silently and crash the frontend at runtime. That's also my answer to the follow-up about teams without an OpenAPI spec. I use zod and let the test schemas themselves be the contract. The tests document the shape, and the team can move toward formal OpenAPI later from the same schemas. The trap is field-by-field assertions that pass when a field is the wrong type.*

Key points to hit

(1) Catches silent shape changes: wrong type, missing required, unknown field. **(2)** ajv for JSON Schema (often generated from OpenAPI). **(3)** zod for TypeScript-first: validator type AND TS type from one source. **(4)** Generate from OpenAPI when available: test enforces published contract. **(5)** They usually ask next: no OpenAPI? Use zod schemas as the contract; formalise later. **(6)** Avoid: field-by-field assertions pass when fields have the wrong type.

CODE

```
import { z } from 'zod';

// One source of truth: runtime validator AND TypeScript type
const UserSchema = z.object({
id: z.number().int().positive(),
email: z.string().email(),
isAdmin: z.boolean(),
createdAt: z.string().datetime(),
});
const UsersResponseSchema = z.object({ users: z.array(UserSchema), total: z.number() });
type UsersResponse = z.infer<typeof UsersResponseSchema>;

test('GET /users matches contract', async ({ request }) => {
const res = await request.get('/api/users');
expect(res.status()).toBe(200);

const json: unknown = await res.json();
const parsed = UsersResponseSchema.parse(json); // throws on shape mismatch
expect(parsed.users.length).toBeGreaterThan(0); // domain assertion after schema OK
});

// Alternative: ajv from OpenAPI-generated JSON Schema
// const validate = ajv.compile(schemaFromOpenApi.paths['/users'].get.responses['200']);
```

Q 106 LEVEL · Mid

How do you upload files via API requests (multipart/form-data)?

CONCEPT

Playwright's APIRequestContext handles multipart by passing the **multipart** option on a post() call. The value is an object mapping field names to either strings (regular form fields), **file paths** (read from disk), or objects with **name, mimeType, buffer** for in-memory files. Playwright sets the Content-Type header to multipart/form-data with the correct boundary automatically. The pattern that matters: a fixtures folder with test files (sample.pdf, small.png) loaded by path for filesystem-based tests, plus a buffer-based approach for generated files, useful for testing rejection of specific MIME types or oversized files without committing huge test fixtures to git. The discipline: commit small fixture files (under ~50 KB); generate larger or invalid ones at test time via Buffer. The latter keeps the repo small while letting you test the 'reject 10 MB upload' path with a synthetic buffer.

INTERVIEW STRATEGY

This question is there to see multipart handled correctly. Lead with the multipart option on post(), then the three value shapes (string, file path, buffer with name+mimeType). The 'commit small fixtures, generate large ones' discipline is the senior signal. The classic error is building FormData by hand. They usually follow with ‘how do you test rejection of oversized files?’, answer generate a large Buffer in-test rather than committing a 10 MB file to git.

How to phrase it

> *Playwright's APIRequestContext handles multipart form-data by passing the multipart option on a post call. The value is an object mapping field names to either strings for regular form fields, file paths read from disk, or objects with name, mimeType, and buffer for in-memory files. Playwright sets the Content-Type header to multipart slash form-data with the correct boundary automatically, which is the part most candidates would otherwise build by hand and get wrong. The pattern that matters in a real test suite is a fixtures folder, typically test-data slash fixtures, with test files like sample.pdf, small.png that I load by file path. That covers the happy path. Plus a buffer-based approach for generated files, which I use for testing rejection of specific MIME types or oversized files. The discipline I would lead with, because it's where the source tree stays sane, is commit small fixture files under about fifty kilobytes; generate larger or invalid ones at test time via Buffer. The reason matters: I don't want a ten-megabyte test PDF in git, but I do want to test the API's rejection of a ten-megabyte upload. So I generate a Buffer of ten million bytes in-test, pass it via multipart with a name and a fake mimeType, and assert the API returns 413 payload too large. The repo stays small; the test still covers the case. And to the follow-up about testing rejection of oversized files: generate a large Buffer in-test rather than committing the file. Same pattern for testing invalid mimeType rejection, a small Buffer with an unexpected mimeType string, and for malformed file content like a corrupted PDF, where I generate the buffer with deliberately invalid headers. The failure mode is building FormData by hand and getting boundary handling wrong, which the multipart option avoids entirely.*

Key points to hit

(1) multipart option on request.post() handles multipart/form-data correctly. **(2)** Three value shapes: string (form field), file path (disk), buffer object. **(3)** Buffer shape: { name, mimeType, buffer } for in-memory files. **(4)** Discipline: commit fixtures <50KB; generate large/invalid ones at test time. **(5)** The next question: 10 MB rejection test uses Buffer.alloc, keeps repo small. **(6)** The easy mistake: hand-built FormData with manual boundary handling, use the option.

CODE

```
// File path from disk: the happy path
test('uploads a PDF', async ({ request }) => {
const res = await request.post('/api/documents', {
multipart: {
file: 'tests/fixtures/sample.pdf', // disk path
category: 'invoice', // regular form field
notes: 'Q1 expenses',
},
});
expect(res.status()).toBe(201);
});

// Generated buffer: test the rejection of oversized files
test('rejects 10 MB upload with 413', async ({ request }) => {
const res = await request.post('/api/documents', {
multipart: {
file: { name: 'big.pdf', mimeType: 'application/pdf', buffer: Buffer.alloc(10 * 1024 *
1024) },
category: 'invoice',
},
});
expect(res.status()).toBe(413);
});
```

Q 107

LEVEL · Mid to Senior

How do you assert on response time and SLA targets in API tests?

CONCEPT

Playwright's request methods don't expose a built-in timing field, so measure explicitly with **performance.now()** or Date.now() around the request. The pattern: capture start, await the request, capture end, assert duration is under the SLA. The discipline that distinguishes a useful test from a flaky one: use a soft upper bound based on the SLA, not the happy-path observed time. If the SLA is 500ms p95, assert under 1000ms in tests to absorb CI machine variance and avoid asserting on a noisy 100ms p50. Track **statistics across runs** rather than per-test, a single 700ms response is fine; ten consecutive 700ms responses indicate regression. For genuine performance testing, dedicated tools (k6, Artillery, Locust) measure better; Playwright is the right tool for catching gross regressions and SLA-breach smoke tests, not for percentile-level performance regression detection.

INTERVIEW STRATEGY

They want to see calibrated performance assertions and the limits. Lead with performance.now() around the request, then the soft-upper-bound discipline based on SLA not happy-path. The 'Playwright for gross regressions, k6 for percentile performance' framing is the senior signal because it shows tool calibration. The risk is asserting on observed happy-path times. The next question is normally ‘what about percentile-level regression?’, answer use a real load-testing tool; Playwright is wrong for that.

How to phrase it

> *Playwright's request methods don't expose a built-in timing field, so I measure explicitly with performance.now or Date.now around the request. Capture start, await the request, capture end, assert duration is under the SLA target. Straightforward in shape. The discipline that distinguishes a useful test from a flaky one, and the framing I would lead with, is use a soft upper bound based on the SLA, not the happy-path observed time. If the documented SLA is five hundred milliseconds at the ninety-fifth percentile, I assert under a thousand milliseconds in tests. That absorbs CI machine variance and avoids the test failing every time a noisy CI runner takes a hundred fifty milliseconds instead of fifty for the request. Asserting on the happy-path observed time, 'I saw a hundred millisecond response in dev, let's assert under two hundred', produces a flaky test that fails the moment load increases anywhere in the system. The second discipline is tracking statistics across runs rather than per-test. A single seven-hundred millisecond response is fine; ten consecutive seven-hundred-millisecond responses across runs indicate regression. Trend-based monitoring catches what instantaneous assertions miss. The calibration that matters, and my answer to the follow-up about percentile-level regression detection, is for genuine performance testing dedicated tools like k6, Artillery, or Locust measure better. They produce real percentiles, real load curves, real concurrent-user scenarios. Playwright is the right tool for catching gross regressions, the API went from one hundred milliseconds to ten seconds, and for SLA-breach smoke tests in CI. It's the wrong tool for ninety-ninth percentile latency tracking under load. Use the right tool for the right question. The trap is asserting on observed happy-path times and shipping a brittle suite that fails every time CI is slow.*

Key points to hit

(1) Measure with performance.now() around the request, no built-in timing field. **(2)** Soft upper bound from SLA, not happy-path observed time. **(3)** SLA 500ms p95 -> assert <1000ms in tests to absorb CI variance. **(4)** Track statistics across runs: trend > single-test value. **(5)** Asked next: percentile-level perf needs k6/Artillery/Locust, not Playwright. **(6)** Trap: asserting on happy-path times, flakes every time CI is slow.

CODE

```
test('GET /products meets SLA', async ({ request }) => {
const start = performance.now();
const res = await request.get('/api/products?limit=50');
const durationMs = performance.now() - start;

expect(res.status()).toBe(200);
// SLA: 500ms p95. Test threshold: 1000ms (soft upper bound)
expect(durationMs).toBeLessThan(1000);

// For trend tracking: write to a metrics file or reporter
test.info().annotations.push({ type: 'duration_ms', description: String(durationMs) });
});

// Wrong: asserting near observed happy-path time, flakes on slow CI runs
// expect(durationMs).toBeLessThan(150);

// For real percentile performance testing: k6, Artillery, Locust
```

Q 108 LEVEL · Mid to Senior

How do you test rate limiting (429 responses) and retry logic?

CONCEPT

Two halves: **verify the API enforces rate limits**, and verify the client (your app) handles 429s correctly. For verifying enforcement, fire requests in a loop until the limit triggers, assert the 429 status with the expected **Retry-After** header, and verify subsequent requests succeed after the window resets. For verifying client behaviour, use page.route to inject 429 responses and assert your app's retry: does it back off with the Retry-After value, does it cap retries to avoid infinite loops, does the UI show a 'rate-limited' state rather than crashing? Both halves matter for different reasons: the enforcement test catches API bugs; the client test catches frontend bugs. The discipline: **respect Retry-After in test code too**. A test that fires more requests after a 429 without honoring the wait is testing the wrong thing and may exhaust the rate limit for other tests.

INTERVIEW STRATEGY

Your job here is to show both sides of rate limiting tested. Lead with the two-halves split, verify enforcement, verify client handling, then make Retry-After honoring the discipline both for the app and the test code itself. The 'firing more requests after 429 exhausts the limit for other tests' detail is the senior signal. The risk is firing 1000 requests and stopping at the first 429 without verifying the recovery. A common follow-up is ‘how do you test exponential backoff?’, answer route.fulfill increasing Retry-After values, assert client waits the right amount each retry.

How to phrase it

> *Rate-limit testing has two halves and a complete answer covers both. First, verify the API enforces its rate limits. I fire requests in a loop until the limit triggers, assert the 429 status with the expected Retry-After header, and verify that subsequent requests succeed after the window resets. This catches the bug where the API documents a hundred-per-minute limit but actually enforces something else, or doesn't enforce one at all. Second, verify the client, my application, handles 429 responses correctly. I use page.route to inject a 429 response with a Retry-After header and assert the app's retry behaviour. Does it back off with the Retry-After value rather than hammering immediately? Does it cap retries to avoid infinite loops? Does the UI show a 'rate-limited' state with a sensible message rather than crashing or hanging? Both halves matter for different reasons. The enforcement test catches API-side bugs; the client test catches frontend bugs in how my app reacts to rate limits. The discipline I would emphasise, because it's where rate-limit tests cause collateral damage, is respect Retry-After in test code too. A test that fires more requests after a 429 without honouring the wait period is testing the wrong thing, it's just spamming, and worse, it may exhaust the shared rate limit for other tests running in parallel, causing flake across the suite. So I read the Retry-After header in the test, wait that long, then continue. That's also my answer to the follow-up about testing exponential backoff in the client. I route .fulfill increasing Retry-After values on successive requests, first 429 returns Retry-After one, second returns two, third returns four, then assert the client waits the right amount each retry by measuring the time between requests in the route handler. That tests the doubling without depending on wall-clock timing in the test itself. The trap is firing a thousand requests, getting a 429, and stopping there without verifying the recovery.*

Key points to hit

(1) Two halves: API enforces 429; client handles 429, both matter. **(2)** Enforcement test: loop requests to trigger, assert 429 + Retry-After. **(3)** Client test: page.route injects 429; assert backoff/cap/UI state. **(4)** Respect Retry-After in test code: spamming exhausts shared rate limits. **(5)** Follow-up: exponential backoff via successive Retry-After values 1, 2, 4... **(6)** Where this goes wrong: fire requests until 429 and stop, doesn't verify recovery.

CODE

```
// Half 1: verify the API enforces the rate limit
test('API enforces 100/min rate limit', async ({ request }) => {
const responses = await Promise.all(
Array.from({ length: 105 }, () => request.get('/api/search?q=test'))
);
const rateLimited = responses.filter(r => r.status() === 429);
expect(rateLimited.length).toBeGreaterThanOrEqual(5);
expect(rateLimited[0]!.headers()['retry-after']).toBeDefined();
});

// Half 2: verify the client handles 429 with backoff and recovery
test('app retries after Retry-After on 429', async ({ page }) => {
let calls = 0;
await page.route('**/api/data', async route => {
calls++;
if (calls === 1) {
return route.fulfill({ status: 429, headers: { 'retry-after': '2' } });
}
return route.fulfill({ status: 200, body: JSON.stringify({ ok: true }) });
});
await page.goto('/dashboard');
await expect(page.getByText('Rate limited: retrying...')).toBeVisible();
await expect(page.getByText('Loaded')).toBeVisible({ timeout: 5_000 });
expect(calls).toBe(2);
});
```

Q 109 LEVEL · Senior

How do you build typed test-data factories that combine branded IDs with automatic cleanup?

CONCEPT

A test-data factory is a class or function that creates an entity via API and returns a typed reference, while tracking the entity for cleanup at test teardown. The pattern uses branded types from S1, UserId, OrderId, so the factory return type can't be confused with a raw string, plus a fluent builder API that reads like the scenario rather than the API: **new UserBuilder().asAdmin().withEmail(...).create(request)**. Combine with a worker-scoped or test-scoped cleanup fixture that records IDs created during the test and deletes them in afterEach, even if the test failed, so the test database never accumulates state. The discipline that separates senior implementations: **cleanup runs regardless of test outcome**, because skipping cleanup on failure leaves orphan data that causes future tests to fail in confusing ways. Branded IDs make orphan-data bugs less likely in the first place by preventing the

wrong-ID-passed-to-wrong-endpoint class entirely.

INTERVIEW STRATEGY

Show typed factories + cleanup as one system. Lead with the builder pattern returning branded IDs (callback to S1 Q14 branded types). Then make the cleanup-runs-regardless-of-outcome rule the senior signal because orphan-data bugs are real. The pitfall is cleanup skipped on failure. The follow-up is often 'what about parallel tests creating conflicting data?', answer factory generates unique data per call (faker + worker index) so parallel tests never collide on natural keys.

How to phrase it

> *A test-data factory is a class or function that creates an entity via API and returns a typed reference, while tracking the entity for cleanup at test teardown. I combine three patterns. First, branded types from earlier in the kit: UserId, OrderId, ProductId. The factory's create method returns the branded type, not a raw string, so downstream code can't accidentally pass a UserId where an OrderId was expected. Second, a fluent builder API that reads like the scenario rather than the API call: new UserBuilder dot asAdmin dot withEmail dot create request. The test reads as the intention; the implementation details are hidden in the builder. Third, automatic cleanup. I combine the factory with a worker-scoped or test-scoped cleanup fixture that records IDs created during the test and deletes them in afterEach. The discipline that separates a senior implementation, and the framing that lands in interviews, is cleanup runs regardless of test outcome. Skipping cleanup on failure leaves orphan data that causes future tests to fail in confusing ways, a test creates a user, fails before deleting it, the next test tries to create the same user and gets a 409 conflict, and now we're debugging a flake that has nothing to do with the actual code change. So the cleanup fixture's teardown runs even on test failure, and I wrap each delete in try-catch so one failed deletion doesn't prevent the others. Branded IDs make orphan-data bugs less likely in the first place by preventing the wrong-ID-passed-to-wrong-endpoint bug class entirely. The follow-up usually probes this, and the answer is about parallel tests creating conflicting data: the factory generates unique data per call using faker plus the worker index, so parallel tests never collide on natural keys like email addresses or slugs. The risk is cleanup that skips on failure: the test database accumulates state and the suite becomes increasingly unstable over weeks.*

Key points to hit

(1) Factory returns branded types (UserId, OrderId), no ID-mixup bugs. **(2)** Fluent builder reads like the scenario: asAdmin().withEmail().create(). **(3)** Cleanup fixture tracks IDs; deletes in afterEach, even on test failure. **(4)** Cleanup wrapped in try-catch so one failure doesn't block other deletions. **(5)** Expect next: faker + worker index for unique data, no parallel collisions. **(6)** What loses the point: cleanup that skips on failure, orphan data flakes future tests.

CODE

```
type UserId = string & { __brand: 'UserId' };

export class UserBuilder {
private data: Partial<{ email: string; isAdmin: boolean }> = {};
asAdmin() { this.data.isAdmin = true; return this; }
withEmail(email: string) { this.data.email = email; return this; }

async create(request: APIRequestContext, registry: Set<string>): Promise<UserId> {
const email = this.data.email ?? `test+${faker.string.uuid()}@local`;
const res = await request.post('/api/users', { data: { email, ...this.data } });
expect(res.status()).toBe(201);
const id = (await res.json()).id as UserId;
registry.add(id);
return id;
}
}

// Cleanup fixture: runs regardless of test outcome
export const test = base.extend<{ cleanup: Set<string>; request: APIRequestContext }>({
cleanup: async ({ request }, use) => {
const ids = new Set<string>();
await use(ids);
for (const id of ids) {
try { await request.delete(`/api/users/${id}`); } catch { /* keep going */ }
}
},
});
```

Q 110 LEVEL · Mid to Senior

How do you test cursor-based vs offset-based pagination?

CONCEPT

Two pagination strategies, each with different test concerns. **Offset-based** (page=1&pageSize;=20): test that page count matches total/pageSize, that page boundaries return contiguous data, that out-of-range pages return empty rather than 500, and that adding new data during pagination doesn't shift results, the classic offset bug where the same item appears on page 2 and page 3 because something was inserted in between. **Cursor-based** (cursor=abc123&limit;=20): test that following the next-cursor link traverses the full set without duplicates or gaps, that an invalid cursor returns 400 not 500, that cursors are opaque (test code should never construct them, only echo them back). The general discipline: **test boundary conditions**, first page, last page, beyond last page, empty result set, page size of 1, page size of max. Pagination bugs almost always live at the boundaries, not in the middle.

INTERVIEW STRATEGY

They are after pagination depth, not just happy-path. Lead with the two strategies and their distinct failure modes, offset has the insertion-shift bug, cursor has the opaque-construction risk. The 'pagination bugs live at boundaries' rule marks senior judgement. The trap is treating offset and cursor as interchangeable tests. Expect the follow-up ‘what's the classic offset bug?’, answer same item appearing on two pages because data inserted between requests shifted the offsets.

How to phrase it

> *Two pagination strategies, each with different test concerns and different failure modes worth knowing. Offset-based pagination, with page and page-size query parameters, has these test concerns: page count matches total divided by page size, page boundaries return contiguous non-duplicate data, out-of-range pages return empty rather than 500, and crucially, and this is the answer to the follow-up about the classic offset bug, adding new data during pagination doesn't shift results. The classic bug: I fetch page two, then before fetching page three, something is inserted into the data set, the offsets shift, and an item that was on page two now appears on page three. The user sees a duplicate. Hard to reproduce, easy to test deterministically by inserting a row between paginated fetches. Cursor-based pagination, with cursor and limit, has different concerns. Test that following the next-cursor link traverses the full set without duplicates or gaps, that an invalid cursor returns 400 not 500, and that cursors are opaque, test code should never construct them, only echo them back from previous responses. Constructing cursors in test code couples the test to the cursor format, which is meant to be implementation-defined and subject to change. The general discipline I would lead with, because it's the rule that lands in interviews, is test boundary conditions. First page, last page, beyond last page, empty result set, page size of one, page size at maximum. Pagination bugs almost always live at the boundaries, the middle of a pagination is usually fine because it follows the same logic as every other middle. The edges are where off-by-one errors, infinite-loop-on-empty, and what-if-the-cursor-is-missing bugs hide. The trap is treating offset and cursor as interchangeable in tests; they have different failure modes and the tests should reflect that.*

Key points to hit

(1) Offset: page count, boundaries, out-of-range, insertion-shift bug. **(2)** Cursor: traversal completeness, invalid cursor -> 400 not 500, opaque rule. **(3)** NEVER construct cursors in test code: only echo from previous response. **(4)** Test boundary conditions: first/last/beyond, empty, size=1, size=max. **(5)** They usually ask next: classic offset bug is the same item on two pages after insert. **(6)** The common miss: offset and cursor tested identically, different failure modes.

CODE

```
// Offset-based: catch the insertion-shift bug
test('offset pagination is stable across inserts', async ({ request, cleanup }) => {
const page1 = await (await request.get('/api/items?page=1&pageSize=20')).json();
await new ItemBuilder().create(request, cleanup); // insert between fetches
const page2 = await (await request.get('/api/items?page=2&pageSize=20')).json();

const ids1 = new Set(page1.items.map((i: any) => i.id));
const dupes = page2.items.filter((i: any) => ids1.has(i.id));
expect(dupes).toEqual([]); // no duplicates across pages
});

// Cursor-based: echo cursor, don't construct
test('cursor pagination traverses the full set', async ({ request }) => {
const seen = new Set<number>();
let cursor: string | undefined = undefined;
for (;;) {
const url = cursor ? `/api/items?cursor=${cursor}&limit=20` : '/api/items?limit=20';
const json = await (await request.get(url)).json();
for (const item of json.items) {
expect(seen.has(item.id)).toBe(false); // no duplicates
seen.add(item.id);
}
if (!json.nextCursor) break;
cursor = json.nextCursor; // echo, never construct
}
});
```

Q 111 LEVEL · Senior

How do you test a microservice in isolation when it depends on other services?

CONCEPT

A microservice typically depends on downstream services for data and side effects, so isolation testing requires mocking the downstream calls while testing the service under test live. The pattern: stand up the service under test in a controlled environment (a test container, a feature-branch deployment), **intercept its outbound requests** to other services either via a per-test proxy or via the service's configurable HTTP client base URL pointed at a mock server (WireMock, MSW, a custom Express stub), and run the test suite against the service's own API while the downstream mocks return controlled responses. The discipline that distinguishes senior implementations: **record the contract between services** and verify the mock matches the real contract, otherwise you're testing against fictional behaviour. Contract tests (Pact, Spring Cloud Contract) are the proper tool here, they share a contract file between the consumer's mock and the provider's real service, so when the provider changes, the consumer's tests fail before integration.

INTERVIEW STRATEGY

They are looking to see real microservice testing patterns. Lead with the mock-downstream + test-service-under-test-live structure, then make contract testing the senior pattern because it's what prevents the mock-drift bug. The Pact/Spring Cloud Contract reference is the depth signal. The thing to avoid is mocking everything and testing nothing. They usually follow with ‘how do you keep the mocks honest?’, answer contract tests with shared contract files between consumer and

provider.

How to phrase it

> *A microservice typically depends on downstream services for data and side effects, so isolation testing requires mocking the downstream calls while testing the service under test live. The pattern is to stand up the service under test in a controlled environment, a test container, a feature-branch deployment, an ephemeral namespace in a Kubernetes test cluster, and intercept its outbound requests to other services. Two ways to do that interception. Either a per-test proxy that sits between the service and its dependencies, or more commonly the service's configurable HTTP client base URL pointed at a mock server like WireMock, MSW, or a custom Express stub. Then I run the test suite against the service's own API while the downstream mocks return controlled responses for each scenario. This isolates the service under test: every test result reflects behaviour of that service, not behaviour of the downstream collection. The discipline that distinguishes senior implementations from junior ones, and the framing I would emphasise, is record the contract between services and verify the mock matches the real contract. Otherwise I'm testing against fictional behaviour: the mock says one thing, the real downstream says another, the test passes, and the integration breaks in production. Contract tests are the proper tool here: Pact and Spring Cloud Contract being the established ones. They share a contract file between the consumer's mock and the provider's real service, so when the provider changes their API the consumer's tests fail before integration, not after deployment. That's also my answer to the follow-up about how I keep the mocks honest. Contract tests with shared contract files between consumer and provider, the contract is the single source of truth, the mock derives from it, and divergence fails the build. The risk is mocking everything without verifying the contract, which gives you a suite of green tests that happen against a fictional version of the system.*

Key points to hit

(1) Stand up the service under test live; mock its downstream dependencies. **(2)** Intercept via proxy or by pointing HTTP client base URL at a mock server. **(3)** Tools: WireMock, MSW, or a custom Express stub, controlled responses. **(4)** Contract tests (Pact, Spring Cloud Contract) prevent mock drift. **(5)** The next question: shared contract files between consumer and provider keep mocks honest. **(6)** Avoid: mocking everything without verifying contract, tests pass against fiction.

CODE

```
// Service under test runs at http://localhost:8080 (real, live)
// Its downstream user-service is configured via env var to use the mock
// USER_SERVICE_URL=http://localhost:9000 (WireMock or MSW)

test('billing-service handles user-service 404 gracefully', async ({ request }) => {
// Configure the downstream mock for this scenario
await fetch('http://localhost:9000/__admin/mappings', {
method: 'POST',
body: JSON.stringify({
request: { method: 'GET', urlPattern: '/users/9999' },
response: { status: 404, body: '{"error":"not found"}' },
}),
});

// Test the service under test: live, real code path
const res = await request.post('http://localhost:8080/invoices', {
data: { userId: 9999, amount: 100 },
});
expect(res.status()).toBe(404); // handles downstream 404
expect((await res.json()).error).toBe('user not found');
});

// To keep mocks honest: Pact contract file shared between consumer and provider
```

Q 112 LEVEL · Mid

What does storageState actually store, and how do you save and load it?

CONCEPT

storageState captures the part of the browser session that proves who you are: cookies and **localStorage**, serialised to a JSON file. You save it with **context.storageState({ path: 'auth/user.json' })** after a successful login, and load it with use: { storageState: 'auth/user.json' } in the config or per project, so the test starts already authenticated with no login step. The caveat worth knowing is sessionStorage is not captured, it is per-tab and deliberately not persisted, so if your app keeps auth there (uncommon, but it happens), storageState alone will not restore the session and you have to seed sessionStorage yourself via an init script. For the common case, cookies or a localStorage token, it just works, and it is the single biggest lever for a fast suite because login happens once, not once per test.

INTERVIEW STRATEGY

They want to know you understand what is actually being persisted, not just the API call. Name the three buckets, cookies and localStorage are saved, sessionStorage is not, because the sessionStorage caveat is the detail that shows you have actually debugged a session that did not restore. Then give the save-and-load pair cleanly. The next question is normally ‘what if auth lives in sessionStorage?’, answer that you seed it with an addInitScript, since storageState will not cover it.

How to phrase it

> *storageState captures the bits of the browser session that prove identity, cookies and localStorage, and writes them to a JSON file. I save it after a successful login with context.storageState and a path, typically in a setup project or globalSetup, and then every test loads it via use storageState pointing at that file, so the test opens already logged in with no UI login step. That is the single biggest lever for a fast suite, because the login cost is paid once rather than once per test. The caveat I would make sure to mention, because it is the thing that catches people out, is that sessionStorage is not captured. It is per-tab and intentionally not persisted, so if an app keeps its auth token in sessionStorage rather than localStorage or a cookie, loading storageState will not restore the session and the test lands on the login page wondering why. For the common case, a cookie or a localStorage token, it just works. And that sets up the follow-up: if auth genuinely lives in sessionStorage, I seed it myself with an addInitScript that writes the token into sessionStorage before the page loads, because storageState alone cannot do it. Knowing exactly what is and is not in that file is what lets me debug an auth-reuse failure in a minute instead of an hour.*

Key points to hit

(1) storageState saves cookies + localStorage to a JSON file. **(2)** Save: context.storageState({ path }); load: use { storageState }. **(3)** Caveat: sessionStorage is NOT captured, it is per-tab, not persisted. **(4)** Login once, not per test: the biggest single lever for suite speed. **(5)** Asked next: sessionStorage auth needs addInitScript to seed the token. **(6)** The easy mistake: assuming storageState covers everything, sessionStorage is the gap.

CODE

```
// Save after login: in auth.setup.ts (a setup project)
import { test as setup } from '@playwright/test';

setup('authenticate as user', async ({ page }) => {
await page.goto('/login');
await page.getByLabel('Email').fill(process.env.USER_EMAIL!);
await page.getByLabel('Password').fill(process.env.USER_PASSWORD!);
await page.getByRole('button', { name: 'Sign in' }).click();
await page.waitForURL('/dashboard');
// Cookies + localStorage written to disk; sessionStorage is NOT
await page.context().storageState({ path: 'auth/user.json' });
});

// Load: in playwright.config.ts
// use: { storageState: 'auth/user.json' }

// If auth lives in sessionStorage, seed it (storageState won't):
// await context.addInitScript(t => sessionStorage.setItem('token', t), token);
```

Q 113 LEVEL · Mid to Senior

storageState reuse vs logging in per test: what's the trade-off, and when is logging in per test the right call?

CONCEPT

storageState reuse logs in once, saves the session, and every test loads it, so the suite pays the login cost a single time and runs fast and stable. **Logging in per test** drives the real login flow at the start of every test, which is slow (five to eight seconds each, multiplied across hundreds of tests) and adds a shared failure point, but it exercises the login itself every time. The default for the vast majority of tests is reuse, you do not want five hundred tests re-validating the login form. The case for logging in per test is narrow and specific: when the login flow is the thing under test (testing the login form, MFA, error states), when a test needs a genuinely fresh, unique session (brand-new user each run, or testing concurrent-session limits), or when the auth token is so **short-lived** that a saved state would already be expired. The principle: reuse by default, log in per test only when the session itself is part of what you are verifying.

INTERVIEW STRATEGY

This is a judgement question, so the interviewer wants the default and the exceptions, not a blanket rule. State clearly that reuse is the default for speed and stability, then name the narrow cases where per-test login is correct, testing the login flow itself, needing a fresh unique session, very short-lived tokens. Saying 'always log in per test for realism' is the trap; it signals you have not run a large suite. Crispness comes from leading with the heuristic 'reuse by default, log in when the session is what you are testing' and then giving two or three concrete exceptions.

How to phrase it

> *The honest answer is that it is a default-plus-exceptions decision, and I would lead with the default: reuse storageState for the vast majority of tests. You log in once in a setup project, save the session, and every test loads it, so the suite pays the login cost a single time and runs fast and stable. Logging in per test means driving the real login flow at the start of every test, five to eight seconds each, and across a few hundred tests that is tens of minutes of pure auth overhead, plus you have turned the login form into a shared dependency that flakes the whole suite when it has a bad day. So I do not re-validate the login form five hundred times. Now the exceptions, because that is the real question. I log in per test in three situations. One, when the login flow is the thing under test, the login form itself, MFA, lockout and error states; there I obviously cannot reuse a saved session because the logging-in is the assertion. Two, when a test genuinely needs a fresh, unique session, a brand-new user created per run, or testing concurrent-session or single-sign-out limits where two saved copies of the same session would defeat the point. Three, when the token is so short-lived that a saved state would already be expired by the time the test runs. So the heuristic I would close on is reuse by default, and log in per test only when the session itself is part of what I am verifying. The trap is logging in everywhere in the name of realism, it buys almost no extra coverage and costs you a slow, fragile suite.*

Key points to hit

(1) Reuse: login once, fast and stable, the default for most tests. **(2)** Per-test login: slow (5-8s each) and a shared failure point. **(3)** Log in per test when the login flow is the thing under test. **(4)** Also when you need a fresh unique session, or the token is very short-lived. **(5)** Heuristic: reuse by default; log in only when the session is what you verify. **(6)** Trap: per-test login everywhere 'for realism', slow, fragile, no real gain.

CODE

```
// Default: reuse saved state (fast, stable)
// playwright.config.ts: use: { storageState: 'auth/user.json' }
test('dashboard shows the user widgets', async ({ page }) => {
await page.goto('/dashboard'); // already logged in
await expect(page.getByTestId('widgets')).toBeVisible();
});

// Exception: the login flow IS the test (no reuse)
test.use({ storageState: { cookies: [], origins: [] } }); // start logged out
test('login rejects a wrong password', async ({ page }) => {
await page.goto('/login');
await page.getByLabel('Email').fill('user@example.com');
await page.getByLabel('Password').fill('wrong-password');
await page.getByRole('button', { name: 'Sign in' }).click();
await expect(page.getByRole('alert')).toHaveText(/invalid credentials/i);
});
```

Q 114 LEVEL · Mid to Senior

How do you handle multiple authenticated roles (admin vs standard user) with separate storageState files and projects?

CONCEPT

Give each role its **own storageState file** and wire it through projects. A setup project logs in as each role and saves to **auth/admin.json**, **auth/user.json**, and so on. Then you define one Playwright project per role, each with use: { storageState: 'auth/<role>.json' } and dependencies: ['setup'] so the login always runs first. Role-specific tests go in role-specific folders or are tagged, so the admin project only runs admin tests and so on. The detail that scales: parameterise the login so adding a new role is a new entry in a roles array, not a copy-pasted setup test. For a single test that needs two roles at once, say an admin approving and a user submitting, you spin up a second context from the other role's storageState inside the test rather than fighting the project model. Separate files and projects keep the roles isolated and let the runner pick the right session for each test automatically.

INTERVIEW STRATEGY

The point of this one is to see you scale auth past a single user without copy-paste. Lead with the structure: one storageState file per role, one project per role, each depending on a setup project that does the logins. Mention parameterising the login over a roles array as the bit that keeps it maintainable. The follow-up is usually ‘what about a test that needs two roles together?’, answer that you create a second context from the other role's saved state inside that test, rather than bending the project model to do it.

How to phrase it

> *The clean way is one storageState file per role, wired through projects. I have a setup project that logs in as each role and saves the session, auth/admin.json, auth/user.json, maybe auth/viewer.json. Then I define a Playwright project per role, each with its own use storageState pointing at the right file, and each with dependencies set to the setup project so the logins always run first and the saved state is fresh. Role-specific tests live in role-specific folders, or I tag them, so the admin project runs only the admin tests and the user project runs only the user ones, each already authenticated as the right person. The detail that keeps this from rotting as roles grow is to parameterise the login, loop a setup over a roles array with credentials, so adding a new role is one new entry, not a copy-pasted setup test that drifts out of sync. That is the maintainability point I would stress. For the follow-up about a single test that genuinely needs two roles at once, an admin approving something a standard user just submitted, I do not fight the project model. I let the test run as one role and spin up a second browser context from the other role's storageState inside the test, so I have an admin page and a user page side by side in the same test. So: separate files and projects for isolation and automatic session selection, parameterised setup for scale, and a second context for the rare two-role test.*

Key points to hit

(1) One storageState file per role: auth/admin.json, auth/user.json. **(2)** One project per role, each with its own use { storageState }. **(3)** Each role project sets dependencies: ['setup'] so logins run first. **(4)** Parameterise login over a roles array: new role = new entry, not copy-paste. **(5)** Two roles in one test: open a second context from the other role's state. **(6)** Where this goes wrong: copy-pasted per-role setup tests that drift out of sync.

CODE

```
// playwright.config.ts: setup project + one project per role
projects: [
{ name: 'setup', testMatch: /auth\.setup\.ts/ },
{
name: 'admin',
testMatch: /.*\.admin\.spec\.ts/,
use: { storageState: 'auth/admin.json' },
dependencies: ['setup'],
},
{
name: 'user',
testMatch: /.*\.user\.spec\.ts/,
use: { storageState: 'auth/user.json' },
dependencies: ['setup'],
},
],

// auth.setup.ts: parameterise over roles, no copy-paste
const roles = [
{ name: 'admin', email: process.env.ADMIN_EMAIL!, file: 'auth/admin.json' },
{ name: 'user', email: process.env.USER_EMAIL!, file: 'auth/user.json' },
];
for (const role of roles) {
setup(`log in as ${role.name}`, async ({ page }) => {
await loginAs(page, role.email);
await page.context().storageState({ path: role.file });
});
}
```

Q 115 LEVEL · Mid

How do you test logged-out behaviour and reset or clear authentication state between tests?

CONCEPT

When the whole suite shares a logged-in storageState, a logged-out test has to override that state to empty. The clean way is test.use({ storageState: { cookies: [], origins: [] } }) on that test or describe block, which starts it with no session regardless of the project default, ideal for testing redirects to login, public pages, and unauthenticated-access guards. To clear state mid-test, you either call **context.clearCookies()** (and clear localStorage via an init or evaluate) or, more cleanly, **open a fresh context**, since a new context is a clean session by definition. The discipline that prevents flaky cross-talk: tests should not mutate the shared storageState file, it is loaded read-only into each context, so logging out in one test does not leak into another. If you need a logged-out variant of an otherwise-authenticated suite, a small **logged-out project** with empty storageState is tidier than overriding test by test.

INTERVIEW STRATEGY

You are being asked to show you can flip off the shared auth deliberately. Lead with overriding storageState to empty via test.use, that is the idiomatic way to make one test logged-out when the project default is logged-in. Then cover clearing mid-test: clearCookies plus localStorage, or better, a fresh context. The reassuring detail is that the shared storageState file is loaded read-only per

context, so logging out in one test cannot leak into another, that addresses the unspoken worry behind the question. For a whole batch of logged-out tests, mention a dedicated logged-out project as the tidy option.

How to phrase it

> *The key thing to know is that when the suite shares a logged-in storageState, a logged-out test has to override that state to empty rather than trying to log out. The idiomatic way is test.use with storageState set to empty cookies and empty origins on that test or describe block, which starts it with no session regardless of the project default. That is exactly what I want for testing redirects to the login page, public pages, and unauthenticated-access guards, the test simply begins as an anonymous visitor. If I need to clear state in the middle of a test, say, verify what happens right after a logout, I either call context.clearCookies and clear localStorage, or, cleaner, I open a fresh context, because a new context is a clean session by definition and I do not have to remember every storage bucket. The discipline that keeps this from causing flake, and the reassurance I would give the interviewer, is that the shared storageState file is loaded read-only into each context. Logging out or clearing cookies in one test mutates only that context, not the file on disk, so it cannot leak into another test or another worker. And if I have a whole batch of logged-out tests, I find a small dedicated logged-out project with empty storageState tidier than sprinkling test.use overrides across files. So: override to empty for logged-out tests, fresh context to clear mid-test, and trust that the shared file is read-only so nothing leaks.*

Key points to hit

(1) Override the shared state: test.use({ storageState: { cookies: [], origins: [] } }). **(2)** Use it for login redirects, public pages, and unauthenticated guards. **(3)** Clear mid-test: context.clearCookies() + localStorage, or open a fresh context. **(4)** Shared storageState is loaded read-only per context, logout doesn't leak. **(5)** A dedicated logged-out project beats per-test overrides for a whole batch. **(6)** What loses the point: trying to mutate the shared state file, override to empty instead.

CODE

```
// Logged-out test: override the project's shared state to empty
import { test, expect } from '@playwright/test';

test.describe('unauthenticated access', () => {
test.use({ storageState: { cookies: [], origins: [] } });

test('redirects to login when not signed in', async ({ page }) => {
await page.goto('/dashboard');
await expect(page).toHaveURL(/\/login/);
});
});

// Clear state mid-test: cookies + localStorage
test('after logout the session is gone', async ({ page, context }) => {
await page.goto('/dashboard');
await context.clearCookies();
await page.evaluate(() => localStorage.clear());
await page.goto('/dashboard');
await expect(page).toHaveURL(/\/login/); // session truly cleared
});
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Which Playwright fixture provides a full HTTP client with the same expect API as UI tests?

A) page
B) browser
C) context
D) request

2. What is the elegant way to authenticate API requests that reuses browser auth?

A) Hardcode a bearer token in each test
B) Reuse the storageState saved by the setup project
C) Log in through the UI before every API call
D) Disable authentication in staging

3. Why run API contract tests as the first stage in CI?

A) They are fast (no browser) and catch breaking schema changes before UI tests
B) They need a browser, so run them early
C) They replace UI tests entirely
D) They cannot run in parallel

4. In the API-setup-plus-UI-verification pattern, why create the test data via API?

A) The UI cannot create data
B) API creation skips validation
C) It is far faster and keeps the test focused on verifying display, not creation
D) It avoids using fixtures

5. Why must GraphQL mocks match on operation name rather than URL?

A) GraphQL uses GET requests
B) All GraphQL requests share one endpoint, so a URL match catches every query
C) Operation names are faster to parse
D) URLs are not available in route handlers

Section 7 · Answer key

#	Answer	Why and where to revisit
1	D) request	The request fixture is a built-in HTTP client, so API and UI tests share one framework, runner and assertion API. *Revisit Section 7.*
2	B) Reuse the storageState saved by the setup project	Reusing storageState means one auth source feeds both UI and API tests, with no separate token management. Note the standalone request fixture is isolated and does not inherit browser cookies, so you either point its context at the saved storageState or use page.request. *Revisit Section 7.*
3	A) They are fast (no browser) and catch breaking schema changes before UI tests	Contract tests need no browser, so they fail fast on a breaking field change before any slower UI test runs. *Revisit Section 7.*
4	C) It is far faster and keeps the test focused on verifying display, not creation	API setup (~200ms vs ~30s via UI) is faster and lets the test verify the UI displays data, not re-test the form. *Revisit Section 7.*
5	B) All GraphQL requests share one endpoint, so a URL match catches every query	Because every GraphQL query hits the same endpoint, you inspect the body and match the operation name to mock one query. *Revisit Section 7.*

SECTION 08

Assertions and the Expect API

Assertions are where flaky tests are won or lost, so interviewers probe whether you reach for the retrying web-first assertions by default and know the specialised tools for harder cases. These fourteen questions cover the web-first assertion catalogue and soft assertions, visual testing with toHaveScreenshot, expect.poll for eventually consistent systems, asserting on API responses, and extending expect with custom matchers.

In this section

- The web-first assertion catalogue: all retry automatically.
- Soft assertions to collect every failure in one run, not just the first.
- Visual testing with toHaveScreenshot: masking dynamic content, disabling animations.
- expect.poll with progressive backoff for eventually consistent systems.
- API-response assertions: toBeOK, toMatchObject with expect.any().
- Custom matchers (toBeLoggedIn) for readable, reusable domain assertions.
- expect.configure for a named slowExpect or softExpect, instead of a global timeout bump.

Q 116 LEVEL · Junior to Mid

What are all the web-first assertions in Playwright, and what are soft assertions?

CONCEPT

Web-first assertions automatically retry until the condition is met or the timeout expires, the correct tool for all DOM state. The catalogue groups cleanly: visibility (**toBeVisible**, toBeHidden), state (toBeEnabled, toBeDisabled, toBeChecked), content (**toHaveText**, toContainText, toHaveValue), count (**toHaveCount**), attribute (toHaveAttribute, toHaveClass), and page (toHaveURL, toHaveTitle). Soft assertions, expect.soft, collect all failures before throwing instead of stopping at the first. In a form-validation test checking ten field errors, regular assertions stop at the first failure and you see one error; soft assertions run all ten and report every failure at once, which is exactly what you want for a complete regression picture.

INTERVIEW STRATEGY

They need to confirm you default to retrying assertions and know when to gather failures rather than fail fast. Group the catalogue rather than listing it flat, then spotlight soft assertions with the ten-field-form example, that is the bit that shows practical use. The pitfall is mixing in non-retrying checks like isVisible as if they were assertions. A common follow-up is 'when would you use soft vs hard assertions?', answer soft when you want the full picture of what broke, hard when a failure should halt the test immediately.

How to phrase it

> *Web-first assertions are the ones that retry automatically until the condition is met or the timeout expires, and they are my default for anything touching DOM state. Rather than reel off every name, I group them: visibility with toBeVisible and toBeHidden, state with toBeEnabled and toBeChecked, content with toHaveText and toContainText, count with toHaveCount, attributes with toHaveAttribute, and page-level with toHaveURL and toHaveTitle, and they all retry, which is the whole point. The feature I highlight in every interview is soft assertions, expect.soft, which collect all failures before throwing instead of stopping at the first. The example I give is a form-validation test that checks ten field error messages. With regular assertions the first failure stops the test and I only see one broken field; with soft assertions all ten run and I get every failure in one report. That answers the follow-up about soft versus hard: I use soft when I want the complete picture of what broke, which is most valuable in regression, and hard when a failure genuinely should halt the test because everything after it is meaningless. So the headline is: retrying assertions by default, soft assertions when I want the full damage report rather than just the first casualty.*

Key points to hit

(1) Web-first assertions retry until condition met or timeout. **(2)** Catalogue: visibility, state, content, count, attribute, page. **(3)** expect.soft collects all failures before throwing. **(4)** Ten-field form: soft assertions report every broken field, not just the first. **(5)** Follow-up: soft for the full picture, hard to halt immediately. **(6)** The common miss: treating one-shot isVisible as an assertion.

CODE

```
// All retry automatically
await expect(page.locator('#status')).toHaveText('Order Confirmed');
await expect(page.locator('#items')).toHaveCount(3);
await expect(page).toHaveURL('/dashboard');

// Soft assertions: collect every failure in one run
await expect.soft(page.getByTestId('name')).toHaveText('Alice');
await expect.soft(page.getByTestId('email')).toHaveText('alice@test.com');
await expect.soft(page.getByTestId('role')).toHaveText('Admin');
// all three run; the test reports all failures at once
```

Q 117 LEVEL · Mid

How do you use toHaveScreenshot() for visual regression testing?

CONCEPT

toHaveScreenshot captures a screenshot and compares it to a stored baseline, on first run it creates the baseline, on later runs it compares pixel-by-pixel with a configurable threshold (**maxDiffPixelRatio**). Two options make it usable on real apps. **mask** hides dynamic content, prices, stock counts, timestamps, user-specific data, that would otherwise cause false failures, so you test structure and styling rather than volatile values. animations: 'disabled' stops animation frames from producing spurious pixel diffs. Baselines are committed to the repo alongside the tests; when a designer intentionally changes a component, they regenerate the baseline with the update-snapshots flag. Treat that update as a reviewable change, not an automatic overwrite.

INTERVIEW STRATEGY

The question is checking whether you can make visual testing survive a real, dynamic app instead of failing constantly. Lead with mask and animations disabled: those two options are what separate a usable visual suite from a noise machine. Mention committing baselines and updating them deliberately. The failure mode is screenshotting a page full of dynamic data and getting endless false failures. The follow-up is often ‘how do you handle intentional design changes?’, answer regenerate the baseline with update-snapshots and review the diff.

How to phrase it

> *toHaveScreenshot captures a screenshot and compares it against a stored baseline, it creates the baseline on the first run and compares pixel-by-pixel after that, with a configurable threshold. But the part that actually makes visual testing work on a real application is the options, so that is what I would emphasise. The mask option is essential: prices, stock counts, timestamps and user-specific data change between runs and would cause constant false failures, so I mask all of that and test only the structural layout and styling. I also set animations to disabled, because animation frames cause spurious pixel diffs depending on exactly when the screenshot fires. Without those two options, visual tests become a noise machine that everyone learns to ignore, which is the trap. I commit the baseline screenshots to the repo alongside the tests, and that sets up the follow-up about intentional design changes: when a designer deliberately changes a component, they regenerate the baseline with the update-snapshots flag, and I treat that as a reviewable change in the pull request, not an automatic overwrite, so a reviewer can see exactly what visually changed. That combination, masking, disabled animations, and reviewed baseline updates, is what keeps a visual suite trustworthy instead of flaky.*

Key points to hit

(1) Compares to a committed baseline with a configurable maxDiffPixelRatio. **(2)** mask hides dynamic content (prices, timestamps) to avoid false failures. **(3)** animations: 'disabled' stops spurious pixel diffs. **(4)** Baselines live in the repo; update deliberately with update-snapshots. **(5)** Expect next: intentional design change, regenerate baseline, review the diff. **(6)** Avoid: screenshotting dynamic data, endless false failures.

CODE

```
test('product card matches the design', async ({ page }) => {
await page.goto('/products/macbook-pro');
await expect(page.getByTestId('product-card')).toHaveScreenshot('product-card.png', {
mask: [page.locator('.price-badge'), page.locator('.stock-count')],
maxDiffPixelRatio: 0.02,
});
});

await expect(page).toHaveScreenshot('dashboard-full.png', {
fullPage: true,
animations: 'disabled',
});
```

Q 118 LEVEL · Mid to Senior

How do you use expect.poll() for eventually consistent systems?

CONCEPT

expect.poll repeatedly calls a function and asserts on its return value, with configurable backoff intervals and a timeout. It is the right tool for asynchronous, eventually consistent systems where state takes time to propagate, a UI action enqueues a message, a worker processes it, and the database updates some seconds later. You poll the status API at **progressive intervals** (1s, then 2s, then 5s, then 10s) until it reaches the expected value, which is faster than fixed-interval polling and far more readable than a hand-written while loop. The anti-pattern it replaces is the **hard sleep**: a

fixed wait is either too short and flaky or too long and slow, whereas poll returns the instant the condition is true.

INTERVIEW STRATEGY

This one probes whether you handle async propagation correctly or paper over it with sleeps. Define expect.poll, then anchor on the event-driven example, action, queue, worker, database, because that is where it earns its place. Stress progressive backoff as both faster and more readable than a while loop. The trap, and the answer-killer, is suggesting a hard sleep. Expect the follow-up 'why not just sleep and check?', answer that poll returns the moment the state is right, so it is fast when the system is fast and patient when it is slow.

How to phrase it

> *expect.poll repeatedly calls a function and asserts on its return value, with configurable backoff intervals and a timeout, and it is my preferred tool for eventually consistent systems. The scenario where it matters is event-driven architecture: a UI action enqueues a message, a worker processes it asynchronously, and the database updates a few seconds later, so the UI will not reflect the change immediately. Instead of guessing how long that takes, I poll the status API with progressive backoff, check at one second, then two, then five, then ten, until it reaches the expected value. That progressive backoff is both faster than fixed-interval polling, because it does not hammer the API, and far more readable than a hand-written while loop with a counter. This is also my answer to the follow-up about why not just sleep and check: a hard sleep is the anti-pattern here, because a fixed wait is either too short, so it is flaky when the system is slow that run, or too long, so every single run pays the worst-case wait. expect.poll returns the instant the condition is true, so it is fast when the system is fast and patient only when it genuinely needs to be. For anything eventually consistent, poll with backoff is the correct, deterministic solution.*

Key points to hit

(1) expect.poll calls a function repeatedly and asserts on the result. **(2)** Right tool for async, eventually consistent systems (queue -> worker -> DB). **(3)** Progressive backoff: faster than fixed polling, clearer than a while loop. **(4)** Returns the instant the condition is true: fast and patient as needed. **(5)** They usually ask next: why not sleep, fixed waits are flaky if short, slow if long. **(6)** The easy mistake: suggesting a hard sleep for eventual consistency.

CODE

```
// Poll the status API until the order is confirmed
await expect.poll(async () => {
const res = await page.request.get('/api/orders/42');
return (await res.json()).status;
}, {
message: 'Order should reach Confirmed within 30s',
timeout: 30_000,
intervals: [1_000, 2_000, 5_000, 10_000], // progressive backoff
}).toBe('confirmed');
```

Q 119

LEVEL · Mid

How do you assert on API responses using Playwright's expect?

CONCEPT

Playwright provides response-level assertions for the APIResponse object. **toBeOK** checks a 2xx status; you can also assert on the exact status and on headers. For the body, **toMatchObject** combined with **expect.any()** and expect.stringMatching() does flexible schema validation: assert that an id is any number and an email matches a format regex, rather than pinning exact values for generated fields like IDs and timestamps. This makes tests resilient to data changes while still validating the contract, you check the type and the format rule, not a specific test value. The same idea extends to arrays with toHaveProperty and length checks. Asserting a format regex for an email is more valuable than asserting one literal address, because it validates the rule the API must always obey.

INTERVIEW STRATEGY

What scores here is showing resilient assertions, not brittle exact-value checks. Centre on toMatchObject with expect.any() and stringMatching, type-and-format validation rather than literal values. The line that lands: a format regex for email is more valuable than a specific address because it validates the rule. The trap is asserting exact IDs or timestamps, which break on every data change. They usually follow with 'how do you assert on generated fields you cannot predict?', answer expect.any for type, stringMatching for format.

How to phrase it

> *Playwright has response-level assertions for the APIResponse object, so I lead with those: toBeOK for a 2xx, plus exact status and header checks where they matter. But the part worth dwelling on is how I assert on the body, because that is where people make tests brittle. I use toMatchObject combined with expect.any and expect.stringMatching to do flexible schema validation, I assert that an id is any number and an email matches a format regex, rather than pinning the exact id or the exact timestamp. That answers the follow-up about asserting on generated fields I cannot predict: I do not try to predict them, I assert their type with expect.any and their format with stringMatching. The principle, and the line I would actually say, is that asserting a format regex for an email is more valuable than asserting one literal address, because it validates the rule the API must always obey, not one specific test value that happens to be in the database today. That keeps the test resilient to data changes while still genuinely validating the contract. The trap I am avoiding is asserting exact IDs and timestamps, which makes the test fail every time the data changes for reasons that have nothing to do with a real bug. The same idea extends to arrays with toHaveProperty and length checks.*

Key points to hit

(1) toBeOK for 2xx; assert exact status and headers where needed. **(2)** toMatchObject + expect.any() + stringMatching for flexible schema checks. **(3)** Validate type and format, not exact values for generated fields. **(4)** A format regex for email beats a literal address, it tests the rule. **(5)** The next question: unpredictable fields, expect.any for type, stringMatching for format. **(6)** Trap: asserting exact IDs/timestamps, brittle on every data change.

CODE

```
test('API response assertions', async ({ request }) => {
const response = await request.get('/api/users/1');
await expect(response).toBeOK();
expect(response.headers()['content-type']).toContain('application/json');

const user = await response.json();
expect(user).toMatchObject({
id: expect.any(Number),
name: expect.any(String),
email: expect.stringMatching(/^[^\s@]+@[^\s@]+\.[^\s@]+$/),
});
});
```

Q 120

LEVEL · Mid to Senior

How do you use custom matchers to extend the expect API?

CONCEPT

expect.extend adds domain-specific assertions to the expect API, raising readability and removing duplication. A **toBeLoggedIn** matcher, for example, encapsulates several checks, the URL is not the login page and an auth cookie exists, behind one expressive call, and the matcher returns a pass flag plus a clear failure message. Without it, every test that verifies login state duplicates those checks. Adding the **TypeScript declaration** for the matcher gives it full autocomplete and type safety, so it behaves like a first-class assertion. Reserve custom matchers for genuine domain concepts, toBeLoggedIn, toHavePermission, toShowValidationError, rather than wrapping single built-in calls, where they would just add indirection.

INTERVIEW STRATEGY

They are quietly checking for framework-design maturity, do you build reusable abstractions or copy-paste checks. Frame custom matchers as the highest level of assertion abstraction, use toBeLoggedIn as the example, and mention the TypeScript declaration so it gets autocomplete and type safety. The classic error is wrapping a single built-in assertion, which adds indirection for no gain. The next question is normally ‘when is a custom matcher worth it?’, answer when a multi-step domain check repeats across many tests, not for one-line wrappers.

How to phrase it

> *Custom matchers, via expect.extend, are the highest level of assertion abstraction I use, and I reach for them to make tests readable and to remove duplication. The example I give is toBeLoggedIn. It encapsulates several checks, the URL is not the login page and an auth cookie exists, behind one expressive call, and the matcher returns a pass flag plus a clear failure message that says exactly what was wrong. Without it, every test that verifies login state duplicates those same checks, and the day the login rule changes I would be editing dozens of tests instead of one matcher. I also add the TypeScript declaration for the matcher, which gives it full autocomplete and type safety so it behaves like a first-class built-in assertion rather than a loose helper. That sets up the follow-up about when a custom matcher is actually worth it: when a multi-step domain check repeats across many tests, toBeLoggedIn, toHavePermission, toShowValidationError, it earns its keep. What I avoid, and the trap I would name, is wrapping a single built-in assertion in a custom matcher, because that just adds a layer of indirection for no real gain. So I add matchers for genuine domain concepts that recur, and leave the simple one-liners as plain expects.*

Key points to hit

(1) expect.extend adds domain-specific assertions: readable and reusable. **(2)** toBeLoggedIn bundles multi-step checks behind one expressive call. **(3)** Add the TypeScript declaration for autocomplete and type safety. **(4)** Login rule changes -> edit one matcher, not dozens of tests. **(5)** Asked next: worth it for repeated multi-step domain checks, not one-liners. **(6)** Where this goes wrong: wrapping a single built-in assertion, indirection for no gain.

CODE

```
// src/utils/custom-matchers.ts
import { expect } from '@playwright/test';

expect.extend({
async toBeLoggedIn(page) {
const onApp = !page.url().includes('/login');
const hasCookie = await page.evaluate(() => document.cookie.includes('auth_token'));
const pass = onApp && hasCookie;
return { pass, message: () => `Expected logged-in. url ok: ${onApp}, cookie:
${hasCookie}` };
},
});

// TypeScript declaration: autocomplete + type safety
declare global {
namespace PlaywrightTest {
interface Matchers<R> { toBeLoggedIn(): R; }
}
}

await expect(page).toBeLoggedIn();
```

Q 121

LEVEL · Senior

How do you operate visual regression in production: baseline updates, masking, and threshold tuning?

CONCEPT

Three operational concerns turn visual regression from a demo into a production tool. Baseline updates: npx playwright test --update-snapshots regenerates baselines for any test that diffs against the current render. The discipline that matters: baselines belong in git and review, every baseline change is a code review by a human who confirms the new render is intended. Without review, a regression that updates the baseline becomes the new "correct" state. Masking: pass **mask: [locator, ...]** to toHaveScreenshot to black out areas that change legitimately every run, timestamps, ad slots, dynamic content. Masked regions are excluded from the diff. Threshold tuning: **maxDiffPixels** (absolute pixel count) and **maxDiffPixelRatio** (fraction of pixels) absorb anti-aliasing noise on different machines. The trap: tuning thresholds too loose hides real regressions. Set the lowest threshold the suite passes stably on, and revisit when it starts flaking, don't just keep raising it.

INTERVIEW STRATEGY

They want visual regression as a production tool, not just a feature demo. Lead with the three ops concerns: baselines (git + review), masking (dynamic regions), thresholds (anti-alias noise). The 'lowest threshold that passes stably, revisit when flaking' rule is the senior signal. The risk is keep-raising-threshold. A common follow-up is ‘what do you mask?’, answer timestamps, ad slots, anything that legitimately changes every run.

How to phrase it

> *Three operational concerns turn visual regression from a demo into a production tool, and managing each well is what separates a useful suite from a noisy one. First, baseline updates. npx playwright test dash-dash-update-snapshots regenerates baselines for any test that diffs against the current render. The discipline that matters, and the one I would lead with because it's where teams lose the value, is baselines belong in git and review. Every baseline change is a code review by a human who confirms the new render is intended. Without review, a regression that updates the baseline becomes the new correct state, the test passes forever against a broken UI and nobody notices until a user complains. Second, masking. I pass mask colon a list of locators to toHaveScreenshot to black out areas that change legitimately every run, timestamps, ad slots, dynamic content, user avatars when they're randomly assigned. Masked regions are excluded from the diff. When they push for the follow-up about what to mask: anything that legitimately changes every run. Timestamps first, then ad slots, then any dynamic content that isn't the thing under test. Third, threshold tuning. maxDiffPixels for absolute pixel count, maxDiffPixelRatio for fraction of pixels. These absorb anti-aliasing noise on different machines, fonts render slightly differently across operating systems, GPU drivers produce subpixel variation, even browser updates shift a pixel here or there. The trap, and the part I would emphasise as the senior discipline, is tuning thresholds too loose hides real regressions. A button that moves twenty pixels is a real bug; if my threshold absorbs a hundred pixels of difference, I'll never see it. The rule: set the lowest threshold the suite passes stably on, and revisit when it starts flaking, don't just keep raising it. Investigate what changed and either fix the render, add a mask, or adjust the baseline. Raising the threshold should be a last resort with justification.*

Key points to hit

(1) Baselines in git + code review: prevents regression-becomes-new-correct. **(2)** mask: [...] excludes legitimately-changing regions from diff. **(3)** maxDiffPixels / maxDiffPixelRatio absorb anti-alias + OS/GPU noise. **(4)** Set the lowest stable threshold; revisit when flaking, don't just raise. **(5)** Follow-up: mask timestamps, ad slots, random dynamic content. **(6)** What loses the point: keep-raising-threshold, hides real visual regressions.

CODE

```
// Production-grade visual regression
test('product card visual', async ({ page }) => {
await page.goto('/products/sku-42');

await expect(page.getByTestId('product-card')).toHaveScreenshot('product-card.png', {
// 1. Mask legitimately-dynamic regions
mask: [
page.getByTestId('timestamp'),
page.getByTestId('ad-slot'),
],
// 2. Threshold for anti-alias / GPU noise (lowest stable value)
maxDiffPixels: 50,
threshold: 0.2,
// 3. Disable animations for deterministic capture
animations: 'disabled',
});
});

// Baseline update workflow:
// npx playwright test --update-snapshots -> git diff -> PR -> review -> merge
// NEVER bypass review: a bad baseline becomes the new "correct" state
```

Q 122 LEVEL · Mid to Senior

When do you use soft assertions versus fail-fast, and what's the discipline?

CONCEPT

Fail-fast is Playwright's default: the first failed assertion throws and the test stops. **expect.soft** lets the test continue after a failure, collecting multiple failures and reporting them all at the end. The right uses are narrow. **UI dashboards with many independent fields**: collecting all the wrong values in one test run shows the full picture rather than fixing one and discovering another on the next run. **End-of-test verification phases**: after the main flow completes, soft-assert on each state indicator so the test reports every issue at once. The case against using soft everywhere: subsequent operations may depend on earlier ones, and a soft failure on a precondition means later operations execute against unexpected state, producing cascade failures that obscure the real cause. The rule: soft for independent assertions, fail-fast for dependent ones. If the next line of code depends on the assertion passing, fail fast; if the next assertion is unrelated, soft is fine.

INTERVIEW STRATEGY

What earns the tick is soft-vs-fail-fast as a judgement, not a default. Lead with fail-fast being the right default, then soft's narrow right uses (independent dashboard fields, end-of-test verification). The independent-vs-dependent rule is the senior signal. The thing to avoid is soft everywhere causing cascade failures. The follow-up is often 'give an example where soft is wrong?', answer asserting login succeeded with soft, then asserting dashboard content, if login failed, dashboard assertions are meaningless.

How to phrase it

> *Fail-fast is Playwright's default and the right default. The first failed assertion throws and the test stops, which means the failure is precise and the trace shows exactly what went wrong. expect dot soft lets the test continue after a failure, collecting multiple failures and reporting them all at the end of the test. The right uses are narrow, and the framing I would lead with because it scopes the tool correctly, is two cases. First, UI dashboards with many independent fields. If I'm verifying a dashboard shows the correct user name, the correct email, the correct tenant, the correct role, and the correct subscription status, soft-asserting on each lets me see all the wrong values in one test run. With fail-fast I'd see one wrong field, fix it, re-run, see the next wrong field, fix it, re-run, five rounds instead of one diagnosis. Second, end-of-test verification phases. After the main flow completes, I soft-assert on each state indicator so the test reports every issue at once. The case against using soft everywhere, and where most candidates miss the nuance, is subsequent operations may depend on earlier ones. A soft failure on a precondition means later operations execute against unexpected state, producing cascade failures that obscure the real cause. The example I'd give, and my answer to the follow-up about when soft is wrong, is asserting login succeeded with soft and then asserting on dashboard content. If login failed, the user never reached the dashboard, the dashboard assertions all fail too, and the report shows ten failures instead of one, burying the real issue under noise. The rule I would close on, because it captures the discipline, is soft for independent assertions, fail-fast for dependent ones. If the next line of code depends on the assertion passing, fail fast, a login failure should stop the test. If the next assertion is unrelated, soft is fine, five independent dashboard fields, soft-assert each. The thing to avoid is soft everywhere causing cascade failures and obscuring root cause.*

Key points to hit

(1) Fail-fast is Playwright's default: first failure stops the test. **(2)** expect.soft continues, collects, reports all failures at end. **(3)** Right uses: independent dashboard fields, end-of-test verification. **(4)** Rule: soft for independent assertions, fail-fast for dependent ones. **(5)** Expect next: soft on login then assertions on dashboard = cascade noise. **(6)** The common miss: soft everywhere, cascade failures obscure root cause.

CODE

```
// GOOD: soft on independent dashboard fields
test('user profile dashboard', async ({ page }) => {
await page.goto('/profile');
// Each field is independent; one wrong doesn't invalidate the others
await expect.soft(page.getByTestId('name')).toHaveText('Anna Banerjee');
await expect.soft(page.getByTestId('email')).toHaveText('anna@bank.com');
await expect.soft(page.getByTestId('tenant')).toHaveText('Mumbai');
await expect.soft(page.getByTestId('role')).toHaveText('Admin');
// All failures reported at the end of this test
});

// BAD: soft on a dependency causes cascade failures
// await expect.soft(page.getByText('Welcome back')).toBeVisible(); // login succeeded?
// await expect(page.getByTestId('orders').count()).resolves.toBe(5); // meaningless if
login failed

// CORRECT: fail-fast on the precondition
await expect(page.getByText('Welcome back')).toBeVisible(); // fail-fast: stops if login
failed
await expect.soft(page.getByTestId('orders').count()).resolves.toBe(5);
```

Q 123 LEVEL · Mid to Senior

How do you use toMatchSnapshot for non-image artifacts like JSON or HTML?

CONCEPT

toMatchSnapshot compares any string or Buffer against a stored baseline, where toHaveScreenshot is image-specific. It's the right tool for testing the shape of JSON responses that should remain stable, HTML fragments for component output, CSV exports, generated markdown, or any text artifact where the entire serialised form is the thing being verified. The pattern: assert **expect(JSON.stringify(response, null, 2)).toMatchSnapshot('users-response.json')**, Playwright stores the snapshot file alongside the test, fails when output diverges, and offers **--update-snapshots** to regenerate. The discipline that holds up: use toMatchSnapshot when the entire shape matters and is expected to be stable; use targeted assertions when only specific fields matter. Snapshotting an entire response that includes timestamps, IDs, or ordering means every test run looks like a change, normalise the artifact before snapshotting (strip timestamps, sort arrays) or you'll be updating snapshots every run.

INTERVIEW STRATEGY

You need to give snapshot-testing-for-text understood. Lead with toMatchSnapshot for any string/Buffer, then list real uses (JSON, HTML, CSV, markdown). The normalise-before-snapshot discipline is the senior signal because un-normalised snapshots produce constant churn. The classic error is snapshotting raw API responses with timestamps. Expect the follow-up ‘what's the normalisation pattern?’, answer strip non-deterministic fields, sort arrays with non-deterministic order, before stringify.

How to phrase it

> *toMatchSnapshot compares any string or Buffer against a stored baseline, where toHaveScreenshot is image-specific. It's the right tool for testing the shape of artifacts where the entire serialised form is the thing being verified. The cases I'd list to make it concrete: JSON responses that should remain stable across releases, HTML fragments for component output if I'm testing rendered markup, CSV exports where the column order and content both matter, generated markdown from a documentation tool, any text artifact I want to lock in. The pattern is straightforward: I assert expect of JSON dot stringify the response null comma two to match snapshot named users-response dot json. Playwright stores the snapshot file alongside the test, fails when the output diverges, and offers dash-dash-update-snapshots to regenerate when the change is intentional. The discipline that holds up in interviews, and the part that separates a useful snapshot suite from a noisy one, is normalise the artifact before snapshotting. That's also my answer to the follow-up about the normalisation pattern. Strip non-deterministic fields like timestamps, request IDs, generated UUIDs. Sort arrays whose order isn't guaranteed by the API. Round floating point numbers that might drift in the last decimal. Then stringify and snapshot. Without normalisation, every test run looks like a change, the team updates snapshots constantly, and the snapshots stop catching real bugs because every diff is noise. The rule I would close on, because it's the judgement call, is use toMatchSnapshot when the entire shape matters and is expected to be stable. Use targeted assertions, expect response dot users dot length to be five, expect response dot users zero dot email to match a regex, when only specific fields matter. Snapshotting is heavy machinery; the right tool when shape stability is the goal, the wrong tool when only a couple of fields are the actual assertion. The classic error is snapshotting raw API responses with timestamps; every run produces a diff and the suite becomes unmaintainable in weeks.*

Key points to hit

(1) toMatchSnapshot compares any string/Buffer to a stored baseline. **(2)** Uses: JSON shape, HTML fragments, CSV exports, generated markdown. **(3)** Normalise BEFORE snapshot: strip timestamps/IDs, sort arrays, round floats. **(4)** Use when entire shape matters; targeted assertions when only fields matter. **(5)** They usually ask next: normalisation strips non-deterministic fields + sorts unstable arrays. **(6)** Avoid: snapshotting raw responses with timestamps, constant churn, snapshots stop catching bugs.

CODE

```
// JSON snapshot with normalisation
test('GET /users response shape', async ({ request }) => {
const res = await request.get('/api/users');
const json = await res.json() as { users: Array<{ id: string; createdAt: string; email:
string }> };

// Normalise: strip non-deterministic fields, sort by stable key
const normalised = {
users: json.users
.map(({ createdAt, id, ...rest }) => rest) // strip id, createdAt
.sort((a, b) => a.email.localeCompare(b.email)), // stable order
};

expect(JSON.stringify(normalised, null, 2)).toMatchSnapshot('users-response.json');
});

// HTML fragment
// expect(await
page.locator('[data-testid="summary"]').innerHTML()).toMatchSnapshot('summary.html');

// Update workflow: npx playwright test --update-snapshots -> review diff -> PR
```

Q 124 LEVEL · Mid to Senior

How does the .not negation work, and what's the timing trap with it?

CONCEPT

Negation in Playwright assertions works via **.not**: **expect(locator).not.toBeVisible()**, **expect(locator).not.toHaveText('Error')**. The retry semantics are the same as the positive case, Playwright polls until the condition is satisfied or the timeout expires. The trap that catches candidates: not.toBeVisible polls until the element is NOT visible, which means it waits the full timeout if the element is currently visible. A test that asserts an error message disappeared after some user action will wait 5 seconds (the default timeout) for the disappearance, fine if the disappearance was intended, slow if the test was checking that the error never appeared. For the latter, use toBeHidden with a short explicit timeout, or check that the element was never attached, depending on intent. The principle: negative assertions are slow by design, they must wait to confirm the negation, so reach for them deliberately, not as a default.

INTERVIEW STRATEGY

This question is there to see the timing trap behind .not. Lead with how .not retries the same way, polls until the negation is true, then make the slow-by-design framing explicit. The 'reach deliberately, not by default' rule is what separates junior from senior thinking. The trap is using .not.toBeVisible to confirm an element never appeared, paying full timeout. They usually follow with ‘how do you check an element never appeared?’, answer assert the positive thing the page should show instead, and trust that locator's auto-waiting to catch the bug if the wrong element shows up.

How to phrase it

> *Negation in Playwright assertions works via dot not: expect locator dot not dot toBeVisible, expect locator dot not dot toHaveText quote Error. The retry semantics are the same as the positive case, Playwright polls until the condition is satisfied or the timeout expires. The trap that catches candidates, and the framing I would emphasise because it's where tests become slow without anyone noticing, is negative assertions are slow by design. Not dot toBeVisible polls until the element is NOT visible, which means if the element is currently visible, Playwright waits the full timeout for it to disappear. That's the right behaviour if I'm asserting the error message disappeared after the user clicked dismiss, five seconds of waiting is fine because the disappearance was intended and likely happens. But it's the wrong cost if I'm asserting the error message never appeared after a successful save, the assertion sits idle for five seconds confirming that the element it was never expecting hasn't appeared, every test that does this pays five seconds, the suite slows down for no value. For the never-appeared case, two better options. toBeHidden with a short explicit timeout if I genuinely need to verify hiding. Or, more often, assert the positive thing the page should show instead, the success toast, the new dashboard URL, the updated state, and trust that locator's auto-waiting will catch the bug if the wrong element shows up. If they push on this in the follow-up, my answer is about checking an element never appeared: assert the positive expected state, not the negative absence. The principle I would close on, because it captures the calibration, is negative assertions are slow by design, they must wait to confirm the negation, so reach for them deliberately, not as a default. The trap is using not dot toBeVisible to confirm an element never appeared, paying full timeout on every test that does it. Over a thousand-test suite, that's measurable minutes of CI time spent waiting for nothing.*

Key points to hit

(1) .not retries the same as positive: polls until negation true or timeout. **(2)** not.toBeVisible waits the FULL timeout if element is currently visible. **(3)** Right for asserting disappearance; wrong for asserting never-appeared. **(4)** Prefer asserting the POSITIVE expected state instead of negative absence. **(5)** The next question: assert success toast + new URL, auto-wait catches the wrong UI. **(6)** The easy mistake: not.toBeVisible for never-appeared, pays full timeout silently.

CODE

```
// RIGHT USE: asserting disappearance after a known action
await page.getByRole('button', { name: 'Dismiss' }).click();
await expect(page.getByRole('alert')).not.toBeVisible(); // waits up to timeout,
expected to disappear

// SLOW USE: asserting never-appeared pays full timeout for nothing
// await page.getByRole('button', { name: 'Save' }).click();
// await expect(page.getByRole('alert')).not.toBeVisible(); // 5s wait, every test, no
value

// BETTER: assert the positive expected state
await page.getByRole('button', { name: 'Save' }).click();
await expect(page.getByText('Saved successfully')).toBeVisible();
await expect(page).toHaveURL(/\/saved/);
// Auto-waiting catches the bug if the wrong UI appears; no wasted timeout
```

Q 125

LEVEL · Senior

What's the difference between toEqual, toStrictEqual, toMatchObject, and expect.objectContaining?

CONCEPT

Four object-shape matchers with subtly different semantics. **toEqual**: deep equality, ignoring undefined properties and prototype, { a: 1 } equals { a: 1, b: undefined }. The default for most object comparisons. **toStrictEqual**: deep equality plus checks for undefined properties (above example is NOT equal) plus prototype check (class instance is not equal to a plain object with the same fields). Use when type identity matters. **toMatchObject**: partial match, the actual must contain at least the expected fields, but can have more. Useful when asserting on an API response that has fields you don't care about. expect.objectContaining(partial): same partial-match semantics but usable inside other matchers, expecting toEqual(expect.objectContaining({ id: 1 })) or as an argument matcher. The rule: toEqual for full deep equality, toMatchObject for partial, toStrictEqual when type identity matters, objectContaining inside other matchers.

INTERVIEW STRATEGY

They are after precise semantic distinctions. Walk all four: toEqual (loose deep), toStrictEqual (strict deep + prototype), toMatchObject (partial), objectContaining (partial inside matchers). The 'toEqual for full equality, toMatchObject for partial' rule is the seniority marker. The trap is toEqual with API responses that include fields you don't care about. The next question is normally ‘when does prototype matter?’, answer when comparing class instances vs plain objects, toStrictEqual catches the mistake.

How to phrase it

> *Four object-shape matchers with subtly different semantics, and knowing the differences is what stops mysterious failures and false passes. toEqual: deep equality, ignoring undefined properties and prototype. Object curly a colon one equals object curly a colon one b colon undefined under toEqual. This is the default for most object comparisons. toStrictEqual: deep equality plus checks for undefined properties, the same example is NOT equal under toStrictEqual because the second has an extra undefined field, plus prototype check, so a class instance is not equal to a plain object with the same field values. Use when type identity matters. toMatchObject: partial match. The actual must contain at least the expected fields, but can have more. Useful when asserting on an API response that has fields I don't care about, timestamps, server-generated IDs, version numbers, and I only want to lock in the fields that matter to the test. expect dot objectContaining of a partial object: same partial-match semantics but usable inside other matchers. So I can write expect of x toEqual expect dot objectContaining curly id colon one when I want partial inside an equality, or pass it as an argument matcher in mock expectations. The rule I would close on, because it's the heuristic that captures all four cleanly, is toEqual for full deep equality, toMatchObject for partial, toStrictEqual when type identity matters, objectContaining inside other matchers. On the natural follow-up about when prototype matters: when comparing class instances versus plain objects. If I'm asserting a function returns an instance of User and not just a User-shaped plain object, toStrictEqual catches the mistake; toEqual would pass on either. The pitfall is using toEqual with API responses that include fields the test doesn't care about, timestamps, version numbers, IDs, making the test brittle to every API change that adds a field. toMatchObject is almost always what I actually want for API response assertions, and reaching for it instead of toEqual is the senior move.*

Key points to hit

(1) toEqual: deep equality, ignores undefined props and prototype, default. **(2)** toStrictEqual: + checks undefined props + prototype, type identity matters. **(3)** toMatchObject: partial, actual must contain expected, can have more. **(4)** expect.objectContaining: partial-match inside other matchers / arg matchers. **(5)** Asked next: prototype matters for class-vs-plain-object distinctions. **(6)** Trap: toEqual on API responses with fields you don't care about, brittle.

CODE

```
const actual = { id: 1, name: 'Anna', email: 'anna@bank.com', createdAt:
'2026-01-15T10:00:00Z' };

// toEqual: full deep equality (ignores undefined props)
expect({ a: 1 }).toEqual({ a: 1, b: undefined }); // passes

// toStrictEqual: + checks undefined props + prototype
expect({ a: 1 }).not.toStrictEqual({ a: 1, b: undefined }); // strict: NOT equal
class User { constructor(public id: number) {} }
expect(new User(1)).not.toStrictEqual({ id: 1 }); // class vs plain object

// toMatchObject, partial: actual contains at least expected
expect(actual).toMatchObject({ id: 1, name: 'Anna' }); // ignores email, createdAt

// objectContaining: partial inside other matchers
expect([actual]).toEqual([
expect.objectContaining({ name: 'Anna' }), // ignores other fields
]);
```

Q 126 LEVEL · Mid

toHaveText vs toContainText: when do you use each, and what's the exact-match vs substring trap?

CONCEPT

Both retry, but they assert different things. **toHaveText** checks the full, exact text of the element, the whole string must match (trimmed of leading and trailing whitespace), so **toHaveText('Order Confirmed')** fails if the element reads 'Order Confirmed Today'. **toContainText** checks that the text is present as a substring, so the same call passes against the longer string. The trap is reaching for toHaveText on dynamic content, a status line, a greeting with the user's name, a count that includes a label, where extra words legitimately ride alongside the bit you care about, and the exact match then fails for the wrong reason. The flip side: use toHaveText when the entire value is the contract (a heading, a confirmed total, a fixed label) so a stray appended word is genuinely a bug you want to catch. Both also accept a regex for partial-but-anchored matches, and both accept an array to assert across a list of elements in one call, toHaveText with an array does an exact match per item, toContainText a substring per item.

INTERVIEW STRATEGY

They want to see that you pick the matcher by intent, not habit. Contrast them in one breath, toHaveText is the whole string, toContainText is a substring, then make the trap concrete: exact match on dynamic content that carries extra words fails for the wrong reason. Say clearly when each is right: exact when the full value is the contract, substring when you only care about one phrase. Mention the regex and array forms to show range. A common follow-up is ‘how do you assert one phrase inside a long paragraph?’, answer toContainText or an anchored regex, never toHaveText.

How to phrase it

> *They both retry, so the difference is what they assert, not how they wait. toHaveText checks the full, exact text of the element, the whole string has to match, trimmed of surrounding whitespace. toContainText checks that my text is present as a substring, so it passes as long as the phrase appears somewhere in the element. The way I decide is by asking whether the entire value is the contract or just one phrase. If I'm asserting a heading, a fixed label, or a confirmed total, where the whole string is meaningful and a stray extra word would be a real bug, I use toHaveText, because I want the exact match to catch that. If I'm asserting against dynamic content, a status line, a greeting that includes the user's name, a banner that appends a count or a date, I use toContainText for the bit I actually care about, because extra words legitimately ride alongside it. The trap, and where I've seen tests fail for the wrong reason, is reaching for toHaveText on that dynamic content: 'Order Confirmed' fails the moment the UI renders 'Order Confirmed Today', even though nothing is broken. Two things I'd add to show range: both accept a regex when I want partial but anchored, starts-with, ends-with, a pattern, and both accept an array to assert across a list of elements in one call, with toHaveText matching each item exactly and toContainText matching each as a substring. So if the follow-up is how I'd assert one phrase inside a long paragraph, the answer is toContainText or an anchored regex, never an exact toHaveText. The headline: exact when the whole value is the contract, substring when only a phrase is.*

Key points to hit

(1) toHaveText = full exact string (trimmed); toContainText = substring. **(2)** Both retry: the difference is what they assert, not the waiting. **(3)** Exact when the whole value is the contract; substring for dynamic content. **(4)** Where this goes wrong: toHaveText on text that carries extra words, fails for the wrong reason. **(5)** Both accept a regex (anchored partial) and an array (per-item assertion). **(6)** Follow-up: a phrase inside a paragraph, toContainText or anchored regex.

CODE

```
// toHaveText: the WHOLE string is the contract
await expect(page.getByRole('heading')).toHaveText('Order Confirmed');
// fails if the heading reads 'Order Confirmed Today'

// toContainText: a phrase inside dynamic content
await expect(page.getByTestId('greeting')).toContainText('Welcome');
// passes for 'Welcome back, Anna'

// Anchored regex: partial but pattern-controlled
await expect(page.getByTestId('total')).toHaveText(/^Total: ₹[\d,]+$/);

// Array form: assert across a list in one call
await expect(page.getByRole('listitem')).toHaveText(['Apples', 'Bananas', 'Cherries']);
await expect(page.getByRole('listitem')).toContainText(['Apple', 'Banana']);
```

Q 127 LEVEL · Mid

toBeVisible vs toBeHidden vs toBeAttached: what does each actually assert, and why isn't toBeHidden just not(toBeVisible)?

CONCEPT

These three sit on two different axes. **toBeAttached** asks whether the element is in the DOM at all, regardless of whether you can see it, it is the presence check. **toBeVisible** asks whether the element is attached and actually rendered to the user: non-zero size, not display:none, not visibility:hidden, not hidden by an ancestor. **toBeHidden** is the considered opposite: it passes when the element is either not in the DOM or in the DOM but not rendered. That is why it isn't simply not.toBeVisible: the timing and intent differ. toBeHidden treats a detached element as a first-class pass and reads as a positive assertion, whereas not.toBeVisible is a negation that polls until visibility becomes false and tends to wait the full timeout when the element starts visible. If you need to distinguish 'removed from the page' from 'present but hidden', say, a modal that should be torn down versus merely collapsed, you pair toBeAttached (or not.toBeAttached) with toBeHidden to be precise about which state you mean.

INTERVIEW STRATEGY

They want you to separate two axes, presence in the DOM versus rendered to the user, and not blur the three matchers together. Define each crisply: toBeAttached is presence, toBeVisible is presence plus actually rendered, toBeHidden is the deliberate opposite that also accepts detached. The crisp way to answer the 'why not not.toBeVisible' part is to name two differences: toBeHidden reads as a positive assertion and counts a detached element as a pass, while not.toBeVisible is a negation that usually burns the full timeout. The follow-up is often ‘how do you assert a modal was fully removed, not just hidden?’, answer not.toBeAttached, not toBeHidden.

How to phrase it

> *I'd start by separating two axes, because that's what the question is really testing. One axis is presence in the DOM; the other is whether the element is actually rendered to the user. toBeAttached is the presence check: is the element in the DOM at all, visible or not. toBeVisible is stricter: the element is attached and genuinely rendered, non-zero size, not display:none, not visibility:hidden, not hidden by an ancestor. toBeHidden is the considered opposite of toBeVisible: it passes when the element is either not in the DOM at all, or in the DOM but not rendered. Now the interesting part: why isn't toBeHidden just not.toBeVisible? Two reasons. First, intent and timing: toBeHidden reads as a positive assertion and is calibrated to wait for the element to become hidden, whereas not.toBeVisible is a negation that polls until visibility turns false and, if the element starts out visible, tends to sit there for the full timeout. Second, detached handling: toBeHidden treats an element that isn't in the DOM as a clean pass, which is usually exactly what you mean by 'hidden'. So they overlap in outcome but differ in how they wait and how they read. Where I genuinely need to distinguish 'removed from the page' from 'present but collapsed', a modal that should be fully torn down versus one that's merely hidden behind a class, I don't rely on toBeHidden alone, because it can't tell those apart. I use toBeAttached or not.toBeAttached to assert presence precisely, and pair it with toBeHidden when I care about rendering. That's also my answer to the usual follow-up: to prove a modal was fully removed and not just hidden, I assert not.toBeAttached, because toBeHidden would pass even if a collapsed, leaked node were still sitting in the DOM.*

Key points to hit

(1) Two axes: presence in DOM (attached) vs rendered to the user (visible). **(2)** toBeAttached = in the DOM at all; toBeVisible = attached AND actually rendered. **(3)** toBeHidden = deliberate opposite:

not in DOM, or in DOM but not rendered. **(4)** Not just not.toBeVisible: toBeHidden reads positive and counts detached as a pass. **(5)** not.toBeVisible is a negation: tends to burn the full timeout. **(6)** Expect next: modal fully removed, assert not.toBeAttached, not toBeHidden.

CODE

```
// Presence only: in the DOM, visible or not
await expect(page.getByTestId('lazy-panel')).toBeAttached();

// Rendered to the user: attached AND actually shown
await expect(page.getByRole('dialog')).toBeVisible();

// Hidden: passes whether detached OR present-but-not-rendered
await page.getByRole('button', { name: 'Close' }).click();
await expect(page.getByRole('dialog')).toBeHidden();

// Distinguish 'fully removed' from 'merely hidden'
await expect(page.getByRole('dialog')).not.toBeAttached(); // node gone from DOM
// vs toBeHidden(), which would pass even if a collapsed node lingered
```

Q 128 LEVEL · Mid to Senior

expect.poll vs expect.toPass vs manual polling, when do you reach for each, and what does expect.toPass give you that expect.poll doesn't?

CONCEPT

All three solve 'retry until it's true', but at different levels. **expect.poll** repeatedly calls a function and asserts on its **return value** with one matcher, ideal when you have a single value that settles over time, like polling a status API until it returns 'confirmed'. **expect.toPass** wraps a whole block of assertions and retries the entire block until it passes or times out, so it handles cases where you need several assertions to all become true together, or where the action that produces the state must be re-run each attempt (re-fetch, re-click, then re-assert). That is the key thing toPass gives you that poll doesn't: poll retries a value feeding one matcher; toPass retries arbitrary code containing many assertions. Manual polling, a hand-written while loop with sleeps, is the fallback you should almost never need: it lacks the configurable intervals, the timeout integration, and the clear failure messages the built-ins give, and it's where flaky hard-coded sleeps creep in. Reach for poll for a single settling value, toPass for a multi-assertion or re-run-the-action block, and manual only for genuinely exotic control flow the two can't express.

INTERVIEW STRATEGY

They need you to rank the three by fit and articulate the one real distinction between the built-ins. Lead with the axis: poll retries a single return value into one matcher; toPass retries a whole block of assertions. Then say the line that answers the question directly, toPass gives you multiple assertions becoming true together, and the ability to re-run the producing action each attempt, which poll's single-value model can't express. Frame manual polling as the fallback you avoid because it reintroduces hard sleeps. Expect the follow-up ‘give a case only toPass fits’, answer re-fetch then assert several fields, or re-click a flaky control then verify state.

How to phrase it

> *All three are 'retry until it's true', so I rank them by fit. expect.poll repeatedly calls a function and asserts on its return value with a single matcher. I reach for it when I have one value that settles over time, the classic case is polling a status endpoint until it returns 'confirmed'. expect.toPass is different in an important way: it wraps a whole block of code and retries that entire block until it passes or the timeout expires. That's exactly what poll can't do, and it's the heart of the question. poll retries a single return value feeding one matcher; toPass retries arbitrary code that can contain many assertions. So when I need several things to all become true together, a response is OK and the body has the right status and the count matches, toPass lets me assert all of them inside the retried block. And critically, toPass re-runs the action that produces the state on every attempt, so I can re-fetch, or re-click a control, and then re-assert, which poll's value-only model simply can't express. Manual polling, a hand-written while loop with sleeps, is the fallback I almost never want. It loses the configurable intervals, the timeout integration, and the readable failure messages the built-ins give me, and it's precisely where hard-coded sleeps and flakiness creep back in. So my rule: poll for a single settling value, toPass for a multi-assertion block or one where I must re-run the producing action each time, and manual only for genuinely exotic control flow neither can express, which in practice is almost never. If they push for a case only toPass fits, I'd give re-fetching a resource and asserting several fields on it together, or re-clicking a control that occasionally no-ops and then verifying the resulting state, both need the action and the assertions retried as a unit, which is toPass, not poll.*

Key points to hit

(1) poll retries a single return value into one matcher, one settling value. **(2)** toPass retries a whole block of assertions until it all passes or times out. **(3)** Key distinction: poll = one value/one matcher; toPass = arbitrary code, many assertions. **(4)** toPass can re-run the producing action each attempt (re-fetch, re-click, re-assert). **(5)** Manual polling: the fallback to avoid, loses intervals, timeout, messages; invites sleeps. **(6)** They usually ask next: only-toPass case, re-fetch then assert several fields, or re-click then verify.

CODE

```
// expect.poll: ONE settling value into one matcher
await expect.poll(async () => {
const res = await page.request.get('/api/orders/42');
return (await res.json()).status;
}, { timeout: 30_000, intervals: [1_000, 2_000, 5_000] }).toBe('confirmed');

// expect.toPass: a BLOCK of assertions, with the action re-run each attempt
await expect(async () => {
const res = await page.request.get('/api/orders/42'); // re-fetched every attempt
expect(res.ok()).toBeTruthy();
const body = await res.json();
expect(body.status).toBe('confirmed');
expect(body.items).toHaveLength(3); // several assertions together
}).toPass({ timeout: 30_000, intervals: [1_000, 2_000, 5_000] });

// Manual polling, the fallback to avoid: hard sleeps, no built-in timeout/messages
// while (true) { ...; await page.waitForTimeout(1000); } // flaky, don't
```

Q 129 LEVEL · Mid to Senior

What does expect.configure do, and when is a pre-configured expect better than passing options at each call?

CONCEPT

expect.configure returns a new expect instance carrying its own defaults, so you stop repeating the same option on every assertion. The documented options are **timeout** and **soft**. Two patterns earn their keep. A slow expect, const slowExpect = expect.configure({ timeout: 10_000 }), for a genuinely slow area of the app: the rest of the suite keeps its tight default, and only the report-generation page gets the longer budget, which is far better than raising the global expect timeout and hiding regressions everywhere. A soft expect, expect.configure({ soft: true }), for a block of presentation checks you want to collect in one run rather than fail-fast on. The senior framing: expect.configure moves a policy decision out of the assertion and into one named object, so the intent is visible in the name and a reviewer sees the exception rather than a stray option buried in line 40. It also composes with expect.extend, since a configured instance keeps your custom matchers.

INTERVIEW STRATEGY

Very few candidates know this one exists, so naming it is cheap credibility. Lead with what it returns, a new expect instance with its own defaults, then give the two real uses: a slowExpect for one genuinely slow area, and a softExpect for a block of non-blocking checks. The judgement to show is why that beats raising the global expect timeout, which is the lazy fix and hides regressions across the whole suite. They usually follow with ‘why not just pass { timeout } on the assertion?’, answer that you do for a one-off, and you reach for configure when the same exception repeats, because a named instance documents the decision.

How to phrase it

> *expect.configure returns a new expect instance that carries its own defaults, and the documented options are timeout and soft. So instead of passing the same option on twenty assertions, I create one named instance and use it. The case I use it for most is a genuinely slow area of the application. Say report generation takes eight or nine seconds. I could raise the global expect timeout, but that is the lazy fix, because now every assertion in the suite waits longer and a real performance regression somewhere else passes silently. Instead I make a slowExpect with a ten second timeout and use it only on the report page. The tight default protects the rest of the suite, and the exception is visible in the name. The second case is a soft expect. If I have a block of presentation checks, heading text, column order, a footer label, where I want every failure in one run rather than stopping at the first, I configure soft true once and use that instance for the block, rather than writing expect dot soft on each line. To the follow-up about why not just pass the option per call: for a one-off I do exactly that, it is clearer. I reach for configure when the same exception repeats, because at that point a named instance is documentation. Someone reading the file sees slowExpect and knows immediately that this area is slow on purpose. It also keeps any custom matchers I registered with expect.extend, so I am not choosing between the two.*

Key points to hit

(1) expect.configure returns a NEW expect instance with its own defaults. **(2)** Documented options: timeout and soft. **(3)** slowExpect for one genuinely slow area, instead of raising the global timeout. **(4)** softExpect for a block of non-blocking presentation checks. **(5)** Judgement: a named instance documents the exception; a global bump hides regressions. **(6)** The next question: per-call options for a one-off, configure when the exception repeats.

CODE

```
import { expect } from '@playwright/test';

// One slow area gets a longer budget; the rest of the suite stays tight
const slowExpect = expect.configure({ timeout: 10_000 });
await slowExpect(page.getByTestId('quarterly-report')).toBeVisible();

// A block of presentation checks: collect every failure, don't stop at the first
const softExpect = expect.configure({ soft: true });
await softExpect(page.getByRole('heading')).toHaveText('Orders');
await softExpect(page.getByRole('columnheader').first()).toHaveText('Order ID');
await softExpect(page.getByTestId('footer-note')).toContainText('Updated');

// WRONG: raising the global default to fix one slow page
// export default defineConfig({ expect: { timeout: 10_000 } });
// every assertion in the suite now waits longer, and a real
// performance regression elsewhere passes quietly.
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What does expect.soft() do differently from a normal assertion?

A) It retries longer
B) It disables retries
C) It collects all failures before throwing instead of stopping at the first
D) It only works on API responses

2. Which two options make toHaveScreenshot usable on a real, dynamic app?

A) fullPage and timeout
B) mask and animations: 'disabled'
C) threshold and retries
D) clip and scale

3. What is expect.poll() the correct tool for?

A) Eventually consistent systems where state propagates over time
B) Asserting on a static DOM element
C) Replacing web-first assertions
D) Mocking network requests

4. Why assert email with expect.stringMatching(regex) rather than a literal address?

A) Regexes run faster
B) Literal strings are not supported
C) It validates the format rule, staying resilient to data changes
D) It avoids using toMatchObject

5. When is writing a custom matcher (e.g. toBeLoggedIn) worth it?

A) To wrap a single built-in assertion
B) To replace expect entirely
C) Only for API responses
D) When a multi-step domain check repeats across many tests

Section 8 · Answer key

#	Answer	Why and where to revisit
1	C) It collects all failures before throwing instead of stopping at the first	Soft assertions run every check and report all failures at once, ideal for seeing the full set of broken fields. *Revisit Section 8.*
2	B) mask and animations: 'disabled'	Masking dynamic content and disabling animations prevent the false pixel diffs that otherwise make visual tests flaky. *Revisit Section 8.*
3	A) Eventually consistent systems where state propagates over time	expect.poll repeatedly checks with backoff until the expected state arrives, right for async, eventually consistent flows. *Revisit Section 8.*
4	C) It validates the format rule, staying resilient to data changes	Matching the format validates the rule the API must always obey, not one specific value that changes with the data. *Revisit Section 8.*
5	D) When a multi-step domain check repeats across many tests	Custom matchers pay off for repeated multi-step domain checks; wrapping a single built-in just adds indirection. *Revisit Section 8.*

SECTION 09

Parallel Execution and Sharding

Parallelism is what makes a large suite finish in minutes instead of an hour, and interviewers test whether you understand the isolation it demands. The recurring theme across these ten questions is the same: tests must own their data and share no state. They cover the worker model, sharding and how to size it, test isolation with unique data, a systematic flakiness process, and when serial mode is justified versus when it is a code smell.

In this section

- The worker model: isolated Node processes; fullyParallel for in-file parallelism.
- Sharding across CI machines and how to calculate the optimal shard count.
- Test isolation: unique data per test with faker, API setup and teardown.
- A systematic flakiness process: retries, quarantine, trace, root-cause fix.
- Worker-scoped fixtures for expensive once-per-worker setup like auth.
- Serial mode: a deliberate escape hatch, usually a code smell to remove.

Q 130 LEVEL · Mid

How does Playwright's parallel execution model work?

CONCEPT

Playwright runs test files in parallel across worker processes by default. Each worker is an independent Node.js process with its own browser instance, and workers share no memory, browser state, or test data. **fullyParallel: true** also parallelises tests **within** the same file. Because each test gets a fresh context, tests are isolated by default, but tests that share external state (a database record, a user account) will still collide when run together. The optimisation layer is **worker-scoped fixtures**: expensive setup such as an authenticated browser context happens once per worker rather than once per test, while staying isolated from other workers. The rule that falls out: every test creates its own data and tears it down, so it is order-independent and parallelism-safe.

INTERVIEW STRATEGY

The test here is whether you understand isolation, which is what makes parallelism safe. Explain the worker model and fullyParallel, then make isolation the centre of gravity: fresh context per test, but shared external state still collides. The trap is claiming Playwright isolation magically prevents all interference. Mention worker-scoped fixtures as the optimisation. The follow-up is usually 'what breaks when you parallelise?', answer tests that share a DB record or account, fixed by each test owning its data.

How to phrase it

> *By default Playwright runs test files in parallel across worker processes, and each worker is an independent Node process with its own browser instance, workers share no memory, no browser state, no test data. fullyParallel true takes it further and parallelises tests within the same file too. But the key insight, and what the question is really about, is isolation. Each test gets a fresh context, so at the browser level tests are isolated automatically, but that does not protect shared external state, and that is the answer to the follow-up about what breaks when you parallelise. Two tests that both modify the same database record or the same user account will collide when they run at the same time, and you get failures that only appear under parallelism and look random. My rule, which makes the whole thing safe, is that every test creates its own data through the API in setup and deletes it in teardown, so it is order-independent and parallelism-safe. The optimisation on top is worker-scoped fixtures: expensive setup like an authenticated context happens once per worker instead of once per test, while still staying isolated from the other workers. So: isolated workers, fresh contexts, tests that own their data, and per-worker setup for the expensive bits.*

Key points to hit

(1) Files run in parallel; each worker is an isolated Node process + browser. **(2)** fullyParallel: true parallelises tests within a file too. **(3)** Fresh context per test, but shared external state still collides. **(4)** Rule: every test creates and tears down its own data. **(5)** Asked next: what breaks, shared DB record/account; fix with owned data. **(6)** What loses the point: claiming isolation magically prevents all interference.

CODE

```
// playwright.config.ts
export default defineConfig({
fullyParallel: true,
workers: process.env.CI ? 4 : undefined,
});

// Worker-scoped fixture: expensive setup once per worker
const test = base.extend<{}, { adminContext: BrowserContext }>({
adminContext: [async ({ browser }, use) => {
const ctx = await browser.newContext({ storageState: 'auth/admin.json' });
await use(ctx);
await ctx.close();
}, { scope: 'worker' }],
});
```

Q 131 LEVEL · Mid

How does sharding work and how do you calculate the optimal shard count?

CONCEPT

Sharding distributes the suite across multiple CI machines; each shard runs a subset of tests independently with **--shard=index/total**, and the per-shard reports are merged afterwards with **merge-reports**. The sizing formula is **total_suite_time / target_time**: for a 45-minute suite targeting about 10 minutes per shard, that is roughly four shards, and with four parallel workers per shard the effective time drops further, under eight minutes in practice. Target around 10 minutes per shard: fast enough for PR feedback without so many shards that CI infrastructure cost balloons. One important caveat: Playwright balances shards by the number of tests, not by execution time, and it hands each shard a contiguous block of the ordered test list. So a shard that happens to land on the slow tests finishes late while the others sit idle, and the docs even note that some shards may run fewer tests or none at all. The fix is to keep slow tests from clustering: split big slow spec files, and let the last shard be light rather than assuming every shard takes the same time.

INTERVIEW STRATEGY

They want a real number and the reasoning behind it, not ‘add more shards’. Give the formula, total time over target time, and a worked example (45 minutes, four shards, under eight with workers). The detail that signals depth is that Playwright balances shards by test count, not by execution time, so a shard that lands on slow tests lags. The mistake is over-sharding and ignoring CI cost. The next question is normally ‘why is one shard slower than the others?’, answer that test count is not the same as time, fixed by splitting large slow spec files so the slow work spreads.

How to phrase it

> *Sharding distributes the suite across multiple CI machines: each shard runs a subset with the shard index-over-total flag, and afterwards I merge the per-shard reports with merge-reports into one. For sizing, the formula I always give is total suite time divided by target time. If a suite takes forty-five minutes and I target about ten minutes per shard, that is roughly four shards, and with four parallel workers per shard the effective time drops further, under eight minutes in practice. I target around ten minutes per shard deliberately, it is fast enough for pull-request feedback, but not so many shards that the CI infrastructure cost balloons, which is the trap of just adding shards until it is fast. The detail that shows I have actually tuned this, and the answer to the follow-up about why one shard runs slower than the others, is that Playwright balances shards by the number of tests, not by how long they take, and each shard gets a contiguous block of the ordered list. So if the slow tests cluster together, that shard finishes late while the others sit idle, and the overall time is gated by the slowest shard. The docs even say some shards may end up running fewer tests, or none. The fix is to stop the slow work clustering: split big slow spec files so it spreads. So my approach is: size by the formula, target ten minutes, and remember that equal test counts are not equal minutes.*

Key points to hit

(1) Sharding splits the suite across CI machines; merge-reports combines results. **(2)** Formula: total_suite_time / target_time; target ~10 min per shard. **(3)** 45-min suite gives ~4 shards; with 4 workers each, under ~8 min. **(4)** Shards are balanced by NUMBER OF TESTS, not execution time. **(5)** Each shard gets a contiguous block; some may run fewer tests, or none. **(6)** Follow-up: one slow shard means slow tests clustered; split large slow spec files. **(7)** The common miss: over-sharding and ignoring CI infrastructure cost.

CODE

```
# Run one shard
npx playwright test --shard=1/4

# GitHub Actions matrix
strategy:
matrix:
shardIndex: [1, 2, 3, 4]
shardTotal: [4]
steps:
- run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
- uses: actions/upload-artifact@v4
with: { name: blob-${{ matrix.shardIndex }}, path: blob-report/ }

# After all shards: merge into one report
# npx playwright merge-reports --reporter html ./all-blob-reports
```

Q 132

LEVEL · Mid

How do you ensure test isolation in a parallel environment?

CONCEPT

Isolation requires four things: unique test data per test, no shared **database** state, no shared file system state, and no shared **browser** state. The anti-pattern is a hardcoded shared fixture, one well-known test email that every parallel test fights over. The fix is synthetic unique data per test with a library like faker, created via the API in setup and deleted in teardown, so no two running tests touch the same record. Hardcoded data that multiple tests depend on is the single most common cause of parallel failures. For the rare test that genuinely cannot be parallelised, **test.describe.configure({ mode: 'serial' })** exists, but treat it as a code smell and first investigate whether the dependency can simply be removed.

INTERVIEW STRATEGY

They are checking whether you can diagnose the classic parallel-failure cause. Name shared test data as the number-one culprit, then give the fix concretely: faker for unique synthetic data, API setup and teardown per test. The risk is a hardcoded shared user that all tests reuse. A common follow-up is 'what about a test that truly cannot be parallelised?', answer serial mode exists but treat it as a code smell to investigate, not a default reach.

How to phrase it

> *Isolation comes down to four things: unique data per test, no shared database state, no shared file-system state, and no shared browser state. In practice the one that bites everyone, and the answer I lead with, is shared test data. The classic anti-pattern is a hardcoded test email that every test reuses, the moment those tests run in parallel they fight over the same user record and you get failures that only appear under parallelism and look maddeningly random. My fix is synthetic unique data per test with faker, created through the API in setup and deleted in teardown, so no two running tests ever touch the same record, and the tests become order-independent. I never use hardcoded data that multiple tests depend on. That sets up the follow-up about a test that genuinely cannot be parallelised: serial mode exists for that, but I treat it as a code smell rather than a default, when I see it, my first move is to ask why the dependency exists and whether moving setup to the API removes it. Usually it can be removed, and the test goes back to being parallel-safe. So the headline is: every test owns its own faker-generated data, and serial mode is a last resort I justify, not a convenience.*

Key points to hit

(1) Isolation needs unique data and no shared DB/file/browser state. **(2)** Anti-pattern: one hardcoded test record every parallel test fights over. **(3)** Fix: faker for unique data, API setup and teardown per test. **(4)** Shared test data is the #1 cause of parallel failures. **(5)** Expect next: a truly non-parallel test, serial mode, but treat it as a smell. **(6)** Avoid: a hardcoded shared user reused across tests.

CODE

```
import { faker } from '@faker-js/faker';

test('user registration', async ({ request, page }) => {
const email = faker.internet.email(); // unique per run
const created = await request.post('/api/users', {
data: { email, name: faker.person.fullName() },
});
const { id } = await created.json();

await page.goto('/users');
await expect(page.locator('tbody tr').filter({ hasText: email })).toBeVisible();

await request.delete(`/api/users/${id}`); // teardown
});
```

Q 133 LEVEL · Mid to Senior

How do you handle flaky tests systematically in Playwright?

CONCEPT

A systematic process, not ad-hoc reruns: retries in CI as a safety net, **track retry rate per test**, quarantine high-retry tests, **investigate with traces**, then fix the root cause. Configure two retries in CI with **trace: 'on-first-retry'** so every retried failure has a full diagnostic artifact. Any test over a retry-rate threshold (say 15%) gets tagged and moved to a quarantine project so it stops blocking the pipeline while you investigate. The hard-won truth: there is no such thing as a genuinely random flaky test, it is always a missing wait, shared state, or a race condition. The usual culprits and fixes: a missing waitForURL after navigation, a spinner not waited out, shared data between tests, a hard sleep, or a strict-mode violation from an ambiguous locator.

INTERVIEW STRATEGY

What earns the tick is a process, not 'I add retries'. Lay out the phases, retries as a net, track retry rate, quarantine, trace, root-cause, and make the strong claim that no flaky test is truly random; it is always a missing wait, shared state, or a race. That conviction demonstrates calibration. The risk is using retries as the fix rather than a safety net. The follow-up is often 'what are the usual root causes?', list missing waitForURL, unwaited spinner, shared data, hard sleep, strict-mode violation.

How to phrase it

> *I treat flakiness as a process, not as something I paper over with reruns, and I would walk the interviewer through the phases. First, retries as a safety net: two in CI, but with trace on first retry, so every retried failure leaves a full diagnostic artifact rather than just disappearing on the rerun. Second, I track the retry rate per test in the report, and any test over about fifteen percent gets tagged flaky and moved to a quarantine project, so it stops blocking the pipeline for everyone while I work on it. Third, root-cause investigation using the trace, and here is the conviction I would state plainly: in years of Playwright work I have never found a genuinely random flaky test. It is always a concrete cause, and that answers the follow-up about the usual culprits: a missing waitForURL after navigation, a loading spinner that was not waited out, shared data between tests, a hard sleep that is sometimes too short, or a strict-mode violation from an ambiguous locator. The trace viewer makes the diagnosis fast because it shows the DOM, the network and the screenshots at the moment of failure. The crucial point is that retries are the net, not the fix, once I find the root cause I make the test deterministic and take it out of quarantine. A suite that relies on retries to stay green is hiding real bugs.*

Key points to hit

(1) Process: retries as a net -> track retry rate -> quarantine -> trace -> fix. **(2)** trace: 'on-first-retry' gives every retried failure a diagnostic artifact. **(3)** Tag and quarantine tests over a retry-rate threshold so they stop blocking CI. **(4)** Conviction: no flaky test is truly random. **(5)** They usually ask next: root causes, missing waitForURL, unwaited spinner, shared data, sleep, strict-mode. **(6)** The easy mistake: using retries as the fix instead of a safety net.

CODE

```
// playwright.config.ts
export default defineConfig({
retries: process.env.CI ? 2 : 0,
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video:
'retain-on-failure' },
});

// Quarantine a known flaky test while you investigate
test('notification timing @flaky', async ({ page }) => {
// tracked in JIRA-1234: race in notification service
});

// Common root causes -> fixes:
// missing waitForURL -> await page.waitForURL('/dashboard')
// unwaited spinner -> spinner.waitFor({ state: 'hidden' })
// hard sleep -> replace with an explicit waitFor
```

Q 134 LEVEL · Mid to Senior

How does serial mode work, and when is it justified?

CONCEPT

test.describe.configure({ mode: 'serial' }) forces every test in a describe block to run sequentially in the same worker, in order, and if one test fails the rest are **skipped**. It is the opposite of

fullyParallel and a deliberate escape hatch for tests with a genuine sequential dependency, where test B needs the state test A created. The honest stance: treat it as a **code smell** that needs justification. Most of the time the dependency exists only because test A creates data through the UI that test B needs, and the fix is to move setup to the API in a fixture so each test is independent and parallel. Serial mode is legitimate only for testing a real state-machine transition that neither the UI nor the API lets you jump into directly, and that is rare.

INTERVIEW STRATEGY

They are testing your judgement, not just whether you know the API. Explain what serial mode does, then take the opinionated stance that it is a code smell requiring justification, that is the senior signal. Show the better alternative: API setup making tests independent. The risk is reaching for serial mode to chain UI steps out of convenience. Expect the follow-up 'so is it ever legitimate?', answer yes, for a genuine state transition neither UI nor API lets you jump into, and note how rarely that comes up.

How to phrase it

> *Serial mode, test.describe.configure with mode serial, forces every test in a describe block to run sequentially in the same worker and in order, and if one fails the rest are skipped, it is the opposite of fullyParallel. I would frame my answer around judgement, because that is what the question is really about: I treat serial mode as a code smell that requires justification. When I see it in a codebase, my first question is why test B depends on test A, and the answer is usually that test A creates data through the UI that test B needs. The fix, and the better alternative I would show, is to move that data creation to the API in a fixture, after which both tests are independent and can run in parallel, faster and more robust. That sets up the follow-up about whether serial mode is ever legitimate. The honest answer is yes, but narrowly: it is justified for testing a genuine state-machine transition where neither the UI nor the API lets you jump straight to an intermediate state, so you have to walk the states in order. But that is rare: in years of Playwright work I have reached for serial mode only a couple of times. So my position is that it is a deliberate, justified escape hatch, never a default way to chain UI steps for convenience.*

Key points to hit

(1) Serial mode runs a describe block sequentially in one worker; failure skips the rest. **(2)** Opposite of fullyParallel: a deliberate escape hatch. **(3)** Treat it as a code smell that requires justification. **(4)** Usual fix: move UI setup to the API so tests are independent and parallel. **(5)** The next question: legitimate only for a real state transition you can't jump into, rare. **(6)** Trap: using serial mode to chain UI steps out of convenience.

CODE

```
test.describe('Order lifecycle', () => {
test.describe.configure({ mode: 'serial' });
let orderId: string;
test('create order', async ({ page }) => {
await page.goto('/orders/new');
await page.getByRole('button', { name: 'Create Order' }).click();
orderId = await page.getByTestId('order-id').innerText();
});
test('approve order', async ({ page }) => {
await page.goto(`/orders/${orderId}/approve`);
});
});

// Better: independent test using API setup, runs in parallel
test('full order lifecycle', async ({ request, page }) => {
const { id } = await (await request.post('/api/orders', { data: { product: 'MacBook Pro'
} })).json();
await request.patch(`/api/orders/${id}`, { data: { status: 'shipped' } });
await page.goto(`/orders/${id}`);
await expect(page.getByTestId('order-status')).toHaveText('Shipped');
});
```

Q 135 LEVEL · Senior

How do you pick the optimal worker count, given browser tests are I/O-bound rather than CPU-bound?

CONCEPT

The common assumption, workers equal CPU cores, is wrong for browser tests. Playwright tests spend most of their time waiting on the browser, the network, or the application under test, not on test-runner JavaScript. That makes them I/O-bound, so you can run more workers than cores before saturating CPU. The practical numbers: on a typical CI runner with **4 vCPUs**, **6-8 workers** often outperforms 4 because the extra workers run while others are waiting on network. The ceiling: memory and browser-process limits. Each Playwright worker runs its own browser instance; at roughly 200-400 MB per browser, 8 workers means 2-3 GB of RAM consumed by browsers alone. On a 7 GB GitHub runner you'll hit memory limits before CPU. The discipline: measure before assuming. Try **workers: 4**, workers: 6, workers: 8 on the same suite; the right count is the one where total wall-clock time stops decreasing. Going beyond that just adds contention without speedup.

INTERVIEW STRATEGY

Your job here is to show worker tuning as an empirical question, not a default. Lead with the I/O-bound nature of browser tests, then the cores-vs-workers mismatch. The 'memory ceiling before CPU ceiling' framing is what separates junior from senior thinking. The 'measure before assuming' rule is the closer. The mistake is defaulting to cores. They usually follow with ‘how do you know when you've added too many workers?’, answer wall-clock time stops decreasing or test flake increases due to contention.

How to phrase it

> *The common assumption that workers equal CPU cores is wrong for browser tests, and understanding why is what separates a tuned suite from a default one. Playwright tests spend most of their time waiting on the browser, the network, or the application under test, not on test-runner JavaScript or test-code computation. That makes them I/O-bound, so I can run more workers than cores before saturating CPU. The practical numbers I'd give to make this concrete: on a typical CI runner with four vCPUs, six to eight workers often outperforms four because the extra workers run while others are waiting on network. The ceiling that surprises people, and the part senior interviewers probe on, is memory and browser-process limits, not CPU. Each Playwright worker runs its own browser instance, and at roughly two hundred to four hundred megabytes per browser, eight workers means two to three gigabytes of RAM consumed by browsers alone. On a seven-gigabyte GitHub runner I'll hit memory limits before CPU does, which manifests as out-of-memory crashes rather than slowdowns, hard to diagnose if I'm thinking in CPU terms. The discipline I would lead with, because it converts this from guessing to measuring, is measure before assuming. Try workers four, workers six, workers eight on the same suite; the right count is the one where total wall-clock time stops decreasing. Going beyond that just adds contention without speedup. That's also my answer to the follow-up about knowing when there are too many workers. Two signals. Wall-clock time stops decreasing as I add workers, the marginal worker isn't doing useful work, just queueing on shared resources. Or test flake increases due to contention: workers competing for the same browser process pool, the same database connections, the same external service rate limits. Both are signs to back off, not push further. The classic error is defaulting to cores from a Node.js concurrency intuition that doesn't apply to browser workloads. Browser tests aren't CPU-spinning Promises; they're waiting on the world.*

Key points to hit

(1) Browser tests are I/O-bound: most time waiting on browser/network/app. **(2)** Workers > cores typically beats workers = cores (6-8 on a 4-vCPU runner). **(3)** Ceiling: memory (200-400 MB/worker) before CPU, OOM, not slowdown. **(4)** Discipline: measure 4 vs 6 vs 8 on the same suite; pick where time plateaus. **(5)** Asked next: too many workers -> wall-clock plateaus OR flake increases. **(6)** Where this goes wrong: workers = cores from Node.js intuition, doesn't apply to I/O-bound.

CODE

```
// playwright.config.ts: measure before locking in
export default defineConfig({
workers: process.env.CI
? Number(process.env.PW_WORKERS ?? 6) // empirically tuned for this runner
: '50%', // local: half the cores
fullyParallel: true,
});

// Tuning workflow: measure on the actual runner
// PW_WORKERS=4 npx playwright test -> total time T4
// PW_WORKERS=6 npx playwright test -> total time T6
// PW_WORKERS=8 npx playwright test -> total time T8
// PW_WORKERS=10 npx playwright test -> total time T10
//
// Pick the lowest count where time plateaus.
// On a 4-vCPU/7GB runner that's often 6-8, not 4.
```

Q 136 LEVEL · Senior

What is a shared-resource bottleneck, and how does it cap parallel test speedup?

CONCEPT

Even perfectly-isolated tests can be slowed by **shared infrastructure** they don't control: the test database accepting only N concurrent connections, an external service with a 100-per-minute rate limit, a single shared admin account that serialises auth, a captcha-protected endpoint that throttles under load. Adding workers beyond what the shared resource supports doesn't speed the suite up, the workers just wait. The pattern to diagnose: parallel runtime stops decreasing (or increases) when adding workers, but CPU and memory are nowhere near saturated. The fixes depend on the bottleneck: **pool of test accounts** (one per worker, indexed via **testInfo.parallelIndex**) for auth-rate issues; per-test or per-worker tenant isolation in the database so connections aren't fighting for the same rows; mock the rate-limited external service so internal tests aren't subject to its throttle. The principle: parallelism is bounded by the least-parallel resource, finding and removing those constraints is what buys you real speedup beyond what worker tuning alone can do.

INTERVIEW STRATEGY

Show resource-bottleneck diagnosis. Lead with the definition (shared infra capping parallelism below worker count), then the diagnostic signature (runtime plateaus, CPU/RAM not saturated). The three fixes (account pool, tenant isolation, mock external service) make it actionable. The 'bounded by least-parallel resource' rule is the experience marker. The trap is adding workers when the bottleneck is elsewhere. The next question is normally ‘how do you find the bottleneck?’, answer profile the test where it spends time, if waiting on DB or external, that's your bottleneck.

How to phrase it

> *Even perfectly-isolated tests can be slowed by shared infrastructure they don't control, and the part senior interviewers want to hear is that adding workers beyond what the shared resource supports doesn't speed the suite up, the workers just wait. The canonical examples I'd give to make it concrete: the test database accepting only N concurrent connections, so the eleventh worker queues for a connection. An external service with a hundred-per-minute rate limit, so workers fire requests faster than the service serves them and start getting 429s. A single shared admin account that serialises auth, so every test that logs in as admin contends for the same session. A captcha-protected endpoint that throttles under load, making tests intermittently slow. The diagnostic signature, and the part that distinguishes resource bottleneck from worker tuning, is parallel runtime stops decreasing or even increases when adding workers, but CPU and memory are nowhere near saturated. That's the telltale: the machine has capacity, the test isn't using it, something else is the limit. The fixes depend on the bottleneck. For auth-rate issues, a pool of test accounts, one per worker, indexed via testInfo dot parallelIndex, so worker zero uses test-user-0 at example.com, worker one uses test-user-1, no contention. For database connection limits or row-level contention, per-test or per-worker tenant isolation so connections aren't fighting for the same rows; faker plus worker index in the test data generates naturally unique keys. For rate-limited external services, mock them entirely so internal tests aren't subject to their throttle, separately from the dedicated tests that verify the integration with the real service. The principle I would close on, because it captures the model, is parallelism is bounded by the least-parallel resource. Finding and removing those constraints is what buys you real speedup beyond what worker tuning alone can do. That's also my answer to the follow-up about finding the bottleneck: profile the test, where is it spending time? If a five-second test is four seconds waiting on a database query that takes two seconds in isolation, the bottleneck is the database under concurrent load. The failure mode is adding workers when the bottleneck is elsewhere; the suite gets noisier without getting faster.*

Key points to hit

(1) Shared infra (DB connections, rate limits, shared accounts) caps parallelism. **(2)** Diagnostic: runtime plateaus while CPU/RAM not saturated -> resource bottleneck. **(3)** Fix 1: pool of test accounts indexed via testInfo.parallelIndex (auth). **(4)** Fix 2: per-test/per-worker tenant isolation (DB row contention). **(5)** Fix 3: mock the rate-limited external service (throttling). **(6)** What loses the point: adding workers when the bottleneck is elsewhere, noisier, not faster.

CODE

```
// Pool of test accounts: one per worker, no auth contention
export const test = base.extend<{ testAccount: { email: string; password: string } }>({
testAccount: async ({}, use, testInfo) => {
const idx = testInfo.parallelIndex; // 0, 1, 2, ...
await use({
email: `test-user-${idx}@example.com`,
password: process.env[`TEST_USER_${idx}_PW`]!,
});
},
});

// Per-worker unique data: faker + workerIndex avoids row contention
const email = `test+${faker.string.uuid()}-w${testInfo.workerIndex}@local`;

// Mock the rate-limited external service in internal tests
test.beforeEach(async ({ context }) => {
await context.route('**/api.stripe.com/**', route =>
route.fulfill({ status: 200, body: JSON.stringify({ id: 'pi_mock' }) }));
});
```

Q 137 LEVEL · Mid

How do test.describe.serial and test.describe.parallel override parallelism for specific blocks?

CONCEPT

Playwright's parallelism is configurable per scope, with two block-level modifiers complementing the config-level fullyParallel. **test.describe.serial(...)**: tests inside run one after another in declaration order on a single worker, regardless of fullyParallel. If one test fails, subsequent tests in the block are skipped, useful when later tests genuinely depend on earlier ones (multi-step flows that can't be split). **test.describe.parallel(...)**: tests inside run in parallel even when fullyParallel is false at config level. Useful in monorepos where the default is sequential but a specific block of independent tests should run concurrently. The rule: let the framework run tests in parallel by default; reach for serial only when tests genuinely depend on each other; reach for parallel-override only when fullyParallel isn't the suite default. The trap: using describe.serial because tests share state, when the right fix is to make them independent.

INTERVIEW STRATEGY

You need to give block-level parallelism control understood. Lead with both modifiers: serial (run in order on one worker, skip on fail) and parallel (override sequential default). The 'serial only for genuine deps, not shared state' rule is the seniority marker because shared state should be removed, not serialised. The classic error is using serial as a shortcut for shared-state tests. A common follow-up is ‘what happens when one fails in serial?’, answer subsequent tests skip; useful for the multi-step flow case.

How to phrase it

> *Playwright's parallelism is configurable per scope, with two block-level modifiers complementing the config-level fullyParallel. test.describe.serial: tests inside run one after another in declaration order on a single worker, regardless of what fullyParallel says at config level. The important behaviour that makes serial useful, and my answer to the follow-up about what happens on failure, is if one test fails, subsequent tests in the block are skipped. This is the right tool for multi-step flows that genuinely can't be split: a test that creates an order, a test that ships it, a test that delivers it, where each one depends on the previous having succeeded. Failing the create-order test means the ship-order test would just throw a confusing error about an order that doesn't exist; skipping it gives a cleaner report. test.describe.parallel: tests inside run in parallel even when fullyParallel is false at config level. Useful in monorepos or older configurations where the default is sequential but a specific block of independent tests should run concurrently. The rule I would lead with, because it captures the discipline, is let the framework run tests in parallel by default; reach for serial only when tests genuinely depend on each other; reach for parallel-override only when fullyParallel isn't the suite default. The trap, and the part where candidates lose points in senior interviews, is using describe.serial because tests share state, when the right fix is to make them independent. If three tests rely on the same user being logged in and the same data being seeded, the instinct is to put them in serial so they share the setup. The senior move is to refactor: a fixture that handles the setup for each test independently, and they go back to running in parallel. Serial is the right tool for ordered multi-step flows; the wrong tool for convenience-shared-state. The closer is that parallelism is the default for good reason, it makes suites fast, and every serialised block is a small tax on the whole suite's speed.*

Key points to hit

(1) describe.serial: tests run in declaration order on one worker. **(2)** Serial: if one fails, subsequent tests in the block SKIP. **(3)** describe.parallel: tests run in parallel even when fullyParallel is false. **(4)** Rule: parallel default; serial only for genuine multi-step dependencies. **(5)** Follow-up: failure in serial -> subsequent tests skip with clean report. **(6)** The common miss: describe.serial for shared state, refactor to independent tests instead.

CODE

```
// RIGHT USE: multi-step flow with genuine ordering
test.describe.serial('order lifecycle', () => {
let orderId: string;

test('create order', async ({ request }) => {
const r = await request.post('/api/orders', { data: { product: 'X' } });
orderId = (await r.json()).id;
});

test('ship the order', async ({ request }) => {
await request.patch(`/api/orders/${orderId}`, { data: { status: 'shipped' } });
});
// If create-order fails, ship-the-order is skipped automatically
});

// WRONG USE: shared state for convenience (refactor instead)
// test.describe.serial('user tests share login', () => { ... });
// Right fix: a loggedInUser fixture, tests run in parallel

// Force parallel in a sequential-default suite
test.describe.parallel('these can run concurrently', () => { /* ... */ });
```

Q 138 LEVEL · Senior

What data isolation strategies prevent parallel tests from corrupting each other?

CONCEPT

Three strategies layered by scope. Unique data per test: every test generates fresh data with **faker** for natural keys (emails, slugs, names) so two parallel tests never collide on a unique constraint. Combine with worker index in the seed for full uniqueness: **`test+${faker.string.uuid()}-w${testInfo.workerIndex}@local`**. **Worker-scoped tenants**: in multi-tenant systems, each worker operates on its own tenant ID created at worker startup and destroyed at teardown, tests within a worker share the tenant for setup efficiency, but no two workers see the same tenant's data. Test-scoped cleanup registry: every entity created is tracked, deleted in afterEach regardless of outcome (covered in S7 Q12). The combination scales: unique data avoids collision, worker tenants give organisational isolation, cleanup prevents accumulation. The principle that ties them: parallel tests are independent tests; any shared mutable state is a future flake waiting to happen. Independence isn't an optional optimisation: it's the precondition for parallelism to work at all.

INTERVIEW STRATEGY

They are looking to see data-isolation patterns at three scopes. Walk the three: unique data per test (faker + worker index), worker-scoped tenants, test-scoped cleanup registry. The 'independence is the precondition, not an optimisation' framing demonstrates calibration. The pitfall is shared test data 'for convenience' that breaks under load. The follow-up is often ‘what if the system doesn't support multi-tenant?’, answer fall back to unique-data-per-test with namespacing; cleanup becomes more critical.

How to phrase it

> *Three strategies layered by scope, each addressing a different kind of contention. First, unique data per test. Every test generates fresh data with faker for natural keys, emails, slugs, names, anything the database has a unique constraint on, so two parallel tests never collide on the same value. I combine with worker index in the seed for full uniqueness: backtick test plus faker dot string dot uuid dash w testInfo dot workerIndex at local. That guarantees uniqueness across the cartesian product of workers and tests within a worker. Second, worker-scoped tenants. In multi-tenant systems, each worker operates on its own tenant ID created at worker startup and destroyed at teardown. Tests within a worker share the tenant for setup efficiency, same baseline data, same configurations, but no two workers see the same tenant's data, so worker-level concurrency is naturally isolated at the organisational boundary the application already supports. Third, test-scoped cleanup registry. Every entity created in a test is tracked, and deleted in afterEach regardless of outcome, which I covered in the API testing section. Cleanup prevents test data from accumulating in the shared database where it would eventually cause contention or storage issues. The combination scales: unique data avoids collision on natural keys, worker tenants give organisational isolation, cleanup prevents accumulation. The principle I would close on, because it's the rule that captures all three, is parallel tests are independent tests; any shared mutable state is a future flake waiting to happen. Independence isn't an optional optimisation: it's the precondition for parallelism to work at all. Without isolation, adding workers just multiplies the contention. With isolation, adding workers gives near-linear speedup until the worker tuning ceiling. That's also my answer to the follow-up about systems without multi-tenant support. Fall back to unique-data-per-test with explicit namespacing in field values, every string includes a worker-and-test-unique prefix, and cleanup becomes more critical because there's no tenant-level teardown to lean on. The mistake is shared test data for convenience: it works fine until parallel load reveals the contention, and then the suite becomes unreliable under exactly the conditions you needed it to be reliable in.*

Key points to hit

(1) Unique data per test: faker + workerIndex for full uniqueness on natural keys. **(2)** Worker-scoped tenants: each worker its own tenant ID, org-level isolation. **(3)** Test-scoped cleanup registry: tracked entities deleted in afterEach. **(4)** Combination scales: collision-free + isolated + non-accumulating. **(5)** Expect next: no multi-tenant -> explicit namespacing + critical cleanup. **(6)** Avoid: shared test data 'for convenience', contention under parallel load.

CODE

```
// Layer 1: unique data per test (faker + workerIndex)
test('parallel-safe user creation', async ({ request, cleanup }, testInfo) => {
const email = `test+${faker.string.uuid()}-w${testInfo.workerIndex}@local`;
const res = await request.post('/api/users', { data: { email } });
expect(res.status()).toBe(201);
cleanup.add((await res.json()).id);
});

// Layer 2: worker-scoped tenant
export const test = base.extend<{}, { workerTenant: string }>({
workerTenant: [async ({ request }, use) => {
const r = await request.post('/api/tenants', { data: { name:
`worker-${testInfo.workerIndex}` } });
const id = (await r.json()).id;
await use(id);
await request.delete(`/api/tenants/${id}`).catch(() => {});
}, { scope: 'worker' }],
});

// Layer 3: test-scoped cleanup registry (see S7 Q12)
// cleanup fixture tracks IDs, deletes in afterEach regardless of outcome
```

Q 139 LEVEL · Senior

How do you diagnose when adding parallel workers makes the suite slower, not faster?

CONCEPT

Anti-pattern signature: increasing workers **increases or plateaus wall-clock time** instead of decreasing it. Three diagnostic dimensions to isolate the cause. **Resource saturation**: monitor CPU and RAM during the run, if CPU is at 100% across cores, workers are CPU-bound (unusual for browser tests; suggests something is compute-heavy); if RAM is near limit, you're hitting memory pressure that causes swapping and GC churn. Shared-resource contention (from Q7): if CPU/RAM are fine but runtime won't decrease, the bottleneck is downstream, the database, an external service, a captcha, an auth endpoint with rate limits. **Test-flake increase**: more workers reveal more race conditions in the application under test or in the tests themselves, isolation bugs that didn't show up at lower concurrency. The fixes: back off worker count to where speedup last applied; identify and remove the shared-resource bottleneck; fix the isolation bugs causing flake. The principle: parallelism is diagnostic about a system's actual concurrency story, if it breaks under parallel load, the problem is real, not theoretical, and production will hit it eventually.

INTERVIEW STRATEGY

The point of this one is to see anti-pattern diagnosis. Lead with the signature (runtime increases or plateaus with more workers), then walk three diagnostic dimensions: resource saturation, shared-resource contention, flake increase. The 'parallelism is diagnostic about the system's real concurrency story' framing is the senior signal. The classic error is just lowering workers without finding the root cause. Expect the follow-up ‘what's the most common cause?’, answer shared-resource bottleneck more often than CPU or flake.

How to phrase it

> *Anti-pattern signature is increasing workers increases or plateaus wall-clock time instead of decreasing it, and when this happens the diagnosis is a methodical three-dimension check, not a guess. First, resource saturation. Monitor CPU and RAM during the run. If CPU is at a hundred percent across cores, the workers are CPU-bound, which is unusual for browser tests and suggests something compute-heavy is happening, maybe expensive screenshot comparisons, complex JSON parsing in test code, or a misconfigured worker that's running tests serially. If RAM is near limit, I'm hitting memory pressure that causes swapping and garbage collection churn, which manifests as slow tests for no apparent reason. Second, shared-resource contention from Q7. If CPU and RAM are fine but runtime won't decrease, the bottleneck is downstream, the test database, an external service, a captcha, an auth endpoint with rate limits. The machine has capacity, the test isn't using it, something else is the limit. If they push on this in the follow-up, my answer is about the most common cause: shared-resource bottleneck, more often than CPU or flake. Modern CI runners have plenty of CPU; the downstream dependencies usually don't. Third, test-flake increase. More workers reveal more race conditions: either in the application under test or in the tests themselves. Isolation bugs that didn't show up at lower concurrency appear suddenly because the test pattern got faster. Two tests that happened to run sequentially now run together and step on each other's data. The fixes match the cause. Back off worker count to where speedup last applied, as the temporary fix. Identify and remove the shared-resource bottleneck, mock the rate-limited service, add database connections, pool test accounts. Fix the isolation bugs causing flake: unique data per test, worker-scoped tenants, cleanup registries. The principle I would close on, because it captures the deeper insight, is parallelism is diagnostic about a system's actual concurrency story. If the test suite breaks under parallel load, the problem is real, not theoretical, and production will hit it eventually, same database, same rate limits, same race conditions. Fixing it in tests pays off twice: faster CI now, fewer production incidents later. The trap is just lowering workers to make the symptom go away without finding the root cause; the underlying issue still exists, just hidden.*

Key points to hit

(1) Signature: more workers -> same or worse wall-clock time. **(2)** Check 1: CPU/RAM saturation (compute-bound or memory pressure). **(3)** Check 2: shared-resource contention (DB, external service, rate limits). **(4)** Check 3: flake increase (isolation bugs surfaced at higher concurrency). **(5)** Parallelism is diagnostic about real concurrency, prod will hit the same limits. **(6)** The easy mistake: lowering workers to hide symptom without finding root cause.

CODE

```
// Diagnose, don't guess: measure across worker counts on the same suite
//
// PW_WORKERS=2 npx playwright test --reporter=json > r2.json
// PW_WORKERS=4 npx playwright test --reporter=json > r4.json
// PW_WORKERS=8 npx playwright test --reporter=json > r8.json
//
// Plot total duration, retry count, max test duration
//
// If total duration plateaus or increases:
// - top + free -m during the run -> CPU/RAM saturation check
// - DB metrics during the run -> connection wait time, query queue depth
// - external API logs -> 429s or other throttle responses
// - Playwright report retries -> isolation-bug flake signature
//
// Fixes (in order of leverage):
// 1. Remove the shared-resource bottleneck (Q7 patterns)
// 2. Fix isolation bugs (Q9 patterns)
// 3. Back off worker count to the empirical sweet spot (Q6)
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What does fullyParallel: true change about Playwright's execution?

A) It runs tests on more machines
B) It forces serial execution
C) It disables worker isolation
D) It parallelises tests within a file, not just across files

2. How do you calculate the optimal shard count?

A) Always use the number of CPU cores
B) total_suite_time divided by target_time per shard
C) One shard per test file
D) Double the worker count

3. What is the most common cause of failures that appear only under parallelism?

A) Too few workers
B) A missing reporter
C) Shared test data that multiple tests fight over
D) Browser version mismatch

4. In a systematic flakiness process, what role do retries play?

A) They are the fix for flaky tests
B) They make tests deterministic
C) They replace the trace viewer
D) A safety net while you track, quarantine and root-cause the flakiness

5. When is test.describe.configure({ mode: 'serial' }) genuinely justified?

A) For a real state transition neither the UI nor the API lets you jump into
B) Whenever tests are slow
C) To chain UI steps for convenience
D) For every describe block

Section 9 · Answer key

#	Answer	Why and where to revisit
1	D) It parallelises tests within a file, not just across files	By default files run in parallel; fullyParallel also parallelises the tests inside each file. *Revisit Section 9.*
2	B) total_suite_time divided by target_time per shard	Divide total suite time by your target per-shard time (around 10 minutes) to size the shard matrix. *Revisit Section 9.*
3	C) Shared test data that multiple tests fight over	Hardcoded shared data causes collisions in parallel; the fix is unique per-test data via faker with API teardown. *Revisit Section 9.*
4	D) A safety net while you track, quarantine and root-cause the flakiness	Retries are a net, not a fix; you still trace and remove the real cause (missing wait, shared state, race). *Revisit Section 9.*
5	A) For a real state transition neither the UI nor the API lets you jump into	Serial mode is a rare escape hatch for genuine sequential state machines; usually the dependency should be removed via API setup. *Revisit Section 9.*

SECTION 10

CI/CD Integration

CI/CD is where a framework proves its worth, and interviewers want to see that you design a pipeline deliberately rather than copy a starter YAML. The recurring themes: get a full picture of failures, keep diagnostics when tests fail, and make test results a deployment decision. These fourteen questions cover GitHub Actions with sharding, Jenkins with Docker, test tagging, secrets management, quality gates, and a professional tag taxonomy.

In this section

- GitHub Actions sharding: fail-fast false, if always() artifacts, a merge-reports job.
- Jenkins with the official Playwright Docker image and the --ipc=host gotcha.
- Tagging for selective execution: smoke on PRs, regression nightly.
- Secrets injected at runtime with a requireEnv guard and .env.example.
- Quality gates: block deploy on pass-rate < 95% or flaky-rate > 5%.
- A four-tier tag taxonomy with enforced time budgets; the v1.42+ tag option.

Q 140 LEVEL · Mid

How do you set up Playwright in GitHub Actions with sharding?

CONCEPT

GitHub Actions is the most common CI for Playwright. The setup uses a matrix strategy for sharding and uploads artifacts for reports and traces. Three decisions define a good pipeline. **fail-fast: false** on the matrix so that if one shard fails, the others still complete and you get the full failure picture, not just the first shard's. **if: always()** on the artifact upload so traces and reports are kept even when tests fail, which is exactly when you need them. And a separate **merge-reports** job that downloads the per-shard blob reports and merges them into one HTML report so stakeholders see a single unified result. The earlier stages (typecheck, lint, cached browser install) gate the expensive test run.

INTERVIEW STRATEGY

They want to know if you understand a pipeline or just pasted one. Do not narrate the whole YAML, defend the three decisions: fail-fast false for full failure visibility, if always() so you keep diagnostics on failure, and the merge-reports job for one unified report. The classic error is leaving fail-fast at its default, which hides failures in the other shards. The follow-up is often 'why merge the reports?', answer that four separate shard reports are unreadable for stakeholders; one merged HTML report is the deliverable.

How to phrase it

> *GitHub Actions is the common case, and rather than read out the YAML I focus on three decisions I can defend. First, fail-fast false on the matrix. By default, if one shard fails the others get cancelled, but I want every shard to finish so I see the full set of failures in one run rather than just whichever shard failed first, otherwise I fix one thing, rerun, and discover the next failure an hour later. Second, if always() on the artifact upload. Traces and reports are most valuable exactly when tests fail, so I make sure they upload even on failure; the default would skip them on a failed job, which is backwards. Third, a separate merge-reports job, and this is my answer to the follow-up about why merge: each shard produces a blob report, and four separate shard reports are unreadable for a stakeholder, so I download all the blobs and merge them into one HTML report that shows a single unified result. I also order the cheap checks first, typecheck and lint before installing browsers and running tests, so a type error fails in seconds instead of after browser startup. Those decisions are what turn a copy-pasted pipeline into one that actually gives the team fast, complete, diagnosable feedback.*

Key points to hit

(1) Matrix strategy shards the suite across runners. **(2)** fail-fast: false, all shards finish, full failure picture. **(3)** if: always() on uploads, keep traces/reports when tests fail. **(4)** merge-reports job combines blob reports into one HTML report. **(5)** Follow-up: why merge, four shard reports are unreadable; one is the deliverable. **(6)** Trap: leaving fail-fast at default, hiding other shards' failures.

CODE

```
jobs:
test:
strategy:
fail-fast: false # all shards finish
matrix: { shardIndex: [1,2,3,4], shardTotal: [4] }
steps:
- run: npm run typecheck # cheap checks gate the run
- run: npm run lint
- run: npx playwright install --with-deps chromium
- run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
- if: always() # keep diagnostics on failure
uses: actions/upload-artifact@v4
with: { name: blob-${{ matrix.shardIndex }}, path: blob-report/ }
merge-reports:
needs: test
if: always()
steps:
- run: npx playwright merge-reports --reporter html ./all-blob-reports
```

Q 141 LEVEL · Mid

How do you configure Playwright for Jenkins CI?

CONCEPT

Jenkins uses a **Jenkinsfile** with stages, and JUnit XML output integrates with its built-in test-result visualisation. The recommended setup runs inside the official Playwright Docker image (mcr.microsoft.com/playwright), which ships every OS-level dependency pre-installed and is tested against a specific Playwright version, so you never manage browser dependencies by hand. The notorious gotcha is **--ipc=host**: Chromium uses shared memory for inter-process communication, Docker's default **/dev/shm** is too small, and without this flag Chromium crashes silently. Stages typically run install, typecheck, then test, publishing JUnit results and archiving the HTML report in a post-always block so artifacts survive failures.

INTERVIEW STRATEGY

This question rewards real Docker-in-CI scars. Cover the Jenkinsfile structure briefly, then make the --ipc=host gotcha the centrepiece, knowing that Chromium crashes silently without it is the single clearest experience signal here. Mention the official Docker image for pinned dependencies. The thing to avoid is installing browsers manually and fighting missing system libraries. Expect the follow-up 'why does Chromium crash in Docker?', answer that the default /dev/shm is too small and --ipc=host shares the host IPC namespace to fix it.

How to phrase it

> *For Jenkins I use a Jenkinsfile with stages, and I lean on JUnit XML output because it plugs straight into Jenkins' built-in test-result view. I run everything inside the official Playwright Docker image rather than installing browsers by hand, because that image ships all the OS-level dependencies pre-installed and is tested against its specific Playwright version, so I never chase missing system libraries. The thing I always call out, because it has bitten me and it signals real experience, is the --ipc=host flag. Chromium uses shared memory for inter-process communication, and Docker's default /dev/shm is too small, so without that flag Chromium crashes silently, the tests just die with no clear error and you lose an afternoon. That is also my answer to the follow-up about why Chromium crashes in Docker. --ipc=host shares the host's IPC namespace and gives Chromium the shared memory it needs. My stages run install, then typecheck, then test, and in a post-always block I publish the JUnit results and archive the HTML report, so the artifacts survive even when the run fails. The combination of the official image and that one flag is what makes Playwright-on-Jenkins stable rather than mysteriously flaky.*

Key points to hit

(1) Jenkinsfile stages; JUnit XML feeds Jenkins' built-in test view. **(2)** Use the official Playwright Docker image: deps pre-installed, version-pinned. **(3)** --ipc=host is the key gotcha: without it Chromium crashes silently. **(4)** Expect next: why, default /dev/shm too small; --ipc=host shares host IPC. **(5)** post-always block publishes results and archives the report on failure. **(6)** Where this goes wrong: installing browsers manually and fighting missing system libs.

CODE

```
pipeline {
agent {
docker {
image 'mcr.microsoft.com/playwright:v1.45.0-jammy'
args '--ipc=host' // without this, Chromium crashes silently
}
}
environment { CI = 'true'; BASE_URL = credentials('staging-base-url') }
stages {
stage('Install') { steps { sh 'npm ci' } }
stage('Typecheck') { steps { sh 'npm run typecheck' } }
stage('Test') {
steps { sh 'npx playwright test --reporter=junit,html' }
post { always {
junit 'results/junit.xml'
archiveArtifacts artifacts: 'playwright-report/**', fingerprint: true
} }
}
}
}
```

Q 142 LEVEL · Junior to Mid

How do you implement test tagging and selective test execution in CI?

CONCEPT

Tags let you run subsets of tests for different CI triggers, smoke tests on every PR, full regression on nightly builds. You tag a test by including an **@-prefixed label** in its title (or via the tag option), then select with **--grep** and exclude with **--grep-invert**. Wire one npm script per tag (test:smoke, test:regression, test:critical, test:not-slow) so each maps to a CI trigger. In the workflow, PRs run the smoke script for fast feedback while a scheduled cron runs the full regression nightly. The point is balancing speed and coverage: developers get quick PR feedback, and the nightly run still catches regressions across the whole suite.

INTERVIEW STRATEGY

They want to see you think about CI economics, not just that tags exist. Frame tagging as the speed-versus-coverage balance: fast smoke on PRs, full regression nightly. Give your standard tag set with purpose attached to each. The thing to avoid is running the entire suite on every PR, which makes CI slow and erodes trust. They usually follow with 'what runs on a PR versus nightly?', answer the 20 critical smoke tests under three minutes on PRs, the full regression on the nightly cron.

How to phrase it

> *Tags let me run different subsets for different CI triggers, and I frame the whole thing as balancing speed against coverage, because that is what the question is really about. I tag a test with an @-prefixed label, @smoke, @regression, @critical, select with --grep and exclude with --grep-invert, and wire one npm script per tag so each maps cleanly to a trigger. My standard set, with a purpose for each: @smoke for the roughly twenty most critical happy-path tests that run on every PR in under three minutes, @regression for the full suite, @critical for tests that block deployment, and @slow for tests over thirty seconds. That answers the follow-up about PR versus nightly directly: PRs run the smoke script so developers get fast feedback, and a scheduled cron runs the full regression nightly to catch anything the smoke set misses. The mistake I have seen teams make, and the trap here, is running the entire suite on every PR, it makes CI slow, developers start merging without waiting for it, and then CI stops being a safety net at all. Tagging is the optimisation that keeps CI both fast enough to trust and thorough enough to catch regressions, and most teams add it far too late.*

Key points to hit

(1) Tag with @-labels; select with --grep, exclude with --grep-invert. **(2)** One npm script per tag, each mapped to a CI trigger. **(3)** @smoke ~20 critical tests under 3 min on PRs; @regression nightly. **(4)** They usually ask next: PR runs smoke for speed; cron runs full regression nightly. **(5)** Balances fast PR feedback with full nightly coverage. **(6)** What loses the point: running the whole suite on every PR, slow CI erodes trust.

CODE

```
test('@smoke @critical login flow', async ({ page }) => { /* ... */ });
test('@regression user management', async ({ page }) => { /* ... */ });

// package.json
{
"scripts": {
"test:smoke": "playwright test --grep @smoke",
"test:regression": "playwright test --grep @regression",
"test:not-slow": "playwright test --grep-invert @slow"
}
}

# workflow: smoke on PRs, full regression nightly
on:
pull_request: # smoke only
schedule: [{ cron: '0 2 * * *' }] # regression at 2am
```

Q 143 LEVEL · Mid

How do you manage test environment secrets in CI?

CONCEPT

Secrets are injected as **environment variables at runtime** from the CI secret store, never committed, never logged, never hardcoded in test files. The pattern that makes this robust is a **requireEnv** helper that reads each variable and throws immediately with a clear, actionable message if it is missing. Without it, a missing variable surfaces as a cryptic failure deep in a test, a 'cannot read property fill of undefined' that takes ten minutes to trace back to a configuration problem. The explicit startup validation converts a confusing runtime error into a clear configuration error. Pair it with a committed **.env.example** listing every required variable name (no values), which documents the configuration for new team members.

INTERVIEW STRATEGY

Beyond 'use CI secrets', the interviewer is listening for robustness and developer experience. State the boundary, injected at runtime, never committed or logged, then make requireEnv the centrepiece: fail fast with a clear message instead of a cryptic downstream crash. The .env.example-as-documentation point is a nice touch. The risk is hardcoding credentials, even in staging. The next question is normally 'what happens when a variable is missing?', answer that requireEnv throws at startup naming the variable, instead of a confusing undefined error mid-test.

How to phrase it

> *Secrets are injected as environment variables at runtime from the CI secret store, and the rules are firm: never committed, never logged, never hardcoded in a test file. But the part I would actually spend time on, because it is about robustness and developer experience, is the requireEnv helper I add to every project. It reads each required variable and throws immediately with a clear, actionable message if it is missing, something like 'ADMIN_PASSWORD is not set, check your CI secrets'. That is the answer to the follow-up about what happens when a variable is missing. Without that helper, a missing variable does not fail cleanly; it surfaces as a cryptic error deep in a test, the classic cannot-read-property-fill-of-undefined, and someone burns ten minutes tracing it back to a configuration problem that had nothing to do with the test logic. requireEnv turns a confusing runtime error into a clear configuration error at startup. I also commit a .env.example file that lists every required variable name with no values, which serves as living documentation so a new team member knows exactly what to configure. The trap I am avoiding is hardcoding credentials anywhere, even in staging, because secrets in source control are a security incident waiting to happen and they leak into git history permanently.*

Key points to hit

(1) Inject secrets at runtime from the CI store: never commit or log them. **(2)** requireEnv throws at startup with a clear message if a var is missing. **(3)** Turns a cryptic mid-test crash into an actionable config error. **(4)** Commit .env.example (names, no values) as configuration documentation. **(5)** The next question: missing var, requireEnv names it at startup, not undefined mid-test. **(6)** The common miss: hardcoding credentials anywhere, even staging.

CODE

```
// src/config/env.ts
function requireEnv(name: string): string {
const value = process.env[name];
if (!value) {
throw new Error(`Required env var ${name} is not set. Check .env or CI secrets.`);
}
return value;
}

export const config = {
baseURL: requireEnv('BASE_URL'),
adminEmail: requireEnv('ADMIN_EMAIL'),
adminPassword: requireEnv('ADMIN_PASSWORD'),
} as const;

# GitHub Actions injects from repository secrets
# env:
# ADMIN_PASSWORD: ${{ secrets.ADMIN_PASSWORD }}
```

Q 144 LEVEL · Mid to Senior

How do you implement quality gates in CI using Playwright results?

CONCEPT

Quality gates block deployment when test metrics fall below thresholds. A small **custom reporter** tallies passed, failed, and flaky counts in onTestEnd, computes the pass rate and flaky rate in onEnd, and calls **process.exit(1)** if a threshold is breached, failing the pipeline. Typical gates: pass rate above 95% and flaky rate below 5%. This transforms test results from a passive report into an active **deployment decision**, and it creates accountability: a team cannot ship with a degraded suite. The framing for stakeholders is that the suite is a quality signal, not just a report, when the signal degrades, you investigate before shipping rather than after.

INTERVIEW STRATEGY

Your job here is to show you connect testing to delivery. Frame quality gates as turning a report into a deployment decision, give two concrete thresholds, and explain the accountability they create. The stakeholder line, the suite is a quality signal, not just a report, is what turns a mechanics answer into an ownership one. The classic error is treating the report as passive output nobody acts on. A common follow-up is 'what thresholds and why?', answer pass rate above 95% and flaky rate below 5%, tuned so the gate catches real degradation without blocking on noise.

How to phrase it

> *Quality gates block deployment when the test metrics fall below thresholds, and I frame them as the thing that turns test results from a report into a deployment decision. Mechanically it is a small custom reporter. I tally passed, failed and flaky in onTestEnd, compute the pass rate and flaky rate in onEnd, and call process.exit with a failure code if a threshold is breached, which fails the pipeline and blocks the deploy. The two gates I set, which answers the follow-up about thresholds and why, are pass rate above ninety-five percent and flaky rate below five percent, tuned so the gate catches genuine degradation without tripping on a little unavoidable noise. The real value is accountability: a team cannot quietly ship with a degraded suite, because the gate stops them and forces a conversation. The way I present this to stakeholders, and I think this framing is what makes it land, is that our test suite is a quality signal, not just a report, when the signal degrades, we investigate before shipping rather than discovering the problem in production. The trap I am avoiding is the common situation where the report is passive output that nobody looks at; a gate makes the result consequential, which is what changes team behaviour around test quality.*

Key points to hit

(1) A custom reporter tallies results and exits non-zero past a threshold. **(2)** Gates: pass rate > 95%, flaky rate < 5%. **(3)** Turns test results into a deployment decision, not a passive report. **(4)** Creates accountability: cannot ship with a degraded suite. **(5)** Stakeholder framing: the suite is a quality signal, not just a report. **(6)** Avoid: treating the report as output nobody acts on.

CODE

```
import { Reporter, TestCase, TestResult } from '@playwright/test/reporter';

class QualityGateReporter implements Reporter {
private passed = 0; private failed = 0; private flaky = 0;
onTestEnd(_t: TestCase, r: TestResult) {
if (r.status === 'passed') this.passed++;
else if (r.status === 'failed') this.failed++;
else if (r.status === 'flaky') this.flaky++;
}
onEnd() {
const total = this.passed + this.failed + this.flaky;
const passRate = (this.passed / total) * 100;
const flakyRate = (this.flaky / total) * 100;
if (passRate < 95) { console.error('GATE FAILED: pass rate < 95%'); process.exit(1); }
if (flakyRate > 5) { console.error('GATE FAILED: flaky rate > 5%'); process.exit(1); }
}
}
```

Q 145 LEVEL · Mid to Senior

How do you design a professional tagging and filtering strategy in CI/CD?

CONCEPT

Beyond basic tags, a professional taxonomy treats tags as a CI execution contract: each tag maps to a specific pipeline trigger, a specific time budget, and a specific quality signal. A four-tier set works well: **@smoke** (15-20 critical happy-path tests, under 3 minutes, every PR), **@critical** (revenue/security paths like login and checkout, every merge to main), @regression (the full suite, nightly, 20-30 minutes acceptable), and @slow (over 30 seconds, excluded from PR runs). The discipline is enforcing the time budget: if @smoke creeps to 8 minutes, you audit and split or remove tests until it is back under 3. Prefer the **v1.42+ tag option** over inline title tags, it is queryable programmatically and does not pollute the test name in reports.

INTERVIEW STRATEGY

This is the senior version of the tagging question, the interviewer wants taxonomy plus governance. Present the four tiers with a trigger and a time budget each, then make the real point: the discipline is enforcing the budget, auditing @smoke when it creeps. Prefer the v1.42+ tag option and say why. Where this goes wrong is letting smoke tests bloat until PR feedback is slow and developers stop trusting CI. The follow-up is often ‘how do you keep smoke fast over time?’, answer that you treat the budget as non-negotiable and split or cut tests to hold it.

How to phrase it

> *The professional version of tagging treats tags as a CI execution contract: every tag maps to a specific trigger, a specific time budget, and a specific quality signal, not just a label. My taxonomy has four tiers. @smoke is the fifteen to twenty most critical happy-path tests, must pass in under three minutes, runs on every PR. @critical is the tests that directly protect revenue or security, login, checkout, payment, and runs on every merge to main. @regression is the full suite, runs nightly, and twenty to thirty minutes is acceptable there. @slow is anything over thirty seconds, like report generation, excluded from PR runs and included nightly. But the part that makes it professional rather than cosmetic, and my answer to how I keep smoke fast over time, is enforcing the time budget: if @smoke starts taking eight minutes, I audit it and split or remove tests until it is back under three, because fast PR feedback is non-negotiable, the moment CI is slow, developers stop waiting for it and it stops being a safety net. One current detail: I use the v1.42-plus tag option rather than inline title tags, because it is queryable programmatically and does not pollute the test name in reports. So it is taxonomy plus governance, and the governance is the harder, more valuable half.*

Key points to hit

(1) Tags are a CI contract: trigger + time budget + quality signal. **(2)** Four tiers: @smoke (PR, <3 min), @critical (merge to main), @regression (nightly), @slow (excluded from PR). **(3)** Discipline: enforce the budget, audit and split @smoke when it creeps. **(4)** Prefer the v1.42+ tag option: queryable, no test-name pollution. **(5)** Asked next: keep smoke fast by treating the budget as non-negotiable. **(6)** The easy mistake: letting smoke bloat until PR feedback is slow and distrusted.

CODE

```
// v1.42+ tag option (preferred: queryable, clean report names)
test('user can log in', { tag: ['@smoke', '@critical'] }, async ({ page }) => { /* ...
*/ });
test('report generation', { tag: ['@regression', '@slow'] }, async ({ page }) => { /*
... */ });

// package.json
{
"scripts": {
"test:smoke": "playwright test --grep @smoke",
"test:release": "playwright test --grep '@smoke|@critical|@regression'"
}
}

# triggers: smoke on PR, critical on main, release-gate on release/*
# each tier enforces its own time budget
```

Q 146 LEVEL · Senior

How do you balance shards so one shard doesn't dominate the wall-clock time?

CONCEPT

Playwright's default sharding splits the ordered list of tests into **contiguous blocks**, one block per shard, balancing by the number of tests. It is deterministic, but equal test counts are not equal minutes, so it is unbalanced whenever some tests are far slower than others. The docs even note that some shards may run fewer tests, or none at all. The result: shard 3 finishes in 4 minutes and shard 7 takes 18 minutes, so the pipeline waits on shard 7 and the other shards are wasted. Two ways to fix it. **Test-level granularity**: enable **fullyParallel: true** so individual tests can move between shards rather than whole files, evening out the distribution. Duration-aware sharding: parse the JSON report from a previous run to compute average duration per test, then use a bin-packing algorithm to distribute tests so each shard has roughly equal total duration, takes 20 lines of script in CI but turns wildly unbalanced shards into shards within 10% of each other. The principle: wall-clock time is set by the slowest shard, so balanced shards are the investment that compounds across every CI run.

INTERVIEW STRATEGY

Show shard imbalance diagnosed and fixed. Lead with why default sharding is unbalanced (contiguous blocks balanced by test count, not by time), then the two fixes: fullyParallel for test-level distribution, duration-aware bin-packing for serious imbalance. The 'wall-clock = slowest shard' framing is the senior signal. The thing to avoid is adding shards without balancing them. Expect the follow-up ‘how do you measure imbalance?’, answer compare duration of slowest vs fastest shard; if >2x the gap is the problem.

How to phrase it

> *Playwright's default sharding takes the ordered list of tests and gives each shard a contiguous block of it, balanced by the number of tests. That's deterministic, but equal test counts are not equal minutes, so it goes out of balance the moment some tests are much slower than others. The visible result, and the part interviewers want to hear diagnosed, is that shard three finishes in four minutes and shard seven takes eighteen minutes, so the pipeline waits on shard seven while the other shards sit idle. Adding more shards doesn't help if the imbalance is the problem; it just creates more idle workers. Two ways to fix it. First, test-level granularity: enable fullyParallel true in the config so individual tests, not whole files, can move between shards. This evens out distribution naturally, because tests are smaller units than files and fifty tests in one file get spread across the available shards instead of all running on one. Second, for cases where fullyParallel isn't enough or where some individual tests are themselves much longer than others, duration-aware sharding. I parse the JSON report from a previous run to compute the average duration per test, then use a bin-packing algorithm to distribute tests so each shard has roughly equal total duration. That's twenty lines of script in CI, sort tests by duration descending, assign each to the currently-shortest shard, and turns wildly unbalanced shards into shards within ten percent of each other. The principle I would lead with, because it's the rule that explains why this matters at all, is wall-clock time is set by the slowest shard. Eight shards finishing in four minutes plus one shard taking eighteen minutes is an eighteen-minute pipeline, not a four-minute one. Balanced shards are the investment that compounds across every CI run. That's also my answer to the follow-up about measuring imbalance: compare the duration of the slowest shard versus the fastest. If the ratio is greater than two times, the gap is the problem and balancing pays for itself. The mistake is adding shards to make CI faster without balancing them, you just have more shards waiting on the slowest one.*

Key points to hit

(1) Default sharding = contiguous blocks balanced by TEST COUNT, not by time. **(2)** fullyParallel: true, distribute by test, not file (the simple fix). **(3)** Duration-aware bin-packing for serious imbalance, ~20 lines of CI script. **(4)** Wall-clock time = duration of slowest shard: imbalance dominates pipeline. **(5)** Follow-up: measure imbalance as ratio slowest/fastest; >2x = balance pays. **(6)** Trap: adding shards without balancing them, just more idle workers.

CODE

```
// 1. Test-level distribution: the cheap fix
// playwright.config.ts
export default defineConfig({
fullyParallel: true,
workers: process.env.CI ? 4 : undefined,
});

// 2. Duration-aware shard assignment: the precise fix
// scripts/balanced-shards.ts (run in CI before tests)
// 1. Read previous run's report.json
// 2. For each test, compute average duration over N recent runs
// 3. Sort tests descending by duration
// 4. For each test, assign to the shard with the lowest current total
// 5. Emit a per-shard test-list file consumed via --grep-invert / --shard-config
//
// Result: shard durations within ~10% of each other instead of 4 vs 18 minutes
```

Q 147 LEVEL · Mid

How do you cache Playwright browser binaries and node_modules in CI without breaking determinism?

CONCEPT

Two cache layers, two cache keys. **node_modules cache**: key on the **hash of package-lock.json**, restored before npm ci, saving 30-90 seconds per run on a typical Playwright project. Browser binaries cache: key on the Playwright version (read from package.json or the lockfile), restored before npx playwright install, saving 60-120 seconds. The discipline that prevents the silent-stale-cache bug: include the OS in the cache key (Ubuntu 22 binaries won't run on Ubuntu 24), and use restore-keys **conservatively**, a partial-match restore from a previous Playwright version means the binaries might be wrong for the current version, causing mysterious failures. Cache the exact match or rebuild. The principle: cache aggressively, invalidate precisely. A 90% hit rate on a correctly-keyed cache saves real engineering time; a 99% hit rate on a wrongly-keyed cache silently breaks one in every hundred runs.

INTERVIEW STRATEGY

They are looking to see caching discipline, not just enabling cache. Lead with the two cache layers and their keys: package-lock hash for node_modules, Playwright version for browsers. Then make the OS-in-the-cache-key and no-loose-restore-keys disciplines the senior signal. The 'cache aggressively, invalidate precisely' framing is the closer. The mistake is loose restore-keys that match wrong versions. They usually follow with ‘how do you debug a cache-related test failure?’,

answer disable cache for one run; if it passes, the cache key is wrong.

How to phrase it

> *Two cache layers, two cache keys, and both need to be configured precisely or they cause more trouble than they save. Node modules cache: I key on the hash of package-lock dot json, restore before npm ci, and that saves thirty to ninety seconds per run on a typical Playwright project. Browser binaries cache: I key on the Playwright version, read from package.json or directly from the lockfile, restore before npx playwright install, and that saves sixty to a hundred and twenty seconds. The discipline that prevents the silent-stale-cache bug, and the framing I would emphasise because most candidates miss this, is two rules. First, include the OS in the cache key. Ubuntu 22 browser binaries won't run on Ubuntu 24, and a cross-OS cache hit produces baffling failures. Second, use restore-keys conservatively or not at all. A partial-match restore from a previous Playwright version means the cached binaries might be wrong for the current version, causing mysterious failures that take hours to diagnose because the developer assumes the cache hit was correct. Cache the exact match or rebuild; the rebuild cost is much less than the debugging cost of a wrong cache. The principle I close on, because it captures the calibration, is cache aggressively, invalidate precisely. A ninety percent hit rate on a correctly-keyed cache saves real engineering time across every build. A ninety-nine percent hit rate on a wrongly-keyed cache silently breaks one in every hundred runs, and the team spends more time debugging those failures than they ever saved on cache hits. That's also my answer to the follow-up about debugging a cache-related test failure: disable the cache for one run by changing the key. If the test passes with cache disabled, the cache key is wrong; fix the key, not the test. The mistake is loose restore-keys that match a previous version's cache and produce inscrutable failures.*

Key points to hit

(1) node_modules: cache key on hash of package-lock.json. **(2)** Browser binaries: cache key on Playwright version. **(3)** Include OS in the cache key: Ubuntu 22 binaries fail on Ubuntu 24. **(4)** Use restore-keys conservatively: loose matches produce stale binaries. **(5)** Expect next: debug cache failures by invalidating the key for one run. **(6)** Where this goes wrong: loose restore-keys, wrong cache hit = inscrutable failures.

CODE

```
# .github/workflows/test.yml: two-layer caching
- name: Cache node_modules
uses: actions/cache@v4
with:
path: node_modules
key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
# NO restore-keys: exact match or rebuild

- run: npm ci

- name: Get Playwright version
id: pw
run: |
echo "version=$(node -p \"require('@playwright/test/package.json').version\")" >>
"$GITHUB_OUTPUT"

- name: Cache Playwright browser binaries
uses: actions/cache@v4
with:
path: ~/.cache/ms-playwright
key: ${{ runner.os }}-playwright-${{ steps.pw.outputs.version }}
# again, NO restore-keys for browser binaries

- run: npx playwright install --with-deps chromium
```

Q 148

LEVEL · Mid to Senior

When do you use the official Playwright Docker image versus a custom one, and how do you keep image versions deterministic?

CONCEPT

The **official mcr.microsoft.com/playwright** image ships with Node, Playwright, and browsers pre-installed, perfect for fast CI runs because there's no install step. Use it when your CI needs fast, deterministic, consistent runtime across local Docker dev, CI, and any cross-team contributor's machine. Build a custom image extending the official one when you need additional system dependencies (PostgreSQL client for DB-cleanup, ImageMagick for visual regression pre-processing, internal CLI tools), company-specific certificates, or your own test fixtures baked in. The determinism rule: always pin the Playwright image to a specific tag matching your @playwright/test version, **mcr.microsoft.com/playwright:v1.47.0-jammy**, not **:latest**. Pinning to latest means your CI uses different browser binaries than your developers, and tests that pass in dev silently fail in CI when Microsoft pushes a new image.

INTERVIEW STRATEGY

The point of this one is to see image-strategy discipline. Lead with the official image (Node + Playwright + browsers preinstalled, fast CI) vs custom (when you need extra system deps). The pin-to-specific-version rule is the experience marker because :latest is the classic foot-gun. The trap is :latest in production CI. The next question is normally ‘what if dev and CI need different tooling?’, answer custom image extending the official one, pinned to the same Playwright version.

How to phrase it

> *The official mcr.microsoft.com slash playwright image ships with Node, Playwright, and the browsers pre-installed, which is perfect for fast CI runs because there's no install step, no npm ci, no npx playwright install, the container starts ready to run tests. I use it when CI needs fast, deterministic, consistent runtime across local Docker dev, CI, and any cross-team contributor's machine. Same image everywhere means same browser binaries, same Node version, same Playwright version, no works-on-my-machine debugging. I build a custom image extending the official one when I need things the base image doesn't have: additional system dependencies like a PostgreSQL client for database cleanup, ImageMagick for visual regression pre-processing, internal CLI tools, company-specific certificates, or test fixtures baked in. The custom Dockerfile starts FROM the official image at a specific tag, runs the apt-get installs, copies in the extras, and publishes to my company's container registry for use across CI and dev. The determinism rule I would lead with, because it's the classic foot-gun, is always pin the Playwright image to a specific tag matching the at-playwright-slash-test version. So mcr.microsoft.com slash playwright colon v1.47.0-jammy, not colon latest. Pinning to latest means CI uses different browser binaries than developers do, and tests that pass in dev silently fail in CI when Microsoft pushes a new image. The follow-up I would answer is what if dev and CI need different tooling. Custom image extending the official one, pinned to the same Playwright version, distributed to both dev and CI, they get the same baseline plus the team-specific extras. The failure mode is colon latest in production CI: works for weeks, then breaks the day Microsoft rebuilds the image, and nobody can reproduce locally because dev still has the old image cached.*

Key points to hit

(1) Official image: Node + Playwright + browsers preinstalled, fast CI start. **(2)** Custom image extends official when extra system deps or fixtures needed. **(3)** ALWAYS pin to specific tag matching @playwright/test version. **(4)** :latest causes silent dev-vs-CI drift on Microsoft's image rebuild. **(5)** They usually ask next: custom image extending official handles dev+CI extras consistently. **(6)** What loses the point: :latest in production CI, works until Microsoft pushes a new build.

CODE

```
# Custom Dockerfile extending the official image, pinned version
FROM mcr.microsoft.com/playwright:v1.47.0-jammy

# Additional system deps the official image doesn't have
RUN apt-get update && apt-get install -y --no-install-recommends \\
postgresql-client imagemagick \\
&& rm -rf /var/lib/apt/lists/*

# Company certificate for internal HTTPS endpoints
COPY certs/company-ca.crt /usr/local/share/ca-certificates/
RUN update-ca-certificates

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npm", "test"]

# WRONG: silent dev/CI drift
# FROM mcr.microsoft.com/playwright:latest
```

Q 149 LEVEL · Mid to Senior

How do you decide which browsers to run in CI and when each pays its way?

CONCEPT

Running all three browsers (Chromium, Firefox, WebKit) on every PR triples CI time and infrastructure cost, it's the right choice only when cross-browser bugs are a real risk that justifies the spend. The tiered approach: **Chromium on every PR** (the fast, broad-coverage default, catches 90% of issues), Firefox and WebKit on main branch merges (catches the cross-browser-specific bugs without slowing PR feedback), **all three on release branches** (the final gate before customers see the build). The decision data: track which browser-specific tests have actually failed in the last 6 months. If WebKit-only failures are 1 per quarter, running WebKit on every PR is wasted spend; if they're weekly, the PR-level cost is justified. The principle: **cross-browser coverage is infrastructure**, and infrastructure decisions are made by measuring failure modes, not by defaulting to maximum coverage.

INTERVIEW STRATEGY

You are being asked to show cost-aware test-matrix decisions. Lead with the tiered strategy: Chromium PR, all three on main, full on release. Make the measure-failure-data point the senior signal because it converts the decision from religious to empirical. The trap is all-three-on-every-PR. A common follow-up is ‘how do you justify dropping a browser?’, answer track the WebKit-only failures in the last 6 months; if rare, PR-level coverage is wasted spend.

How to phrase it

> *Running all three browsers, Chromium, Firefox, WebKit, on every PR triples CI time and infrastructure cost, and it's only the right choice when cross-browser bugs are a real risk that justifies the spend. The tiered approach I use is: Chromium on every PR, because it's fast and catches roughly ninety percent of issues; Firefox and WebKit on main branch merges, because they catch the cross-browser-specific bugs without slowing PR feedback; all three on release branches, because release is the final gate before customers see the build and the marginal cost is worth it. That tiering keeps PR feedback fast while preserving cross-browser coverage at the points where it matters most. The decision data, and the framing I would lead with because it converts this from a religious argument to an empirical one, is track which browser-specific tests have actually failed in the last six months. If WebKit-only failures are one per quarter, running WebKit on every PR is wasted spend, the cost-per-bug-caught is too high. If they're weekly, the PR-level cost is justified because you're catching something real. The principle I would close on is cross-browser coverage is infrastructure, and infrastructure decisions are made by measuring failure modes, not by defaulting to maximum coverage. The default isn't always-on; the default is measure-then-decide. That's also my answer to the follow-up about justifying dropping a browser. I pull the data: how many PRs in the last six months were blocked or caught by WebKit-only failures? If it's negligible relative to the cost of running WebKit on every PR, I move WebKit to main-only with a clear policy document. The data makes the conversation about measurement rather than tribal preference. The thing to avoid is all-three-on-every-PR by default, wastes money and time on coverage you're not actually using.*

Key points to hit

(1) Tiered: Chromium on PR; +Firefox/WebKit on main; all on release. **(2)** Chromium catches ~90% of issues at lowest cost, PR-friendly default. **(3)** Measure browser-specific failure data over 6 months, empirical decision. **(4)** Cross-browser coverage is infrastructure: decide by data, not defaults. **(5)** The next question: justify dropping via failure-rate data; document the policy. **(6)** The common miss: all three on every PR, wastes time/money on unused coverage.

CODE

```
# .github/workflows/test.yml: tiered browser matrix
jobs:
test:
strategy:
matrix:
# PR builds: Chromium only, fast feedback
# Push to main: + Firefox + WebKit
# Release: all three explicitly
browser: ${{ github.event_name == 'pull_request'
&& fromJSON('["chromium"]')
|| fromJSON('["chromium","firefox","webkit"]') }}
steps:
- run: npx playwright test --project=${{ matrix.browser }}

# Decision data: measure before defaulting
# grep "webkit" reports/*.json | grep "failed" -> frequency
# if < 1/quarter: move WebKit to main-only, document the policy
# if weekly or more: PR-level WebKit pays its way
```

Q 150

LEVEL · Senior

What is a merge queue, and how does it combine with retry-on-flake to ship safely?

CONCEPT

A **merge queue** serialises PR merges into the main branch, PRs are queued and tested against the actual tip of main rather than their stale base, so two PRs that each pass individually but break together are caught before the merge. GitHub's native merge queue and tools like Bors or Mergify implement it. The pattern combines well with **retry-on-flake**: Playwright's **retries: 2** in CI means a flaky test gets two more shots before failing, and the merge queue absorbs that retry time because PRs are queued anyway. Together they ship reliable code without blocking on flakes. The discipline that separates a good implementation: track which tests needed retries to pass as a separate dashboard, those are tomorrow's flake-fix backlog. Retries are a tactical fix that masks an underlying problem; the dashboard prevents flaky tests from accumulating indefinitely. The principle: retries unblock the build; the dashboard prevents the flake debt from growing.

INTERVIEW STRATEGY

What scores here is showing merge queue + retry as a system, not isolated features. Lead with merge queue serialising merges against main's tip, then retry-on-flake absorbed by queueing. The 'retries unblock build, dashboard prevents debt' framing demonstrates calibration because it shows you know retries are tactical. The trap is uncapped retries with no tracking. The follow-up is often ‘what about flake debt?’, answer the dashboard of tests that needed retries is the prioritised fix backlog.

How to phrase it

> *A merge queue serialises PR merges into the main branch. PRs are queued and tested against the actual tip of main rather than their stale base, so two PRs that each pass individually but break together are caught before the merge. GitHub's native merge queue and tools like Bors or Mergify implement this. The failure mode it prevents, and worth naming because it's invisible without a queue, is the classic merge-race bug: PR A modifies a shared helper, PR B writes tests using the shared helper, both pass against their own bases, the merge of B against A's new helper breaks. Without a queue you find that after the merge; with a queue it fails before. The pattern combines well with retry-on-flake. Playwright's retries two in CI means a flaky test gets two more shots before failing, and the merge queue absorbs that retry time because PRs are queued anyway, the marginal minute of retries doesn't slow developer feedback because the developer isn't watching the build, the queue is. Together they ship reliable code without blocking on flakes. The discipline that separates a good implementation from a sloppy one, and my answer to the follow-up about flake debt, is track which tests needed retries to pass as a separate dashboard. Those are tomorrow's flake-fix backlog. Retries are a tactical fix that masks an underlying problem; without the dashboard flaky tests accumulate indefinitely until the suite is so noisy nobody trusts it. With the dashboard I can say this week we have ten tests that needed retries; the platform team owns five, the auth team owns three, the rest are mine, and fixes get prioritised by failure frequency. The principle I close on is retries unblock the build; the dashboard prevents the flake debt from growing. The trap is uncapped retries with no tracking; the suite degrades silently until it's useless.*

Key points to hit

(1) Merge queue serialises merges; tests run against actual main tip. **(2)** Catches merge-race bugs: A+B pass individually, fail together. **(3)** Retry-on-flake (retries: 2) absorbed by queueing, no dev wait penalty. **(4)** Track retry-needed tests in a dashboard: the flake-fix backlog. **(5)** Asked next: dashboard turns invisible flake debt into prioritised work. **(6)** Avoid: uncapped retries with no tracking, silent degradation.

CODE

```
# .github/workflows/test.yml: retry-on-flake + merge queue
on:
pull_request:
merge_group: # GitHub native merge queue

jobs:
test:
runs-on: ubuntu-latest
steps:
- run: npx playwright test --retries=2 --reporter=json,html
# JSON output drives the retry-needed dashboard

# Dashboard query (parse JSON reports)
# for each test where result.retries > 0:
# count occurrences over the last 30 days
# sort descending -> prioritised flake-fix backlog
# owner = team-from-CODEOWNERS for that test file
```

Q 151

LEVEL · Senior

In a monorepo, how do you run only the tests affected by a code change?

CONCEPT

Selective testing in a monorepo cuts CI time by an order of magnitude on small PRs while keeping full coverage on broad changes. Two complementary strategies. **Path-based filtering**: a CODEOWNERS-style mapping from source paths to test paths, changes to apps/checkout/** trigger tests/checkout/**, changes to libs/auth/** trigger every test importing the auth lib. Compute affected test files via git diff and run only those. **Build-graph-based filtering**: tools like Nx, Turborepo, or Bazel know the import graph and compute the true set of affected projects (**nx affected:test**). More precise than path-based because it follows actual imports, not directory conventions. The discipline: always run the full suite on main-branch merges as a backstop. Path-based filtering misses indirect dependencies; build-graph-based filtering can be fooled by dynamic imports. The full-suite-on-main backstop catches what selective testing misses.

INTERVIEW STRATEGY

This question is there to see monorepo selective testing with the backstop discipline. Lead with path-based and build-graph as the two strategies (Nx/Turborepo/Bazel for the latter). Make the full-suite-on-main backstop the senior signal because selective testing without a backstop ships indirect-dependency bugs. The trap is selective only, no backstop. Expect the follow-up ‘which is more reliable, path or graph?’, answer graph follows real imports; path-based is rule-based and can miss indirect deps.

How to phrase it

> *Selective testing in a monorepo cuts CI time by an order of magnitude on small PRs while keeping full coverage on broad changes, it's one of the highest-leverage CI optimisations for a large codebase. Two complementary strategies. Path-based filtering: a CODEOWNERS-style mapping from source paths to test paths, changes to apps slash checkout double-star trigger tests slash checkout double-star, changes to libs slash auth double-star trigger every test importing the auth lib. I compute affected test files via git diff against the merge-base and run only those. It's simple, fast, and works for monorepos with a clear directory convention. Build-graph-based filtering: tools like Nx, Turborepo, or Bazel know the import graph and compute the true set of affected projects. nx affected colon test runs only the tests for projects depending on changed code, transitively. More precise than path-based because it follows actual imports, not directory conventions. The discipline that matters, and the senior signal I would emphasise because skipping it ships indirect-dependency bugs, is always run the full suite on main-branch merges as a backstop. Path-based filtering misses indirect dependencies, a util buried three directories away that several services import. Build-graph filtering can be fooled by dynamic imports, like a string-based require or a lazy-loaded module the graph doesn't see. The full-suite-on-main backstop catches what selective testing misses, so the team gets the speed benefit on PRs without losing safety overall. On the natural follow-up about which is more reliable. Build-graph is more precise because it follows real imports; path-based is rule-based and depends on the directory convention matching the actual dependency structure. But path-based is easier to implement without adopting a new build tool, so for teams not on Nx or Bazel, it's the pragmatic choice with the full-suite backstop. The classic error is selective testing without a backstop, an indirect dep breaks and ships.*

Key points to hit

(1) Path-based: directory-convention mapping; git diff finds affected tests. **(2)** Build-graph (Nx/Turborepo/Bazel): follows real imports, more precise. **(3)** ALWAYS run full suite on main-branch merge as a backstop. **(4)** Path-based misses indirect deps; graph-based can miss dynamic imports. **(5)** Follow-up: graph more reliable; path easier without new build tool. **(6)** The easy mistake: selective testing without the backstop, indirect-dep bugs ship.

CODE

```
# Path-based filtering on PRs (no new build tool required)
- name: Compute affected tests
id: affected
run: |
BASE=$(git merge-base origin/main HEAD)
CHANGED=$(git diff --name-only "$BASE" HEAD)
# Map source paths to test paths via convention
echo "$CHANGED" | sed -E 's#^apps/([^/]+)/.*#tests/\1/#;
s#^libs/([^/]+)/.*#tests/**/*.spec.ts#' \\
| sort -u > affected.txt

- run: npx playwright test $(cat affected.txt)

# Build-graph alternative (more precise)
- run: npx nx affected --target=test --base=origin/main

# BACKSTOP: full suite on main-branch merges, catches what selective misses
# trigger: on: push: branches: [main]
```

Q 152

LEVEL · Senior

How do you tier CI feedback (smoke vs regression vs full e2e) to balance speed with coverage?

CONCEPT

Three-tier strategy with explicit time budgets. **Tier 1, Smoke (under 2 minutes)**: a handful of critical-path tests tagged **@smoke**, runs on every commit including draft PRs, verifies the build is alive (auth works, the homepage loads, checkout reaches the payment step). Goal: quickest possible feedback that the PR isn't catastrophic. **Tier 2, Regression (under 15 minutes)**: the full suite tagged @critical plus @regression, runs on PR-ready and on main merges. Goal: catch every documented regression before merge. Tier 3, Full e2e (nightly or pre-release): everything including @slow, cross-browser, mobile viewports, the genuinely-long tests like full user onboarding flows. Goal: complete coverage on a schedule that doesn't block merges. Each tier enforces its own time budget and triggers at a different point in the workflow. The principle: match feedback speed to the decision being made. Draft PR wants smoke; merge wants regression; release wants everything.

INTERVIEW STRATEGY

They want to see CI tiering with explicit budgets. Walk the three tiers, smoke under 2 min, regression under 15 min, full overnight, with the trigger points and goals. The 'match feedback speed to decision' framing is what separates junior from senior thinking. The mistake is one tier (full suite every PR) that's too slow or too narrow. They usually follow with ‘what goes in smoke?’, answer the critical-path tests: login, homepage, checkout-reach-payment. About 10-20 tests, never more.

How to phrase it

> *Three-tier strategy with explicit time budgets, because the right feedback at the right time is what makes CI useful rather than annoying. Tier one, smoke, under two minutes. A handful of critical-path tests tagged at-smoke that run on every commit, including draft PRs. The goal is the quickest possible feedback that the PR isn't catastrophic, auth works, the homepage loads, checkout reaches the payment step. If they push on this in the follow-up, my answer is about what goes in smoke: the critical-path tests, around ten to twenty of them, never more. If smoke takes more than two minutes, it's not smoke any more. Tier two, regression, under fifteen minutes. The full suite tagged at-critical plus at-regression, runs on PR-ready and on main merges. The goal is to catch every documented regression before merge, the cases users have hit, the edge cases tests have been written for, the boundary conditions. This is the gate the PR has to pass to merge. Tier three, full e2e, nightly or pre-release. Everything including at-slow, cross-browser, mobile viewports, the genuinely-long tests like full user onboarding flows that take minutes each. The goal is complete coverage on a schedule that doesn't block merges. Run overnight, surface failures in the morning, address before the next release. The principle I would lead with, because it captures the tiering logic, is match feedback speed to the decision being made. Draft PR wants smoke because the dev is exploring. Merge wants regression because the change is locking in. Release wants everything because customers see it next. Each tier enforces its own time budget and triggers at a different point. The pitfall is one tier, the full suite every PR, which is either too slow to get feedback while iterating or too narrow to catch real bugs. Tiering gives both.*

Key points to hit

(1) Three tiers with budgets: smoke <2min, regression <15min, full nightly. **(2)** Smoke (10-20 tests): critical paths, auth/home/checkout reach payment. **(3)** Regression: @critical + @regression on PR-ready and main merges. **(4)** Full e2e: @slow + cross-browser + mobile, nightly schedule. **(5)** Expect next: smoke is 10-20 tests max, over 2 min and it isn't smoke any more. **(6)** Trap: one tier (full suite on every PR), too slow for iteration, too narrow for safety.

CODE

```
# Tier 1: smoke (runs on every commit, every push, every draft PR)
name: smoke
on: [push, pull_request]
jobs:
smoke:
timeout-minutes: 3 # hard ceiling, not 2 (small buffer)
steps:
- run: npx playwright test --grep @smoke --reporter=line

# Tier 2: regression (PR ready or main merge)
name: regression
on:
pull_request:
types: [ready_for_review, synchronize]
push:
branches: [main]
jobs:
regression:
timeout-minutes: 18 # 15 + small buffer
steps:
- run: npx playwright test --grep '@critical|@regression'

# Tier 3: full e2e (nightly, cross-browser, mobile)
name: nightly
on:
schedule:
- cron: '0 2 * * *' # 02:00 UTC every day
```

Q 153

LEVEL · Mid to Senior

How do you manage trace, video, and screenshot artifacts in CI without exploding storage costs?

CONCEPT

Artifacts are essential for debugging CI failures but expensive at scale. Do the arithmetic on your own suite rather than quoting a figure: take your measured trace size, multiply by tests per run and runs per day, and the number lands in tens of gigabytes a day quickly. Three discipline points. **Generate conditionally**: trace: 'on-first-retry', video: 'retain-on-failure', screenshot: 'only-on-failure', zero cost on passing tests, full diagnostics on failures. **Retain selectively**: keep PR artifacts for 7 days (long enough to debug), main-branch artifacts for 30 days (long enough to investigate post-merge issues), release-tag artifacts indefinitely via a separate promotion to long-term storage. **Upload conditionally**: don't upload trace.zip when the test passed even if it was recorded; the GitHub Actions if-failure pattern saves uploads on green builds. The principle: diagnostics matter on failures; storage discipline matters on the other 99% of builds.

INTERVIEW STRATEGY

Your job here is to show cost-aware artifact management. Walk the three disciplines: conditional generation, selective retention, conditional upload. Quantify the cost from your own measured trace size times tests times runs per day, which makes it real and is defensible. The 'diagnostics on failure, storage on the 99% of green' framing is the senior signal. Where this goes wrong is trace: 'on' + indefinite retention. The next question is normally ‘how do you debug an old failure?’, answer

7-day window for PRs is enough; promote release artifacts to long-term storage.

How to phrase it

> *Artifacts are essential for debugging CI failures but expensive at scale, and the trade-off is what most teams get wrong either way, too little and CI failures are unanalysable; too much and storage costs grow without bound. I'd anchor the conversation with arithmetic from our own suite rather than a number off a blog: take the measured trace size, multiply by tests per run and runs per day, and it reaches tens of gigabytes a day quickly, which is a real line on the cloud bill. Three discipline points. First, generate conditionally. Trace on-first-retry, video retain-on-failure, screenshot only-on-failure. Zero cost on passing tests, full diagnostics on failures. Most tests pass; the artifacts most tests generate are unused. Second, retain selectively. Keep PR artifacts for seven days, which is long enough to debug a PR-time failure. Main-branch artifacts for thirty days, long enough to investigate post-merge issues. Release-tag artifacts indefinitely via a separate promotion to long-term storage, S3 with lifecycle rules for cheap archival. Different retention for different signal value. Third, upload conditionally. Don't upload trace dot zip when the test passed even if it was recorded. The GitHub Actions if-failure pattern handles this, the artifact-upload step runs only when earlier steps failed. Saves uploads on every green build, which is the vast majority. The principle I close on, because it captures the calibration, is diagnostics matter on failures; storage discipline matters on the other ninety-nine percent of builds. Generosity with artifacts on failure, parsimony on green. And to the follow-up about debugging an old failure: the seven-day window for PRs is enough, if someone needs to debug a PR-time failure after a week, the right answer is reproducing locally, not paying for indefinite cloud retention. Release artifacts get promoted to long-term because those are the ones a customer incident might trace back to. The trap is trace on plus indefinite retention, the bill grows linearly with PR volume forever.*

Key points to hit

(1) Generate conditionally: trace on-first-retry, video retain-on-failure. **(2)** Retain selectively: PR 7d, main 30d, release indefinite (promoted to long-term). **(3)** Upload conditionally: if-failure pattern, no upload on green builds. **(4)** Quantify from YOUR measured trace size x tests x runs per day, not a quoted figure. **(5)** They usually ask next: 7d for PRs; promote release artifacts to long-term archive. **(6)** Where this goes wrong: trace: 'on' + indefinite retention, cost grows with PR volume forever.

CODE

```
// playwright.config.ts: conditional generation
export default defineConfig({
use: {
trace: 'on-first-retry',
video: 'retain-on-failure',
screenshot: 'only-on-failure',
},
});

# .github/workflows/test.yml: conditional upload + tiered retention
- name: Upload trace/video/screenshots (failures only)
if: failure()
uses: actions/upload-artifact@v4
with:
name: playwright-failure-${{ github.run_id }}
path: |
test-results/**
playwright-report/**
retention-days: ${{ github.ref == 'refs/heads/main' && 30 || 7 }}

# Release-tag promotion to long-term storage
- if: startsWith(github.ref, 'refs/tags/v')
run: aws s3 cp playwright-report/ s3://artifacts-archive/${{ github.ref_name }}/
--recursive
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Why set fail-fast: false on a GitHub Actions sharding matrix?

A) So all shards finish and you get the full failure picture, not just the first
B) To make the job run faster
C) To disable retries
D) To reduce CI cost

2. What is the key Jenkins-plus-Docker gotcha for Playwright?

A) Setting node-version
B) Caching the browsers
C) Using JUnit output
D) --ipc=host, without which Chromium crashes silently on a too-small /dev/shm

3. What is the purpose of the requireEnv helper for secrets?

A) It encrypts the secrets
B) It fails fast with a clear message if a required variable is missing
C) It commits secrets to the repo
D) It logs all variables

4. Which two thresholds make sensible Playwright quality gates?

A) Pass rate > 50%, flaky rate > 20%
B) Any failures block deployment
C) Pass rate > 95% and flaky rate < 5%
D) Pass rate > 80% only

5. Why prefer the v1.42+ tag option over inline @tags in the test title?

A) It runs faster
B) It supports more tags
C) It is the only way to tag tests
D) It is queryable programmatically and does not pollute the test name in reports

Section 10 · Answer key

#	Answer	Why and where to revisit
1	A) So all shards finish and you get the full failure picture, not just the first	Without fail-fast: false, one shard failing cancels the rest and you only see that shard's failures. *Revisit Section 10.*
2	D) --ipc=host, without which Chromium crashes silently on a too-small /dev/shm	Chromium needs shared memory; Docker's default /dev/shm is too small, so --ipc=host prevents silent crashes. *Revisit Section 10.*
3	B) It fails fast with a clear message if a required variable is missing	requireEnv turns a cryptic mid-test undefined error into a clear configuration error at startup. *Revisit Section 10.*
4	C) Pass rate > 95% and flaky rate < 5%	Pass rate above 95% and flaky rate below 5% catch real degradation without tripping on minor noise. *Revisit Section 10.*
5	D) It is queryable programmatically and does not pollute the test name in reports	The tag option keeps report names clean and lets tooling query tags, unlike tags embedded in titles. *Revisit Section 10.*

SECTION 11

Debugging and Tracing

Debugging is where Playwright quietly wins the most engineer-hours, and the Trace Viewer is the centrepiece, it lets you diagnose a CI-only failure without reproducing it locally. Interviewers want to see you know which tool fits which moment. These eleven questions cover the Inspector and page.pause, the Trace Viewer, capturing console and network errors, test.step for readable traces, VS Code debugging, page.evaluate, and UI Mode.

In this section

- The Inspector, page.pause(), and codegen for interactive local debugging.
- The Trace Viewer: diagnose CI failures without reproducing locally.
- Capturing console errors and network failures; asserting no console errors.
- test.step() for traces that read as logical steps, not 20 raw actions.
- VS Code extension and the PWDEBUG / DEBUG=pw:api flags.
- page.evaluate() as a setup-and-debug escape hatch, never for assertions.
- UI Mode for active development; knowing which debug tool fits each moment.

Q 154 LEVEL · Junior to Mid

How do you use the Playwright Inspector for local debugging?

CONCEPT

The Playwright Inspector is a GUI debugger that launches alongside the browser. It shows the current action, allows step-through execution, and includes a live selector picker for finding locators interactively. Launch it with **--debug** (pauses before the first action), or pause mid-test at an exact line with **page.pause()**. Headed mode with **--slow-mo** lets you watch the run in real time. codegen opens a browser plus the Inspector and generates locators as you click through a flow, a fast starting point for a new page object, after which you upgrade the generated CSS selectors to getByRole equivalents. page.pause is the most underused of these: drop it at the failing line, run with --debug, and inspect the frozen page state.

INTERVIEW STRATEGY

Show a real local workflow, not just tool names. Lead with page.pause() as the underused gem: drop it at the failing line, run --debug, inspect the frozen DOM and try locators live. Mention codegen as a starting point you then upgrade to getByRole. The thing to avoid is implying you debug by sprinkling console.log. A common follow-up is 'do you keep codegen's output as-is?', answer no, you upgrade the generated CSS selectors to semantic getByRole locators.

How to phrase it

> *The Inspector is a GUI debugger that launches with the browser, shows the current action, lets me step through, and has a live selector picker. I launch it with --debug to pause before the first action, or I use headed mode with slow-mo to watch a run in real time. The tool I would single out, because it is the most underused, is page.pause(). I drop it at the exact line where a test fails, run with --debug, and the Inspector opens with the browser frozen at that point, so I can inspect the DOM, try candidate locators in the Inspector's selector input, and see precisely what state the page is in, rather than guessing. That is far more effective than sprinkling console.log, which is the trap. codegen is my starting point for a new page object: I click through the flow and Playwright generates the locators for me. But that sets up the follow-up about whether I keep codegen's output as-is, and the answer is no, codegen tends to produce CSS selectors, so I upgrade them to getByRole equivalents that survive UI change, treating the generated code as a draft, not the final locator strategy. So my local loop is codegen to scaffold, page.pause to inspect, and the Inspector's picker to refine locators.*

Key points to hit

(1) Inspector: step-through GUI with a live selector picker. **(2)** --debug pauses before the first action; --slow-mo to watch in real time. **(3)** page.pause() at the failing line freezes the page for inspection. **(4)** codegen scaffolds locators by clicking through a flow. **(5)** The next question: upgrade codegen's CSS selectors to semantic getByRole. **(6)** What loses the point: implying you debug by sprinkling console.log.

CODE

```
# Pause before the first action
npx playwright test login.spec.ts --debug

# Watch a run in real time
npx playwright test --headed --slow-mo=500

# Pause at an exact point
test('debug this', async ({ page }) => {
await page.goto('/login');
await page.pause(); // Inspector opens, browser frozen here
await page.getByLabel('Email').fill('admin@test.com');
});

# Generate locators interactively, then upgrade to getByRole
npx playwright codegen https://example.com
```

Q 155 LEVEL · Mid

How does the Playwright Trace Viewer work, and how do you use it for CI debugging?

CONCEPT

The Trace Viewer records a complete timeline of every action: screenshots before and after, DOM snapshots, network requests and responses, console logs, and the source location, the primary tool for diagnosing CI failures without reproducing them locally. Configure trace: 'on-first-retry' so a full trace is captured exactly when a test fails and retries, without the storage cost of tracing every passing run. View locally with **show-trace**, or open the trace zip at **trace.playwright.dev** with no install. You can also attach custom data to a trace with **testInfo.attach**, an API response, for instance, so it sits alongside the timeline for correlation. The payoff is dramatic: a CI-only failure goes from roughly an hour of reproduce-and-log to about five minutes of scrubbing a trace.

INTERVIEW STRATEGY

This is a flagship Playwright feature, so show you actually use it. Quantify the before-and-after, ~60 minutes of reproduce-add-logging-push-wait versus ~5 minutes of download-and-scrub, because the numbers sell it. Stress on-first-retry, not on, to show cost awareness. The risk is setting trace: on and wasting storage on every passing run. The follow-up is often ‘why on-first-retry over on?’, answer you get the full trace exactly when it failed, at a fraction of the storage.

How to phrase it

> *The Trace Viewer records a complete timeline of a test, before and after screenshots, DOM snapshots, network, console logs, and the source location, and it is the single biggest productivity gain Playwright brought our team, so I would back that with the numbers. Before, a CI-only failure meant reproduce locally, maybe thirty minutes, add logging, push, wait ten minutes for CI, then analyse logs, about an hour per failure, and sometimes I could not reproduce it at all. With the Trace Viewer I download the artifact in a minute, open it at trace.playwright.dev with no install, scrub to the failing action, and see the DOM snapshot and the network response right there, about five minutes total. I have diagnosed race conditions, environment-specific rendering bugs, and session-expiry issues entirely from traces without ever reproducing locally. On configuration, I use trace on-first-retry rather than on, and that is my answer to the follow-up about why: tracing every passing run wastes a lot of storage for no benefit, whereas on-first-retry gives me a full trace exactly when a test failed and is retrying, which is the only time I need it. I also attach custom data like an API response with testInfo.attach so it sits next to the timeline for correlation. The Trace Viewer is the reason CI failures stopped being dreaded on our team.*

Key points to hit

(1) Records screenshots, DOM snapshots, network, console, source location. **(2)** Diagnose CI failures without reproducing locally. **(3)** trace: 'on-first-retry', full trace when it fails, minimal storage. **(4)** View via show-trace or trace.playwright.dev (no install). **(5)** Asked next: on-first-retry over on, same diagnostics, far less storage. **(6)** Quantify: ~60 min reproduce-and-log vs ~5 min scrubbing a trace.

CODE

```
// playwright.config.ts
export default defineConfig({
use: { trace: 'on-first-retry' }, // not 'on', storage cost
});

// View locally, or upload the zip to trace.playwright.dev
// npx playwright show-trace trace.zip

// Attach data to the trace for correlation
test('order flow', async ({ page }, testInfo) => {
const res = await page.request.get('/api/cart');
await testInfo.attach('cart-api-response', {
body: await res.text(), contentType: 'application/json',
});
});
```

Q 156 LEVEL · Mid

How do you capture and analyse console logs and network failures in tests?

CONCEPT

Playwright event listeners capture browser output in real time: **page.on('console')** for console messages, **page.on('requestfailed')** for failed network requests (with request.failure().errorText),

and **page.on('pageerror')** for uncaught exceptions. A high-value test is asserting no console errors occurred during a flow: JavaScript errors are often invisible to functional tests, the page looks correct but an uncaught exception is firing in the background. Collect error-type console messages into an array and assert the array is empty, with the messages in the failure text. A complementary pattern puts the requestfailed listener in a fixture that attaches every network failure to the report, so a failing test immediately shows which request failed without any reproduction.

INTERVIEW STRATEGY

They want proactive diagnostics, not just reactive debugging. Highlight the assert-no-console-errors test as something you add to every smoke suite, and explain why it matters: silent JS errors that functional tests miss entirely. Mention the fixture that attaches network failures to the report. The failure mode is only checking visible UI and ignoring the console. Expect the follow-up 'how do you surface the failing request when a test fails?', answer the requestfailed listener in a fixture that attaches failures to the report.

How to phrase it

> *Playwright lets me capture browser output in real time with event listeners, console for console messages, requestfailed for failed network requests with the failure reason, and pageerror for uncaught exceptions. The pattern I would lead with, because it is proactive rather than reactive, is a test that asserts no console errors occurred during a flow, which I add to every smoke suite. The reason it matters is that JavaScript errors in the console are often completely invisible to functional tests: the page renders correctly, all the assertions pass, but there is an uncaught exception firing in the background that will bite users in edge cases. I collect the error-type console messages into an array, then assert the array is empty with the actual messages in the failure text, so when it fails I see exactly what errored. That catches silent failures a normal functional test sails straight past, which is the trap of only checking the visible UI. For the follow-up about surfacing the failing request when a test fails, my answer is a fixture that registers the requestfailed listener and attaches every network failure to the test report, so the moment a test fails I can see whether a request failed and which one, with no reproduction needed. Together those turn the browser's own error channels into part of the test signal.*

Key points to hit

(1) Listeners: console (messages), requestfailed (network), pageerror (exceptions). **(2)** Assert no console errors: catches silent JS failures functional tests miss. **(3)** Collect error messages into an array; assert empty with messages in the failure. **(4)** Fixture attaches network failures to the report for instant diagnosis. **(5)** Follow-up: surface the failing request via a requestfailed fixture. **(6)** The common miss: only checking the visible UI and ignoring the console.

CODE

```
// Capture failures in real time
page.on('requestfailed', req =>
console.error(`Network fail: ${req.method()} ${req.url()},
${req.failure()?.errorText}`));
page.on('pageerror', err => console.error('Uncaught:', err.message));

// Assert no console errors during a flow
test('no console errors on dashboard', async ({ page }) => {
const errors: string[] = [];
page.on('console', m => { if (m.type() === 'error') errors.push(m.text()); });
await page.goto('/dashboard');
expect(errors, `Console errors: ${errors.join(', ')}`).toHaveLength(0);
});
```

Q 157 LEVEL · Junior to Mid

What is test.step() and how does it improve tracing and reporting?

CONCEPT

test.step() groups a sequence of actions under a **named step**. Steps appear as collapsible groups in the HTML report and Trace Viewer, so you can see which logical step failed without reading individual action lines, and they show in the test output on failure. They are purely **organisational**: they do not affect execution. Without steps, a checkout test shows 20 raw actions (click, fill, click, fill); with steps it shows three named groups, Add items to cart, Complete checkout, Verify confirmation, each expandable. You can also wrap a multi-action page-object method in a step so its name appears in the trace and tells the story of what the test was doing at the point of failure. This is the difference between a trace that takes five minutes to read and one that takes thirty seconds.

INTERVIEW STRATEGY

They are checking whether you make tests diagnosable for the whole team. Sell the readability jump, 20 raw actions versus three named steps, and add the senior move of wrapping page-object methods in steps so the trace narrates itself. The mistake is dismissing test.step as cosmetic. They usually follow with ‘does it change how the test runs?’, answer no, it is purely organisational, which is exactly why there is no reason not to use it in complex flows.

How to phrase it

> *test.step groups a sequence of actions under a named step, and those steps show up as collapsible groups in both the HTML report and the Trace Viewer. It is the single biggest improvement to trace readability in complex tests, so let me make the contrast concrete. Without steps, a checkout test shows twenty individual actions in the trace, click, fill, click, fill, and when it fails in CI I have to read through all of them to work out where it broke. With steps, the same test shows three named groups: Add items to cart, Complete checkout, Verify confirmation. So when a failure happens I immediately see which step failed and only expand that one. The senior move I would add is wrapping multi-action page-object methods in a step too, so the step name appears in the trace and the trace literally narrates what the test was doing at the point of failure. That answers the follow-up about whether it changes execution: no, test.step is purely organisational, it does not affect how the test runs at all, which is exactly why there is no reason not to use it in any complex flow, there is no runtime cost and a large readability gain. The practical effect is the difference between a trace that takes five minutes to read and one that takes thirty seconds, and at team scale that compounds across every failure investigation.*

Key points to hit

(1) Groups actions under named, collapsible steps in report and trace. **(2)** 20 raw actions become a few readable logical steps. **(3)** See which step failed without reading every action line. **(4)** Wrap multi-action page-object methods in steps so the trace narrates itself. **(5)** Expect next: it does not change execution, purely organisational. **(6)** Avoid: dismissing test.step as cosmetic.

CODE

```
test('checkout flow', async ({ cartPage, checkoutPage }) => {
await test.step('Add items to cart', async () => {
await cartPage.navigate();
await cartPage.addItem('MacBook Pro');
});
await test.step('Complete checkout', async () => {
await checkoutPage.fillShipping({ name: 'Alice', address: '123 Main St' });
await checkoutPage.placeOrder();
});
await test.step('Verify confirmation', async () => {
await expect(checkoutPage.confirmationHeading).toBeVisible();
});
});

// In a page object: the step name appears in the trace
async completeCheckout(d: CheckoutDetails): Promise<void> {
await test.step(`Complete checkout for ${d.name}`, async () => {
await this.fillShipping(d); await this.placeOrder();
});
}
```

Q 158 LEVEL · Junior to Mid

How do you debug Playwright tests in VS Code?

CONCEPT

The official **Playwright VS Code extension** provides a test explorer, inline test running, and breakpoint debugging directly in the editor. Set a breakpoint in the test file, run via the test explorer or a launch configuration, and VS Code pauses at the breakpoint **with the browser open**, so you inspect variables and step through code while seeing the browser state at the same time. The environment flags are worth knowing: PWDEBUG=1 opens the Inspector, PWDEBUG=console gives verbose logging, and **DEBUG=pw:api** logs every low-level API call with timing, which is how you diagnose unexpected retries or performance issues. This is the fastest workflow for local development because code and browser state are visible together in one tool.

INTERVIEW STRATEGY

Keep this concrete and workflow-focused: the interviewer wants to know you debug efficiently locally. Describe the breakpoint-with-browser-open loop, then add the flags, especially DEBUG=pw:api for diagnosing unexpected retries, which is the depth signal. The risk is describing only console.log debugging. The next question is normally 'how do you diagnose why a test is retrying or slow?', answer DEBUG=pw:api, which logs every API call with timing.

How to phrase it

> *The official Playwright VS Code extension is my fastest local debugging workflow, and the core loop is simple: I set a breakpoint in the test file, click debug in the test explorer, and VS Code pauses at the breakpoint with the browser open. The value is that I can inspect variables, step through the code, and see the browser state at the same time, all in one tool, instead of switching between a terminal and a browser window. Beyond breakpoints, the environment flags are worth knowing and they are where I would show some depth. PWDEBUG=1 opens the Inspector, PWDEBUG=console gives verbose logging, and the one I reach for in tricky cases is DEBUG=pw:api, which logs every low-level Playwright API call with timing. That is my answer to the follow-up about diagnosing why a test is retrying or running slowly: the pw:api log shows me exactly what Playwright is doing internally, including the retries on actionability checks and how long each is taking, so I can see whether the page is genuinely slow or a locator is waiting on something. The trap I would avoid is implying I debug only with console.log; the breakpoint workflow plus pw:api logging is far faster and tells me what is actually happening under the hood rather than what I guessed to print.*

Key points to hit

(1) VS Code extension: test explorer, inline running, breakpoint debugging. **(2)** Pauses at the breakpoint with the browser open, code + state together. **(3)** PWDEBUG=1 opens the Inspector; PWDEBUG=console for verbose logging. **(4)** DEBUG=pw:api logs every API call with timing. **(5)** They usually ask next: diagnose retries/slowness with DEBUG=pw:api. **(6)** The easy mistake: describing only console.log debugging.

CODE

```
// .vscode/launch.json
{
"configurations": [{
"name": "Debug Playwright Test",
"type": "node",
"request": "launch",
"program": "${workspaceFolder}/node_modules/.bin/playwright",
"args": ["test", "--debug", "${file}"],
"env": { "PWDEBUG": "1" },
"console": "integratedTerminal"
}]
}

# PWDEBUG=1: opens the Inspector
# PWDEBUG=console: verbose logging
# DEBUG=pw:api, logs every API call with timing
```

Q 159 LEVEL · Mid

How do you use page.evaluate() for advanced debugging?

CONCEPT

page.evaluate() executes JavaScript **in the browser context** and returns the result to Node, the bridge between the test and the browser's JavaScript runtime. It is an **escape hatch** for non-standard scenarios: inspecting application state (a global app-state object, localStorage, cookies, feature flags), setting up state that the UI cannot (injecting a feature flag, seeding localStorage, dispatching a custom event), or reading framework internals such as a React fibre tree to inspect component props. The firm rule: use it for setup and debugging, not for assertions. Assertions should always use the Playwright expect API with its built-in retrying, an evaluate-based check is a one-shot snapshot and reintroduces flakiness.

INTERVIEW STRATEGY

They are looking to see power used with discipline. Position page.evaluate as an escape hatch for state inspection and setup the UI cannot do, with a concrete story (reading React component props). Then state the firm rule clearly: setup and debugging, never assertions, because evaluate is a one-shot snapshot. That boundary is the senior signal. The risk is asserting on an evaluate result and reintroducing flakiness. A common follow-up is 'why not assert with evaluate?', answer it does not retry, so use the retrying expect API instead.

How to phrase it

> *page.evaluate runs JavaScript in the browser context and returns the result to Node, so it is the bridge between my test and the browser's runtime, and I treat it as an escape hatch for non-standard scenarios. I use it to inspect application state, a global app-state object, localStorage, cookies, feature flags, and to set up state that the UI simply cannot, like injecting a feature flag, seeding a localStorage value, or dispatching a custom event. The story I would give is debugging a React app where the UI looked correct but the component state was wrong: page.evaluate let me reach into the React fibre tree and inspect the component props directly, which is how I found the bug. But the part that matters most, and the discipline an interviewer is listening for, is the rule: I use evaluate for setup and debugging, never for assertions. That is also my answer to the follow-up about why not assert with it, evaluate is a one-shot snapshot that does not retry, so asserting on its result reintroduces exactly the flakiness Playwright's web-first assertions are designed to remove. Assertions always go through the expect API with its built-in retrying. So page.evaluate is a powerful tool I reach for deliberately and narrowly, to inspect and to set up, and I keep it well away from the assertion path.*

Key points to hit

(1) Runs JS in the browser context and returns the result to Node. **(2)** Escape hatch: inspect app state, or set up state the UI cannot. **(3)** Real use: read React fibre/component props to find a state bug. **(4)** Firm rule: setup and debugging only, never assertions. **(5)** The next question: why not assert, it is one-shot, no retry; use expect instead. **(6)** Trap: asserting on an evaluate result and reintroducing flakiness.

CODE

```
// Inspect application state at the point of failure
const appState = await page.evaluate(() => ({
cart: JSON.parse(localStorage.getItem('cart') || '{}'),
flags: (window as any).__FEATURE_FLAGS__,
}));
console.log('State at failure:', JSON.stringify(appState, null, 2));

// Set up state the UI cannot
await page.evaluate((token: string) => {
localStorage.setItem('auth_token', token);
}, 'test-token-12345');

// Rule: setup/debugging only, assertions use the retrying expect API
```

Q 160 LEVEL · Mid

What is Playwright UI Mode, and when do you use it?

CONCEPT

UI Mode (launched with **--ui**) is an interactive test runner with a watch mode, a timeline view, and a live browser preview. It lists all tests in a sidebar, runs individual tests or groups, and shows the trace timeline inline as tests run, and because it watches files, saving a test re-runs it automatically. It is the fastest workflow for active development, with a time-travel scrubber to move back and forward through actions and find exactly where a test diverges. Knowing which tool

fits which moment is the real signal: **--debug** for step-through breakpoint debugging, **--ui** for watch-mode development, --headed for a quick visual check, the Trace Viewer for CI post-mortems, and page.pause() for targeted mid-test inspection.

INTERVIEW STRATEGY

This one is checking for tool fluency and judgement. Describe UI Mode as your default for writing and debugging tests, watch mode, inline traces, time-travel scrubber, then close with the tool-selection map, because knowing when to use --ui versus --debug versus the Trace Viewer is what signals maturity. The thing to avoid is treating every tool as interchangeable. The follow-up is often 'when do you use UI Mode versus --debug?', answer UI Mode for active development, --debug for targeted breakpoint debugging.

How to phrase it

> *UI Mode, launched with --ui, is an interactive runner with watch mode, a timeline view and a live browser preview, and it is my default workflow when I am writing new tests or debugging locally. I open the test I am working on, and every time I save the file it re-runs automatically with the trace visible inline, so I never switch between terminal and browser, everything is in one window. The time-travel scrubber lets me move backward and forward through the actions to find exactly where the test diverges from what I expected. But the real signal here, and what I would close on, is knowing which tool fits which moment, which answers the follow-up about UI Mode versus --debug directly. I use --ui for active development, --debug for targeted step-through breakpoint debugging when I want to pause at a specific line, --headed for a quick visual sanity check in a normal browser window, the Trace Viewer for CI post-mortems where I am diagnosing a failure I cannot reproduce, and page.pause for targeted mid-test inspection. The mistake is treating those as interchangeable; each has a moment where it is clearly the right one. So my honest answer is UI Mode is where I live day to day, and I reach deliberately for the others when the situation calls for them, and being able to make that call quickly is itself a sign of framework maturity.*

Key points to hit

(1) --ui: watch mode, inline traces, live preview, time-travel scrubber. **(2)** Default workflow for writing and debugging tests; auto-reruns on save. **(3)** Tool map: --debug (breakpoints), --ui (dev), --headed (visual check). **(4)** Trace Viewer for CI post-mortems; page.pause() for mid-test inspection. **(5)** Asked next: --ui for active development, --debug for targeted breakpoints. **(6)** Where this goes wrong: treating every debug tool as interchangeable.

CODE

```
# Watch mode with inline traces: default dev workflow
npx playwright test --ui
npx playwright test login.spec.ts --ui

# Choosing the right tool for the moment:
# --debug -> step-through Inspector for targeted debugging
# --ui -> watch mode with inline traces and live preview
# --headed -> quick visual check in a normal window
# trace viewer -> CI post-mortem and failure analysis
# page.pause() -> targeted mid-test inspection
```

Q 161 LEVEL · Mid

How do PWDEBUG and DEBUG=pw:api change Playwright's runtime behaviour for diagnosis?

CONCEPT

Two environment variables expose internal Playwright behaviour without changing test code. **PWDEBUG=1** enables Inspector mode: every test runs headed with the Playwright Inspector attached, default timeouts disabled, and tests pause at each step so you can step through manually. **PWDEBUG=console** is the lighter version: headed, no Inspector, no auto-pause, but exposes the playwright object in the browser devtools console so you can run **playwright.locator('button').highlight()** interactively. DEBUG=pw:api logs every Playwright API call with timing, useful for understanding what auto-waiting did, why an action timed out, or which actionability check failed. DEBUG=pw:protocol shows the raw CDP/WebSocket protocol traffic between Playwright and the browser, the level below the API, for diagnosing framework bugs or strange browser behaviour. The discipline: PWDEBUG for live interactive debugging, DEBUG=pw:api for understanding what happened in a failed run, DEBUG=pw:protocol only when you suspect Playwright or the browser themselves.

INTERVIEW STRATEGY

The point of this one is to see env-var-level diagnosis tools. Lead with the four variables (PWDEBUG=1 full inspector, PWDEBUG=console light, DEBUG=pw:api logs, DEBUG=pw:protocol raw CDP) and what each exposes. The matched-tool-to-diagnostic-need rule is the seniority marker. The pitfall is reaching for protocol-level when api-level would have answered. Expect the follow-up ‘which do you use most?’, answer DEBUG=pw:api on CI failures, it tells me what auto-waiting actually waited on.

How to phrase it

> *Two environment variables expose internal Playwright behaviour without changing test code, and using each at the right time is what separates effective from random debugging. PWDEBUG equals one enables Inspector mode: every test runs headed with the Playwright Inspector attached, default timeouts disabled, and tests pause at each step so I can step through manually. The right tool when I'm actively debugging a test on my machine. PWDEBUG equals console is the lighter version: headed, no Inspector, no auto-pause, but exposes the playwright object in the browser DevTools console so I can run playwright dot locator quote button highlight interactively. Useful when I want a normal test run but with the option to inspect locators from DevTools. DEBUG equals pw colon api logs every Playwright API call with timing, useful for understanding what auto-waiting actually did, why an action timed out, or which actionability check failed. This is the variable I reach for most when investigating a CI failure where the trace doesn't make the cause obvious, the timing log shows me click waited four seconds before timing out, which check it was waiting on, and what the element looked like at each retry. For the follow-up question about which I use most: DEBUG equals pw colon api on CI failures. DEBUG equals pw colon protocol shows the raw CDP and WebSocket protocol traffic between Playwright and the browser, the level below the API, for diagnosing framework bugs or strange browser behaviour. I almost never need this; it's the tool for when I suspect Playwright itself or the browser are misbehaving rather than my test code. The discipline I would close on, because it captures the calibration, is PWDEBUG for live interactive debugging, DEBUG equals pw colon api for understanding what happened in a failed run, DEBUG equals pw colon protocol only when you suspect Playwright or the browser themselves. Match the variable to the question I'm trying to answer. The classic error is reaching for protocol-level diagnostics when api-level would have answered the question, you spend hours reading raw CDP frames when the API log would have shown the failing actionability check in one line.*

Key points to hit

(1) PWDEBUG=1: headed + Inspector + no timeouts + auto-pause, live debug. **(2)** PWDEBUG=console: lighter, exposes playwright object in DevTools console. **(3)** DEBUG=pw:api: logs every API call with timing, understand failed runs. **(4)** DEBUG=pw:protocol: raw CDP/WebSocket, only for framework/browser-level bugs. **(5)** Discipline: match the variable to the question (api-level usually answers it). **(6)** What loses the point: reaching for protocol-level when api-level would have answered.

CODE

```
# Live debugging on my machine: full inspector, no timeouts
PWDEBUG=1 npx playwright test tests/checkout.spec.ts

# Headed run with playwright object in DevTools console
PWDEBUG=console npx playwright test --headed
# In DevTools: > playwright.locator('button[name="Save"]').highlight()

# Investigate a CI failure: see what auto-waiting actually did
DEBUG=pw:api npx playwright test tests/failing.spec.ts 2>&1 | tee debug.log
# Log shows: 'click' -> waiting for 'visible' -> waiting for 'stable' -> ...

# Last resort: raw protocol traffic for framework/browser-level issues
DEBUG=pw:protocol npx playwright test tests/strange.spec.ts
```

Q 162

LEVEL · Mid to Senior

How do you use the trace viewer's time-travel features beyond opening a trace zip?

CONCEPT

The trace viewer is far more than a screenshot gallery. Five capabilities worth knowing for senior diagnosis. **Action timeline with DOM scrubbing**: click any action on the left, and the centre pane shows the rendered DOM at that exact moment, before and after, so you can compare what changed. Network tab: every request and response with full headers and body, filterable by status code or URL, the right tool when a test failed because an API returned an unexpected response. **Console tab**: all browser console messages including those that fired during background async operations, often containing the real error your assertion missed. Source tab: the test source at the action's call site, essential for diagnosing failures in tests with complex helpers or POMs. **The metadata panel**: per-action duration, retries, snapshots taken. The combination lets you reconstruct the exact sequence: what the page looked like, what the network was doing, what console said, where in the code, all timestamped. The discipline: start from the failing action, scrub the DOM, then check network and console for the cause.

INTERVIEW STRATEGY

What earns the tick is trace viewer used at senior depth, not just opened. Walk the five capabilities (timeline+DOM scrub, network, console, source, metadata) and what each tells you. The 'start from failing action, scrub DOM, check network and console' workflow is the senior signal. The risk is treating traces as screenshot galleries. They usually follow with ‘what's the most useful tab?’, answer DOM scrubbing on the failing action, then network for cause.

How to phrase it

> *The trace viewer is far more than a screenshot gallery, and using its five capabilities at depth is what makes CI failures actually diagnosable. First, action timeline with DOM scrubbing. I click any action on the left, and the centre pane shows the rendered DOM at that exact moment, both before and after, so I can compare what changed. This is the single highest-leverage feature, and my answer to the follow-up about most useful tab: DOM scrubbing on the failing action. I can see whether the button I clicked was actually visible, whether it had moved between frames, whether something was overlaying it. Second, network tab: every request and response with full headers and body, filterable by status code or URL. The right tool when a test failed because an API returned an unexpected response, I filter to four-xx and five-xx and see which calls failed, often surfacing the real cause that the UI assertion didn't name. Third, console tab: all browser console messages including those that fired during background async operations, often containing the real error my assertion missed. JavaScript errors that the test ignored because they happened in a different code path show up here. Fourth, source tab: the test source at the action's call site, essential for diagnosing failures in tests with complex helpers or POMs, because I can see exactly which line of which helper caused the action, not just the top-level test. Fifth, the metadata panel: per-action duration, retries, snapshots taken. The combination lets me reconstruct the exact sequence: what the page looked like, what the network was doing, what console said, where in the code, all timestamped. The discipline I would lead with, because it's the workflow that converts trace viewer from a tool into a methodology, is start from the failing action, scrub the DOM, then check network and console for the cause. Failing action is the symptom; DOM tells me the page state; network and console tell me the cause. The mistake is treating traces as screenshot galleries, opening it, looking at a few images, closing it without going deep, and giving up on the failure as inscrutable.*

Key points to hit

(1) Action timeline + DOM scrubbing: before/after at every action. **(2)** Network tab: requests/responses with full headers/body, status filter. **(3)** Console tab: browser console messages including background async errors. **(4)** Source tab: call-site source, critical for diagnosing helper/POM failures. **(5)** Workflow: failing action -> DOM scrub -> network + console for cause. **(6)** The common miss: opening trace as a screenshot gallery, not using DOM/network/console.

CODE

```
# Open a trace from CI
npx playwright show-trace test-results/checkout-test/trace.zip
# or drag-drop into https://trace.playwright.dev

# Senior diagnostic workflow (top-to-bottom in trace viewer):
#
# 1. Left panel -> click the failing action (red bar)
# 2. Centre -> use the slider above DOM to scrub before/after
# Did the button I clicked actually exist? Was it covered?
# 3. Network -> filter to 4xx/5xx: did an API call fail silently?
# 4. Console -> any uncaught JS errors during the action's time window?
# 5. Source -> jump to the helper or POM method that called this
# 6. Metadata -> how long did it take? How many retries? snapshots stored?
#
# Compare DOM before vs after to isolate exactly what state changed.
```

Q 163 LEVEL · Mid

How do you use page.pause() with Chrome DevTools for mid-test inspection?

CONCEPT

page.pause() stops the test at the call site, opens the Playwright Inspector, and lets the browser stay live for inspection. Once paused, you can switch to the browser's regular Chrome DevTools, same DevTools the developer uses, and get the full debugging surface: DOM inspector, network panel showing all requests, console for running JavaScript against the page, application tab for localStorage and cookies, source panel for stepping through application code. The combination is the closest thing to live debugging a test in production: the test machinery is paused, the application is live, you can poke at it with the same tools you'd use against a deployed app. The discipline: page.pause() inside the test, at the line you want to inspect, removed before merging. ESLint rules can flag page.pause calls in committed code so they don't sneak into main. The follow-up power move: in Inspector, click Record to capture interactive steps as Playwright code, then paste into the test, useful for diagnosing flaky tests where you want to capture the exact selector path that worked manually.

INTERVIEW STRATEGY

You need to give the page.pause + DevTools workflow understood. Lead with what pause does (stops test, browser stays live), then the DevTools surfaces (DOM, network, console, application, source) that become available. The 'closest thing to debugging in production' framing signals depth here. Where this goes wrong is leaving page.pause in committed code. The next question is normally ‘what else can you do in the Inspector?’, answer record interactive steps as Playwright code via the Record button.

How to phrase it

> *page dot pause stops the test at the call site, opens the Playwright Inspector, and lets the browser stay live for inspection. That last part is the key insight: the test machinery is paused but the browser is running normally, so I can switch to the browser's regular Chrome DevTools, the same DevTools the developer uses, and get the full debugging surface. DOM inspector to see the rendered tree as it is right now. Network panel showing all requests that fired, including the ones after I paused. Console for running JavaScript directly against the page, evaluate state, call functions, modify the DOM to test hypotheses. Application tab for localStorage and cookies, useful for diagnosing auth state issues. Source panel for stepping through the application code, not just the test code. The combination is the closest thing to live debugging a test in production: the test machinery is paused, the application is live, I can poke at it with the same tools I'd use against a deployed app. This is the framing I'd lead with because it's what makes pause uniquely powerful, every other Playwright debugging tool shows me a record of what happened; pause lets me interact with what's happening. The discipline that prevents this from polluting the suite is page dot pause inside the test, at the line I want to inspect, removed before merging. ESLint rules can flag page dot pause calls in committed code so they don't sneak into main. The follow-up power move, and my answer to the follow-up question about what else the Inspector can do, is the Record button. In Inspector, click Record to capture interactive steps as Playwright code, then paste into the test. Useful for diagnosing flaky tests where I want to capture the exact selector path that worked manually, or for adding a new test step without writing the selector by hand. The trap is leaving page dot pause in committed code; CI hangs forever waiting on an Inspector nobody is there to drive.*

Key points to hit

(1) page.pause() stops the test; browser stays live for inspection. **(2)** Full Chrome DevTools available: DOM, network, console, application, source. **(3)** Closest thing to debugging in production: interact, not just observe. **(4)** Discipline: at the line you want to inspect; removed before merging. **(5)** Follow-up: Inspector's Record button captures interactive steps as code. **(6)** Avoid: leaving page.pause in committed code, CI hangs forever.

CODE

```
test('debug a flaky checkout step', async ({ page }) => {
await page.goto('/checkout');
await page.getByLabel('Card number').fill('4111 1111 1111 1111');
await page.getByRole('button', { name: 'Place order' }).click();

await page.pause(); // <- test pauses here, browser stays live
// In the Inspector window:
// - Step over each command
// - Click Record to capture new actions as Playwright code
//
// In Chrome DevTools attached to the browser:
// - Elements: inspect why the success toast isn't appearing
// - Network: did the /api/orders call return 200?
// - Console: any uncaught errors? Try page.evaluate equivalents
// - Application > Local Storage: is the auth token still set?

await expect(page.getByText('Order placed')).toBeVisible();
});

// .eslintrc.json: prevent page.pause from being merged
// "no-restricted-syntax": ["error", { "selector":
"CallExpression[callee.property.name='pause']" }]
```

Q 164

LEVEL · Junior to Mid

What is codegen, how do you run it, and how do you treat its output as a starting point rather than final tests?

CONCEPT

codegen is Playwright's recorder: run npx playwright codegen <url> and it opens a browser alongside the Inspector, then writes test code as you click, type and navigate, generating locators and actions for every interaction. It is a fast way to scaffold a new flow or discover how to reach a tricky element, and it has handy options: **--target** to pick the language, **--save-storage** to capture a logged-in session, and a Record button inside the Inspector so you can resume recording mid-pause. The catch is that codegen output is a draft, not a finished test. It favours CSS and text selectors over semantic ones, has no assertions beyond what you explicitly record, repeats setup instead of using fixtures, and knows nothing of your page-object structure. So the workflow is: record to scaffold, then upgrade the generated CSS selectors to getByRole and getByLabel, add web-first expect assertions, lift shared setup into fixtures, and fold the actions into the relevant page object. Treated that way it saves real time; committed raw, it produces brittle, assertion-free tests.

INTERVIEW STRATEGY

They are testing whether you know codegen's limits, not whether you can run it. Lead with what it does, records clicks into locators and actions, a fast scaffold, then pivot quickly to the discipline: it is a draft. Name the specific gaps (CSS selectors over semantic, no real assertions, no fixtures, no page objects) and the upgrade you do on every recording. That honesty is the signal. The trap is implying you commit codegen output as-is. A common follow-up is ‘so what do you change before it goes in?’, answer swap CSS for getByRole, add expect assertions, lift setup into fixtures, fold into the page object.

How to phrase it

> *codegen is Playwright's recorder. I run npx playwright codegen with a URL, it opens a browser next to the Inspector, and as I click, type and navigate it writes the locators and actions for me. So it is genuinely useful for two things: scaffolding a brand new flow quickly, and working out how to reach a tricky element when I am not sure of the right locator. There are nice options too: target to choose the language, save-storage to capture a logged-in session so I do not record the login every time, and a Record button inside the Inspector to resume recording after a pause. But the part an interviewer actually cares about is that I treat the output as a draft, not a finished test, and I can say exactly why. codegen favours CSS and text selectors rather than semantic ones, it has no assertions beyond what I explicitly record, it repeats setup instead of using fixtures, and it knows nothing about my page-object structure. So my workflow is record to scaffold, then do the upgrade pass, which is also my answer to the follow-up about what changes before it goes in: I swap the generated CSS selectors for getByRole and getByLabel so they survive UI change, I add web-first expect assertions because a recording without assertions is not really a test, I lift any shared setup like login into a fixture, and I fold the actions into the relevant page object rather than leaving a flat script. Treated that way codegen saves me real time on the boring scaffolding. Committed raw, it produces exactly the brittle, assertion-free tests that give recorded automation a bad name, so I would never let the generated code go in untouched.*

Key points to hit

(1) codegen records clicks/typing into locators and actions, a scaffold. **(2)** Run: npx playwright codegen <url>; --target, --save-storage, Record button. **(3)** Output is a draft: CSS selectors, no real assertions, no fixtures/POM. **(4)** Upgrade pass: CSS -> getByRole/getByLabel for resilience. **(5)** Add expect assertions, lift setup into fixtures, fold into the page object. **(6)** The easy mistake: committing codegen output as-is, brittle, assertion-free tests.

CODE

```
# Record a flow: opens a browser plus the Inspector
npx playwright codegen https://example.com

# Record already logged in, and save the session for reuse
npx playwright codegen --save-storage=auth.json https://example.com
npx playwright codegen --load-storage=auth.json https://example.com/account

// What codegen generates (a draft, note the CSS, no assertions):
await page.click('#nav > ul > li:nth-child(2) > a');
await page.fill('input[name="q"]', 'playwright');

// After the upgrade pass, semantic locators + a real assertion:
await page.getByRole('link', { name: 'Products' }).click();
await page.getByRole('searchbox').fill('playwright');
await expect(page.getByRole('heading', { name: /results/i })).toBeVisible();
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What does page.pause() do during local debugging?

A) Adds a fixed sleep
B) Captures a screenshot
C) Disables auto-waiting
D) Freezes the test at that line and opens the Inspector to inspect page state

2. Why configure trace: 'on-first-retry' rather than trace: 'on'?

A) It captures more data
B) It is required for CI
C) It gives a full trace exactly when a test fails and retries, without wasting storage on passing runs
D) It disables screenshots

3. Why add a test that asserts no console errors occurred?

A) JS errors are often invisible to functional tests, the page looks fine but throws in the background
B) Console errors always fail tests automatically
C) It speeds up the suite
D) It replaces network mocking

4. What does test.step() change about a test?

A) It changes execution order
B) Nothing functional: it groups actions into named, collapsible steps in the report and trace
C) It adds retries
D) It runs steps in parallel

5. What is the firm rule for using page.evaluate()?

A) Use it for setup and debugging, never for assertions, it is a one-shot snapshot
B) Use it for all assertions
C) Never use it
D) Use it only in CI

Section 11 · Answer key

#	Answer	Why and where to revisit
1	D) Freezes the test at that line and opens the Inspector to inspect page state	Run with --debug and page.pause() opens the Inspector with the browser frozen so you can inspect and try locators. *Revisit Section 11.*
2	C) It gives a full trace exactly when a test fails and retries, without wasting storage on passing runs	on-first-retry captures the trace only when needed (on a retry), avoiding the storage cost of tracing every passing test. *Revisit Section 11.*
3	A) JS errors are often invisible to functional tests, the page looks fine but throws in the background	Functional tests can pass while an uncaught exception fires silently; asserting no console errors catches it. *Revisit Section 11.*
4	B) Nothing functional: it groups actions into named, collapsible steps in the report and trace	test.step is purely organisational; it makes traces readable by grouping raw actions into named logical steps. *Revisit Section 11.*
5	A) Use it for setup and debugging, never for assertions, it is a one-shot snapshot	page.evaluate does not retry, so assertions must use the retrying expect API; reserve evaluate for setup/inspection. *Revisit Section 11.*

SECTION 12

Reporting

Reporting is how test results reach people who are not in the terminal, so interviewers want to see you run multiple reporters for different audiences and push failures to where the team already looks. These eight questions cover configuring the built-in HTML reporter alongside JUnit/JSON/GitHub, integrating Allure for history trends, and writing a custom reporter that posts to Slack or Teams.

In this section

- Multiple reporters at once: HTML for humans, JUnit for CI, github for PR annotations.
- open: 'never' in CI, the setting that stops the pipeline hanging.
- Allure for history-trend charts that correlate flakiness with deployments.
- allure.step and labels for stakeholder-readable reports.
- A custom Slack/Teams reporter that posts pass rate and top failures.
- Guarding the notifier so it is a no-op locally and only fires in CI.

Q 165 LEVEL · Junior to Mid

How do you configure Playwright's built-in HTML reporter?

CONCEPT

The HTML reporter generates a rich interactive report with results, screenshots, traces, and video, the primary report for local review and CI artifacts. The professional setup runs **multiple reporters at once**, each for a different audience: html for human review, junit for CI test-result parsing, **json** for custom tooling, and github, which annotates the PR inline with exactly which tests failed, so a reviewer sees failures without downloading the HTML report. The setting that matters most in CI is open: 'never': without it, Playwright tries to launch a browser to show the report and **hangs the pipeline**. View locally with show-report.

INTERVIEW STRATEGY

You are being asked to show you report for multiple audiences, not just generate one HTML file. List the reporters with the audience each serves, and call out the github reporter for inline PR annotations as the team-friendly touch. The must-mention operational detail is open: 'never', the setting whose absence hangs CI, which signals you have actually run this. The failure mode is leaving the reporter trying to open a browser in CI. The follow-up is often ‘which reporter for which consumer?’, map html to humans, junit to CI, github to reviewers.

How to phrase it

> *Playwright's HTML reporter gives a rich interactive report with results, screenshots, traces and video, and it is my primary report for both local review and CI artifacts. But the professional setup, and what I would lead with, is running several reporters at once because each serves a different audience. HTML is for human review, JUnit is for CI to parse test results into its own dashboards, JSON is for any custom tooling we have, and the github reporter is the one I highlight: it annotates the pull request inline with exactly which tests failed and why, so a reviewer sees the failures directly in the PR without having to download and open the HTML report. That answers the follow-up about which reporter for which consumer, html for humans, junit for CI, github for reviewers. The operational detail I always call out, because forgetting it has burned teams, is open never in CI: without it Playwright tries to launch a browser to display the report and the pipeline just hangs, which looks like a stuck build with no obvious cause. So I set open never, run the multi-reporter array, and the result is that every audience, developers, reviewers, CI dashboards, gets the results in the form they actually use, instead of one HTML file someone has to go hunting for.*

Key points to hit

(1) HTML report: results, screenshots, traces, video for review. **(2)** Run multiple reporters: html (humans), junit (CI), json (tooling), github (PR). **(3)** github reporter annotates the PR inline: no download needed. **(4)** open: 'never' in CI or Playwright tries to open a browser and hangs. **(5)** Expect next: map reporter to consumer, html/junit/github. **(6)** Trap: leaving the reporter trying to open a browser in CI.

CODE

```
// playwright.config.ts
export default defineConfig({
reporter: [
['html', { outputFolder: 'playwright-report', open: 'never' }], // never in CI
['junit', { outputFile: 'results/junit.xml' }], // CI parsing
['json', { outputFile: 'results/results.json' }], // custom tooling
['github'], // PR annotations
['dot'], // minimal console
],
});

// npx playwright show-report
```

Q 166 LEVEL · Mid

How do you integrate Allure reporting with Playwright?

CONCEPT

Allure provides richer reporting than the built-in HTML report, test categorisation, history trends, environment information, and custom metadata, via the **allure-playwright** package. You add it as a reporter, then attach metadata to tests: labels (feature, severity), descriptions, links to tickets, and **allure.step** for a human-readable breakdown of what each test did. The killer feature is the history-trend chart: it shows pass rate over time across builds, so when a test starts failing intermittently you can see exactly when it began and correlate it with the deployment history. Using allure.step on every test makes the report meaningful for non-technical stakeholders who need to understand what was tested, not just whether it passed.

INTERVIEW STRATEGY

They need to know why you would reach past the built-in report. Lead with Allure's differentiator, the history-trend chart that correlates a test going flaky with the deployment that caused it, because that is the capability the built-in report lacks. Mention allure.step and labels for stakeholder-readable output. The thing to avoid is adding Allure with no reason beyond ‘it looks nicer’. Expect the follow-up ‘when is Allure worth the extra setup?’, answer when you need history trends or reports non-technical stakeholders can read.

How to phrase it

> *Allure gives richer reporting than the built-in HTML report, categorisation, history trends, environment info and custom metadata, through the allure-playwright package, and I would justify it by its differentiator rather than by looks. The killer feature is the history-trend chart: it shows pass rate over time across builds, so when a test starts failing intermittently I can see exactly when it began and line it up against the deployment history to find the change that introduced the flakiness. The built-in report shows me this run; Allure shows me the trend, and that is the answer to the follow-up about when it is worth the extra setup, when I need history trends or reports that non-technical stakeholders can actually read. To make it readable for those stakeholders, I add allure.step to every test so the report shows a human breakdown of what the test did, not just a pass or fail, plus labels for feature and severity and a link to the JIRA ticket. That turns the report into something a product owner can open and understand what was covered. The trap I would avoid is adding Allure just because it is prettier; if I am only ever looking at a single run, the built-in HTML report is enough. I bring in Allure specifically when the trend over time and the stakeholder readability are the things I need.*

Key points to hit

(1) allure-playwright adds categorisation, trends, environment info, metadata. **(2)** Killer feature: history-trend chart, pass rate over time across builds. **(3)** Correlate a test going flaky with the deployment that caused it. **(4)** allure.step + labels make reports readable for non-technical stakeholders. **(5)** They usually ask next: worth it for history trends or stakeholder-readable reports. **(6)** Where this goes wrong: adding Allure with no reason beyond 'it looks nicer'.

CODE

```
# npm install --save-dev allure-playwright allure-commandline

// playwright.config.ts
reporter: [['allure-playwright', { detail: true, outputFolder: 'allure-results' }]]

import { allure } from 'allure-playwright';
test('checkout flow', async ({ page }) => {
allure.label('feature', 'Checkout');
allure.label('severity', 'critical');
allure.link('https://jira.example.com/PROJ-123', 'JIRA Ticket');
await allure.step('Navigate to checkout', async () => { await page.goto('/checkout');
});
await allure.step('Place order', async () => {
await page.getByRole('button', { name: 'Place Order' }).click();
});
});
# npx allure generate allure-results --clean -o allure-report
```

Q 167 LEVEL · Mid

How do you create a custom reporter for Slack or Teams notifications?

CONCEPT

A custom reporter implements the Reporter interface and can send notifications to any external system when a run completes. In **onTestEnd** you tally results; in **onEnd** you post a message to a Slack or Teams webhook with the pass rate, the first few failures by name, and a link to the full report. This moves failures to where the team already looks, a message appears in the alerts channel within seconds of a run completing, so people start investigating before the next deployment instead of checking CI manually. The essential guard is **if (!process.env.SLACK_WEBHOOK_URL) return** at the top: the reporter is then a no-op locally and only activates in CI where the webhook is configured, so local runs never spam the channel.

INTERVIEW STRATEGY

What scores here is showing you close the loop from test result to team action. Frame the Slack reporter as pushing failures to where the team already is, so nobody has to poll CI. Specify the useful payload, pass rate, top failures by name, report link, not just 'it sends a message'. The must-mention detail is the webhook guard so it is a no-op locally; that shows you have shipped it without spamming the channel. The pitfall is a reporter that fires on local runs too. They usually follow with 'how do you keep it from spamming during development?', answer the env-var guard.

How to phrase it

> *A custom reporter implements the Reporter interface, so I can send a notification anywhere when a run completes, and the Slack version genuinely changed how our team responds to failures. Before it, someone had to remember to check CI; now a message appears in the qa-alerts channel within seconds of a run finishing. I tally results in onTestEnd, and in onEnd I post to the Slack webhook with a useful payload, not just a pass-or-fail, the pass rate, the first five failures by name so people can see at a glance what broke, and a link to the full report. The effect is that the team sees failures immediately and can start investigating before the next deployment, instead of finding out later. The detail I always include, and the answer to the follow-up about keeping it from spamming during development, is the guard at the top: if the SLACK_WEBHOOK_URL environment variable is not set, the reporter returns immediately and does nothing. So it is a no-op on local runs and only activates in CI where the webhook is configured, which means I am not flooding the channel every time I run a test on my machine. That guard is the difference between a notifier the team values and one they mute in a week. So the pattern is: tally, post a rich message to where the team already looks, and guard it so it only fires in CI.*

Key points to hit

(1) Implement the Reporter interface; post to a webhook in onEnd. **(2)** Payload: pass rate, first few failures by name, link to the report. **(3)** Pushes failures to where the team already looks, no manual CI polling. **(4)** Guard: if (!process.env.SLACK_WEBHOOK_URL) return, no-op locally. **(5)** The next question: the env-var guard stops it spamming during development. **(6)** What loses the point: a reporter that fires on local runs too.

CODE

```
import { Reporter, TestCase, TestResult, FullResult } from '@playwright/test/reporter';

class SlackReporter implements Reporter {
private failures: string[] = [];
private passed = 0;
onTestEnd(test: TestCase, result: TestResult) {
if (result.status === 'passed') this.passed++;
else if (result.status === 'failed') this.failures.push(test.title);
}
async onEnd(_result: FullResult) {
if (!process.env.SLACK_WEBHOOK_URL) return; // no-op locally
const total = this.passed + this.failures.length;
const passRate = ((this.passed / total) * 100).toFixed(1);
const text = `Pass rate ${passRate}%, failures: ${this.failures.slice(0, 5).join(', ')
|| 'none'}`;
await fetch(process.env.SLACK_WEBHOOK_URL, {
method: 'POST', body: JSON.stringify({ text }),
});
}
}
export default SlackReporter;
```

Q 168 LEVEL · Mid

How do you run multiple reporters at once, and what's the right combination for production?

CONCEPT

The **reporter** config option accepts an array, running each reporter against the same events. The right production combination layers reporters by audience and purpose. list for the terminal during local development (live output as tests run); **html** for human investigation (browsable report with screenshots and traces); json for machine-readable output to drive dashboards or custom analysis; junit for CI integration with tools that expect xUnit XML; a custom reporter for team-specific integrations like Slack on failure or JIRA traceability. The discipline that matters: **each reporter serves a different audience**, list for developers, html for engineers investigating failures, json/junit for tools, custom for stakeholder notifications. Stacking reporters has minimal cost because they're event subscribers, not separate test runs. The trap: choosing one reporter and trying to make it serve everyone, ending up with output that's good for nobody.

INTERVIEW STRATEGY

They are after reporter composition understood. Lead with the array config, then walk the five-reporter production combination (list/html/json/junit/custom) and what audience each serves. The 'reporters are subscribers, not test runs' framing covers the cost question. The trap is one-reporter-fits-all. The next question is normally ‘what's the cost of multiple reporters?’, answer minimal, they're event listeners, not extra test passes.

How to phrase it

> *The reporter config option accepts an array, running each reporter against the same events as tests execute. The production combination I'd configure layers reporters by audience and purpose, because each output format serves a different consumer. List for the terminal during local development, live output as tests run, the format developers actually watch. Html for human investigation when something fails, browsable report with screenshots, traces, retry history, everything an engineer needs to investigate. Json for machine-readable output that drives dashboards or custom analysis, the file my flake-tracking script parses, the input to test-duration trend visualisation. Junit for CI integration with tools that expect xUnit XML format, Jenkins test result widgets, Azure DevOps test panels, anything in that ecosystem. A custom reporter for team-specific integrations, Slack on failure, JIRA traceability, the dashboard post-test hooks. The discipline I would lead with, because it captures the design principle, is each reporter serves a different audience. List for developers, html for engineers investigating failures, json or junit for tools, custom for stakeholder notifications. Stacking reporters has minimal cost because they're event subscribers, not separate test runs, Playwright fires onTestEnd once, every registered reporter gets the event, no extra browser work, no extra timing. When they push for the follow-up about cost: minimal, they're listeners not extra test passes. The risk is choosing one reporter and trying to make it serve everyone, ending up with output that's good for nobody. The HTML reporter isn't grep-friendly; the JSON isn't human-readable; the list isn't archivable. Use the right format for each audience and accept that production CI needs four or five reporters running together.*

Key points to hit

(1) reporter accepts an array: multiple reporters run against the same events. **(2)** Production stack: list + html + json + junit + custom. **(3)** Each reporter serves a different audience: dev, engineer, tool, stakeholder. **(4)** Cost is minimal: reporters are event subscribers, not extra test passes. **(5)** Asked next: stacking is essentially free, one test run, many outputs. **(6)** The common miss: one-reporter-fits-all, output that serves nobody well.

CODE

```
// playwright.config.ts: layered reporters for different audiences
export default defineConfig({
reporter: [
['list'], // dev terminal
['html', { outputFolder: 'playwright-report', open: 'never' }], // engineer
investigation
['json', { outputFile: 'reports/results.json' }], // dashboards/analysis
['junit', { outputFile: 'reports/junit.xml' }], // CI tool integration
['./reporters/slack-on-failure.ts'], // stakeholder notification
['./reporters/jira-traceability.ts'], // requirements coverage
],
});

// Stack is essentially free: all reporters subscribe to the same events
```

Q 169

LEVEL · Mid

How do you operate the HTML report in production: attachments, hosting, and trace embedding?

CONCEPT

The HTML report is more than a static page; it's a self-contained application with embedded traces and attachments. Three production operations matter. **Attachments**: tests call **testInfo.attach(name, { body, contentType })** to embed files, screenshots, or data in the report, viewable inline, downloadable from the failure page. Useful for attaching custom diagnostic data: API request/response bodies, generated CSVs, log dumps. **Hosting**: the report is a folder of static files, push it to S3, Netlify, GitHub Pages, an internal Nginx, anywhere static hosting works. The url ends up in PR comments or Slack messages so engineers click straight to the failure. Trace embedding: when trace recording is enabled, the HTML report includes a trace viewer link per failure that opens the trace directly, no separate download. The discipline: upload the HTML report from every PR run, link it from the PR comment. The faster the failing engineer can click from “test failed” to “trace open in browser”, the shorter the debug cycle.

INTERVIEW STRATEGY

They want HTML report as a production tool. Lead with the three operations: attachments via testInfo.attach, static hosting, embedded trace links. The 'fast click from failure-notice to trace-open' framing is the senior signal. The thing to avoid is HTML report kept as a local artifact only. A common follow-up is ‘what do you attach?’, answer custom diagnostic data: API bodies, generated CSVs, log dumps.

How to phrase it

> *The HTML report is more than a static page; it's a self-contained application with embedded traces and attachments, and three production operations matter when running it for real. First, attachments. Tests call testInfo dot attach with a name and a body or path to embed files, screenshots, or data in the report. The attachment is viewable inline if it's an image or text, downloadable from the failure page otherwise. The use case I'd give to make it concrete, and my answer to the follow-up about what to attach: custom diagnostic data. API request and response bodies for failed calls. Generated CSVs the test produced. Log dumps from the application under test. Anything that helps the engineer diagnose without re-running the test. Second, hosting. The HTML report is a folder of static files, so I push it to S3, Netlify, GitHub Pages, an internal Nginx, anywhere static hosting works. The URL ends up in PR comments or Slack messages so engineers click straight to the failure. Third, trace embedding. When trace recording is enabled, the HTML report includes a trace viewer link per failure that opens the trace directly in the same browser tab, no separate download, no asking the dev to fetch the trace zip from CI artifacts and drag-drop into the trace viewer. One click from the report's failure page to the trace timeline. The discipline I would close on, because it's the production move that compounds, is upload the HTML report from every PR run and link it from the PR comment. The faster the failing engineer can click from test failed to trace open in browser, the shorter the debug cycle. Friction adds time: every extra step between failure and diagnosis is time spent not fixing. The failure mode is keeping HTML report as a local artifact only, where engineers have to know to npx playwright show-report to look at it, the dev who isn't the test author won't, and the failure goes investigated less than it should.*

Key points to hit

(1) Attachments via testInfo.attach(): inline for images, downloadable otherwise. **(2)** Static hosting: S3, Netlify, GitHub Pages, internal Nginx. **(3)** Trace links embedded per failure: one click from report to trace timeline. **(4)** Upload + link from PR comment: minimise friction to investigation. **(5)** Follow-up: attach API bodies, CSVs, log dumps, custom diagnostic data. **(6)** Avoid: HTML report kept local-only, only the test author looks at it.

CODE

```
// Attach custom diagnostic data: visible in HTML report
test('order flow', async ({ request }, testInfo) => {
const res = await request.post('/api/orders', { data: { product: 'X' } });
const body = await res.json();
await testInfo.attach('order-response.json', {
body: JSON.stringify(body, null, 2),
contentType: 'application/json',
});
expect(res.status()).toBe(201);
});

# .github/workflows/test.yml: upload + link from PR comment
- if: always()
uses: actions/upload-artifact@v4
with: { name: html-report, path: playwright-report/, retention-days: 7 }
- if: always()
uses: peter-evans/create-or-update-comment@v4
with:
issue-number: ${{ github.event.pull_request.number }}
body: |
Playwright report: ${{ steps.deploy-report.outputs.url }}
(trace viewer linked per failure)
```

Q 170 LEVEL · Mid to Senior

How do test annotations turn tests into living documentation?

CONCEPT

testInfo.annotations attaches arbitrary metadata to test results that flows through to every reporter. Three real uses. **Requirements traceability**: annotate each test with the requirement IDs it covers, **testInfo.annotations.push({ type: 'requirement', description: 'REQ-1234' })**, and a custom reporter aggregates which requirements are covered and which aren't, producing a coverage dashboard that proves compliance. Test classification: annotate with severity (P0, P1), category (smoke, integration), team owner (auth-team, billing-team), reporters segment failures by owner, dashboards filter by severity. Failure context: when a test detects something non-fatal but worth flagging, annotate with the context for later review. The discipline that makes annotations valuable: structured types with consistent vocabulary. Adopt a small set of annotation types (requirement, owner, severity, jira, issue) and enforce them via a helper function so the data stays queryable. Free-form annotation strings produce a rubbish dump nobody can analyse.

INTERVIEW STRATEGY

What earns the tick is annotations as documentation, not just metadata. Lead with three real uses (requirements traceability, classification, failure context). The 'structured types with consistent vocabulary' rule is the senior signal because free-form annotations produce unqueryable rubbish. The classic error is string-typed annotations across the suite. The follow-up is often ‘what do you actually do with the annotation data?’, answer the custom reporter aggregates it into a coverage dashboard or owner filter.

How to phrase it

> *testInfo dot annotations attaches arbitrary metadata to test results that flows through to every reporter. The objects have a type and a description, and they appear in JSON output, HTML report, every reporter that subscribes to onTestEnd, so they become structured data the team can act on. Three real uses I'd give to make this concrete. First, requirements traceability. I annotate each test with the requirement IDs it covers, testInfo dot annotations dot push curly type colon requirement description colon REQ-1234. A custom reporter aggregates which requirements are covered and which aren't, producing a coverage dashboard that proves compliance. For regulated industries like finance or healthcare where requirements traceability is audit material, this turns tests into the documentation auditors actually want. Second, test classification. I annotate with severity like P0 or P1, category like smoke or integration, team owner like auth-team or billing-team. Reporters then segment failures by owner, the auth team gets pinged on auth-test failures, billing on billing failures, and dashboards filter by severity so P0 failures get attention before P3 ones. Third, failure context. When a test detects something non-fatal but worth flagging, an unexpected console warning, a slow API response within SLA but trending up, I annotate with the context for later review. The discipline that makes annotations valuable, and where most teams ship a useful idea badly, is structured types with consistent vocabulary. Adopt a small set of annotation types like requirement, owner, severity, jira, issue, and enforce them via a helper function so the data stays queryable. Free-form annotation strings produce a rubbish dump nobody can analyse. The whole point is producing structured data; loose strings defeat that. That's also my answer to the follow-up about what we do with the data: the custom reporter aggregates it into the coverage dashboard or owner filter or severity-segmented Slack channel. Annotations are the raw material; the reporter is the kitchen. The pitfall is string-typed annotations across the suite, every team picks their own vocabulary, the aggregation produces nothing useful, the idea gets abandoned.*

Key points to hit

(1) testInfo.annotations: metadata attached to tests, flows through all reporters. **(2)** Use 1: requirements traceability, REQ-IDs, coverage dashboard. **(3)** Use 2: classification, severity/category/owner for filtering and routing. **(4)** Use 3: non-fatal failure context, console warnings, slow trends. **(5)** Discipline: structured types via helper function, queryable data. **(6)** The easy mistake: free-form annotation strings, unqueryable rubbish dump.

CODE

```
// Helper enforces vocabulary: keeps annotations queryable
type Annotation =
| { type: 'requirement'; description: string } // REQ-1234
| { type: 'owner'; description: string } // 'auth-team'
| { type: 'severity'; description: 'P0' | 'P1' | 'P2' }
| { type: 'jira'; description: string }; // PROJ-456

function annotate(testInfo: TestInfo, a: Annotation): void {
testInfo.annotations.push(a);
}

test('admin can delete user', async ({ page }, testInfo) => {
annotate(testInfo, { type: 'requirement', description: 'REQ-1234' });
annotate(testInfo, { type: 'owner', description: 'auth-team' });
annotate(testInfo, { type: 'severity', description: 'P0' });
annotate(testInfo, { type: 'jira', description: 'PROJ-789' });
// ... test body
});

// Custom reporter aggregates: REQ coverage report, owner dashboards, P0 alerts
```

Q 171 LEVEL · Senior

How do you build a PR-comment reporter that posts results back to the originating pull request?

CONCEPT

The pattern: a custom reporter that listens for **onEnd**, summarises the run, and posts a comment to the originating PR via the GitHub API. Three implementation details that distinguish a useful version from a noisy one. **Idempotency**: find-or-create the comment by a marker string in the body (**<!-- playwright-results -->**) and update rather than append, so the PR doesn't accumulate dozens of test-result comments across force-pushes. Failure detail with context: post the first 3-5 failures with test name, error first line, and a link to the trace in the hosted HTML report, enough for the dev to know which tests to investigate, not so much that the comment becomes unreadable. Cross-link to artifacts: the comment includes a link to the full HTML report and the trace artifact, so investigation is one click away. The discipline: the comment is the entry point, not the summary, it tells the dev what to investigate and where to click, not every detail of the run. The trap: a verbose comment with full stack traces that drowns the PR's discussion thread.

INTERVIEW STRATEGY

You need to give production-grade PR-comment integration. Walk the three implementation details: idempotency via marker string, failure detail at 3-5 tests max, cross-links to artifacts. The 'comment is entry point, not summary' framing is the senior signal. The trap is comment noise drowning PR discussion. Expect the follow-up ‘how do you avoid spamming the PR?’, answer marker string for find-or-create idempotency, no append-on-rerun.

How to phrase it

> *The pattern is a custom reporter that listens for onEnd, summarises the run, and posts a comment to the originating PR via the GitHub API. Conceptually simple; the implementation details are where it stops being a nuisance and becomes useful. Three details that distinguish a useful version from a noisy one. First, idempotency. I find-or-create the comment by a marker string in the body, something like an HTML comment containing playwright-results, and update rather than append. Without this, every push to the PR adds a new comment, and after ten force-pushes during iteration the PR has ten test-result comments crowding the actual discussion. That's also my answer to the follow-up about avoiding PR spam: marker string for find-or-create idempotency, never append on rerun. Second, failure detail with context. I post the first three to five failures with test name, the first line of the error message, and a link to the trace in the hosted HTML report. Three to five is the empirical sweet spot: enough for the dev to know which tests to investigate, not so much that the comment becomes unreadable. If twenty tests failed, the comment says first five and a link to see all of them, not the full twenty inline. Third, cross-link to artifacts. The comment includes a link to the full HTML report and the trace artifact, so investigation is one click away. The developer reads the comment, clicks the trace link, lands in the trace viewer at the failing action. The discipline I would close on, because it captures the design principle, is the comment is the entry point, not the summary. It tells the dev what to investigate and where to click, not every detail of the run. Details live in the linked HTML report; the comment is the navigation. The trap is a verbose comment with full stack traces that drowns the PR's discussion thread, the comment becomes noise everyone scrolls past, the dev investigates from the GitHub Actions logs instead, and the integration provides no value.*

Key points to hit

(1) Custom reporter on onEnd posts to GitHub API. **(2)** Idempotency: marker string in body, find-or-create, update not append. **(3)** Failure detail: first 3-5 failures with trace link, not the full list. **(4)** Cross-link to HTML report + trace artifact: one click to investigation. **(5)** Principle: the comment is entry point, not summary, details live in the link. **(6)** Trap: verbose comment with full stack traces, noise everyone scrolls past.

CODE

```
// reporters/pr-comment.ts
import type { Reporter, TestCase, TestResult } from '@playwright/test/reporter';
import { Octokit } from '@octokit/rest';

const MARKER = '<!-- playwright-results -->';

export default class PrCommentReporter implements Reporter {
private failed: { test: TestCase; result: TestResult }[] = [];

onTestEnd(test: TestCase, result: TestResult): void {
if (result.status === 'failed' || result.status === 'timedOut') {
this.failed.push({ test, result });
}
}

async onEnd(): Promise<void> {
const prNumber = Number(process.env.GITHUB_PR_NUMBER);
if (!prNumber) return;
const summary = this.failed.slice(0, 5).map((f, i) =>
`${i + 1}. **${f.test.title}**: ${f.result.error?.message?.split('\n')[0]?.slice(0,
200)}`
).join('\n');
const body =
`${MARKER}\n` +
`### Playwright: ${this.failed.length} failed\n\n` +
`${summary}\n\n` +
`[Full report](${process.env.REPORT_URL}) · [Trace artifact](${process.env.TRACE_URL})`;

const gh = new Octokit({ auth: process.env.GITHUB_TOKEN });
const existing = (await gh.issues.listComments({
owner: 'org', repo: 'repo', issue_number: prNumber,
})).data.find(c => c.body?.includes(MARKER));

if (existing) {
await gh.issues.updateComment({ owner: 'org', repo: 'repo', comment_id: existing.id,
body });
} else {
await gh.issues.createComment({ owner: 'org', repo: 'repo', issue_number: prNumber, body
});
}
}
}
```

Q 172 LEVEL · Senior

How do you export test metadata to an observability platform for long-term trend analysis?

CONCEPT

Built-in reporters answer per-run questions; observability answers trend questions. Is this test getting slower over time? Are flakes concentrated in one part of the suite? Did yesterday's deploy increase failure rate? Answering these requires shipping test metadata to a time-series-aware system. The pattern: a custom reporter that emits structured events per test, test name, duration, status, retries, browser, annotations, to Datadog, New Relic, Honeycomb, Prometheus pushgateway,

an internal observability stack, or just a JSON-line file ingested by a scheduled job. The events become queryable: p95 duration per test over the last 30 days, failure rate per team-owner from annotations, flake correlation with time-of-day. The discipline: **emit structured fields, not free-form strings**, so the data can be aggregated and filtered. The follow-up insight: this is how flake-tracking dashboards from S10 actually work, the retry-needed list is a query on the emitted metadata, not a separate manual tally. Observability turns test results into product insight.

INTERVIEW STRATEGY

They are after test results as an observability data source. Lead with the trend questions built-in reporters can't answer, then the per-test event emission pattern. Name real platforms (Datadog, New Relic, Honeycomb). The 'structured fields, not free-form strings' rule and the callback to S10's flake dashboard are the senior signals. The mistake is free-form log lines. They usually follow with ‘what queries do you actually run?’, answer p95-duration-per-test, flake-rate-per-owner, deploy-impact-on-failure.

How to phrase it

> *Built-in reporters answer per-run questions: did this run pass, what failed, how long. Observability answers trend questions, which is the level senior interviewers probe because it's where the test suite stops being a gate and starts being insight. Is this test getting slower over time? Are flakes concentrated in one part of the suite? Did yesterday's deploy increase failure rate? Did the new framework version make the auth tests more reliable? Answering these requires shipping test metadata to a time-series-aware system. The pattern is a custom reporter that emits structured events per test, test name, duration, status, retries, browser, annotations, to wherever the team's observability lives. Datadog, New Relic, Honeycomb, Prometheus pushgateway, an internal observability stack, or even just a JSON-line file ingested by a scheduled job. The events become queryable: p95 duration per test over the last thirty days, failure rate per team-owner from annotations, flake correlation with time-of-day, regression after specific commits. If they push on this in the follow-up, my answer is about queries I actually run: p95 duration per test, flake rate per owner, deploy impact on failure rate, browser-specific reliability. The discipline I would emphasise, because free-form data defeats the purpose, is emit structured fields, not free-form strings. test_name as a tag, duration_ms as a number, status as an enum, owner as a tag from the annotations. That lets the data be aggregated and filtered. Free-form log lines force me to regex against text every time I want to ask a question, which is exactly the wrong direction. The follow-up insight that lands as the senior framing, and connects back to earlier in the kit, is this is how flake-tracking dashboards from CI/CD section actually work. The retry-needed list is a query on the emitted metadata, select tests where retries greater than zero over the last seven days, group by test name, order by count descending, not a separate manual tally. Observability turns test results from a gate that says pass or fail into a data source that says how the system is trending. The mistake is free-form log lines: data exists but is unqueryable, the value is theoretical, the insights never get extracted.*

Key points to hit

(1) Trend questions need observability: slowing tests, flake hotspots, deploy impact. **(2)** Custom reporter emits per-test structured events to Datadog/Honeycomb/etc. **(3)** Fields: test_name, duration_ms, status, retries, browser, annotation tags. **(4)** Queries: p95 duration, flake rate per owner, deploy impact on failure rate. **(5)** Flake dashboards from S10 are queries against this

metadata, not manual tallies. **(6)** Where this goes wrong: free-form log lines, data exists but is unqueryable, insights never extracted.

CODE

```
// reporters/observability.ts: emit structured events per test
import type { Reporter, TestCase, TestResult } from '@playwright/test/reporter';

export default class ObservabilityReporter implements Reporter {
onTestEnd(test: TestCase, result: TestResult): void {
const owner = test.annotations.find(a => a.type === 'owner')?.description ?? 'unknown';
const severity = test.annotations.find(a => a.type === 'severity')?.description ?? 'P3';
const event = {
timestamp: new Date().toISOString(),
test_name: test.titlePath().join(' > '),
project: test.parent.project()?.name,
duration_ms: result.duration,
status: result.status,
retries: result.retry,
owner, // queryable tag
severity, // queryable tag
ci_commit: process.env.GITHUB_SHA,
};
// ship to Datadog/Honeycomb/Prometheus/etc.: or jsonl file ingested by a cron
fetch('https://obs.internal/v1/events', { method: 'POST', body: JSON.stringify(event)
});
}
}

// Queries that become possible:
// SELECT p95(duration_ms) FROM events WHERE test_name=$1 AND ts > NOW()-30d
// SELECT count(*) FROM events WHERE retries > 0 GROUP BY owner ORDER BY 1 DESC
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Why set open: 'never' on the HTML reporter in CI?

A) To hide the report
B) To speed up tests
C) Otherwise Playwright tries to launch a browser to show the report and hangs the pipeline
D) To disable screenshots

2. What is the value of running multiple reporters at once?

A) Each serves a different audience: html for humans, junit for CI, github for PR annotations
B) It runs tests faster
C) It reduces flakiness
D) Only one reporter is allowed

3. What is Allure's standout capability over the built-in HTML report?

A) Faster generation
B) The history-trend chart: pass rate over time, correlating flakiness with deployments
C) It needs no setup
D) It blocks deployment

4. What payload makes a Slack test-failure notification useful?

A) Just 'tests failed'
B) Only the total test count
C) The entire HTML report inline
D) Pass rate, the first few failures by name, and a link to the full report

5. Why guard a Slack reporter with if (!process.env.SLACK_WEBHOOK_URL) return?

A) So it is a no-op locally and only fires in CI: local runs do not spam the channel
B) To encrypt the webhook
C) To speed up the reporter
D) To require the webhook everywhere

Section 12 · Answer key

#	Answer	Why and where to revisit
1	C) Otherwise Playwright tries to launch a browser to show the report and hangs the pipeline	Without open: 'never', the reporter attempts to open a browser in CI, which has no display, and the build hangs. *Revisit Section 12.*
2	A) Each serves a different audience: html for humans, junit for CI, github for PR annotations	Different consumers need different formats; a reporter array gives each audience the output it actually uses. *Revisit Section 12.*
3	B) The history-trend chart: pass rate over time, correlating flakiness with deployments	Allure's history trend shows when a test started failing across builds, which the single-run HTML report cannot. *Revisit Section 12.*
4	D) Pass rate, the first few failures by name, and a link to the full report	A useful alert shows the rate, names the top failures so the team sees what broke, and links to the full report. *Revisit Section 12.*
5	A) So it is a no-op locally and only fires in CI: local runs do not spam the channel	The guard makes the notifier inactive without the webhook, so development runs never flood the alerts channel. *Revisit Section 12.*

SECTION 13

Configuration and Fixtures

Fixtures are Playwright's dependency-injection system, and interviewers use them to tell people who wire setup by hand from people who let the framework do it with guaranteed teardown. These fifteen questions cover how fixtures resolve and tear down, worker-scoped fixtures for expensive setup, composing fixture files at scale, why fixtures beat beforeAll/afterAll, test.use for per-block configuration, and an environment fixture that keeps destructive tests out of production.

In this section

- Fixtures as DI: the yield pattern, dependency graph, reverse-order teardown.
- Worker-scoped fixtures for expensive once-per-worker setup like auth.
- Composing fixture files by merging extended test objects, scale to 50+.
- Fixtures vs beforeAll: guaranteed teardown and composability.
- test.use for per-test and per-describe config (storageState, viewport, locale).
- An environment fixture that skips destructive tests in production.

Q 173 LEVEL · Mid

How do fixtures work internally in Playwright?

CONCEPT

Fixtures are Playwright's **dependency-injection** system. Each fixture is a function that does setup, yields the value to the test with use(), then does teardown after. Playwright resolves the dependency graph automatically: if fixture B depends on fixture A, A is set up first and torn down last. Teardown runs in **reverse order**: so an authenticatedPage that depends on a testUser is closed before the user is deleted, never the other way round. The crucial property is that the yield pattern gives try/finally semantics: teardown runs **even if the test throws**, and even if a later fixture's setup fails. That guarantee is what makes fixtures safe for resource creation in a way ad-hoc setup is not.

INTERVIEW STRATEGY

They want the mental model, not the syntax. Frame fixtures as dependency injection, then nail the two things that show real understanding: automatic dependency-graph resolution and reverse-order teardown that still runs when the test throws. The try/finally framing is the experience marker. The thing to avoid is describing a fixture as just 'setup before the test' and missing the teardown guarantee. The next question is normally 'what happens to teardown if the test fails?', answer it runs regardless, in reverse dependency order.

How to phrase it

> *I describe fixtures as Playwright's dependency-injection system. A fixture does its setup, yields the value to the test with use, then runs teardown after the test. The two things I would emphasise because they show I actually understand the model are dependency resolution and teardown order. Playwright reads the dependency graph automatically: if my authenticatedPage fixture depends on a testUser fixture, it creates the testUser first, and then on the way out it tears down in reverse order, it closes the page before it deletes the user, never the reverse, which would leave a closed page trying to act on a deleted user. The property that actually matters in practice, and my answer to the follow-up about what happens to teardown if the test fails, is that the yield pattern gives me try/finally semantics: the teardown runs even if the test throws, and even if a later fixture's setup blows up partway through. That is the whole reason I trust fixtures for creating real resources like users and orders, I know the cleanup will run no matter how the test ends, so I do not leak test data. Ad-hoc setup in the test body does not give me that guarantee, which is exactly why I moved everything to fixtures.*

Key points to hit

(1) Fixtures are DI: setup, yield with use(), then teardown. **(2)** Playwright resolves the dependency graph automatically. **(3)** Teardown runs in reverse order: page closed before user deleted. **(4)** yield gives try/finally semantics: teardown runs even on throw. **(5)** Expect next: teardown on failure, runs regardless, reverse order. **(6)** What loses the point: calling a fixture just 'setup' and missing the teardown guarantee.

CODE

```
export const test = base.extend<{ testUser: User; authedPage: Page }>({
testUser: async ({ request }, use) => {
const res = await request.post('/api/users', { data: { email: `t-${Date.now()}@x.com` }
});
const user = await res.json();
await use(user); // test runs here
await request.delete(`/api/users/${user.id}`); // teardown, always runs
},
authedPage: async ({ page, testUser }, use) => { // depends on testUser
await page.goto('/login');
await page.getByLabel('Email').fill(testUser.email);
await page.getByRole('button', { name: 'Sign in' }).click();
await page.waitForURL('/dashboard');
await use(page); // closed before testUser is deleted (reverse order)
},
});
```

Q 174 LEVEL · Mid

What are worker-scoped fixtures and when do you use them?

CONCEPT

A **worker-scoped** fixture is created once per worker process and shared across every test in that worker, declared with { scope: 'worker' }. The default is **function scope**: fresh per test. Worker scope is the right home for expensive, stateful setup like an authenticated BrowserContext: if 50 tests in a worker all need an authenticated admin context and you build it per test, you pay the setup cost 50 times; worker-scoped pays it once. For a 2-second context setup that is roughly 98 seconds saved per worker. The standard pairing: **worker-scoped for the BrowserContext** (expensive, stateful) and function-scoped for the Page (cheap, and must be fresh per test to avoid state leaking between tests).

INTERVIEW STRATEGY

They are probing performance instinct. Contrast worker scope with the function default, then quantify the saving (50 setups versus one) because the number sells it. The pairing rule, worker-scoped context, function-scoped page, is the detail that shows you have tuned this. The thing to avoid is making the Page worker-scoped, which leaks state between tests. A common follow-up is 'why not share the page too?', answer that a shared page carries state across tests and breaks isolation.

How to phrase it

> *A worker-scoped fixture is created once per worker process and shared across every test in that worker, and you declare it with scope worker; the default is function scope, which is fresh per test. Worker scope is the performance optimisation most teams miss, so it is the one I lead with. The example is an authenticated admin context. If fifty tests in a worker all need that context and I build it function-scoped, I create and destroy it fifty times; worker-scoped builds it once. For a context that takes two seconds to set up, that is around ninety-eight seconds saved per worker, multiplied across every worker in the run. But there is a pairing rule I always state, and it sets up the follow-up about why I do not just share the page too: I make the BrowserContext worker-scoped because it is expensive and stateful, but I keep the Page function-scoped because a page carries state, open dialogs, scroll position, in-progress forms, and sharing it across tests would leak that state and break isolation, reintroducing exactly the flakiness parallelism is supposed to avoid. So the pattern is worker-scoped context for the expensive auth, function-scoped page off that context for each test. That gives me both the speed and the isolation.*

Key points to hit

(1) Worker-scoped: created once per worker, shared across its tests. **(2)** Default is function scope (fresh per test). **(3)** Quantify: 50 setups vs 1, ~98s saved per worker for a 2s setup. **(4)** Pairing: worker-scoped context, function-scoped page. **(5)** They usually ask next: don't share the page, it leaks state and breaks isolation. **(6)** The common miss: making the Page worker-scoped.

CODE

```
// Worker-scoped: expensive context, once per worker
export const test = base.extend<{}, { adminContext: BrowserContext }>({
adminContext: [async ({ browser }, use) => {
const ctx = await browser.newContext({ storageState: 'auth/admin.json' });
await use(ctx);
await ctx.close(); // when the worker finishes
}, { scope: 'worker' }],
});

// Function-scoped page off the shared context, fresh per test
export const test2 = test.extend<{ adminPage: Page }>({
adminPage: async ({ adminContext }, use) => {
const page = await adminContext.newPage();
await use(page);
await page.close();
},
});
```

Q 175 LEVEL · Mid to Senior

How do you compose multiple fixture files in a large framework?

CONCEPT

Fixture files compose by **merging extended test objects**: each domain owns a fixture file, and each file **extends the previous one** rather than the base. A page-fixtures file extends base, an api-fixtures file extends the page test, and an index file re-exports the final composed test plus

expect. Tests then **import from one place** and get every fixture. The payoff is organisational: the API team owns api.fixtures, the UI team owns page.fixtures, and they compose at the index level, so adding a fixture is a one-file change that touches no test files. This is module composition from application code applied to test infrastructure, and it is what lets a framework reach 50+ fixtures without a single monolithic file everyone edits and conflicts over.

INTERVIEW STRATEGY

What they are checking is whether you can scale fixtures past a single file. Describe the extend-chain composition and the single import for tests, then sell the ownership angle: each domain owns its file, adding a fixture touches no tests. That organisational point is the senior signal. The risk is one giant monolithic fixture file that becomes a merge-conflict magnet. The follow-up is often 'how does adding a fixture not break tests?', answer that composition is additive and tests import from the single index.

How to phrase it

> *I compose fixtures by merging extended test objects: each domain owns its own fixture file, and each file extends the previous one rather than extending base directly. So page.fixtures extends base, api.fixtures extends the page test, and an index file re-exports the final composed test along with expect. Tests import from that one index and get everything. The reason this matters, and what I would emphasise, is organisational rather than technical. The API team owns api.fixtures, the UI team owns page.fixtures, and they compose at the index level, so the teams are not all editing one file. That sets up the follow-up about how adding a fixture does not break tests: composition is purely additive, a new fixture is a one-file change in the relevant domain file, the index picks it up, and because tests only declare the fixtures they actually use in their arguments, existing tests are completely untouched. This is just module composition from application code applied to test infrastructure, and it is what lets a framework grow to fifty or more fixtures without a single monolithic file that everyone edits and constantly conflicts over. The monolith is the trap; the extend-chain with a single index export is what scales.*

Key points to hit

(1) Each file extends the previous test object; index re-exports the final one. **(2)** Tests import from one index and get every fixture. **(3)** Each domain team owns its fixture file; compose at the index. **(4)** Adding a fixture is a one-file change touching no test files. **(5)** The next question: composition is additive, tests declare only what they use. **(6)** Avoid: a single monolithic fixture file that becomes a conflict magnet.

CODE

```
// page.fixtures.ts
export const pageTest = base.extend<{ loginPage: LoginPage }>({
loginPage: ({ page }, use) => use(new LoginPage(page)),
});

// api.fixtures.ts: extends the page test
import { pageTest } from './page.fixtures';
export const apiTest = pageTest.extend<{ userAPI: UserAPI }>({
userAPI: ({ request }, use) => use(new UserAPI(request)),
});

// index.ts: single composed export
export const test = apiTest;
export { expect } from '@playwright/test';

// any test: one import, all fixtures
import { test, expect } from '@fixtures/index';
```

Q 176 LEVEL · Mid

What is the difference between beforeAll/afterAll and fixtures, and when do you use each?

CONCEPT

beforeAll/afterAll are hooks that run once per describe block; **fixtures** are DI that run per test or per worker. The differences that decide it: fixtures are composable and reusable across files, hooks are local to the describe block; fixtures guarantee teardown even if setup throws, afterAll does not (if beforeAll fails partway, afterAll may not clean up, leaking resources); fixtures support **dependency injection**, hooks rely on shared mutable variables. Playwright recommends fixtures for almost everything. The narrow case where beforeAll is still appropriate: one-time, read-only setup with no cleanup, loading a large static dataset once for a block of read-only tests, or incremental migration of a legacy codebase.

INTERVIEW STRATEGY

This question is there to see you default to fixtures for the right reasons. Lead with the teardown guarantee, fixtures clean up even when setup throws, afterAll may not, because that is the decisive difference. Add composability across files. Then show balance by naming the narrow case where beforeAll still fits. The failure mode is using beforeAll with shared mutable variables for resource setup. Expect the follow-up 'is beforeAll ever the right choice?', answer yes, for one-time read-only setup needing no cleanup.

How to phrase it

> *beforeAll and afterAll are hooks that run once per describe block, while fixtures are dependency injection that run per test or per worker, and I use fixtures for essentially everything beforeAll used to do. The decisive reason, which I lead with, is the teardown guarantee. A fixture's teardown runs even if the test throws and even if setup fails partway through, because of the yield-based try/finally semantics. afterAll does not give me that: if beforeAll itself throws halfway, afterAll may never run, and I am left with a half-created user orphaned in the database. The second reason is composability: a fixture defined once is available to every test in the project that imports it, whereas a beforeAll block is local to one describe block, and fixtures use dependency injection instead of shared mutable variables that tests reach into. Now, to show balance and answer the follow-up about whether beforeAll is ever right, yes, narrowly. I will use beforeAll for truly one-time, read-only setup that needs no cleanup: loading a large static dataset once for a block of read-only tests, or when I am migrating a legacy suite to fixtures incrementally. But the moment setup creates a resource that needs tearing down, I reach for a fixture, because the cleanup guarantee is the thing that keeps the suite from leaking data.*

Key points to hit

(1) Hooks run per describe block; fixtures run per test/worker via DI. **(2)** Fixtures guarantee teardown even if setup throws; afterAll may not. **(3)** Fixtures are composable across files; hooks are local to the block. **(4)** Default to fixtures for anything that creates a resource. **(5)** Asked next: beforeAll fits one-time read-only setup with no cleanup. **(6)** The easy mistake: beforeAll + shared mutable variables for resource setup.

CODE

```
// Fragile: afterAll may not run if beforeAll throws midway
test.describe('admin flow', () => {
let userId: number;
test.beforeAll(async ({ request }) => {
userId = (await (await request.post('/api/users', { data: {} })).json()).id;
});
test.afterAll(async ({ request }) => { await request.delete(`/api/users/${userId}`); });
});

// Robust: fixture teardown ALWAYS runs
export const test = base.extend<{ adminUser: User }>({
adminUser: async ({ request }, use) => {
const user = await (await request.post('/api/users', { data: { role: 'admin' }
})).json();
await use(user);
await request.delete(`/api/users/${user.id}`); // guaranteed
},
});
```

Q 177 LEVEL · Junior to Mid

How do you use test.use() for per-test and per-describe configuration?

CONCEPT

test.use() overrides configuration for a specific test or describe block without changing the global config, the correct way to set **storageState**, viewport, locale, **timezoneId** and similar per-scope settings. The cleanest pattern is to organise tests into describe blocks by user role and set storageState once at the block level, so every test inside inherits the auth state, far more maintainable than repeating it per test. The same applies to a mobile viewport block or a German-locale block for date-formatting tests. This also makes the suite self-documenting: the describe block's name tells you which role, device, or locale is under test, so a reader understands the context without reading the setup.

INTERVIEW STRATEGY

They want to see clean configuration over duplication. Present test.use as the per-scope override and lead with the role-based describe-block pattern: set storageState once, every test inherits it. Note that it makes the suite self-documenting: the block name signals the context. The classic error is repeating storageState in every test. They usually follow with 'how do you organise tests by role?', answer describe blocks per role with test.use at the block level.

How to phrase it

> *test.use overrides configuration for a specific test or describe block without touching the global config, and it is the right tool for things like storageState, viewport, locale and timezone. The pattern I use most, and the one that answers the follow-up about organising tests by role, is to group tests into describe blocks by user role, an admin block, a user block, a viewer block, and set storageState once at the block level with test.use. Every test inside the block then inherits that auth state, which is far more maintainable than repeating storageState in every single test and remembering to update them all when the auth file changes. I do the same for a mobile-checkout block where I set a phone viewport once, or a date-formatting block where I set a German locale and a Berlin timezone so I can verify localised date rendering. A side benefit I would point out is that this makes the suite self-documenting: the describe block's name tells a reader exactly which role, device or locale is under test, so they understand the context without digging through setup code. So test.use is how I keep per-scope configuration in one obvious place rather than scattered and duplicated, and the block structure communicates intent on its own.*

Key points to hit

(1) test.use overrides config per test/describe without changing global config. **(2)** Sets storageState, viewport, locale, timezoneId per scope. **(3)** Group by role; set storageState once at the block level. **(4)** More maintainable than repeating storageState per test. **(5)** Follow-up: organise tests in role-based describe blocks with test.use. **(6)** Trap: duplicating storageState in every test.

CODE

```
// Per-describe: role-based blocks
test.describe('Admin panel', () => {
test.use({ storageState: 'auth/admin-state.json' });
test('can view all users', async ({ page }) => { /* ... */ });
test('can delete users', async ({ page }) => { /* ... */ });
});

// Mobile viewport block
test.describe('Mobile checkout', () => {
test.use({ viewport: { width: 390, height: 844 } });
test('checkout works on mobile', async ({ page }) => { /* ... */ });
});

// Locale + timezone block
test.describe('Date formatting', () => {
test.use({ locale: 'de-DE', timezoneId: 'Europe/Berlin' });
test('dates display in German format', async ({ page }) => { /* ... */ });
});
```

Q 178 LEVEL · Mid

How do you implement environment-specific configuration in fixtures?

CONCEPT

An **environment fixture** reads environment variables and exposes a typed config object, baseURL, apiURL, isCI, and an **environment** enum (local, staging, production), so tests switch behaviour without changing code. Its highest-value use is preventing destructive tests from running in production: every test that creates, modifies, or deletes data checks the environment and calls **test.skip()** if it is production, while read-only smoke tests run everywhere. This is safer than relying on developers to remember a skip condition, the fixture makes the environment available to every test and the skip logic is explicit and reviewable rather than implicit and forgettable. It is the difference between a documented safety rule and a hope that nobody runs a delete test against prod.

INTERVIEW STRATEGY

You are being marked on for production safety instincts. Present the typed env fixture, then make its killer use the centre: skipping destructive tests in production via an explicit, reviewable check. Framing it as safer than trusting developers to remember is the senior signal. The mistake is hardcoding environment branching in scattered tests. The next question is normally ‘how do you stop a destructive test hitting prod?’, answer the fixture exposes the environment and the test skips on production, with read-only tests running everywhere.

How to phrase it

> *An environment fixture reads the environment variables and exposes a typed config object, baseURL, apiURL, an isCI flag, and an environment enum of local, staging or production, so tests can switch behaviour without changing code. The highest-value use, and what I would build the answer around, is preventing destructive tests from running in production. Every test that creates, modifies or deletes data checks the environment from the fixture and calls test.skip if it is production, while read-only smoke tests run everywhere including prod. That is also my answer to the follow-up about how I stop a destructive test hitting production: the guard is explicit in the test and the environment comes from one fixture, so it is reviewable in code review. The reason I prefer this over scattering environment branching through individual tests, which is the trap, is safety through structure rather than memory, I am not relying on every developer to remember to add a skip condition on every new destructive test, because the fixture makes the environment available everywhere and the pattern is consistent and visible. It is the difference between a documented, enforceable safety rule and a hope that nobody accidentally points a delete-user test at prod. For tests running against production I also keep them to a read-only smoke subset on a single browser, so even the non-destructive prod runs stay lightweight.*

Key points to hit

(1) Env fixture exposes a typed config: baseURL, apiURL, isCI, environment enum. **(2)** Tests switch behaviour by environment without code changes. **(3)** Destructive tests check environment and test.skip() in production. **(4)** Explicit, reviewable guard beats relying on developer memory. **(5)** Expect next: read-only smoke runs everywhere; destructive skips in prod. **(6)** Where this goes wrong: scattering environment branching across individual tests.

CODE

```
interface EnvConfig {
baseURL: string; apiURL: string; isCI: boolean;
environment: 'local' | 'staging' | 'production';
}

export const test = base.extend<{ envConfig: EnvConfig }>({
envConfig: async ({}, use) => {
const environment = (process.env.TEST_ENV || 'staging') as EnvConfig['environment'];
await use({ baseURL: process.env.BASE_URL!, apiURL: process.env.API_URL!, isCI:
!!process.env.CI, environment });
},
});

test('destructive: delete user', async ({ page, envConfig }) => {
test.skip(envConfig.environment === 'production', 'No destructive tests in production');
// ... safe to create/delete data
});
```

Q 179 LEVEL · Senior

How does Playwright resolve the fixture dependency graph, and what happens with a circular dependency?

CONCEPT

Playwright resolves fixtures via **topological sort of the dependency graph**: when a test requests fixtures, Playwright walks the graph, determines the order each fixture must run in, and sets them up bottom-up so every fixture's dependencies are ready before it runs. The graph is built from the destructured argument list in each fixture: a fixture declared as authenticatedPage: async ({ page }, use) => ... depends on the page fixture, so page resolves first. **Circular dependencies**, fixture A depending on B and B depending on A, cause Playwright to throw a clear error at test-discovery time, before any test runs: *circular fixture dependency*. This is the design's safety net: the failure is loud and immediate, not silent at runtime. The discipline that prevents circular deps in the first place: fixtures should flow one direction, from atomic (page, request) to composed (authenticatedPage, adminPage). If you find yourself wanting A to depend on B and B on A, extract the shared piece into a third fixture they both depend on.

INTERVIEW STRATEGY

Your job here is to show fixture resolution understood. Lead with topological sort, the compiler-y answer, then circular deps caught at discovery time as the safety net. The atomic-to-composed direction rule is the senior signal. The thing to avoid is bidirectional deps. A common follow-up is 'what's the error if you have a circular dep?', answer Playwright throws at discovery time before any test runs.

How to phrase it

> *Playwright resolves fixtures via topological sort of the dependency graph. When a test requests fixtures, Playwright walks the graph, determines the order each fixture must run in, and sets them up bottom-up so every fixture's dependencies are ready before it runs. The graph is built from the destructured argument list in each fixture's definition: a fixture declared as authenticatedPage async curly page use depends on the page fixture, so page resolves before authenticatedPage. This is the same dependency-injection model TypeScript developers know from frameworks like NestJS or Angular, just adapted to test scope. The interesting case, and where the design shows real care, is circular dependencies. If fixture A depends on B and B depends on A, Playwright throws a clear error at test-discovery time, before any test runs, circular fixture dependency, with the cycle named. That's the safety net: the failure is loud and immediate, not silent at runtime where it would be much harder to debug. That's also my answer to the follow-up about what happens with a circular dep: it's caught at discovery time. The thing to emphasise first, because it prevents the situation in the first place, is fixtures should flow one direction, from atomic to composed. Atomic fixtures, page, request, browser, sit at the bottom of the graph. Composed fixtures, authenticatedPage, adminPage, userBuilderWithCleanup, build on top of atomic ones. Test-specific fixtures sit at the top. If I find myself wanting A to depend on B and B on A, the right answer is to extract the shared piece into a third fixture they both depend on. That refactors the cycle into a tree. The failure mode is bidirectional deps, usually introduced by convenience helpers that gradually accrete responsibility until they need each other.*

Key points to hit

(1) Topological sort of the dependency graph: deps resolved bottom-up. **(2)** Graph built from destructured argument lists. **(3)** Circular deps caught at TEST DISCOVERY time, not runtime, fail-fast safety net. **(4)** Flow rule: atomic fixtures (page, request) -> composed -> test-specific. **(5)** Extract

shared piece into a third fixture to break a circular dep. **(6)** What loses the point: bidirectional deps, usually convenience helpers that grew responsibilities.

CODE

```
// Atomic -> composed -> test-specific (one direction)
type MyFixtures = {
authenticatedPage: Page; // composed
adminPage: Page; // composed
userBuilder: UserBuilder; // test-specific
};

export const test = base.extend<MyFixtures>({
authenticatedPage: async ({ page }, use) => { // depends on page
await page.goto('/login'); /* sign in */ await use(page);
},
adminPage: async ({ authenticatedPage }, use) => { // depends on authenticatedPage
await authenticatedPage.goto('/admin'); await use(authenticatedPage);
},
userBuilder: async ({ request }, use) => { // depends on request
await use(new UserBuilder(request));
},
});

// CIRCULAR (Playwright errors at discovery time):
// fixtureA: async ({ fixtureB }, use) => { ... }
// fixtureB: async ({ fixtureA }, use) => { ... }
// Error: Circular fixture dependency detected: fixtureA -> fixtureB -> fixtureA
```

Q 180 LEVEL · Mid to Senior

What are auto-fixtures, and when are they the right pattern?

CONCEPT

An **auto-fixture** (declared with auto: true) runs for every test in scope **without being explicitly requested** in the test's argument list. The right use cases are narrow but valuable. Cross-cutting setup everyone needs: a test-id tagging fixture that annotates each test with metadata; an OpenTelemetry tracer that wraps every test with a span; a console-error guard that fails the test if the browser console logged an error during the run. **Cleanup that must happen regardless**: a fixture that ensures temp files are cleaned even for tests that don't touch the filesystem explicitly. The discipline: auto-fixtures should be invisible, they shouldn't change test behaviour, only observe or guard it. A fixture that auto-changes the page's locale or auto-sets the viewport is the wrong use case; that's per-describe test.use territory. Auto-fixtures that do real work rather than observe are how surprise behaviour leaks into tests and people lose hours debugging "why is this test using French".

INTERVIEW STRATEGY

Show auto-fixtures used deliberately. Lead with what auto: true does, runs without being requested, then the narrow right use cases (observation/guarding, not modification). The 'invisible: observe or guard, never change' rule is the experience marker. The trap is auto-fixtures that modify behaviour (locale, viewport) when test.use is the right tool. The follow-up is often ‘how do you debug an

auto-fixture causing unexpected behaviour?', answer search for auto: true in the fixtures file; the implicit invocation is exactly what makes it hard to find.

How to phrase it

> *An auto-fixture, declared with auto true, runs for every test in scope without being explicitly requested in the test's argument list. Most fixtures only run when a test destructures them; auto-fixtures run regardless. The right use cases are narrow but valuable, and I'd give two categories. First, cross-cutting setup everyone needs. A test-id tagging fixture that annotates each test with metadata for the dashboard. An OpenTelemetry tracer that wraps every test with a span for observability. A console-error guard that fails the test if the browser console logged any error during the run, catches the class of bug where the app threw an error but the test passed because the assertion didn't depend on the broken code path. Second, cleanup that must happen regardless. A fixture that ensures temp files are cleaned even for tests that don't touch the filesystem explicitly, because forgetting to register cleanup is a common mistake. The discipline I would emphasise, because it's the rule that prevents auto-fixtures from becoming a nightmare, is auto-fixtures should be invisible. They should observe or guard test behaviour, not change it. A fixture that auto-changes the page's locale or auto-sets the viewport is the wrong use case; that's per-describe test.use territory where the test author can see the configuration in the same file. Auto-fixtures that do real work rather than observe are how surprise behaviour leaks into tests and people lose hours debugging why is this test running in French. That's also my answer to the follow-up about debugging unexpected behaviour from an auto-fixture. Search for auto true in the fixtures file. The implicit invocation, the very thing that makes auto-fixtures convenient, is exactly what makes them hard to find when they go wrong. The trap is using auto-fixtures for things that change behaviour rather than observe or guard.*

Key points to hit

(1) auto: true, runs for every test in scope without being requested. **(2)** Right uses: observation (tracer, console-error guard) and guaranteed cleanup. **(3)** Wrong uses: changing behaviour (locale, viewport), use test.use instead. **(4)** Rule: invisible, observe or guard, never change. **(5)** They usually ask next: debug by grep'ing 'auto: true', the implicit invocation is the cost. **(6)** The common miss: auto-fixtures that modify behaviour, surprise behaviour from invisible code.

CODE

```
// GOOD: console-error guard (observes; fails the test on browser error)
export const test = base.extend<{ failOnConsoleError: void }>({
failOnConsoleError: [async ({ page }, use, testInfo) => {
const errors: string[] = [];
page.on('console', msg => { if (msg.type() === 'error') errors.push(msg.text()); });
await use();
if (errors.length > 0) {
testInfo.fail(true, `Console errors:\n${errors.join('\n')}`);
}
}, { auto: true }], // <- runs for every test, no destructure needed
});

// BAD: auto-fixture that CHANGES behaviour (use test.use instead)
// localeFixture: [async ({ context }, use) => {
// await context.addInitScript(() => Object.defineProperty(navigator, 'language', {
value: 'fr-FR' }));
// await use();
// }, { auto: true }], // <- surprise! every test runs in French
```

Q 181 LEVEL · Senior

How do you handle teardown failures in fixtures so they don't mask the original test failure?

CONCEPT

Fixture teardown runs the code **after** the **await use(...)** call. If teardown throws, a cleanup API call fails, a file deletion errors, a context.close() hangs, it can **mask the original test failure** by replacing the meaningful test error with a noisy teardown error. The discipline that distinguishes a senior implementation: catch and log teardown errors rather than letting them throw, so the original failure stays visible. For each cleanup operation, wrap in try-catch, log the failure with enough context to investigate later, and continue with the remaining cleanup. The principle: one cleanup failure should not prevent other cleanups. The exception worth knowing: resource leaks that would corrupt the next test should still throw, a browser context that won't close, a database session left open, anything that would make subsequent tests unreliable. The judgement: fail loudly on resource leaks that affect subsequent tests, fail quietly on cleanup of data that no longer matters.

INTERVIEW STRATEGY

They are looking to see teardown error handling as a distinct concern. Lead with the masking problem (teardown error replaces test error), then make the try-catch-log-continue pattern the right move with one judgement exception: resource leaks that corrupt the next test still throw. The 'fail loudly on leaks, quietly on dead data' rule is the senior signal. Where this goes wrong is letting cleanup exceptions throw and lose the real failure. Expect the follow-up ‘how do you debug a swallowed cleanup error?’, answer the log with test name and entity ID gives you everything; check the test runner logs.

How to phrase it

> *Fixture teardown runs the code after the await use call. If teardown throws, a cleanup API call fails, a file deletion errors, a context dot close hangs, it can mask the original test failure by replacing the meaningful test error with a noisy teardown error. The dev opens the failure, sees DELETE returned 500, and spends an hour investigating the cleanup, only to discover the test was actually failing on a real assertion before teardown ran. The discipline that distinguishes a senior implementation, and the pattern I would lead with, is catch and log teardown errors rather than letting them throw, so the original failure stays visible. For each cleanup operation, wrap in try-catch, log the failure with enough context to investigate later, test name, entity ID, the error message, and continue with the remaining cleanup operations. The principle is one cleanup failure should not prevent other cleanups. If I have ten entities to delete and the third one's delete returns 500, the rest should still be cleaned up; otherwise one transient downstream error leaves nine orphan records. There's one exception worth knowing, and it's the judgement call that separates a sloppy implementation from a careful one: resource leaks that would corrupt the next test should still throw. A browser context that won't close, a database session left open, anything that would make subsequent tests unreliable. The judgement is fail loudly on resource leaks that affect subsequent tests; fail quietly on cleanup of data that no longer matters once this test is done. That's also my answer to the follow-up about debugging a swallowed cleanup error. The log includes test name and entity ID, so I can find it in the test runner output, the cleanup failure isn't hidden, just demoted from a test failure to a logged warning. The mistake is letting cleanup exceptions throw and lose the real failure; the dev investigates the cleanup, the real bug remains unfixed.*

Key points to hit

(1) Teardown errors mask the original test failure if uncaught. **(2)** Wrap each cleanup op in try-catch: log with context, continue. **(3)** Principle: one cleanup failure should not prevent other cleanups. **(4)** Exception: resource leaks affecting subsequent tests SHOULD still throw. **(5)** Judgement: fail loudly on leaks; fail quietly on dead-data cleanup. **(6)** Avoid: swallowed test failure because teardown threw, dev chases wrong bug.

CODE

```
export const test = base.extend<{ cleanup: Set<string>; request: APIRequestContext }>({
cleanup: async ({ request }, use, testInfo) => {
const ids = new Set<string>();
await use(ids); // - test runs here -

for (const id of ids) {
try {
await request.delete(`/api/users/${id}`);
} catch (err) {
// Log, don't throw: don't mask the original test failure
console.warn(`[cleanup] ${testInfo.title}, delete user ${id} failed:`, err);
}
// Loop continues to the next ID even if this one failed
}
},
});

// EXCEPTION, resource leak: STILL throw so it's loud
// const ctx = await browser.newContext();
// await use(ctx);
// await ctx.close(); // no try-catch: if this hangs/fails, subsequent tests are unsafe
```

Q 182 LEVEL · Senior

What is fixture scope leakage, and how do you diagnose state surviving where it shouldn't?

CONCEPT

Scope leakage is when state from one test or worker **contaminates another** because a fixture is scoped too broadly or because state was stored outside the fixture entirely. The classic cases: a **worker-scoped fixture mutating shared state** (a cached lookup object) that tests then read in the order they happened to run; a module-level variable outside any fixture (let userId; userId = …) that survives tests; a **filesystem path** that's shared across workers because it doesn't include the worker index. Diagnosis: run the suspected test in isolation, if it passes alone but fails when run after others, that's scope leakage; run tests in reverse order, if a previously-passing pair now fails, ordering dependency confirms leakage; check fixtures for worker-scoped state that's mutable. The rule that prevents leakage in the first place: worker-scoped fixtures should be immutable after setup, or expose mutations through explicit methods, not mutable shared objects. State that needs to vary per test belongs in a test-scoped fixture.

INTERVIEW STRATEGY

The point of this one is to see scope leakage diagnosed methodically. Lead with the three classic patterns (worker fixture mutating state, module-level vars, shared file paths), then the two diagnostic tests (run in isolation, run reverse order). The 'worker fixtures immutable after setup' rule is the senior signal. The mistake is mutable worker-scoped state. They usually follow with ‘how do you know it's leakage and not flake?’, answer leakage is order-dependent; flake is order-independent.

How to phrase it

> *Scope leakage is when state from one test or worker contaminates another because a fixture is scoped too broadly or because state was stored outside the fixture entirely. The three classic cases I'd describe. First, a worker-scoped fixture that mutates shared state, a cached lookup object the fixture creates once per worker, and tests then read or write in the order they happened to run. Second, a module-level variable outside any fixture, let userId, then userId equals something inside a test, that survives tests because it's hoisted out of any scope. Third, a filesystem path that's shared across workers because it doesn't include the worker index, so two workers write to slash tmp slash report dot pdf and stomp on each other. Diagnosis is methodical. Run the suspected test in isolation: if it passes alone but fails when run after others, that's scope leakage. Run tests in reverse order: if a previously-passing pair now fails, ordering dependency confirms leakage. Check fixtures for worker-scoped state that's mutable. That's also my answer to the follow-up about distinguishing leakage from flake: leakage is order-dependent. A test that passes in isolation but fails after specific other tests is leakage. A test that fails randomly regardless of order is flake. The rule that prevents leakage in the first place, and the framing I would close on, is worker-scoped fixtures should be immutable after setup. Worker fixtures set things up once; tests read but don't write to that state. If state needs to vary per test, it belongs in a test-scoped fixture, where Playwright disposes it between tests automatically. The mutable shared state is what makes order matter, and order-mattering is what produces leakage.*

Key points to hit

(1) Three patterns: worker fixture mutating state, module-level var, shared paths. **(2)** Diagnose: isolate the test, reverse the order, leakage is order-dependent. **(3)** Rule: worker-scoped fixtures immutable after setup; vary state per test via test-scope. **(4)** Module-level vars outside fixtures are NEVER cleaned up between tests. **(5)** The next question: leakage is order-dependent; flake is order-independent. **(6)** The easy mistake: mutable worker-scoped state, order-of-run determines correctness.

CODE

```
// LEAKAGE PATTERN: worker fixture with mutable shared state
export const test = base.extend<{}, { cache: Map<string, string> }>({
cache: [async ({}, use) => {
const c = new Map<string, string>(); // shared across all tests in worker
await use(c);
}, { scope: 'worker' }],
});
// test A writes cache.set('user', 'A'); test B reads cache.get('user'), sees 'A'.
// Order-dependent. Order-dependent = leakage.

// DIAGNOSE
// 1. Run the suspect test alone: npx playwright test some.spec.ts --grep "failing test"
// 2. Reverse order: --workers=1 with tests in reversed order
// 3. Inspect for mutable worker fixtures or module-level vars

// CORRECT: test-scoped state, fresh per test
// const myFixture = async ({}, use) => { const x = new Map(); await use(x); };
```

Q 183

LEVEL · Mid

How does defineConfig give you type safety, and what happens without it?

CONCEPT

defineConfig is a helper from @playwright/test that wraps the configuration object and applies the **PlaywrightTestConfig** type to its argument. Without it, TypeScript can't validate the config shape, you can write retires: 2 instead of retries: 2 and the config compiles cleanly, silently disabling the feature you thought you enabled. With defineConfig, every typo is a compile error. The pattern matters especially for project-level config inheritance, each project's use block is typed against the same interface, so options the team has agreed on get enforced. The discipline that builds on this: extract reusable config fragments into typed constants instead of duplicating across projects. Define a **baseUse: PlaywrightTestConfig['use']** object and spread it into each project's use, with project-specific overrides explicit. The framing: defineConfig is to playwright.config.ts what TypeScript itself is to test code, the type system catching mistakes before they ship.

INTERVIEW STRATEGY

You are being asked to show config type-safety valued. Lead with the silent-typo failure mode without defineConfig (retires vs retries), then the project-inheritance benefit. The 'extract fragments into typed constants' discipline is the senior signal. The risk is plain-object config that compiles with typos. The next question is normally ‘what's the worst typo you've seen?’, answer retries vs retires; the test ran without retries for months.

How to phrase it

> *defineConfig is a helper from at-playwright-slash-test that wraps the configuration object and applies the PlaywrightTestConfig type to its argument. The reason it matters, and the framing I would lead with, is silent-typo prevention. Without defineConfig, TypeScript can't validate the config shape because the object literal has no inferred type. You can write retires colon two instead of retries colon two and the config compiles cleanly, silently disabling the feature you thought you enabled. With defineConfig, every typo is a compile error and the IDE autocompletes valid options. The pattern matters especially for project-level config inheritance, because each project's use block is typed against the same PlaywrightTestConfig dot use interface, so options the team has agreed on get enforced everywhere. The discipline that builds on this, and where senior interviewers probe, is extract reusable config fragments into typed constants instead of duplicating across projects. I define a baseUse constant typed as PlaywrightTestConfig at use, and spread it into each project's use with project-specific overrides explicit. So the chromium project's use is base-use spread plus the chromium device settings, the firefox project is base-use spread plus firefox device settings, and anyone adding a new option to base-use has it applied across every project automatically. If they push on this in the follow-up, my answer is about the worst typo I've seen: retries versus retires. I joined a team where the test suite had been running without retries for months because the config had retires, with two i's instead of three, and the typo compiled. Tests were flaking and failing on first attempt because retries weren't configured, and the team thought their suite was just unreliable. The framing I close on, because it lands as the senior insight, is defineConfig is to playwright.config.ts what TypeScript itself is to test code: the type system catching mistakes before they ship. The classic error is plain-object config that compiles with typos and silently disables features.*

Key points to hit

(1) defineConfig applies PlaywrightTestConfig type, typos become compile errors. **(2)** Without it: 'retires: 2' compiles fine, silently disables retries. **(3)** Project use blocks typed against the same interface, inheritance enforced. **(4)** Extract fragments: baseUse: PlaywrightTestConfig['use'] spread per project. **(5)** Asked next: retires-vs-retries is a real incident; months without retries. **(6)** Trap: plain-object config, typos compile and silently disable features.

CODE

```
import { defineConfig, devices, PlaywrightTestConfig } from '@playwright/test';

// Reusable typed fragment: changes here propagate to every project
const baseUse: PlaywrightTestConfig['use'] = {
baseURL: process.env.BASE_URL,
trace: 'on-first-retry',
screenshot: 'only-on-failure',
testIdAttribute: 'data-test',
};

export default defineConfig({
retries: process.env.CI ? 2 : 0, // ✓ typo here = compile error
// retires: 2 // ✗ would compile WITHOUT defineConfig
projects: [
{ name: 'chromium', use: { ...baseUse, ...devices['Desktop Chrome'] } },
{ name: 'firefox', use: { ...baseUse, ...devices['Desktop Firefox'] } },
{ name: 'mobile', use: { ...baseUse, ...devices['Pixel 7'] } },
],
});
```

Q 184 LEVEL · Mid to Senior

How do you use projects to model different test data environments?

CONCEPT

Projects in playwright.config.ts aren't just for cross-browser, they're a general mechanism for running the same tests against different configurations. Useful applications include **environment matrices** (dev, staging, production-readonly with different baseURLs and storageStates), **user-role matrices** (run the same tests as admin, user, viewer to verify role-appropriate behaviour), locale matrices (run the same tests in English, French, Japanese to catch i18n regressions), and **feature-flag matrices** (project with the flag on, project with the flag off). Each project gets its own use config, storageState, and testMatch, and tests automatically run across the cartesian product. The discipline that scales: filter aggressively with --project flag in CI to avoid running the full matrix when you only care about a subset. Three roles times three locales times two flags is eighteen project runs; without filtering, CI runs all eighteen on every PR.

INTERVIEW STRATEGY

What scores here is showing projects beyond cross-browser. List the four matrix categories: environment, role, locale, feature flag. The cartesian product warning + filtering discipline demonstrates calibration. The pitfall is treating projects as browser-only and missing their

general-purpose power. A common follow-up is 'how do you avoid running the full matrix on every PR?', answer --project=chromium-admin-en on PR, full matrix on main.

How to phrase it

> *Projects in playwright.config.ts aren't just for cross-browser, they're a general mechanism for running the same tests against different configurations, and using them deliberately is one of the highest-leverage moves a senior SDET makes. Four matrix categories I'd give as concrete uses. First, environment matrices, dev, staging, production-readonly with different baseURLs and storageStates, letting the same tests verify multiple environments without duplicating code. Second, user-role matrices: run the same tests as admin, user, viewer to verify role-appropriate behaviour. The admin sees the delete button, the viewer doesn't, all three projects assert against the same test file with different storageState files. Third, locale matrices: run the same tests in English, French, Japanese to catch internationalisation regressions without writing locale-specific tests. Fourth, feature-flag matrices, a project with the flag on, a project with the flag off, testing both code paths the way S28's feature flag scenario described. Each project gets its own use config, storageState, testMatch, and tests automatically run across the cartesian product of projects configured. The discipline that scales, and the part that prevents this from becoming unmanageable, is filter aggressively with the dash-dash-project flag in CI to avoid running the full matrix when you only care about a subset. Three roles times three locales times two flags is eighteen project runs; without filtering, CI runs all eighteen on every PR, which is slow and wasteful. So I use --project flags to pick the right subset per CI tier. PR runs chromium-admin-en, main runs the role matrix, nightly runs the full cartesian product. That's also my answer to the follow-up about avoiding the full matrix on every PR. The mistake is treating projects as browser-only and missing their general-purpose power; the same mechanism that handles cross-browser handles role and locale and flag matrices with no code duplication.*

Key points to hit

(1) Projects are general: environment, role, locale, feature-flag matrices. **(2)** Same test file runs across cartesian product of projects. **(3)** Each project: own use, storageState, testMatch, no test duplication. **(4)** Filter aggressively with --project in CI: don't run full matrix on every PR. **(5)** Follow-up: PR -> chromium-admin-en; main -> role matrix; nightly -> full. **(6)** Where this goes wrong: treating projects as browser-only, missing the general-purpose power.

CODE

```
// playwright.config.ts: projects model role + locale matrix
export default defineConfig({
projects: [
{ name: 'setup', testMatch: /.*\.setup\.ts/ },
{
name: 'admin-en',
dependencies: ['setup'],
use: { storageState: 'auth/admin.json', locale: 'en-US' },
},
{
name: 'admin-fr',
dependencies: ['setup'],
use: { storageState: 'auth/admin.json', locale: 'fr-FR' },
},
{
name: 'viewer-en',
dependencies: ['setup'],
use: { storageState: 'auth/viewer.json', locale: 'en-US' },
},
// 6+ more projects as roles × locales grows
],
});

// Filter per CI tier
// PR: npx playwright test --project=admin-en
// main: npx playwright test --project='admin-en|viewer-en'
// nightly: npx playwright test # full cartesian product
```

Q 185 LEVEL · Mid

How do you configure outputDir and write parallel-safe file paths from tests?

CONCEPT

outputDir is where Playwright writes per-test artifacts (screenshots, videos, traces). It defaults to test-results/ and Playwright automatically namespaces files within it by test ID, so tests writing screenshots via toHaveScreenshot or the trace recording never collide across parallel workers. For files your test code writes directly, generated CSVs, downloaded artifacts, intermediate data, you must namespace by worker yourself, because the OS filesystem isn't isolated across contexts the way browser state is (covered in S2). The pattern: use **testInfo.outputPath('filename')** for paths Playwright manages, automatically scoped per test, or **`${testInfo.outputDir}/${testInfo.parallelIndex}/file.csv`** for explicit worker namespacing. The discipline: never write to a hardcoded path like /tmp/output.csv; parallel tests collide silently. testInfo.outputPath is the safe default for any file the test produces that you want preserved in artifacts.

INTERVIEW STRATEGY

This question is there to see parallel-safe file handling. Lead with outputDir defaults and automatic namespacing, then make testInfo.outputPath the senior pattern for tests that write files. Connect back to S2: filesystem isn't isolated by browser context. The thing to avoid is hardcoded paths like /tmp/output. The follow-up is often 'why isn't filesystem isolated by context?', answer because OS

filesystem is outside the browser process; context isolates browser state, not OS state.

How to phrase it

> *outputDir is where Playwright writes per-test artifacts, screenshots, videos, traces. It defaults to test-results slash, and Playwright automatically namespaces files within it by test ID. So tests writing screenshots via toHaveScreenshot or producing trace recordings never collide across parallel workers, each test has its own subdirectory, organised by project name and test path. That part is handled automatically by the framework. The case where I need to be careful, and where interviews probe, is files my test code writes directly, generated CSVs, downloaded artifacts I want to preserve, intermediate data files. I must namespace those by worker myself, because the OS filesystem isn't isolated across browser contexts the way browser state is. That's the connection back to context isolation from earlier in the kit: context isolates cookies, storage, cache, permissions, all the in-browser stuff, but not the filesystem, because the filesystem lives outside the browser process entirely. Two patterns. testInfo dot outputPath of a filename returns a path scoped per test automatically, Playwright manages it, my code just gets a safe path. Or testInfo.outputDir slash testInfo dot parallelIndex slash file dot csv for explicit worker namespacing when I want a worker-level shared directory instead of per-test. The discipline I would lead with, because it's where parallel tests silently break, is never write to a hardcoded path like slash tmp slash output dot csv. Two parallel tests both writing there will collide, the second one wins or both corrupt, and the failure is intermittent and hard to debug. testInfo dot outputPath is the safe default for any file the test produces. That's also my answer to the follow-up about why filesystem isn't isolated by context: because the OS filesystem is outside the browser process. BrowserContext isolates browser state, not OS state. The risk is treating filesystem the way you treat cookies, expecting isolation, you don't get it.*

Key points to hit

(1) Playwright auto-namespaces its own artifacts by test ID under outputDir. **(2)** testInfo.outputPath('file') for paths Playwright manages, per-test scope. **(3)** testInfo.parallelIndex for explicit worker-level namespacing. **(4)** Filesystem is NOT context-isolated: OS lives outside the browser process. **(5)** Expect next: context isolates browser state; filesystem stays shared. **(6)** What loses the point: hardcoded paths like /tmp/output, parallel workers collide silently.

CODE

```
test('generates a CSV report', async ({ page }, testInfo) => {
await page.goto('/reports');
const rows = await page.locator('.row').allTextContents();

// SAFE: per-test scoped path managed by Playwright
const csvPath = testInfo.outputPath('report.csv');
fs.writeFileSync(csvPath, rows.join('\n'));
await testInfo.attach('report.csv', { path: csvPath });

// ALSO SAFE: explicit worker namespacing
const sharedPath = path.join(testInfo.outputDir, `worker-${testInfo.parallelIndex}`,
'shared.csv');
fs.mkdirSync(path.dirname(sharedPath), { recursive: true });
fs.writeFileSync(sharedPath, '...');
});

// WRONG: silent collision under parallel execution
// fs.writeFileSync('/tmp/output.csv', data);
```

Q 186 LEVEL · Senior

What are the override semantics of test.use across config, project, describe, and test scopes?

CONCEPT

test.use options resolve in a precise hierarchy, with more specific scopes overriding more general ones. The order from least to most specific: **1. config-level use** (the top-level use in playwright.config.ts, applies to all projects); **2. project-level use** (overrides config-level for that project); **3. describe-block test.use** (overrides project-level for tests inside that describe); 4. per-test test.use (overrides describe-level for that single test). Options are merged shallowly: setting viewport at describe level replaces the viewport entirely, not merges fields, so deep nested options like storageState options have to be redeclared in full when overriding. The discipline: put the broadest setting at the config level, override progressively as scope narrows. A test that needs a unique configuration uses per-test test.use; a describe of related tests sharing a config uses describe-level. The trap: test.use inside a test function body, it must be at describe-level or above to apply, never inside an individual test.

INTERVIEW STRATEGY

They want to see the resolution hierarchy exactly. List the four scopes in order of specificity (config -> project -> describe -> per-test), then the shallow-merge detail. The 'test.use inside a test body doesn't apply' trap is the test of seniority on this because it's where bugs hide. Expect the follow-up ‘what if I call test.use inside a test?’, answer it's an error, test.use must be at describe level or higher.

How to phrase it

> *test.use options resolve in a precise hierarchy, with more specific scopes overriding more general ones, and knowing the exact order is what stops "why-isn't-my-setting-applying" debugging. Four levels from least to most specific. Level one, config-level use, the top-level use in playwright.config.ts, applies to all projects by default. Level two, project-level use, inside a project definition, overrides config-level for that specific project. Level three, describe-block test.use inside a test.describe block, overrides project-level for tests inside that describe. Level four, per-test test.use inside an individual test or in a test.describe with a single test, overrides describe-level for that single test. Options are merged shallowly, which is the detail worth knowing because it surprises people. Setting viewport at describe level replaces the viewport entirely, not merges fields, so deep nested options like storageState options have to be redeclared in full when overriding. If I have storageState set with options at config level and want to add an option at describe level, I have to repeat all the existing storageState options or they're lost. The discipline that lands as the senior approach is put the broadest setting at the config level and override progressively as scope narrows. A test that needs a unique configuration uses per-test test.use. A describe of related tests sharing a configuration uses describe-level. A whole project uses project-level. The framework gives me a ladder; I use the rung that matches the scope of the change. The trap, and my answer to the follow-up about calling test.use inside a test, is test.use inside a test function body does not apply. It must be at describe level or above to take effect; Playwright now errors at lint or runtime if you try, but in older versions the call silently did nothing and the developer wondered why the test ran with default config. Always outside the test, always at describe level or higher.*

Key points to hit

(1) 4 levels (specific wins): config -> project -> describe -> per-test. **(2)** Options merge SHALLOWLY: nested fields don't merge, full redeclare needed. **(3)** Place the broadest setting at config; override progressively as scope narrows. **(4)** test.use must be at describe-level or above: inside a test it doesn't apply. **(5)** They usually ask next: test.use inside a test body errors (or silently fails in older versions). **(6)** The common miss: expecting nested options to merge, they don't; redeclare in full.

CODE

```
// 1. Config-level (top-level use): applies to all projects
export default defineConfig({
use: { viewport: { width: 1280, height: 720 }, locale: 'en-US' },
projects: [
// 2. Project-level: overrides config-level for this project
{ name: 'mobile', use: { viewport: { width: 390, height: 844 } } },
],
});

// 3. Describe-level: overrides project for tests inside
test.describe('French locale block', () => {
test.use({ locale: 'fr-FR' });
test('renders in French', async ({ page }) => { /* ... */ });
});

// 4. Per-test: overrides describe-level for this one test
test('this one needs a custom viewport', async ({ page }) => {
// test.use({ viewport: { width: 800 } }); // ✗ WRONG, must be outside the test
});
test.use({ viewport: { width: 800, height: 600 } }); // ✓ applies to next test
```

Q 187

LEVEL · Mid to Senior

globalSetup/globalTeardown vs project dependencies (setup projects), which do you use for auth and one-time setup, and why are setup projects usually better?

CONCEPT

Both run setup once before the suite, but they sit in different worlds. **globalSetup** is a single function in playwright.config.ts that runs outside the test runner before anything else, with a paired **globalTeardown** after. Because it runs outside the runner it gets no fixtures, no test context, no built-in tracing or reporting, you drive a browser by hand with chromium.launch, and a failure shows up as a raw stack trace, not a test in the report. Project dependencies (a 'setup project') instead model setup as a real test: you define a project matched to *.setup.ts and list it under **dependencies** for the main projects, so it runs first as an ordinary test. Setup projects are usually better for auth and one-time setup because they run inside the runner, full fixtures (page, request), traces and screenshots on failure, a line in the HTML report, and retries if the login flow is flaky. They also compose: different main projects can depend on different setup projects (admin vs viewer auth). The narrow case where globalSetup still fits: environment-level work that has nothing to do with a browser, seeding a database, starting a mock server, setting process-wide env vars.

INTERVIEW STRATEGY

This is a contrast question, so answer it as a contrast: both do once-before-everything setup, but globalSetup runs outside the runner and a setup project runs inside it as a real test. Lead with that one sentence, then let everything else fall out of it, fixtures, traces, report line, retries all come for free inside the runner and are all missing outside it. Say crisply that for auth you reach for a setup project, then concede the narrow globalSetup case (non-browser environment work) so you sound balanced rather than dogmatic. The trap to avoid is reaching for globalSetup to do the login, you lose tracing exactly where you most want it. They usually follow with 'so is globalSetup ever right?',

answer yes, for browser-less environment setup like seeding a database or starting a mock server.

How to phrase it

> *Both run once before the suite, so I frame the difference in one line: globalSetup is a function that runs outside the test runner, whereas a project dependency, a setup project, runs inside the runner as an ordinary test. Everything that matters follows from that. globalSetup lives in the config, runs before the runner spins up, and has no fixtures, no test context, no automatic tracing or reporting. If I want to log in there I'm launching a browser by hand with chromium.launch and saving storage state myself, and if that login fails I get a raw stack trace in the console, not a failed test in the report, which is painful because auth setup is exactly the flow I most want a trace for when it breaks. A setup project flips all of that. I define a project matched to dot-setup-dot-ts files and list it under dependencies for my main projects, so Playwright runs it first as a real test. That means I get the page and request fixtures, traces and screenshots on failure, a line in the HTML report, and retries if the login flow is occasionally flaky, none of which I'd have in globalSetup. It also composes: my admin project can depend on an admin-auth setup project and my viewer project on a viewer-auth one, each writing its own storage state. So for auth and one-time setup my default is a setup project, almost always. The follow-up is usually whether globalSetup is ever the right tool, and yes, narrowly, for environment-level work that has nothing to do with a browser. seeding a database, starting a mock server, setting process-wide env vars before any worker boots. That's genuinely global and browser-less, so the runner's fixtures and tracing buy me nothing, and globalSetup is the simpler home. But the moment the setup involves driving a browser, and auth always does, I move it into a setup project so I get the report line and the trace when it fails. The trap is doing the login inside globalSetup out of habit and then having no trace the day it breaks.*

Key points to hit

(1) One-line contrast: globalSetup runs OUTSIDE the runner; setup project runs INSIDE it as a real test. **(2)** globalSetup: no fixtures, no trace, no report line, failures are raw stack traces. **(3)** Setup project: page/request fixtures, traces + screenshots on failure, report line, retries. **(4)** Setup projects compose: different main projects depend on different auth setups (admin vs viewer). **(5)** Default for auth/one-time browser setup: a setup project via dependencies. **(6)** The next question: globalSetup still fits browser-less env work, seed a DB, start a mock server, set env vars. **(7)** Avoid: doing the login in globalSetup, no trace the day auth breaks.

CODE

```
// == OPTION A: globalSetup (outside the runner) ==
// playwright.config.ts
export default defineConfig({ globalSetup: require.resolve('./global-setup') });

// global-setup.ts: no fixtures, no trace; drive the browser by hand
async function globalSetup() {
const browser = await chromium.launch();
const page = await browser.newPage();
await page.goto('https://app/login'); /* ...sign in... */
await page.context().storageState({ path: 'auth/admin.json' });
await browser.close(); // a failure here = raw stack trace, no report line
}
export default globalSetup;

// == OPTION B (usually better): setup project via dependencies ==
// auth.setup.ts, a REAL test: gets fixtures, trace, retries, a report line
setup('authenticate as admin', async ({ page }) => {
await page.goto('/login'); /* ...sign in... */
await page.context().storageState({ path: 'auth/admin.json' });
});

// playwright.config.ts
export default defineConfig({
projects: [
{ name: 'setup', testMatch: /.*\.setup\.ts/ },
{ name: 'admin', dependencies: ['setup'], use: { storageState: 'auth/admin.json' } },
],
});
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. How does Playwright run fixture teardown when a test throws?

A) Teardown is skipped on failure
B) Teardown runs in setup order
C) Only the last fixture tears down
D) Teardown still runs, in reverse dependency order (try/finally semantics)

2. When is a worker-scoped fixture the right choice?

A) For data that must be fresh per test
B) Never: it breaks isolation
C) Only for the Page object
D) For expensive, stateful setup (e.g. an authenticated context) shared across a worker's tests

3. What is the decisive advantage of fixtures over beforeAll/afterAll?

A) Fixtures run faster
B) beforeAll cannot do setup
C) Teardown is guaranteed even if setup throws, and fixtures compose across files
D) Fixtures avoid using the API

4. What is test.use() the correct tool for?

A) Overriding config (storageState, viewport, locale) per test or describe block
B) Changing the global config file
C) Importing fixtures
D) Running tests in serial

5. How does an environment fixture keep destructive tests out of production?

A) It blocks all network calls
B) It exposes the environment so tests call test.skip() when it is production
C) It deletes production data first
D) It runs only read-only tests automatically

Section 13 · Answer key

#	Answer	Why and where to revisit
1	D) Teardown still runs, in reverse dependency order (try/finally semantics)	The yield pattern gives try/finally semantics: teardown runs even on throw, in reverse dependency order. *Revisit Section 13.*
2	D) For expensive, stateful setup (e.g. an authenticated context) shared across a worker's tests	Worker scope creates expensive setup once per worker; keep the Page function-scoped to preserve isolation. *Revisit Section 13.*
3	C) Teardown is guaranteed even if setup throws, and fixtures compose across files	Fixture teardown always runs (unlike afterAll if beforeAll fails), and fixtures are reusable across files. *Revisit Section 13.*
4	A) Overriding config (storageState, viewport, locale) per test or describe block	test.use overrides config for a scope without touching global config, e.g. storageState per role-based block. *Revisit Section 13.*
5	B) It exposes the environment so tests call test.skip() when it is production	The fixture surfaces the environment; destructive tests skip on production via an explicit, reviewable guard. *Revisit Section 13.*

SECTION 14

Advanced Playwright Patterns

These patterns are what interviewers look for when they want a framework designer, not just a test writer. They cover the Builder pattern for readable test data, a generic retry utility for flaky external dependencies, a cleanup registry that prevents orphaned data, Playwright Component Testing for the gap between unit and E2E, and a page object built for an SPA with dynamic routing and query-parameter filters.

In this section

- The Builder pattern: fluent test data where tests state only what matters.
- A generic withRetry utility with exponential backoff for external systems.
- A cleanup registry that deletes resources even on partial-setup failure.
- Component Testing: real rendering and CSS without the full app stack.
- SPA page objects: URLSearchParams filters and waiting for async content.

Q 188 LEVEL · Mid

How do you implement the Builder pattern for test data?

CONCEPT

The Builder pattern creates complex test-data objects with a fluent API, so a test specifies only the fields relevant to its scenario. A UserBuilder seeds sensible defaults with faker, then exposes chainable methods, withRole, asAdmin, asViewer, each returning this, plus a **build()** for the plain object and a **create()** that posts it via the API. The win is intent: instead of ten lines of faker setup per test, a test reads **new UserBuilder().asAdmin().create(request)**, which communicates 'I need an admin' without naming the admin's phone or address. And when the User API adds a required field, you update the builder once and every test stays correct. It is the test-data analogue of deriving from a single source of truth.

INTERVIEW STRATEGY

Your job here is to show you reduce test noise and centralise data. Sell the Builder on intent, a one-line, self-describing setup versus ten lines of faker, and on the single-edit maintainability when the schema changes. The risk is duplicating data setup across every test. The next question is normally 'what happens when the user schema changes?', answer you update the builder once and all tests stay correct, the same single-source-of-truth idea as typed contracts.

How to phrase it

> *The Builder pattern is the most impactful test-data pattern I have implemented, so I would say that up front. It creates complex data objects with a fluent API: a UserBuilder seeds sensible defaults with faker, then exposes chainable methods like withRole, asAdmin and asViewer that each return this, plus a build for the plain object and a create that posts it through the API. The reason it matters is intent. Before the builder, every test had about ten lines of data setup with faker calls cluttering the top. After it, a test reads new UserBuilder asAdmin create request, one line that says exactly what the test needs, an admin user, without specifying irrelevant details like the admin's name or phone number, so the reader instantly sees what scenario is being exercised. That answers the follow-up about what happens when the user schema changes: because all construction goes through the builder, when the User API adds a new required field I update UserBuilder once and every test stays correct, instead of editing dozens of faker blocks. It is the test-data version of deriving everything from a single source of truth, the same principle I apply to typed API contracts. So the builder buys me readable, intent-revealing tests and one-place maintenance, which is exactly what keeps a large suite from drowning in setup noise.*

Key points to hit

(1) Fluent API: chainable methods returning this, build() and create(). **(2)** faker seeds defaults; tests override only what matters. **(3)** One line communicates intent ('I need an admin') vs ten lines of setup. **(4)** Schema change -> update the builder once, all tests stay correct. **(5)** Asked next: single source of truth for test data, like typed contracts. **(6)** The easy mistake: duplicating faker data setup across every test.

CODE

```
import { faker } from '@faker-js/faker';

export class UserBuilder {
private user = { name: faker.person.fullName(), email: faker.internet.email(), role:
'user' };
withRole(role: string): this { this.user.role = role; return this; }
asAdmin(): this { this.user.role = 'admin'; return this; }
asViewer(): this { this.user.role = 'viewer'; return this; }
build() { return { ...this.user }; }
async create(request: APIRequestContext) {
return (await request.post('/api/users', { data: this.user })).json();
}
}

// Tests state only what matters
const admin = await new UserBuilder().asAdmin().create(request);
const viewer = await new UserBuilder().asViewer().create(request);
```

Q 189 LEVEL · Mid

How do you implement retry logic for unstable external dependencies?

CONCEPT

A generic **withRetry** utility wraps an operation that can fail **transiently**, an external API call, email verification, an SMS code, with configurable **attempts**, delay, and exponential backoff, plus an optional onRetry callback for logging. The important distinction: this is not the same as Playwright's built-in test retries. Playwright retries handle a whole-test failure; withRetry handles a specific operation inside a test that talks to an eventually consistent external system. Those systems have inherent latency and occasional failures that are not bugs in the application under test, so retrying the operation with backoff is correct, whereas failing the test or sleeping a fixed time would be wrong, the first is a false negative, the second is slow and still flaky.

INTERVIEW STRATEGY

The question is checking whether you distinguish test-level retries from operation-level ones. Make that distinction the spine of the answer: Playwright retries the whole test, withRetry retries one external operation. Ground it in email/SMS/webhook latency that is not an app bug. The pitfall is conflating the two or using a fixed sleep. A common follow-up is ‘why not just use Playwright retries?’, answer that they re-run the entire test, not the single flaky external call, so they are too coarse for this.

How to phrase it

> *withRetry is a generic utility that wraps an operation which can fail transiently, an external API call, fetching an email verification code, an SMS code, with configurable attempts, an initial delay, exponential backoff, and an optional onRetry callback for logging. The distinction I lead with, because it is what the question is really testing, is that this is not Playwright's built-in test retries. Playwright retries re-run the whole test on failure; withRetry retries a specific operation inside a test. That difference is also my answer to the follow-up about why not just use Playwright retries: they are too coarse here, because re-running the entire test to get past one flaky external call wastes everything else the test did and is far slower. These external systems, email services, SMS gateways, payment webhooks, have inherent latency and occasional hiccups that are not bugs in the application I am testing, so I do not want them to fail my test or to dictate a long fixed sleep. A fixed sleep is either too short and still flaky or too long and slow on every run; withRetry with exponential backoff returns as soon as the operation succeeds and only waits longer when it genuinely needs to. So I reach for withRetry for eventually consistent external dependencies, and I keep Playwright's retries for genuine whole-test flakiness, which are two different problems with two different tools.*

Key points to hit

(1) withRetry wraps a transient operation with attempts, delay, backoff. **(2)** Not the same as Playwright test retries: operation-level vs whole-test. **(3)** For external systems whose latency/failures aren't app bugs. **(4)** Exponential backoff returns on success; fixed sleep is flaky or slow. **(5)** Follow-up: Playwright retries re-run the whole test, too coarse here. **(6)** Trap: conflating the two retry levels or using a fixed sleep.

CODE

```
export async function withRetry<T>(
operation: () => Promise<T>,
{ attempts = 3, delayMs = 1000, backoff = 2, onRetry }: {
attempts?: number; delayMs?: number; backoff?: number;
onRetry?: (attempt: number, error: Error) => void;
} = {},
): Promise<T> {
for (let attempt = 1; attempt <= attempts; attempt++) {
try { return await operation(); }
catch (error) {
if (attempt === attempts) throw error;
onRetry?.(attempt, error as Error);
await new Promise(r => setTimeout(r, delayMs * backoff ** (attempt - 1)));
}
}
throw new Error('unreachable');
}

const code = await withRetry(() => emailClient.getLatestCode('user@test.com'), {
attempts: 5, delayMs: 2000 });
```

Q 190 LEVEL · Mid

How do you implement a test data cleanup strategy?

CONCEPT

A **cleanup registry** fixture collects deletion functions as resources are created during a test, then runs them all in teardown **in reverse order**, even if the test fails partway through setup. The fixture provides a **cleanup(fn)** function; each time the test creates a resource it registers the matching delete. If the test fails after creating a user but before creating an order, only the user deletion runs, nothing is orphaned. A try/catch around each deletion ensures one failed cleanup does not block the rest. This solves the orphaned-data problem: without it, a test that dies mid-setup leaves partial records that accumulate and cause cross-test interference over time, the kind of slow rot that makes a suite flaky months later.

INTERVIEW STRATEGY

This one probes whether you keep the test environment clean under failure. Describe the registry that collects deletes and runs them in reverse, stressing it works even on partial-setup failure. The try/catch per deletion is the robustness detail. The trap is cleanup that only runs on success, leaving orphans when a test dies mid-setup. The follow-up is often 'what happens if the test fails halfway through creating data?', answer only the resources created so far are cleaned up, because each was registered as it was made.

How to phrase it

> *I use a cleanup registry fixture that solves the orphaned-test-data problem. The fixture hands the test a cleanup function, and every time the test creates a resource it immediately registers the matching deletion, create a user, register the user delete; create an order, register the order delete. In teardown the fixture runs all the registered deletions in reverse order. The key property, and my answer to the follow-up about a test failing halfway through creating data, is that because each delete is registered the instant its resource is created, if the test dies after creating the user but before creating the order, only the user deletion runs, there is no order to delete and nothing is left orphaned. I wrap each deletion in its own try/catch so that one failed cleanup, maybe a resource already gone, does not block the others from running. The reason this matters is the slow rot it prevents: without a registry, a test that fails mid-setup leaves partial records in the database, those accumulate over weeks, and eventually they cause cross-test interference that shows up as mysterious flakiness months later, long after the test that created the mess. The classic error is cleanup that only runs on the happy path. The registry makes cleanup track creation exactly, so the environment stays clean no matter how a test ends.*

Key points to hit

(1) Registry collects delete fns as resources are created; runs in reverse. **(2)** Works even on partial-setup failure: only created resources are cleaned. **(3)** try/catch per deletion so one failure doesn't block the rest. **(4)** Prevents orphaned data that accumulates into cross-test interference. **(5)** Expect next: fail mid-setup -> only resources created so far are deleted. **(6)** Where this goes wrong: cleanup that only runs on the happy path.

CODE

```
export const test = base.extend<{ cleanup: (fn: () => Promise<void>) => void }>({
cleanup: async ({}, use) => {
const fns: Array<() => Promise<void>> = [];
await use(fn => fns.push(fn));
for (const fn of fns.reverse()) {
try { await fn(); } catch (e) { console.warn('Cleanup failed:', e); }
}
},
});

test('complex setup with cleanup', async ({ request, cleanup }) => {
const user = await (await request.post('/api/users', { data: {} })).json();
cleanup(() => request.delete(`/api/users/${user.id}`));
const order = await (await request.post('/api/orders', { data: { userId: user.id }
})).json();
cleanup(() => request.delete(`/api/orders/${order.id}`));
// if the test fails here, both deletions still run in reverse
});
```

Q 191 LEVEL · Mid to Senior

How do you implement visual component testing with Playwright?

CONCEPT

Playwright **Component Testing** (@playwright/experimental-ct-react, and Vue/Svelte variants) mounts a component directly in a real browser without the full application, so tests are fast and isolated. You **mount** the component with props, then assert on rendering and interaction with the normal Playwright API, text content, classes, click handlers. The value is that it fills the gap between unit and E2E: a unit test mocks the DOM and cannot catch CSS or rendering bugs, while an E2E test needs the whole application stack; component testing gives real rendering, real CSS, real event handling with none of the full-app overhead. It is ideal for **design-system components** where both visual correctness and interaction behaviour matter and you want to test them in isolation.

INTERVIEW STRATEGY

What they want is to know if you understand the testing pyramid's middle layer. Frame component testing as the gap-filler between unit and E2E, and be specific about what each neighbour misses, unit mocks the DOM, E2E needs the whole stack. The design-system use case shows real application. The thing to avoid is positioning it as a replacement for E2E. Expect the follow-up ‘when do you use it over E2E?’, answer for isolated component rendering and interaction, not for full user journeys that span the app.

How to phrase it

> *Playwright Component Testing mounts a component directly in a real browser without the full application, so I get fast, isolated component tests. I mount the component with its props and then assert with the normal Playwright API, that it renders the right text, has the right class, fires its onClick when clicked. The way I frame its value, which is what the interviewer is really after, is that it fills the gap between unit tests and end-to-end tests. A unit test mocks the DOM, so it cannot catch a CSS or rendering bug, the logic passes but the button is invisible. An E2E test catches that but needs the entire application stack running, which is slow and heavyweight for checking one component. Component testing sits in between: real rendering, real CSS, real event handling, but no full-app overhead. That answers the follow-up about when I use it over E2E: I use it for isolated component correctness, especially design-system components like buttons and form controls where both the visual rendering and the interaction behaviour are critical and I want to test them in isolation from the rest of the app. I do not position it as a replacement for E2E, that is the trap, because E2E still owns the full user journeys that span multiple pages and the real backend. Component tests verify the pieces; E2E verifies they work together. Using both, each for what it is best at, is the point.*

Key points to hit

(1) Mounts a component in a real browser without the full app. **(2)** Assert rendering and interaction with the normal Playwright API. **(3)** Fills the unit-to-E2E gap: unit mocks the DOM, E2E needs the whole stack. **(4)** Real rendering/CSS/events without full-app overhead. **(5)** They usually ask next: use it for isolated components (design system), not full journeys. **(6)** What loses the point: positioning it as a replacement for E2E.

CODE

```
// playwright-ct.config.ts
import { defineConfig } from '@playwright/experimental-ct-react';
export default defineConfig({ testDir: './src', use: { ctPort: 3100 } });

// Button.spec.tsx
import { test, expect } from '@playwright/experimental-ct-react';
import { Button } from './Button';

test('renders with label and variant', async ({ mount }) => {
const component = await mount(<Button label="Submit Order" variant="primary" />);
await expect(component).toContainText('Submit Order');
await expect(component).toHaveClass(/btn-primary/);
});

test('calls onClick when clicked', async ({ mount }) => {
let clicked = false;
const component = await mount(<Button label="Click" onClick={() => { clicked = true; }}
/>);
await component.click();
expect(clicked).toBe(true);
});
```

Q 192

LEVEL · Mid to Senior

How do you implement a Page Object for a complex SPA with dynamic routing?

CONCEPT

SPAs with dynamic routing need page objects that handle URL patterns, query parameters, and async content loading. For parameterised routes, navigate to the templated path and **waitForURL** on the pattern, then wait for a content element. For filters, build the query string with **URLSearchParams** from a typed options object, category, price range, sort, so the test calls **navigateWithFilters({ category, sort })** and the page object owns the URL construction. The detail that makes SPA page objects correct: the URL changes immediately but the content loads asynchronously, so after navigation you must wait for the real content (the product grid visible) before proceeding. Skip that wait and the test races the data load, the classic SPA flakiness. Centralising URL construction also means a routing change is a one-method edit.

INTERVIEW STRATEGY

Show you handle SPA-specific issues, not generic POM. Lead with two specifics: typed URLSearchParams filter construction in the page object, and waiting for real content because the URL changes before the data loads. That async-content wait is the SPA-flakiness insight. The pitfall is asserting right after navigation, racing the data load. They usually follow with 'why wait after the URL changes?', answer that in an SPA the route updates instantly but content renders asynchronously, so you wait on a content element.

How to phrase it

> *For an SPA with dynamic routing I build page objects that handle URL patterns, query parameters and asynchronous content. Two specifics are what I would highlight, because generic POM does not cover them. First, filters: I construct the query string with URLSearchParams from a typed options object, category, min and max price, sort, so a test calls navigateWithFilters with category laptops and sort price, and the page object owns building the URL. That is type-safe and readable, and if the URL structure changes I update one method instead of every test that filters. Second, and this is the SPA-specific correctness point, the URL changes immediately on navigation but the content loads asynchronously, so after navigating I wait for the real content, the product grid visible, the heading rendered, before the test proceeds. That answers the follow-up about why I wait after the URL already changed: in an SPA the route updates instantly client-side but the data fetch and render happen after, so if I assert right away I am racing the data load, which is the classic SPA flakiness. Waiting for a content element rather than the URL alone makes it deterministic. So the SPA page object centralises URL construction for maintainability and always waits on rendered content rather than the route change, which is what keeps tests against single-page apps reliable.*

Key points to hit

(1) Handle URL patterns, query params, and async content in the page object. **(2)** Build filter query strings with typed URLSearchParams, one-method edits. **(3)** URL changes instantly; content loads async: wait for real content. **(4)** waitForURL on the pattern, then wait for a content element. **(5)** The next question: why wait after the URL changes, SPA renders data asynchronously. **(6)** The common miss: asserting right after navigation, racing the data load.

CODE

```
export class ProductPage extends BasePage {
readonly productTitle = this.page.getByRole('heading', { level: 1 });

async navigateToProduct(productId: string): Promise<void> {
await this.page.goto(`/products/${productId}`);
await this.page.waitForURL(`**/products/${productId}`);
await this.productTitle.waitFor({ state: 'visible' }); // content loads async
}

async navigateWithFilters(f: { category?: string; sort?: 'price' | 'rating' | 'newest'
}): Promise<void> {
const params = new URLSearchParams();
if (f.category) params.set('category', f.category);
if (f.sort) params.set('sort', f.sort);
await this.page.goto(`/products?${params.toString()}`);
await this.page.locator('#product-grid').waitFor({ state: 'visible' });
}
}
```

Q 193 LEVEL · Mid to Senior

What's the difference between Object Mother, Builder, and Factory patterns for test data?

CONCEPT

Three creational patterns for test data, each with a different sweet spot. Object Mother: a registry of named pre-configured instances, **UserMother.aValidAdmin()**, **UserMother.aLockedOutCustomer()**, each method returns a fully-specified domain object for a specific scenario. Strength: tests read at the scenario level (“a valid admin”), and Object Mother centralises domain knowledge about what “valid” means. Weakness: explodes combinatorially when you need many variations, add a tenant dimension, a role dimension, a status dimension, and you need dozens of methods. Builder: a fluent interface that constructs incrementally, **new UserBuilder().asAdmin().withEmail('x').build()**. Strength: composes well for variation; the test specifies only the dimensions that matter and sensible defaults handle the rest. Weakness: tests can become noisy when many dimensions need configuring. Factory function: a function with an options bag, createUser({ role: 'admin', email: 'x' }). Strength: simplest, least machinery, fine for small data shapes. Weakness: no method-chaining clarity for complex objects. Rule: factory for small/flat data, builder for complex multi-dimensional objects, Object Mother layered on top for the handful of named canonical scenarios.

INTERVIEW STRATEGY

What earns the tick is creational-pattern fluency. Walk the three (Object Mother = named scenarios, Builder = fluent composition, Factory = options bag) with the strength and weakness of each. The 'factory for small, builder for complex, Object Mother layered on top' rule is the senior signal. The thing to avoid is using Object Mother exclusively and exploding combinatorially. The next question is normally ‘which do you actually use most?’, answer Builder + a small Object Mother facade for canonical cases.

How to phrase it

> *Three creational patterns for test data, each with a different sweet spot, and the senior answer combines them rather than picking one tribe. Object Mother is a registry of named pre-configured instances. UserMother dot aValidAdmin, UserMother dot aLockedOutCustomer, UserMother dot aTrialAccountNearExpiry. Each method returns a fully-specified domain object for a specific scenario. The strength, and what makes it appealing, is that tests read at the scenario level, a valid admin, a locked-out customer, and Object Mother centralises domain knowledge about what valid actually means in this system. The weakness, and where teams using only Object Mother lose the war, is it explodes combinatorially when you need many variations. Add a tenant dimension, a role dimension, a status dimension, and you need dozens of methods named in different combinations. aValidAdminInTenantAOnTrial, aValidAdminInTenantBProduction, and so on. Builder is a fluent interface that constructs incrementally. New UserBuilder dot asAdmin dot withEmail x dot build. The strength is composition: it scales with variation because the test specifies only the dimensions that matter and sensible defaults handle the rest. The weakness is tests can become noisy when many dimensions need configuring, a builder call with eight chained methods is harder to read than a Mother call with one. Factory function is the simplest: a function with an options bag, createUser open-brace role colon admin email colon x close-brace. Strength: least machinery, fine for small data shapes. Weakness: no method-chaining clarity for complex objects. The rule I would close on, and my answer to the follow-up about what I use most, is factory for small or flat data, builder for complex multi-dimensional objects, Object Mother layered on top for the handful of named canonical scenarios. UserBuilder is the engine; UserMother dot aValidAdmin is just a one-line wrapper that calls UserBuilder with the right defaults. Tests get scenario naming when they want it and full builder composition when they need it, without combinatorial explosion or boilerplate. The mistake is using Object Mother exclusively and watching the named-scenario list grow until nobody can navigate it.*

Key points to hit

(1) Object Mother: named pre-configured instances, reads as scenarios. **(2)** Builder: fluent composition for multi-dimensional variation. **(3)** Factory function: simplest, options-bag, fine for small/flat data. **(4)** Object Mother explodes combinatorially with many dimensions. **(5)** Rule: factory for small, builder for complex, Mother for canonical scenarios. **(6)** Avoid: Object Mother only, named-scenario list grows until unnavigable.

CODE

```
// FACTORY: simplest, fine for small flat data
export function createUser(opts: Partial<UserData> = {}): UserData {
return { email: faker.internet.email(), role: 'user', ...opts };
}

// BUILDER: fluent composition for complex objects
export class UserBuilder {
private data: Partial<UserData> = {};
asAdmin() { this.data.role = 'admin'; return this; }
withEmail(email: string) { this.data.email = email; return this; }
inTenant(tenantId: string) { this.data.tenantId = tenantId; return this; }
build(): UserData { return { ...defaults, ...this.data }; }
}

// OBJECT MOTHER: named canonical scenarios layered on builder
export const UserMother = {
aValidAdmin: () => new UserBuilder().asAdmin().build(),
aLockedOutCustomer: () => new UserBuilder().withStatus('locked').build(),
aTrialNearExpiry: () => new UserBuilder().onTrial().withTrialDaysRemaining(1).build(),
};

// Tests pick the right level of abstraction:
// const adminA = UserMother.aValidAdmin(); // canonical scenario
// const customA = new UserBuilder().asAdmin().inTenant('acme').build(); // custom
```

Q 194 LEVEL · Senior

What's the Meszaros taxonomy of test doubles, and why does precise vocabulary matter?

CONCEPT

Gerard Meszaros's **xUnit Test Patterns** defined five distinct kinds of test doubles, and using the right term changes how teams talk about test design. **Dummy**: an object passed but never used, fills a parameter slot so the code compiles. **Stub**: returns canned answers to calls made during the test, “when asked for the user, return this user”. Has state but no behaviour verification. Spy: a stub that records the calls made to it, you can assert later that the function was called with specific arguments. Mock: pre-programmed with expectations about how it will be called, fails the test if those expectations aren't met. The verification is built into the mock itself. Fake: a working implementation that takes shortcuts, an in-memory database instead of PostgreSQL, a localhost SMTP that captures emails instead of sending. Functionally correct but not production-grade. The discipline: most teams say “mock” for everything, which loses the distinction between behaviour verification and state stubbing. Saying “we stub the auth service” versus “we mock the email send” communicates different intent: the auth response shape matters, the email-send call happening matters.

INTERVIEW STRATEGY

You need to give Meszaros taxonomy fluency. Walk all five (Dummy, Stub, Spy, Mock, Fake) with one-line distinctions. The 'stub-for-state, mock-for-behaviour-verification' precision is the senior signal. The 'most teams say mock for everything' framing shows you know the common loss of distinction. The classic error is treating mock as a generic term. A common follow-up is ‘give an

example of each?', answer the auth response stub vs email-send mock contrast lands well.

How to phrase it

> *Gerard Meszaros's xUnit Test Patterns defined five distinct kinds of test doubles, and using the right term changes how teams talk about test design. Dummy: an object passed but never used. Fills a parameter slot so the code compiles: useful when a function requires a logger argument but the test path being exercised never calls log. Stub: returns canned answers to calls made during the test. When asked for the user, return this user. Has state but no behaviour verification. This is what most route handlers in Playwright actually are, they stub the response. Spy: a stub that records the calls made to it. I can assert later that the function was called with specific arguments. Useful when I care both that the call happened and what it received. Mock: pre-programmed with expectations about how it will be called, fails the test if those expectations aren't met. The verification is built into the mock itself, declared upfront. Fake: a working implementation that takes shortcuts, an in-memory database instead of PostgreSQL, a localhost SMTP that captures emails instead of sending. Functionally correct but not production-grade. The discipline that lands in interviews, and the framing I would emphasise, is most teams say mock for everything, which loses the distinction between behaviour verification and state stubbing. Saying "we stub the auth service" versus "we mock the email send" communicates different intent: the auth response shape matters, we're just controlling what the auth service returns. The email-send call happening matters: we're verifying that our code did invoke the email send. Two different test designs hiding behind the word mock. That's also my answer to the follow-up about examples. Dummy: a logger parameter the test path doesn't exercise. Stub: page route returning a synthetic API response. Spy: a wrapped function recording calls so I can assert on arguments later. Mock: an expectation that the email service is called once with specific to-address. Fake: a Docker container running PostgreSQL in tmpfs for fast tests. The trap is treating mock as a generic term. Once you have the vocabulary, design discussions become more precise: we don't need to mock this, just stub it; we want a spy here, not a mock, because we want to assert flexibly.*

Key points to hit

(1) Dummy: passed but never used, fills parameter slots. **(2)** Stub: returns canned answers, state, no behaviour verification. **(3)** Spy: stub + records calls, assert arguments after the fact. **(4)** Mock: pre-programmed expectations, fails test if not called as expected. **(5)** Fake: working impl with shortcuts, in-memory DB, localhost SMTP. **(6)** Asked next: stub the auth response (state) vs mock the email send (behaviour).

CODE

```
// STUB: controls returned state, no verification of call
await page.route('**/api/user', route =>
route.fulfill({ status: 200, body: JSON.stringify({ id: 1, role: 'admin' }) }));

// SPY: records the calls made to it; assert on them after the test action
const calls: string[] = [];
await page.route('**/api/audit', route => {
calls.push(route.request().postData() ?? '');
route.fulfill({ status: 200 });
});
// ... perform action ...
expect(calls).toHaveLength(1);
expect(JSON.parse(calls[0]!).event).toBe('user_login');

// MOCK: expectation that the call happens with specific shape (sinon-style)
// emailService.expects('send').once().withArgs(sinon.match({ to: 'anna@bank.com' }));

// FAKE: working implementation with shortcuts (testcontainers)
// const pg = await new PostgreSqlContainer('postgres:17').withTmpFs({
'/var/lib/postgresql/data': 'rw' }).start();
```

Q 195 LEVEL · Mid to Senior

What is the Specification pattern, and when does it cleaner than chained assertions?

CONCEPT

A Specification is an object that encapsulates a single business rule as a predicate, with the ability to compose multiple specifications via and, or, not. Instead of writing **expect(order.total).toBeGreaterThan(100); expect(order.items.length).toBeGreaterThan(0); expect(order.status).toBe('confirmed')**, the specification pattern says **expect(OrderIsConfirmedAndNonEmpty.isSatisfiedBy(order)).toBe(true)**. The named specification carries the business meaning. The strength: complex business rules become reusable, composable, and named. A test asserting that an order is ready-to-ship uses OrderReadyToShip; a test asserting that an order is eligible-for-refund uses OrderEligibleForRefund. Specs compose: **OrderReadyToShip.and(NotMarkedFraudulent)**. The weakness: it's machinery for cases that don't need it, a simple status check is clearer as one assertion. The discipline: reach for Specification when business rules involve three-plus conditions and are referenced across many tests; stick with chained assertions for single-test rules. The pattern shines when the rule has a name in the domain (“eligible for refund”) and the test wants to assert on that name, not on the implementation details.

INTERVIEW STRATEGY

They are after Specification pattern understood as test-design depth, not just a DDD reference. Lead with the predicate encapsulation + composition, then make 'reach when business rules have names in the domain' the senior signal. Where this goes wrong is over-using it for simple checks. The follow-up is often ‘when is it overkill?’, answer single-test rules with one or two conditions; just write the assertions inline.

How to phrase it

> *A Specification is an object that encapsulates a single business rule as a predicate, with the ability to compose multiple specifications via and, or, not. The contrast lands quickly with an example. Instead of writing three chained assertions, expect order dot total to be greater than a hundred, expect order dot items dot length to be greater than zero, expect order dot status to be confirmed, the specification pattern says expect OrderIsConfirmedAndNonEmpty dot isSatisfiedBy of order to be true. The named specification carries the business meaning, and the test reads as the intent rather than the implementation. The strength, and where this pattern earns its place in a senior framework, is complex business rules become reusable, composable, and named. A test asserting that an order is ready-to-ship uses OrderReadyToShip; a test asserting that an order is eligible-for-refund uses OrderEligibleForRefund. Specs compose: OrderReadyToShip dot and NotMarkedFraudulent. Several tests across the suite share the same OrderReadyToShip specification, so if the definition changes, say, the business adds a new condition like must-have-valid-shipping-address, I update one spec and every test downstream picks it up. Without the spec pattern, that same change requires hunting through the assertion chains in every test that touches the concept. The weakness is it's machinery for cases that don't need it. A simple status check is clearer as one assertion. The discipline I would close on, and my answer to the follow-up about when it's overkill, is reach for Specification when business rules involve three or more conditions and are referenced across many tests; stick with chained assertions for single-test rules with one or two conditions. The pattern shines when the rule has a name in the domain, eligible for refund, ready to ship, requires manual review, and the test wants to assert on that name, not on the implementation details of what eligible means. The thing to avoid is over-using Specification for trivial checks, ending up with a class hierarchy for what should be one expect call.*

Key points to hit

(1) Spec encapsulates a business rule as a composable predicate. **(2)** Composes via .and(), .or(), .not(): complex rules from simple ones. **(3)** Strength: shared definition, change rule once, every test updates. **(4)** Weakness: machinery overhead for simple checks. **(5)** Rule: reach when rule has 3+ conditions and a name in the domain. **(6)** The easy mistake: over-using for trivial checks, class hierarchy for one expect.

CODE

```
// Specification encapsulates a domain rule
interface Spec<T> { isSatisfiedBy(item: T): boolean; }

class And<T> implements Spec<T> {
constructor(private a: Spec<T>, private b: Spec<T>) {}
isSatisfiedBy(item: T): boolean { return this.a.isSatisfiedBy(item) &&
this.b.isSatisfiedBy(item); }
}

const OrderIsConfirmed: Spec<Order> = { isSatisfiedBy: o => o.status === 'confirmed' };
const OrderHasItems: Spec<Order> = { isSatisfiedBy: o => o.items.length > 0 };
const OrderHasShippingAddr: Spec<Order> = { isSatisfiedBy: o =>
Boolean(o.shippingAddress) };

// Composed, named domain rule
const OrderReadyToShip = new And(OrderIsConfirmed, new And(OrderHasItems,
OrderHasShippingAddr));

// Test reads as intent, not implementation details
test('shipping fulfilment picks up ready orders', async ({ request }) => {
const order = await (await request.get('/api/orders/123')).json();
expect(OrderReadyToShip.isSatisfiedBy(order)).toBe(true);
});
```

Q 196 LEVEL · Senior

What is the Saga pattern in test orchestration, and when do you need it?

CONCEPT

A Saga in test code is a long, multi-step flow with compensating actions for each step's failure. Distinct from a serial test or a linear scenario because the Saga explicitly defines what to undo if a later step fails. The pattern: each step records its compensating action (delete the created order, refund the payment, cancel the shipment); if any step fails, the Saga walks the compensating actions in reverse order so the system ends up in a clean state. Real use cases: **payment-checkout-shipment flows** where partial completion leaves real money or inventory in limbo; **multi-system orchestrations** spanning your app, payment gateway, and fulfilment provider where each has its own state; tests that interact with services that don't support transactions so cleanup-on-failure isn't free. The discipline: declare compensating actions at the moment of creation, not at the end. Pushing cleanup to the end of the test means a step that throws halfway never runs the cleanup; declaring compensation as you go means the test framework can walk back automatically. **Trap**: confusing Saga with serial mode. Serial is about ordering; Saga is about reliable cleanup of multi-step side effects.

INTERVIEW STRATEGY

They want Saga as test-design depth, applied to multi-step orchestration. Lead with the compensating-actions-in-reverse-order definition, then the real use cases (payment-checkout-shipment, multi-system, no-transaction services). The 'declare compensation at creation, not end' discipline is the experience marker. The trap is confusing Saga with serial mode. Expect the follow-up ‘how is this different from cleanup fixtures?’, answer Saga is ordered

compensation, fixtures are flat cleanup.

How to phrase it

> *A Saga in test code is a long, multi-step flow with compensating actions for each step's failure. The distinction from a serial test or a linear scenario, and the thing senior interviewers want to hear named, is that the Saga explicitly defines what to undo if a later step fails. Not just cleanup at the end; cleanup that knows what was created up to the point of failure and walks the unwinding in reverse. The pattern: each step records its compensating action. Create an order: compensating action is delete that order. Charge the payment: compensating action is refund the payment. Schedule the shipment: compensating action is cancel the shipment. If any later step fails, the Saga walks the compensating actions in reverse order so the system ends up in a clean state. Real use cases I'd give because they make the abstraction concrete.*
> *Payment-checkout-shipment flows where partial completion leaves real money or inventory in limbo. Multi-system orchestrations spanning my app, the payment gateway, and the fulfilment provider where each has its own state and I can't just rollback a database. Tests that interact with services that don't support transactions, so cleanup-on-failure isn't free, I have to define it operationally. The discipline I would lead with, because it's where Saga differs from naive cleanup, is declare compensating actions at the moment of creation, not at the end of the test. Pushing cleanup to a single end-of-test block means a step that throws halfway never runs the cleanup at all, the throw skips to teardown but the teardown block doesn't know what was created before the throw. Declaring compensation as I go means the test framework can walk back automatically, executing only the compensations for steps that actually completed.*
> *That's also my answer to the follow-up about how this differs from cleanup fixtures: Saga is ordered compensation tied to specific steps; fixtures are flat unordered cleanup of tracked entities. Fixtures handle the database-row case well; Saga handles the cross-system multi-step side-effect case where order of unwind matters. The mistake is confusing Saga with serial mode. Serial is about test ordering: run A before B. Saga is about reliable cleanup of multi-step side effects within one test.*

Key points to hit

(1) Saga = multi-step flow + compensating action per step + reverse unwind on failure. **(2)** Use cases: payment-checkout-shipment, multi-system, no-transaction services. **(3)** Declare compensation AT creation, not at end: throw mid-test still unwinds. **(4)** Walks compensations in reverse for only the steps that completed. **(5)** Follow-up: differs from cleanup fixtures, ordered, step-tied, not flat. **(6)** Trap: confusing Saga with serial mode, different concerns.

CODE

```
class Saga {
private compensations: Array<() => Promise<void>> = [];
async run<T>(step: () => Promise<T>, compensate: (result: T) => Promise<void>):
Promise<T> {
const result = await step();
this.compensations.unshift(() => compensate(result)); // unshift = reverse order
return result;
}
async unwind(): Promise<void> {
for (const c of this.compensations) { try { await c(); } catch { /* keep going */ } }
}
}

test('full checkout flow with reliable unwind', async ({ request }) => {
const saga = new Saga();
try {
const order = await saga.run(
() => request.post('/api/orders', { data: { product: 'X' } }).then(r => r.json()),
(o) => request.delete(`/api/orders/${o.id}`).then(() => undefined),
);
const charge = await saga.run(
() => request.post('/api/payments', { data: { orderId: order.id, amount: 100 } }).then(r
=> r.json()),
(c) => request.post(`/api/payments/${c.id}/refund`).then(() => undefined),
);
// If charge throws mid-flow, the order delete still runs.
} finally {
await saga.unwind();
}
});
```

Q 197 LEVEL · Senior

How do you apply hexagonal architecture (ports and adapters) to test code itself?

CONCEPT

Hexagonal architecture in test code means separating what the test verifies (the domain) from how it interacts with the system (the adapters). The test code declares ports, interfaces like **UserRepository**, **OrderApi**, **NotificationChannel**, describing what operations the test needs. Adapters implement those ports for specific environments: a PlaywrightUserRepository for UI tests, an HttpUserRepository for API tests, a SqlUserRepository for direct database verification. The test itself depends on the ports, not the adapters, so the same test logic runs against different environments by swapping the adapter at fixture level. The value: tests stay readable and stable while environments change. Moving from REST API to GraphQL means writing a new GraphQL adapter; the test code is unchanged. Adding a separate verification path through the database for data-correctness tests means another adapter; tests still read the same. The discipline that makes it work: ports describe operations, not implementation. Define UserRepository dot findByEmail, not UserRepository.executeQueryWithEmailFilter. Operations stay stable; implementations are free to vary.

INTERVIEW STRATEGY

What earns the tick is hexagonal architecture applied to tests, not just production code. Lead with port-as-interface and adapter-as-implementation, then the same-test-different-environment benefit. The 'ports describe operations, not implementation' rule is the senior signal. The risk is putting implementation details in the port interface. They usually follow with ‘give a real example?’, answer moving from REST to GraphQL: new adapter, unchanged test code.

How to phrase it

> *Hexagonal architecture in test code means separating what the test verifies, the domain operations, from how it interacts with the system, the adapters. The test code declares ports, interfaces like UserRepository, OrderApi, NotificationChannel, describing what operations the test needs. Adapters implement those ports for specific environments: a PlaywrightUserRepository for UI tests that interacts via the browser, an HttpUserRepository for API tests that uses APIRequestContext, a SqlUserRepository for direct database verification when I need to check rows that aren't exposed through the API. The test itself depends on the ports, not the adapters, so the same test logic runs against different environments by swapping the adapter at fixture level. The value, and the part senior interviewers want to hear articulated, is tests stay readable and stable while environments change. That's the same idea Alistair Cockburn had for production code, applied to tests. The real example that lands, and my answer to the follow-up, is moving from REST API to GraphQL. Without hexagonal architecture, every test that uses the API has to be updated when the team migrates the API to GraphQL. With ports and adapters, I write a new GraphQL adapter for UserRepository, swap the implementation in the fixture, and the test code is unchanged. The tests now run against GraphQL, verifying the same domain operations, with no rewrite. Another example: adding a database-level verification path for data-correctness tests where I want to assert on rows the API doesn't expose. Another adapter, same test pattern. The discipline that makes this work, and where teams that try hexagonal in tests often go wrong, is ports describe operations, not implementation. Define UserRepository dot findByEmail, not UserRepository dot executeQueryWithEmailFilter or UserRepository dot httpGetWithEmail Query. Operations are stable in the domain; implementations are free to vary. If the port leaks implementation details, the abstraction is just a renamed adapter, and swapping environments still requires rewriting tests. The thing to avoid is putting implementation in the port interface, which defeats the whole point.*

Key points to hit

(1) Tests depend on ports (interfaces describing domain operations). **(2)** Adapters implement ports for environments: HTTP, browser, SQL. **(3)** Same test logic across environments: swap adapter at fixture level. **(4)** Real example: REST -> GraphQL migration, new adapter, unchanged tests. **(5)** Discipline: ports describe operations, not implementation details. **(6)** Where this goes wrong: port leaks implementation, abstraction becomes renamed adapter.

CODE

```
// PORT: describes domain operations the test needs
interface UserRepository {
findByEmail(email: string): Promise<User | null>;
create(user: NewUser): Promise<User>;
delete(id: string): Promise<void>;
}

// ADAPTERS: environment-specific implementations
class HttpUserRepository implements UserRepository { /* uses APIRequestContext */ }
class GraphQLUserRepository implements UserRepository { /* same operations via GraphQL
*/ }
class SqlUserRepository implements UserRepository { /* direct DB for verification */ }
class PlaywrightUserRepository implements UserRepository { /* via browser UI */ }

// Fixture chooses the adapter for this test run
export const test = base.extend<{ users: UserRepository }>({
users: async ({ request }, use) => {
await use(new HttpUserRepository(request)); // swap to GraphQL/SQL without changing
tests
},
});

// Test depends on the port: stable across adapter changes
test('user lookup', async ({ users }) => {
const u = await users.findByEmail('anna@bank.com');
expect(u?.role).toBe('admin');
});
```

Q 198

LEVEL · Senior

Beyond toHaveScreenshot, what is the production playbook for visual regression at scale?

CONCEPT

page.toHaveScreenshot() is the entry point: fast, free, no extra service, baselines committed to the repo. But running it at scale exposes three real problems and corresponding solutions. Problem 1: cross-OS rendering differences -- the test passes on the Mac dev machine, fails on Linux CI because of font anti-aliasing. Solution: pin baselines to a specific Docker image and never run visual tests outside it. The official mcr.microsoft.com slash playwright image is the canonical choice. Problem 2: dynamic content noise -- timestamps, randomised avatars, animations, ads. Solution: mask the volatile regions via the mask option, or clip to the stable section via the clip option. Problem 3: baseline-update review burden -- every intentional design change requires updating dozens of baselines, and reviewers cannot tell which diffs are intentional. Solution: AI-based visual diff tools like Applitools or Percy that group similar changes, highlight semantic changes, and run perception-based comparisons rather than pixel-perfect ones. The principle that matters most: start with toHaveScreenshot for stable critical pages; reach for Applitools or Percy when the baseline-update burden becomes the bottleneck. The mistake is enabling visual tests across every page, watching every PR fail on unrelated diffs, and abandoning the entire approach. Pick stable pages first.

INTERVIEW STRATEGY

You need to give visual regression calibrated as a real production discipline, not waved at. Lead with the three real problems (cross-OS rendering, dynamic content, baseline-update burden) and the specific solution for each. The toHaveScreenshot-for-stable-pages and AI-tools-for-bottleneck tiering marks senior judgement. The pitfall is enabling visual tests everywhere and watching them fail constantly. The next question is normally Applitools vs Percy -- answer Applitools for richer AI grouping and self-healing; Percy for tighter GitHub integration.

How to phrase it

> *page.toHaveScreenshot is the entry point: fast, free, no extra service, baselines committed to the repo as PNG files. That works fine for a handful of stable pages. But running visual regression at scale exposes three real problems, and the calibration that interviewers probe is knowing the specific solution for each. Problem one: cross-OS rendering differences. The test passes on the Mac dev machine, fails on Linux CI because of font anti-aliasing, kerning, or subpixel rendering. Solution: pin baselines to a specific Docker image and never run visual tests outside it. The official Microsoft Playwright Docker image is the canonical choice because it is the same image the framework ships against. Devs running visual tests locally use the same Docker image as CI; baselines are captured inside the image; flake from OS differences disappears. Problem two: dynamic content noise. Timestamps, randomised avatars, animations, ad slots, user-specific data. Every visual test fails on the noise rather than on real regressions, the suite gets distrusted, people start ignoring failures. Solution: mask the volatile regions via the mask option, which paints them solid pink so the pixel diff ignores them. Or clip to the stable section via the clip option, capturing only the portion of the page that is deterministic. Both let me test the parts that should be stable while ignoring the parts that legitimately vary. Problem three: baseline-update review burden. Every intentional design change requires updating dozens of baselines, and reviewers cannot tell which diffs are intentional and which are regressions. The PR has fifty PNG changes; the reviewer rubber-stamps them; the regression slips through. Solution: AI-based visual diff tools like Applitools or Percy that group similar changes, highlight semantic changes, and run perception-based comparisons rather than pixel-perfect ones. On the follow-up about Applitools vs Percy: Applitools for richer AI grouping and self-healing baselines; Percy for tighter GitHub integration. The principle that matters most is start with toHaveScreenshot for stable critical pages -- checkout, login, dashboard -- and reach for Applitools or Percy when the baseline-update burden becomes the bottleneck. The mistake is enabling visual tests across every page at once, watching every PR fail on unrelated diffs, and abandoning the entire approach because it slowed the team down. Pick stable pages first; expand coverage as the team's discipline around baselines matures.*

Key points to hit

(1) toHaveScreenshot baseline approach: free, in-repo, fine for stable pages. **(2)** Cross-OS flake: pin to mcr.microsoft.com/playwright Docker image. **(3)** Dynamic content: mask (paints pink) or clip (capture only stable region). **(4)** Baseline-update burden: Applitools or Percy for AI grouping plus semantic diffs. **(5)** Rule: toHaveScreenshot for stable pages; AI tools when burden bottlenecks. **(6)** Pitfall: enabling visual tests everywhere -- noise, distrust, abandonment.

CODE

```
// toHaveScreenshot with mask plus clip for noise control
test('checkout page visual regression', async ({ page }) => {
await page.goto('/checkout');
await expect(page.getByRole('heading', { name: 'Checkout' })).toBeVisible();

await expect(page).toHaveScreenshot('checkout.png', {
fullPage: true,
mask: [
page.locator('[data-testid="timestamp"]'),
page.locator('[data-testid="user-avatar"]'),
],
maxDiffPixelRatio: 0.01,
animations: 'disabled',
});
});

// Pin visual baselines to the Playwright Docker image
// Update baselines from inside the image, not from your laptop:
// docker run -it --rm -v $(pwd):/work mcr.microsoft.com/playwright:v1.45.0-jammy \
// bash -c 'cd /work && npx playwright test --update-snapshots --project=visual'

// Applitools alternative when baseline-update burden becomes the bottleneck
// import { Eyes, Target } from '@applitools/eyes-playwright';
// const eyes = new Eyes();
// await eyes.open(page, 'AcmeApp', 'Checkout');
// await eyes.check('Checkout', Target.window().fully());
// await eyes.close();
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What problem does the Builder pattern solve for test data?

A) Tests state only the fields that matter; schema changes update one builder
B) It makes tests run in parallel
C) It removes the need for the API
D) It replaces fixtures

2. How does withRetry differ from Playwright's built-in test retries?

A) They are identical
B) withRetry retries one operation inside a test; Playwright retries the whole test
C) withRetry only works in CI
D) Playwright retries individual operations

3. Why use a cleanup registry instead of deleting data at the end of the test?

A) It is faster
B) It runs cleanup in setup order
C) It avoids using the API
D) It deletes only the resources created so far, even if the test fails mid-setup

4. What gap does Playwright Component Testing fill?

A) It replaces end-to-end tests
B) It tests only APIs
C) Between unit (mocks the DOM) and E2E (needs the whole stack), real rendering, isolated
D) It is only for visual screenshots

5. Why must an SPA page object wait for content after navigation, not just the URL?

A) URLs are unreliable
B) It speeds up the test
C) Playwright cannot read URLs
D) In an SPA the URL changes instantly but content loads asynchronously

Section 14 · Answer key

#	Answer	Why and where to revisit
1	A) Tests state only the fields that matter; schema changes update one builder	A fluent builder seeds defaults and lets tests override only what matters; a schema change is a one-place edit. *Revisit Section 14.*
2	B) withRetry retries one operation inside a test; Playwright retries the whole test	Playwright retries re-run the entire test; withRetry retries a single flaky external operation with backoff. *Revisit Section 14.*
3	D) It deletes only the resources created so far, even if the test fails mid-setup	Registering a delete as each resource is created means a partial-setup failure still cleans up what exists. *Revisit Section 14.*
4	C) Between unit (mocks the DOM) and E2E (needs the whole stack), real rendering, isolated	It mounts a real component in a real browser, real CSS and events, without the full application stack. *Revisit Section 14.*
5	D) In an SPA the URL changes instantly but content loads asynchronously	The route updates client-side immediately; asserting before the async content renders races the data load. *Revisit Section 14.*

SECTION 15

AI for SDET Engineers

AI is the question every 2026 interview now asks, and the wrong answer is either dismissing it or claiming it replaces engineers. The right one positions AI as an accelerator paired with the expertise to evaluate its output. These ten questions cover where AI changes SDET work, generating tests from requirements, self-healing locators, AI-assisted failure analysis, and a review checklist for validating AI-generated test code.

In this section

- AI as accelerator, not replacement: generate scaffolding, review every line.
- LLM test generation: acceptance criteria in the prompt drive real coverage.
- Self-healing locators as a maintenance strategy with logged technical debt.
- AI failure analysis from error + test + diff: 20 minutes to 3.
- A five-point review checklist that catches 90% of AI mistakes.

Q 199 LEVEL · Mid

How is AI transforming test automation in 2026?

CONCEPT

AI touches five areas of SDET work. **Test generation**: LLMs draft test cases from user stories and acceptance criteria, with review. Self-healing locators: a model identifies the right element when a locator breaks after a UI change. **Intelligent maintenance**: LLMs suggest fixes for failing tests given the error and the diff. Defect prediction: ML trained on commit history flags high-risk code areas. Visual AI: semantic screenshot comparison that understands meaningful change versus noise. The professional framing is **accelerator, not replacement**: use Copilot to scaffold page objects and stubs from a spec, then review every line, AI defaults to CSS selectors, which you upgrade to getByRole. The critical 2026 skill is prompt engineering for test generation plus the expertise to evaluate the output.

INTERVIEW STRATEGY

This question screens for attitude as much as knowledge. Avoid both extremes, dismissing AI or claiming it replaces you, and land on accelerator-plus-judgement. Name a few of the five areas, then make it concrete: AI scaffolds, you review, and you upgrade its CSS selectors to getByRole. The mistake is sounding either threatened or naive. A common follow-up is 'so what is the SDET's job now?', answer prompt engineering for generation plus the expertise to evaluate and correct what the model produces.

How to phrase it

> *I position AI as an accelerator, not a replacement, and I would say that up front because this question is really testing my attitude. AI touches about five areas of our work: generating test cases from user stories, self-healing locators when the UI changes, suggesting fixes for failing tests, predicting which code areas are high-risk from commit history, and semantic visual comparison. In my own day-to-day I use Copilot to generate the scaffolding of a page object or a set of test stubs from a specification, and then I review every line, because AI consistently defaults to CSS selectors, and I upgrade those to getByRole, which is exactly the kind of judgement it cannot make for me. That leads into the follow-up about what the SDET's job actually is now: the critical 2026 skill is prompt engineering for test generation combined with the expertise to evaluate the output. The generation is cheap; knowing whether a generated test uses the right locator strategy, has proper awaits, and covers the negative cases is the expensive, valuable part, and that is squarely the engineer's job. So I am neither threatened by it nor naive about it, I treat AI as a force multiplier that makes me faster on the boilerplate while my review and design judgement is what keeps the quality bar where it needs to be. That balance is the answer that lands in 2026 interviews.*

Key points to hit

(1) Five areas: generation, self-healing, maintenance, defect prediction, visual AI. **(2)** Framing: accelerator, not replacement. **(3)** AI scaffolds; you review every line and upgrade CSS to getByRole. **(4)** Expect next: the SDET job is prompt engineering + evaluating the output. **(5)** Generation is cheap; judgement on locators/awaits/coverage is the value. **(6)** What loses the point: sounding either threatened by AI or naive about it.

Q 200 LEVEL · Mid

How do you use LLMs for test case generation from requirements?

CONCEPT

LLMs can generate test-case matrices from user stories, but **quality depends entirely on the prompt**. A vague prompt yields happy-path-only tests; a detailed prompt with explicit **acceptance criteria**, redirect on valid login, exact error text on invalid password, the required-field validation, the account-lock behaviour, produces comprehensive coverage. An effective prompt also supplies the **available page objects** and asks for a specific output (the test file only, no explanation, using the project's fixtures). With good criteria the output covers roughly 80% of what you would write by hand, leaving the engineer the 20% the LLM misses: auth-state setup, API validation assertions, and test-data strategy. Every generated test is reviewed before committing, AI output is a starting point, not a finished product.

INTERVIEW STRATEGY

They are looking to see disciplined prompting, not 'paste-and-pray'. Make the acceptance criteria the centrepiece: without them you get only the happy path; with them you get ~80% coverage. Then name the 20% you still own, auth setup, API assertions, data strategy, and that you review every line. The trap is committing generated tests unreviewed. The follow-up is often 'what does the LLM miss?', answer the cross-cutting setup and the negative/edge coverage that needs domain judgement.

How to phrase it

> *LLMs are good at generating a test-case matrix from a user story, but the quality depends entirely on the prompt, and that is what I would build the answer around. The single biggest lever is putting explicit acceptance criteria in the prompt. Without them, the model gives me only the happy path. With them, valid credentials redirect to the dashboard, an invalid password shows this exact error text, an empty email shows the required-field error, the account locks after five attempts, the output covers maybe eighty percent of what I would write by hand. I also give it the available page objects and ask for the test file only, using our fixtures, with no explanation, so I get something I can drop in. That sets up the follow-up about what the LLM misses, which is the twenty percent I spend my time on: the auth-state setup, the API validation assertions, and the test-data strategy, the cross-cutting decisions that need domain judgement and knowledge of our system. And I review every generated test before committing, because AI output is a starting point, not a finished product, the trap is committing it unreviewed and shipping CSS-selector, happy-path-only tests. So my approach is detailed criteria-driven prompts to get a strong first draft fast, then engineer judgement on the parts that actually require it. That is a real productivity gain without lowering the bar.*

Key points to hit

(1) Prompt quality decides everything; vague prompts give happy-path only. **(2)** Explicit acceptance criteria drive ~80% coverage. **(3)** Supply page objects; ask for the test file only, using project fixtures. **(4)** Engineer owns the 20%: auth setup, API assertions, data strategy. **(5)** They usually ask next: the LLM misses cross-cutting setup and negative/edge coverage. **(6)** The

common miss: committing generated tests unreviewed.

CODE

```
/* Effective test-generation prompt
User Story: As a user I can log in with email and password.
Acceptance Criteria:
- Valid credentials redirect to /dashboard
- Invalid password shows "Invalid email or password"
- Empty email shows "Email is required"
- Account locks after 5 failed attempts ("Account locked")
Page Objects: LoginPage (navigate, login, getErrorMessage)
Generate Playwright TypeScript tests using @fixtures/pages.fixture.
Cover positive, negative, and edge cases. Output only the test file. */
```

Q 201 LEVEL · Mid

How do you implement self-healing locators in a Playwright framework?

CONCEPT

Self-healing locators use **multiple fallback strategies**: when the primary locator fails, the helper tries alternatives in order and logs a warning identifying what healed so a developer can fix the primary. A healingLocator function takes an array of strategy functions and a description, returns the first that resolves to exactly one element, and warns if it had to fall back. The critical framing: this is a **maintenance strategy, not a permanent solution**. A healed locator is technical debt: the warning should become a ticket to update the primary. Healing prevents a CI failure from blocking the team while the proper fix is scheduled. For larger frameworks a tool like **Healenium** does more sophisticated AI-based healing, but a custom implementation demonstrates the concept cleanly.

INTERVIEW STRATEGY

They are quietly checking whether you treat self-healing as a crutch or a managed stopgap. State plainly it is a maintenance strategy, not a permanent fix, and that every heal logs technical debt to be paid down. That governance framing is the experience marker. Where this goes wrong is treating healing as a reason never to fix locators, which lets the suite rot. Expect the follow-up ‘doesn't this hide broken locators?’, answer that the logged warning becomes a ticket, so it surfaces the debt rather than burying it.

How to phrase it

> *Self-healing locators try multiple fallback strategies: when the primary locator fails, a helper tries alternatives in order, returns the first that resolves to exactly one element, and logs a warning saying which fallback it used. My healingLocator function takes an array of strategy functions and a description, so the primary might be getByRole, the first fallback a testid, the second a data attribute. But the framing I lead with, because it is what separates good use from bad, is that this is a maintenance strategy, not a permanent solution. A locator that heals is technical debt: the warning it logs should become a ticket to update the primary locator. That is also my answer to the follow-up about whether this just hides broken locators: it would, if the warning went nowhere, but I treat each heal as a debt item that surfaces in the logs and gets scheduled, so it is visible rather than buried. What healing buys me is that a single changed locator does not block the whole team's CI while the fix is scheduled, the test heals, stays green, and I fix the primary properly soon after. The thing to avoid is treating self-healing as a reason never to fix locators, which lets the suite quietly rot on fallbacks. For larger frameworks I would use something like Healenium for more sophisticated AI-based healing, but a custom implementation like this demonstrates the concept clearly and keeps the governance explicit.*

Key points to hit

(1) Try fallback strategies in order; return the one that resolves uniquely. **(2)** Log a warning identifying what healed. **(3)** Framing: a maintenance strategy, not a permanent fix. **(4)** Each heal is technical debt: the warning becomes a ticket. **(5)** The next question: doesn't hide breakage because the heal is logged and scheduled. **(6)** Avoid: using healing as a reason never to fix locators.

CODE

```
export async function healingLocator(
page: Page, strategies: Array<() => Locator>, description: string,
): Promise<Locator> {
for (let i = 0; i < strategies.length; i++) {
const locator = strategies[i]();
if (await locator.count() === 1) {
if (i > 0) console.warn(`[HEALING] Primary failed for "${description}"; used fallback
${i + 1}. Update the primary.`);
return locator;
}
}
throw new Error(`All locator strategies failed for: ${description}`);
}

const submit = await healingLocator(page, [
() => page.getByRole('button', { name: 'Submit Order' }), // primary
() => page.getByTestId('submit-order-btn'), // fallback 1
() => page.locator('[data-action="submit-order"]'), // fallback 2
], 'Submit Order button');
```

Q 202 LEVEL · Mid to Senior

How do you use AI for test maintenance and failure analysis?

CONCEPT

LLMs can analyse a failing test given three inputs, the **error message**, the **test code**, and the recent diff, and suggest a fix, which accelerates maintenance sharply. It is most valuable for strict-mode violations and locator failures after a UI change: the model sees that a new button was added in the diff, recognises why locator('button') is now ambiguous, and proposes the precise getByRole fix. The practical implementation is a small tool (for example a VS Code extension) that automatically pulls the failing test, the error, and the last few commits' diff and sends them to the LLM, surfacing the suggested fix as a code action. Done well this can cut average locator fix time from around **20 minutes to 3**, with the engineer still reviewing and applying the suggestion, never auto-merging it.

INTERVIEW STRATEGY

The point of this one is to see you feed the model the right context, not just the error. Stress the three inputs, error, test code, recent diff, because the diff is what lets the LLM reason about cause ('a new button was added'). Quantify the speedup (20 to 3 minutes) and keep the human in the loop. The mistake is auto-applying AI fixes. They usually follow with 'how does the LLM know the cause?', answer it correlates the error with the diff, which is why supplying the recent commits matters.

How to phrase it

> *For maintenance I have the LLM analyse a failing test from three inputs: the error message, the test code, and the recent diff. The three together are the whole trick, and I would emphasise the diff in particular. It is most valuable for strict-mode violations and locator failures after a UI change, the kind where a test that worked yesterday suddenly resolves to multiple elements. That is also my answer to the follow-up about how the LLM knows the cause: it correlates the error with the diff. If the error says locator('button') resolved to eight elements, and the diff shows a new Cancel button was just added next to the Submit button, the model can reason that the new button made the selector ambiguous and suggest the precise fix, getByRole button with the name Submit Order. Without the diff it could only guess. The practical implementation I built is a small VS Code extension that automatically pulls the failing test, the error, and the last few commits' diff, sends them to the LLM, and surfaces the suggested fix as a code action. That took our average locator fix time from about twenty minutes of investigation down to around three. The important guard, and the trap I avoid, is that the engineer still reviews and applies the suggestion, I never auto-merge an AI fix, because a plausible-looking wrong fix is worse than an honest failure. So AI does the diagnosis legwork; I keep the decision.*

Key points to hit

(1) Feed the LLM three inputs: error message, test code, recent diff. **(2)** Best for strict-mode violations and post-UI-change locator failures. **(3)** The diff lets the model reason about cause (a new button was added). **(4)** A VS Code tool can cut locator fix time from ~20 min to ~3. **(5)** Asked next: the LLM infers cause by correlating error with diff. **(6)** The easy mistake: auto-applying AI fixes instead of reviewing them.

CODE

```
/* AI-assisted failure-analysis prompt
ERROR: strict mode violation: locator('button') resolved to 8 elements
TEST CODE: await page.locator('button').click();
RECENT DIFF:
+ <button class="btn-secondary">Cancel</button>
+ <button class="btn-primary">Submit Order</button>
PAGE OBJECTS: CheckoutPage (submitButton, cancelButton)
Suggest the correct fix using Playwright best practices. */

// AI output (reviewed, then applied):
// await page.getByRole('button', { name: 'Submit Order' }).click();
// Reason: a new Cancel button made locator('button') ambiguous.
```

Q 203 LEVEL · Mid to Senior

How do you evaluate and validate AI-generated test code?

CONCEPT

AI-generated tests need **systematic review against a quality checklist** before merging, because the model makes predictable mistakes. Five checks catch most of them. Locator quality: upgrade CSS selectors to getByRole/getByLabel, since AI defaults to CSS. **Await coverage**: verify every Playwright action is awaited. Assertion type: replace one-shot isVisible() with retrying toBeVisible(). Test independence: ensure no shared state, since AI often reuses data across tests. **Negative cases**: add the error and edge cases AI skips in favour of the happy path. Two of these (await coverage, assertion type) are caught automatically by ESLint (no-floating-promises, prefer-web-first-assertions), leaving manual review to focus on independence and negative coverage. The result: about 15 minutes of review versus 45 writing from scratch.

INTERVIEW STRATEGY

They are after a repeatable validation process, not 'I read it over'. Present the five-point checklist tied to AI's predictable mistakes, then show leverage: ESLint auto-catches two of the five, so manual review focuses on independence and negative coverage. Quantify the net gain (15 min review vs 45 writing). The trap is trusting AI output by default. The next question is normally 'which mistakes can you automate catching?', answer no-floating-promises for awaits and prefer-web-first-assertions for assertion type.

How to phrase it

> *I review AI-generated tests against a checklist built specifically for the mistakes these models reliably make, rather than reading them over ad hoc. Five checks catch about ninety percent of the problems. One, locator quality: AI defaults to CSS selectors, so I upgrade them to getByRole or getByLabel. Two, await coverage: verify every Playwright action is awaited, because AI sometimes drops them. Three, assertion type: replace one-shot isVisible with retrying toBeVisible, since AI loves point-in-time checks. Four, test independence: AI often reuses the same data across tests, so I ensure there is no shared state. Five, negative cases: AI focuses on the happy path, so I add the error and edge cases. The leverage, and my answer to the follow-up about which mistakes I can automate catching, is that I run ESLint with no-floating-promises and prefer-web-first-assertions before I even read the code, which automatically catches checks two and three, the missing awaits and the assertion type. That frees my manual review to concentrate on test independence and negative-case coverage, the parts that genuinely need domain judgement. With this process an AI-generated test takes about fifteen minutes to review versus forty-five to write from scratch, so it is roughly a three-times productivity gain with the quality bar held constant. The mistake is trusting AI output by default; the checklist plus the linter is what lets me move fast without lowering the bar.*

Key points to hit

(1) Review against a checklist targeting AI's predictable mistakes. **(2)** Five checks: locator quality, awaits, assertion type, independence, negative cases. **(3)** ESLint auto-catches awaits (no-floating-promises) and assertion type. **(4)** Manual review focuses on independence and negative coverage. **(5)** Net: ~15 min review vs ~45 writing, ~3x gain, bar held. **(6)** Trap: trusting AI output by default.

CODE

```
// AI-generated-test review checklist
// 1. Locator quality: upgrade CSS selectors to getByRole/getByLabel
// 2. Await coverage: every Playwright action awaited
// 3. Assertion type: isVisible() -> toBeVisible() (retrying)
// 4. Test independence: no shared state / reused data
// 5. Negative cases: add error + edge cases AI skipped

// Automate 2 and 3 before manual review:
// '@typescript-eslint/no-floating-promises': 'error'
// 'playwright/prefer-web-first-assertions': 'error'
```

Q 204 LEVEL · Senior

What is Playwright MCP, and how does it change the developer experience around test authoring?

CONCEPT

MCP (Model Context Protocol) is Anthropic's open standard for giving AI assistants structured access to external tools and data sources. **Playwright MCP** is a server that exposes Playwright's capabilities, navigate, click, fill, query the DOM, to MCP-compatible AI clients like Claude Desktop,

Cursor, Windsurf. The developer experience shift: instead of writing test code manually, you describe the scenario in natural language and the AI drives a real browser session to discover the selectors, the flow, and the assertions. The AI queries the live DOM, finds working selectors based on accessibility roles or test IDs, and emits Playwright TypeScript code with proper auto-waiting. Real uses: scaffolding the first test for a new feature in minutes instead of hours; diagnosing why a failing test broke by letting the AI walk the page state; **generating data-driven test variations** from one example. The discipline: MCP-generated tests are starting points, not finished work. They still need human review for the same reasons covered in Q5, selector quality, auto-waiting discipline, negative-case coverage. But as a productivity multiplier, the time saved on the first draft is real and growing fast in 2026.

INTERVIEW STRATEGY

They want Playwright MCP understood as the 2026 productivity story. Lead with what MCP is (Anthropic's standard for AI tool access), then Playwright MCP as the server that gives AI a browser. The 'starting point not finished work' discipline is what separates junior from senior thinking because it ties to Q5's review process. The risk is treating MCP-generated tests as production-ready. A common follow-up is ‘what does it actually save?’, answer time on scaffolding the first draft and on diagnosing failures.

How to phrase it

> *MCP, the Model Context Protocol, is Anthropic's open standard for giving AI assistants structured access to external tools and data sources. Playwright MCP is a server that exposes Playwright's capabilities, navigate, click, fill, query the DOM, to MCP-compatible AI clients like Claude Desktop, Cursor, Windsurf. So an AI running in my IDE or in a chat window can actually drive a real browser, see what the page looks like, find selectors, and emit code. The developer experience shift, and the framing I would lead with because it's the actually-new thing about 2026, is this. Instead of writing test code manually, open the app, inspect element, guess a selector, write the line, watch it fail, refine, I describe the scenario in natural language and the AI drives a real browser session to discover the selectors, the flow, and the assertions. The AI queries the live DOM, finds working selectors based on accessibility roles or test IDs, and emits Playwright TypeScript code with proper auto-waiting. The selectors are based on what the page actually exposes, not on the AI's guess about typical web apps, which is the big advance over plain LLM code generation. Real uses worth naming. Scaffolding the first test for a new feature in minutes instead of hours, I describe the user journey, the AI produces a working draft. Diagnosing why a failing test broke by letting the AI walk the page state and compare against the expected flow. Generating data-driven test variations from one example, I show the AI one test, ask for five more covering edge cases, and the AI uses MCP to verify each actually runs. The discipline I would close on, and my answer to the follow-up about what it saves, is MCP-generated tests are starting points, not finished work. They still need human review for the same reasons covered in the previous question, selector quality, auto-waiting discipline, negative-case coverage. The time saved is on the first draft and on diagnosing failures; the time still spent is on review, on adding the cases the AI didn't think of, and on the architectural concerns the AI doesn't see. As a productivity multiplier, the time saved on the first draft is real and growing fast in 2026, especially for teams willing to invest in good prompts and review discipline. The mistake is treating MCP-generated tests as production-ready and skipping review.*

Key points to hit

(1) MCP = Anthropic's standard for AI access to external tools. **(2)** Playwright MCP server exposes browser ops to AI clients (Claude, Cursor). **(3)** AI drives real browser, queries live DOM, emits selector-correct code. **(4)** Uses: scaffold first test, diagnose failures, generate variations. **(5)** Discipline: starting points, not finished work, still need review. **(6)** Where this goes wrong: treating MCP-generated tests as production-ready, skip review at cost.

CODE

```
// .mcp/playwright.config.json: register Playwright MCP server with the AI client
{
"mcpServers": {
"playwright": {
"command": "npx",
"args": ["@playwright/mcp", "--browser", "chrome", "--headless"]
}
}
}

// Developer workflow:
// 1. Prompt: "Write a Playwright test for the checkout flow on staging.acme.com.
// Use storageState 'auth/user.json'. Cover happy path + card decline."
// 2. AI uses MCP to navigate to the URL, inspect the actual DOM, find selectors.
// 3. AI emits TypeScript code with selectors verified against the live page.
// 4. ALWAYS review: selectors, auto-waiting, negative cases, independence.
// 5. Add the cases the AI missed; commit.
//
// What actually saves time: the first draft. What still needs you: the polish.
```

Q 205 LEVEL · Mid to Senior

What's the structure of a prompt that reliably produces high-quality Playwright tests?

CONCEPT

Prompts for test generation follow a structure that lands consistently better than free-form requests. Five sections worth including. **Role/context**: “You are an SDET writing Playwright TypeScript tests for a checkout flow. Use the @playwright/test API, fullyParallel mode, and the test fixture pattern.” Constraints: “All locators must use getByRole or getByTestId. No CSS selectors. Use expect().toBeVisible() instead of toHaveCount > 0. Maximum one assertion per test.” Domain context: “The app uses a stripe-hosted payment widget. Test users have storageState in auth/user.json.” **Example output format**: a few lines of the expected style so the model matches your conventions. The actual task: the scenario to cover. The discipline: constraints do most of the work. Without explicit constraints, models default to whatever style is most common in their training data, which mixes Playwright with Cypress, Selenium, Jest, and the result needs heavy editing. With clear constraints, output quality improves dramatically. **The follow-up insight**: codify your team's conventions in a reusable prompt template rather than retyping them each time.

INTERVIEW STRATEGY

What earns the tick is prompt-engineering structure. Walk the five sections (role, constraints, domain, example, task). The 'constraints do most of the work' rule is the senior signal. The codified-template idea closes well. The risk is free-form prompts that get inconsistent output. The follow-up is often ‘what's the highest-leverage section?’, answer constraints, without them the model picks whatever's common in training data.

How to phrase it

> *Prompts for test generation follow a structure that lands consistently better than free-form requests, and the difference is dramatic enough that prompt engineering is a real skill rather than chat-with-AI novelty. Five sections worth including. Role and context: you are an SDET writing Playwright TypeScript tests for a checkout flow. Use the at-playwright-slash-test API, fullyParallel mode, and the test fixture pattern. This tells the model what kind of code to produce and which framework. Constraints: all locators must use getByRole or getByTestId. No CSS selectors. Use expect to-be-visible instead of to-have-count greater than zero. Maximum one assertion per test. This is where most of the work happens, and the framing I would lead with because it's the highest-leverage section. Domain context: the app uses a Stripe-hosted payment widget. Test users have storageState in auth slash user dot json. This gives the model app-specific knowledge it can't infer. Example output format: a few lines of the expected style so the model matches my team's conventions, import style, test description format, comment discipline. Then the actual task: the scenario to cover. The discipline I would emphasise, because it's where prompt quality compounds, is constraints do most of the work. Without explicit constraints, models default to whatever style is most common in their training data, which mixes Playwright with Cypress, Selenium, Jest patterns, jQuery selectors. The output needs heavy editing because half of it is the wrong framework's idioms. With clear constraints, output quality improves dramatically and review becomes a polish step rather than a rewrite. And to the follow-up about the highest-leverage section: constraints. Role and context set the tone; example output sets the style; the task names what to do; but constraints are what stop the model from defaulting to common-but-wrong patterns. The follow-up insight that lands as the senior framing is codify the team's conventions in a reusable prompt template rather than retyping them each time. I keep a playwright-test-prompt dot md file in the repo that defines the constraints, the team's conventions, the example format, and prepend it to every test-generation request. The failure mode is free-form prompts every time, getting inconsistent output, and never building the prompt template that would make the AI consistently useful.*

Key points to hit

(1) Five sections: role, constraints, domain, example output, task. **(2)** Constraints do the most work: stop the model defaulting to common patterns. **(3)** Without constraints: output mixes Playwright/Cypress/Selenium/Jest idioms. **(4)** Example output format teaches the model your team's style. **(5)** Codify conventions in a reusable prompt template, don't retype each time. **(6)** What loses the point: free-form prompts, inconsistent output, no leverage from prior work.

CODE

```
# prompts/playwright-test.md: reusable team prompt template

## Role & context
You are an SDET writing **Playwright TypeScript** tests using @playwright/test.
Tests run with `fullyParallel: true` on Chromium, Firefox, WebKit.

## Constraints (MUST follow)
- Locators: only `getByRole`, `getByLabel`, `getByTestId`, no CSS, no XPath.
- Assertions: only web-first (`toBeVisible`, `toHaveText`, etc), never `isVisible()` in
expects.
- Auto-waiting: never `page.waitForTimeout`. Use `expect.toHaveURL` or
`waitForResponse`.
- Test independence: each test creates its own data via the `userBuilder` fixture.
- One scenario per test; describe blocks group related tests.

## Domain context
App: acme.com. Stripe-hosted payment widget at iframe `[title="Payment"]`.
Test users: storageState at `auth/{role}.json`. Roles: admin, user, viewer.

## Example output
```ts
test('admin removes a user', async ({ adminPage, userBuilder }) => {
const user = await userBuilder.create();
await adminPage.users.deleteUser(user.email);
await expect(adminPage.users.row(user.email)).toBeHidden();
});
```

Task
{{ paste the scenario here }}
```

Q 206

LEVEL · Senior

What is AI-driven exploratory testing, and where does it fit in a test strategy?

CONCEPT

AI-driven exploratory testing uses an autonomous agent to navigate an application as a curious user would, discovering states the human test designer didn't think of. The agent has browser access (via MCP or similar), a high-level goal (“explore the checkout flow looking for broken states”), and a set of tools to interact with the page. It tries things, clicks buttons, fills forms with weird inputs, navigates routes, and reports what it found: crashes, broken layouts, console errors, unexpected redirects. This complements **scripted regression tests**, which verify known scenarios, by covering the unknown unknowns: input combinations no human thought to test, navigation paths nobody tried, edge cases hidden in feature interactions. **The right place in a strategy**: as discovery before scripting. The agent explores; the human reviews findings; genuine bugs become tickets; reproducible scenarios become scripted regression tests. Replacing scripted regression with exploratory AI is the wrong move, you lose deterministic coverage of the cases that matter most. **Trap**: trusting the agent's reports without verification, false positives (things the agent flagged that are actually fine) waste time, and false negatives (things the agent missed) give false confidence.

INTERVIEW STRATEGY

You need to give exploratory AI calibrated against scripted testing, not treated as replacement. Lead with what the agent does (autonomous browser nav with a goal), then its place (discovery before scripting, complement to regression). The 'unknown unknowns' framing is the senior signal. The mistake is replacing scripted regression with exploratory AI. Expect the follow-up ‘how do you validate what the agent finds?’, answer reproducible findings become scripted tests; non-reproducible flagged for investigation.

How to phrase it

> *AI-driven exploratory testing uses an autonomous agent to navigate an application as a curious user would, discovering states the human test designer didn't think of. The agent has browser access via MCP or similar, a high-level goal like explore the checkout flow looking for broken states, and a set of tools to interact with the page. It tries things, clicks buttons, fills forms with weird inputs, navigates routes, and reports what it found: crashes, broken layouts, console errors, unexpected redirects. The important framing, and the part senior interviewers want to hear calibrated, is what this complements rather than what it replaces. It complements scripted regression tests, which verify known scenarios, by covering the unknown unknowns: input combinations no human thought to test, navigation paths nobody tried, edge cases hidden in feature interactions. A scripted test asks did the known scenario still work; an exploratory agent asks is there something broken we don't know about. Different questions, both valuable. The right place in a strategy is as discovery before scripting. The agent explores; the human reviews findings; genuine bugs become tickets; reproducible scenarios become scripted regression tests. So exploratory AI is a feeder into the scripted suite, not a replacement for it. Replacing scripted regression with exploratory AI is the wrong move because you lose deterministic coverage of the cases that matter most, the ones you already know to verify on every deploy. Exploratory is wonderful for discovery, terrible for verification, because it's not deterministic and you can't trust it to find the same bug twice. The trap I would close on is trusting the agent's reports without verification. False positives, things the agent flagged that are actually fine, waste team time investigating non-issues. False negatives, things the agent missed, give false confidence that the area is well-tested when actually the agent just didn't try the broken case. That's also my answer to the follow-up about validation: every agent finding gets a human reproducing it deterministically. If it reproduces, it's a bug: it becomes a ticket and a scripted regression test. If it doesn't, the finding is investigated but not trusted; either the agent was confused or the bug is genuinely flaky and that itself is worth knowing.*

Key points to hit

(1) Agent + browser + goal: explores autonomously, reports findings. **(2)** Covers unknown unknowns: input combos, paths, feature interactions. **(3)** Complements scripted regression: doesn't replace it. **(4)** Place: discovery before scripting, reproducible bugs become scripted tests. **(5)** Follow-up: every finding validated by human reproduction. **(6)** The common miss: replacing scripted regression with exploratory, lose deterministic coverage.

CODE

```
// Agent prompt for exploratory testing (Playwright MCP)
//
// You are an exploratory testing agent for acme.com checkout.
// Goal: find any state where the UI breaks, hangs, or shows console errors.
// Constraints: don't make actual payments; use test card 4000 0000 0000 0002
(declines).
// Tools: navigate, click, fill, screenshot, get console errors.
// For each finding, report:
// 1. Steps to reproduce
// 2. Expected vs actual
// 3. Console errors at the time
// 4. Screenshot of the broken state
//
// Output: a list of findings, each verified by a human:
// - Reproducible -> file as bug + add scripted regression test
// - Not reproducible -> investigate (intermittent? agent confusion?)
// - Not a bug -> dismiss but log for prompt-tuning
//
// Exploratory AI feeds the scripted suite; never replaces it.
```

Q 207 LEVEL · Senior

When is LLM-as-judge a valid assertion technique, and how do you keep it from being arbitrary?

CONCEPT

LLM-as-judge means using a language model to evaluate whether a piece of output meets a rubric, rather than comparing it to a fixed expected value. Useful when the output is **semantically variable but rule-bound**: an AI chatbot's response (should be polite, factual, on-topic), a marketing email (should mention three benefits without exaggeration), a code-review comment (should be constructive, specific). Strict string matching fails because two correct outputs can be word-for-word different; LLM-as-judge applies a rubric to both. The discipline that prevents arbitrary judgements: **rubrics with explicit criteria, not vibes**. Say “the response mentions at least one specific benefit and uses no superlatives,” not “the response is good.” Calibrate against a labelled dataset: hand-judge fifty outputs first, then test that the LLM judge agrees with the humans at least 80% of the time before trusting it in CI. The trap: using LLM-as-judge for things deterministic assertions handle, a function returning **{ ok: true }** doesn't need an LLM to verify, just toEqual. LLM-as-judge for outputs that legitimately vary; deterministic assertions for the rest. The 2026 use case growing fastest: testing applications that themselves use AI.

INTERVIEW STRATEGY

They are after LLM-as-judge calibrated, not adopted blindly. Lead with the right cases (semantically variable but rule-bound outputs), then the rubric + calibration discipline that prevents arbitrary judgements. The 'testing apps that themselves use AI' framing is the 2026 angle. The pitfall is LLM-as-judge for deterministic outputs. They usually follow with ‘how do you calibrate the judge?’, answer hand-label 50 outputs, test 80%+ agreement with humans before trusting it in CI.

How to phrase it

> *LLM-as-judge means using a language model to evaluate whether a piece of output meets a rubric, rather than comparing it to a fixed expected value. The useful case, and where it earns its place, is when the output is semantically variable but rule-bound. An AI chatbot's response to a user question: should be polite, factual, on-topic, but the exact words vary. A marketing email generated dynamically: should mention three benefits without exaggeration. A code-review comment from an AI tool: should be constructive, specific, actionable. Strict string matching fails for these because two correct outputs can be word-for-word different. LLM-as-judge applies a rubric to both and decides whether each meets it. The discipline that prevents arbitrary judgements, and where this technique stops being rigorous if you're sloppy, is rubrics with explicit criteria, not vibes. Say the response mentions at least one specific benefit and uses no superlatives, not the response is good. Specific criteria the judge can apply consistently, not vague qualities the judge interprets differently each call. And calibrate against a labelled dataset, which is my answer to the follow-up about calibration. Hand-judge fifty outputs first, then test that the LLM judge agrees with the humans at least eighty percent of the time before trusting it in CI. Without calibration you don't know if the judge is too lenient, too strict, or just inconsistent. With calibration you have a measured quality bar and can re-calibrate when the upstream model changes. The classic error is using LLM-as-judge for things deterministic assertions handle. A function returning an object curly-brace ok colon true doesn't need an LLM to verify, just toEqual. Spending tokens to ask an LLM whether ok is true is expensive, non-deterministic, and worse than the five-character assertion that does the same thing. The rule is LLM-as-judge for outputs that legitimately vary; deterministic assertions for the rest. The 2026 use case growing fastest, and the closer that lands as the forward-looking framing, is testing applications that themselves use AI. An app whose features include AI-generated content produces outputs that are different on every call. Deterministic assertions are the wrong tool. LLM-as-judge with a calibrated rubric is the right one. Test the AI-using app with an AI judge: same non-determinism problem, same technique.*

Key points to hit

(1) LLM-as-judge: model evaluates output against a rubric, not a fixed value. **(2)** Use when output is semantically variable but rule-bound (AI responses, dynamic email). **(3)** Rubric with explicit criteria: 'mentions one benefit, no superlatives', not 'good'. **(4)** Calibrate: hand-label 50, test 80%+ agreement before CI use. **(5)** 2026 growth case: testing apps that themselves use AI. **(6)** Avoid: LLM-as-judge for deterministic outputs, expensive, non-deterministic, wrong.

CODE

```
// LLM-as-judge with an explicit rubric (anthropic SDK)
import Anthropic from '@anthropic-ai/sdk';
const client = new Anthropic();

async function judgeChatbotResponse(userQuery: string, botResponse: string):
Promise<boolean> {
const prompt = `
Rubric (return JSON):
- on_topic: response addresses the user's question
- factual: no claims contradicting our docs
- polite: no aggressive or dismissive tone
- no_personal_data: no leaked PII

User query: ${userQuery}
Bot response: ${botResponse}

Return JSON: {"on_topic": bool, "factual": bool, "polite": bool, "no_personal_data":
bool}`;
const result = await client.messages.create({
model: 'claude-opus-4-7', max_tokens: 200,
messages: [{ role: 'user', content: prompt }],
});
const verdict = JSON.parse((result.content[0] as any).text);
return Object.values(verdict).every(Boolean);
}

// In a Playwright test
test('chatbot answers refund policy correctly', async ({ page }) => {
await page.goto('/chat');
await page.getByRole('textbox').fill('What is your refund policy?');
await page.getByRole('button', { name: 'Send' }).click();
const response = await page.getByTestId('bot-response').innerText();
expect(await judgeChatbotResponse('refund policy?', response)).toBe(true);
});

// Calibrate first: 50 labelled examples -> measure judge agreement -> trust in CI
```

Q 208 LEVEL · Senior

How do you protect a test suite from AI hallucinations and AI-generated bugs?

CONCEPT

AI-generated code carries failure modes deterministic code doesn't: hallucinated API signatures, incorrect-but-plausible selectors, made-up framework methods, hallucinated test data. Four lines of defence. **Compile-time validation**: strict TypeScript catches non-existent API calls and wrong signatures before code runs. Most hallucinations, a method that doesn't exist on Page, a property that doesn't exist on Locator, fail at compile time. Linting + ESLint rules: catch Playwright-specific anti-patterns (non-web-first assertions, missing awaits, wrong-shape getByRole calls). **Schema validation of test data**: if the AI invents request bodies or response shapes, JSON Schema validation against the actual API contract fails the test deterministically. Human review with a hallucination checklist: do all referenced selectors exist on the actual page? Do all API calls match the OpenAPI spec? Does test data match the schema? The principle: **defence in depth**. Each layer

catches a different category of hallucination, and no single layer is sufficient. The trap: relying on the AI to verify its own work, the same model that hallucinated the API method will confidently claim the method is correct. External verification is the only reliable check.

INTERVIEW STRATEGY

They want hallucination defence as a layered system. Walk the four layers (compile, lint, schema, human review). The 'defence in depth' framing is the senior signal. The 'AI can't verify its own work' warning lands. The failure mode is single-layer defence. The next question is normally ‘which layer catches the most?’, answer compile-time TypeScript catches the most hallucinated APIs; schema validation catches the most data-shape ones.

How to phrase it

> *AI-generated code carries failure modes deterministic code doesn't: hallucinated API signatures, incorrect-but-plausible selectors, made-up framework methods, hallucinated test data values. These look right on first read, the model produces syntactically valid TypeScript with plausible-sounding method names, but fail in specific patterns I need to defend against. Four lines of defence, and the framing I would lead with is defence in depth: each layer catches a different category of hallucination, and no single layer is sufficient. First, compile-time validation via strict TypeScript. This catches non-existent API calls and wrong signatures before code runs. Most hallucinations, a method that doesn't exist on Page, a property that doesn't exist on Locator, an option name the AI invented, fail at compile time. That's also my answer to the follow-up about which layer catches the most: TypeScript catches the most hallucinated APIs. Schema validation catches the most data-shape ones. Second, linting and ESLint rules tuned for Playwright. Catch non-web-first assertions, missing awaits, wrong-shape getByRole calls, expect-with-no-await. The playwright-eslint plugin handles most of these out of the box. Third, schema validation of test data. If the AI invents request bodies or response shapes, JSON Schema or zod validation against the actual API contract fails the test deterministically. The API expects an email field; the AI invented an emailAddress field; the schema catches it. Fourth, human review with a hallucination checklist. Do all referenced selectors exist on the actual page? Do all API calls match the OpenAPI spec? Does test data match the schema? Are there negative cases or did the AI only cover happy path? The checklist makes review systematic rather than vibes-based. The principle of defence in depth matters because each layer catches a category the others miss. TypeScript catches the API shape; linting catches the framework idioms; schemas catch the data; humans catch the coverage gaps. Together they're reliable; alone none is enough. The trap I would close on is relying on the AI to verify its own work. The same model that hallucinated the API method will confidently claim the method is correct when asked, the verification has the same hallucination risk as the generation. External verification is the only reliable check, which is why every layer of defence is a deterministic tool, not another AI prompt.*

Key points to hit

(1) Four-layer defence: TypeScript compile, ESLint, schema validation, human review. **(2)** TypeScript catches non-existent APIs (hallucinated methods/properties). **(3)** Schema validation catches data-shape hallucinations (wrong field names). **(4)** Human checklist: selectors exist? APIs match OpenAPI? data shape valid? **(5)** Expect next: TypeScript catches most API hallucinations; schemas catch most data ones. **(6)** The easy mistake: AI verifying its own work, same hallucination risk as

generation.

CODE

```
// Defence layer 1: strict TypeScript, hallucinated APIs fail at compile time
// tsconfig.json: "strict": true, "noImplicitAny": true, "strictNullChecks": true
// await page.getByRole('button').clickIt(); <- TS2339: clickIt does not exist on
Locator

// Defence layer 2: ESLint catches framework idiom hallucinations
// .eslintrc.json:
// "plugins": ["playwright"],
// "rules": {
// "playwright/no-wait-for-timeout": "error",
// "playwright/no-conditional-in-test": "warn",
// "playwright/prefer-web-first-assertions": "error",
// "playwright/missing-playwright-await": "error",
// }

// Defence layer 3: schema validation of test data, hallucinated shapes fail at runtime
const UserRequestSchema = z.object({ email: z.string().email(), role:
z.enum(['admin','user']) });
const body = UserRequestSchema.parse(aiGeneratedBody); // throws on shape mismatch

// Defence layer 4: human-review checklist (PR template)
// [ ] All getByRole/getByLabel/getByTestId selectors verified against live page
// [ ] All API endpoints match the OpenAPI spec
// [ ] All test data conforms to schemas
// [ ] Negative cases present (not just happy path)
// [ ] Tests are independent (no shared state)
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What is the professional framing for AI in test automation?

A) An accelerator paired with the expertise to evaluate its output
B) A full replacement for SDETs
C) A tool to avoid
D) Only useful for visual testing

2. What most improves LLM-generated test coverage?

A) A longer prompt
B) Asking for an explanation
C) Explicit acceptance criteria in the prompt
D) Using GPT over Claude

3. How should a healed self-healing locator be treated?

A) As a permanent fix: no action needed
B) As technical debt: the logged warning becomes a ticket to fix the primary
C) As a test failure
D) As a reason to delete the test

4. Which three inputs let an LLM diagnose a failing test's root cause?

A) Screenshot, video, trace
B) Config, fixtures, and reporter
C) Test name, author, and date
D) Error message, test code, and the recent diff

5. Which AI-generated-test mistakes can ESLint catch automatically?

A) Test independence and negative cases
B) Locator naming and comments
C) Missing awaits (no-floating-promises) and assertion type (prefer-web-first-assertions)
D) None: all review is manual

Section 15 · Answer key

#	Answer	Why and where to revisit
1	A) An accelerator paired with the expertise to evaluate its output	AI scaffolds quickly; the engineer's prompt skill and review judgement keep the quality bar. *Revisit Section 15.*
2	C) Explicit acceptance criteria in the prompt	Without acceptance criteria the LLM produces happy-path only; with them it covers ~80% of what you'd write. *Revisit Section 15.*
3	B) As technical debt: the logged warning becomes a ticket to fix the primary	Healing is a stopgap that keeps CI green; the logged heal should be scheduled as a primary-locator fix. *Revisit Section 15.*
4	D) Error message, test code, and the recent diff	Correlating the error with the recent diff lets the model reason about cause (e.g. a newly added button). *Revisit Section 15.*
5	C) Missing awaits (no-floating-promises) and assertion type (prefer-web-first-assertions)	ESLint auto-catches awaits and assertion type, freeing manual review for independence and negative coverage. *Revisit Section 15.*

SECTION 16

Test Design Techniques

Test design is what separates someone who writes tests from someone who decides which tests are worth writing, and interviewers use it to gauge whether you can justify coverage rather than just chase it. These eight questions cover equivalence partitioning, boundary value analysis, decision tables, state transition testing, and how to shape a test pyramid for a Playwright framework so coverage is high and the CI run stays fast.

In this section

- Equivalence partitioning: one value per partition, coverage you can justify.
- Boundary value analysis: min-1/min/min+1 catches off-by-one bugs.
- Decision tables: every business-rule combination, traceable to product.
- State transition testing: valid transitions plus rejected invalid ones.
- A Playwright test pyramid: 30/30/30/10, ~95% coverage in a fast CI run.

Q 209 LEVEL · Junior to Mid

How do you apply Equivalence Partitioning in Playwright tests?

CONCEPT

Equivalence Partitioning divides input data into **groups where every value behaves identically**, so you test **one value per partition** instead of every possible value. For an age field valid 0-120, the partitions are below-min (<0), valid (0-120), and above-max (>120), you pick one representative from each rather than testing 121 integers. In Playwright this is a parameterised test: an array of cases, each with its input, expected outcome, and a **partition name in the test title** so the report shows which partition each test covers. That naming makes coverage gaps visible at a glance. The technique is how you justify coverage to stakeholders: equivalent confidence to exhaustive testing at a fraction of the test count.

INTERVIEW STRATEGY

You are being asked to show you justify coverage, not just produce tests. Define partitioning quickly, then make the stakeholder argument concrete with numbers, 5 partition tests give the same confidence as 121 exhaustive ones. Putting the partition name in the test title is the practical detail that shows you make coverage visible. The trap is testing many values inside one partition, which adds no coverage. A common follow-up is 'how do you choose the representative value?', answer any value works since the partition is defined to behave identically.

How to phrase it

> *Equivalence partitioning divides input into groups where every value behaves the same, so I test one representative per partition instead of every value. The way I would pitch it, because it is really about justifying coverage, is with numbers: for an age field valid from zero to a hundred and twenty, testing every integer is a hundred and twenty-one tests, but testing one value from each partition, below minimum, valid, above maximum, gives equivalent coverage in a handful of tests. In Playwright I implement it as a parameterised test: an array of cases each with an input, an expected outcome, and the partition name in the test title, so the report shows which partition each test covers and any gap is visible at a glance. That visibility is the point: a stakeholder can see exactly which input classes are exercised. For the follow-up about choosing the representative value, the answer is that any value in the partition works, because the partition is defined as the set that behaves identically, so I usually pick a clean mid-range value, and I leave the exact edges to boundary value analysis. The trap I avoid is testing lots of values inside the same partition, which feels thorough but adds zero real coverage. Partitioning is how I get defensible coverage without a combinatorial test count.*

Key points to hit

(1) Partition inputs into groups that behave identically; test one each. **(2)** Numbers sell it: ~5 partition tests vs 121 exhaustive, equal confidence. **(3)** Parameterised test with the partition name in the title, gaps visible. **(4)** Justifies coverage to stakeholders without exhaustive testing. **(5)** They usually ask next: any value in a partition works since it behaves identically. **(6)** Trap: testing many values inside one partition for no extra coverage.

CODE

```
const ageCases = [
{ age: -1, expect: 'error', partition: 'below-min' },
{ age: 25, expect: 'success', partition: 'valid' },
{ age: 121, expect: 'error', partition: 'above-max' },
];
for (const { age, expect: outcome, partition } of ageCases) {
test(`age validation, ${partition}: ${age}`, async ({ page }) => {
await page.goto('/profile');
await page.getByLabel('Age').fill(String(age));
await page.getByRole('button', { name: 'Save' }).click();
if (outcome === 'success') await expect(page.locator('.success-message')).toBeVisible();
else await expect(page.locator('.field-error')).toBeVisible();
});
}
```

Q 210 LEVEL · Junior to Mid

How do you apply Boundary Value Analysis in Playwright?

CONCEPT

Boundary Value Analysis tests the **exact boundary and its immediate neighbours**, the spots where off-by-one errors live. For a password length of min 8, max 64, you test 7, 8, 9 and **63, 64, 65** (min-1, min, min+1; max-1, max, max+1). The reasoning: a developer implementing 'minimum 8' might write length > 8 instead of **length >= 8**, and only the test at exactly 8 catches that. It pairs naturally with equivalence partitioning, partitioning picks the classes, BVA hardens the edges between them. The one-line framing: the boundary is where the condition changes, and conditions are where bugs live, so that is exactly where to aim tests.

INTERVIEW STRATEGY

What earns the tick is the reasoning, not just the recipe. Give the min-1/min/min+1 pattern, then the why with the concrete > vs >= bug that only the exact-boundary test catches. The pairing with partitioning shows you see them as a system. The classic error is testing only mid-range values and missing the edges where bugs cluster. The follow-up is often 'why both boundary and partition?', answer partitioning covers the classes, BVA hardens the transition points between them.

How to phrase it

> *Boundary value analysis tests the exact boundary and its immediate neighbours, because that is where off-by-one errors live. For a password length with a minimum of eight and a maximum of sixty-four, I test seven, eight and nine at the low end, and sixty-three, sixty-four and sixty-five at the high end, min-1, min, min+1 and the same at the max. The reasoning is what I would emphasise, with a concrete bug: a developer implementing minimum eight might write length greater than eight instead of greater than or equal to eight, and the only test that catches that is the one with exactly eight characters, a mid-range value of, say, twenty would pass either way and hide the bug. The phrasing I use in interviews is that the boundary is where the condition changes, and conditions are where bugs live, so that is exactly where to aim. That also answers the follow-up about why I use both boundary and partition analysis: equivalence partitioning picks the representative classes so I am not testing every value, and BVA hardens the edges between those classes where off-by-one mistakes cluster. The risk is testing only comfortable mid-range values, which feels safe but skips precisely the inputs most likely to expose a bug. Together the two techniques give me confident validation coverage with very few tests.*

Key points to hit

(1) Test min-1/min/min+1 and max-1/max/max+1: the boundary neighbours. **(2)** Catches > vs >= off-by-one that mid-range values miss. **(3)** One-liner: the boundary is where the condition changes, where bugs live. **(4)** Pairs with partitioning: classes from EP, edges hardened by BVA. **(5)** The next question: why both, EP covers classes, BVA covers the transitions. **(6)** Where this goes wrong: testing only mid-range values and skipping the edges.

CODE

```
// Password length: min 8, max 64
const bva = [
{ len: 7, valid: false, label: 'below-min' },
{ len: 8, valid: true, label: 'min-boundary' },
{ len: 9, valid: true, label: 'min-plus-one' },
{ len: 64, valid: true, label: 'max-boundary' },
{ len: 65, valid: false, label: 'above-max' },
];
for (const { len, valid, label } of bva) {
test(`password BVA: ${label} (${len})`, async ({ page }) => {
await page.goto('/register');
await page.getByLabel('Password').fill('A'.repeat(len - 2) + '1!');
await page.getByRole('button', { name: 'Register' }).click();
if (valid) await expect(page.locator('.password-error')).toBeHidden();
else await expect(page.locator('.password-error')).toBeVisible();
});
}
```

Q 211 LEVEL · Mid

How do you implement Decision Table testing in Playwright?

CONCEPT

A decision table maps **combinations of conditions to expected outcomes**, ensuring every meaningful combination is tested without combinatorial explosion. For discount rules driven by membership tier and order total, the table enumerates each condition combination and its expected discount, and each row becomes one parameterised test. The real value beyond coverage is **communication**: you present the table to product managers, conditions as columns, outcomes as rows, and they verify that every business-rule combination is covered. When a new rule is added, you add a row and a test. This makes test coverage **traceable to business requirements**, which is an SDET skill well beyond writing the test code itself.

INTERVIEW STRATEGY

What scores here is showing you handle business-rule combinations systematically and traceably. Describe the table-to-test mapping, then take it to the communication angle: a PM can read the table and confirm coverage. That traceability-to-requirements point is the senior signal. Where this goes wrong is testing rule combinations ad hoc and missing one. Expect the follow-up 'how do you keep coverage in sync when a rule changes?', answer add a row to the table and a matching test, so coverage tracks requirements.

How to phrase it

> *A decision table maps combinations of conditions to expected outcomes, so I cover every meaningful combination without combinatorial explosion. For discount rules driven by membership tier and order total, I enumerate each combination and its expected discount, and each row becomes one parameterised test. But the part I would emphasise, because it is what makes this a senior technique rather than just a loop, is communication. I present the table to product managers with conditions as columns and outcomes as rows, and they can verify directly that every business-rule combination is covered, they do not have to read test code to trust the coverage. That answers the follow-up about keeping coverage in sync when a rule changes: when a new discount rule is added, I add a row to the table and a matching test, so the coverage tracks the requirements one-to-one and a gap is obvious because it is a missing row. This makes test coverage traceable to business requirements, which is an SDET skill well beyond writing test code, it turns the test suite into something a non-technical stakeholder can audit. The thing to avoid is testing rule combinations ad hoc, where it is easy to silently miss one combination and never notice. The table makes completeness visible and the missing case loud.*

Key points to hit

(1) Map condition combinations to outcomes; each row is one test. **(2)** Covers all meaningful combinations without combinatorial explosion. **(3)** Communication: PMs verify coverage from the table directly. **(4)** New rule -> add a row and a test; coverage tracks requirements. **(5)** Asked next: keeping coverage in sync, one row per rule, gaps are obvious. **(6)** What loses the point: testing combinations ad hoc and silently missing one.

CODE

```
// premium + >$100 = 20% | premium + <=$100 = 10%
// regular + >$100 = 5% | regular + <=$100 = 0%
const discountTable = [
{ premium: true, total: 150, discount: '20%' },
{ premium: true, total: 50, discount: '10%' },
{ premium: false, total: 150, discount: '5%' },
{ premium: false, total: 50, discount: '0%' },
];
for (const { premium, total, discount } of discountTable) {
test(`discount: ${premium ? 'premium' : 'regular'} + $${total} = ${discount}`, async ({
page, request }) => {
await new UserBuilder().withRole(premium ? 'premium' : 'user').create(request);
await page.goto(`/checkout?total=${total}`);
await expect(page.locator('#discount-display')).toHaveText(discount);
});
}
```

Q 212 LEVEL · Mid

How do you apply State Transition testing in Playwright?

CONCEPT

State Transition testing validates that an entity moves through its **valid states correctly** and that **invalid transitions are rejected**. For an order machine Draft to Submitted to Confirmed to Shipped to Delivered, you test each valid step and also that an illegal jump, Draft straight to Shipped, is refused with the right error and status (a 422). The discipline: draw the state diagram first, then write a test for every valid transition, every invalid transition, and every terminal state. The invalid-transition tests are often the most valuable because they verify the application enforces business rules, not just the happy path, they catch the case where the API allows a transition the UI prevents, which is a genuine security and data-integrity gap.

INTERVIEW STRATEGY

This question is there to see you test lifecycles rigorously, including what should be forbidden. Lead with the draw-the-diagram-first discipline, then stress that the invalid-transition tests are the high-value ones, they verify rule enforcement, not the happy path. The API-allows-what-the-UI-prevents security gap is the war-story detail. The thing to avoid is testing only valid transitions. They usually follow with ‘why test invalid transitions?’, answer they catch the API accepting an illegal jump the UI blocks, a data-integrity hole.

How to phrase it

> *State transition testing checks that an entity moves through its valid states correctly and that invalid transitions are rejected. For an order machine, Draft, Submitted, Confirmed, Shipped, Delivered, I test each valid step, and I also test that an illegal jump like Draft straight to Shipped is refused, with the right error message and a 422 status. My discipline is to draw the state diagram first, then write a test for every valid transition, every invalid transition, and every terminal state, so I am working from the machine rather than guessing. The part I emphasise is that the invalid-transition tests are often the most valuable, which is also my answer to the follow-up about why test them at all: they verify that the application actually enforces the business rules, not just that the happy path works. I have caught genuine bugs this way where the API allowed an invalid state transition that the UI happened to prevent, so the button was hidden, but a direct API call could ship a draft order, which is a real data-integrity and security gap. The happy-path tests would never have found that because the UI looked correct. The mistake is testing only the valid transitions and assuming the invalid ones are handled; the invalid cases are exactly where the enforcement bugs hide, so I treat them as first-class tests, not an afterthought.*

Key points to hit

(1) Test valid transitions and that invalid ones are rejected (right error + 422). **(2)** Discipline: draw the diagram, then cover every transition and terminal state. **(3)** Invalid-transition tests verify rule enforcement, not the happy path. **(4)** Catches the API allowing a jump the UI prevents, a real security gap. **(5)** Follow-up: why test invalid transitions, data-integrity enforcement. **(6)** The common miss: testing only valid transitions.

CODE

```
test('order follows valid transitions', async ({ request, page }) => {
const order = await new OrderBuilder().asDraft().create(request);
await request.post(`/api/orders/${order.id}/submit`);
await page.goto(`/orders/${order.id}`);
await expect(page.locator('#order-status')).toHaveText('Submitted');
});

test('cannot ship a draft order: invalid transition', async ({ request }) => {
const order = await new OrderBuilder().asDraft().create(request);
const res = await request.post(`/api/orders/${order.id}/ship`);
expect(res.status()).toBe(422);
expect((await res.json()).error).toContain('Cannot ship an order in Draft status');
});
```

Q 213 LEVEL · Mid to Senior

How do you design a test pyramid for a Playwright-based framework?

CONCEPT

The test pyramid guides how tests are distributed across layers: more at the lower, faster, cheaper layers, fewer at the higher, slower ones. A practical Playwright distribution is roughly **30% unit** (utilities, data transforms, business logic, fastest), **30% component** (UI components in isolation, real browser, no full app), 30% API integration (contract, CRUD, error states, fast, no browser), and **10%**

E2E (only the critical journeys: login, checkout, core CRUD, slow but high-confidence). The reasoning is cost and confidence: cover everything possible at a fast layer, reserve expensive E2E for the flows that immediately impact revenue if broken. Done well this yields high coverage with a CI run measured in minutes, not the hour an E2E-heavy suite would take.

INTERVIEW STRATEGY

You need to give a deliberate distribution with reasoning, not 'write lots of tests'. Give concrete percentages and what each layer covers, then justify by cost and confidence: push coverage down to fast layers, reserve E2E for revenue-critical journeys. Quantifying the CI run (minutes vs an hour) lands it. The mistake is an inverted pyramid: mostly slow E2E tests. The next question is normally 'what goes in the 10% E2E?', answer login, checkout, core CRUD: the flows that hit revenue immediately if broken.

How to phrase it

> *The test pyramid guides how I distribute tests across layers, more at the lower, faster, cheaper layers and fewer at the slow, expensive top. For a Playwright framework my practical split is roughly thirty percent unit tests for utilities, data transformations and business logic, which are the fastest; thirty percent component tests for UI components in isolation, real browser but no full app; thirty percent API integration tests for contract, CRUD and error states, fast because there is no browser; and ten percent end-to-end tests. The reasoning, which is the part that matters, is cost versus confidence: I cover everything I can at a fast layer and reserve the expensive E2E tests for the flows that immediately impact revenue if they break. That answers the follow-up about what goes in the ten percent: login, checkout and the core CRUD operations, the journeys where a failure directly costs money, so they are worth the slow, high-confidence full-stack test. Everything else is covered faster and cheaper below. The mistake is the inverted pyramid, a suite that is mostly slow E2E tests, which is brittle and takes an hour to run, so developers stop running it. With this distribution I get around ninety-five percent coverage with a CI run in the low double-digit minutes rather than an hour, which keeps the feedback loop fast enough that the team actually relies on it.*

Key points to hit

(1) Distribution: ~30% unit, 30% component, 30% API, 10% E2E. **(2)** Lower layers are faster/cheaper; push coverage down the pyramid. **(3)** Reserve E2E for revenue-critical journeys (login, checkout, core CRUD). **(4)** ~95% coverage with a CI run in minutes, not an hour. **(5)** Expect next: the 10% E2E is the flows that hit revenue if broken. **(6)** Avoid: an inverted pyramid that is mostly slow E2E tests.

CODE

```
// Target distribution for a Playwright framework
// Unit ~30% utilities, data transforms, business logic (fastest)
// Component ~30% UI components in isolation (real browser, no full app)
// API ~30% contract, CRUD, error states (fast, no browser)
// E2E ~10% critical journeys only: login, checkout, core CRUD
//
// playwright.config.ts: separate projects per layer
// projects: [
// { name: 'api', testDir: './tests/api' },
// { name: 'component', testDir: './tests/component' },
// { name: 'e2e', testDir: './tests/e2e', use: { ...devices['Desktop Chrome'] } },
// ]
```

Q 214 LEVEL · Mid to Senior

How do you apply pairwise testing to features with many input parameters?

CONCEPT

When a feature has multiple input parameters, the number of combinations grows multiplicatively, 5 browsers × 4 plans × 3 countries × 4 payment methods = 240 combinations. Testing all 240 is infeasible; testing just a few leaves coverage gaps. **Pairwise testing (combinatorial testing, orthogonal array)** is the mathematical answer: cover every pair of values from any two parameters in at least one test, which catches most multi-parameter bugs with dramatically fewer test cases. The 240-case example reduces to about 20 cases that achieve pairwise coverage. The justification, backed by research from NIST: most defects in multi-parameter features are triggered by single values or pairs, not by three-way or higher interactions. Triple-wise coverage exists but the additional defect-catching power rarely justifies the test explosion. Tools: **AllPairs**, **PICT** (Microsoft), pict-node for TypeScript-integrated generation. The discipline: identify parameters and values, generate the pairwise matrix, run the matrix as parameterised tests. The trap: treating pairwise as a replacement for boundary and equivalence testing. Pairwise covers the combination space; you still need BVA and EP within each parameter.

INTERVIEW STRATEGY

They are after combinatorial testing understood mathematically, not waved at. Lead with the parameter-explosion problem (240 cases), then pairwise as the answer (20 cases for all-pairs coverage). The NIST research backing is what separates junior from senior thinking. The 'pairwise complements BVA/EP, doesn't replace them' rule lands. The failure mode is pairwise instead of, not in addition to. A common follow-up is ‘what tool do you use?’, answer PICT for general use, pict-node for TypeScript test generation.

How to phrase it

> *When a feature has multiple input parameters, the number of combinations grows multiplicatively. Five browsers times four subscription plans times three countries times four payment methods is two hundred forty combinations. Testing all two hundred forty is infeasible; testing just a few leaves coverage gaps where multi-parameter bugs hide. Pairwise testing, also called combinatorial testing or orthogonal array testing, is the mathematical answer. It covers every pair of values from any two parameters in at least one test, which catches most multi-parameter bugs with dramatically fewer test cases. The two-hundred-forty-case example reduces to about twenty cases that achieve pairwise coverage, more than a ten-times reduction with minimal coverage loss. The justification, and what makes this rigorous rather than convenient, is NIST research showing that most defects in multi-parameter features are triggered by single values or pairs of values, not by three-way or higher interactions. So pairwise coverage catches the high-probability defect space; three-way and beyond catch a long tail that rarely justifies the test explosion. The tooling is mature. AllPairs as the classic open-source generator. PICT, Microsoft's pairwise tool, more configurable. pict-node for TypeScript-integrated test generation, my preferred choice because it produces the test matrix in JavaScript objects that Playwright can iterate over directly. When they push for the follow-up about which tool: PICT for general use, pict-node when I want the matrix directly in TypeScript. The discipline is identify parameters and their values, generate the pairwise matrix, run the matrix as parameterised tests. The matrix becomes a data-driven loop where each row is one test case. The trap I would emphasise, and where most candidates lose marks on this question, is treating pairwise as a replacement for boundary and equivalence testing. Pairwise covers the combination space across parameters; BVA and EP cover the value space within each parameter. They're orthogonal. I still need to test boundary values for each numeric input, zero, max, max-plus-one, within the pairwise matrix. Pairwise tells me which combinations to test; EP and BVA tell me which values to test within each parameter. Both techniques applied together produce comprehensive coverage; either alone leaves significant gaps.*

Key points to hit

(1) Parameter explosion: 5×4×3×4 = 240 combinations, infeasible to test all. **(2)** Pairwise covers every value-pair across any two parameters, 240 -> ~20. **(3)** NIST research: most multi-param defects are 1- or 2-way, not higher. **(4)** Tools: AllPairs (classic), PICT (Microsoft), pict-node (TS-integrated). **(5)** Generate matrix -> run as parameterised tests, data-driven loop. **(6)** The easy mistake: pairwise instead of BVA/EP, complementary, not replacement.

CODE

```
// pict-node generates the pairwise matrix for a multi-parameter feature
import { pict } from 'pict-node';

const cases = await pict({
model: [
{ key: 'browser', values: ['chrome', 'firefox', 'safari', 'edge', 'opera'] },
{ key: 'plan', values: ['free', 'basic', 'pro', 'enterprise'] },
{ key: 'country', values: ['US', 'UK', 'IN'] },
{ key: 'payment', values: ['card', 'paypal', 'bank', 'crypto'] },
],
// generates ~20 cases covering all pairs (vs 240 for full cartesian)
});

for (const c of cases) {
test(`signup: ${c.browser}/${c.plan}/${c.country}/${c.payment}`, async ({ page }) => {
await page.goto(`/signup?plan=${c.plan}&country=${c.country}`);
await selectPaymentMethod(page, c.payment);
await page.getByRole('button', { name: 'Complete signup' }).click();
await expect(page.getByText('Welcome')).toBeVisible();
});
}

// Still apply BVA/EP within each parameter: test plan='pro' with seats=1, max, max+1
```

Q 215 LEVEL · Senior

What is risk-based test design, and how do you decide where to invest test coverage?

CONCEPT

Risk-based test design **allocates test effort proportionally to risk**, where risk = probability of failure × impact of failure. High-impact, high-probability areas get deep coverage; low-impact, low-probability areas get minimal. The matrix has four quadrants: high-impact + high-probability = comprehensive coverage (checkout flow on a payment app, login on any product); high-impact + low-probability = important regression tests but not exhaustive (disaster recovery flows); low-impact + high-probability = monitor for trends but don't deep-test (UI cosmetic glitches); **low-impact + low-probability** = deprioritise (rare edge cases on internal tools). The inputs to assess risk: incident history (where have bugs actually happened?), code churn (the files changing fastest carry the highest risk of new defects), business criticality (revenue paths versus internal admin), complexity (cyclomatic complexity, code review heat maps). The discipline that distinguishes risk-based from preference-based: **quantify the inputs, don't intuit them**. The trap: treating risk-based as an excuse to under-test areas the team finds boring, risk has to be evidence-based, not vibes-based.

INTERVIEW STRATEGY

They want risk-based design applied rigorously, not vaguely. Lead with the probability×impact formula, walk the four-quadrant matrix, name the inputs (incident history, code churn, business criticality, complexity). The 'quantify inputs, don't intuit' rule is the senior signal. Where this goes wrong is under-testing 'boring' areas the team rationalises as low-risk. The follow-up is often ‘what's your strongest signal?’, answer incident history; past failures predict future failures.

How to phrase it

> *Risk-based test design allocates test effort proportionally to risk, where risk equals probability of failure times impact of failure. High-impact, high-probability areas get deep coverage; low-impact, low-probability areas get minimal. The framework, and what makes it actionable rather than philosophical, is the four-quadrant matrix. High-impact plus high-probability: comprehensive coverage. The checkout flow on a payment app: fails often if not protected, costs the business directly when it does. Login on any product: everyone hits it, everything depends on it. High-impact plus low-probability: important regression tests but not exhaustive. Disaster recovery flows, the rarely-triggered emergency paths, they matter when they fire but they don't fire often. Low-impact plus high-probability: monitor for trends but don't deep-test. UI cosmetic glitches that occur frequently but don't block users. Low-impact plus low-probability: deprioritise. Rare edge cases on internal admin tools used by two people once a quarter. The inputs to assess risk, and where most teams skip the rigour, are four. Incident history: where have bugs actually happened in the past six to twelve months? That's also my answer to the follow-up about strongest signal: incident history. Past failures predict future failures more reliably than any other signal because they reveal both probability and the real impact pattern. Code churn: the files changing fastest carry the highest risk of new defects, a module with twenty commits this sprint is more dangerous than one with two. Business criticality: revenue paths versus internal admin, customer-facing versus back-office. Complexity: cyclomatic complexity, code review heat maps, the parts of the code where developers themselves disagree about behaviour. The discipline I would lead with, because it's where risk-based design goes wrong, is quantify the inputs, don't intuit them. Pull incident reports, run churn analysis via git log, query the bug tracker by module. Numbers, not gut feel. The trap is treating risk-based as an excuse to under-test areas the team finds boring, configuration screens, admin tools, settings pages. The team rationalises them as low-risk because nobody wants to write the tests, then a config bug ships and the rationalisation looks embarrassing in the post-mortem. Risk has to be evidence-based, not vibes-based. If incident history shows three production bugs in the settings screen last year, that's a high-risk area regardless of how boring it is to test.*

Key points to hit

(1) Risk = probability of failure × impact of failure, allocate effort accordingly. **(2)** Four-quadrant matrix: high/low impact × high/low probability. **(3)** Inputs: incident history (strongest), code churn, business criticality, complexity. **(4)** Quantify each input from real data: git log, bug tracker, complexity tools. **(5)** They usually ask next: incident history is the strongest signal, past predicts future. **(6)** Trap: rationalising boring areas as low-risk, evidence not vibes.

CODE

```
// Risk-based prioritisation pipeline (conceptual)
//
// 1. Pull incident history (last 12 months)
// SELECT module, COUNT(*) AS incidents
// FROM production_incidents
// WHERE created_at > NOW() - INTERVAL '12 months'
// GROUP BY module ORDER BY 2 DESC;
//
// 2. Pull code churn
// git log --since='3 months ago' --pretty=format: --name-only | sort | uniq -c | sort
-rn
//
// 3. Score each area: impact (1-5) × probability (1-5)
// risk_score = (incident_count * 0.5) + (churn_rank * 0.3) + (business_critical * 0.2)
//
// 4. Allocate test depth by quadrant:
// risk >= 20 -> comprehensive coverage (every flow, BVA, EP, negative, security)
// risk 10-19 -> happy + critical edges only
// risk 5-9 -> smoke + regression
// risk < 5 -> monitor; don't gate releases on it
//
// 5. Revisit quarterly; risk shifts as features change
```

Q 216 LEVEL · Mid to Senior

What is scenario-based testing, and how does it differ from feature-based testing?

CONCEPT

Feature-based testing organises around the application's structure, a test file per feature, tests grouped by component. Scenario-based testing organises around **user journeys that span multiple features**, a test that represents what a real user actually does in one session. The contrast: a feature test for the search feature verifies search works in isolation; a scenario test for the 'new customer browses, signs up, makes first purchase, leaves a review' journey verifies the entire flow works end-to-end. The scenario test catches integration bugs that feature tests miss, data flowing wrong between steps, state leaking across boundaries, side effects interacting. The discipline that makes scenarios useful: derive them from real user journeys, not from feature lists. Analytics data, customer support tickets, user research studies, sales call transcripts, these surface what users actually do. The right place in a strategy: feature tests as the broad coverage layer; a small number of scenario tests for the critical journeys. Five scenario tests for the top user journeys plus comprehensive feature coverage is the right ratio. The trap: **only scenarios, no features**, you miss the isolated-feature edge cases. Or **only features, no scenarios**: you miss the integration bugs.

INTERVIEW STRATEGY

What earns the tick is scenario vs feature testing calibrated, not picked. Define both, then make 'feature tests for coverage breadth; scenario tests for critical journeys' the senior framing. 'Derive scenarios from real user data' is where this gets rigorous. The risk is one without the other. Expect the follow-up 'how many scenarios?', answer five to ten for the top user journeys; more becomes a maintenance burden.

How to phrase it

> *Feature-based testing organises around the application's structure, a test file per feature, tests grouped by component. The search feature gets its own test suite, the cart feature gets its own, the auth feature gets its own. Each verifies its feature works in isolation. Scenario-based testing organises around user journeys that span multiple features, a test that represents what a real user actually does in one session. New customer browses, finds a product, signs up, makes their first purchase, leaves a review. That's one scenario, spanning five or six features, verified as one flow. The contrast that lands concretely: a feature test for search verifies search works in isolation, type a query, results appear, click a result, the right page loads. A scenario test for the new-customer journey verifies the entire flow works end-to-end, including the interactions between features. The scenario test catches integration bugs that feature tests miss, data flowing wrong between steps, state leaking across boundaries, side effects interacting in ways isolated tests don't see. The discipline that makes scenarios useful, and where they go wrong when teams skip this part, is derive them from real user journeys, not from feature lists. Analytics data showing what users actually do, customer support tickets describing the flows users have trouble with, user research studies where you watched real users use the product, sales call transcripts where prospects ask about specific journeys. These surface what users actually do, not what the product manager planned for or what the engineers built. The right place in a strategy is feature tests as the broad coverage layer, running fast and exhaustively against every feature; a small number of scenario tests for the critical journeys, running slower but covering what actually matters to users. That's also my answer to the follow-up about how many scenarios: five to ten for the top user journeys. More than that becomes a maintenance burden: every feature change risks breaking multiple scenarios, each scenario takes non-trivial time to debug. Fewer scenarios, kept current and comprehensive, beat many scenarios kept inconsistently. The thing to avoid is one approach without the other. Only scenarios, no feature tests: you miss the isolated-feature edge cases, the search-with-empty-query test that scenarios don't naturally cover. Only features, no scenarios: you miss the integration bugs that emerge only when the features interact, exactly the bugs that hit production with the highest visibility because users actually exercise the integrations.*

Key points to hit

(1) Feature: per-component tests; scenario: end-to-end user journeys. **(2)** Scenarios catch integration bugs: features miss them. **(3)** Derive from real journeys: analytics, support tickets, research, sales calls. **(4)** Strategy: features for breadth, 5-10 scenarios for critical journeys. **(5)** The next question: 5-10 scenarios, more becomes maintenance burden. **(6)** Where this goes wrong: one without the other, miss edge cases OR integration bugs.

CODE

```
// FEATURE TEST: isolated, comprehensive for one capability
test.describe('search feature', () => {
test('returns matching results', async ({ page }) => { /* ... */ });
test('handles empty query', async ({ page }) => { /* ... */ });
test('paginates beyond 20 results', async ({ page }) => { /* ... */ });
test('filters by category', async ({ page }) => { /* ... */ });
});

// SCENARIO TEST: end-to-end journey across multiple features
test('new-customer journey: browse -> signup -> buy -> review', async ({ page,
userBuilder }) => {
// 1. Browse as anonymous
await page.goto('/');
await page.getByPlaceholder('Search').fill('headphones');
await page.getByRole('button', { name: 'Search' }).click();
await page.getByRole('link', { name: /Sony WH/ }).click();
// 2. Sign up (auth feature integration)
await page.getByRole('button', { name: 'Buy now' }).click();
const user = await userBuilder.unsavedDetails();
await page.getByLabel('Email').fill(user.email);
await page.getByLabel('Password').fill(user.password);
await page.getByRole('button', { name: 'Create account' }).click();
// 3. First purchase (checkout + payment feature integration)
await page.getByLabel('Card number').fill('4242 4242 4242 4242');
await page.getByRole('button', { name: 'Place order' }).click();
await expect(page.getByText('Order confirmed')).toBeVisible();
// 4. Leave review (post-purchase feature)
await page.getByRole('link', { name: 'Leave a review' }).click();
await page.getByRole('radio', { name: '5 stars' }).check();
await page.getByRole('button', { name: 'Submit review' }).click();
await expect(page.getByText('Review posted')).toBeVisible();
});
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What is the core idea of Equivalence Partitioning?

A) Test every possible input value
B) Group inputs that behave identically and test one value per group
C) Test only boundary values
D) Randomly sample inputs

2. Why does Boundary Value Analysis test min-1, min, and min+1?

A) To test more values for thoroughness
B) To satisfy code coverage
C) Because the middle is never buggy
D) Because off-by-one errors (> vs >=) live exactly at the boundary

3. What is the main benefit of a decision table beyond coverage?

A) It makes business-rule coverage traceable and verifiable by product managers
B) It runs tests faster
C) It removes the need for assertions
D) It avoids parameterised tests

4. Which state transition tests are often the most valuable?

A) Valid transitions only
B) Invalid transitions: they verify the app enforces business rules
C) Terminal states only
D) Transitions the UI never shows

5. What is a sound test-pyramid distribution for a Playwright framework?

A) Mostly E2E tests for maximum confidence
B) 50% E2E, 50% unit
C) ~30% unit, 30% component, 30% API, 10% E2E (critical journeys only)
D) Only E2E and unit, no API

Section 16 · Answer key

#	Answer	Why and where to revisit
1	B) Group inputs that behave identically and test one value per group	One representative per partition gives coverage equivalent to exhaustive testing at a fraction of the count. *Revisit Section 16.*
2	D) Because off-by-one errors (> vs >=) live exactly at the boundary	Only the exact-boundary test distinguishes length > 8 from length >= 8, where off-by-one bugs hide. *Revisit Section 16.*
3	A) It makes business-rule coverage traceable and verifiable by product managers	Conditions-to-outcomes mapping lets a PM verify every rule combination is covered; a new rule is one new row. *Revisit Section 16.*
4	B) Invalid transitions: they verify the app enforces business rules	Invalid-transition tests catch the API allowing an illegal jump the UI prevents, a real data-integrity gap. *Revisit Section 16.*
5	C) ~30% unit, 30% component, 30% API, 10% E2E (critical journeys only)	Push coverage to fast layers; reserve slow E2E for revenue-critical journeys, high coverage, fast CI. *Revisit Section 16.*

SECTION 17

QA Fundamentals and Test Strategy

These questions test whether you can think above the test file, strategy, risk, and communication with non-technical stakeholders, which is what separates a senior SDET from a strong scripter. They cover defining a test strategy with a risk matrix, verification versus validation, reporting coverage in terms product understands, shift-left testing, and a tiered regression approach for agile teams.

In this section

- Test strategy from a risk matrix: probability x impact drives test investment.
- Verification vs validation: building it right vs building the right thing.
- Reporting coverage as requirement traceability, not a code-coverage percent.
- Shift-left: write test cases from acceptance criteria before development.
- Tiered agile regression: smoke per PR, targeted per sprint, full per release.

Q 217 LEVEL · Mid to Senior

How do you define a test strategy for a new product?

CONCEPT

A test strategy defines what to test, how, who, and when: scope, risk assessment, test types, environments, entry/exit criteria, and tooling. The anchor is a **risk matrix**: list every feature and rate it on two axes, **probability of failure** and business impact of failure. High probability plus high impact gets maximum coverage; low plus low gets minimal. This is how you **justify test investment to stakeholders**: you are explicitly not testing everything equally, you are testing the riskiest things most thoroughly, payment processing gets fifty tests, the about page gets two. The strategy then layers on test types (functional, regression, performance, security, accessibility), an environment plan, an automation scope with the pyramid distribution, and explicit entry/exit criteria like pass-rate thresholds.

INTERVIEW STRATEGY

They want to see strategic, prioritised thinking, not a generic list of test types. Make the risk matrix the spine: rate every feature on probability and impact, and let that drive where the effort goes. The payment-gets-50, about-page-gets-2 contrast makes it concrete and shows you can defend the spend. The pitfall is proposing to test everything equally. They usually follow with 'how do you justify the testing budget?', answer the risk matrix is the justification: effort is proportional to risk, and that is visible to stakeholders.

How to phrase it

> *A test strategy defines what to test, how, who, and when, scope, risk, test types, environments, entry and exit criteria, tooling, but I start every new product with a risk matrix, and that is what I would lead with because it is what makes the strategy defensible. I list every feature and rate each on two dimensions: the probability that it fails and the business impact if it does. High probability plus high impact gets maximum coverage; low plus low gets minimal. The reason I anchor on this, and my answer to the follow-up about justifying the testing budget, is that the matrix is the justification, I am explicitly not testing everything equally, I am testing the riskiest things most thoroughly, and I can show a stakeholder exactly why. Payment processing might get fifty tests; the about page gets two. That conversation lands far better than asking for blanket coverage. On top of the matrix the strategy layers the test types that apply, functional, regression, performance, security, accessibility, an environment plan for what runs where, the automation scope with the test-pyramid distribution, and explicit entry and exit criteria like pass-rate thresholds so done is defined rather than argued about. The trap is proposing to test everything equally, which is both impossible and impossible to justify. Risk-based prioritisation is what turns a wish list into a strategy.*

Key points to hit

(1) Strategy = what/how/who/when: scope, risk, types, envs, criteria, tooling. **(2)** Risk matrix: rate every feature on probability x business impact. **(3)** Effort proportional to risk: payment gets 50 tests, about page gets 2. **(4)** Layer on test types, environment plan, pyramid, entry/exit criteria. **(5)** Asked next: the risk matrix IS the budget justification. **(6)** What loses the point: proposing to test everything equally.

Q 218 LEVEL · Junior to Mid

What is the difference between verification and validation in QA?

CONCEPT

Verification asks 'are we building the product right?', does it conform to the specification (reviews, static analysis, unit tests). Validation asks 'are we building the right product?', does it meet real user needs (UAT, beta, usability testing). The classic illustration: a calculator that correctly implements the **wrong formula** passes verification but fails validation. Automated Playwright tests are primarily verification, they check behaviour against the spec. **UAT with real users is validation**: it checks whether the spec was correct in the first place. Both are necessary: a product can pass every automated test and still fail UAT because the requirements themselves were wrong, which is a failure no amount of verification can catch.

INTERVIEW STRATEGY

This is a fundamentals check; answer crisply and show you understand where automation sits. Use the two questions, building it right vs building the right thing, then the wrong-formula calculator, which is the example that proves you get it. Placing Playwright tests as verification and UAT as validation shows applied understanding. The trap is conflating the two or implying tests prove the product is right. The next question is normally 'which do your automated tests provide?', answer verification, with UAT covering validation.

How to phrase it

> *Verification asks are we building the product right, does it conform to the specification, checked through reviews, static analysis and unit tests. Validation asks are we building the right product, does it meet actual user needs, checked through UAT, beta and usability testing. The example I use because it makes the distinction click is a calculator that correctly implements the wrong formula: it passes verification, because it does exactly what the spec says, but it fails validation, because the spec was wrong and users get wrong answers. Applying that to my own work, which is the follow-up about which my automated tests provide: my Playwright tests are primarily verification, they check that the application behaves according to the specification. UAT with real users is validation: it checks whether the specification was correct in the first place. Both are necessary, and I would stress that, because I have seen products pass every automated test and still fail UAT because the requirements themselves were wrong, and no amount of verification can catch a wrong requirement, my tests would faithfully confirm the product does the wrong thing correctly. So I am clear-eyed that automation gives strong verification and real confidence the build matches the spec, but it does not replace validating that the spec was right, which is why I still advocate for UAT alongside a strong automated suite. The classic error is implying green tests prove the product is right; they prove it is built right.*

Key points to hit

(1) Verification: building it right (conforms to spec), reviews, unit tests. **(2)** Validation: building the right thing (meets user needs), UAT, beta. **(3)** Wrong-formula calculator passes verification, fails validation. **(4)** Playwright tests are verification; UAT is validation. **(5)** Follow-up: automated tests

provide verification, not validation. **(6)** The common miss: implying green tests prove the product is the right product.

Q 219 LEVEL · Mid

How do you measure and report test coverage to stakeholders?

CONCEPT

For E2E tests, coverage is best framed three ways: **requirement coverage** (which user stories have tests), risk coverage (which high-risk areas are tested), and code coverage (which paths run). The practical mechanism is a **requirement traceability matrix** built from requirement-ID tags, every test is tagged with the requirement it covers, and a script parses the results into a report: which requirements have passing tests, which have failing tests, and which have **no test at all**. You present this to stakeholders as a requirements coverage dashboard, not a code-coverage percentage, because a product manager understands 'REQ-103 has no automated test' far better than '73% branch coverage'. The gap, the untested requirement, is the most actionable thing on the report.

INTERVIEW STRATEGY

They are checking whether you can communicate coverage to non-technical people. Lead with requirement traceability via ID tags, and contrast it sharply with a code-coverage percentage, the PM understands 'REQ-103 has no test', not '73% branch'. That audience-awareness signals depth here. The mistake is reporting a raw code-coverage number to product. A common follow-up is 'what is the most useful thing on the report?', answer the requirements with no test, because that is the actionable gap.

How to phrase it

> *For E2E tests I frame coverage three ways: requirement coverage, which user stories have tests; risk coverage, which high-risk areas are tested; and code coverage, which paths run. But the mechanism I actually use and present is a requirement traceability matrix built from requirement-ID tags. Every test is tagged with the requirement it covers, REQ-101, REQ-102, and a script parses the run into a report showing which requirements have passing tests, which have failing tests, and which have no test at all. The key decision, and what I would emphasise, is that I present this to stakeholders as a requirements coverage dashboard, not a code-coverage percentage, because I am matching the report to the audience. A product manager understands REQ-103 has no automated test far better than seventy-three percent branch coverage, the first is actionable, the second is an abstraction they cannot act on. That also answers the follow-up about the most useful thing on the report: it is the requirements with no test, because that is the gap someone can immediately decide to close or accept. A code-coverage number, by contrast, can be high while a whole critical requirement sits untested, so it can actively mislead. Reporting coverage in the language of requirements keeps the conversation about risk and product, which is where stakeholders can make real decisions, rather than about a percentage that means little to them.*

Key points to hit

(1) Frame coverage as requirement, risk, and code coverage. **(2)** Build a traceability matrix from requirement-ID tags on tests. **(3)** Report shows requirements covered, failing, and with NO test. **(4)** Present as a requirements dashboard, not a code-coverage percent. **(5)** Expect next: the untested requirement is the most actionable item. **(6)** Avoid: reporting a raw code-coverage number to product.

CODE

```
// Requirement-ID tags create the traceability matrix
test('user can log in with valid credentials', { tag: ['@REQ-101', '@smoke'] }, async ({
page }) => {
// maps to REQ-101
});
test('invalid password shows error', { tag: ['@REQ-102', '@regression'] }, async ({ page
}) => {
// maps to REQ-102
});

// Generate a coverage report from the tags:
// npx playwright test --reporter=json | node scripts/coverage-report.js
// -> REQ-101: covered | REQ-102: covered | REQ-103: NOT COVERED
```

Q 220 LEVEL · Mid

What is shift-left testing and how do you implement it?

CONCEPT

Shift-left moves testing activities **earlier in the lifecycle**, from after development to during and before it, so defects are found when they are cheapest to fix. In practice that spans the stages: at requirements, review acceptance criteria and write test cases before development starts; at **design**, review API contracts and data models for testability; during development, unit tests alongside code with type-checking and ESLint enforced in CI; at integration, contract and component tests plus smoke on every PR; at release, full regression, performance, and security. The most impactful single practice is writing test cases from acceptance criteria **before** a developer starts, the defect found there costs essentially zero, whereas the same defect in production can cost on the order of 100x more.

INTERVIEW STRATEGY

You need to give concrete practices and a cost argument, not a buzzword. Define shift-left as finding defects earlier where they are cheaper, then name your highest-impact practice: writing test cases from acceptance criteria before development. Quantify it (zero cost pre-code vs ~100x in production, and a real defect-reduction number). The pitfall is reciting the phrase with no concrete action. The follow-up is often ‘how do you prove it works?’, answer with a metric like a measured drop in production defects.

How to phrase it

> *Shift-left moves testing activities earlier in the lifecycle, from after development to during and before it, so defects are found when they are cheapest to fix. In practice it spans the stages: at requirements I review acceptance criteria and write the test cases before development starts; at design I review API contracts and data models for testability gaps; during development it is unit tests alongside the code with type-checking and ESLint enforced in CI; at integration it is contract and component tests plus smoke on every PR; and at release it is full regression, performance and security. But the single most impactful practice, and what I would lead with, is writing the test cases from acceptance criteria before a developer picks up the ticket. When they start, the test cases already exist, so they know exactly what to implement and which edge cases to handle, and a defect found at that stage costs essentially zero because the code has not been written yet, whereas the same defect found in production costs on the order of a hundred times more. That cost curve is the whole argument. For the follow-up about proving it works, I quantify it to stakeholders rather than asserting it: in one case our shift-left practices reduced production defects by around forty percent in the first quarter, which turns shift-left from a nice idea into a measurable result they can fund. Where this goes wrong is just saying the word; the value is in the specific early-stage practices and the cost math behind them.*

Key points to hit

(1) Move testing earlier so defects are found when cheapest to fix. **(2)** Spans stages: requirements, design, development, integration, release. **(3)** Highest-impact practice: write test cases from criteria before dev starts. **(4)** Cost: ~0 pre-code vs ~100x in production. **(5)** They usually ask next: prove it with a metric (e.g. ~40% fewer production defects). **(6)** The easy mistake: reciting the buzzword with no concrete practice.

Q 221 LEVEL · Mid

How do you handle regression testing in an agile environment?

CONCEPT

Agile regression needs a **tiered approach** rather than running everything every time. Tier 1, smoke on every PR: the critical happy paths, around 3 minutes. **Tier 2, targeted regression** per sprint: the areas changed, determined by the PR's changed files or labels, around 15 minutes. Tier 3, full regression on release candidates, the whole suite across browsers, around 45 minutes. The key is **automating the tier selection** from PR labels or changed files so developers never have to decide which tests to run: a payment-change PR triggers the payment and checkout suites, a release-candidate triggers full regression, everything else gets smoke. This balances fast feedback with confidence: PRs are not blocked by a 45-minute run, and releases are not shipped on a 3-minute one.

INTERVIEW STRATEGY

Your job here is to show you balance speed and confidence in a fast-moving team. Lay out the three tiers with their time budgets and triggers, then stress the differentiator: automate tier selection from labels or changed files so no human decides. The classic error is one-size regression: either always-full (too slow for PRs) or always-smoke (too thin for releases). Expect the follow-up 'how do you decide which tier runs?', answer automatically from PR labels or the set of

changed files.

How to phrase it

> *Agile regression needs a tiered approach rather than running everything every time, and I would frame it as three tiers with explicit time budgets. Tier one is smoke on every PR: the critical happy paths, about three minutes, fast enough not to block developers. Tier two is targeted regression per sprint: the areas that actually changed, around fifteen minutes. Tier three is full regression on release candidates, the whole suite across Chromium, Firefox and WebKit, around forty-five minutes, where I want maximum confidence. The key that makes this work, and my answer to the follow-up about how the tier is chosen, is automating the selection from PR labels or changed files so developers never have to decide which tests to run: a PR labelled payment-change triggers the payment and checkout suites, a release-candidate triggers full regression, and everything else gets smoke. Making the machine choose is what keeps it consistent, if I leave it to people, someone runs full regression on a tiny PR and wastes everyone's time, or runs only smoke before a release and ships a regression. This balances fast feedback with confidence: PRs are not blocked by a forty-five-minute run, and releases are not shipped on the back of a three-minute one. The thing to avoid is one-size-fits-all regression in either direction, always-full is too slow for PRs, always-smoke is too thin for releases, and the tiering with automated selection is how I avoid both.*

Key points to hit

(1) Three tiers: smoke per PR (~3min), targeted per sprint (~15min), full per release (~45min). **(2)** Targeted tier runs the areas changed, by changed files or labels. **(3)** Automate tier selection so developers never choose which tests run. **(4)** Balances fast PR feedback with release confidence. **(5)** The next question: tier chosen automatically from PR labels/changed files. **(6)** Trap: one-size regression, always-full (slow) or always-smoke (thin).

CODE

```
# Tier 1: smoke on every PR (~3 min)
npx playwright test --grep @smoke --project=chromium

# Tier 2: targeted regression per sprint (~15 min)
npx playwright test --grep "@checkout|@payment" --project=chromium

# Tier 3: full regression on release candidates (~45 min)
npx playwright test --project=chromium --project=firefox --project=webkit

// Automated tier selection in CI
if (pr.labels.includes('release-candidate')) runFullRegression();
else if (pr.labels.includes('payment-change')) runTargeted(['@payment', '@checkout']);
else runSmoke();
```

Q 222 LEVEL · Senior

How do you design a test environment strategy that supports parallel teams without contention?

CONCEPT

Test environments are **infrastructure shared by many testers**, and contention between them is the source of most flake that isn't a code bug. Three strategies layered by need. Ephemeral environments per PR: each PR spins up its own isolated stack (Kubernetes namespace, Docker Compose, Vercel preview), no contention, production-realistic, but expensive in compute and DevOps overhead. Best for the highest-stakes teams. **Shared staging with tenant isolation**: one persistent staging environment, but every test creates data under its own tenant ID or namespace so concurrent tests don't see each other's data, the pattern from S9 Q9 applied at strategy level. Cheaper than ephemeral, scales to dozens of engineers. Dedicated environments per team: each team gets its own staging instance, refreshed nightly from production-shaped seed data. Sits between ephemeral and shared in cost. The discipline: match strategy to team size and budget. Five engineers and a single QA can run on shared staging with tenant isolation. Fifty engineers across ten teams need either ephemeral per PR or dedicated per team to avoid contention storms. The trap: **one shared environment for everyone** when the team has grown past it, every release becomes a coordination problem, every flaky test gets blamed on someone else's concurrent run.

INTERVIEW STRATEGY

They are after environment strategy as an org-scaling question, not a tool choice. Walk the three options (ephemeral per PR, shared with tenant isolation, dedicated per team) with the trade-offs. The 'match strategy to team size' rule is the senior signal. The mistake is one shared environment for everyone. They usually follow with ‘what's your default recommendation?’, answer shared staging with tenant isolation, best balance of cost and scale for most teams.

How to phrase it

> *Test environments are infrastructure shared by many testers, and contention between them is the source of most flake that isn't a code bug. Two tests running concurrently on the same shared data interact in non-obvious ways and produce intermittent failures that look like test bugs but are actually environment bugs. Three strategies layered by need and budget. First, ephemeral environments per PR. Each PR spins up its own isolated stack: Kubernetes namespace, Docker Compose, Vercel preview, Render preview. No contention because each PR is on its own infrastructure; production-realistic because the whole stack comes up the same way as prod does; but expensive in compute cost and DevOps overhead to maintain the spin-up pipeline. Best for the highest-stakes teams where contention cost is greater than infrastructure cost. Second, shared staging with tenant isolation. One persistent staging environment, but every test creates data under its own tenant ID or namespace so concurrent tests don't see each other's data. This is the pattern from the parallel execution section applied at the strategy level: parallel safety through tenant isolation, not through reduced concurrency. Cheaper than ephemeral, scales to dozens of engineers, doesn't require ephemeral infrastructure investment. That's also my answer to the follow-up about default recommendation: shared staging with tenant isolation. Best balance of cost and scale for most teams, and the pattern that survives team growth without re-architecting. Third, dedicated environments per team. Each team gets its own staging instance, refreshed nightly from production-shaped seed data. Sits between ephemeral and shared in cost: cheaper than ephemeral because it's persistent, more isolated than shared because each team has full control. Good for regulated environments where data isolation between teams matters for compliance reasons, not just convenience. The discipline I would lead with, because it's where teams make strategic mistakes, is match strategy to team size and budget. Five engineers and a single QA can run on shared staging with tenant isolation, low overhead, sufficient scale. Fifty engineers across ten teams need either ephemeral per PR or dedicated per team to avoid contention storms. The transitions between these strategies are themselves the work: knowing when the org has outgrown shared staging is what prevents the slow-burning frustration of contention-driven flake as teams scale. The pitfall is one shared environment for everyone when the team has grown past it. Every release becomes a coordination problem: don't deploy on Friday because the marketing team is testing the campaign flow. Every flaky test gets blamed on someone else's concurrent run. The right fix is tenant isolation or environment tiering, not asking the teams to schedule around each other.*

Key points to hit

(1) Ephemeral per PR: max isolation, max cost, best for highest-stakes teams. **(2)** Shared staging + tenant isolation: best balance, default for most teams. **(3)** Dedicated per team: between ephemeral and shared, good for regulated orgs. **(4)** Match strategy to team size and budget: transitions are themselves the work. **(5)** Asked next: default recommendation is shared + tenant isolation. **(6)** Where this goes wrong: one shared environment past team size, contention storms, scheduling pain.

CODE

```
# Environment strategy by team size / budget

# Strategy A: ephemeral per PR (Vercel/Render/K8s namespace)
# .github/workflows/preview.yml
deploy_preview:
steps:
- run: kubectl create namespace pr-${{ github.event.number }}
- run: helm install acme ./chart --namespace pr-${{ github.event.number }} \
--set imageTag=${{ github.sha }}
- run: BASE_URL=https://pr-${{ github.event.number }}.staging.acme.com npx playwright
test
- if: always()
run: helm uninstall acme --namespace pr-${{ github.event.number }}

# Strategy B: shared staging with tenant isolation (S9 Q9 pattern)
test.use({
storageState: 'auth/per-worker.json',
});
test.beforeEach(async ({ request }, testInfo) => {
await request.post('/api/test/tenant', { data: { id:
`tenant-w${testInfo.workerIndex}-t${testInfo.testId}` } });
});

# Strategy C: dedicated per team (config per team in CI)
# config/staging-payments.ts, config/staging-platform.ts, config/staging-growth.ts
# Each team's CI points at its own staging URL
```

Q 223

LEVEL · Senior

What quality metrics actually matter, and which are vanity metrics that mislead leadership?

CONCEPT

Quality metrics fall into two categories: **outcome metrics** measure what users actually experience; vanity metrics measure activity that may or may not produce outcomes. The ones that matter. Defect escape rate: bugs that reached production per release, the inverse of test effectiveness. **Mean time to detect (MTTD)**: from defect occurring to defect known about, measures monitoring and alerting quality. **Mean time to restore (MTTR)**: from defect known to defect fixed, measures operational responsiveness. Change failure rate: percentage of deploys causing a production incident, from DORA, predicts org health. Test stability rate: percentage of test runs that pass without retry, measures suite trustworthiness. The vanity metrics that mislead leadership. Test count: more tests aren't better, a thousand brittle tests are worse than two hundred reliable ones. Code coverage percentage: hitting lines doesn't mean verifying behaviour; high coverage with weak assertions is worse than focused coverage with strong ones. Tests passing: passing tests are the baseline expectation, not an achievement. The discipline: report outcomes to leadership, activity to your own team. Leadership cares about defect escape and change failure rate; the team uses test stability and coverage to operate. Conflating the two produces misleading dashboards and bad incentives.

INTERVIEW STRATEGY

They want metric discrimination: which signal vs which noise. List five outcome metrics (defect escape, MTTD, MTTR, change failure rate, test stability) and three vanity ones (test count, coverage percentage, tests passing). The 'outcomes to leadership, activity to team' rule demonstrates calibration. The trap is leadership dashboards built on vanity metrics. The next question is normally ‘which single metric matters most?’, answer defect escape rate, it's the direct measure of test effectiveness.

How to phrase it

> *Quality metrics fall into two categories. Outcome metrics measure what users actually experience; vanity metrics measure activity that may or may not produce outcomes. The distinction matters because the wrong dashboard at the leadership level produces bad incentives across the org. The ones that matter, in priority order. Defect escape rate: bugs that reached production per release. The inverse of test effectiveness. If fewer bugs are escaping, testing is working; if more, it isn't. That's also my answer to the follow-up about which single metric matters most: defect escape rate. It's the direct measure of whether the test suite is doing its job. Mean time to detect: from defect occurring in production to defect being known about. Measures monitoring and alerting quality. A bug that takes four hours to detect has had four hours of user impact; one detected in four minutes has had four minutes. Mean time to restore: from defect known to defect fixed. Measures operational responsiveness. Together with MTTD, describes the recovery story. Change failure rate, from the DORA metrics: percentage of deploys causing a production incident. Predicts org health better than almost any single metric because it captures the interaction of testing, deployment practices, and code quality. Test stability rate: percentage of test runs that pass without retry. Measures suite trustworthiness, the thing that determines whether engineers actually pay attention to test failures or rerun them reflexively. The vanity metrics that mislead leadership, and where I push back when asked to put them on a dashboard. Test count: more tests aren't better. A thousand brittle tests are worse than two hundred reliable ones because the brittle ones produce noise that drowns out signal. Leadership rewarding test count incentivises producing more tests of decreasing quality. Code coverage percentage: hitting lines doesn't mean verifying behaviour. High coverage with weak assertions, exercising the code but not asserting the right outcomes, is worse than focused coverage with strong assertions, because it gives false confidence. Coverage is a floor check, not a quality measure. Tests passing: passing tests are the baseline expectation, not an achievement. Reporting all-green to leadership as if it's a win conflates the absence of regression with the presence of quality. The discipline I would close on, because it's what keeps these metrics useful, is report outcomes to leadership, activity to your own team. Leadership cares about defect escape and change failure rate, the business signals. The team uses test stability and coverage to operate, the technical signals. Conflating the two produces misleading dashboards and bad incentives: leadership thinks we're winning because the dashboard is green; production tells a different story.*

Key points to hit

(1) Outcome metrics: defect escape rate (top), MTTD, MTTR, change failure, stability. **(2)** Vanity metrics: test count, coverage %, tests-passing, don't put on leadership dash. **(3)** Discipline: outcomes to leadership, activity to your own team. **(4)** Defect escape rate is the single most important metric, direct test effectiveness. **(5)** Follow-up: coverage is a floor check, not a quality measure. **(6)** What loses the point: leadership rewarding vanity metrics, incentivises noise over

signal.

CODE

```
// Outcome dashboard for leadership
{
defectEscapeRate: bugsReachingProduction / totalReleasesThisQuarter,
mttdMinutes: avg(timeFromDefectStartToAlert), // monitoring quality
mttrMinutes: avg(timeFromAlertToFix), // operational responsiveness
changeFailureRate: incidentCausingDeploys / totalDeploys, // DORA
testStabilityRate: passesFirstTry / totalTestRuns, // suite trustworthiness
}

// Activity dashboard for the team (NOT for leadership)
{
testCount: totalTestsInSuite, // direction-of-travel only
codeCoverage: percentLinesHit, // floor check, not quality measure
passingTests: passedThisRun, // baseline, not achievement
flakeRate: retriedTests / totalRuns, // tracks toward stability
}

// Anti-pattern: putting test count on the executive dashboard
// Result: someone optimises for the metric, mass-produces brittle tests
// Outcome: dashboard goes up, defect escape stays flat, leadership confused
```

Q 224

LEVEL · Senior

How do you scale QA expertise when you're the only SDET across multiple teams?

CONCEPT

Solo-SDET-across-teams is a leverage problem: the SDET can't write every test for every team, so the strategy has to be enable developers to write their own tests well. Four leverage moves. **Reusable framework**: fixtures, page objects, helpers, custom matchers, the SDET builds the infrastructure developers use to write tests in minutes, not hours. Pair-programming sessions: rotating two-hour pairing slots where the SDET works with one developer on their feature's tests, the developer learns the patterns from doing, not from documentation. **Code review on test PRs**: every developer-written test goes through the SDET for review, with specific feedback on selector quality, async discipline, and negative-case coverage, review is teaching at scale. Documented patterns library: examples in the repo (recipes for common tasks, anti-patterns to avoid, common gotchas), updated as the framework evolves. The discipline that makes this scale: the SDET's job is the multiplier, not the producer. Writing tests directly is the lowest-leverage activity; building infrastructure and teaching others to write tests well multiplies the SDET's impact across the team count. The trap: **the SDET becomes the bottleneck**, every test needs the SDET, every framework change waits on the SDET, the role doesn't scale and the SDET burns out. Multiplier first, producer second.

INTERVIEW STRATEGY

What earns the tick is leverage strategy for the solo-SDET role. Walk the four moves (framework, pairing, review, documented patterns). The 'SDET as multiplier, not producer' framing is the senior

signal. Where this goes wrong is becoming the bottleneck. A common follow-up is 'how do you handle teams resistant to writing tests?', answer make it trivially easy via framework + pairing; resistance often comes from the test-writing being too hard.

How to phrase it

> *Solo-SDET-across-teams is a leverage problem. The SDET can't write every test for every team, the math doesn't work even at modest team sizes, so the strategy has to be enable developers to write their own tests well. The job is the multiplier, not the producer. Four leverage moves I would apply in order. First, reusable framework. Fixtures, page objects, helpers, custom matchers, test data builders, authentication setup, common assertions, the SDET builds the infrastructure developers use to write tests in minutes, not hours. If writing a basic test takes a developer an afternoon, they won't write tests. If it takes ten minutes because the fixtures and patterns are all there, they will. Second, pair-programming sessions. Rotating two-hour pairing slots where the SDET works with one developer on their feature's tests. The developer learns the patterns from doing, not from documentation, which is dramatically more effective. After three or four pairing sessions, that developer can write tests independently for the next twenty features. Third, code review on test PRs. Every developer-written test goes through the SDET for review, with specific feedback on selector quality, async discipline, negative-case coverage, the things that distinguish a good test from a brittle one. Review is teaching at scale because every developer who reads the feedback improves, and the patterns spread across the codebase. Fourth, documented patterns library. Examples in the repo, recipes for common tasks, anti-patterns to avoid, common gotchas, updated as the framework evolves. The library is the asynchronous teaching channel for the cases pairing doesn't cover. The discipline I would lead with, because it's where solo SDETs fail when they fail, is the SDET's job is the multiplier, not the producer. Writing tests directly is the lowest-leverage activity. Building infrastructure and teaching others to write tests well multiplies the SDET's impact across the team count. One SDET writing tests for ten teams produces less than one SDET enabling ten teams to write their own tests well. The math is decisive. If they push on this in the follow-up, my answer is about teams resistant to writing tests: make it trivially easy via framework plus pairing. Resistance often comes from the test-writing being too hard or too time-consuming. If the developer can write a covering test in ten minutes during their regular feature work, resistance evaporates because the cost has dropped below the value. The thing to avoid is the SDET becomes the bottleneck, every test needs the SDET to write or review at the last minute, every framework change waits on the SDET, the role doesn't scale and the SDET burns out. Multiplier first, producer second. Write tests yourself only where the test is unusually hard or where it serves as a teaching example for the patterns; otherwise enable and review.*

Key points to hit

(1) Solo SDET = leverage problem; can't write every test, so enable devs to. **(2)** Move 1: reusable framework (fixtures, POs, helpers), tests in minutes not hours. **(3)** Move 2: pair-programming, patterns learned by doing, not documentation. **(4)** Move 3: code review on test PRs, teaching at scale via specific feedback. **(5)** Move 4: documented patterns library, async teaching for what pairing doesn't. **(6)** The common miss: SDET as bottleneck, writes/reviews everything, role doesn't scale.

CODE

```
// The reusable framework that enables developers to write tests in minutes
//
// /tests/support/ <- the SDET builds and owns this
// fixtures.ts authenticated pages, test data builders
// page-objects/
// CheckoutPage.ts, AccountPage.ts consistent patterns per area
// matchers.ts custom expect.extend matchers
// builders/
// userBuilder.ts, orderBuilder.ts fluent test-data construction
//
// /tests/features/ <- developers write tests here
// checkout.spec.ts
// account.spec.ts
//
// Developer's typical test, made trivial by the framework:
test('user can update billing address', async ({ accountPage, userBuilder }) => {
const user = await userBuilder.withAddress().create();
await accountPage.gotoAs(user);
await accountPage.updateAddress({ city: 'Chennai' });
await expect(accountPage.savedConfirmation).toBeVisible();
});
// 5 lines because the framework did the heavy lifting.
//
// Pair on the first 3-4 tests with each developer; review every PR; document patterns.
// One SDET enabling 10 teams beats one SDET writing for 10 teams. Multiplier >
producer.
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What anchors a risk-based test strategy?

A) A risk matrix rating each feature on probability x business impact
B) Testing every feature equally
C) Code coverage targets
D) The number of available testers

2. Which question does validation (not verification) answer?

A) Are we building the product right?
B) Are we building the right product?
C) Is the code well-formatted?
D) Do the unit tests pass?

3. How should test coverage be reported to product stakeholders?

A) As a branch-coverage percentage
B) As the total number of tests
C) As a requirement traceability matrix (which requirements have tests)
D) As lines of test code

4. What is the most impactful shift-left practice?

A) Running more tests in production
B) Increasing the timeout
C) Adding retries in CI
D) Writing test cases from acceptance criteria before development starts

5. How is regression best handled in an agile team?

A) Run the full suite on every PR
B) Tiered: smoke per PR, targeted per sprint, full per release, selected automatically
C) Run only smoke before releases
D) Manual regression each sprint

Section 17 · Answer key

#	Answer	Why and where to revisit
1	A) A risk matrix rating each feature on probability x business impact	Probability x impact decides where effort goes, payment processing gets many tests, the about page few. *Revisit Section 17.*
2	B) Are we building the right product?	Validation checks the product meets user needs; a correct implementation of the wrong spec fails validation. *Revisit Section 17.*
3	C) As a requirement traceability matrix (which requirements have tests)	'REQ-103 has no automated test' is actionable for a PM; '73% branch coverage' is not. *Revisit Section 17.*
4	D) Writing test cases from acceptance criteria before development starts	A defect found before code is written costs ~0; the same defect in production costs ~100x more. *Revisit Section 17.*
5	B) Tiered: smoke per PR, targeted per sprint, full per release, selected automatically	Automated tier selection from labels/changed files balances fast PR feedback with release confidence. *Revisit Section 17.*

SECTION 18

Mobile Testing with Playwright

Mobile questions are where candidates either over-claim or under-explain, so interviewers listen for whether you know the boundary: Playwright does mobile web emulation, not native app testing. These eight questions cover device emulation for responsive web, testing touch gestures and their limits, and parameterised responsive-breakpoint testing with per-viewport snapshots.

In this section

- Device descriptors emulate viewport, user agent, DPR and touch, web, not native.
- Scope honesty: Appium/Detox for native; Playwright for mobile web.
- Touch gestures: tap() is solid; complex gestures are verbose and fragile.
- Real devices (BrowserStack/Sauce) for gesture-heavy features.
- Parameterised viewport tests with a visual snapshot per breakpoint.

Q 225 LEVEL · Junior to Mid

How do you test mobile web applications with Playwright?

CONCEPT

Playwright emulates mobile devices with **device descriptors** that set viewport, user agent, device pixel ratio, and touch support, you apply one with **test.use({ ...devices['iPhone 14'] })** or as a project. Critically this is browser-based mobile web emulation, not native app testing: for native iOS and Android you use Appium or Detox. For mobile web, responsive layouts, touch interactions, mobile-specific UI, Playwright's emulation is excellent, and you run mobile projects in nightly regression to catch responsive regressions. The two most common mobile-specific bugs it surfaces: elements visible on desktop but hidden behind the mobile nav, and touch targets too small to tap reliably. Stating the emulation-versus-native scope upfront signals you know the tool's real boundaries.

INTERVIEW STRATEGY

This one is checking whether you understand the tool's boundary. The single most important move is clarifying scope upfront: Playwright does mobile web emulation, not native app testing, and native means Appium or Detox. That honesty is the seniority marker. Then describe device descriptors and the bugs it catches. The thing to avoid is claiming Playwright tests native apps. The follow-up is often 'what about native apps?', answer Appium/Detox, with Playwright owning responsive mobile web.

How to phrase it

> *Playwright emulates mobile devices with device descriptors that set the viewport, user agent, device pixel ratio and touch support, and I apply one with test.use spreading devices iPhone 14, or as a project in the config. The first thing I always do in an interview, though, is clarify the scope, because it is exactly what distinguishes someone who knows the tool: this is browser-based mobile web emulation, not native app testing. That is also my answer to the inevitable follow-up about native apps, for native iOS and Android, Appium or Detox are the correct tools; Playwright does not drive native apps. For mobile web, though, responsive layouts, touch interactions, mobile-specific UI, its emulation is genuinely excellent, and I run mobile projects in nightly regression to catch responsive regressions. The most common mobile-specific bugs it surfaces are elements that are visible on desktop but hidden behind the mobile nav, and touch targets that are too small to tap reliably, both invisible to a desktop-only suite. Stating the emulation-versus-native boundary upfront matters because over-claiming that Playwright tests native apps is the fastest way to lose credibility in a mobile question, whereas being precise about what it does and does not do shows I have actually used it for mobile work and know where its line is.*

Key points to hit

(1) Device descriptors set viewport, user agent, DPR, touch (test.use or project). **(2)** Scope upfront: mobile web emulation, NOT native app testing. **(3)** Native apps -> Appium or Detox; Playwright owns responsive mobile web. **(4)** Run mobile projects nightly; catches hidden-behind-nav and tiny-tap-target bugs. **(5)** Expect next: native apps need Appium/Detox. **(6)** Avoid: claiming Playwright tests native mobile apps.

CODE

```
import { test, expect, devices } from '@playwright/test';

test.use({ ...devices['iPhone 14'] });
test('mobile checkout', async ({ page }) => {
await page.goto('/checkout');
await page.getByRole('button', { name: 'Menu' }).tap();
await expect(page.locator('.mobile-nav')).toBeVisible();
await expect(page.locator('.desktop-nav')).toBeHidden();
});

// playwright.config.ts: mobile projects for nightly regression
// projects: [
// { name: 'iPhone 14', use: { ...devices['iPhone 14'] } },
// { name: 'Pixel 7', use: { ...devices['Pixel 7'] } },
// ]
```

Q 226

LEVEL · Mid

How do you test touch gestures and mobile-specific interactions?

CONCEPT

Playwright supports **tap()**, the touchscreen API, and **dispatchEvent** for complex gestures. The honest picture: **tap() works perfectly** as the mobile equivalent of click, and simple touchscreen taps are reliable. There is no built-in swipe or pinch helper: page.touchscreen only offers tap. Complex gestures, swipe, pinch-to-zoom, mean building real Touch objects and dispatching the full touchstart, touchmove, touchend sequence yourself through dispatchEvent, which is verbose and fragile. A helper that taps and then calls mouse.move is a common copy-paste trap: it fires no touch events, so no gesture handler ever sees a swipe. For applications with heavy gesture interaction (maps, image galleries, drawing tools), the right call is real devices via BrowserStack or Sauce Labs using Playwright's remote browser connection. The rule of thumb worth stating: emulation covers around 90% of mobile web testing well, and real devices are needed for the gesture-heavy 10%, knowing where that line falls is the point of the question.

INTERVIEW STRATEGY

You need to give honesty about emulation's limits, not bravado. Be candid: tap() is solid, complex gestures via dispatchEvent are verbose and fragile. Then give the practical escalation, real devices on BrowserStack/Sauce for gesture-heavy apps, and the 90/10 rule of thumb. That calibrated judgement is the experience marker. The thing to avoid is claiming emulation handles every gesture flawlessly. Expect the follow-up ‘when do you move to real devices?’, answer for gesture-heavy features like maps and galleries.

How to phrase it

> *Playwright gives me tap, the touchscreen API, and dispatchEvent for complex gestures, and I would be candid about where the line is rather than overstate it. Simple tap works perfectly, it is the mobile equivalent of click, and basic touchscreen taps are reliable. The complex gestures, swipe and pinch-to-zoom, require constructing TouchEvent objects through dispatchEvent, and that is verbose and fragile; it works, but it is the kind of code that breaks easily and is unpleasant to maintain. So for applications with heavy gesture interaction, maps, image galleries, drawing tools, I do not pretend emulation is enough; I recommend testing on real devices through BrowserStack or Sauce Labs using Playwright's remote browser connection. That is my answer to the follow-up about when I move to real devices: gesture-heavy features are the trigger. The rule of thumb I state is that emulation covers around ninety percent of mobile web testing well, responsive layout, taps, mobile UI, and real devices are needed for the gesture-heavy ten percent. The reason I answer this way is that the question is really testing calibrated judgement: claiming emulation handles every gesture flawlessly is the trap and it reads as inexperience, whereas knowing exactly where emulation stops and real devices start is what shows I have actually shipped mobile web tests and hit those limits myself.*

Key points to hit

(1) tap() and simple touchscreen taps are reliable. **(2)** No built-in swipe or pinch: touchscreen only has tap(). **(3)** Swipe = build Touch objects, then dispatchEvent touchstart/touchmove/touchend. **(4)** The easy mistake: a tap plus mouse.move fires no touch events, so the swipe silently does nothing. **(5)** Real devices via BrowserStack/Sauce for gesture-heavy apps (remote connection). **(6)** Rule of thumb: emulation ~90%, real devices for the gesture-heavy 10%. **(7)** They usually ask next: move to real devices for maps, galleries, drawing tools. **(8)** Trap: claiming emulation handles every gesture flawlessly.

CODE

```
// tap: the mobile click; reliable
await page.getByRole('button', { name: 'Add to Cart' }).tap();
await page.touchscreen.tap(200, 300);

// Swipe: there is NO touchscreen.swipe(). You dispatch the whole
// touchstart/touchmove/touchend run yourself, which is why it is verbose.
// A tap plus a mouse.move fires no touch events at all and does nothing.
async function swipeLeft(page: Page, el: Locator): Promise<void> {
const box = await el.boundingBox();
if (!box) throw new Error('Element has no bounding box');
const y = box.y + box.height / 2;
const target = await el.elementHandle();

const touchAt = (x: number) => page.evaluateHandle(
({ target, x, y }) => new Touch({ identifier: 1, target, clientX: x, clientY: y }),
{ target, x, y });

const start = await touchAt(box.x + box.width * 0.8);
const end = await touchAt(box.x + box.width * 0.2);
await el.dispatchEvent('touchstart',
{ touches: [start], changedTouches: [start], targetTouches: [start] });
await el.dispatchEvent('touchmove',
{ touches: [end], changedTouches: [end], targetTouches: [end] });
await el.dispatchEvent('touchend',
{ touches: [], changedTouches: [end], targetTouches: [] });
}
// Needs hasTouch: true on the context (the device profiles set it)
// Gesture-heavy apps -> real devices via BrowserStack/Sauce remote connection
```

Q 227 LEVEL · Mid

How do you test responsive design breakpoints?

CONCEPT

Parameterise a test across **viewport sizes**, mobile (375), tablet (768), desktop (1440), and assert the correct layout at each: below the breakpoint the hamburger menu is visible and the desktop nav hidden, above it the reverse. Add a **visual snapshot per viewport** (toHaveScreenshot with a per-size name) so a baseline exists at every breakpoint and CSS regressions are caught. These are the most valuable mobile tests because they catch the most common responsive bugs: elements overflowing at narrow widths, navigation breaking at tablet size, text becoming unreadable on small screens. They are too slow for every PR but important enough to run in the **nightly build**, a deliberate placement on the speed-versus-coverage spectrum.

INTERVIEW STRATEGY

They are after a systematic approach, not eyeballing. Lead with parameterising across viewports and asserting layout per breakpoint, then add the per-viewport snapshot for CSS-regression safety. Naming the common bugs it catches makes the value concrete. Placing it in nightly, not per-PR, shows you think about the speed/coverage trade-off. The classic error is one fixed viewport or manual visual checks. They usually follow with ‘why not run these on every PR?’, answer they are too slow but valuable enough for the nightly build.

How to phrase it

> *I parameterise a test across viewport sizes, mobile at 375, tablet at 768, desktop at 1440, and assert the correct layout at each: below the breakpoint the hamburger menu is visible and the desktop nav is hidden, above it the reverse. On top of that I add a visual snapshot per viewport with toHaveScreenshot and a per-size name, so I have a baseline at every breakpoint and a CSS change that breaks one size gets caught. I would call these the most valuable mobile tests I run, because they catch the most common responsive bugs, elements overflowing at narrow widths, navigation breaking at tablet size, text becoming unreadable on small screens, all of which are invisible to a single-viewport suite. On placement, and this answers the follow-up about why I do not run them on every PR: they are too slow for that, between the multiple viewports and the screenshot comparisons, but they are important enough to run in the nightly build, so I put them exactly where they belong on the speed-versus-coverage spectrum, not gating every PR, but never skipped either. The thing to avoid is testing a single fixed viewport or relying on someone manually eyeballing the layout, which does not scale and misses regressions. Parameterising across the real breakpoints with a snapshot at each makes responsive coverage systematic and gives me a baseline that turns a silent CSS regression into a failing test.*

Key points to hit

(1) Parameterise across mobile/tablet/desktop viewports; assert layout at each. **(2)** Add a visual snapshot per viewport to catch CSS regressions. **(3)** Catches overflow, broken tablet nav, unreadable small-screen text. **(4)** Run in nightly, not per-PR: too slow but high value. **(5)** The next question: why not per-PR, too slow; nightly is the right home. **(6)** Where this goes wrong: a single fixed viewport or manual visual checks.

CODE

```
const viewports = [
{ name: 'mobile', width: 375, height: 812 },
{ name: 'tablet', width: 768, height: 1024 },
{ name: 'desktop', width: 1440, height: 900 },
];
for (const v of viewports) {
test(`nav layout at ${v.name} (${v.width}px)`, async ({ page }) => {
await page.setViewportSize({ width: v.width, height: v.height });
await page.goto('/');
if (v.width < 768) {
await expect(page.locator('.hamburger-menu')).toBeVisible();
await expect(page.locator('.desktop-nav')).toBeHidden();
} else {
await expect(page.locator('.desktop-nav')).toBeVisible();
await expect(page.locator('.hamburger-menu')).toBeHidden();
}
await expect(page).toHaveScreenshot(`nav-${v.name}.png`);
});
}
```

Q 228

LEVEL · Senior

What does Playwright's device emulation actually emulate, and where does it differ from a real device?

CONCEPT

Device emulation in Playwright sets viewport size, user-agent string, device pixel ratio, touch support flag, isMobile flag, and a few other browser-detectable properties from the predefined **devices** object. What it doesn't change: the underlying rendering engine (Chromium regardless of whether you emulate iPhone or Galaxy), CPU and memory characteristics, real touch hardware (events are synthesised, not physical), iOS-specific Safari/WebKit quirks outside the WebKit project version Playwright ships with, and any feature that depends on the actual OS (push notifications, deep links, biometric auth). The implication: emulation catches layout, responsive-design, and user-agent-conditional code paths; it does not catch **device-specific browser bugs**, performance issues on real CPU, or **iOS Safari behaviour** that diverges from Playwright's WebKit build. The discipline: use emulation for layout + flow regression in CI; supplement with BrowserStack/Sauce or actual devices for the cross-device compatibility check before release. Treating emulation as equivalent to real-device testing is the trap that ships iOS-specific bugs to production.

INTERVIEW STRATEGY

They want emulation calibrated against real devices, not treated as equivalent. Lead with what Playwright sets (viewport, UA, DPR, touch flag, isMobile) and what it doesn't (rendering engine, CPU, real hardware, OS-level). The 'emulation for layout/flow, real for compatibility' discipline is the test of seniority on this. The trap is shipping iOS bugs from emulation-only testing. The next question is normally ‘what's the biggest gap?’, answer iOS Safari specifically; Playwright's WebKit isn't iOS Safari, it's a related build.

How to phrase it

> *Device emulation in Playwright sets viewport size, user-agent string, device pixel ratio, touch-support flag, isMobile flag, and a few other browser-detectable properties from the predefined devices object. So a responsive layout reading from window dot innerWidth sees the iPhone viewport; JavaScript reading navigator dot userAgent sees the iPhone UA string; CSS media queries fire correctly. That's the layer emulation handles well. What emulation doesn't change, and the part senior interviewers want to hear listed, is the underlying rendering engine, Chromium regardless of whether I emulate iPhone or Galaxy, CPU and memory characteristics, real touch hardware so events are synthesised rather than physical, iOS-specific Safari and WebKit quirks outside the WebKit project version Playwright ships with, and any feature that depends on the actual operating system: push notifications, deep links, biometric authentication, app-store-installed PWAs behaving differently from browser PWAs. The implication, and the framing that lands, is emulation catches layout, responsive design, and user-agent-conditional code paths reliably. It does not catch device-specific browser bugs, performance issues on real CPU and memory constraints, or iOS Safari behaviour that diverges from Playwright's WebKit build. That's also my answer to the follow-up about the biggest gap: iOS Safari specifically. Playwright's WebKit is a related build maintained by Microsoft, not Apple's iOS Safari, and the two have known divergences in areas like memory limits, video playback, private browsing behaviour, certain CSS features. The discipline I would close on is use emulation for layout and flow regression in CI, fast, deterministic, good enough for the bulk of mobile-web testing, and supplement with BrowserStack, Sauce Labs, or actual devices for the cross-device compatibility check before release. Real devices are slow and expensive; emulation is fast and cheap. Use each for what it's good at. The trap is treating emulation as equivalent to real-device testing, which ships iOS-specific bugs to production because the tests passed in emulation. Tests pass; users on iPhones see broken features.*

Key points to hit

(1) Emulation sets: viewport, UA, DPR, touch flag, isMobile from devices object. **(2)** Doesn't change: rendering engine, CPU/RAM, real hardware, iOS-specific quirks. **(3)** Catches: layout, responsive, UA-conditional paths. **(4)** Misses: device browser bugs, real perf, iOS Safari divergences. **(5)** Discipline: emulation in CI; BrowserStack/real devices before release. **(6)** What loses the point: emulation-as-equivalent ships iOS-specific bugs to production.

CODE

```
// What emulation sets (devices['iPhone 15'] approximation)
test.use({
viewport: { width: 393, height: 852 },
deviceScaleFactor: 3,
isMobile: true,
hasTouch: true,
userAgent: 'Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/...',
});

// What it does NOT set / cannot set:
// - Real iOS Safari rendering (Playwright WebKit ≠ iOS Safari)
// - CPU and memory constraints of a real phone
// - Real touch hardware (events are synthesised)
// - iOS-specific bugs in font rendering, video, scroll behaviour
// - OS features: push notifications, biometrics, deep links

// Right CI strategy:
// - Emulation in PR + main pipelines (fast, deterministic)
// - BrowserStack/SauceLabs or real-device lab on release branches
// - Smoke test on actual iPhone Safari before customer-visible deploy
```

Q 229 LEVEL · Mid

How do you simulate mobile network conditions (slow 3G, intermittent connectivity, offline)?

CONCEPT

Mobile users experience network conditions desktop testing ignores: slow connections, transitions between wifi and cellular, complete offline. Three patterns. Throttled bandwidth and latency: via CDP's **Network.emulateNetworkConditions** through **context.newCDPSession()**, set **downloadThroughput**, uploadThroughput, and latency to simulate slow 3G (downlink 500 kbps, uplink 500 kbps, 400 ms RTT) or 4G (4 Mbps / 3 Mbps / 70 ms). Chromium-only. Offline: context.setOffline(true) drops all network for the context, tests PWA behaviour, cached content, error states. Intermittent connectivity: toggle offline on and off during the test to simulate spotty cellular, or use a route handler that fails randomly to simulate flaky connections. The discipline: test the slow-network case explicitly for any feature mobile users will use, checkout flows, login, search. A feature that works on a fast connection in dev but takes twelve seconds on real cellular is a production bug that emulation lets you catch.

INTERVIEW STRATEGY

What earns the tick is mobile network simulation handled at depth. Walk three patterns (CDP throttle, setOffline, intermittent toggle). The 'test slow-network explicitly for mobile-user features' discipline is the senior signal. The mistake is testing only on fast networks. A common follow-up is ‘what 3G numbers do you use?’, answer 500 kbps down, 500 kbps up, 400 ms latency, the Chrome DevTools default slow 3G profile.

How to phrase it

> *Mobile users experience network conditions desktop testing ignores: slow connections, transitions between wifi and cellular, complete offline. Three patterns to simulate each, and using the right one is what catches mobile-specific bugs. First, throttled bandwidth and latency. Via CDP's Network dot emulateNetworkConditions through context dot newCDPSession, I set downloadThroughput, uploadThroughput, and latency to simulate slow 3G, downlink five hundred kilobits per second, uplink five hundred, four hundred millisecond round-trip time, or 4G at four megabits down, three up, seventy ms latency. Chromium only, which is a real limitation; Firefox and WebKit don't support CDP throttling. That's also my answer to the follow-up about what 3G numbers I use: five hundred kbps each way and four hundred ms latency, the Chrome DevTools default slow 3G profile. Second, offline. Context dot setOffline true drops all network for the context. Tests PWA behaviour, cached content, error states. When users lose connectivity in the middle of using my app, do they see a sensible offline indicator? Does the service worker serve cached pages? Are in-flight requests retried when connectivity returns? All testable with setOffline. Third, intermittent connectivity. Toggle offline on and off during the test to simulate spotty cellular, the user goes through a tunnel, connectivity drops for ten seconds, returns, drops again. Or use a route handler that fails randomly to simulate flaky connections with packets dropping. Useful for testing retry logic, auto-save behaviour, error UI under real-world conditions rather than the idealised online-or-offline binary. The discipline I would close on is test the slow-network case explicitly for any feature mobile users will use, checkout flows, login, search. A feature that works on a fast connection in dev but takes twelve seconds on real cellular is a production bug that emulation lets you catch before users hit it. The pitfall is testing only on the dev machine's fast wifi; the feature ships, mobile users rage-quit because it never loads.*

Key points to hit

(1) CDP Network.emulateNetworkConditions: throttle bandwidth + latency (Chromium only). **(2)** Slow 3G profile: 500 kbps down, 500 kbps up, 400 ms latency. **(3)** context.setOffline(true): full network drop for PWA and error-state testing. **(4)** Intermittent: toggle offline during test, or random-fail route handler. **(5)** Discipline: test slow-network for any mobile-user feature. **(6)** The common miss: testing on fast wifi only, ships features that break on real cellular.

CODE

```
// Slow 3G simulation via CDP
test('checkout works on slow 3G', async ({ page, context }) => {
const cdp = await context.newCDPSession(page);
await cdp.send('Network.emulateNetworkConditions', {
offline: false,
downloadThroughput: (500 * 1024) / 8, // 500 kbps in bytes/s
uploadThroughput: (500 * 1024) / 8,
latency: 400, // ms RTT
});
// Run the checkout flow under slow-3G conditions; assert loading states render
});

// Offline scenario
test('PWA serves cached content offline', async ({ page, context }) => {
await page.goto('/dashboard');
await context.setOffline(true);
await page.reload();
await expect(page.getByText('You’re offline')).toBeVisible();
await expect(page.getByTestId('cached-content')).toBeVisible();
await context.setOffline(false);
});

// Intermittent connectivity
test('auto-save retries on connection drop', async ({ page, context }) => {
await page.getByLabel('Draft text').fill('important content');
await context.setOffline(true);
// ... assert offline indicator + queued save
await context.setOffline(false);
await expect(page.getByText('Saved')).toBeVisible({ timeout: 10_000 });
});
```

Q 230 LEVEL · Mid

How do you handle mobile-specific browser APIs: geolocation, permissions, device orientation?

CONCEPT

Mobile browsers expose APIs desktop browsers don't use the same way, and Playwright provides context-level controls for each. Geolocation: context.setGeolocation({ latitude, longitude }) sets the position returned by navigator.geolocation, useful for testing location-aware features like store locators or delivery zones. Permissions: **context.grantPermissions(['geolocation', 'notifications', 'camera'])** grants without the permission prompt. To test the prompt itself, skip granting and assert the dialog appears. Device orientation isn't natively controlled by Playwright, there's no portrait/landscape API, but you can simulate by setting the viewport to portrait then landscape dimensions via **page.setViewportSize**, which matches what happens when a user rotates. The discipline: permissions and geolocation set per test or per context, not globally, because tests need specific permission states to verify the feature, a test of the permission-denied path needs the permission explicitly denied via **context.clearPermissions()**. The trap: leaving permissions granted at config level and missing tests of the prompt-rejection path.

INTERVIEW STRATEGY

You need to give mobile-API control fluency. List the three (geolocation, permissions, orientation-via-viewport) with the right API for each. The 'set per test, verify permission states explicitly' discipline is what separates junior from senior thinking. The trap is global permissions hiding prompt-rejection paths. The follow-up is often ‘how do you test the rotation handler?’, answer call setViewportSize to swap portrait/landscape dimensions; layout changes are what the rotation handler actually responds to.

How to phrase it

> *Mobile browsers expose APIs desktop browsers don't use the same way, and Playwright provides context-level controls for each. Three to know well. Geolocation: context dot setGeolocation with latitude and longitude sets the position returned by navigator dot geolocation, useful for testing location-aware features like store locators, delivery zones, regional pricing. Combined with permissions, context dot grantPermissions array geolocation, the test bypasses the location prompt and sees a specific position. Permissions: context dot grantPermissions array geolocation notifications camera grants without the permission prompt. To test the prompt itself, I skip granting and assert the dialog appears. To test the rejection path, I deny via context dot clearPermissions before the action and verify the app handles the denial gracefully, the permission-denied path is what users hit when they tap deny, and it needs explicit test coverage. Device orientation isn't natively controlled by Playwright, there's no portrait-or-landscape API, but I can simulate by setting the viewport to portrait then landscape dimensions via page dot setViewportSize, which matches what happens when a user rotates: the viewport dimensions swap, layout responds, JS resize handlers fire. That's also my answer to the follow-up about testing the rotation handler: setViewportSize swap produces the same DOM and layout events a real rotation would. The discipline I would close on, because it's where most candidates miss the nuance, is permissions and geolocation set per test or per context, not globally. Tests need specific permission states to verify the feature: a test of the permission-denied path needs the permission explicitly denied via context dot clearPermissions; a test of the granted path needs it granted. Setting permissions globally at config level means every test starts with the same state, and the prompt-rejection path never gets tested. The thing to avoid is leaving permissions granted at config level and missing tests of the prompt-rejection path, which is exactly the path that crashes the app for real users who tap deny.*

Key points to hit

(1) context.setGeolocation({ latitude, longitude }) for location-aware features. **(2)** context.grantPermissions for geolocation, notifications, camera, clipboard. **(3)** Orientation via page.setViewportSize: swap dimensions to simulate rotation. **(4)** Set permissions per test, not globally: prompt-rejection paths need coverage. **(5)** Asked next: rotation handler tested via setViewportSize, layout/resize events fire. **(6)** Avoid: global granted permissions, prompt-rejection path never tested.

CODE

```
// Geolocation + permission for store locator
test.use({
geolocation: { latitude: 19.0760, longitude: 72.8777 }, // Mumbai
permissions: ['geolocation'],
});

test('store locator shows nearest store', async ({ page }) => {
await page.goto('/stores');
await expect(page.getByTestId('nearest-store')).toContainText('Mumbai');
});

// Test the permission-denied path explicitly
test('app handles location denial gracefully', async ({ page, context }) => {
await context.clearPermissions(); // no permissions granted
await page.goto('/stores');
await page.getByRole('button', { name: 'Use my location' }).click();
await expect(page.getByText('Allow location access in settings')).toBeVisible();
});

// Rotation via viewport swap
test('layout responds to rotation', async ({ page }) => {
await page.setViewportSize({ width: 393, height: 852 }); // portrait
await expect(page.locator('[data-orientation="portrait"]')).toBeVisible();
await page.setViewportSize({ width: 852, height: 393 }); // landscape
await expect(page.locator('[data-orientation="landscape"]')).toBeVisible();
});
```

Q 231 LEVEL · Senior

How do you test a Progressive Web App's service worker, offline mode, and install flow?

CONCEPT

PWAs add capabilities desktop web apps don't have, each requiring its own test approach. Service worker registration: verify the SW registers via **navigator.serviceWorker.ready** on page load, a Playwright test polls until the service worker is in the 'activated' state, then proceeds. **Offline mode**: combine **context.setOffline(true)** with assertions that cached pages render from the SW cache rather than failing with no-network errors. Cache management: test that the SW serves the right cached version after a deploy, tricky because cache invalidation is the bug category PWAs are most prone to. Install prompt (the beforeinstallprompt event): Playwright can't trigger the native install prompt directly, but you can assert the install UI appears when conditions are met, and manually trigger the event in a page evaluate for the code path that handles it. The discipline: test the cache-update path specifically because it's the PWA failure mode most likely to ship a stale UI to users. Run a test that loads the app, updates the SW, reloads, and asserts the new version is served, not the cached old one.

INTERVIEW STRATEGY

They are after PWA testing at depth. Walk the four areas (SW registration, offline, cache management, install prompt) with what each tests. The 'cache-update path specifically' discipline is the senior signal because stale-cache bugs are the PWA classic. The thing to avoid is testing only the happy installation path. Expect the follow-up 'what's the worst PWA bug you've seen?', answer

stale cache serving an old build for hours after deploy.

How to phrase it

> *PWAs add capabilities desktop web apps don't have, and each requires its own test approach. Service worker registration first: I verify the SW registers via navigator dot serviceWorker dot ready on page load. A Playwright test polls until the service worker is in the activated state, then proceeds with the rest of the test. Without that wait, subsequent tests interact with a page where the SW hasn't taken over yet, producing inconsistent caching behaviour. Offline mode: I combine context dot setOffline true with assertions that cached pages render from the SW cache rather than failing with no-network errors. This is the core PWA value proposition, the app works without network, and the test verifies it actually does. Cache management: I test that the SW serves the right cached version after a deploy. This is tricky because cache invalidation is the bug category PWAs are most prone to. Install prompt: Playwright can't trigger the native install prompt directly because that's a browser-OS interaction, but I can assert the install UI appears when the conditions are met, and manually trigger the beforeinstallprompt event in a page dot evaluate to test the code path that handles user acceptance. The discipline I would lead with, and where PWA test suites earn their place, is test the cache-update path specifically. It's the PWA failure mode most likely to ship a stale UI to users, the deploy completes, but users keep seeing yesterday's build because the SW is serving cached responses and not checking for updates. My test loads the app, programmatically updates the SW, reloads, and asserts the new version is served, not the cached old one. That's also my answer to the follow-up about the worst PWA bug I've seen: stale cache serving an old build for hours after deploy. Customers see the old UI, support fields confused tickets, the feature that was supposed to ship today is invisible. The fix, cache-busting headers, SW skipWaiting, service-worker-update-on-reload, is well-known, but the regression test for it is what makes sure it doesn't come back.*

Key points to hit

(1) SW registration: poll navigator.serviceWorker.ready until 'activated'. **(2)** Offline: setOffline(true) + assert cached pages render from SW cache. **(3)** Cache management: test the update path, deploy serves new version, not stale. **(4)** Install prompt: can't trigger natively, but test the handler via page.evaluate. **(5)** Discipline: cache-update path is the PWA failure most likely to ship. **(6)** The easy mistake: only testing the happy install path, stale-cache regressions ship.

CODE

```
test('service worker registers and activates', async ({ page }) => {
await page.goto('/');
await page.waitForFunction(async () => {
const reg = await navigator.serviceWorker.ready;
return reg.active?.state === 'activated';
});
});

test('app works offline', async ({ page, context }) => {
await page.goto('/');
await page.waitForFunction(() => navigator.serviceWorker.ready);
await context.setOffline(true);
await page.reload();
await expect(page.getByRole('heading', { name: 'Welcome' })).toBeVisible();
});

test('cache-update path serves new version after deploy', async ({ page }) => {
await page.goto('/');
await page.evaluate(async () => {
const reg = await navigator.serviceWorker.ready;
await reg.update(); // simulate a deploy-triggered update
});
await page.reload();
await expect(page.getByTestId('build-id')).not.toHaveText(/old-build-/);
});

// Trigger beforeinstallprompt manually for install-handler testing
// await page.evaluate(() => window.dispatchEvent(new Event('beforeinstallprompt')));
```

Q 232 LEVEL · Mid to Senior

Where does Playwright's mobile-web scope end, and when do you reach for Appium or native tools?

CONCEPT

Playwright tests **mobile web only**: it drives browsers, not native apps. The boundary matters because mobile teams often have three artifact types: mobile web (responsive websites and PWAs), native apps (iOS Swift/Objective-C, Android Kotlin/Java), and hybrid apps (web views wrapped in a native shell via Cordova, Capacitor, Ionic). Playwright is the right tool for mobile web and the web-view portions of hybrid apps you can access via a browser URL. Appium is the right tool for native apps and the native-shell portions of hybrid apps (navigation transitions, biometric prompts, share sheets, native widgets). **Detox** (React Native) and XCUITest/Espresso (platform-native) are the right tools for deeper native automation when Appium's abstractions get in the way. The pragmatic rule: match the tool to the artifact, not the team's framework preference. A team using Playwright for web testing doesn't extend Playwright to native automation; they use Appium alongside Playwright, with the test suite for each artifact in its own tooling. The trap: **forcing Playwright into the native-app space** via uncommon workarounds, wrappers around Appium that pretend to be Playwright, ends up with the worst of both.

INTERVIEW STRATEGY

They want tool calibration across the range of real devices. Walk the three artifact types and the right tool for each (Playwright for web/web-views, Appium for native, Detox/XCUITest/Espresso for deep native). The 'match tool to artifact, not team framework preference' rule signals depth here. The failure mode is Playwright wrappers around Appium. They usually follow with ‘how do you split ownership?’, answer one suite per artifact, share contracts at the API boundary.

How to phrase it

> *Playwright tests mobile web only: it drives browsers, not native apps. The boundary matters because mobile teams often have three artifact types, and treating them as one is where senior candidates lose points. Mobile web: responsive websites and PWAs, what users see when they visit your URL on a phone. Native apps: iOS in Swift or Objective-C, Android in Kotlin or Java, downloaded from the App Store or Play Store. Hybrid apps: web views wrapped in a native shell via Cordova, Capacitor, or Ionic, mostly web under the hood but with native chrome around it. Playwright is the right tool for mobile web and the web-view portions of hybrid apps that I can access via a browser URL. Same DOM, same JavaScript, same auto-waiting discipline. Appium is the right tool for native apps and the native-shell portions of hybrid apps, navigation transitions, biometric prompts, share sheets, native widgets, anything outside the web view. Appium speaks WebDriver against the iOS Simulator or Android Emulator and drives native UI elements. Detox for React Native, and XCUITest and Espresso for platform-native, are the right tools for deeper native automation when Appium's abstractions get in the way, deeply-integrated UI tests, performance harnesses, anything where the WebDriver indirection is a cost. The pragmatic rule I would lead with, because it's the calibration senior interviewers want to hear, is match the tool to the artifact, not the team's framework preference. A team using Playwright for web testing doesn't extend Playwright to native automation. They use Appium alongside Playwright, with the test suite for each artifact in its own tooling. Web team owns Playwright, native team owns Appium-or-deeper, shared contracts at the API boundary so both can verify the same backend. That's also my answer to the follow-up about splitting ownership: one suite per artifact, shared contracts at the API. The trap is forcing Playwright into the native-app space via uncommon workarounds, wrappers around Appium that pretend to be Playwright, headless WebKit instances driven outside their normal use case. These end up with the worst of both worlds: not as integrated with the native platform as a real native tool, more complex than just using Appium directly. Use the right tool for the artifact.*

Key points to hit

(1) Playwright = mobile web + web-view portions of hybrid apps. **(2)** Appium = native apps + native-shell portions of hybrid apps. **(3)** Detox/XCUITest/Espresso = deep native, when Appium's overhead matters. **(4)** Three artifact types: mobile web, hybrid, native, each gets the right tool. **(5)** Rule: match tool to artifact, not team's framework preference. **(6)** Trap: forcing Playwright into native space via wrappers, worst of both.

CODE

```
// Mobile WEB: Playwright (responsive site, PWA, hybrid web-view)
test.use({ ...devices['iPhone 15 Pro'] });
test('mobile checkout flow', async ({ page }) => {
await page.goto('https://m.acme.com/checkout');
// ... web test, same patterns as desktop with mobile viewport
});

// NATIVE APP: Appium (iOS Simulator / Android Emulator)
// (different tooling, different test runner: NOT Playwright)
//
// const driver = await remote({
// capabilities: {
// platformName: 'iOS', 'appium:deviceName': 'iPhone 15', 'appium:app': '/path/Acme.app'
// },
// });
// await driver.$('~Sign In').click();

// HYBRID APP: Playwright for the web-view portion + Appium for native chrome
// Two test suites, shared API contracts, different tools for what each does best.
//
// The architecture decision: one suite per artifact, not one tool everywhere.
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What kind of mobile testing does Playwright provide?

A) Browser-based mobile web emulation (viewport, UA, DPR, touch)
B) Native iOS and Android app testing
C) Real-device cloud testing only
D) Performance profiling of native apps

2. What is the honest limitation of touch-gesture testing in Playwright?

A) tap() does not work
B) Touch is unsupported entirely
C) Complex gestures (swipe, pinch) via dispatchEvent are verbose and fragile
D) Gestures only work on desktop

3. Where in CI should parameterised responsive-breakpoint tests run?

A) On every PR
B) They should not be automated
C) Only on release candidates
D) In the nightly build: too slow for every PR but high value

4. Roughly what share of mobile web testing does emulation cover well?

A) About 50%: real devices needed for the rest
B) 100%, real devices are never needed
C) About 90%: real devices needed for the gesture-heavy 10%
D) About 10%: mostly real devices

5. Which mobile-specific bugs does device emulation most commonly surface?

A) Database deadlocks and API timeouts
B) Elements hidden behind the mobile nav and touch targets too small to tap
C) Memory leaks in native code
D) Server-side rendering errors

Section 18 · Answer key

#	Answer	Why and where to revisit
1	A) Browser-based mobile web emulation (viewport, UA, DPR, touch)	Playwright emulates mobile web via device descriptors; native apps need Appium or Detox. *Revisit Section 18.*
2	C) Complex gestures (swipe, pinch) via dispatchEvent are verbose and fragile	tap() is reliable; gesture-heavy apps (maps, galleries) are better tested on real devices via BrowserStack/Sauce. *Revisit Section 18.*
3	D) In the nightly build: too slow for every PR but high value	Multiple viewports plus snapshots are too slow to gate PRs but valuable enough to run nightly. *Revisit Section 18.*
4	C) About 90%: real devices needed for the gesture-heavy 10%	Emulation handles layout, taps and mobile UI well (~90%); gesture-heavy features need real devices. *Revisit Section 18.*
5	B) Elements hidden behind the mobile nav and touch targets too small to tap	Emulation catches responsive issues a desktop suite misses, hidden-behind-nav elements and tiny tap targets. *Revisit Section 18.*

SECTION 19

Microservices and API Architecture

Microservices questions test whether you can reason about a system, not just a single app, independent services, the boundaries between them, and the asynchronous glue that connects them. These eight questions cover the contract-versus-integration testing split, treating the API gateway as a security surface, and testing event-driven services through a message queue with expect.poll.

In this section

- Three levels: contract per service, integration per interaction, E2E per release.
- Contract tests as the highest-value investment; Pact for consumer-driven contracts.
- API-gateway tests are security tests: auth enforcement, rate limiting, CORS.
- Event-driven testing: trigger the action, poll for the downstream effect.
- expect.poll with backoff validates the whole event chain, not just the endpoint.

Q 233 LEVEL · Mid to Senior

How do you test microservices with Playwright?

CONCEPT

Microservices testing means testing each service **independently** (contract tests) and testing service interactions (integration tests), both via the request fixture. There are effectively three levels. **Contract tests per service**: fast, isolated, run on every PR for that service, validating the API shape (id is any number, email matches a pattern, role is one of the allowed values) rather than the implementation. Integration tests for interactions, create a user via the User Service, then an order referencing it via the Order Service, run when either service changes. E2E tests through the UI: reserved for release candidates. Contract tests are the highest-value investment because they catch breaking changes before integration tests even run; **Pact** adds consumer-driven contract testing for complex environments.

INTERVIEW STRATEGY

Show you reason in levels, not treat microservices as one big app. Lay out the three, contract, integration, E2E, with what each runs on, then single out contract tests as the highest-value investment because they fail fast on a breaking change. Mentioning Pact for consumer-driven contracts shows depth. The risk is testing everything only through the UI/E2E. The next question is normally 'which level catches a breaking change first?', answer the per-service contract test, before integration even runs.

How to phrase it

> *I distinguish three levels in microservices testing, and laying them out is how I show I think about the system rather than one app. First, contract tests per service, fast, isolated, run on every PR for that service, and they validate the contract, not the implementation: that the id is any number, the email matches a pattern, the role is one of the allowed values. Second, integration tests for service interactions, like creating a user through the User Service and then an order that references that user through the Order Service, which I run when either service changes. Third, E2E tests through the UI, reserved for release candidates because they are slow and broad. The contract tests are the most important investment, and that is my answer to the follow-up about which level catches a breaking change first: a per-service contract test fails the instant the backend renames or retypes a field, before the integration tests even run, so the break is caught at the cheapest, fastest stage and attributed to the right service. The trap I avoid is testing everything through the UI, which is slow and makes it impossible to tell which service actually broke. For complex environments I use Pact for consumer-driven contract testing, where each consumer declares what it needs from a provider and the provider is verified against all its consumers' expectations, that scales the contract idea across many services without an explosion of brittle integration tests.*

Key points to hit

(1) Three levels: contract (per PR), integration (on change), E2E (release). **(2)** Contract tests validate the API shape, not the implementation. **(3)** Contract tests are the highest-value investment, fail fast on breaks. **(4)** Pact for consumer-driven contracts in complex environments. **(5)** Follow-up: the per-service contract test catches a break first. **(6)** Where this goes wrong: testing everything through the UI/E2E.

CODE

```
// Contract test: validates a service in isolation
test('User Service: GET /users/:id contract', async ({ request }) => {
const res = await request.get(`${process.env.USER_SERVICE_URL}/users/1`);
await expect(res).toBeOK();
expect(await res.json()).toMatchObject({
id: expect.any(Number),
email: expect.stringMatching(/@/),
role: expect.stringMatching(/^(admin|user|viewer)$/),
});
});

// Integration test: validates a service interaction
test('Order Service references User Service', async ({ request }) => {
const { id: userId } = await (await
request.post(`${process.env.USER_SERVICE_URL}/users`, { data: { email: 't@x.com', role:
'user' } })).json();
const order = await request.post(`${process.env.ORDER_SERVICE_URL}/orders`, { data: {
userId, items: [{ productId: 1, quantity: 2 }] } });
expect(order.status()).toBe(201);
});
```

Q 234 LEVEL · Mid

How do you test the API gateway and authentication middleware?

CONCEPT

API-gateway tests validate routing, authentication enforcement, rate limiting, and CORS, pure API tests via the request fixture. The framing that matters: these are **security tests disguised as functional tests**. Testing that an unauthenticated request returns **401** verifies the auth middleware is actually applied. Testing that the threshold trips a 429 verifies the gateway protects backend services from abuse. Testing **CORS headers** for an allowed origin matters especially for SPAs, where a misconfigured CORS policy causes silent browser failures that are painful to diagnose. You run these in the security suite as well as the functional one, because the gateway is the single enforcement point for cross-cutting protections, if it is wrong, every service behind it is exposed.

INTERVIEW STRATEGY

They are looking to see you recognise the gateway as a security surface, not just routing. Frame these as security tests in functional clothing, and give the three concrete checks, 401 for auth, 429 for rate limiting, CORS headers, each with what it actually verifies. The SPA/CORS silent-failure note shows real experience. The classic error is treating gateway tests as plain routing checks. A common follow-up is ‘why test the gateway separately from the services?’, answer it is the single enforcement point for cross-cutting protections.

How to phrase it

> *API-gateway tests validate routing, authentication enforcement, rate limiting and CORS, and they are pure API tests through the request fixture, but the framing I lead with is that these are really security tests disguised as functional tests. Testing that an unauthenticated request to a protected resource returns 401 verifies the authentication middleware is genuinely applied, not just assumed. Testing that firing past the threshold returns 429 verifies the gateway protects the backend services from abuse. And testing the CORS headers for an allowed origin matters especially for SPAs, because a misconfigured CORS policy causes silent failures in the browser that are genuinely painful to diagnose, the request just fails with no obvious server error. That is also my answer to the follow-up about why I test the gateway separately from the services: the gateway is the single enforcement point for these cross-cutting protections, so if its auth or rate limiting is wrong, every service behind it is exposed regardless of how well each service is tested. I run these in the security suite as well as the functional suite, on every release candidate. The failure mode is treating gateway tests as plain routing checks and missing that they are the verification that the system's perimeter actually holds. Catching a missing auth check here is far cheaper than discovering it as a breach in production.*

Key points to hit

(1) Gateway tests cover routing, auth, rate limiting, CORS, via request fixture. **(2)** Framing: security tests disguised as functional tests. **(3)** 401 verifies auth applied; 429 verifies abuse protection; CORS for SPAs. **(4)** Misconfigured CORS causes silent, hard-to-diagnose browser failures. **(5)** Expect next: test the gateway separately, it's the single enforcement point. **(6)** What loses the point: treating gateway tests as plain routing checks.

CODE

```
test('unauthenticated request returns 401', async ({ request }) => {
const res = await request.get('/api/protected-resource');
expect(res.status()).toBe(401);
});

test('rate limiting returns 429 past the threshold', async ({ request }) => {
const responses = await Promise.all(Array.from({ length: 101 }, () =>
request.get('/api/public-endpoint')));
expect(responses.filter(r => r.status() === 429).length).toBeGreaterThan(0);
});

test('CORS header present for an allowed origin', async ({ request }) => {
const res = await request.get('/api/data', { headers: { Origin:
'https://allowed-origin.com' } });
expect(res.headers()['access-control-allow-origin']).toBe('https://allowed-origin.com');
});
```

Q 235 LEVEL · Mid to Senior

How do you test event-driven microservices with message queues?

CONCEPT

Event-driven systems need you to verify that events are published correctly and consumers process them correctly. The pattern is trigger an action, then poll for the downstream effect, because the trigger is synchronous but the effect is asynchronous. Placing an order publishes an OrderPlaced event; the Inventory Service consumes it and decrements stock some time later. So you capture the initial stock, place the order, then **expect.poll** the inventory endpoint with progressive backoff (500ms, 1s, 2s, 5s) until it reaches the expected value. In a healthy system the update arrives within a second or two; if it takes longer than the timeout, something is wrong with the event pipeline and the test should **fail loudly**. This validates the **entire event chain**, publish, queue, consume, persist, not just one endpoint.

INTERVIEW STRATEGY

The aim is to know you handle async propagation correctly, not with sleeps. Make the trigger-then-poll pattern explicit and tie it to expect.poll with backoff, stressing that it validates the whole event chain rather than a single endpoint. Noting that a too-long wait should fail loudly shows you treat a slow pipeline as a real signal. The classic error is a fixed sleep before checking inventory. The follow-up is often 'what does this test actually cover?', answer the entire chain: publish, queue, consume, persist.

How to phrase it

> *Event-driven systems need me to verify two things: that events are published correctly and that consumers process them correctly. The pattern I use is trigger the action, then poll for the downstream effect, because the trigger is synchronous but the effect is not. Concretely, placing an order publishes an OrderPlaced event, the Inventory Service consumes it off the queue and decrements stock a little later. So I capture the initial stock, place the order, then use expect.poll on the inventory endpoint with progressive backoff, check at five hundred milliseconds, one second, two, five, until the quantity reaches the expected value. In a healthy system that update arrives within a second or two; if it takes longer than my fifteen-second timeout, something is genuinely wrong with the event pipeline and I want the test to fail loudly rather than wait forever, because a slow pipeline is itself a defect worth surfacing. That is also my answer to the follow-up about what this test actually covers: it validates the entire event chain, the order endpoint publishing the event, the queue delivering it, the inventory consumer processing it, and the new state persisting, not just a single API call. The trap is reaching for a fixed sleep before checking inventory, which is either too short and flaky on a slow run or too long and wasteful on every run. expect.poll returns the moment the state is correct, so it is both fast and reliable for testing asynchronous, eventually consistent event flows.*

Key points to hit

(1) Verify events are published and consumers process them. **(2)** Pattern: trigger the synchronous action, poll for the async effect. **(3)** expect.poll with backoff (500ms/1s/2s/5s) to the expected value. **(4)** A too-long wait should fail loudly: a slow pipeline is a defect. **(5)** They usually ask next: validates the whole chain, publish, queue, consume, persist. **(6)** The common miss: a fixed sleep before checking the downstream effect.

CODE

```
test('order placement triggers inventory update', async ({ request }) => {
const initial = (await (await request.get('/api/inventory/PROD-001')).json()).quantity;

await request.post('/api/orders', { data: { items: [{ productId: 'PROD-001', quantity: 2
}] } });

await expect.poll(async () => {
return (await (await request.get('/api/inventory/PROD-001')).json()).quantity;
}, {
message: 'Inventory should decrease by 2 after the order',
timeout: 15_000,
intervals: [500, 1000, 2000, 5000],
}).toBe(initial - 2);
});
```

Q 236 LEVEL · Senior

How do you test gRPC services from Playwright, given Playwright is HTTP-focused?

CONCEPT

Playwright's APIRequestContext is HTTP/HTTPS, not gRPC. For gRPC services, the pattern is add gRPC-Web to the service and let Playwright speak HTTP/2 to it, or use a dedicated gRPC client library alongside Playwright. Three approaches by service architecture. **gRPC-Web gateway**: most production gRPC services expose a gRPC-Web endpoint for browser clients; Playwright tests can hit that endpoint with standard request methods, treating gRPC-Web as another HTTP API. **Dedicated gRPC client**: for tests that need pure gRPC (not gRPC-Web), instantiate a Node gRPC client (**@grpc/grpc-js** with the generated stubs) inside a Playwright fixture and call it directly, you get Playwright's test runner, parallelism, and reporting around a non-HTTP client. Hybrid: UI in browser + gRPC verification: the test drives the UI via Playwright, then verifies state through a gRPC client called from the test code. The discipline: match the protocol to how production clients actually call the service. If real browsers use gRPC-Web, test through gRPC-Web; if the service is called server-to-server in production, test with a pure gRPC client. Testing a different protocol than production uses misses protocol-specific bugs.

INTERVIEW STRATEGY

What earns the tick is gRPC tested correctly despite Playwright's HTTP focus. Walk the three approaches: gRPC-Web gateway (Playwright as HTTP), dedicated gRPC client in fixture, hybrid UI+gRPC. The 'match protocol to how production calls the service' rule is the seniority marker. The trap is testing only gRPC-Web when production is pure gRPC. Expect the follow-up ‘what if there's no gRPC-Web?’, answer @grpc/grpc-js with the generated stubs inside a Playwright fixture.

How to phrase it

> *Playwright's APIRequestContext is HTTP and HTTPS, not gRPC. So for gRPC services I have to choose between adding gRPC-Web to the service and letting Playwright speak HTTP-slash-two to it, or using a dedicated gRPC client library alongside Playwright. Three approaches by service architecture. First, gRPC-Web gateway. Most production gRPC services expose a gRPC-Web endpoint for browser clients because raw gRPC doesn't work in browsers. Playwright tests can hit that gRPC-Web endpoint with standard request methods, treating it as another HTTP API. Status codes, headers, body: all the patterns from earlier in the kit apply. Second, dedicated gRPC client. For tests that need pure gRPC, not gRPC-Web, I instantiate a Node gRPC client, at-grpc-slash-grpc-js with the generated stubs from the proto files, inside a Playwright fixture and call it directly. I get Playwright's test runner, parallelism, and reporting around a non-HTTP client, which is the productivity win even if the protocol is different. Third, hybrid: UI in browser plus gRPC verification. The test drives the UI via Playwright, then verifies state through a gRPC client called from the test code. Common when the UI is web but the verification API is internal and gRPC-only. The discipline I would lead with, because it's the rule that prevents the worst-case mismatch, is match the protocol to how production clients actually call the service. That's also my answer to the follow-up about what if there's no gRPC-Web: at-grpc-slash-grpc-js with the generated stubs inside a Playwright fixture, full pure-gRPC test. If real browsers use gRPC-Web, test through gRPC-Web; if the service is called server-to-server in production, test with a pure gRPC client. Testing a different protocol than production uses misses protocol-specific bugs, streaming responses behave differently in pure gRPC versus gRPC-Web, deadlines work differently, the wire format diverges. The mistake is testing the gRPC-Web gateway because Playwright makes it easy when the service ships to production as pure gRPC. The tests pass through gRPC-Web, the production traffic uses gRPC, and the gRPC-specific bug ships.*

Key points to hit

(1) Playwright is HTTP/HTTPS only: no native gRPC support. **(2)** Option 1: gRPC-Web gateway, Playwright hits it as HTTP/2. **(3)** Option 2: @grpc/grpc-js client in a fixture for pure gRPC. **(4)** Option 3: hybrid, Playwright drives UI, gRPC client verifies state. **(5)** Rule: match the test protocol to how production callers actually use it. **(6)** Avoid: testing gRPC-Web when production traffic is pure gRPC, misses protocol bugs.

CODE

```
// Approach 1: gRPC-Web gateway, Playwright as HTTP client
test('order service via gRPC-Web', async ({ request }) => {
const res = await request.post('/grpc.OrderService/CreateOrder', {
headers: { 'content-type': 'application/grpc-web+proto' },
data: encodeOrderRequest({ product: 'X', qty: 1 }),
});
expect(res.status()).toBe(200);
});

// Approach 2: dedicated gRPC client inside a Playwright fixture
import * as grpc from '@grpc/grpc-js';
import { OrderServiceClient } from './generated/order_grpc_pb';

export const test = base.extend<{ orderClient: OrderServiceClient }>({
orderClient: async ({}, use) => {
const client = new OrderServiceClient('order-service:50051',
grpc.credentials.createInsecure());
await use(client);
client.close();
},
});

// Approach 3: hybrid, Playwright drives UI, gRPC client verifies state
test('UI creates order, gRPC verifies state', async ({ page, orderClient }) => {
// ... drive UI via page ...
const order = await new Promise<Order>(r => orderClient.getOrder({ id: 'ord_123' }, (_,
res) => r(res!)));
expect(order.status).toBe('confirmed');
});
```

Q 237

LEVEL · Mid to Senior

How do you use distributed tracing to diagnose test failures across microservices?

CONCEPT

Distributed tracing (OpenTelemetry, Jaeger, Zipkin, Honeycomb) gives each request a unique **trace ID** that propagates through every service it touches. The diagnostic pattern: have the test emit a known trace ID with each request via the W3C Trace Context header (**traceparent**), and when a test fails, query the tracing backend by that trace ID to see exactly which service produced the error and what it received. The implementation: Playwright sets an **extraHTTPHeaders** containing the traceparent on the request context; the test logs the trace ID on failure; the engineer pastes it into Jaeger or Honeycomb to see the full distributed call tree. The win that scales: diagnosing a failure that involves five services is ten times faster with trace IDs than without. Without trace IDs, the engineer manually grep s logs across services trying to find correlated entries; with them, one query returns the entire call graph timestamped and ordered. The discipline: generate trace IDs at the test level, log them on every test failure, surface them in the HTML report, so the trace ID is always at the engineer's fingertips when they open a failure.

INTERVIEW STRATEGY

You need to give distributed tracing as a test-diagnosis tool. Lead with trace IDs via traceparent header and how the test surfaces them on failure. The '10x faster diagnosis on multi-service failures' framing signals depth here. The classic error is logging trace IDs but not linking them in test output. They usually follow with ‘where do you log the trace ID?’, answer test annotations + HTML report attachment so it's visible at the failure page.

How to phrase it

> *Distributed tracing gives each request a unique trace ID that propagates through every service it touches, OpenTelemetry, Jaeger, Zipkin, Honeycomb all work this way. The diagnostic pattern, and the part that makes microservice testing actually tractable, is have the test emit a known trace ID with each request via the W3C Trace Context header, traceparent, and when a test fails, query the tracing backend by that trace ID to see exactly which service produced the error and what it received. The implementation: Playwright sets an extraHTTPHeaders containing the traceparent on the request context. The test logs the trace ID on failure, ideally as a test annotation so it surfaces in the HTML report. The engineer investigating the failure pastes the trace ID into Jaeger or Honeycomb and sees the full distributed call tree, which services were called in what order, with what latencies, with what payloads, and which span produced the error. The win that scales, and the framing I would lead with because it's where this pays off in practice, is diagnosing a failure that involves five services is ten times faster with trace IDs than without. Without trace IDs, the engineer manually greps logs across services trying to find correlated entries by timestamp and request shape, which works for one failure and gets impossible at scale. With trace IDs, one query returns the entire call graph timestamped and ordered, with payloads and timing on every span. The discipline I would close on is generate trace IDs at the test level, log them on every test failure, surface them in the HTML report. The trace ID is always at the engineer's fingertips when they open a failure, one click from the test report to the trace in Jaeger, like the trace viewer link pattern from S12 but pointing at the distributed tracing system instead of the Playwright trace. That's also my answer to the follow-up about where to log the trace ID: test annotations plus HTML report attachment so it's visible at the failure page. The pitfall is logging trace IDs but not linking them where engineers will see them; the data exists but nobody finds it during the rushed investigation of a failed CI run.*

Key points to hit

(1) Trace ID via traceparent header propagates across all services. **(2)** Test emits a generated trace ID; failure surfaces it in annotations + report. **(3)** Engineer queries Jaeger/Honeycomb by trace ID: full call tree in one query. **(4)** 10x faster diagnosis on multi-service failures vs grepping logs. **(5)** Discipline: generate at test level, log on failure, surface in HTML report. **(6)** The easy mistake: logging trace IDs but not linking where engineers look during investigation.

CODE

```
// Generate a trace ID per test, propagate via traceparent
import { randomBytes } from 'crypto';

function newTraceparent(): string {
const traceId = randomBytes(16).toString('hex');
const spanId = randomBytes(8).toString('hex');
return `00-${traceId}-${spanId}-01`; // W3C format
}

export const test = base.extend<{ tracedRequest: APIRequestContext; traceId: string }>({
traceId: async ({}, use) => { await use(newTraceparent()); },
tracedRequest: async ({ traceId, playwright }, use) => {
const ctx = await playwright.request.newContext({
extraHTTPHeaders: { traceparent: traceId },
});
await use(ctx);
},
});

test('cross-service order flow', async ({ tracedRequest, traceId }, testInfo) => {
testInfo.annotations.push({ type: 'trace_id', description: traceId });
// ... test body ...
// On failure: trace ID appears in HTML report + annotation -> paste into Jaeger
});
```

Q 238

LEVEL · Senior

How do you test resilience patterns: circuit breakers, retries, and timeouts?

CONCEPT

Resilience patterns prevent cascade failures, and each needs explicit test coverage because they only fire under conditions that don't occur in normal operation. **Circuit breakers**: after N consecutive failures, the breaker opens and short-circuits subsequent requests for a cooldown period. Test by mocking the downstream to fail N times, then verifying the next call fails fast (without hitting the downstream) and returns to closed state after cooldown. **Retries**: failed requests retry with exponential backoff. Test by mocking transient failures and asserting the request is retried the expected number of times with the expected delays. **Timeouts**: requests exceeding a deadline are aborted. Test by mocking a slow downstream and asserting the caller gives up at the right time, not waiting indefinitely. The discipline that distinguishes a senior implementation: each resilience pattern is tested in isolation, with its expected behaviour explicit. Not ‘the system is resilient’ but ‘after 5 consecutive failures, the breaker opens for 30 seconds; during open, requests fail in under 100ms with a CircuitOpen error; after 30 seconds, the next request is allowed through’. Resilience is specific behaviour; the tests have to be specific too.

INTERVIEW STRATEGY

They are after resilience patterns tested specifically, not vaguely. Walk circuit breaker, retry, timeout with the specific behaviour each test should verify. The 'specific numeric behaviour, not the system is resilient' framing is the senior signal. The risk is vague resilience claims without numbers. The next question is normally ‘what's the hardest to test?’, answer the recovery path after the breaker closes;

needs precise timing control.

How to phrase it

> *Resilience patterns prevent cascade failures, and each needs explicit test coverage because they only fire under conditions that don't occur in normal operation. Three to test specifically. Circuit breakers: after N consecutive failures, the breaker opens and short-circuits subsequent requests for a cooldown period to give the downstream time to recover. I test by mocking the downstream to fail N times, then verifying the next call fails fast, without actually hitting the downstream, and returns to closed state after the cooldown. The hard part, and my answer to the follow-up about hardest to test, is the recovery path: after the cooldown, the next request is allowed through, and if it succeeds, the breaker closes normally; if it fails, it stays open. Testing this needs precise timing control and a mock that I can switch from failing to succeeding mid-test. Retries: failed requests retry with exponential backoff. I test by mocking transient failures and asserting the request is retried the expected number of times with the expected delays, first retry after one second, second after two, third after four. The mock counts calls and measures time between them. Timeouts: requests exceeding a deadline are aborted. I test by mocking a slow downstream, a route handler with setTimeout that delays past the caller's timeout, and asserting the caller gives up at the right time, not waiting indefinitely. The discipline that distinguishes a senior implementation, and the framing I would emphasise because it's where most resilience tests go vague, is each resilience pattern is tested in isolation with its expected behaviour explicit. Not the system is resilient but after five consecutive failures, the breaker opens for thirty seconds; during open, requests fail in under a hundred milliseconds with a CircuitOpen error; after thirty seconds, the next request is allowed through. Resilience is specific behaviour; the tests have to be specific too. The pitfall is vague resilience claims without numbers: the system is resilient passes review and ships, then production reveals the breaker takes ten minutes to recover or retries forever or the timeout isn't enforced. Specific numbers in the spec, specific assertions in the tests.*

Key points to hit

(1) Test each pattern in isolation: breaker, retry, timeout, separate scenarios. **(2)** Circuit: N failures -> breaker opens -> fast-fail -> cooldown -> recovery. **(3)** Retries: count calls + measure delays, verify exponential backoff numbers. **(4)** Timeouts: slow downstream + assert caller aborts at the right time. **(5)** Discipline: specific numeric behaviour, not 'the system is resilient'. **(6)** Trap: vague resilience claims, production reveals breaker takes 10 min to recover.

CODE

```
// Circuit breaker test: specific behaviour assertions
test('breaker opens after 5 failures, fast-fails until 30s cooldown', async ({ page })
=> {
let calls = 0;
await page.route('**/api/downstream', route => {
calls++;
return route.fulfill({ status: 503 });
});

// 5 failing calls open the breaker
for (let i = 0; i < 5; i++) {
await page.evaluate(() => fetch('/api/downstream'));
}
expect(calls).toBe(5);

// 6th call fast-fails without hitting downstream
const start = Date.now();
const status = await page.evaluate(async () => (await fetch('/api/downstream')).status);
const elapsed = Date.now() - start;
expect(elapsed).toBeLessThan(100); // fast-fail
expect(status).toBe(503);
expect(calls).toBe(5); // downstream NOT called
});

// Retry test: count calls, measure delays
test('client retries 3 times with exponential backoff', async ({ page }) => {
const callTimes: number[] = [];
await page.route('**/api/data', route => {
callTimes.push(Date.now());
return route.fulfill({ status: 503 });
});
await page.evaluate(() => fetch('/api/data').catch(() => undefined));
expect(callTimes).toHaveLength(4); // 1 original + 3 retries
expect(callTimes[1]! - callTimes[0]!).toBeGreaterThan(900); // ~1s
expect(callTimes[2]! - callTimes[1]!).toBeGreaterThan(1900); // ~2s
});
```

Q 239 LEVEL · Senior

What is chaos engineering applied to test environments, and how do you implement it?

CONCEPT

Chaos engineering verifies the system's resilience claims by **deliberately injecting failures** and observing what happens. Applied to testing, it means running tests against a system where you've induced a known failure, killed a pod, dropped a network connection, introduced latency, and asserting the system behaves correctly under degradation. Tools: Chaos Mesh and **Litmus** for Kubernetes, Toxiproxy for network manipulation at the connection level, Gremlin as a commercial platform. The test pattern: **fixture-injected chaos** that brings up a controlled fault before the test body and removes it after, a fixture that uses the Chaos Mesh API to kill the payment-service pod, then the test verifies the order flow degrades gracefully (returns the right error, doesn't double-charge, queues the retry). The discipline: chaos tests are not for every PR; they're for release-gate verification. Chaos tests are slow, harder to debug, and require specific infrastructure.

Run them on release branches and nightly schedules, not on every commit. The principle: resilience claims need evidence; chaos engineering is how you produce it. Without chaos tests, resilience is a hope; with them, it's a verified property.

INTERVIEW STRATEGY

They want chaos engineering calibrated for test environments. Define it (deliberate failure injection), name tools (Chaos Mesh, Litmus, Toxiproxy, Gremlin), then make the 'release-gate, not every PR' tiering the senior signal. The failure mode is chaos tests on every PR: too slow, too brittle. A common follow-up is ‘what do you actually inject?’, answer pod kills, network latency, packet drops, process pauses, the failures production actually sees.

How to phrase it

> *Chaos engineering verifies the system's resilience claims by deliberately injecting failures and observing what happens. Applied to testing, it means running tests against a system where I've induced a known failure, killed a pod, dropped a network connection, introduced latency, and asserting the system behaves correctly under degradation. Distinct from unit tests of resilience patterns from the previous question, because here the failure is real infrastructure failure, not a mocked response. Tools depend on the platform. Chaos Mesh and Litmus for Kubernetes, where the platform itself can kill pods, partition networks, throttle resources via CRDs. Toxiproxy for network manipulation at the connection level, sitting between services and introducing latency, packet loss, or complete connection drops. Gremlin as a commercial platform that wraps a lot of this with UI and policy controls. The test pattern is fixture-injected chaos: a fixture that brings up a controlled fault before the test body and removes it after. The fixture uses the Chaos Mesh API to kill the payment-service pod, then the test verifies the order flow degrades gracefully, returns the right error, doesn't double-charge, queues the retry for when payment comes back. That's also my answer to the follow-up about what I inject: pod kills, network latency, packet drops, process pauses, the failures production actually sees. Disk full, DNS resolution failures, slow disk I-O for more advanced scenarios. The discipline I would lead with, because this is where chaos engineering goes wrong in test suites, is chaos tests are not for every PR; they're for release-gate verification. Chaos tests are slow because real infrastructure failures take time to materialise and resolve. They're harder to debug because the failure mode is more varied. They require specific infrastructure that not every CI environment has. Run them on release branches and nightly schedules, not on every commit. The principle I would close on, because it captures why this matters at all, is resilience claims need evidence; chaos engineering is how you produce it. Without chaos tests, resilience is a hope: we built circuit breakers and retries and the deploy looks fine. With chaos tests, it's a verified property: the system demonstrably handles pod kills in under five seconds with zero data loss. The thing to avoid is chaos tests on every PR; the suite gets slow, brittle, abandoned, and the resilience claims go unverified anyway.*

Key points to hit

(1) Deliberate failure injection: pod kills, network drops, latency injection. **(2)** Tools: Chaos Mesh, Litmus (K8s); Toxiproxy (network); Gremlin (commercial). **(3)** Fixture-injected chaos: bring up fault, run test, remove fault. **(4)** Tiering: release-gate + nightly only, NOT every PR (too slow/brittle). **(5)** Resilience claims need evidence: chaos engineering produces it. **(6)** Where this goes wrong: chaos tests on every PR, brittle, slow, eventually abandoned.

CODE

```
// Fixture that injects chaos via Chaos Mesh (Kubernetes CRD API)
export const test = base.extend<{ chaos: ChaosFault }>({
chaos: async ({}, use) => {
const fault: ChaosFault = {
kind: 'PodChaos',
target: 'app=payment-service',
action: 'pod-kill',
};
await applyChaosMesh(fault); // CRD applied
await use(fault);
await removeChaosMesh(fault); // teardown
},
});

test('order flow degrades gracefully when payment-service is down', async ({ page,
chaos, request }) => {
// chaos fixture already killed payment-service
await page.goto('/checkout');
await page.getByRole('button', { name: 'Place Order' }).click();

// System degrades gracefully:
await expect(page.getByText('Payment is temporarily unavailable')).toBeVisible();
await expect(page.getByText('We will retry your order')).toBeVisible();

// Verify no double-charge in the order log
const audit = await (await request.get('/api/audit/recent')).json();
expect(audit.filter((e: any) => e.type === 'charge')).toHaveLength(0);
});

// Run on release branches + nightly schedule, NOT every PR
```

Q 240

LEVEL · Senior

How do you tier contract testing in CI: fast consumer-driven vs slow end-to-end verification?

CONCEPT

Contract tests verify that services agree on the shape of their interactions, and they tier across CI by speed and scope. Tier 1: consumer contracts in PR pipelines (seconds): each consumer service declares contracts about the providers it depends on (Pact files), and PR tests verify the consumer's expectations match a stored contract. No real network calls; the tests are unit-test-fast. Tier 2: provider verification on provider PRs (minutes): when the provider changes, its CI verifies against the published consumer contracts to confirm backward compatibility. If the change breaks a consumer's expectations, the provider PR fails. Tier 3: end-to-end verification on integration branches (slower): real services talking to real services in a staging environment, catching the integration bugs contract tests miss, race conditions, latency-dependent behaviour, state-dependent interactions. The discipline: tier-1 contracts gate every PR; tier-3 e2e gates merge to main. Most interaction bugs surface at tier 1; tier 3 is the backstop for the rest. The trap: **only running tier-3 e2e**, which is slow, expensive, and discovers bugs too late in the cycle to fix cheaply. Contract tests catch the same bugs earlier; e2e catches what they miss.

INTERVIEW STRATEGY

What earns the tick is contract testing tiered explicitly. Walk three tiers (consumer contracts PR-fast, provider verification on provider PRs, e2e on integration). The 'tier-1 gates every PR, tier-3 gates merge' rule is the senior signal. The thing to avoid is only running tier-3 e2e. The follow-up is often ‘what's Pact?’, answer consumer-driven contract testing framework: consumers publish expectations, providers verify against them.

How to phrase it

> *Contract tests verify that services agree on the shape of their interactions, and they tier across CI by speed and scope. Three tiers worth knowing because they map to different gates. Tier one: consumer contracts in PR pipelines, running in seconds. Each consumer service declares contracts about the providers it depends on, typically as Pact files or similar consumer-driven contracts, and PR tests verify the consumer's expectations match a stored contract. No real network calls; the tests are unit-test-fast. The auth service's consumer of the user service tests against a Pact file saying ‘given a user with email X, GET slash users slash X returns 200 with body shape Y’. The contract is the test fixture. Tier two: provider verification on provider PRs, taking minutes. When the provider changes, its CI verifies against the published consumer contracts to confirm backward compatibility. If the change breaks a consumer's expectations, say, removes a field the auth service relies on, the provider PR fails before merge. This is what makes the consumer-provider pact actually enforce: providers can't merge breaking changes without the consumers explicitly accepting them. Tier three: end-to-end verification on integration branches, slower again. Real services talking to real services in a staging environment, catching the integration bugs contract tests miss, race conditions, latency-dependent behaviour, state-dependent interactions that the contract test's static fixture can't simulate. The discipline I would lead with, because it's the tiering rule that makes the system work, is tier-one contracts gate every PR; tier-three e2e gates merge to main. Most interaction bugs surface at tier one because contracts are explicit about field shapes and required values; tier three is the backstop for the rest. That's also my answer to the follow-up about Pact: consumer-driven contract testing framework where consumers publish expectations as machine-readable contracts and providers verify against them, shared via a Pact Broker or similar registry so both sides see the same contract. The classic error is only running tier-three e2e, which is slow, expensive, and discovers bugs too late in the cycle to fix cheaply. A PR that breaks a contract is fixed in minutes if tier one catches it; the same break caught at e2e an hour later means the dev has context-switched, the fix takes longer, and the cost compounds. Contract tests catch the same bugs earlier; e2e catches what they miss.*

Key points to hit

(1) Tier 1: consumer contracts in PR pipelines (seconds), unit-test-fast. **(2)** Tier 2: provider verification on provider PRs (minutes), backward compat. **(3)** Tier 3: real-service e2e on integration branches, race/state-dependent bugs. **(4)** Rule: tier-1 gates every PR; tier-3 gates merge to main. **(5)** The next question: Pact = consumer-driven contracts shared via broker. **(6)** What loses the point: only tier-3 e2e, slow, expensive, bugs found too late to fix cheaply.

CODE

```
# CI tiering for contract tests

# Tier 1: consumer PR pipeline (seconds)
on: pull_request
jobs:
consumer-contracts:
runs-on: ubuntu-latest
steps:
- run: npm test -- --grep '@contract' # Pact-based unit tests

# Tier 2: provider PR (minutes), verify against published consumer contracts
on: pull_request # in the provider's repo
jobs:
provider-verification:
runs-on: ubuntu-latest
steps:
- run: |
pact-broker can-i-deploy --pacticipant=auth-service \\
--version=$GITHUB_SHA --to-environment=production

# Tier 3: e2e on integration branches (slower), real services
on:
push:
branches: [main, release/*]
jobs:
end-to-end:
runs-on: ubuntu-latest
steps:
- run: npx playwright test --project=e2e-staging
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Which microservices test level catches a breaking change first?

A) E2E tests through the UI
B) Manual exploratory testing
C) Per-service contract tests: before integration tests even run
D) Load tests

2. Why are API-gateway tests best thought of as security tests?

A) They verify auth enforcement, rate limiting and CORS, the system's perimeter
B) They run faster than other tests
C) They only check routing
D) They replace contract tests

3. What pattern tests an event-driven flow through a message queue?

A) A fixed sleep then a single check
B) Trigger the action, then expect.poll for the async downstream effect
C) Assert immediately after the trigger
D) Mock the entire queue

4. What does Pact provide in a microservices environment?

A) Load testing
B) A message queue
C) Browser automation
D) Consumer-driven contract testing across services

5. How should contract tests validate a response?

A) Assert exact values for every field
B) Compare against a screenshot
C) Only check the status code
D) Validate the contract, types, formats, allowed values, not the implementation

Section 19 · Answer key

#	Answer	Why and where to revisit
1	C) Per-service contract tests: before integration tests even run	Fast, isolated contract tests fail the instant a service's API shape changes, at the cheapest stage. *Revisit Section 19.*
2	A) They verify auth enforcement, rate limiting and CORS, the system's perimeter	The gateway is the single enforcement point; a 401/429/CORS test confirms the perimeter actually holds. *Revisit Section 19.*
3	B) Trigger the action, then expect.poll for the async downstream effect	The trigger is sync but the effect is async; polling with backoff validates the whole publish-consume-persist chain. *Revisit Section 19.*
4	D) Consumer-driven contract testing across services	Pact lets each consumer declare what it needs and verifies providers against all consumer expectations. *Revisit Section 19.*
5	D) Validate the contract, types, formats, allowed values, not the implementation	Contract tests check shape (id is a number, role is one of the allowed values), staying resilient to data changes. *Revisit Section 19.*

SECTION 20

Security and Performance Testing

Security and performance questions check whether you can apply Playwright beyond functional flows, and, just as importantly, whether you know its limits. These nine questions cover testing common vulnerabilities with the standard API, capturing Web Vitals as performance-regression guards (not load tests), and wiring OWASP ZAP in as a proxy for automated security scanning.

In this section

- Security tests with a security lens: XSS, auth bypass, sensitive-data exposure.
- The password-not-in-response test that catches real data leaks.
- Web Vitals/Navigation Timing as performance-regression tests, not load tests.
- Knowing the boundary: k6/JMeter for load, Playwright for single-load budgets.
- OWASP ZAP as a proxy: zero test-code change, scans your test surface.

Q 241 LEVEL · Mid

How do you implement security testing with Playwright?

CONCEPT

Playwright can test common vulnerabilities with the **standard API**, they are functional tests with a security lens. XSS: submit a script payload, then assert the script did not execute and the payload is displayed as escaped text, not live HTML. **Authentication bypass**: clear cookies and localStorage, hit a protected route, and assert a redirect to login. Sensitive-data exposure: assert that a user API response does **not** contain password or passwordHash properties. These run in the regression suite on every release candidate. The data-exposure test in particular catches real bugs, for example a debug endpoint that accidentally returned the full user object including the hashed password, before it reaches production, which is exactly the kind of leak that is invisible to a purely functional test.

INTERVIEW STRATEGY

You are being asked to show you apply a security lens with the tools you already have. Frame these as functional tests with a security angle, give the three concrete checks, XSS, auth bypass, sensitive-data exposure, and land the password-not-in-response war story, which proves it catches real leaks. The failure mode is implying Playwright replaces a dedicated pen test. The follow-up is often 'what real bug has this caught?', answer the debug endpoint leaking the hashed password, caught before production.

How to phrase it

> *Playwright can test common vulnerabilities with the standard API, so I frame security tests as functional tests with a security lens. For XSS prevention I submit a script payload into a field, then assert two things: that the script did not execute, and that the payload is displayed as escaped text rather than live HTML, so I am verifying the app sanitises input. For authentication bypass I clear the cookies and localStorage, hit a protected route directly, and assert it redirects to login, which confirms the route is actually guarded and not just hidden in the UI. For sensitive-data exposure I assert that a user API response does not contain a password or passwordHash property. That last one is also my answer to the follow-up about a real bug it has caught: a developer added a debug endpoint that returned the full user object including the hashed password, and the test caught it before it reached production, a leak that is completely invisible to a functional test because the screen looked fine. I run these in the regression suite on every release candidate. The honest boundary I would state, to avoid the trap of overclaiming, is that this is not a substitute for a dedicated penetration test or a full security audit, it is a fast, automated safety net for the well-understood vulnerability classes, so the obvious regressions get caught continuously rather than once a year.*

Key points to hit

(1) Functional tests with a security lens, using the standard API. **(2)** XSS: payload not executed and shown as escaped text. **(3)** Auth bypass: clear cookies/storage, hit protected route, expect redirect. **(4)** Sensitive data: assert no password/passwordHash in API responses. **(5)** Asked next: caught a debug endpoint leaking the hashed password pre-prod. **(6)** The easy mistake: implying it replaces a dedicated pen test or security audit.

CODE

```
test('user input is sanitised: XSS prevented', async ({ page }) => {
const payload = '<script>window.__XSS__ = true;</script>';
await page.goto('/profile/edit');
await page.getByLabel('Display Name').fill(payload);
await page.getByRole('button', { name: 'Save' }).click();
await page.goto('/profile');
expect(await page.evaluate(() => (window as any).__XSS__)).toBeUndefined();
expect(await page.locator('#display-name').textContent()).toContain('<script>');
});

test('password not returned in user API response', async ({ request }) => {
const user = await (await request.get('/api/users/1')).json();
expect(user).not.toHaveProperty('password');
expect(user).not.toHaveProperty('passwordHash');
});
```

Q 242

LEVEL · Mid

How do you measure performance metrics with Playwright?

CONCEPT

Playwright can capture **Core Web Vitals** and the Navigation Timing API from the page, LCP, TTFB, DOMContentLoaded, load complete, and assert them against a **performance budget**. The crucial framing: these are performance-regression tests, not load tests. You are not testing behaviour under a thousand concurrent users, that is k6 or JMeter territory, you are testing that a single page load meets the budget. When a developer adds a large unoptimised image or a blocking script, the test fails and they know immediately. Set thresholds from your SLA (for example TTFB under 500ms, DOMContentLoaded under 2s) and track them over time to catch **gradual degradation**, the kind that never trips a single run but slowly erodes the experience release over release.

INTERVIEW STRATEGY

What scores here is showing you know exactly what kind of performance testing this is. State the boundary first and loudly: regression-not-load, with k6/JMeter owning load. Then describe capturing Web Vitals and asserting an SLA-derived budget, and the value of trending to catch gradual degradation. The mistake is conflating this with load testing. Expect the follow-up ‘how would you load test then?’, answer that is k6 or JMeter, a different tool for a different question.

How to phrase it

> *Playwright can capture Core Web Vitals and the Navigation Timing API from the page, LCP, TTFB, DOMContentLoaded, load complete, and I assert them against a performance budget. The first thing I always clarify, because it is exactly what the question is probing, is that these are performance-regression tests, not load tests. I am not testing how the application behaves under a thousand concurrent users; that is k6 or JMeter territory, and that is also my answer to the follow-up about how I would actually load test, a different tool for a different question. What I am testing is that a single page load meets our budget. So when a developer adds a large unoptimised image or a render-blocking script, the performance test fails and they find out immediately rather than from a user complaint weeks later. I set the thresholds from our SLA, TTFB under five hundred milliseconds, DOMContentLoaded under two seconds, load complete under five, and I track them over time, which is the part that catches gradual degradation: any single release might be a hair slower and pass, but the trend reveals the slow erosion across releases that no individual run would flag. The classic error is conflating this with load testing and claiming Playwright does something it does not. Being precise, Playwright for single-load performance budgets, k6 or JMeter for load, is what shows I understand where each tool belongs.*

Key points to hit

(1) Capture Web Vitals / Navigation Timing (LCP, TTFB, DOMContentLoaded). **(2)** Framing: performance-regression tests, NOT load tests. **(3)** Load testing is k6/JMeter territory: a different question. **(4)** Set thresholds from the SLA; fail on a regression like a heavy image. **(5)** Follow-up: load test with k6/JMeter; track trends for gradual degradation. **(6)** Trap: conflating single-load budgets with load testing.

CODE

```
test('dashboard meets the performance budget', async ({ page }) => {
await page.goto('/dashboard');
const m = await page.evaluate(() => {
const nav = performance.getEntriesByType('navigation')[0] as
PerformanceNavigationTiming;
return {
domContentLoaded: nav.domContentLoadedEventEnd - nav.startTime,
ttfb: nav.responseStart - nav.requestStart,
loadComplete: nav.loadEventEnd - nav.startTime,
};
});
expect(m.ttfb).toBeLessThan(500); // SLA-derived budget
expect(m.domContentLoaded).toBeLessThan(2000);
expect(m.loadComplete).toBeLessThan(5000);
});
```

Q 243 LEVEL · Mid to Senior

How do you integrate OWASP ZAP with Playwright for security scanning?

CONCEPT

OWASP ZAP is configured as a **proxy** for Playwright: you point the config's proxy at ZAP, and as the tests run normally, ZAP **silently records every endpoint and parameter** they exercise. After the run, you trigger a ZAP active scan via its API against the recorded in-scope URLs, and it tests each for SQL injection, XSS, missing security headers, insecure cookie flags, and more. The elegance is that it requires **zero changes to test code**, the functional tests double as a crawler that maps the application's attack surface. The consequence worth stating: security coverage becomes proportional to functional coverage, the more tests you have, the more endpoints ZAP scans, so investment in functional tests pays a second dividend in security reach.

INTERVIEW STRATEGY

This question is there to see you integrate a real security tool, not just write assertions. Lead with the zero-test-code-change elegance: ZAP proxies the run and records the attack surface, then active-scans it. The insight that lands is that security coverage scales with functional coverage. The failure mode is describing ZAP as a separate manual effort disconnected from the suite. They usually follow with 'how do you get broad endpoint coverage?', answer that the functional tests are the crawler, so more tests means more scanned endpoints.

How to phrase it

> *I integrate OWASP ZAP by configuring it as a proxy for Playwright, I point the config's proxy setting at the ZAP server, and then my tests run completely normally while ZAP silently records every endpoint and parameter they touch. After the run, I trigger a ZAP active scan via its API against the recorded in-scope URLs, and it tests each one for SQL injection, XSS, missing security headers, insecure cookie flags and the rest. The reason I like this approach, and what I would emphasise, is that it requires zero changes to the test code, the functional tests effectively double as a crawler that maps the application's attack surface, every URL, every form field, every API parameter the tests exercise. That leads directly to the follow-up about how I get broad endpoint coverage: I do not write security-specific navigation, the existing functional tests are the crawler, so the more functional tests we have, the more endpoints ZAP scans. Security coverage becomes proportional to functional coverage, which means the investment we already make in functional tests pays a second dividend in security reach. Where this goes wrong is treating ZAP as a separate, manual scanning effort disconnected from the suite, which is slower and always lags the app; wiring it in as a proxy means the security scan rides along with every test run and grows automatically as the suite grows. I run the active scan on release candidates and review ZAP's findings as part of the release security check.*

Key points to hit

(1) Configure ZAP as Playwright's proxy; tests run normally. **(2)** ZAP silently records every endpoint/parameter, then active-scans them. **(3)** Scans for SQLi, XSS, missing headers, insecure cookie flags. **(4)** Zero test-code change: functional tests map the attack surface. **(5)** Expect next: coverage scales, more functional tests, more scanned endpoints. **(6)** Where this goes wrong: treating ZAP as a separate manual effort detached from the suite.

CODE

```
// playwright.config.ts: route traffic through ZAP
export default defineConfig({
use: { proxy: { server: 'http://localhost:8080' } }, // ZAP proxy
});

// Tests run normally; ZAP records every endpoint + parameter.
// After the run, trigger an active scan via the ZAP API:
// curl -X POST "http://localhost:8080/JSON/ascan/action/scan/" \
// -d "url=${BASE_URL}&recurse=true&inScopeOnly=true"
//
// ZAP reports: SQL injection, XSS, missing security headers,
// insecure cookie flags: across everything the tests exercised.
```

Q 244 LEVEL · Senior

How do you systematically test authorisation boundaries across user roles?

CONCEPT

Authorisation bugs are the wrong user could perform an action they shouldn't, the viewer sees admin data, a tenant-A user accesses tenant-B records, a logged-out user reaches an authenticated endpoint. The systematic approach: matrix of roles × actions × expected outcomes. For each protected action, test as every role that shouldn't be allowed (expect 403 or 401) and as every role that should (expect 200). The pattern in Playwright: parameterised tests across the role matrix using storageState files per role, admin storageState, user storageState, viewer storageState, anonymous (no storageState), with a data-driven loop asserting each role gets the right response for each endpoint. The discipline that catches the bugs others miss: test the negative path explicitly, not just the positive. A test asserting admin can delete passes when everyone can delete. The test that catches the bug is asserting viewer can NOT delete. Cover the matrix completely; the holes are where vulnerabilities live.

INTERVIEW STRATEGY

You need to give systematic authorisation testing, not ad-hoc. Lead with the roles×actions matrix concept, then parameterised tests with per-role storageState. The 'test negative path explicitly' rule demonstrates calibration because positive-only tests miss the bug. The pitfall is testing only happy admin paths. The next question is normally ‘what about row-level / tenant-level authorisation?’, answer same matrix pattern: tenant-A user trying tenant-B records, expect 403.

How to phrase it

> *Authorisation bugs are the wrong user could perform an action they shouldn't, the viewer sees admin data, a tenant-A user accesses tenant-B records, a logged-out user reaches an authenticated endpoint. These are typically high-severity bugs because they're not just functional failures, they're data leaks or privilege escalation. The systematic approach, and where most teams stop short, is a matrix of roles times actions times expected outcomes. For each protected action, I test as every role that shouldn't be allowed, expect 403 or 401, and as every role that should, expect 200. Anonymous user, viewer, regular user, admin, super-admin: each one runs every protected endpoint, and the test verifies the right outcome for that combination. The pattern in Playwright is parameterised tests across the role matrix using storageState files per role. Admin storageState, user storageState, viewer storageState, anonymous with no storageState, each as a project in playwright.config.ts or as a parameterised loop. A data-driven loop asserts each role gets the right response for each endpoint. The discipline that catches the bugs others miss, and the framing I would lead with because positive-only testing is everywhere, is test the negative path explicitly, not just the positive. A test asserting admin can delete passes when everyone can delete, including viewers. The test that catches the bug is asserting viewer cannot delete, the negative case where the expected response is 403, not 200. Without the negative case, a broken authorisation check that allows everything still passes the suite. That's also my answer to the follow-up about row-level or tenant-level authorisation: same matrix pattern but with two users from different tenants. Tenant-A user trying to access tenant-B records, expect 403. Cover the matrix completely; the holes are where vulnerabilities live. The risk is testing only happy admin paths because they're easiest to write and pass first, the suite looks comprehensive, real privilege boundaries go untested, the bug ships.*

Key points to hit

(1) Authorisation bugs: wrong user could perform an action they shouldn't. **(2)** Matrix approach: roles × actions × expected outcomes, cover all cells. **(3)** Per-role storageState files: admin, user, viewer, anonymous. **(4)** Parameterised loop runs every endpoint as every role with expected outcome. **(5)** Test negative path explicitly: viewer canNOT delete, not just admin can. **(6)** What loses the point: positive-only testing, broken auth that allows everything still passes.

CODE

```
// Parameterised authorisation matrix
const matrix = [
{ role: 'anonymous', file: undefined, delete: 401, view: 401 },
{ role: 'viewer', file: 'auth/viewer.json', delete: 403, view: 200 },
{ role: 'user', file: 'auth/user.json', delete: 403, view: 200 },
{ role: 'admin', file: 'auth/admin.json', delete: 204, view: 200 },
];

for (const { role, file, delete: del, view } of matrix) {
test.describe(`role: ${role}`, () => {
test.use({ storageState: file });
test(`DELETE /api/users/X returns ${del}`, async ({ request }) => {
const res = await request.delete('/api/users/123');
expect(res.status()).toBe(del); // covers BOTH positive + negative
});
test(`GET /api/users returns ${view}`, async ({ request }) => {
const res = await request.get('/api/users');
expect(res.status()).toBe(view);
});
});
}

// Tenant-boundary variant: tenant-A user accessing tenant-B resources -> 403
```

Q 245 LEVEL · Mid to Senior

How do you test for XSS and injection vulnerabilities with input fuzzing?

CONCEPT

XSS and injection vulnerabilities live in any input that flows through to rendered output or executed query. Manual testing with a few payloads misses most cases; systematic fuzzing with a catalogue of known payloads catches more. The pattern: a curated list of payloads (XSS strings from OWASP XSS cheat sheet, SQL injection from sqlmap's payloads, command injection patterns, NoSQL injection) plus a test that pumps each through every user input and asserts the rendered output is properly escaped. For XSS: the payload appears as text on the page, not as executed script (**page.locator('script').count() shouldn't increase**). For SQL injection: the request returns a normal error or 400, not a 500 with a database stack trace, and not data the user shouldn't see. The discipline that scales: generate input-fuzzing tests from the API schema. Every string field becomes a fuzzing target; every fuzzing payload runs against every field. The trap: writing one fuzz test against one endpoint and feeling covered, vulnerabilities live in the endpoints you didn't think to test. Comprehensive fuzzing is the right strategy; dedicated SAST/DAST tools (Burp, ZAP) complement it for what manual fuzzing misses.

INTERVIEW STRATEGY

They are after input fuzzing as a systematic security practice. Lead with the payload catalogue + test-every-input pattern, then assertion targets (text not script for XSS, normal error not stack trace for SQL injection). The 'generate from schema, every field becomes a target' discipline is what separates junior from senior thinking. The trap is one-fuzz-per-endpoint. A common follow-up is ‘where do payloads come from?’, answer OWASP XSS cheat sheet, sqlmap's payloads,

command/NoSQL injection lists.

How to phrase it

> *XSS and injection vulnerabilities live in any input that flows through to rendered output or executed query. The classic failure mode: a user submits a string containing script tags, the application stores it, another user views the record, and the script executes in their browser, stored XSS, the high-impact variant. Manual testing with a few payloads misses most cases because there are dozens of encoding variations that all bypass naive escaping. Systematic fuzzing with a catalogue of known payloads catches more. The pattern is a curated list of payloads plus a test that pumps each through every user input and asserts the rendered output is properly escaped. That's also my answer to the follow-up about where payloads come from: the OWASP XSS cheat sheet for HTML and JavaScript injection variants, sqlmap's payload list for SQL injection, command-injection patterns for shell escape attempts, NoSQL injection patterns for MongoDB-style attacks. Curated lists maintained by security researchers; far more comprehensive than anything I'd think of manually. The assertion targets differ by vulnerability. For XSS, the payload appears as text on the page, not as executed script. I can assert that the rendered output equals the literal payload string, and that page dot locator script dot count hasn't increased, the payload didn't create new script tags. For SQL injection, the request returns a normal error or 400, not a 500 with a database stack trace, and not data the user shouldn't see. The discipline that scales, because hand-writing fuzz tests for every endpoint doesn't, is generate input-fuzzing tests from the API schema. Every string field becomes a fuzzing target; every fuzzing payload runs against every field. The OpenAPI spec becomes the source of truth for what to fuzz. The risk is writing one fuzz test against one endpoint and feeling covered. Vulnerabilities live in the endpoints you didn't think to test, the admin-only field that should have been sanitised but wasn't, the legacy endpoint nobody remembered, the new search filter that bypassed the input sanitiser. Comprehensive fuzzing is the right strategy; dedicated SAST and DAST tools like Burp and ZAP complement it for what manual fuzzing misses, particularly authenticated scanning of complex flows.*

Key points to hit

(1) XSS/injection live in any input flowing to rendered output or executed query. **(2)** Payload catalogue: OWASP XSS cheat sheet, sqlmap, command/NoSQL patterns. **(3)** XSS assertion: payload as text + no new script tags created. **(4)** SQL injection assertion: normal error/400, not 500 with stack trace. **(5)** Generate fuzzing from OpenAPI schema: every string field is a target. **(6)** The common miss: one fuzz test per endpoint, vulnerabilities live where you didn't test.

CODE

```
// Curated payload catalogue (subset of OWASP cheat sheet)
const xssPayloads = [
'<script>alert(1)</script>',
'"><script>alert(1)</script>',
'<img src=x onerror=alert(1)>',
'javascript:alert(1)',
'<svg/onload=alert(1)>',
'<iframe src="javascript:alert(1)">',
'<body onload=alert(1)>',
];

for (const payload of xssPayloads) {
test(`comment field escapes XSS: ${payload.slice(0, 30)}`, async ({ page }) => {
await page.goto('/posts/1');
await page.getByLabel('Comment').fill(payload);
await page.getByRole('button', { name: 'Post' }).click();

// 1. Payload appears as text, not executed
await expect(page.getByTestId('comment-body').last()).toHaveText(payload);
// 2. No new <script> tags created
expect(await page.locator('script').count()).toBeLessThan(20); // baseline
});
}

// SQL injection variant
const sqlPayloads = ["' OR '1'='1", "1; DROP TABLE users--", "' UNION SELECT password
FROM users--"];
// For each: request returns 400 / normal error, never 500 with DB error
```

Q 246 LEVEL · Mid

How do you verify security headers and cookie flags are correctly set?

CONCEPT

Security headers and cookie flags are deployment-config concerns that the application code might set correctly in dev but lose in production due to proxy stripping, framework defaults, or environment-specific config drift. The pattern: a test suite that fetches the application's key endpoints and asserts each security header matches the policy. Headers to verify: **Content-Security-Policy** (specific directives, not just present), Strict-Transport-Security (max-age, includeSubDomains), X-Frame-Options or CSP frame-ancestors, X-Content-Type-Options nosniff, Referrer-Policy. Cookie flags: **HttpOnly** (JavaScript can't read), Secure (only over HTTPS), **SameSite=Lax or Strict** (CSRF protection). The discipline: run this suite against every environment, dev, staging, production smoke, because environment-specific config drift is exactly what these tests catch. A header set correctly in staging but missing in production is a real failure mode.

INTERVIEW STRATEGY

They want security header verification systematic, not occasional. Lead with the deployment-config-drift framing, then the specific headers and cookie flags. The 'run against every environment' discipline catches the prod-specific failures. The pitfall is asserting header is present without checking value. The follow-up is often ‘what's the biggest gotcha?’, answer CSP that's present but permissive enough to be useless.

How to phrase it

> *Security headers and cookie flags are deployment-config concerns that the application code might set correctly in dev but lose in production due to proxy stripping, framework defaults, or environment-specific config drift. The important framing is that these are not code bugs in the application; they're configuration bugs in the deployment, which means they pass local testing and fail in environments specifically. The pattern is a test suite that fetches the application's key endpoints and asserts each security header matches the policy. Headers to verify, in order of impact. Content-Security-Policy with specific directives, not just present, a CSP that allows everything is the biggest gotcha and my answer to the follow-up. Strict-Transport-Security with max-age and includeSubDomains, so browsers refuse HTTP downgrades. X-Frame-Options or CSP frame-ancestors to prevent clickjacking. X-Content-Type-Options nosniff to prevent MIME-type confusion. Referrer-Policy to control what referrers leak. Cookie flags: HttpOnly so JavaScript can't read the cookie via document dot cookie, Secure so cookies only travel over HTTPS, SameSite=Lax or Strict for CSRF protection. The trap I would emphasise, because asserting presence without value is the common mistake, is asserting the header is present without checking it's actually restrictive. A CSP header that says default-src star is technically present but provides no protection, same as not having one. The assertions need to verify specific directives that match the security policy: script-src self only, no unsafe-inline, object-src none. The discipline I would close on, because it's what makes these tests valuable, is run this suite against every environment, dev, staging, production smoke. Environment-specific config drift is exactly what these tests catch. A header set correctly in staging but missing in production because a load balancer is stripping it is a real failure mode and one that only this kind of test finds. Run it as a separate environment-smoke job in CI, not bundled with regular regression.*

Key points to hit

(1) Security headers are deployment-config concerns; pass dev, fail in prod. **(2)** Headers to verify: CSP (specific directives!), HSTS, X-Frame-Options, nosniff, Referrer-Policy. **(3)** Cookies: HttpOnly, Secure, SameSite=Lax or Strict. **(4)** Run against every environment: catches LB/proxy stripping drift. **(5)** They usually ask next: permissive CSP (default-src *) is technically present, provides no protection. **(6)** Avoid: assert header present without verifying restrictive value.

CODE

```
test('production security headers and cookies', async ({ request }) => {
const res = await request.get(`${process.env.BASE_URL}/`);
const headers = res.headers();

// CSP: specific directives, not just present
const csp = headers['content-security-policy'] ?? '';
expect(csp).toContain("default-src 'self'");
expect(csp).not.toContain('unsafe-inline'); // catches permissive CSP
expect(csp).not.toContain('*');

// HSTS
expect(headers['strict-transport-security']).toMatch(/max-age=\d{7,}.*includeSubDomains/
);

// Anti-clickjacking + MIME sniff
expect(headers['x-frame-options']).toMatch(/DENY|SAMEORIGIN/);
expect(headers['x-content-type-options']).toBe('nosniff');

// Cookies (auth flow)
const setCookie = res.headersArray().find(h => h.name.toLowerCase() ===
'set-cookie')?.value ?? '';
expect(setCookie).toMatch(/HttpOnly/i);
expect(setCookie).toMatch(/Secure/i);
expect(setCookie).toMatch(/SameSite=(Lax|Strict)/i);
});

// Run as 'security-headers' project against dev/staging/production smoke endpoints
```

Q 247 LEVEL · Mid to Senior

How do you measure Core Web Vitals (LCP, INP, CLS) from Playwright tests?

CONCEPT

Core Web Vitals are Google's user-experience metrics that affect search ranking and real user satisfaction. Three primary metrics worth measuring. LCP (Largest Contentful Paint): time to the largest visible element rendered, target under 2.5 seconds. INP (Interaction to Next Paint): latency from a user interaction to the next paint, target under 200 ms. Replaced FID in 2024. CLS (Cumulative Layout Shift): visual stability score, target **under 0.1**. Measure via the **PerformanceObserver API** inside **page.evaluate**, capturing the metrics during the test and asserting against thresholds. The discipline that distinguishes useful measurement from misleading: measure on a representative environment, not your dev machine. Web Vitals on a fast wired connection in dev say nothing about a user on a mid-range Android over 4G. Combine with network throttling (CDP slow 3G) and CPU throttling for realistic measurements. The complementary tool: Lighthouse produces a fuller report with optimisation advice, runnable via lighthouse-ci, but PerformanceObserver from Playwright is lighter-weight for per-test threshold enforcement.

INTERVIEW STRATEGY

What earns the tick is Web Vitals as a Playwright-measurable concern. Define the three metrics with target numbers (LCP <2.5s, INP <200ms, CLS <0.1), then PerformanceObserver pattern. The

'measure on representative environment, not dev machine' discipline is the test of seniority on this. The thing to avoid is asserting Web Vitals on a wired-connection dev box. Expect the follow-up 'how is this different from Lighthouse?', answer PerformanceObserver is per-test threshold; Lighthouse is a fuller report for optimisation advice.

How to phrase it

> *Core Web Vitals are Google's user-experience metrics that affect search ranking and real user satisfaction. Three primary metrics worth knowing because they're what gets measured and graded. LCP, Largest Contentful Paint: time to the largest visible element rendered, target under two and a half seconds. INP, Interaction to Next Paint: latency from a user interaction to the next paint, target under two hundred milliseconds. INP replaced FID in twenty-twenty-four because INP captures the full interaction rather than just first input. CLS, Cumulative Layout Shift: visual stability score measuring how much elements move around after initial render, target under zero point one. Layout shift is the thing that makes users tap the wrong button because the page jumped just as they touched the screen. I measure these via the PerformanceObserver API inside page dot evaluate, capturing the metrics during the test and asserting against thresholds. PerformanceObserver fires browser events as each metric is calculated; I collect them, return the values to the test, and assert. The discipline that distinguishes useful measurement from misleading numbers, and the framing I would emphasise, is measure on a representative environment, not on the dev machine. Web Vitals on a fast wired connection in dev say nothing about a user on a mid-range Android over 4G. The LCP is two seconds in dev but six seconds for the actual user; the test passes, the user experiences the slow site. Combine with network throttling, the CDP slow-3G profile from earlier in this section, and CPU throttling for realistic measurements. The CDP Emulation.setCPUThrottlingRate slows the JavaScript engine to mimic a slower device. That's also my answer to the follow-up about how this differs from Lighthouse. PerformanceObserver from Playwright is lighter-weight for per-test threshold enforcement, run on every PR, fail when LCP exceeds budget. Lighthouse, runnable via lighthouse-ci, produces a fuller report with optimisation advice and aggregates across many runs, better suited for periodic deep audits than per-test gating. Use both: PerformanceObserver for budgets in CI, Lighthouse for the occasional comprehensive check. The trap is asserting Web Vitals on a wired-connection dev box; the numbers are meaningless for real users.*

Key points to hit

(1) LCP <2.5s, INP <200ms (replaced FID in 2024), CLS <0.1, Google's targets. **(2)** Measure via PerformanceObserver API in page.evaluate. **(3)** Throttle network (slow 3G) and CPU for representative environment. **(4)** Dev-machine numbers are misleading; real users have slower devices/networks. **(5)** The next question: PerformanceObserver = per-test threshold; Lighthouse = fuller report. **(6)** The easy mistake: Web Vitals on wired dev box, numbers don't reflect real users.

CODE

```
test('home page meets Web Vitals on slow 3G', async ({ page, context }) => {
// Realistic environment
const cdp = await context.newCDPSession(page);
await cdp.send('Network.emulateNetworkConditions', {
offline: false, latency: 400, downloadThroughput: 500 * 1024 / 8, uploadThroughput: 500
* 1024 / 8,
});
await cdp.send('Emulation.setCPUThrottlingRate', { rate: 4 }); // 4x slower CPU

await page.goto('/');

const vitals = await page.evaluate(() => new Promise<{ lcp: number; cls: number
}>(resolve => {
let lcp = 0, cls = 0;
new PerformanceObserver(list => {
for (const entry of list.getEntries()) {
if (entry.entryType === 'largest-contentful-paint') lcp = entry.startTime;
if (entry.entryType === 'layout-shift' && !(entry as any).hadRecentInput) cls += (entry
as any).value;
}
}).observe({ type: 'largest-contentful-paint', buffered: true });
setTimeout(() => resolve({ lcp, cls }), 5000);
}));

expect(vitals.lcp).toBeLessThan(2500); // LCP target
expect(vitals.cls).toBeLessThan(0.1); // CLS target
// INP requires user interaction; capture during a click test
});
```

Q 248

LEVEL · Senior

How do you enforce performance budgets in CI without producing constant flake?

CONCEPT

Performance budgets are fixed thresholds that fail the build if exceeded, LCP under 2.5s, bundle size under 250 KB, total page weight under 1 MB. They prevent gradual regression, but they flake unless designed carefully. Three disciplines that make them reliable. **Soft upper bounds, not happy-path numbers**: covered for SLA in S7 Q10 but applies here too, if LCP is typically 1.5s, set the budget at 2.5s, not 1.7s. Absorb CI variance, fail when regression is real. **Median across N runs, not single-run**: run the measurement three to five times within the same test job and assert on the median or p95, not any single value. A single bad run on a noisy CI runner shouldn't fail the build. Trend tracking beats absolute thresholds for catching real regression: a metric drifting from 1.5s to 1.9s passes a 2.5s budget but indicates regression. Track the metric over time in the observability system from S12 Q8, alert on drift even when within budget. The principle: budgets prevent acute regression, trends catch chronic drift. Both matter. The trap: tight budgets without variance absorption produce a flaky suite the team stops trusting.

INTERVIEW STRATEGY

You need to give performance budgets enforced without flake. Walk three disciplines: soft upper bounds, median across N runs, trend tracking for chronic drift. The 'budgets prevent acute, trends

catch chronic' rule marks senior judgement because it shows both are needed. The trap is tight budgets without variance absorption. They usually follow with ‘what do you budget?’, answer LCP, bundle size, total page weight, JS execution time.

How to phrase it

> *Performance budgets are fixed thresholds that fail the build if exceeded, LCP under two and a half seconds, JavaScript bundle size under two hundred fifty kilobytes, total page weight under one megabyte. That's also my answer to the follow-up about what I budget: LCP, bundle size, total page weight, JavaScript execution time, sometimes specific metrics like time to interactive. Budgets prevent gradual regression, the feature that adds two seconds to LCP fails CI before merge, but they flake unless designed carefully because performance measurements are noisy. Three disciplines that make them reliable. First, soft upper bounds, not happy-path numbers. This is the same principle from API SLA assertions: if LCP is typically one and a half seconds, I set the budget at two and a half, not one point seven. Absorb CI variance, fail when regression is real, not when the runner happens to be slow. Tight budgets without variance absorption produce a flaky suite the team stops trusting. Second, median across N runs, not single-run. I run the measurement three to five times within the same test job and assert on the median or p95, not any single value. A single bad run on a noisy CI runner shouldn't fail the build, but a median that exceeds the budget is real. Three runs is usually enough; five is better for noisy metrics. Third, and this is the framing that lands as the senior insight, trend tracking beats absolute thresholds for catching real regression. A metric drifting from one and a half seconds to one point nine over a month passes a two and a half second budget but indicates regression. The trend is the signal; the absolute threshold isn't sensitive enough to catch it. Track the metric over time in the observability system from earlier in the kit, alert on drift even when within budget. The principle I would close on, because it captures the calibration, is budgets prevent acute regression, trends catch chronic drift. Both matter. Budgets are the gate; trends are the monitoring. Without budgets a single bad commit ships a fifty-percent regression. Without trends a slow ten-percent-per-month creep is invisible until it's a real problem. The classic error is tight budgets without variance absorption, which produces flaky tests and a team that starts disabling the budget checks to unblock CI.*

Key points to hit

(1) Budgets: fixed thresholds that fail CI if exceeded, prevent regression. **(2)** Soft upper bounds: budget for the variance, not the happy path. **(3)** Median across N runs (3-5): single bad run shouldn't fail the build. **(4)** Trend tracking catches chronic drift that absolute thresholds miss. **(5)** Both needed: budgets for acute regression, trends for chronic. **(6)** Trap: tight budgets without variance absorption, flake -> team disables them.

CODE

```
test('LCP performance budget', async ({ page }) => {
const samples: number[] = [];
// Median across 5 runs absorbs CI variance
for (let i = 0; i < 5; i++) {
await page.goto('/', { waitUntil: 'networkidle' });
const lcp = await page.evaluate(() => new Promise<number>(resolve => {
new PerformanceObserver(list => {
const entries = list.getEntries();
resolve(entries[entries.length - 1]!.startTime);
}).observe({ type: 'largest-contentful-paint', buffered: true });
}));
samples.push(lcp);
}

samples.sort((a, b) => a - b);
const median = samples[Math.floor(samples.length / 2)]!;

// Soft upper bound: target is 1500ms typical, budget is 2500ms (variance absorbed)
expect(median).toBeLessThan(2500);

// Ship the value to observability for trend tracking (S12 Q8 pattern)
test.info().annotations.push({ type: 'lcp_median_ms', description:
String(Math.round(median)) });
});

// Bundle size budget enforced separately at build time
// size-limit config: [{ path: 'dist/main.js', limit: '250 KB' }]
```

Q 249 LEVEL · Senior

Where does Playwright performance measurement end, and when do you reach for k6, JMeter, or Artillery?

CONCEPT

Playwright measures **single-user performance under realistic conditions**: how fast does this page load, how snappy is this interaction, what are the Web Vitals for a real user. It does not measure system behaviour under concurrent load: what happens to the API when a thousand users hit it at once, where does the system degrade, what's the breaking point. Those are different questions, and they need different tools. **k6**: JavaScript-scriptable load testing, modern developer experience, integrates well with CI, Grafana dashboards. Best when the team already writes JS and the load model is scriptable. JMeter: GUI-driven, battle-tested, huge plugin ecosystem, used by enterprise QA teams for decades. Best when the team has JMeter skills and complex test plans matter more than developer ergonomics. Artillery: YAML-config or JS-script, lightweight, fast to spin up, good for simple smoke load tests in CI. The principle that matters most: **match the tool to the question**. Playwright answers ‘is the page fast enough for one user?’. Load tools answer ‘is the system stable enough for many users?’. The pitfall is conflating the two and claiming Playwright ‘does load testing’, it doesn't, and saying so loses senior credibility instantly.

INTERVIEW STRATEGY

They are after tool-boundary calibration. Define what Playwright measures (single-user, realistic conditions) and what it doesn't (concurrent load behaviour). Name three load tools (k6, JMeter,

Artillery) and when each wins. The 'match tool to question' framing demonstrates calibration. The pitfall is claiming Playwright does load testing. The next question is normally ‘which load tool would you actually choose?’, answer k6 for modern JS-stack teams, JMeter for enterprise QA, Artillery for fast CI smoke checks.

How to phrase it

> *Playwright measures single-user performance under realistic conditions. How fast does this page load, how snappy is this interaction, what are the Web Vitals for a real user navigating the app. It does not measure system behaviour under concurrent load. What happens to the API when a thousand users hit it at once, where does the system degrade, what's the breaking point under sustained traffic. Those are different questions, single-user perf and concurrent-load behaviour are different concerns at different layers, and they need different tools. Naming three that matter. k6: JavaScript-scriptable load testing with a modern developer experience, integrates well with CI and Grafana dashboards for visualising results over time. Best when the team already writes JS and the load model is scriptable. JMeter: GUI-driven, battle-tested, huge plugin ecosystem, used by enterprise QA teams for decades. Best when the team has JMeter skills and complex test plans matter more than developer ergonomics; the GUI can be a win for non-coders building load scenarios. Artillery: YAML-config or JS-script, lightweight, fast to spin up, good for simple smoke load tests in CI, the equivalent of a quick perf check that runs in five minutes rather than an hour. That's also my answer to the follow-up about which load tool to actually choose: k6 for modern JS-stack teams; JMeter for enterprise QA with existing JMeter skills; Artillery for fast CI smoke checks. The principle that matters most is match the tool to the question. Playwright answers is the page fast enough for one user. Load tools answer is the system stable enough for many users. Both questions need answers; one tool can't honestly answer both. A real performance strategy uses both: Playwright on every PR for single-user perf budgets, load tools on release candidates and scheduled jobs for capacity verification. The pitfall worth naming is conflating the two and claiming Playwright does load testing, it doesn't, and saying so loses senior credibility instantly. Interviewers watch for this exact tell; precision about what each tool actually does is what shows you've shipped both kinds of testing in production, not just read about them.*

Key points to hit

(1) Playwright = single-user perf under realistic conditions (Web Vitals, page load). **(2)** Load tools = system behaviour under concurrent load (degradation, breaking point). **(3)** k6: JS-scriptable, modern DX, Grafana dashboards, best for JS-stack teams. **(4)** JMeter: GUI-driven, plugin ecosystem, best for enterprise QA with existing skills. **(5)** Artillery: YAML or JS, lightweight, best for fast CI smoke load tests. **(6)** Pitfall: claiming Playwright does load testing, loses senior credibility instantly.

CODE

```
// Playwright: single-user perf budget (this is its lane)
test('checkout LCP under 2500ms', async ({ page }) => {
await page.goto('/checkout');
const lcp = await page.evaluate(() => new Promise<number>(res => {
new PerformanceObserver(list => {
const e = list.getEntries(); res(e[e.length - 1]!.startTime);
}).observe({ type: 'largest-contentful-paint', buffered: true });
}));
expect(lcp).toBeLessThan(2500);
});

# k6, concurrent load testing (different lane, different tool)
# k6/checkout-load.js
# import http from 'k6/http';
# export const options = {
# stages: [
# { duration: '2m', target: 100 },
# { duration: '5m', target: 500 },
# { duration: '2m', target: 0 },
# ],
# thresholds: { http_req_duration: ['p(95)<800'] },
# };
# export default function () {
# http.post('https://api.acme.com/orders', JSON.stringify({ /* ... */ }));
# }
#
# Run: k6 run --vus 100 --duration 5m k6/checkout-load.js
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. How are security tests in Playwright best characterised?

A) A replacement for penetration testing
B) Load tests
C) Functional tests with a security lens (XSS, auth bypass, data exposure)
D) Manual-only checks

2. What does a 'password not in the API response' test catch?

A) Slow responses
B) Sensitive-data exposure: e.g. a debug endpoint leaking the password hash
C) CORS misconfiguration
D) Rate-limit failures

3. What kind of performance testing does Playwright provide?

A) Performance-regression tests against a single-page-load budget
B) Load testing under thousands of concurrent users
C) Stress testing the database
D) Network throughput benchmarking

4. Which tool is appropriate for load testing, not Playwright?

A) ESLint
B) Pact
C) Allure
D) k6 or JMeter

5. Why is integrating OWASP ZAP as a proxy elegant?

A) It rewrites the tests automatically
B) It removes the need for assertions
C) Zero test-code change: functional tests map the attack surface ZAP then scans
D) It only scans the login page

Section 20 · Answer key

#	Answer	Why and where to revisit
1	C) Functional tests with a security lens (XSS, auth bypass, data exposure)	Standard Playwright assertions cover well-known vulnerability classes; they complement, not replace, a pen test. *Revisit Section 20.*
2	B) Sensitive-data exposure: e.g. a debug endpoint leaking the password hash	It asserts no password/passwordHash property is returned, catching leaks invisible to functional tests. *Revisit Section 20.*
3	A) Performance-regression tests against a single-page-load budget	Playwright asserts Web Vitals against an SLA budget; load testing is k6/JMeter territory. *Revisit Section 20.*
4	D) k6 or JMeter	Playwright measures single-load performance budgets; k6 or JMeter handle concurrent-user load. *Revisit Section 20.*
5	C) Zero test-code change: functional tests map the attack surface ZAP then scans	ZAP silently records every endpoint the tests exercise, so security coverage scales with functional coverage. *Revisit Section 20.*

SECTION 21

Behavioural and Soft Skills

Behavioural rounds decide as many SDET offers as technical ones, and the format is what trips people up: interviewers want a STAR-structured story with a quantified result, not a vague philosophy. These eight questions cover a test-suite performance win, handling coverage disagreements with developers, onboarding a new engineer, communicating results to non-technical stakeholders, and prioritising automation work in a sprint. Each model answer lands a concrete number: that is what makes it memorable.

In this section

- STAR structure (Situation, Task, Action, Result) with a quantified result.
- Reframing coverage disagreements as risk-and-cost conversations.
- Progressive-complexity onboarding measured by time-to-first-merged-test.
- Translating test results into business confidence for stakeholders.
- Prioritising automation work: a ranked, non-negotiable capacity allocation.

Q 250 LEVEL · Mid

Tell me about a time you improved test suite performance significantly.

CONCEPT

Answer with the **STAR method**, Situation, Task, Action, Result, and **quantify** the outcome. A strong structure: a slow regression suite blocking deployments (Situation), a target to cut it without losing coverage (Task), a profiled, specific intervention (Action), and a measured improvement (Result). The strongest Actions are concrete and diagnostic: profile with the JSON reporter to find the slowest tests, move their setup from UI flows to API calls, enable fullyParallel, and shard across CI. The Result must carry numbers, a runtime drop from 45 minutes to 8 (an 82% reduction) and a jump in deployment frequency, because in a behavioural answer the number is what proves the story is real rather than rehearsed.

INTERVIEW STRATEGY

The interviewer is evaluating structured communication and whether you drive measurable impact, not just activity. Use STAR explicitly and front-load that you measured the problem before acting, profiling, not guessing. The decisive element is the quantified result; a story without a number reads as hypothetical. Where this goes wrong is a vague 'I made the tests faster' with no diagnosis or metric. A common follow-up is 'how did you find the slow tests?', answer profiling with the JSON reporter, so be ready to show the diagnostic step.

How to phrase it

> *I will answer this with STAR. The situation: our Playwright regression suite took forty-five minutes in CI, which blocked deployments for nearly an hour and was throttling how often we could ship. The task I set myself was to get it under fifteen minutes without reducing coverage, faster, but not by deleting tests. For the action, the first thing I did, and this answers the follow-up about how I found the slow tests, was profile rather than guess: I ran the suite with the JSON reporter and identified the ten slowest tests, which turned out to account for about forty percent of total runtime. They were slow because they did their setup through the UI, filling forms, waiting for redirects, so I moved that setup to API calls, which dropped per-test setup from around thirty seconds to three. Then I enabled fullyParallel and implemented four-shard execution in CI. The result, and I lead with the number because it is what makes the story real: the suite went from forty-five minutes to eight, an eighty-two percent reduction, and deployment frequency went from twice a day to on-demand. The reason I tell it this way is that the diagnosis is the point, I did not blindly parallelise everything, I measured where the time actually went and fixed the biggest contributors first. The trap in this kind of question is saying I made the tests faster with no method and no metric; the profiling step and the eighty-two percent are what show it was real engineering.*

Key points to hit

(1) Use STAR explicitly: Situation, Task, Action, Result. **(2)** Profile first (JSON reporter): diagnose before acting. **(3)** Action: API setup over UI, fullyParallel, sharding. **(4)** Quantify: 45 min -> 8 min (82%), deploys twice-daily -> on-demand. **(5)** Asked next: how you found the slow tests, profiling. **(6)** Where this goes wrong: 'I made the tests faster' with no method or number.

Q 251 LEVEL · Mid

How do you handle disagreements with developers about test coverage?

CONCEPT

Frame the disagreement as a **risk conversation, not an opinion debate**, and use data plus business impact to settle it. The classic case: a developer argues that payment error handling does not need automated tests because 'it never fails in practice'. Rather than argue, reframe to the business question, what is the impact if it breaks in production? Quantify it: if 2% of transactions fail, average order value is $150, and an hour of broken error handling means roughly $3,000 in abandoned carts, against a cost of two hours to write three tests. Stated that way, the decision is obvious and the developer agrees immediately. The principle: coverage decisions are **risk management, not perfectionism**, so you make them with numbers, not with who feels more strongly.

INTERVIEW STRATEGY

The interviewer is assessing collaboration and whether you persuade with data rather than authority. Lead with reframing the dispute as a risk-and-cost conversation, then show the actual back-of-envelope math that ended it. That quantified reframing marks senior judgement and it keeps the relationship intact. The risk is escalating it into a who's-right standoff. The follow-up is often 'what if the numbers had favoured the developer?', answer that then they would be right and you would deprioritise it; risk math cuts both ways.

How to phrase it

> *I frame these as risk conversations, not opinion debates, because two people asserting what they believe never resolves and it damages the working relationship. The specific situation I would give: a developer argued that the payment error handling did not need automated tests because, in his words, it never fails in practice. Instead of arguing the point, I reframed it to the business question, what is the impact if it does break in production? We worked the numbers together. about two percent of transactions fail and hit that error path, average order value is around a hundred and fifty dollars, so an hour of broken error handling is roughly three thousand dollars in abandoned carts. Against that, the cost of writing the three tests was about two hours. Put that way, the developer agreed immediately, there was nothing to argue about once it was a number. That cuts both ways, which is my answer to the follow-up about what if the math had favoured him: if the risk and cost genuinely did not justify the tests, then he would have been right and I would have deprioritised it, because the whole point is risk management, not testing everything out of perfectionism. The trap is letting it become a who-is-right standoff, where I pull rank as the QA person and he digs in as the developer. Turning it into shared math means we are solving the same problem together rather than competing, and the data, not the loudest voice, makes the call.*

Key points to hit

(1) Reframe as a risk conversation, not an opinion debate. **(2)** Use business impact: ~2% fail x $150 AOV ≈ $3,000/hr vs 2 hrs of tests. **(3)** Stated as a number, the decision becomes obvious and

agreement is fast. **(4)** Principle: coverage is risk management, not perfectionism. **(5)** Follow-up: if the math favours the developer, they're right, it cuts both ways. **(6)** What loses the point: escalating into a who's-right standoff.

Q 252 LEVEL · Mid

How do you onboard a new engineer to a Playwright framework?

CONCEPT

Structured onboarding with **progressive complexity** gets a new engineer to contribution fast. A proven arc: Day 1 clone, install, run the suite locally and understand the structure; Day 2 read around ten existing tests to absorb fixture usage, locator patterns, and assertion style; Day 3 pair on writing a new test, explaining locator priority and fixture composition; Week 2 write a test solo with a detailed code review; Week 3 contribute a new page object or fixture and learn the architecture. A maintained **CONTRIBUTING.md** documenting locator priority, fixture patterns, and the review checklist lets new engineers self-serve most questions. Measure success by **time-to-first-merged-test**, a concrete metric (e.g. an average of 4 days) that proves the program works rather than just exists.

INTERVIEW STRATEGY

The interviewer is gauging mentorship and whether you build repeatable process, not heroics. Lead with the progressive-complexity arc, run, read, pair, solo, contribute, then add the two things that make it scale: a CONTRIBUTING.md for self-service and a time-to-first-merged-test metric. Measuring onboarding is the senior signal. The trap is ad-hoc 'ask me anything' onboarding that does not scale. Expect the follow-up 'how do you know it works?', answer with the time-to-first-merged-test number.

How to phrase it

> *My onboarding gets new engineers writing production tests in their first week, and the key is progressive complexity rather than throwing them at the codebase. Day one, they clone the repo, install, and run the suite locally so they understand the structure and config before touching anything. Day two, they read about ten existing tests to absorb how we use fixtures, our locator patterns, and our assertion style, reading good examples first means their first test looks like ours. Day three, we pair on writing a new test together, and I explain locator priority and fixture composition as we go. Week two, they write a test solo and I give a detailed code review. Week three, they contribute a new page object or fixture and start to see the full architecture. Two things make this scale beyond me personally. I maintain a CONTRIBUTING.md documenting our locator priority, fixture patterns and review checklist, so a new engineer can answer most questions themselves before asking me, and I measure the program rather than assume it works, which is my answer to the follow-up about how I know it works: I track time-to-first-merged-test, and our current average is about four days. That metric tells me whether a change to the onboarding actually helped. The thing to avoid is ad-hoc onboarding where the new person just asks whoever is free; it does not scale, the answers are inconsistent, and there is no signal on whether it is working. Structured, documented, and measured is what makes onboarding repeatable across a growing team.*

Key points to hit

(1) Progressive complexity: run -> read -> pair -> solo -> contribute. **(2)** Read existing tests before writing so the first test matches house style. **(3)** CONTRIBUTING.md (locator priority, fixtures, review checklist) for self-service. **(4)** Measure with time-to-first-merged-test (e.g. ~4 days). **(5)** Expect next: how you know it works, the time-to-first-merged-test metric. **(6)** The common miss: ad-hoc 'ask anyone' onboarding that doesn't scale.

Q 253 LEVEL · Junior to Mid

How do you communicate test results to non-technical stakeholders?

CONCEPT

Non-technical stakeholders need **business impact and confidence**, not technical metrics, so you translate results into risk language. The practice: never send a raw link to the HTML report. Send a **one-paragraph summary**: pass count and rate, what the failures are and whether they are critical, explicit confirmation that the high-stakes flows (payment, authentication, checkout) passed, and a clear confidence-to-deploy verdict. That gives them what they need to make a deployment decision without understanding test infrastructure. Backing it with a **pass-rate trend dashboard** lets them see quality improving or degrading at a glance. The skill is meeting the audience where they are: a 99.6% pass rate with 'confidence: HIGH' is actionable; a stack trace is not.

INTERVIEW STRATEGY

You are being marked on whether you can translate for a non-technical audience. Contrast sharply: never a raw report link, always a one-paragraph business summary with a confidence verdict. Naming the high-stakes flows that passed (payment, auth, checkout) is what gives them comfort. The mistake is dumping technical detail or a report URL on someone who cannot act on it. They usually follow with 'what do they actually need from you?', answer a deploy/no-deploy confidence call, not the test internals.

How to phrase it

> *Non-technical stakeholders need business impact and confidence, not technical metrics, so I translate results into risk language. Concretely, I never send a stakeholder a link to the HTML report, that is the trap, handing someone a stack trace and a test tree they cannot act on. Instead I send a one-paragraph summary, something like: this release passed 847 of 850 automated tests, 99.6 percent; the three failures are in the report-export feature, which is non-critical and has a workaround, and the fix is scheduled for next sprint; the payment, authentication and checkout flows all passed; confidence to deploy is high. That gives them exactly what they need to make a deployment decision without understanding the test infrastructure, and it answers the follow-up about what they actually need from me: a deploy or no-deploy confidence call, with the high-stakes flows explicitly called out, not the internals. Naming payment, auth and checkout specifically matters, because those are the flows a business owner actually worries about, confirming they are green is what gives real comfort. I back the summary with a pass-rate trend dashboard so they can see quality improving or degrading at a glance over time, which builds trust between releases. The whole skill is meeting the audience where they are: 99.6 percent passing with a clear high-confidence verdict is information they can act on; the same data as a raw report is noise to them.*

Key points to hit

(1) Translate results into business impact and confidence, not metrics. **(2)** Never send a raw report link: send a one-paragraph summary. **(3)** Include pass rate, failure criticality, and a confidence-to-deploy verdict. **(4)** Explicitly confirm the high-stakes flows (payment, auth, checkout) passed. **(5)** They usually ask next: they need a deploy/no-deploy call, not test internals. **(6)** Avoid: dumping a report URL or technical detail on a non-technical audience.

Q 254 LEVEL · Mid

How do you prioritise test automation work in a sprint?

CONCEPT

Automation work competes with feature development for capacity, so you treat it as a **product with a backlog** and a ranked priority order. A proven ranking: P1 blocking failures, failing tests that block CI for the whole team; P2 new-feature coverage, automate the features delivered this sprint, because coverage debt compounds; **P3 high-risk gaps** from the risk matrix; P4 flaky-test fixes, since flakiness erodes confidence in the suite; P5 framework improvements, refactors and utilities, important but not urgent. The enabling discipline: allocate a fixed, **non-negotiable share of capacity** (for example 20%) to automation every sprint, and never let coverage debt accumulate, a feature shipped without tests requires manual regression forever. Tracking coverage debt as a metric keeps it visible in sprint review.

INTERVIEW STRATEGY

They want to see disciplined prioritisation under competing demands. Give a clear ranked order P1-P5 and the principle behind it, then add the enabler: a fixed, non-negotiable capacity allocation and treating coverage debt as a tracked metric. That product-with-a-backlog framing is the seniority marker. The thing to avoid is letting automation be whatever is left over after features, which is nothing. The next question is normally ‘how do you stop coverage debt piling up?’, answer the fixed allocation plus making the debt visible in sprint review.

How to phrase it

> *I treat test automation as a product with a backlog rather than whatever is left after features, and I prioritise it in a clear ranked order. Priority one is blocking failures: tests that are failing and blocking CI, because those block the entire team, so they come before anything. Priority two is coverage for features delivered in the current sprint, because coverage debt compounds, a feature shipped without tests this sprint is harder to retrofit next sprint. Priority three is high-risk gaps identified in the risk matrix. Priority four is flaky-test fixes, because flakiness quietly erodes the team's confidence in the suite until they stop trusting red. Priority five is framework improvements, refactors, new utilities, performance, important but not urgent. The discipline that makes this actually work, and my answer to the follow-up about stopping coverage debt piling up, is two things: I allocate a fixed twenty percent of capacity to automation every sprint and treat it as non-negotiable, and I track coverage debt as a metric that shows up in sprint review so it stays visible. The key principle is never letting coverage debt accumulate, because a feature shipped without automated tests is a feature that needs manual regression forever, which is a tax that compounds release over release. The classic error is letting automation be the thing that gets cut when the sprint gets tight, then there is never any capacity for it and the debt grows unbounded. Protecting the allocation and making the debt visible is what keeps the suite from rotting.*

Key points to hit

(1) Treat automation as a product with a ranked backlog. **(2)** P1 blocking failures, P2 new-feature coverage, P3 high-risk gaps, P4 flaky fixes, P5 framework. **(3)** Allocate a fixed, non-negotiable ~20% of capacity each sprint. **(4)** Never let coverage debt accumulate: it becomes permanent manual regression. **(5)** The next question: stop debt piling up via fixed allocation + visible metric in review. **(6)** The easy mistake: letting automation be the leftover that gets cut when sprints tighten.

Q 255 LEVEL · Mid to Senior

Tell me about a critical bug that escaped your test suite. What did you do, and what changed afterwards?

CONCEPT

This is a post-mortem question disguised as a behavioural one, testing whether you take ownership of escapes without deflection, diagnose the root cause beyond “we needed more tests”, and turn the incident into systemic improvement rather than a one-off fix. The interviewer wants to see **blameless post-mortem thinking**: the bug escaped because of a gap in test design or process, not because someone messed up. They want the **specific category of gap** identified, was it a class of test we didn't run, an assumption we didn't verify, an environment difference we didn't cover? They want the systemic change that prevents the whole category, not just the one bug, from recurring. Strong answers name a specific bug, walk through the diagnosis honestly including what they got wrong, and close with the framework or process change that resulted. Weak answers are defensive (“it wasn't really our fault”), generic (“we added more tests”), or blame-shifting (“the developers didn't tell us”). The discipline: **treat escapes as data, not shame**. Every escaped bug is information about where the test strategy has gaps.

INTERVIEW STRATEGY

This one tests ownership, root-cause thinking, and systemic-improvement instinct. Pick a real bug, walk diagnosis honestly, name the category of gap, close with the framework change. The 'escapes as data, not shame' framing is the senior signal. The trap is blame-shifting or ‘we needed more tests’ vagueness. A common follow-up is ‘what would you do differently next time?’, answer the specific check that's now in place for this category of bug.

How to phrase it

> *About eighteen months ago a payment rounding bug shipped to production in our enterprise billing flow. The system calculated taxes by line item, summed them, then rounded the total. The bug was the rounding direction, banker's rounding in some code paths, half-up in others, produced one-cent discrepancies on roughly two percent of invoices. Finance noticed during reconciliation, raised a ticket, and I owned the post-mortem. The diagnosis honestly: we had hundreds of tests for the billing flow, but every test used round numbers like a hundred dollars times eight percent equals exactly eight. Our test data never produced the kind of awkward decimals that exposed the inconsistency, things like a hundred and twenty-seven dollars and forty-three cents times six and a quarter percent. The bug wasn't that we needed more tests; the bug was that our test data was systematically unrepresentative. The category of gap had a name once I identified it: input-space coverage. We tested behaviour exhaustively but only on a narrow slice of the input space, the easy-arithmetic slice. That's a different problem from needing more tests; it's needing different tests. The systemic change was twofold. First, for the billing domain specifically, I added property-based testing using fast-check: generate thousands of random line-item amounts and tax rates, run them through the calculation, assert mathematical properties hold, totals are sum-consistent, rounding is direction-consistent, no path produces NaN or negative tax. That caught the bug immediately on the first run. Second, and this is the part that scales, I wrote up the pattern as a team practice: any calculation-heavy domain gets property-based tests, not just example-based. We applied it to discount calculations, currency conversion, shipping cost. The escape stopped being a billing-bug-fix and became a test-strategy change. What I learned, and the framing I'd close on, is treat escapes as data, not shame. Every escaped bug is information about where the test strategy has gaps. The instinct to be defensive or blame the developer for not catching it during code review misses the point: tests exist to catch what humans miss; if a class of bug escapes, the test strategy has a category-shaped hole. That's also my answer to the follow-up about what I would do differently. The property-based tests are now part of our framework conventions; new billing-domain features get them by default. The specific check that's now in place is the property test for rounding-direction consistency that would have caught the original bug.*

Key points to hit

(1) Pick a real bug; walk diagnosis honestly including what was missed. **(2)** Identify the category of gap (input space, environment, assumption), not 'need more tests'. **(3)** Close with the systemic change that prevents the whole category. **(4)** 'Escapes as data, not shame': information about where strategy has gaps. **(5)** Asked next: name the specific check now in place for this category. **(6)** Trap: defensive answers, blame-shifting, or vague 'we added more tests'.

Q 256

LEVEL · Senior

How have you convinced leadership to invest in test infrastructure when they wanted feature delivery instead?

CONCEPT

This question tests whether you can **translate engineering concerns into business language**, and whether you understand that “invest in test infrastructure” is a hard sell that requires evidence, not principle. The interviewer wants to see **quantified cost of the current state** (time spent on flake, escapes per quarter, feature velocity drag from manual regression), specific proposed investment with a scoped budget, and **measurable outcome** tied to a metric leadership already cares about. Strong answers convert “we need a better framework” into “our current framework costs eight engineer-days per release cycle in flake triage; investing two engineer-months in framework hardening pays back in three quarters and reduces release-blocking flakes by 70%”. Weak answers argue from principle (“we need quality”) or technical detail (“the framework has technical debt”), both ignore that leadership trades investments against each other and needs to compare on the same axis. The discipline: quantify the pain, quantify the proposal, tie outcomes to business metrics. Engineering investment wins arguments when it's presented as ROI, not as virtue.

INTERVIEW STRATEGY

What is being tested is business-translation skill. Quantify the current cost, propose a scoped investment, tie to a metric leadership cares about. The 'ROI not virtue' framing signals depth here. The thing to avoid is arguing from principle. The follow-up is often ‘what if leadership still says no?’, answer revisit with data after the next big incident; escapes and outages are the highest-leverage moments to ask.

How to phrase it

> *About two years ago I needed leadership buy-in to invest two engineer-months in framework hardening when the natural instinct was to keep the team on feature work for the upcoming release. The first version of my pitch was the wrong version: I argued from principle, talked about technical debt, said we needed better infrastructure for quality. Leadership listened, agreed in principle, and kept us on feature work because feature work had measurable revenue impact and quality didn't. I lost the argument because I'd brought engineering language to a business conversation. So I rebuilt the pitch in business terms. First, I quantified the current cost. I pulled three months of CI data and calculated: forty-seven engineer-hours per release cycle were going to flake triage, reruns, investigation, fixing tests that were broken, not the system. Twelve hours per cycle were going to manual regression that automation could have covered but didn't because the framework made certain test categories impractical to write. Three escaped bugs in the previous quarter had been in areas the framework couldn't easily test, with a combined cost of customer credits and engineering response time of roughly twelve thousand dollars. The current state had a number, and the number was real money. Second, I quantified the proposal. Two engineer-months of focused framework work, scoped to specific outcomes: parallel-safe test isolation, structured fixtures, and a flake-tracking dashboard. Concrete deliverables on a concrete timeline, not open-ended. Third, I tied outcomes to a metric leadership already cared about: release velocity. The release cycle was currently twelve days from code-freeze to ship, with three of those days as test-triage. The pitch was: two months of framework investment reduces post-freeze test work to one day, shortening release cycles by two days, which let us ship one extra feature per quarter. Now the investment had a business-side return leadership understood. The pitch worked the second time. The discipline I would close on, because it's where engineering-coded arguments fail and ROI-coded arguments win, is quantify the pain, quantify the proposal, tie outcomes to business metrics. Engineering investment wins arguments when it's presented as ROI, not as virtue. Leadership isn't anti-quality; they're trading investments against each other and need to compare on the same axis. That's also my answer to the follow-up about what to do if they still say no: revisit with data after the next big incident. Escapes and outages are the highest-leverage moments to ask because the cost just became concrete, the same proposal lands differently the week after a production incident than it did the week before. The pitfall is arguing from principle or technical detail; both ignore that leadership is comparing your ask against other asks on a business axis.*

Key points to hit

(1) Quantify the current cost: engineer-hours, escapes, customer impact in dollars. **(2)** Scope the proposal: specific deliverables, concrete timeline, measurable outcomes. **(3)** Tie to a metric leadership already cares about (velocity, revenue, customer satisfaction). **(4)** Engineering arguments fail; ROI arguments win: same axis as other investments. **(5)** Follow-up: revisit after the next incident, cost just became concrete. **(6)** Where this goes wrong: arguing from principle or technical detail, wrong conversational frame.

Q 257 LEVEL · Senior

How do you handle a critically important test that's flaky and can't be quickly fixed?

CONCEPT

This question tests **pragmatism under constraint**: you can't simply delete a flaky test of a critical flow because the coverage matters, but you can't let it keep blocking every PR with noise either. The interviewer wants to see **short-term containment** (retries, quarantine queue, allow-fail in PR but must-pass in nightly), investigation discipline (treat as bug, file the ticket, track to resolution), and **communication** with the team about the state, a quarantined test is not a forgotten test. Strong answers walk a specific case: what the test covered, why it was flaking, how they contained without deleting, the investigation that eventually fixed it. Weak answers are extremes: “delete the flaky test” (loses critical coverage) or “keep failing the build until we fix it” (loses team velocity and trust). The discipline: quarantine is a temporary state with an owner and a deadline, not a graveyard. Tests in quarantine for months stop being containment and become lies about coverage. The team needs to know what's quarantined and why; the fix needs an owner and a date.

INTERVIEW STRATEGY

The test here is pragmatism under tension between coverage and velocity. Walk three layers: short-term containment (retries / quarantine / nightly-only), investigation as a tracked bug, communication about state. The 'quarantine has owner and deadline' rule is the seniority marker. The mistake is extremes (delete or keep failing). Expect the follow-up ‘how long before you delete it?’, answer never delete without a replacement; the critical coverage has to come from somewhere.

How to phrase it

> *About a year ago we had a critically important checkout-flow test that started flaking after a deploy. The flake rate was about fifteen percent: enough to make every PR feel unreliable but not enough that we could easily reproduce it locally. The test covered the high-value path: cart to payment to confirmation. Deleting it wasn't an option because that's where revenue happens; if a regression breaks checkout we lose money the day it ships. But letting it keep failing was also breaking the team, PRs were getting re-run constantly, engineers started ignoring failures because 'probably just that test again', and the suite was losing credibility. So I worked three layers in parallel. First, short-term containment. I moved the test to a quarantine queue: it still ran on every PR but with retries enabled and marked allow-fail in PR context. The PR-level signal stopped blocking merges. In nightly schedule, the same test ran without retries and must-pass. If it failed nightly, we investigated; if it failed PR-only, we logged but didn't block. That stopped the velocity hit while preserving the regression-catch capability. Second, investigation discipline. The test wasn't a flaky-test problem; it was a real-system-race problem. I filed it as a bug, not a test issue, because the system had a race condition between the payment confirmation callback and the order-update message. Two engineers from the payment team owned it; I owned the test reproduction and verification. The fix took three weeks because the race required rearchitecting the message-ordering guarantees, but the test went back to blocking once the race was fixed. Third, and the part most candidates miss, communication. I posted the quarantine state in the team's standup channel: which tests, why, who owned the fix, target date. A quarantined test that nobody knows is quarantined becomes a lie about coverage, the suite says checkout is tested, but the team has implicitly agreed to ignore the test, so it isn't really tested. Visibility kept the quarantine honest. The discipline I would close on, because it's where quarantine systems go wrong, is quarantine is a temporary state with an owner and a deadline, not a graveyard. Tests in quarantine for months stop being containment and become lies about coverage. The team needs to know what's quarantined and why; the fix needs an owner and a date. That's also my answer to the follow-up about when to delete: never delete without a replacement. The critical coverage has to come from somewhere; if the original test was fundamentally unfixable, write a different test at a different layer that covers the same risk. The trap is extremes: deleting critical coverage to make CI green, or keeping the failing test in place until it breaks the team's trust in the suite entirely.*

Key points to hit

(1) Containment: quarantine queue, allow-fail in PR, must-pass nightly. **(2)** Investigation: treat as a bug, file the ticket, assign an owner, track to resolution. **(3)** Communication: team knows what's quarantined, why, who owns it, target date. **(4)** Quarantine = temporary state with owner + deadline, never a graveyard. **(5)** Expect next: never delete without a replacement at a different layer. **(6)** What loses the point: extremes, delete (lose coverage) or keep blocking (lose team trust).

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What structure should a 'tell me about a time' answer follow?

A) STAR, Situation, Task, Action, Result, with a quantified result
B) A list of technologies used
C) A chronological diary
D) Only the result

2. How do you best resolve a coverage disagreement with a developer?

A) Pull rank as the QA lead
B) Escalate to the manager immediately
C) Reframe it as a risk-and-cost conversation backed by numbers
D) Add the tests without telling them

3. What metric measures onboarding success for a test framework?

A) Lines of test code written
B) Number of meetings attended
C) Time-to-first-merged-test
D) Total tests in the suite

4. How should you report test results to a non-technical stakeholder?

A) Send a link to the HTML report
B) A one-paragraph summary with pass rate, failure criticality, and a confidence-to-deploy verdict
C) A full stack trace of failures
D) The raw JSON results

5. What is the key discipline in prioritising automation work?

A) Doing it only when there is spare time
B) Letting developers decide each sprint
C) Always prioritising framework refactors first
D) A fixed, non-negotiable capacity allocation so coverage debt never accumulates

Section 21 · Answer key

#	Answer	Why and where to revisit
1	A) STAR, Situation, Task, Action, Result, with a quantified result	STAR keeps the story structured; the quantified result is what proves it is real rather than rehearsed. *Revisit Section 21.*
2	C) Reframe it as a risk-and-cost conversation backed by numbers	Quantifying business impact vs cost turns an opinion debate into a shared decision, and it cuts both ways. *Revisit Section 21.*
3	C) Time-to-first-merged-test	Time-to-first-merged-test shows whether progressive-complexity onboarding actually gets people contributing fast. *Revisit Section 21.*
4	B) A one-paragraph summary with pass rate, failure criticality, and a confidence-to-deploy verdict	Stakeholders need a deploy/no-deploy confidence call and confirmation the high-stakes flows passed, not internals. *Revisit Section 21.*
5	D) A fixed, non-negotiable capacity allocation so coverage debt never accumulates	Protecting a fixed allocation (e.g. 20%) and tracking coverage debt keeps the suite from rotting over time. *Revisit Section 21.*

SECTION 22

Advanced TypeScript for SDETs

These are the TypeScript features senior interviewers ask about to see whether you use the type system to prevent bug classes, not just to annotate variables. They cover template literal types for API endpoints, mapped types for self-maintaining test-data generators, decorators for metadata-driven test execution, discriminated unions for state machines, and module augmentation to extend Playwright's types cleanly.

In this section

- Template literal types pin API endpoint shapes at compile time.
- Mapped types create test-data generators that stay in sync with interfaces.
- Decorators for metadata-driven test discovery: when you need them.
- Discriminated unions for state machines, with exhaustive switch checking.
- Module augmentation extends Page and Matchers without forking Playwright.

Q 258 LEVEL · Mid to Senior

How do you use conditional and template literal types in a Playwright framework?

CONCEPT

Conditional types create type-level logic: the resulting type depends on another type, useful for a flexible API client whose response shape varies by HTTP method. **infer** extracts a type from inside another type, for example Awaited<T> pulling the resolved value out of a Promise. The practical winner in automation frameworks is template literal types for typed endpoints: defining **UserEndpoint as `/api/users/${number}`** means TypeScript catches **fetchUser('/api/users/abc')** at compile time, a bug that would otherwise only surface as a 404 at runtime. Use this for every typed API client. It is the kind of advanced TypeScript that signals senior-level fluency in an interview.

INTERVIEW STRATEGY

Your job here is to show you apply advanced types to real problems, not recite syntax. Lead with template literal types for endpoints, the compile-time catch on a malformed URL, because it is the most concretely useful pattern in automation. Mention conditional types and infer briefly for breadth. The risk is describing conditional types in the abstract. They usually follow with 'what bug does this prevent?', answer the path-with-the-wrong-type 404 caught at compile time instead of runtime.

How to phrase it

> *Conditional types create type-level logic where the resulting type depends on another type, useful for an API client whose response shape varies by HTTP method, and infer extracts a type from inside another, like Awaited unwrapping a Promise to get the resolved value. But the pattern I would lead with because it pays off most in a framework is template literal types for typed endpoints. If I define UserEndpoint as the template literal slash-api-slash-users-slash-dollar-number, then calling fetchUser with '/api/users/42' is valid, but calling it with '/api/users/abc' is a TypeScript error at compile time, string is not assignable to number. That answers the follow-up about what bug this prevents: without it, the wrong path type only surfaces at runtime as a 404, which means the test goes red for a reason that looks like a backend problem when really it is a path format mistake. Catching it at compile time is dramatically faster to diagnose because the error names exactly where and what is wrong. I use this pattern for every typed API client in the framework, and across hundreds of API calls it removes a whole class of subtle bugs for essentially zero runtime cost. This is the kind of TypeScript knowledge that signals senior-level fluency, because it is using the type system to enforce a contract the language would otherwise happily let you violate.*

Key points to hit

(1) Conditional types: type-level logic; infer extracts a type (e.g. Awaited<T>). **(2)** Template literal types pin endpoint shapes: `/api/users/${number}`. **(3)** Catches /api/users/abc at compile time instead of as a runtime 404. **(4)** Apply to every typed API client: enforces the contract for free. **(5)** They usually ask next: prevents the wrong-path 404, fast diagnosis vs runtime hunt. **(6)** The common miss: describing conditional types in the abstract with no application.

CODE

```
type APIResponse<T extends 'GET' | 'POST' | 'DELETE'> =
T extends 'GET' ? { data: unknown; status: number } :
T extends 'POST' ? { data: unknown; id: number; status: number } :
T extends 'DELETE' ? { success: boolean; status: number } : never;

type Awaited<T> = T extends Promise<infer U> ? U : T;
type LoginResult = Awaited<ReturnType<typeof loginPage.login>>;

// Template literal types pin endpoint shapes at compile time
type UserEndpoint = `/api/users/${number}`;
function fetchUser(endpoint: UserEndpoint) { /* ... */ }
fetchUser('/api/users/42'); // Valid
// fetchUser('/api/users/abc'); // TS error: string not assignable to number
```

Q 259 LEVEL · Mid to Senior

How do you use mapped types for test data generation?

CONCEPT

Mapped types transform every property of an existing type, and combined with faker they create type-safe data generators that stay in sync with the interface. A **DataGenerator<T>** mapped type produces a record where each key of T maps to a function returning the value of that key, so a userGenerator object has one generator per field of the User interface, each typed correctly. The self-maintaining property is the win: when a new required field is added to the User interface, TypeScript immediately shows userGenerator is **missing a generator for that field**. Without the mapped type, a new required field would cause silent undefined values in generated test data, the kind of bug that only surfaces as a confusing test failure days later.

INTERVIEW STRATEGY

Show you use the type system to prevent test-data drift. Centre on the self-maintaining property: add a field to the interface, the compiler points at the missing generator. That compile-time-fan-out is the same principle as deriving request types from a single source of truth, applied to fakers. The pitfall is treating the data generator as a plain object that silently goes out of sync. The next question is normally ‘what happens if a new field is added?’, answer TypeScript names the missing generator immediately.

How to phrase it

> *Mapped types transform every property of an existing type, and paired with faker they give me type-safe data generators that stay in sync with the interface they generate for. The pattern is a DataGenerator of T mapped type, which says for every key in T, give me a function that returns the value of that key. So userGenerator is an object with one generator per field of the User interface, a name generator, an email generator, an age generator, each typed correctly to the field it produces. The property that makes this worth using, and my answer to the follow-up about what happens when a new field is added, is that it is self-maintaining. The moment someone adds a phoneNumber field to the User interface, TypeScript immediately tells me userGenerator is missing a generator for phoneNumber, and the build fails until I add one. Without the mapped type, that new field would silently default to undefined in every generated user, and the bug would only surface downstream as confusing test failures days later when something tried to use the missing data. The thing to avoid is treating the generator as a plain object that quietly drifts out of sync. With the mapped type, drift is structurally impossible, the compiler is the safety net, the same single-source-of-truth principle I apply to typed API contracts, applied to test data. It is the kind of type-level safety that prevents an entire category of test-data bugs in a large framework.*

Key points to hit

(1) Mapped types transform every property of a type into a derived shape. **(2)** DataGenerator<T> = one generator function per field of T, fully typed. **(3)** Self-maintaining: add a field to the interface, the compiler names the gap. **(4)** Without it, a new required field silently becomes undefined in test data. **(5)** The next question: new field -> TS immediately flags missing generator. **(6)** Avoid: a plain-object generator that drifts silently out of sync.

CODE

```
type DataGenerator<T> = { [K in keyof T]: () => T[K] };

interface User { name: string; email: string; age: number; role: 'admin' | 'user'; }

const userGenerator: DataGenerator<User> = {
name: () => faker.person.fullName(),
email: () => faker.internet.email(),
age: () => faker.number.int({ min: 18, max: 80 }),
role: () => faker.helpers.arrayElement(['admin', 'user'] as const),
};

function generateUser(overrides?: Partial<User>): User {
const base = Object.fromEntries(
Object.entries(userGenerator).map(([k, gen]) => [k, (gen as () => unknown)()]),
) as User;
return { ...base, ...overrides };
}
```

Q 260 LEVEL · Mid to Senior

How do you use decorators for test metadata in TypeScript?

CONCEPT

TypeScript decorators (with **experimentalDecorators** enabled) attach metadata to test classes and methods, enabling automatic test registration and tagging. A **@TestSuite** decorator records the suite name on the class; a **@TestCase** decorator records tags and priority on each method. A test runner can then query that metadata programmatically, every test tagged @smoke, without string-matching test titles. The honest take: decorators are more common in Java/JUnit frameworks and increasingly used in TypeScript, but for most Playwright projects the simpler approach of @smoke in the title with --grep is sufficient. Reach for decorators only when the framework needs sophisticated metadata querying (a custom orchestrator, dynamic test selection, an internal reporting platform that consumes structured metadata).

INTERVIEW STRATEGY

This question screens for judgement on when complexity is warranted. Explain decorators briefly, but the real signal is naming when they are overkill: for most Playwright projects, --grep on title tags is enough, and decorators only earn their place when you need programmatic metadata querying. That calibration is the senior answer. The failure mode is recommending decorators for every project. A common follow-up is 'when would you actually use them?', answer when a custom orchestrator or internal platform needs structured metadata, not for ordinary tag-based selection.

How to phrase it

> *TypeScript decorators, with experimentalDecorators enabled, let me attach metadata to test classes and methods. A TestSuite decorator on the class records the suite name; a TestCase decorator on each method records tags and priority. A test runner can then query that metadata programmatically rather than parsing strings out of test titles. The honest position I take in interviews, which I think is the senior signal, is that decorators are more common in Java and JUnit frameworks and are increasingly used in TypeScript, but for most Playwright projects the simpler approach of an at-smoke label in the test title with --grep is sufficient and easier to read. So I do not recommend decorators for every project; I reach for them only when the framework needs sophisticated metadata querying, and that is my answer to the follow-up about when I actually use them. The concrete case is a custom orchestrator that selects tests dynamically by priority and tag, or an internal reporting platform that consumes structured metadata about every test for analytics. In both, string-matching titles is fragile and metadata via decorators is the robust answer. The classic error is recommending decorators reflexively because they sound advanced; for ordinary tag-based selection it is overkill and adds a Reflect-metadata dependency and a layer of indirection for no gain. The judgement, use the simplest mechanism that does the job, escalate only when the job demands more, is more impressive than the decorator syntax itself.*

Key points to hit

(1) Decorators attach metadata to test classes/methods; queryable programmatically. **(2)** @TestSuite on class, @TestCase on methods (tags, priority). **(3)** Most Playwright projects: simpler @smoke title + --grep is enough. **(4)** Reach for decorators when a custom orchestrator/platform needs structured metadata. **(5)** Asked next: actually used when string-matching titles is fragile. **(6)** The easy mistake: recommending decorators for every project, overkill.

CODE

```
// tsconfig.json: "experimentalDecorators": true
import 'reflect-metadata'; // Reflect.defineMetadata comes from this polyfill
function TestSuite(name: string) {
return function(ctor: Function) { Reflect.defineMetadata('suite:name', name, ctor); };
}
function TestCase(opts: { tags?: string[]; priority?: 'high' | 'medium' | 'low' }) {
return function(target: any, key: string) { Reflect.defineMetadata('test:options', opts,
target, key); };
}

@TestSuite('Authentication')
class LoginTests {
@TestCase({ tags: ['@smoke', '@critical'], priority: 'high' })
async validLoginRedirects() { /* ... */ }

@TestCase({ tags: ['@regression'], priority: 'medium' })
async invalidPasswordShowsError() { /* ... */ }
}
```

Q 261 LEVEL · Mid

How do you use discriminated unions for test state management?

CONCEPT

Discriminated unions model **mutually exclusive states**, and TypeScript narrows the type based on the discriminant field, preventing access to fields that do not exist in the current state. A TestState union with statuses pending, running, passed, and failed carries different fields per state, startTime on running, duration on passed, error and trace on failed, and a switch on the discriminant lets the compiler tell you which fields are available in each case. The deeper win is **exhaustiveness checking**: if you later add a **skipped** state to the union, TypeScript immediately flags every switch that does not handle it. This is the same principle as a Record keyed by an enum, applied to state machines, add a state, the compiler tells you every site that must be updated.

INTERVIEW STRATEGY

They are looking to see you use the type system to keep state machines honest. Describe discriminated unions and the narrowing in a switch, then make exhaustiveness the climax, add a state, compiler fan-out tells you every switch to update. That is the same single-source-of-truth pattern as Record-by-enum. The failure mode is using a single broad interface with optional fields, which loses narrowing. The follow-up is often ‘what happens if you add a new state?’, answer the compiler flags every unhandled switch.

How to phrase it

> *Discriminated unions model mutually exclusive states, and TypeScript narrows the type by the discriminant field, so I can only access fields that exist in the current state. For a TestState union with pending, running, passed and failed, each state carries different fields, startTime on running, duration on passed, error and trace on failed, and a switch on the status field lets the compiler tell me, in each case, exactly which fields are safe to read. That is already useful, but the part that earns its place in a framework, and my answer to the follow-up about what happens if I add a new state, is exhaustiveness checking. If I later add a skipped state to the union, TypeScript immediately flags every switch and every handler that does not deal with it. So a state-machine change becomes a compiler-guided edit rather than a hunt for every site that might need updating, which is exactly the same single-source-of-truth idea as a Record keyed by an enum, applied to state machines. The alternative people sometimes reach for, and the trap I would name, is a single broad interface with everything optional, startTime, duration, error all optional. That technically works but you lose narrowing entirely, and you have to defensively check whether duration is defined every time, even when the state guarantees it. Discriminated unions are the cleaner choice for any state-bearing data in the framework.*

Key points to hit

(1) Mutually exclusive states modelled as a union with a discriminant field. **(2)** switch on the discriminant narrows the type to that case's fields. **(3)** Add a new state: compiler flags every unhandled switch. **(4)** Same single-source-of-truth idea as Record-by-enum, for state machines. **(5)** Follow-up: new state -> TS fan-out points at every site to update. **(6)** Trap: a broad interface with all fields optional, loses narrowing.

CODE

```
type TestState =
| { status: 'pending' }
| { status: 'running'; startTime: number }
| { status: 'passed'; duration: number; screenshot?: string }
| { status: 'failed'; duration: number; error: string; trace: string };

function handleTestResult(state: TestState): string {
switch (state.status) {
case 'pending': return 'Waiting to run';
case 'running': return `Running for ${Date.now() - state.startTime}ms`;
case 'passed': return `Passed in ${state.duration}ms`;
case 'failed': return `Failed: ${state.error}`; // TS knows .error exists
}
}
```

Q 262

LEVEL · Mid to Senior

How do you use module augmentation to extend Playwright types?

CONCEPT

Module augmentation extends an existing TypeScript module declaration without modifying the source library. In a Playwright framework you use it to add custom methods to Page and custom

assertions to Matchers, so your helpers appear as first-class members of the Playwright API, fully typed, autocompleted, and integrated with the library's docs. Declare module '@playwright/test' { interface Page { … } and declare global { namespace PlaywrightTest { interface Matchers<R> { … } } } in a single .d.ts file, then provide the runtime implementations (for example via **Page.prototype.loginAs**). The result is the cleanest possible extension surface: **page.loginAs ('admin')** and **expect(page).toBeLoggedIn()** behave exactly like built-ins, with no forking and no awkward wrapper imports.

INTERVIEW STRATEGY

The question is checking whether you extend third-party libraries cleanly. Explain that module augmentation declares additions to @playwright/test in a .d.ts file plus runtime implementations on the prototype, and the headline is that helpers feel first-class, autocomplete, type safety, no fork. That clean-extension framing is the senior signal. The mistake is wrapping Page in a custom class and asking everyone to import that everywhere. Expect the follow-up 'how do tests get the new methods?', answer they work on the ordinary Page, no special import, the augmentation is global.

How to phrase it

> *Module augmentation extends an existing TypeScript module without modifying the source library, which means I can add custom methods to Page and custom assertions to Matchers in a way that feels first-class rather than bolted on. In a .d.ts file I write declare module @playwright/test with an extended Page interface that adds methods like loginAs role and clearAllCookiesAndStorage, and an extended Matchers interface that adds toBeLoggedIn, toHavePermission and toShowValidationError. Then I provide the runtime implementations on Page.prototype, and the same for the matchers via expect.extend. The outcome, and what I would emphasise, is that page.loginAs admin and expect-page-toBeLoggedIn behave exactly like Playwright built-ins, full autocomplete in the IDE, full type safety, integrated with the library's docs surface. That is the cleanest way to extend a third-party library's types. It is also my answer to the follow-up about how tests get the new methods: they work on the ordinary Page object, no special import, because the augmentation is global once the .d.ts file is in the project. The trap I would call out is wrapping Page in a custom class and asking everyone to import that everywhere, which feels right but ends up indirected and verbose. Module augmentation is the framework-design choice that signals senior TypeScript fluency, you extend the platform, you do not wrap around it.*

Key points to hit

(1) Extend @playwright/test's Page/Matchers without modifying the library. **(2)** declare module + runtime impl on Page.prototype / expect.extend. **(3)** Custom methods feel first-class: autocomplete, type safety, no fork. **(4)** Use for page-level utilities and custom matchers (toBeLoggedIn, etc.). **(5)** Expect next: tests use the new methods on ordinary Page, no special import. **(6)** Where this goes wrong: wrapping Page in a custom class everyone must import.

CODE

```
// src/types/playwright.d.ts: extend Playwright's types
import { Page } from '@playwright/test';

declare module '@playwright/test' {
interface Page {
loginAs(role: 'admin' | 'user' | 'viewer'): Promise<void>;
clearAllCookiesAndStorage(): Promise<void>;
}
}
declare global {
namespace PlaywrightTest {
interface Matchers<R> {
toBeLoggedIn(): R;
toHavePermission(permission: string): R;
}
}
}

// Runtime implementation
(Page.prototype as any).loginAs = async function(role: string) {
await this.context().addCookies([/* role-specific cookies */]);
};

// Fully typed usage
await page.loginAs('admin');
await expect(page).toBeLoggedIn();
```

Q 263

LEVEL · Senior

How do generic constraints and infer let you build type-safe test helpers that work for any shape?

CONCEPT

Generic constraints (**T extends Something**) and the **infer** keyword let TypeScript express ‘a function that works for any type meeting these rules and returns a type derived from the input.’ Two patterns earn their place in a senior framework. Constraint with keyof: pick<T, K extends keyof T> guarantees the key exists on T, so test helpers can't be called with typo'd field names. Conditional inference with infer: extract the resolved type from a promise or array, type Awaited<T> = T extends Promise<infer U> ? U : T, so test utilities returning async values give callers the right unwrapped type. Real test use case: **expectFieldEquals(user, 'email', 'x@y.z')** where the third argument's type is inferred from the user object's email field automatically. If user dot email is a string, the third arg must be a string; if user dot age is a number, the third arg must be a number. The compiler catches type mismatches before the test runs. The discipline: add generic constraints to test helpers that take field names or shapes so typos and shape mismatches fail at compile time, not at runtime in a confusing assertion error.

INTERVIEW STRATEGY

They want generic constraints and infer applied to test helpers, not explained abstractly. Lead with both patterns (extends keyof for field safety, infer for type extraction) with the expectFieldEquals example. The 'compile-time over runtime' framing is the senior signal. The thing to avoid is helpers typed as string + any. They usually follow with ‘where does this matter most?’, answer data-driven

tests and assertion helpers that take field names.

How to phrase it

> *Generic constraints with extends and the infer keyword let TypeScript express a function that works for any type meeting these rules and returns a type derived from the input. Two patterns earn their place in a senior framework. Constraint with keyof: pick of T, K extends keyof T guarantees the key exists on T, so test helpers can't be called with typo'd field names. If I write expectFieldEquals user emial, misspelled, the compiler catches it. The helper is generic over the object type and constrained to keys of that type, so misspellings become compile errors. Conditional inference with infer: extract the resolved type from a Promise or array. Type Awaited of T equals T extends Promise of infer U question mark U colon T. The infer keyword introduces a type variable in the conditional position and binds to whatever the actual type is. Test utilities returning async values give callers the right unwrapped type. The real test use case that lands concretely, and the example I'd lead with, is expectFieldEquals user email x at y dot z, where the third argument's type is inferred from the user object's email field automatically. If user dot email is a string, the third arg must be a string; if user dot age is a number, the third arg must be a number. The compiler catches type mismatches before the test runs, preventing the runtime bug where I'd compare a number to a string and the assertion would silently fail in confusing ways. That's also my answer to the follow-up about where it matters most: data-driven tests and assertion helpers that take field names. These are the helpers called dozens of times across the suite, and a typo in a field name produces a test that compiles, runs, and silently passes because it's asserting on a non-existent field. Generic constraints turn that into a compile error. The principle worth ending with is add generic constraints to test helpers that take field names or shapes, so typos and shape mismatches fail at compile time, not at runtime in a confusing assertion error. The trap is helpers typed as string + any: they work for anything, which means they don't enforce anything, which means typos slip through.*

Key points to hit

(1) K extends keyof T: field-name typos become compile errors. **(2)** infer U inside conditional: extract the unwrapped type from Promise/array. **(3)** expectFieldEquals(user, 'email', value): third arg type inferred from field. **(4)** Compiler catches type mismatches before tests run. **(5)** They usually ask next: highest leverage in data-driven tests + assertion helpers. **(6)** What loses the point: helpers typed string + any, enforce nothing, typos slip through.

CODE

```
// Pattern 1: extends keyof, field-name typos fail at compile time
function expectFieldEquals<T, K extends keyof T>(obj: T, key: K, value: T[K]): void {
if (obj[key] !== value) throw new Error(`Expected ${String(key)}=${value}, got
${obj[key]}`);
}

interface User { email: string; age: number; }
const user: User = { email: 'a@b.c', age: 30 };

expectFieldEquals(user, 'email', 'a@b.c'); // OK
expectFieldEquals(user, 'email', 30); // error: number not string
expectFieldEquals(user, 'emial', 'x'); // error: 'emial' not in User keys

// Pattern 2: infer extracts type from a wrapping type
type UnwrapPromise<T> = T extends Promise<infer U> ? U : T;

async function fetchUser(): Promise<User> { /* ... */ return {} as User; }
type Fetched = UnwrapPromise<ReturnType<typeof fetchUser>>; // Fetched = User

// Test helper using infer to give callers the unwrapped type
async function expectResolves<T>(p: Promise<T>): Promise<T> {
const v = await p;
if (v === undefined || v === null) throw new Error('promise resolved to nullish');
return v; // caller sees T, not Promise<T>
}
```

Q 264 LEVEL · Senior

How does the `satisfies` operator help you keep test config and test data both type-safe and inference-rich?

CONCEPT

Before the **satisfies** operator, you had two imperfect options when annotating a config object. Annotate with **: SomeType** and lose narrow literal types (the union of literal strings becomes the wider type); skip the annotation and lose the compile-time check that the shape matches the expected interface. satisfies threads the needle: const config = { … } satisfies PlaywrightTestConfig checks that the object conforms to PlaywrightTestConfig at compile time, but preserves the narrowest inferred types of the literal values. The test-code uses where this matters: environment-specific config maps (test.use values per project), fixture option tables (per-test option dictionaries), and test-data builders that mix typed shape with literal narrowing. Compare with the alternatives: as const freezes the values but doesn't verify the shape; : Type verifies the shape but widens. satisfies does both in one operator. The principle that matters most: reach for satisfies on any object literal where you want both shape-checking AND literal preservation. The pitfall is using : Type on test config and then wondering why **config.projects[0].name** is typed as string rather than the literal "chromium" you wrote, breaking downstream code that switches on the project name.

INTERVIEW STRATEGY

What earns the tick is `satisfies` understood as a real workflow tool, not a syntax curiosity. Lead with the two-bad-options problem (`: Type` widens, no annotation skips checking), then `satisfies` as the threading. The shape-check-plus-literal-preservation framing demonstrates calibration. The pitfall

is `: Type` on configs widening literals downstream. The next question is normally 'when do you NOT use satisfies?', answer when you genuinely want type widening, e.g. a variable meant to hold any of the union members over its lifetime.

How to phrase it

> *Before the satisfies operator, I had two imperfect options when annotating a config object. First option: annotate with colon SomeType and lose narrow literal types, the union of literal strings becomes the wider type. So if I wrote const config colon PlaywrightTestConfig equals open-brace projects colon array of open-brace name colon quote chromium quote close-brace close-brace, the inferred type of config dot projects index zero dot name is string, not the literal quote chromium quote. Anything downstream that switches on the project name loses the exhaustiveness check. Second option: skip the annotation entirely. Then I lose the compile-time check that the shape matches the expected interface. If I mistyped projetcs instead of projects, no error, the object is valid in isolation. satisfies threads the needle. Const config equals open-brace dot dot dot close-brace satisfies PlaywrightTestConfig checks that the object conforms to PlaywrightTestConfig at compile time, but preserves the narrowest inferred types of the literal values. I get both: shape verification and literal preservation. The test-code uses where this matters are three. Environment-specific config maps: dev versus staging versus production config objects where the project names matter as literals downstream. Fixture option tables: per-test option dictionaries that other tests destructure by specific key. Test-data builders that mix typed shape with literal narrowing, the builder enforces the shape, but the literal field values stay narrow for downstream type-narrowing. Compare with the alternatives. As const freezes the values but doesn't verify the shape, a different tool for a different job, useful for deeply-frozen constants but not for validating against an interface. Colon Type verifies the shape but widens. Satisfies does both in one operator. The rule worth leading with is reach for satisfies on any object literal where I want both shape-checking AND literal preservation. The pitfall worth naming is using colon Type on test config and then wondering why config dot projects index zero dot name is typed as string rather than the literal quote chromium quote I wrote, breaking downstream code that switches on the project name. As for the follow-up about when NOT to use satisfies: when I genuinely want type widening. A variable meant to hold any of the union members over its lifetime, like a let binding reassigned across iterations, should be annotated, not satisfies-ed. Satisfies is for one-shot const declarations; mutable bindings still want explicit annotation.*

Key points to hit

(1) satisfies checks shape AND preserves narrow literal inference. **(2)** Colon Type widens literals; no annotation skips shape-checking. **(3)** Uses: env-specific config maps, fixture option tables, narrowing-aware builders. **(4)** Compare with as const (freezes but no shape check). **(5)** Pitfall: colon Type on test config, then literal-typed code downstream breaks. **(6)** Don't use for mutable bindings that need widened type, annotate those.

CODE

```
// BEFORE satisfies: two bad options
// Option A: : Type widens literals
const config1: PlaywrightTestConfig = {
projects: [{ name: 'chromium', use: { browserName: 'chromium' } }],
};
// config1.projects[0].name is now `string`, not `'chromium'`

// Option B: no annotation, no shape check
const config2 = {
projects: [{ name: 'chromium', use: { browserName: 'chromium' } }],
};
// Typo in field name? No error. config2.projetcs is silently undefined.

// satisfies: shape verified, literals preserved
const config3 = {
projects: [
{ name: 'chromium', use: { browserName: 'chromium' } },
{ name: 'firefox', use: { browserName: 'firefox' } },
],
} satisfies PlaywrightTestConfig;

// config3.projects[0].name is `'chromium'`: literal preserved
// Misspelled field would error: 'projetcs' not in PlaywrightTestConfig

// Downstream switch can narrow exhaustively
function isMobileProject(name: typeof config3.projects[number]['name']): boolean {
switch (name) {
case 'chromium': return false;
case 'firefox': return false;
// TS error if literal preserved: missing case
}
}
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What concrete bug do template literal types like `/api/users/${number}` prevent?

A) A wrong-type URL path: caught at compile time instead of as a runtime 404
B) Race conditions in tests
C) Memory leaks
D) Locator strict-mode violations

2. Why use a mapped type like DataGenerator<T> for test data?

A) It stays in sync with the interface: a new field becomes a compile error until a generator is added
B) It makes tests run faster
C) It replaces faker
D) It removes the need for fixtures

3. When are TypeScript decorators the right choice over @tag-in-title + --grep?

A) For every project: they're always better
B) When a custom orchestrator or platform needs to query structured metadata programmatically
C) For simple smoke-test filtering
D) Whenever the suite has more than 10 tests

4. What property of discriminated unions makes them ideal for state machines?

A) They run faster than enums
B) They reduce bundle size
C) They avoid using TypeScript
D) Exhaustiveness: adding a new state makes the compiler flag every unhandled switch

5. What does module augmentation let you do for Playwright?

A) Replace the Playwright runner
B) Bypass the Playwright config
C) Extend Page and Matchers with custom methods/assertions that feel first-class, no fork
D) Disable type checking

Section 22 · Answer key

#	Answer	Why and where to revisit
1	A) A wrong-type URL path: caught at compile time instead of as a runtime 404	fetchUser('/api/users/abc') fails to compile; without the template type, it would only surface as a 404 at runtime. *Revisit Section 22.*
2	A) It stays in sync with the interface: a new field becomes a compile error until a generator is added	Without the mapped type, a newly added required field silently becomes undefined in generated data. *Revisit Section 22.*
3	B) When a custom orchestrator or platform needs to query structured metadata programmatically	For ordinary tag-based selection, title tags + --grep are simpler; decorators earn their place only with programmatic metadata querying. *Revisit Section 22.*
4	D) Exhaustiveness: adding a new state makes the compiler flag every unhandled switch	A switch on the discriminant narrows the type; adding a state fans out compile errors at every site that must handle it. *Revisit Section 22.*
5	C) Extend Page and Matchers with custom methods/assertions that feel first-class, no fork	declare module '@playwright/test' + runtime impl on Page.prototype gives autocomplete and type safety without wrapping Page. *Revisit Section 22.*

SECTION 23

Docker and Containerised Testing

Docker questions test whether you have actually run Playwright in containers or only read about it, because the gotchas, silent Chromium crashes, missing artifacts, oversized images, are the kind you only learn by hitting them. These seven questions cover the official image plus the non-negotiable flags, image-size optimisation with concrete time savings, display handling for headless and headed modes, parallel sharding via Kubernetes Jobs, and the volume mounts that save your test artifacts.

In this section

- Official Playwright image + ipc:host + depends_on service_healthy.
- Multi-stage build + browser pruning: 2GB -> 800MB, 90 seconds saved per run.
- Headless by default; Xvfb for headed; --no-sandbox as a documented last resort.
- Kubernetes Jobs for >20-shard sharding past GitHub Actions' runner limits.
- Volume mounts + --exit-code-from so artifacts survive and CI sees true status.

Q 265 LEVEL · Mid

How do you run Playwright tests in Docker?

CONCEPT

The official Playwright Docker image (mcr.microsoft.com/playwright) ships with every browser binary and OS-level dependency pre-installed, so there is no environment drift between local and CI. The two non-negotiable settings that mark someone who has actually run this in anger: **ipc: host**, because Chromium uses shared memory for inter-process communication and Docker's default /dev/shm is 64MB, too small, which makes Chromium **crash silently**; and **depends_on with service_healthy** on the application service in docker-compose, so Playwright only starts after the app's health check passes, otherwise the first tests fail against a still-starting app. Add a baseURL pointing at the app service name (http://app:3000) so the network resolves correctly inside the compose network.

INTERVIEW STRATEGY

They are quietly checking for real Docker-on-CI scars. State the official image, then make ipc:host and the silent-Chromium-crash story the centrepiece because it is the detail that proves you have lived this. depends_on service_healthy is the second-tier signal that you have debugged race conditions. The thing to avoid is installing browsers manually inside the Dockerfile. A common follow-up is 'what is the ipc:host setting for?', answer that Chromium needs shared memory larger than Docker's default 64MB or it crashes silently.

How to phrase it

> *I use the official Playwright Docker image because it ships every browser binary and OS-level dependency pre-installed, which eliminates the environment drift between local and CI that bites everyone running Selenium in Docker. But the two settings I always emphasise, and the ones that prove I have actually run this rather than read about it, are ipc:host and depends_on with service_healthy. ipc:host answers the follow-up about what it is for. Chromium uses shared memory for inter-process communication, and Docker's default /dev/shm is only 64MB, which is too small for Chromium, so it crashes silently, the test just dies with no clear error, which is brutal to debug the first time you hit it. ipc:host shares the host's IPC namespace and fixes it completely. The second setting is depends_on with service_healthy on the app service in docker-compose: it ensures the application is fully started and passing its health check before Playwright begins, because without it the first few tests fail against an app that is still booting and you get failures that look like flakiness but are actually a race condition with startup. I also set the baseURL to the service name, http-colon-slash-slash-app-colon-3000, so the network resolves correctly inside the compose network. The trap I would call out is installing browsers manually in the Dockerfile, which reintroduces the version-skew problem the official image was designed to remove.*

Key points to hit

(1) Use the official mcr.microsoft.com/playwright image, no manual installs. **(2)** ipc:host fixes Chromium's silent crash from a too-small /dev/shm. **(3)** depends_on service_healthy gates Playwright on the app's health check. **(4)** baseURL = http://app:3000 (the compose service name). **(5)** The next question: ipc:host is for Chromium's shared-memory IPC. **(6)** The common miss: installing browsers manually inside the Dockerfile.

CODE

```
# Dockerfile
FROM mcr.microsoft.com/playwright:v1.45.0-jammy
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npx", "playwright", "test"]

# docker-compose.yml
services:
app:
image: my-app:latest
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
interval: 5s
playwright:
image: mcr.microsoft.com/playwright:v1.45.0-jammy
depends_on:
app: { condition: service_healthy }
environment:
- BASE_URL=http://app:3000
ipc: host # required, prevents silent Chromium crash
```

Q 266 LEVEL · Mid

How do you optimise Docker image size for Playwright?

CONCEPT

The full Playwright image with all three browsers is around **2GB**, which is wasteful when CI only needs one. A multi-stage build isolates the install step from the runtime image, and **removing the Firefox and WebKit binaries** from the image after install brings Chromium-only down to roughly **800MB**, about a 60% size reduction. The framing that lands with stakeholders is the time math: a 90-second pull-time saving per CI run multiplied across 50 daily CI runs is roughly 75 minutes of CI time saved per day. The pattern: run Chromium-only in Docker for speed, run Firefox and WebKit on the host for cross-browser coverage in the nightly suite. Optimisation decisions framed in time and cost are far more persuasive than a percentage size reduction.

INTERVIEW STRATEGY

The point of this one is to see optimisation justified by impact, not cosmetics. Lead with the multi-stage build and the Chromium-only prune to 800MB, then make the time-and-cost calculation the centrepiece, 60% reduction is fine, but 75 minutes of CI time saved per day is what lands. The risk is over-optimising on a low-frequency pipeline where the saving is theoretical. The follow-up is often ‘how do you keep cross-browser coverage?’, answer Chromium in Docker for speed, Firefox/WebKit on the host nightly.

How to phrase it

> *The full Playwright image with all three browsers is around 2GB, and that is wasteful when most of CI only needs one. So I use a multi-stage build to isolate the dependency install from the runtime image, and I remove the Firefox and WebKit binaries from the image after install, which brings Chromium-only down to roughly 800MB, a sixty percent reduction. But the way I would actually pitch this in an interview, and the way I pitch optimisations to stakeholders, is the time math rather than the percentage. A ninety-second pull-time saving per CI run multiplied across fifty daily CI runs is about seventy-five minutes of CI time saved per day. That is a number a stakeholder understands immediately, whereas sixty percent of an image size is meaningless to them. Always frame optimisation in time and cost: it makes the value concrete and gets sign-off fast. For the follow-up about keeping cross-browser coverage, my pattern is to run Chromium-only in Docker for PR speed and run Firefox and WebKit on the host in the nightly suite, so I get fast feedback on every PR without sacrificing cross-browser coverage on the regression. The trap I would name is over-optimising on a low-frequency pipeline, where the saving is theoretical and the multi-stage complexity is not worth it. On a high-frequency pipeline, though, the math compounds quickly and the optimisation pays for itself in the first week.*

Key points to hit

(1) Full image ~2GB; Chromium-only ~800MB: ~60% reduction. **(2)** Multi-stage build + rm -rf /ms-playwright/firefox*/webkit*. **(3)** Pitch in time/cost: 90s saved x 50 CI runs = ~75 min/day. **(4)** Pattern: Chromium in Docker for PRs, all browsers nightly on the host. **(5)** Asked next: keep cross-browser coverage by running all three nightly. **(6)** Avoid: over-optimising a low-frequency pipeline for theoretical gains.

CODE

```
# Multi-stage build: Chromium only
FROM mcr.microsoft.com/playwright:v1.45.0-jammy AS base
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM mcr.microsoft.com/playwright:v1.45.0-jammy AS test
WORKDIR /app
COPY --from=base /app/node_modules ./node_modules
COPY . .
RUN rm -rf /ms-playwright/firefox* /ms-playwright/webkit* # ~2GB -> ~800MB
ENV PLAYWRIGHT_BROWSERS_PATH=/ms-playwright
CMD ["npx", "playwright", "test", "--project=chromium"]
```

Q 267

LEVEL · Junior to Mid

How do you handle browser display in headless Docker environments?

CONCEPT

Docker containers have no display server by default, but Playwright runs headless by default so no display is needed, headless in Docker is the standard case and just works. For **headed mode** in Docker (rare, usually for debugging), **Xvfb** provides a virtual framebuffer you start before the test

process. In CI, enforce headless explicitly via config (headless: !!process.env.CI) so a forgotten --headed flag locally does not break the pipeline. The **--no-sandbox** flag is sometimes needed when the container runs as root, because Chromium's sandbox requires kernel capabilities that may be missing, but treat it as a documented last resort, since the sandbox is a real security feature, and prefer running the container as a non-root user with the sandbox enabled.

INTERVIEW STRATEGY

You are being asked to show you treat security flags as deliberate, not reflexive. Lead with the good news, headless is the default and just works in Docker, then cover headed via Xvfb for completeness. The senior signal is the --no-sandbox stance: a last resort, documented when used, with a preference for fixing the container as non-root instead. The mistake is sprinkling --no-sandbox in every config because Stack Overflow said to. Expect the follow-up 'when is --no-sandbox actually needed?', answer when the container runs as root and you cannot change that easily.

How to phrase it

> *Docker containers have no display server by default, but Playwright runs headless by default so for normal CI usage there is nothing to configure, headless in Docker just works. For headed mode in Docker, which I only use when actively debugging something I cannot reproduce headless, I install Xvfb in the image and start it before the test process with a display on :99. In CI I enforce headless explicitly in the config, headless equals double-bang process.env.CI, so a developer running --headed locally cannot accidentally push a config that breaks the pipeline. The part I would spend my real time on is the --no-sandbox question. It is sometimes needed when the container runs as root, because Chromium's sandbox requires specific Linux kernel capabilities that the container may not have. That is also my answer to the follow-up about when it is actually needed. But I treat --no-sandbox as a documented last resort, not a default, because the sandbox is a real security feature, it is what stops a compromised renderer process from doing damage. My preference is to run the container as a non-root user, which lets me keep the sandbox enabled and avoid the flag entirely. When I do have to use it, I leave a comment in the Dockerfile explaining why, so the next person doesn't remove or duplicate it blindly. The risk is sprinkling --no-sandbox everywhere because a tutorial mentioned it, deliberate, documented, minimal is the right posture.*

Key points to hit

(1) Headless is the default: no display needed in Docker for normal CI. **(2)** Xvfb provides a virtual display for the rare headed-in-Docker case. **(3)** Enforce headless in CI via config (headless: !!process.env.CI). **(4)** --no-sandbox is a last resort; prefer running as non-root with sandbox on. **(5)** Follow-up: --no-sandbox needed when container runs as root and you can't change that. **(6)** The easy mistake: copy-pasting --no-sandbox everywhere from tutorials.

CODE

```
# Headless: default, no display configuration needed
npx playwright test

# Headed in Docker via Xvfb (debugging only)
RUN apt-get update && apt-get install -y xvfb
CMD ["sh", "-c", "Xvfb :99 -screen 0 1280x720x24 & DISPLAY=:99 npx playwright test
--headed"]

// playwright.config.ts: enforce headless in CI
export default defineConfig({
use: {
headless: !!process.env.CI,
launchOptions: { args: ['--no-sandbox'] }, // documented last resort
},
});
```

Q 268

LEVEL · Mid to Senior

How do you implement parallel Docker execution for Playwright sharding?

CONCEPT

Beyond GitHub Actions' matrix sharding, **Kubernetes Jobs** are the enterprise-scale solution: each pod runs one shard independently, completions and parallelism set to the shard count so they start simultaneously, and Kubernetes handles pod scheduling, failure recovery, and artifact collection. The shard index comes from the built-in **batch.kubernetes.io/job-completion-index** annotation and feeds the **--shard=N/total** flag. The advantage over a CI runner matrix: Kubernetes can scale to 20 or more shards without hitting the runner limits that constrain GitHub Actions, and pod failure does not kill the whole run. Reach for this pattern only when the suite is over roughly 1,000 tests and a four-shard matrix is no longer enough; for smaller suites it is overkill.

INTERVIEW STRATEGY

What scores here is showing you scale beyond a single CI runner without over-engineering. Explain Kubernetes Jobs with completions and parallelism, then make the boundary explicit: over 1,000 tests with GitHub Actions limits as the trigger, not your default. That calibration is the seniority marker. The thing to avoid is reaching for Kubernetes for a 200-test suite. They usually follow with ‘when is GitHub Actions matrix not enough?’, answer when you need more shards than the runner limit allows, typically past 20.

How to phrase it

> *For Playwright sharding beyond what a GitHub Actions matrix can do, I use Kubernetes Jobs, and the framing matters because this is enterprise-scale infrastructure and easy to over-reach for. The pattern is: completions and parallelism both set to the shard count, say four, so all four pods start simultaneously; each pod runs one shard independently using the --shard equals N over total flag; and the shard index comes from the built-in batch.kubernetes.io/job-completion-index annotation, which Kubernetes fills in automatically per pod. Kubernetes handles pod scheduling, failure recovery, and the result collection across pods, so a single pod crash does not kill the whole run. The reason I reach for this over a CI runner matrix, and my answer to the follow-up about when GitHub Actions matrix is not enough, is the runner limit: Actions caps matrix size, and once I genuinely need twenty or more shards, Kubernetes is the path forward. But the framing I would emphasise, to show I am not reflexively over-engineering, is that I only reach for this when the suite is over roughly a thousand tests and a four-shard matrix is no longer enough, for smaller suites it is overkill, and the operational cost of running on Kubernetes is not worth it. Where this goes wrong is using a Kubernetes Job for a two-hundred-test suite because it sounds impressive; the right answer most of the time is the simpler matrix in CI, and Kubernetes is the escalation when scale demands it.*

Key points to hit

(1) Kubernetes Jobs: completions=parallelism=shard count, all pods start together. **(2)** Shard index from batch.kubernetes.io/job-completion-index annotation. **(3)** K8s handles scheduling, recovery, artifact collection across pods. **(4)** Use when GitHub Actions matrix runner limit is hit (>20 shards / >1k tests). **(5)** Expect next: GH matrix is enough until runner limit, K8s is the escalation. **(6)** Trap: using K8s for a 200-test suite, over-engineering.

CODE

```
apiVersion: batch/v1
kind: Job
metadata: { name: playwright-tests }
spec:
completions: 4
parallelism: 4 # all 4 pods run simultaneously
template:
spec:
restartPolicy: Never
containers:
- name: playwright
image: my-playwright:latest
env:
- name: SHARD_INDEX
valueFrom:
fieldRef:
fieldPath: metadata.annotations['batch.kubernetes.io/job-completion-index']
- name: SHARD_TOTAL
value: "4"
command: ["npx", "playwright", "test", "--shard=$(SHARD_INDEX)/$(SHARD_TOTAL)"]
```

Q 269 LEVEL · Mid

How do you collect and merge test artifacts from Docker containers?

CONCEPT

Test artifacts, HTML reports, traces, screenshots, disappear when a container exits unless you mount the directories that hold them. Mount three: **test-results** for raw output, **blob-report** for the mergeable format, and playwright-report for the final HTML. After all containers complete, a one-shot playwright merge-reports run combines the per-shard blob reports into a single HTML report for stakeholders. The second non-obvious detail is **--exit-code-from playwright** on docker-compose up: it makes the compose command exit with the Playwright container's exit code, so CI correctly marks the step as failed when tests fail. Without this flag, docker-compose always exits 0 even when tests fail, which is exactly the kind of silent green that hides real regressions.

INTERVIEW STRATEGY

They are checking for the gotchas that only show up in production CI use. Lead with volume mounts for the three artifact dirs, then make --exit-code-from the punchline because it explains a subtle CI failure mode: docker-compose defaults to exit 0 on success of the compose command, not on the test container's exit code. That silent-green-on-failure detail is the test of seniority on this. Where this goes wrong is forgetting to mount artifact dirs and losing every trace. The next question is normally 'why does CI not catch the failure without that flag?', answer docker-compose exits 0 unless told to use the container's exit code.

How to phrase it

> *Test artifacts disappear when a container exits, so the first thing I do is mount the directories that hold them. I mount three: test-results for the raw output, blob-report for the mergeable per-shard format, and playwright-report for the final HTML. Without these mounts every trace and screenshot is gone the moment the container stops, which is brutal for debugging a failed CI run. After all the shard containers complete, I run a one-shot playwright merge-reports to combine the per-shard blob reports into a single HTML report, stakeholders see one unified result, not four fragments. The detail I emphasise because it catches everyone the first time, and my answer to the follow-up about why CI does not catch a failure without it, is the --exit-code-from playwright flag on docker-compose up. Without this flag, docker-compose always exits zero even when tests inside the playwright container failed, because exit zero just means the compose command itself ran successfully. So CI marks the step green and you ship a broken build, which is the worst possible failure mode, silent green on real red. With --exit-code-from playwright, docker-compose exits with the Playwright container's actual exit code, so a test failure correctly fails the CI step. The pitfall is forgetting either the volume mounts, in which case you lose all diagnostics, or the --exit-code-from flag, in which case you lose the failure signal itself. Both have to be right or the Docker integration is worse than running on the host.*

Key points to hit

(1) Mount test-results, blob-report, playwright-report, or artifacts vanish. **(2)** Use playwright merge-reports to combine per-shard blob reports into one HTML. **(3)** --exit-code-from playwright on compose up surfaces the container's exit code. **(4)** Without it, docker-compose exits 0 on failure: CI marks green incorrectly. **(5)** They usually ask next: CI misses failures because compose exits 0 by default. **(6)** Where this goes wrong: missing mounts (lose artifacts) or missing flag (lose failure signal).

CODE

```
# docker-compose.yml: mount artifact dirs
services:
playwright:
image: my-playwright:latest
volumes:
- ./test-results:/app/test-results
- ./playwright-report:/app/playwright-report
- ./blob-report:/app/blob-report

# Run + propagate the Playwright container's exit code (else CI thinks 0)
docker-compose up --exit-code-from playwright playwright

# Merge per-shard blob reports into one HTML
docker run --rm -v $(pwd)/blob-reports:/blob-reports -v
$(pwd)/merged-report:/merged-report \
mcr.microsoft.com/playwright:v1.45.0-jammy \
npx playwright merge-reports --reporter html /blob-reports
```

Q 270

LEVEL · Senior

How do you set up Docker networking for a Playwright suite that tests a multi-service app?

CONCEPT

When the app under test is multi-service, API backend, database, queue, cache, payment mock, the Playwright container needs to reach all of them by stable name, not by IP. Three patterns depending on orchestration. **docker-compose**: all services join the same bridge network by default, with DNS resolution by service name. Playwright reaches the API at **http://api:3000**, the API reaches Postgres at postgres:5432, no IP gymnastics. The compose file is the network topology. Custom networks: explicit named networks let you isolate groups of services, a frontend network for the app and Playwright, a backend network for database and queue. Useful when Playwright shouldn't have direct database access. host network mode: **--network host** bypasses container networking entirely, the container shares the host's network namespace. Faster, but loses the isolation containerisation provides; use only when specific tools require it. The discipline that saves time: name services consistently and use those names as URLs in the test config. BASE_URL=http://app:3000 in the Playwright service's environment matches the app service name in compose. No hard-coded IPs. The trap: localhost in a container, localhost inside a container is the container itself, not the host, so every URL like http://localhost:3000 fails. Always reach other services by their service name, never by localhost.

INTERVIEW STRATEGY

You need to give Docker networking operationally understood. Walk three patterns (compose default bridge, custom networks, host mode) with when each applies. The 'service names as URLs' rule is the seniority marker. The classic error is localhost-in-a-container. A common follow-up is ‘how do you debug network issues?’, answer docker-compose exec into the container, then ping or curl the service name to verify resolution.

How to phrase it

> *When the app under test is multi-service, API backend, database, queue, cache, payment mock, the Playwright container needs to reach all of them by stable name, not by IP, because IPs change across container restarts and environments. Three patterns depending on orchestration. First, docker-compose. All services declared in the same compose file join the same bridge network by default, with DNS resolution by service name. Playwright reaches the API at http colon slash slash api colon three-thousand, the API reaches Postgres at postgres colon five-four-three-two, no IP gymnastics. The compose file is the network topology, which makes the setup self-documenting. Second, custom networks. Explicit named networks let me isolate groups of services, a frontend network for the app and Playwright, a backend network for database and queue. Useful when Playwright shouldn't have direct database access, for example, when I want to enforce that tests can only verify state through the API, not by querying the database, so the tests reflect what users actually see. Third, host network mode. Dash dash network host bypasses container networking entirely; the container shares the host's network namespace. Faster because there's no bridge overhead, but loses the isolation containerisation provides. Use only when specific tools require it: some debugging or performance-measurement tools want direct host network access. The discipline that saves time, and the framing I would lead with, is name services consistently and use those names as URLs in the test config. BASE_URL equals http colon slash slash app colon three-thousand in the Playwright service's environment matches the app service name in compose. No hard-coded IPs, no container ID lookups, just service names that the docker DNS resolves automatically. The trap, and where every Docker-newcomer trips first, is localhost in a container. Localhost inside a container is the container itself, not the host machine, so every URL like http colon slash slash localhost colon three-thousand fails because the API isn't running inside the Playwright container. Always reach other services by their service name, never by localhost. That's also my answer to the follow-up about debugging network issues: docker-compose exec into the container, then ping or curl the service name to verify resolution. If curl http colon slash slash api colon three-thousand works from inside the Playwright container, networking is fine; if it doesn't, the service isn't running or isn't named correctly in compose.*

Key points to hit

(1) Compose default bridge network: services reach each other by service name. **(2)** Custom networks: isolate groups (frontend net, backend net) when needed. **(3)** Host network mode: fast, no isolation, only when tools require it. **(4)** Service names as URLs in config: no hard-coded IPs, no container IDs. **(5)** The next question: debug via docker-compose exec + curl/ping the service name. **(6)** What loses the point: localhost in a container = the container itself, not the host.

CODE

```
# docker-compose.yml: multi-service test setup
version: '3.8'
services:
app:
build: .
ports: ['3000:3000']
environment:
DATABASE_URL: postgres://postgres:postgres@db:5432/test
QUEUE_URL: redis://redis:6379
depends_on: [db, redis]

db:
image: postgres:17
environment: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: test }

redis:
image: redis:7-alpine

playwright:
image: mcr.microsoft.com/playwright:v1.45.0-jammy
working_dir: /work
volumes: ['.:/work']
environment:
BASE_URL: http://app:3000 # service name, NOT localhost
DATABASE_URL: postgres://postgres:postgres@db:5432/test
depends_on: [app]
command: npx playwright test

# Debug network resolution from inside the Playwright container
# docker-compose exec playwright sh -c 'curl -sf http://app:3000/health'
```

Q 271

LEVEL · Senior

How do you use BuildKit and layer caching to keep Playwright Docker images fast to build in CI?

CONCEPT

Image build time is wasted CI minutes on every commit. Three techniques that compound. **BuildKit**: Docker's modern builder, enabled via **DOCKER_BUILDKIT=1** or docker buildx. Faster than the classic builder, supports parallel layer builds, and introduces cache mounts. Layer caching order: put rarely-changing operations early in the Dockerfile so they cache across builds. COPY package.json package-lock.json before COPY . . means npm install only re-runs when lockfiles change, not on every code change. **Cache mounts with BuildKit**: RUN --mount=type=cache,target=/root/.npm npm install persists the npm cache across builds even when the layer rebuilds, dramatically reducing install time. Registry-based caching in CI: GitHub Actions cache, Docker Hub registry, or buildx --cache-to mode=max push the cache to a registry so the next CI run starts warm. The discipline: order Dockerfile instructions from least to most frequently changing. Base image first, package install second, source code last. Layer cache invalidation cascades downwards, so the early layers staying stable is what makes the build fast. The trap: COPY . . near the top: a single code change invalidates everything after, making every build a full rebuild.

INTERVIEW STRATEGY

They are after Docker build optimisation as a CI-cost question, not a theoretical one. Three techniques: BuildKit, layer ordering, cache mounts + registry cache. The 'order from least to most frequently changing' rule is the senior signal. The failure mode is COPY . . early. The follow-up is often ‘how much does this actually save?’, answer minutes per build, multiplied across every PR, hours per week.

How to phrase it

> *Image build time is wasted CI minutes on every commit, and a slow Playwright image build costs the team real time, every PR pipeline pays the same cost. Three techniques that compound. First, BuildKit. Docker's modern builder, enabled via DOCKER underscore BUILDKIT equals one or docker buildx. Faster than the classic builder, supports parallel layer builds, and introduces cache mounts that the classic builder doesn't have. Turning BuildKit on is essentially free, same Dockerfile, faster build, and yet I see teams still using the classic builder by default. Second, layer caching order. Put rarely-changing operations early in the Dockerfile so they cache across builds. COPY package dot json package-lock dot json before COPY dot dot means npm install only re-runs when lockfiles change, not on every code change. This is the single highest-leverage technique: a typical Playwright image install takes a couple of minutes; caching it correctly means those couple of minutes only happen when dependencies actually change, maybe once a week. Third, cache mounts with BuildKit. RUN dash dash mount equals type equals cache target equals slash root slash dot npm npm install persists the npm cache across builds even when the layer rebuilds, dramatically reducing install time for the case where the layer does have to rebuild. Combined with registry-based caching in CI, GitHub Actions cache, Docker Hub registry, or buildx dash dash cache dash to mode equals max push the cache to a registry so the next CI run starts warm, the cache survives even across runner instances. The discipline I would close on, because it's the rule that captures the whole approach, is order Dockerfile instructions from least to most frequently changing. Base image first, system packages second, language deps third, source code last. Layer cache invalidation cascades downwards, so the early layers staying stable is what makes the build fast. The risk is COPY dot dot near the top: a single code change invalidates everything after, making every build a full rebuild. That's also my answer to the follow-up about how much this saves: minutes per build, multiplied across every PR. Hours per week in CI time, real money in compute cost, and faster feedback for developers.*

Key points to hit

(1) BuildKit: enable via DOCKER_BUILDKIT=1 or buildx, parallel layers + cache mounts. **(2)** Layer ordering: COPY package*.json + install BEFORE COPY . . **(3)** Cache mounts: RUN --mount=type=cache,target=/root/.npm, persists across rebuilds. **(4)** Registry caching: cache-to/cache-from in CI, warm starts across runners. **(5)** Rule: order from least to most frequently changing, cache invalidation cascades down. **(6)** The common miss: COPY . . near the top: any code change rebuilds everything after.

CODE

```
# Dockerfile: optimised for layer caching
# syntax=docker/dockerfile:1.4 <- enables BuildKit features
FROM mcr.microsoft.com/playwright:v1.45.0-jammy

WORKDIR /work

# Step 1: copy only dependency manifests, stable, cached aggressively
COPY package.json package-lock.json ./

# Step 2: install deps with cache mount, cache survives layer rebuilds
RUN --mount=type=cache,target=/root/.npm \
npm ci --prefer-offline --no-audit

# Step 3: now copy the source, changes here don't invalidate the install layer
COPY . .

CMD ["npx", "playwright", "test"]

# CI build with registry caching (GitHub Actions example)
# docker buildx build \
# --cache-from type=registry,ref=ghcr.io/org/pw-test:cache \
# --cache-to type=registry,ref=ghcr.io/org/pw-test:cache,mode=max \
# -t ghcr.io/org/pw-test:${{ github.sha }} .
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What does ipc:host fix when running Playwright in Docker?

A) Network routing between containers
B) Browser binary downloads
C) Chromium's silent crash from a too-small /dev/shm (default 64MB)
D) Headed-mode display

2. What is the realistic Chromium-only image size after multi-stage build and browser pruning?

A) ~800MB, about 60% reduction; ~90s saved per CI pull
B) ~2GB, same as full image
C) ~50MB, alpine-minimal
D) ~5GB, with extra layers

3. How should --no-sandbox be treated in a Docker Playwright setup?

A) Always on by default
B) A documented last resort; prefer non-root container with sandbox enabled
C) Never used under any circumstance
D) Required for headless mode

4. When is a Kubernetes Job the right choice over a GitHub Actions matrix?

A) Any suite over 50 tests
B) When the suite exceeds ~1,000 tests and >20 shards hits GitHub Actions runner limits
C) When using a single browser
D) Always: GH Actions is unreliable

5. Why is --exit-code-from playwright critical on docker-compose up?

A) It speeds up tests
B) It mounts artifact directories
C) It enables Chromium
D) Without it, docker-compose exits 0 even when tests fail, CI marks the step green incorrectly

Section 23 · Answer key

#	Answer	Why and where to revisit
1	C) Chromium's silent crash from a too-small /dev/shm (default 64MB)	Chromium uses shared memory for IPC; Docker's default 64MB /dev/shm is too small, ipc:host shares the host's IPC namespace. *Revisit Section 23.*
2	A) ~800MB, about 60% reduction; ~90s saved per CI pull	rm -rf the firefox/webkit binaries from the official image brings it from ~2GB to ~800MB; pitch the saving as time/cost, not %. *Revisit Section 23.*
3	B) A documented last resort; prefer non-root container with sandbox enabled	The sandbox is a real security feature; --no-sandbox is for environments where the container must run as root. *Revisit Section 23.*
4	B) When the suite exceeds ~1,000 tests and >20 shards hits GitHub Actions runner limits	K8s Jobs scale to many shards with pod recovery and artifact collection; for smaller suites the matrix is enough and simpler. *Revisit Section 23.*
5	D) Without it, docker-compose exits 0 even when tests fail, CI marks the step green incorrectly	The flag propagates the test container's exit code; otherwise compose's own success masks test failures. *Revisit Section 23.*

SECTION 24

Database and SQL Testing

Database questions test whether you can verify that the UI's claims are actually true at the data layer, a higher-confidence test pattern than anything pure UI can give you. These seven questions cover injecting a database client via a fixture, idempotent seeding, transaction rollback, migration verification in CI, and the three isolation strategies that keep parallel tests from fighting over rows.

In this section

- DB validation as a fixture: UI says success, DB confirms persistence.
- Idempotent seeding: ON CONFLICT DO NOTHING + scoped DELETE patterns.
- Transaction rollback tests catch partial-write data-integrity bugs.
- Migration tests: schema correctness + existing-data preservation.
- Parallel isolation: unique data (default), schema-per-worker, transaction rollback.

Q 272 LEVEL · Mid

How do you integrate database validation into Playwright tests?

CONCEPT

Use a **fixture** to inject a database client (pg.Pool for Postgres, etc.) into tests, so a test can query the database **directly after UI interactions**. The pool is shared across tests, do not close per test, and the fixture just hands it to use. The pattern: perform the UI action, assert the UI outcome, then query the DB and assert the row exists with the expected fields and the right values. This is the **highest-confidence test pattern**: the UI says the registration succeeded, the database confirms the data was actually persisted with the right role and required fields. It catches bugs the UI alone hides: a registration that displayed success but silently failed to write, or saved with the wrong role. Always include a cleanup query at the end, or test data accumulates and trips unique-constraint violations in parallel runs.

INTERVIEW STRATEGY

This question is there to see you verify persistence, not just rendering. Frame DB validation as the highest-confidence pattern, give a war story of a UI-says-success-DB-says-nothing bug, and call out the cleanup query so test data does not poison parallel runs. The trap is closing the pool per test: it kills connections needlessly. Expect the follow-up 'why is this stronger than UI assertion alone?', answer because the UI can show success while the write failed silently, and only the DB query catches that.

How to phrase it

> *I inject a database client through a fixture, a pg.Pool for Postgres, for instance, and the fixture hands it to use without closing per test, because the pool is shared. A test then performs its UI action, asserts the UI outcome, and queries the database directly to assert the row exists with the right fields and values. I would frame this as the highest-confidence test pattern I write, and my answer to the follow-up about why it is stronger than asserting on the UI alone is concrete: the UI can show a success banner while the database write failed silently, and only a DB query catches that. I have seen exactly that in production: a registration that flashed a green confirmation and then nothing was persisted, and the bug would have been invisible to any UI-only test. I have also caught data saved with the wrong role or with required fields missing, which are equally invisible from the front end. One detail I always include is a cleanup query at the end of the test deleting the row it created, because without it test data accumulates over time and eventually trips unique-constraint violations in parallel runs, which looks like flakiness but is really compound test-data pollution. The trap I would name is closing the connection pool per test, the pool is meant to be shared across tests, and closing it churns connections for no reason. So: shared pool via fixture, UI plus DB assertions per test, scoped cleanup at the end. That combination is what gives the test real teeth.*

Key points to hit

(1) Fixture injects a shared DB client (pg.Pool): don't close per test. **(2)** Pattern: UI action -> assert UI outcome -> query DB -> assert row + values. **(3)** Highest-confidence pattern: catches silent-write-failure bugs. **(4)** Also catches wrong-role saves and missing required fields. **(5)** Asked next: stronger than UI alone because UI can lie about persistence. **(6)** Avoid: closing the pool per test (and forgetting to clean up test rows).

CODE

```
// src/fixtures/db.fixture.ts
import { test as base } from '@playwright/test';
import { Pool } from 'pg';
const pool = new Pool({
host: process.env.DB_HOST, port: Number(process.env.DB_PORT) || 5432,
database: process.env.DB_NAME, user: process.env.DB_USER, password:
process.env.DB_PASSWORD,
});
export const test = base.extend<{ db: Pool }>({
db: async ({}, use) => { await use(pool); }, // shared, do not close per test
});

test('registration persists to the database', async ({ page, db }) => {
const email = `test-${Date.now()}@example.com`;
await page.goto('/register');
await page.getByLabel('Email').fill(email);
await page.getByLabel('Password').fill('SecurePass@123');
await page.getByRole('button', { name: 'Create Account' }).click();
await expect(page).toHaveURL('/dashboard');

const result = await db.query('SELECT id, email, role FROM users WHERE email = $1',
[email]);
expect(result.rows).toHaveLength(1);
expect(result.rows[0].role).toBe('user');
expect(result.rows[0]).not.toHaveProperty('password');

await db.query('DELETE FROM users WHERE email = $1', [email]); // cleanup
});
```

Q 273 LEVEL · Mid

How do you use database seeding for test data setup?

CONCEPT

Database seeding creates known test data before tests run, and **globalSetup** is the right place for seed data every test depends on. Two patterns make it safe. ON CONFLICT DO NOTHING for reference data and standard test users makes the seed idempotent, if globalSetup runs twice, which can happen in some CI configurations, the seed data is not duplicated. **Scoped DELETE-before-INSERT** for test-specific data ensures a clean state at the start of every run. The discipline rule: use an email pattern like **%@test.com** to identify test data, so cleanup queries are safe and specific. Never run DELETE FROM users without a WHERE clause in a shared environment, a single careless seed can wipe out other engineers' work.

INTERVIEW STRATEGY

They want idempotency and a safety mindset. Lead with ON CONFLICT DO NOTHING for idempotent seeding, then the scoped DELETE-before-INSERT pattern, and close on the never-DELETE-without-WHERE rule as the operational safety principle. That safety-first framing is the seniority marker. The mistake is an unscoped DELETE that could nuke real data. They usually follow with ‘what happens if globalSetup runs twice?’, answer ON CONFLICT keeps the seed safe and

idempotent.

How to phrase it

> *Seeding creates the known test data tests depend on, and globalSetup is where I put it. Two patterns make it safe, and I would frame the answer around them rather than just describe insert statements. The first is ON CONFLICT DO NOTHING for reference data and standard test users, product categories, an admin, a user, a viewer, which makes the seed idempotent. That answers the follow-up about what happens if globalSetup runs twice, which can happen in some CI configurations: the seed data is not duplicated because ON CONFLICT tells the database to leave the existing row alone. Without that, a second run would either error or create duplicates, both of which poison the suite. The second pattern is a scoped DELETE before INSERT for test-specific data, so I start every run from a known clean state. The operational rule I always state, because it has saved me from real disasters, is to use an email pattern like percent-at-test.com to identify test data, so my cleanup queries are safe and specific, DELETE FROM users WHERE email LIKE percent-at-test.com. And I never, ever run DELETE FROM users without a WHERE clause in a shared environment. The mistake is exactly that: an unscoped DELETE in a seed script could wipe out other engineers' work in seconds, and you cannot get it back. So: ON CONFLICT for idempotency, scoped DELETE for clean state, identifiable test-data patterns, and never an unscoped destructive query. That keeps seeding both repeatable and safe.*

Key points to hit

(1) globalSetup is the right home for data every test depends on. **(2)** ON CONFLICT DO NOTHING makes the seed idempotent across re-runs. **(3)** Scoped DELETE-before-INSERT for test-specific data, known clean state. **(4)** Use %@test.com email pattern so cleanup queries are safe and specific. **(5)** Follow-up: globalSetup runs twice, ON CONFLICT keeps it safe. **(6)** The easy mistake: DELETE FROM users without a WHERE clause in a shared env.

CODE

```
// src/setup/global.setup.ts
import { Pool } from 'pg';
export default async function globalSetup() {
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
try {
// Scoped DELETE: only test-prefixed rows
await pool.query("DELETE FROM users WHERE email LIKE '%@test.com'");
await pool.query("DELETE FROM orders WHERE created_by LIKE '%@test.com'");

// Idempotent reference data
await pool.query(`
INSERT INTO product_categories (id, name, slug) VALUES
(1, 'Electronics', 'electronics'),
(2, 'Clothing', 'clothing'),
(3, 'Books', 'books')
ON CONFLICT (id) DO NOTHING
`);

// Idempotent test users
await pool.query(`
INSERT INTO users (email, password_hash, role) VALUES
('admin@test.com', '$2b$10$hashed', 'admin'),
('user@test.com', '$2b$10$hashed', 'user'),
('viewer@test.com', '$2b$10$hashed', 'viewer')
ON CONFLICT (email) DO NOTHING
`);
} finally { await pool.end(); }
}
```

Q 274 LEVEL · Mid to Senior

How do you test database transactions and rollback behaviour?

CONCEPT

Transaction tests verify that multi-step operations are **atomic**, either all steps succeed or all are rolled back. This is **critical for financial and inventory operations**, where a partial write produces orphaned data and accounting nightmares. The pattern: force the second step to fail (mock the payment API to return 402), trigger the operation, assert the UI shows the error, and then **assert the database is unchanged**, the order count before equals the order count after. That confirms the transaction was rolled back completely instead of leaving an orphaned order record with no payment. These tests are some of the highest-value DB tests you will write, they catch the bug class that produces partial orders, double-charged customers, and customer-service incidents that take weeks to clean up.

INTERVIEW STRATEGY

They want to see you test atomicity, not just happy paths. Make the pattern crisp: mock the dependency to fail, assert the UI error, assert the DB count is unchanged. Anchor on a real bug class, partial orders or double-charges, because that is what makes this test class so valuable. The mistake is asserting only on the UI error and not on the DB state. The next question is normally 'which operations need this most?', answer financial and inventory flows, where a partial write is a real data-integrity bug.

How to phrase it

> *Transaction tests verify that multi-step operations are atomic, either every step succeeds or every step is rolled back, and they are critical for financial and inventory operations because a partial write there is a real data-integrity bug. The pattern is: force one step to fail, usually by mocking the second API to fail, a payment returning 402 insufficient funds, trigger the operation through the UI, assert the UI shows the error, and then assert the database is unchanged. That is the part that separates this from a UI-only assertion: I count the order rows before and after, and they must be equal, which confirms the transaction was rolled back completely instead of leaving an orphaned order with no payment. The follow-up is usually about which operations most need this, and the answer is financial and inventory, payment processing, order placement, stock decrements, because a partial write there produces the bugs that cause real customer pain. I have caught two transaction bugs with this pattern, both of which would have left orphaned order records in production and triggered weeks of customer-service cleanup. The trap I would name is asserting only on the UI error banner: the UI can correctly show insufficient funds while the backend still half-committed the order, and you only catch that with the database assertion. So the UI assertion proves the user-facing behaviour, the database assertion proves the integrity, and you need both. Transaction rollback tests are some of the highest-value database tests I write, full stop.*

Key points to hit

(1) Verify multi-step operations are atomic: all-or-nothing. **(2)** Pattern: mock the dependency to fail, assert UI error, assert DB unchanged. **(3)** Critical for financial/inventory: partial writes are real data bugs. **(4)** Compare row counts before and after to confirm rollback. **(5)** Expect next: most-needed for payments, orders, inventory decrements. **(6)** Trap: asserting only on the UI error and trusting the rollback.

CODE

```
test('failed payment rolls back order creation', async ({ page, db }) => {
await page.route('**/api/payments', route => route.fulfill({
status: 402, body: JSON.stringify({ error: 'Insufficient funds' }),
}));

const before = (await db.query('SELECT COUNT(*) FROM orders')).rows[0].count;

await page.goto('/checkout');
await page.getByRole('button', { name: 'Place Order' }).click();
await expect(page.locator('.error-banner')).toContainText('Insufficient funds');

const after = (await db.query('SELECT COUNT(*) FROM orders')).rows[0].count;
expect(after).toBe(before); // rollback confirmed
});
```

Q 275 LEVEL · Mid to Senior

How do you test database migrations in a CI pipeline?

CONCEPT

Migration tests run as a separate CI stage immediately after the migration is applied, and they verify two things: the **schema change is correct** (column exists with the right type and constraints, queried from information_schema) and **existing data is preserved** (no corruption, correct null handling for newly nullable columns). They catch real migration bugs: a column accidentally changed from NOT NULL to nullable, a data transformation that dropped rows, a default value that did not apply correctly. These tests are fast and cheap to write but prevent expensive data-integrity incidents downstream, which is the value proposition that gets them added to the pipeline.

INTERVIEW STRATEGY

Your job here is to show you treat schema changes as code that needs tests. Frame two distinct checks, schema correctness via information_schema, data preservation via row queries, and stress that the stage runs right after migration apply. The cheap-to-write-but-prevents-expensive-incidents framing is the value argument. The mistake is assuming the migration tool's success is verification enough. A common follow-up is 'what real bug has this caught?', answer a column accidentally made nullable, surfaced before staging.

How to phrase it

> *Migration tests run as a separate stage in CI immediately after the migration is applied, and they verify two distinct things, which is the structure I would walk through. First, the schema change is correct: I query information_schema dot columns to confirm the new column exists with the right data type and nullability. Second, the existing data is preserved: I query specific test users created during seeding and assert that their fields are intact and that any newly nullable column is null on existing rows, which is the correct behaviour for a nullable add. That answers the follow-up about a real bug this caught: I once had a migration that accidentally changed a NOT NULL column to nullable, which the migration tool happily applied without error, and the schema test caught it before it reached staging. Without the test, that would have shipped, and downstream code expecting non-null values would have started returning the wrong results or throwing in production. The framing I would close on, and the one that gets these tests added to the pipeline, is the value argument: migration tests are fast and cheap to write, a few queries against information_schema and the seeded data, but they prevent expensive data-integrity incidents that take days to clean up in production. The mistake is assuming the migration tool's success means the migration is correct; success just means it ran, not that it produced the schema you expected. Verifying the schema explicitly is what closes that gap.*

Key points to hit

(1) Run migration tests as a separate CI stage right after migration apply. **(2)** Verify two things: schema correctness + existing-data preservation. **(3)** Schema check: information_schema for column existence, type, nullability. **(4)** Data check: query seeded rows, assert fields intact and nulls handled. **(5)** They usually ask next: caught NOT NULL -> nullable before staging. **(6)** Where this goes wrong: assuming migration-tool success means the migration is correct.

CODE

```
test('migration adds phone_number column to users', async ({ db }) => {
const r = await db.query(`
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'users' AND column_name = 'phone_number'
`);
expect(r.rows).toHaveLength(1);
expect(r.rows[0].data_type).toBe('character varying');
expect(r.rows[0].is_nullable).toBe('YES'); // nullable for existing users
});

test('migration preserves existing data', async ({ db }) => {
const r = await db.query(
'SELECT id, email, role, phone_number FROM users WHERE email = $1',
['admin@test.com'],
);
expect(r.rows[0].role).toBe('admin');
expect(r.rows[0].phone_number).toBeNull(); // correct null handling
});
```

Q 276 LEVEL · Mid to Senior

How do you handle test database isolation in parallel test execution?

CONCEPT

Parallel tests that share a database can interfere with each other, and there are three viable strategies. **Unique data per test**: each test creates rows with a unique key (timestamp + random) and deletes them in cleanup. The default for most cases: simple, reliable, works on any database. **Schema-per-worker**: globalSetup creates a separate schema per worker (test_worker_1, test_worker_2) and each worker connects to its own; complete isolation, used when unique-data discipline is hard to maintain at scale. Transaction rollback: each test runs inside BEGIN/ROLLBACK so nothing persists; elegant for **read-only tests only**, because it undoes the very persistence a write-verification test is meant to confirm. Choose by the test's purpose.

INTERVIEW STRATEGY

Show you choose strategy by use case, not apply one approach to everything. Name the three explicitly, unique data, schema-per-worker, transaction rollback, and the boundary on each. The crucial limit: transaction rollback only fits read-only tests, because rollback undoes the persistence write-tests need to verify. That contraindication is what separates junior from senior thinking. Where this goes wrong is reaching for rollback for write tests and being baffled by false-negative assertions. The follow-up is often ‘which is your default?’, answer unique data per test.

How to phrase it

> *Parallel tests that share a database can interfere with each other, and there are three viable strategies, which is the structure I would lay out. First, unique data per test, where every test creates rows with a unique key, timestamp plus a random number in the email, for instance, and deletes them in cleanup. That is also my answer to the follow-up about which is my default, because it is simple and works on any database without extra infrastructure. Second, schema-per-worker, where globalSetup creates a separate schema per worker process, test_worker_1, test_worker_2, and each worker connects to its own schema with its own copy of the seed data. That gives complete isolation and is what I reach for when the test suite is complex enough that maintaining unique-data discipline becomes brittle. Third, transaction rollback, where each test runs inside a BEGIN and ROLLBACK so nothing persists, which is elegant for read-only tests. But there is a crucial contraindication I always state: rollback only fits read-only tests, because it undoes the very persistence that a write-verification test is meant to confirm. The mistake is reaching for transaction rollback for write tests, the test rolls back the insert it just made and then asserts the row exists, which it does not, because the rollback removed it, and you get false-negative failures that look like real bugs. So the rule is: unique data per test by default, schema-per-worker for complex suites, transaction rollback only for read-only, and the choice is driven by the test's purpose, never by what feels clever.*

Key points to hit

(1) Three strategies: unique data, schema-per-worker, transaction rollback. **(2)** Unique data per test: simple, reliable, default for most cases. **(3)** Schema-per-worker: complete isolation, for complex suites. **(4)** Transaction rollback: read-only tests only; rollback undoes persistence. **(5)** The next question: default is unique data per test. **(6)** What loses the point: using rollback for write tests, false-negative assertions.

CODE

```
// 1) Unique data per test: default
test('user appears in the list', async ({ db, page }) => {
const email = `test-${Date.now()}-${Math.random()}@example.com`;
await db.query('INSERT INTO users (email, role) VALUES ($1, $2)', [email, 'user']);
await page.goto('/admin/users');
await expect(page.locator('tbody tr').filter({ hasText: email })).toBeVisible();
await db.query('DELETE FROM users WHERE email = $1', [email]);
});

// 2) Schema-per-worker (in globalSetup): CREATE SCHEMA test_worker_$WORKER_INDEX

// 3) Transaction rollback: READ-ONLY tests only
test('read-only count', async ({ db }) => {
await db.query('BEGIN');
try {
const r = await db.query('SELECT COUNT(*) FROM users');
expect(Number(r.rows[0].count)).toBeGreaterThan(0);
} finally { await db.query('ROLLBACK'); }
});
```

Q 277

LEVEL · Senior

How do you adapt SQL testing patterns when the database is NoSQL (MongoDB, DynamoDB, Firestore)?

CONCEPT

NoSQL databases break assumptions SQL testing relies on, and the test patterns adapt accordingly. Three differences worth knowing. **No transactions across documents**: MongoDB has transactions but only within a replica set and with performance cost; DynamoDB and Firestore have limited transactional APIs. The pattern from S24 Q3, BEGIN, work, ROLLBACK, doesn't apply. Cleanup becomes explicit per-document deletion via cleanup registry from S7 Q12. **Schema flexibility means schema drift**: NoSQL documents don't enforce structure at the database level, so two records can have different fields. Tests that assume a specific shape need to assert the shape exists, not just the values, schema validation with zod or ajv against actual fetched documents. **Eventual consistency**: DynamoDB and Firestore default to eventual consistency, so a write followed by an immediate read may see stale data. Tests use the consistency options the database provides (ConsistentRead in DynamoDB) or poll with expect dot poll from S5 Q8 to wait for convergence. The discipline: understand the consistency model before writing the tests. Strong-consistency reads test the happy path; tests of eventual-consistency behaviour need the poll-and-wait pattern. The trap: treating NoSQL like SQL with different syntax, the differences are semantic, not syntactic, and tests that ignore them produce flake under load.

INTERVIEW STRATEGY

What earns the tick is NoSQL testing calibrated against SQL patterns. Walk three differences: no cross-doc transactions, schema drift, eventual consistency. The 'understand consistency model first' rule signals depth here. The thing to avoid is NoSQL-as-SQL-with-different-syntax. Expect the follow-up ‘what's the biggest gotcha?’, answer eventual consistency; writes don't appear in reads immediately.

How to phrase it

> *NoSQL databases break assumptions SQL testing relies on, and the test patterns adapt accordingly. Three differences worth knowing and the test implications of each. First, no transactions across documents: or limited transactions, depending on the database. MongoDB has transactions but only within a replica set and with performance cost; DynamoDB and Firestore have limited transactional APIs that don't span operations the way a SQL BEGIN-COMMIT does. The pattern from the transactions question earlier, BEGIN, work, ROLLBACK, doesn't apply, or applies with caveats. Cleanup becomes explicit per-document deletion via cleanup registry, the pattern I covered in the API testing section. Track every document created, delete each individually in afterEach. Second, schema flexibility means schema drift. NoSQL documents don't enforce structure at the database level, so two records in the same collection can have different fields, different types for the same field name, optional fields that appear in some documents and not others. Tests that assume a specific shape need to assert the shape exists, not just the values, schema validation with zod or ajv against actual fetched documents catches the bug where a service has started writing a renamed field but older code still reads the old name. Third, eventual consistency. DynamoDB and Firestore default to eventual consistency, so a write followed by an immediate read may see stale data, the write hasn't propagated to the read replica yet. Tests use the consistency options the database provides, like ConsistentRead true in DynamoDB, or poll with expect dot poll from the assertions section to wait for convergence. That's also my answer to the follow-up about the biggest gotcha: eventual consistency. It's the failure mode most likely to produce intermittent test failures that look like flake but are actually the test asserting too eagerly on data that hasn't propagated yet. Strong-read APIs or polling fixes it; ignoring it produces a suite that's unreliable under exactly the conditions you needed it reliable in. The point worth closing on is understand the consistency model before writing the tests. Strong-consistency reads test the happy path where I want deterministic behaviour; tests of eventual-consistency behaviour itself need the poll-and-wait pattern because that's the behaviour they're verifying. The trap is treating NoSQL like SQL with different syntax. The differences are semantic, not syntactic, and tests that ignore them produce flake under load, the kind of flake that goes away when the test suite runs slowly and reappears when it runs fast.*

Key points to hit

(1) No cross-document transactions: cleanup via registry, not BEGIN/ROLLBACK. **(2)** Schema flexibility -> drift: validate shape with zod/ajv, not just values. **(3)** Eventual consistency: writes don't appear in reads immediately, strong-read or poll. **(4)** Understand the consistency model BEFORE writing tests. **(5)** Asked next: biggest gotcha = eventual consistency, produces flake disguised as bugs. **(6)** The common miss: NoSQL-as-SQL-with-different-syntax, semantic differences, not just syntax.

CODE

```
// MongoDB: cleanup registry (no easy ROLLBACK across documents)
export const test = base.extend<{ mongo: MongoClient; cleanup: { add: (col: string, id:
ObjectId) => void } }>({
mongo: async ({}, use) => {
const client = await MongoClient.connect(process.env.MONGO_URL!);
await use(client);
await client.close();
},
cleanup: async ({ mongo }, use) => {
const tracked: { col: string; id: ObjectId }[] = [];
await use({ add: (col, id) => tracked.push({ col, id }) });
for (const { col, id } of tracked.reverse()) {
await mongo.db('test').collection(col).deleteOne({ _id: id }).catch(() => {});
}
},
});

// DynamoDB: strong-read option to bypass eventual consistency
const result = await dynamo.send(new GetCommand({
TableName: 'Users',
Key: { userId: 'u_123' },
ConsistentRead: true, // <- forces strong read
}));

// Validate document shape with zod (catches schema drift)
import { z } from 'zod';
const UserDoc = z.object({
_id: z.any(), email: z.string().email(), role: z.enum(['admin','user']), createdAt:
z.date(),
});
const doc = UserDoc.parse(fetched); // throws on shape mismatch
```

Q 278 LEVEL · Senior

How do you anonymise production data for use in test environments without losing test value?

CONCEPT

Production-cloned data gives tests the realism synthetic data lacks: real distributions, edge cases the team forgot to invent, the actual shape of legacy records. But production data carries PII that can't legally or ethically ship to test environments. **Anonymisation** is the bridge: keep the distributions and structure, replace identifying values. Three techniques layered by data type. Deterministic hashing: replace email with sha256 of email plus salt, truncate to twelve chars, append at example dot com. Same input produces same output, so relational integrity is preserved, the same customer's orders all link correctly, but the email isn't reversible. **Format-preserving substitution**: replace phone numbers with valid-format fake numbers, replace names with faker-generated equivalents, replace addresses with addresses in the same country but at different streets. The format matches production so validation logic still works; the values are fake. Field-level encryption or redaction: for sensitive fields that don't need to look real, internal IDs, audit notes, just hash or redact. The discipline: preserve distributions and relationships, not exact values. The shape of the data is what makes tests realistic; the actual email address is what makes it a privacy violation. The trap: **partial anonymisation**, anonymising the obvious fields but leaving

free-text fields (notes, comments, descriptions) full of PII. Free-text fields need specific scrubbing tools or full redaction.

INTERVIEW STRATEGY

You need to give production data anonymisation as a real engineering concern. Walk three techniques (deterministic hashing, format-preserving substitution, encryption/redaction) with what each preserves. The 'preserve distributions and relationships, not exact values' rule is what separates junior from senior thinking. The risk is partial anonymisation leaving free-text fields un-scrubbed. They usually follow with ‘what about free-text fields?’, answer specific PII-scrubbing tools or full redaction; can't safely anonymise prose at scale.

How to phrase it

> *Production-cloned data gives tests the realism synthetic data lacks: real distributions, edge cases the team forgot to invent, the actual shape of legacy records that accumulated over years. The synthetic alternative, faker-generated everything, misses the long tail of weird real-world data. But production data carries PII that can't legally or ethically ship to test environments, especially with GDPR, CCPA, and equivalents. Anonymisation is the bridge: keep the distributions and structure, replace identifying values. Three techniques layered by data type. First, deterministic hashing. Replace email with sha-two-five-six of email plus a salt, truncate to twelve characters, append at example dot com. Same input produces same output, so relational integrity is preserved, the same customer's orders all link correctly because every reference to their email resolves to the same anonymised value, but the email isn't reversible to the original. The salt is the key piece: without it, hashed emails are guessable by dictionary attack. Second, format-preserving substitution. Replace phone numbers with valid-format fake numbers, replace names with faker-generated equivalents, replace addresses with addresses in the same country but at different streets. The format matches production so validation logic still works, phone-number regex still passes, address parsers still parse, but the values are fake. Faker is the workhorse for this technique. Third, field-level encryption or redaction. For sensitive fields that don't need to look real, internal IDs, audit log notes, anything not user-visible, just hash or redact to a constant value. The test doesn't depend on the content, it depends on the field existing with some value. The thing to emphasise first, because it captures the design rule, is preserve distributions and relationships, not exact values. The shape of the data is what makes tests realistic, the percentage of users with certain attributes, the relationship between users and their orders, the timing patterns of transactions. The actual email address is what makes it a privacy violation. Keep the structure, replace the content. The trap, and where most anonymisation pipelines leak data despite trying not to, is partial anonymisation. Anonymising the obvious fields, email, name, phone, but leaving free-text fields full of PII: support ticket notes with full names and addresses, comment fields users wrote with their phone numbers in, profile bios containing email signatures. That's also my answer to the follow-up about free-text fields: specific PII-scrubbing tools that recognise and replace named entities, like AWS Comprehend or Azure's PII detection, or full redaction of free-text fields if the tests don't depend on the content. Can't safely anonymise prose at scale with regex; either use a proper NER tool or redact the whole field.*

Key points to hit

(1) Production data = realism; PII = legal/ethical risk. Anonymisation bridges. **(2)** Deterministic hashing: same input -> same output. Relations preserved, values fake. **(3)** Format-preserving substitution: faker for phone/name/address, validation still works. **(4)** Field-level encryption/redaction: for non-user-visible sensitive fields. **(5)** Rule: preserve distributions and relationships, NOT exact values. **(6)** Avoid: partial anonymisation, free-text fields still full of PII.

CODE

```
// Anonymisation pipeline for production-cloned data
import { createHash } from 'crypto';
import { faker } from '@faker-js/faker';

const SALT = process.env.ANON_SALT!; // never in code; pulled from secrets

function anonEmail(real: string): string {
const h = createHash('sha256').update(real + SALT).digest('hex').slice(0, 12);
return `${h}@example.com`; // deterministic + format-preserving
}

function anonName(real: string): string {
faker.seed(hashToInt(real + SALT)); // deterministic faker output
return faker.person.fullName();
}

function anonPhone(real: string): string {
faker.seed(hashToInt(real + SALT));
return faker.phone.number(); // valid format, fake number
}

// Free-text fields need a real PII detector, not regex
// await comprehend.detectPiiEntities({ Text: notes, LanguageCode: 'en' })
// -> replace detected spans, or REDACT the whole field if test doesn't need content

// Apply to a snapshot before loading into test env
for (const user of productionSnapshot.users) {
user.email = anonEmail(user.email);
user.name = anonName(user.email); // seeded by email keeps consistency
user.phone = user.phone ? anonPhone(user.phone) : null;
user.notes = '[REDACTED]'; // free-text field
}

// Distributions preserved (count, age range, role mix); identifying values gone
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Why is a DB-assertion test stronger than a UI-only assertion?

A) It is faster
B) The UI can show success while the write failed silently, only a DB query catches that
C) It avoids using fixtures
D) It doesn't need a database client

2. Which two patterns make database seeding safe and idempotent?

A) ON CONFLICT DO NOTHING for reference data; scoped DELETE-before-INSERT for test-specific data
B) DROP TABLE before insert; RAISE on duplicates
C) Random data per run; no cleanup
D) Only seed in tearDown

3. What does a transaction-rollback test verify?

A) That the UI shows a loading spinner
B) That the test is fast
C) That a failed multi-step operation leaves the database unchanged (atomicity)
D) That the API returns 200

4. What two things does a migration test verify?

A) Speed and memory
B) Code coverage and lint rules
C) Only that the migration ran
D) Schema correctness (via information_schema) AND existing-data preservation

5. Which DB isolation strategy fits ONLY read-only tests?

A) Transaction rollback (BEGIN/ROLLBACK): the rollback undoes any persistence write-tests must verify
B) Unique data per test
C) Schema-per-worker
D) Truncating tables before each test

Section 24 · Answer key

#	Answer	Why and where to revisit
1	B) The UI can show success while the write failed silently, only a DB query catches that	DB validation confirms actual persistence (right values, right role), the highest-confidence test pattern. *Revisit Section 24.*
2	A) ON CONFLICT DO NOTHING for reference data; scoped DELETE-before-INSERT for test-specific data	Idempotent INSERTs survive a double globalSetup; scoped DELETE keeps test data identifiable via %@test.com. *Revisit Section 24.*
3	C) That a failed multi-step operation leaves the database unchanged (atomicity)	Mock the dependency to fail, assert UI error, AND assert row counts before == after, confirms full rollback, no orphans. *Revisit Section 24.*
4	D) Schema correctness (via information_schema) AND existing-data preservation	A migration tool's success only means it ran; explicit assertions catch nullable/NOT NULL slips and data corruption. *Revisit Section 24.*
5	A) Transaction rollback (BEGIN/ROLLBACK): the rollback undoes any persistence write-tests must verify	Rollback is elegant for reads, but used on a write test it undoes the very persistence the test is asserting. *Revisit Section 24.*

SECTION 25

Git and Version Control

Git questions in an SDET interview test whether you treat the test framework like production code, with branch protection, pre-commit enforcement, and deliberate visual-baseline review, not as scripts you commit at will. These five questions cover a Git workflow for a Playwright framework, the discipline for screenshot baselines, hooks that block anti-patterns before they reach the repo, the upgrade process that keeps you current, and git bisect for finding the commit that broke a test.

In this section

- Framework Git workflow: feature branches, protected main, conventional commits.
- Visual baselines: deliberate human review, never --update-snapshots in CI.
- Pre-commit hooks: typecheck, lint, block test.only, warn on waitForTimeout.
- Monthly Playwright upgrades via Dependabot: stay current cheaply.
- git bisect run finds the breaking commit in ~6 steps for 100 commits.

Q 279 LEVEL · Mid

How do you structure Git workflows for a Playwright framework?

CONCEPT

Treat the test framework like application code: **feature branches with a naming convention** (feature/add-checkout-tests, fix/flaky-login-test, chore/update-playwright-1.46), pull-request reviews, a **protected main branch** requiring passing CI and at least one reviewer, and pre-commit hooks via husky running lint-staged so ESLint and Prettier fix and format on every commit. Add **commitlint** with @commitlint/config-conventional to enforce a consistent commit-message format (feat, fix, refactor, chore, docs, test), which then powers automatic changelog generation. The framing that matters: these are not bureaucratic processes, they are automation that enforces quality without human intervention, which is exactly the same standard a production codebase deserves.

INTERVIEW STRATEGY

They are looking to see you treat the framework as code, not scripts. List the four standards, branch naming, protected main, pre-commit hooks, conventional commits, and frame them as automation rather than process. What reads senior is calling them quality enforcement, not bureaucracy. The pitfall is suggesting direct pushes to main are fine 'because it is only tests'. The next question is normally 'why conventional commits?', answer they enable automatic changelog generation and consistent history.

How to phrase it

> *I treat a Playwright framework like application code, which is the framing I would lead with because it is the senior position. So the workflow has four standards. First, a branch naming convention, feature-slash for new tests, fix-slash for flaky-test fixes, chore-slash for upgrades, which makes the branch list self-documenting. Second, a protected main branch that requires passing CI and at least one reviewer, with no direct pushes, so nothing untested lands on main. Third, pre-commit hooks via husky running lint-staged, so ESLint and Prettier fix and format every commit, nobody can commit code that violates the standards because the tooling blocks it. Fourth, commitlint with the conventional config enforcing the prefix, feat, fix, refactor, chore, docs, test, which gives me a consistent history and answers the follow-up about why conventional commits: they enable automatic changelog generation, so my release notes come from the commit log instead of being hand-written. The framing I close on is that none of this is bureaucracy. These are automated quality gates that enforce standards without human intervention, which is exactly what a production codebase deserves, and the test framework is part of the production codebase, not a script collection. The classic error is treating the framework as second-class with looser rules 'because it is only tests'; that is how the framework rots.*

Key points to hit

(1) Treat the framework as application code: same standards. **(2)** Branch naming convention: feature/, fix/, refactor/, chore/. **(3)** Protected main: passing CI + 1 reviewer, no direct pushes. **(4)** husky + lint-staged: ESLint + Prettier on every commit. **(5)** Follow-up: conventional commits power automatic changelog generation. **(6)** The easy mistake: looser rules for the framework 'because it's only tests'.

CODE

```
# Branch naming convention
feature/add-checkout-tests
fix/flaky-login-test
refactor/extract-data-table-component
chore/update-playwright-1.46

// package.json: husky + lint-staged + commitlint
{
"husky": {
"hooks": {
"pre-commit": "lint-staged",
"commit-msg": "commitlint --edit $1"
}
},
"lint-staged": { "*.ts": ["eslint --fix", "prettier --write"] }
}

// commitlint.config.js: enforce conventional commits
module.exports = {
extends: ['@commitlint/config-conventional'],
rules: { 'type-enum': [2, 'always', ['feat','fix','refactor','chore','docs','test']] },
};
```

Q 280 LEVEL · Mid

How do you handle Playwright baseline screenshot updates in Git?

CONCEPT

Visual-test baselines are binary PNG files committed to Git, and updates need a deliberate human process so accidental baseline changes don't silently mask regressions. The single most important rule: **never run --update-snapshots in CI automatically**. If CI updates baselines on failure, every visual regression is silently accepted and the visual test gives no protection at all. Updates happen locally with **npx playwright test --update-snapshots**, then the diff is reviewed in the HTML report, the updated PNGs are committed with a message explaining the intentional visual change, and the PR reviewer verifies the change really is intentional before merging. Mark PNGs as binary in **.gitattributes** so diffs behave correctly, and never gitignore baseline files even when ignoring test-results output.

INTERVIEW STRATEGY

The point of this one is to see you protect visual tests from silent acceptance. Lead with the single non-negotiable rule, never --update-snapshots in CI, and explain the failure mode: if CI does it, every regression is silently accepted. Then describe the human-review process with PR diff verification. The pitfall is auto-updating baselines on failure 'to keep tests green'. A common follow-up is 'how do you enforce that rule?', answer by gating --update-snapshots behind a specific local environment variable so it cannot run unguarded in CI.

How to phrase it

> *Visual-test baselines are binary PNG files committed to Git, and updates need a deliberate human process so accidental baseline changes do not silently mask regressions. The single rule I would lead with, because it is non-negotiable and people get it wrong, is never run --update-snapshots in CI automatically. If CI updates baselines on failure to keep the build green, every visual regression is silently accepted and the visual test gives zero protection, it is worse than no test, because it provides false confidence. So my process is: I run --update-snapshots locally, review the diff image in the HTML report to confirm the visual change is what I intended, commit the updated PNGs with a clear message like update visual baselines for nav redesign, and the PR reviewer must verify the visual change is intentional before merging. That answers the follow-up about enforcement, which is the second part of the answer. I gate --update-snapshots behind a specific environment variable so it physically cannot run unguarded in CI, and the human review of the diff image is part of the PR template. I also mark PNGs as binary in .gitattributes so Git treats them sensibly, and I never gitignore baseline files even while ignoring test-results output. The risk is auto-updating on failure to keep tests green; the whole point of a visual test is to fail loudly when the look changes, and the human decides whether the new look is correct.*

Key points to hit

(1) Never --update-snapshots in CI: silently accepts every regression. **(2)** Update locally; review diff image in the HTML report. **(3)** Commit baselines with a clear intentional-change message. **(4)** PR reviewer verifies the visual change is intentional before merge. **(5)** Expect next: enforce via a gated env variable plus PR-template review. **(6)** Trap: auto-updating baselines on CI failure 'to keep tests green'.

CODE

```
# Update baselines locally only
npx playwright test --update-snapshots

# .gitattributes: mark PNGs as binary
*.png binary

# .gitignore: ignore test output, KEEP baselines
test-results/
!tests/**/*.png

# Baseline update process
# 1. Visual test fails with diff image in HTML report
# 2. Reviewer opens diff image and verifies the change is intentional
# 3. If intentional: npx playwright test --update-snapshots
# 4. Commit: git commit -m "test: update visual baselines for nav redesign"
# 5. PR reviewer confirms visual change matches the PR's intent
```

Q 281 LEVEL · Mid

How do you use Git hooks to enforce Playwright best practices?

CONCEPT

Git hooks automate enforcement of framework standards before code reaches the repository. **Pre-commit** runs fast checks on every commit; **pre-push** runs slower checks (e.g. smoke tests) before pushing to main. The hooks worth their weight: typecheck and lint with zero warnings; block any test.only / it.only / describe.only from being committed (focused tests in CI run only that test and let the rest of the suite pass silently, a real incident pattern); **warn on waitForTimeout** as an anti-pattern (warning, not block, so education happens without disruption). The principle that lands with interviewers: education through tooling is more effective than documentation, a hook that fires once teaches more than a wiki page nobody reads.

INTERVIEW STRATEGY

You are being asked to show automation of standards, not policing. Lead with the highest-value hook: blocking test.only, with the story of how a focused test masks 849 other tests passing trivially. Then the waitForTimeout warning (not block) shows judgement, educate without disrupting. The phrase 'education through tooling' is the senior signal. The classic error is heavy blocking hooks that frustrate developers into bypassing. The follow-up is often 'what real bug has a hook caught?', answer the test.only that would have made CI report passing while skipping everything else.

How to phrase it

> *Git hooks automate enforcement of framework standards before code reaches the repository. I split them into pre-commit for fast checks on every commit, typecheck, lint with zero warnings, and pre-push for slower checks like running smoke tests before pushing to main. But the hook I am most proud of, and the answer to the follow-up about a real bug a hook has caught, is the grep in pre-commit that blocks test.only, it.only, or describe.only. It has prevented three incidents where a developer would have committed a focused test, which in CI runs only that single test and lets every other test pass silently, so the CI report shows all green while eight hundred and forty-nine tests were actually skipped, and a broken build ships looking healthy. The hook takes about a hundred milliseconds and has saved hours of debugging and at least one near-incident. I also add a grep for waitForTimeout, but I make this a warning rather than a block. The developer sees the message that waitForTimeout is an anti-pattern, but the commit goes through. That deliberate softness is the senior choice, because heavy blocking hooks frustrate developers into bypassing them with --no-verify and then you lose the protection entirely. The principle I close on is education through tooling is more effective than documentation, a hook firing once with a clear message teaches more than a wiki page nobody reads. So hooks are how I make the standards self-enforcing without needing me to police every PR.*

Key points to hit

(1) Pre-commit for fast checks; pre-push for slower (e.g. smoke). **(2)** Block test.only/it.only/describe.only: prevents silent-skip incidents. **(3)** Warn (not block) on waitForTimeout: educate without disrupting. **(4)** Heavy blocks get bypassed with --no-verify: design hooks people accept. **(5)** They usually ask next: caught 3 test.only incidents that would have hidden 849 skips. **(6)** Where this goes wrong: making every check a hard block until people --no-verify everything.

CODE

```
#!/bin/sh
# .husky/pre-commit: fast checks on every commit
. "$(dirname "$0")/_/husky.sh"

npm run typecheck || exit 1
npm run lint || exit 1

# Block focused tests: the most valuable hook
if grep -r "test\.only\|it\.only\|describe\.only" tests/; then
echo "ERROR: test.only found. Remove before committing."
exit 1
fi

# Warn (not block) on anti-patterns
if grep -r "waitForTimeout" tests/ src/; then
echo "WARNING: waitForTimeout is an anti-pattern. Use explicit waits."
fi

# .husky/pre-push: smoke before pushing to main
BRANCH=$(git rev-parse --abbrev-ref HEAD)
[ "$BRANCH" = "main" ] && npm run test:smoke || exit 1
```

Q 282

LEVEL · Junior to Mid

How do you manage Playwright framework versioning and upgrades?

CONCEPT

Playwright releases roughly every **month** with new features and browser updates, so a structured upgrade process minimises disruption while keeping the framework current. The cadence that works: **monthly** upgrades via a chore branch, npm install @playwright/test@latest, npx playwright install for browsers, run the full suite, check release notes for breaking changes, update baseline screenshots if browser rendering shifted. The whole cycle takes about 30 minutes when done regularly. Automate it with **Dependabot** creating weekly PRs, CI runs automatically, you merge if green. The cost of falling behind: a six-month gap accumulates breaking changes and turns a quick upgrade into a multi-day migration with conflicts at every layer.

INTERVIEW STRATEGY

What scores here is showing a disciplined cadence, not heroic annual migrations. Give a concrete rhythm, monthly, ~30 minutes, and the Dependabot automation that makes it cheap. The currency-cost-compounds framing (six-month gap = multi-day migration) is what separates junior from senior thinking. The classic error is putting off upgrades until forced. Expect the follow-up ‘what happens if you fall behind?’, answer accumulated breaking changes turn a 30-minute upgrade into a multi-day project.

How to phrase it

> *Playwright releases roughly every month with new features and browser updates, so I upgrade monthly on a structured process, not annually, not when forced. The cycle is a chore branch, npm install at-playwright-slash-test at latest, npx playwright install for the browser binaries, run the full suite, check the release notes for breaking changes, and update baseline screenshots if browser rendering shifted. Done regularly it takes about thirty minutes start to finish. I automate it further with Dependabot creating a weekly PR for the @playwright/test package, which runs CI automatically, and I merge it if the tests pass, so most months the upgrade is essentially zero effort. The reason I am strict about monthly, and the answer to the follow-up about what happens if I fall behind, is that the cost compounds. A six-month gap accumulates breaking changes from multiple releases, browser rendering shifts that invalidate dozens of visual baselines at once, deprecated APIs that have to be migrated together, and what would have been thirty minutes spread across six painless upgrades becomes a multi-day migration with conflicts at every layer. The pitfall is putting off upgrades because the current version is working fine; staying current is much cheaper than catching up. So the rule is: monthly, automated, Dependabot-driven, painless. Falling behind is a self-inflicted wound.*

Key points to hit

(1) Playwright releases roughly monthly: upgrade monthly, not annually. **(2)** Cycle: chore branch, npm install, browsers, full suite, release notes, baselines. **(3)** Whole cycle ~30 minutes when done regularly. **(4)** Dependabot weekly PRs: CI runs automatically, merge if green. **(5)** The next question: fall behind, 6 months -> multi-day migration with conflicts. **(6)** What loses the point: putting off upgrades because the current version 'works fine'.

CODE

```
# Monthly upgrade cycle (~30 min when done regularly)
git checkout -b chore/upgrade-playwright-1.46
npm install @playwright/test@latest
npx playwright install
npm run test # full suite
# Check release notes: https://playwright.dev/docs/release-notes
npx playwright test --update-snapshots --project=chromium # if rendering shifted

# .github/dependabot.yml: automated weekly PR
version: 2
updates:
- package-ecosystem: npm
directory: /
schedule: { interval: weekly }
allow:
- dependency-name: "@playwright/test"
```

Q 283 LEVEL · Mid to Senior

How do you use git bisect to find the commit that introduced a test failure?

CONCEPT

Git bisect performs a **binary search through commit history** to find the exact commit that introduced a regression, essential when a test that worked last week is failing now and you have no obvious suspect. Start with git bisect start, mark the current commit **bad** and a known-good earlier tag good, then run the failing test at the midpoint Git checks out, mark good or bad, and repeat, typically 6-8 steps for 100 commits. Even better, **git bisect run** with a test script automates the entire binary search: bisect runs the test at each step and tells you the bad commit with zero manual input. The result is dramatic: a regression that would take hours of manual code inspection across 47 commits is named in about 10 minutes.

INTERVIEW STRATEGY

This question is there to see you reach for the right diagnostic tool for regressions in big histories. Describe the manual bisect briefly, then make git bisect run the centrepiece because it is the version most engineers never use. Quantify the saving: 47 commits, manual = hours, bisect = ~10 minutes. The bit that marks you as senior is naming the surprising finding (a 2-pixel CSS shift breaking a stability check) because it shows you have actually used it. The mistake is doing the binary search by hand when the automated form exists. They usually follow with 'when is bisect not the right tool?', answer when the test is intermittent, because a flaky test confuses the good/bad signal.

How to phrase it

> *Git bisect performs a binary search through commit history to find the exact commit that introduced a regression, and I would call it the most powerful debugging tool most developers never use. The manual form is straightforward: git bisect start, mark the current commit bad and a known-good earlier tag good, then Git checks out the midpoint, I run the failing test, mark the result good or bad, and repeat, typically six to eight steps for a hundred commits. But the form I actually use, and the one that distinguishes the answer, is git bisect run with a test script. That automates the entire binary search: bisect runs the test command at each step and completes the whole search without any manual input. I used this on a real regression that appeared after forty-seven commits over two weeks. Manual inspection would have taken hours; git bisect run found the exact commit in six steps and about ten minutes, and the commit turned out to be a CSS change that moved a button by two pixels, which broke Playwright's stability actionability check intermittently. There is no way I would have guessed that from reading diffs. For the follow-up about when bisect is not the right tool, the answer is when the test is intermittent rather than consistently failing, a flaky test confuses the good-or-bad signal and bisect lands on the wrong commit. So bisect for a clean regression, flake investigation for an intermittent one, and reach for git bisect run by default because the automated form is so much faster.*

Key points to hit

(1) Binary search through commit history: ~6-8 steps for 100 commits. **(2)** git bisect run with a test script automates the whole search. **(3)** Real example: 47 commits, ~10 min to find a 2-pixel CSS shift. **(4)** Dramatic saving vs hours of manual diff inspection. **(5)** Asked next: don't use bisect on intermittent tests, it lands wrong. **(6)** The common miss: doing the binary search by hand when 'bisect run' exists.

CODE

```
# Manual bisect
git bisect start
git bisect bad # current commit fails
git bisect good v1.44.0 # known-good tag
# Git checks out the midpoint: run the failing test
npx playwright test tests/checkout.spec.ts --project=chromium
git bisect good # or: git bisect bad
# repeat … typically 6-8 steps for 100 commits
git bisect reset

# Automated bisect: zero manual input
git bisect start HEAD v1.44.0
git bisect run npx playwright test tests/checkout.spec.ts --project=chromium
# "abc1234 is the first bad commit"
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What's the right mindset for Git workflows on a Playwright framework?

A) Looser rules: it's only tests
B) Treat the framework like application code: protected main, hooks, conventional commits
C) No PR reviews needed
D) Commit only to main

2. What's the single non-negotiable rule for visual screenshot baselines?

A) Update baselines on every commit
B) Use JPEG instead of PNG
C) Delete baselines weekly
D) Never run --update-snapshots in CI: it silently accepts every regression

3. Which pre-commit hook has prevented real incidents?

A) A hook that auto-formats code
B) A hook that increments version numbers
C) A grep that blocks test.only / it.only / describe.only, prevents silent-skip in CI
D) A hook that runs all tests

4. What is the right Playwright upgrade cadence?

A) Monthly, ~30 minutes when done regularly: Dependabot can automate it
B) Yearly major migrations
C) Only when forced by breaking changes
D) Never: keep stable

5. What does 'git bisect run' do that manual bisect doesn't?

A) It's slower
B) It automates the binary search by running a test script at each step, zero manual input
C) It bypasses the commit history
D) It modifies the bad commit

Section 25 · Answer key

#	Answer	Why and where to revisit
1	B) Treat the framework like application code: protected main, hooks, conventional commits	The framework is part of the production codebase; the same standards apply, automated rather than policed. *Revisit Section 25.*
2	D) Never run --update-snapshots in CI: it silently accepts every regression	Auto-updating in CI means visual tests provide zero protection; updates must be a deliberate human decision. *Revisit Section 25.*
3	C) A grep that blocks test.only / it.only / describe.only, prevents silent-skip in CI	A focused test in CI runs only that one and lets every other test 'pass', the hook catches it in 100ms. *Revisit Section 25.*
4	A) Monthly, ~30 minutes when done regularly: Dependabot can automate it	Releases come roughly monthly; falling 6 months behind turns a 30-min upgrade into a multi-day migration. *Revisit Section 25.*
5	B) It automates the binary search by running a test script at each step, zero manual input	git bisect run + a test command finds the breaking commit in ~6 steps with no human between iterations. *Revisit Section 25.*

SECTION 26

Playwright vs Other Frameworks

Framework-comparison questions test whether you can argue with judgement rather than tribalism, because the wrong answer in 2026 is 'Playwright is always better'. These seven questions cover Playwright vs Selenium, Cypress, WebdriverIO, and Puppeteer with the contexts where each is the right choice, plus a structured framework-selection process anchored on a proof of concept rather than benchmarks.

In this section

- Playwright vs Selenium: WebSocket/CDP vs HTTP, default for new, keep for legacy.
- Playwright vs Cypress: coverage breadth vs developer experience.
- WebdriverIO wins when web + native mobile must live in one framework.
- Puppeteer is for scripting (scraping, PDFs); Playwright is for testing.
- Selection by POC, not benchmark: 10 representative tests, real DX measured.

Q 284 LEVEL · Mid

Playwright vs Selenium: when do you choose each?

CONCEPT

Both are production-grade. The architectural difference: Playwright uses a **persistent WebSocket connection** to each browser (CDP for Chromium and Edge, Playwright's own patched protocols for Firefox and WebKit); classic Selenium uses stateless HTTP requests to browser drivers for every command. That is what makes Playwright's auto-waiting, native network interception, and trace viewer work the same way on every engine, because they all depend on listening to browser events as they happen. **Do not overclaim here**: Selenium 4 also has an event channel now, CDP for Chromium and, since 4.18, WebDriver BiDi over a WebSocket with genuine network interception. The gap is uniformity and maturity, not capability. The honest selection rule: Playwright for new projects, TypeScript teams, and modern frontend frameworks; Selenium when there is significant existing investment (hundreds of Selenium tests is gradual migration territory, not a rewrite), when **real Safari testing via SafariDriver** is required, or when enterprise policy mandates Java. The message worth stating: both are production-grade. Playwright is the better starting point today. Selenium is the better choice for existing investment.

INTERVIEW STRATEGY

They are after judgement, not advocacy. Never say ‘Playwright is always better’. Lead with the WebSocket-vs-HTTP architectural difference, it shows technical depth, then frame the choice on existing investment and real Safari testing. The both-are-production-grade closer is the calibration signal. The trap is dismissing Selenium as outdated. The next question is normally ‘when would you stay on Selenium?’, answer significant existing investment, real Safari requirement, or Java mandate.

How to phrase it

> *Both Playwright and Selenium are production-grade tools, and the first thing I would say in an interview, because it is what separates a senior answer from a junior one, is never claim Playwright is always better. The architectural difference is the depth signal: Playwright uses a persistent WebSocket connection to the browser, CDP for Chromium and Edge and its own patched protocols for Firefox and WebKit, while classic Selenium uses stateless HTTP requests to browser drivers for every single command. The persistent connection is what makes Playwright's auto-waiting, native network interception, and trace viewer work identically on every engine, because they all depend on listening to browser events as they happen. I'd be careful not to overclaim, though. Selenium 4 has CDP access and, since 4.18, WebDriver BiDi over its own WebSocket with real network interception, so the difference is uniformity and maturity rather than one tool being able to do something the other cannot. When we migrated from Selenium to Playwright, our test flakiness dropped about sixty-five percent because auto-waiting replaced every line of manual explicit wait code. On selection, I choose Playwright for new projects, TypeScript teams, and applications using modern frontend frameworks. But, and this is the follow-up about when I would stay on Selenium, I keep Selenium when there is a large existing Selenium framework with significant investment, hundreds of tests is gradual migration territory, not a rewrite, when Safari testing via real SafariDriver is required, because Playwright's WebKit is a genuine WebKit build but it is not Safari the application, so Safari specific behaviour still needs Safari, or when enterprise policy mandates Java. The message I close on is both are production-grade. Playwright is the better starting point today; Selenium is the better choice for existing investment. The classic error is dismissing Selenium as outdated, which signals tribalism rather than judgement.*

Key points to hit

(1) Architecture: Playwright = persistent WebSocket, classic Selenium = HTTP WebDriver. **(2)** WebSocket enables auto-waiting, network interception, trace viewer, on every engine. **(3)** Currency: Selenium 4 has CDP, and 4.18+ has WebDriver BiDi interception. Don't say it cannot. **(4)** Playwright for new projects / TypeScript / modern SPAs. **(5)** Selenium for large existing investment, real SafariDriver, Java mandate. **(6)** Follow-up: stay on Selenium when migration cost exceeds benefit. **(7)** Avoid: 'Playwright is always better', reads as tribal, not senior.

CODE

```
// Architectural snapshot: Playwright vs Selenium (2026)
// Protocol persistent WebSocket HTTP WebDriver + BiDi
// Auto-waiting built-in manual explicit waits
// Network interception native, all engines CDP or BiDi (4.18+)
// Debugging trace viewer limited built-in tools
// Browser drivers auto-managed Selenium Manager
// Mobile emulation device profiles ChromeOptions emulation
// Languages JS/TS/Py/Java/C# Java/Py/C#/Ruby/JS
// Safari real WebKit build real SafariDriver
```

Q 285 LEVEL · Mid

Playwright vs Cypress: what are the key architectural differences?

CONCEPT

Cypress runs inside the same browser process as the application, powerful for synchronous DOM inspection and the famous time-travel debugging GUI, but architecturally limited: Chromium-based browsers plus stable Firefox (WebKit still experimental), **no native multi-tab support**, and **multi-origin tests require cy.origin workarounds**. Playwright runs outside the browser via WebSocket, which gives it Chromium, Firefox and WebKit, native multi-tab and multi-origin, mobile emulation, and bindings in Python, Java, C# beyond JS/TS. The honest assessment: Cypress wins developer experience for pure frontend JavaScript teams (the GUI, hot reload, time-travel debugging are outstanding); **Playwright wins coverage** (multiple browsers, tabs, origins, mobile, non-JS languages). Use both where both fit: Cypress for component-level tests, Playwright for end-to-end and cross-browser.

INTERVIEW STRATEGY

They want to see you reason about architecture, not pick a tribe. Lead with the in-process vs out-of-process distinction, it explains every other difference. Give Cypress its honest win (developer experience) before naming Playwright's wins (coverage), and close with both-can-coexist. That balance is the test of seniority on this. The failure mode is dismissing Cypress's DX. A common follow-up is ‘when would you choose Cypress?’, answer a pure frontend JS team where DX matters more than multi-browser coverage.

How to phrase it

> *The architectural difference that explains every other contrast is where the framework runs. Cypress runs inside the same browser process as the application, which is powerful for synchronous DOM inspection and gives it the famous time-travel debugging GUI, but architecturally limited: Chromium-based browsers plus stable Firefox, with WebKit still experimental, no native multi-tab support, and multi-origin tests require cy.origin workarounds because the in-process model cannot cross origins cleanly. Playwright runs outside the browser via WebSocket, which gives it Chromium plus Firefox and WebKit out of the box, native multi-tab and multi-origin, mobile emulation, and language bindings beyond JS, Python, Java, C-Sharp. Now, the honest assessment, because tribalism is the trap. Cypress wins developer experience for pure frontend JavaScript teams, the GUI, hot reloading, time-travel debugging really are outstanding, and dismissing that signals you haven't actually used Cypress. Playwright wins coverage: multiple browsers, tabs, origins, mobile emulation, non-JavaScript languages. That answers the follow-up about when I would choose Cypress: a pure frontend JS team where developer experience matters more than multi-browser coverage. I have worked in shops that used both, Cypress for unit and component-level tests, Playwright for end-to-end and cross-browser, and they are complementary, not mutually exclusive. The answer that lands is judgement: each is excellent at what it was designed for, and the choice depends on what the team actually needs.*

Key points to hit

(1) Cypress in-process (sync DOM, GUI); Playwright out-of-process (WebSocket). **(2)** Cypress limits: Chromium-mainly, no native multi-tab, cy.origin for multi-origin. **(3)** Playwright wins: all three browsers, multi-tab/origin, mobile, multi-language. **(4)** Cypress wins: developer experience, GUI, hot reload, time-travel debugging. **(5)** Expect next: choose Cypress for pure frontend JS team prioritising DX. **(6)** The easy mistake: dismissing Cypress's DX, reads as tribal, not informed.

CODE

```
// Architectural snapshot: Playwright vs Cypress
// Browser support Chromium + Firefox + WebKit Chromium + Firefox stable, WebKit
experimental
// Multiple origins native cy.origin workaround required
// Multiple tabs native no native support
// Network interception excellent excellent
// Language JS/TS/Py/Java/C# JavaScript / TypeScript only
// Component testing experimental CT mature component testing
// Developer experience excellent outstanding GUI
```

Q 286 LEVEL · Mid

Playwright vs WebdriverIO: when does WebdriverIO win?

CONCEPT

WebdriverIO supports both WebDriver protocol and Chrome DevTools Protocol, making it a bridge between the Selenium ecosystem and modern browser automation. The distinguishing strength: first-class Appium integration for native mobile testing, more mature than any Playwright-based mobile solution. The choice is clean: WebdriverIO wins when a team needs both web and native mobile testing in one framework, same tooling, same test runner, same reporting across Web/iOS/Android. For teams with existing WebdriverIO investment plus a native mobile requirement, it stays the right choice. For teams starting fresh with **web-only testing**, Playwright's simpler setup, better TypeScript support, and richer built-in features (trace viewer, fixtures, auto-waiting) make it the better choice. Pick by the breadth of platforms actually needed.

INTERVIEW STRATEGY

Your job here is to show you recognise where WebdriverIO genuinely wins, not dismiss it. The decisive answer: native mobile via Appium, in one framework with web. State that clearly, then make Playwright the answer for web-only fresh starts. That clean platform-breadth rule demonstrates calibration. The risk is treating WebdriverIO as Selenium's smaller cousin. The follow-up is often ‘would you migrate a WebdriverIO team to Playwright?’, answer no if they need native mobile; the Appium maturity gap is real.

How to phrase it

> *WebdriverIO supports both the WebDriver protocol and Chrome DevTools Protocol, which makes it a bridge between the Selenium ecosystem and modern browser automation. But its distinguishing strength, and the thing I would lead with because it is what wins the decision, is first-class Appium integration for native mobile testing, more mature than any Playwright-based mobile solution today. So the choice is genuinely clean: WebdriverIO wins when a team needs both web and native mobile testing in one framework. Same tooling, same test runner, same reporting across Web, iOS and Android, with no stitching together two separate frameworks. For teams with existing WebdriverIO investment plus a native mobile requirement, it stays the right choice and that is also my answer to the follow-up about migrating a WebdriverIO team to Playwright, if they need native mobile, the answer is no, because the Appium maturity gap is real and a migration loses them capability they actually use. For teams starting fresh with web-only testing, though, Playwright's simpler setup, better TypeScript support, and richer built-in features like the trace viewer, fixtures, and auto-waiting make it the better starting point. The principle I would close on is to pick by the breadth of platforms actually needed, not by which framework feels more modern. The risk is treating WebdriverIO as Selenium's smaller cousin and missing that for native-mobile-plus-web it is genuinely the right tool.*

Key points to hit

(1) WebdriverIO supports both WebDriver and CDP: bridge tool. **(2)** Wins: first-class Appium for native mobile in one framework with web. **(3)** Playwright wins for web-only fresh starts (simpler, better TS, richer built-ins). **(4)** Decision driver: breadth of platforms actually needed, not modernity vibes. **(5)** They usually ask next: don't migrate WDIO+Appium team to Playwright if they need native mobile. **(6)** Trap: treating WebdriverIO as Selenium's smaller cousin.

CODE

```
// When WebdriverIO is the right choice
// Need web + native iOS + native Android in ONE framework
// Why first-class Appium integration, mature mobile story
// Trade setup is heavier than Playwright; web DX is slightly behind
//
// When Playwright is the right choice
// Need web-only, fresh start, TypeScript team
// Why simpler setup, trace viewer, fixtures, native auto-waiting
// Trade native mobile remains an Appium/Detox conversation
```

Q 287 LEVEL · Junior to Mid

Playwright vs Puppeteer: what is the relationship?

CONCEPT

Playwright was created by the same team that built Puppeteer at Google, and Playwright is the spiritual successor, it adds Firefox and WebKit support, a built-in test runner with fixtures, the trace viewer, and a richer API. **Puppeteer remains Chromium-only** and is a lower-level automation library without a test runner, you bring your own (Jest, Mocha). The right way to position them in an interview: they serve **different purposes**. Puppeteer for automation scripts, web scraping, PDF generation, screenshot pipelines, where you don't need a test runner. **Playwright for test**

automation where you need fixtures, retries, parallel execution, and reporting. The APIs are similar enough that switching is easy when the purpose changes.

INTERVIEW STRATEGY

Show you understand the relationship and the right use for each. Lead with the same-team history (it shows you actually know the lineage), then split by purpose: Puppeteer for scripting, Playwright for testing. That purpose-driven split is the senior signal. The mistake is implying Puppeteer is obsolete. Expect the follow-up 'is Puppeteer obsolete?', answer no, it is the right tool for scraping, PDF generation, and screenshot pipelines where no test runner is needed.

How to phrase it

> *Playwright was created by the same team that built Puppeteer at Google, and Playwright is the spiritual successor, it adds Firefox and WebKit support, a built-in test runner with fixtures and parallel execution, the trace viewer, and a generally richer API. Puppeteer remains Chromium-only and is a lower-level automation library without a test runner; you bring your own, like Jest or Mocha. The way I would position them, and the framing that lands in interviews, is that they serve different purposes rather than competing. Puppeteer is the right tool for automation scripts, web scraping, PDF generation, screenshot pipelines, where you don't need a test runner, fixtures, or parallel execution; you need a clean browser-control library you can drop into a script. Playwright is the right tool for test automation where you do need fixtures, retries, parallel execution, and reporting, all the things a real test framework provides. That answers the follow-up about whether Puppeteer is obsolete: it is not, and saying so would be wrong. I use Puppeteer for our PDF generation service in production and Playwright for our test suite, side by side. The APIs are similar enough that switching between them is straightforward when the purpose changes. The pitfall is implying Puppeteer is obsolete because Playwright is newer; they are siblings designed for different jobs.*

Key points to hit

(1) Same team at Google: Playwright is the spiritual successor. **(2)** Puppeteer: Chromium-only, no test runner; Playwright: 3 browsers + runner. **(3)** Different purposes: Puppeteer for scripts, Playwright for tests. **(4)** Use Puppeteer for scraping/PDFs/screenshots; Playwright for the suite. **(5)** The next question: Puppeteer is not obsolete, it's the right tool for scripting. **(6)** Where this goes wrong: implying Puppeteer is dead because Playwright is newer.

CODE

```
// Architectural snapshot: Playwright vs Puppeteer
// Browser support Chromium + Firefox + WebKit Chromium only
// Test runner built-in (@playwright/test) none (use Jest/Mocha)
// Fixtures built-in DI none
// Auto-waiting built-in limited
// Trace viewer yes no
// Best for test automation scripting (scrape/PDF/screenshot)
```

Q 288 LEVEL · Mid to Senior

How do you make the framework-selection decision for a new project?

CONCEPT

Framework selection should be a **structured evaluation**, not personal preference or hype. Five steps that scale: (1) Assess team expertise, TypeScript teams to Playwright, Java teams to Selenium/WebdriverIO, JS-only to Cypress or Playwright. (2) Evaluate application type: modern SPA to Playwright, legacy multi-page to Selenium, native mobile to Appium/WebdriverIO, component library to Cypress CT or Playwright CT. (3) **Existing investment**: 500 Selenium tests means gradual migration, not immediate rewrite. (4) CI requirements: real Safari to Selenium, fast parallel to Playwright, native + web to WebdriverIO. (5) Run a proof of concept with about **10 representative tests** in each candidate, a complex multi-step flow, an API test, and a known-flaky scenario, and measure setup time, execution time, flakiness rate, and lines of code per test. The POC reveals real developer experience better than any benchmark.

INTERVIEW STRATEGY

They are looking to see structured decision-making, not advocacy. Walk the five steps and make the POC the headline, ten representative tests, real measurements, beats any abstract benchmark or vendor pitch. Giving concrete numbers from your own POC (Playwright 2 hours, Selenium 4) makes it lived experience. The trap is committing to a framework based on hype. They usually follow with 'how do you handle stakeholder pressure to pick a specific tool?', answer present POC data; numbers beat opinions.

How to phrase it

> *Framework selection should be a structured evaluation, not personal preference or hype, and I would walk through five steps. First, assess team expertise, TypeScript teams point to Playwright, Java teams to Selenium or WebdriverIO, JavaScript-only teams to Cypress or Playwright, because fighting language is the easiest way to kill adoption. Second, evaluate the application type: modern SPA to Playwright, legacy multi-page to Selenium, native mobile to Appium or WebdriverIO, component library to Cypress CT or Playwright CT. Third, consider existing investment: five hundred Selenium tests is gradual-migration territory, not an immediate rewrite, because the rewrite cost almost always exceeds the framework benefit. Fourth, evaluate CI requirements: real Safari needs Selenium, fast parallel execution favours Playwright, native plus web favours WebdriverIO. But the step I would emphasise, and the answer to the follow-up about handling stakeholder pressure to pick a specific tool, is the fifth: always run a proof of concept. Ten representative tests in each candidate framework, a complex multi-step flow, an API test, and a known-flaky scenario, and I measure setup time, execution time, flakiness rate, and lines of code per test. That reveals the real developer experience better than any benchmark or vendor pitch. For our last new project the Playwright POC took about two hours to set up and produced cleaner tests than the Selenium POC that took four, the decision was obvious from the data, not from advocacy. The thing to avoid is picking a framework on hype or familiarity; the POC numbers are what justify the decision to stakeholders.*

Key points to hit

(1) Five steps: team expertise, app type, existing investment, CI needs, POC. **(2)** Match language: TypeScript -> Playwright, Java -> Selenium/WDIO, JS -> Cypress/Playwright. **(3)** Big existing investment -> gradual migration, not rewrite. **(4)** POC = 10 representative tests; measure setup, runtime, flakiness, LOC. **(5)** Asked next: POC data beats advocacy under stakeholder pressure. **(6)** What loses the point: committing to a framework on hype rather than measured DX.

CODE

```
// Selection process: 5 steps
// 1. Team expertise TS -> Playwright | Java -> Selenium/WDIO | JS -> Cypress/PW
// 2. Application type SPA -> PW | legacy MPA -> Selenium | native mobile -> WDIO
// 3. Existing investment 500 Selenium tests = gradual migration, not rewrite
// 4. CI requirements real Safari -> Selenium | speed -> PW | mobile+web -> WDIO
// 5. Proof of concept 10 representative tests; measure:
// • setup time (PW 2h vs Selenium 4h, our last POC)
// • execution time
// • flakiness rate
// • lines of code per test
```

Q 289 LEVEL · Mid to Senior

How do you make the call between Playwright and a BDD framework (Cucumber, Behave, Robot Framework) for a new project?

CONCEPT

BDD frameworks separate **test intent (Gherkin scenarios)** from **test implementation (step definitions)**. The pitch is that non-technical stakeholders read scenarios and contribute to them; the reality is more nuanced. Three real scenarios where BDD wins. Truly cross-functional collaboration: when product, business, and engineering actively write and maintain scenarios together. Rare but powerful when it happens. Regulated industries: finance, healthcare, where requirements traceability matters and scenarios double as compliance documentation. Heavy reuse of step phrases across many tests, the step library amortises across hundreds of scenarios. Playwright wins everywhere else: faster feedback (no Gherkin layer to maintain), better debugging (the stack trace points at code, not at a Gherkin line), simpler tooling (one less abstraction layer to explain to new joiners). The combination exists: Playwright with @cucumber/cucumber or the **playwright-bdd** adapter, you get Gherkin on top of Playwright's engine. Useful when the team genuinely needs BDD's stakeholder-readable layer but wants Playwright's runtime. The pitfall is BDD as cargo cult: adopted because it sounds professional, but no non-technical person ever reads the scenarios, and the Gherkin layer becomes a maintenance tax with no benefit. Adopt BDD only when the collaboration it enables is real.

INTERVIEW STRATEGY

They want a calibrated BDD answer, not tribalism. Walk the three real wins for BDD (cross-functional collaboration, regulated industries, heavy step reuse), then where Playwright wins (speed, debugging, simplicity). The 'BDD only when collaboration is real' framing marks senior judgement. The pitfall is BDD as cargo cult. The next question is normally ‘what about playwright-bdd?’, answer: real option when the team needs Gherkin AND Playwright's runtime, but rarely the right default.

How to phrase it

> *BDD frameworks separate test intent in Gherkin scenarios from test implementation in step definitions. Given a user is logged in, when they click checkout, then they see the payment form, readable, structured, traceable. The pitch is that non-technical stakeholders read scenarios and contribute to them; the reality is more nuanced and senior interviewers want to see that nuance. Three real scenarios where BDD wins. First, truly cross-functional collaboration. When product managers, business analysts, and engineers actively write and maintain scenarios together, not engineering writing scenarios and business pretending to review them, the Gherkin layer is genuinely valuable as a shared specification language. Rare but powerful when it happens. Second, regulated industries. Finance, healthcare, anywhere requirements traceability matters and scenarios double as compliance documentation that auditors actually read. The Gherkin layer carries audit value beyond testing. Third, heavy reuse of step phrases across many tests. The step library amortises across hundreds of scenarios; a well-designed step definition like 'given a user with role admin' is implemented once and reused widely, which is harder to achieve cleanly without the Gherkin layer. Playwright wins everywhere else. Faster feedback because there's no Gherkin parsing layer to maintain. Better debugging because the stack trace points at code, not at a Gherkin line that maps to code somewhere else. Simpler tooling: one less abstraction layer to explain to new joiners, one less thing to maintain when the framework upgrades. The combination exists: Playwright with at-cucumber slash cucumber or the playwright-bdd adapter, you get Gherkin scenarios on top of Playwright's runtime engine. Useful when the team genuinely needs BDD's stakeholder-readable layer but wants Playwright's speed and capabilities. That's also my answer to the follow-up about playwright-bdd: real option when the team needs Gherkin AND Playwright's runtime, but rarely the right default. The principle that matters most is BDD only when the collaboration it enables is real. Adopting BDD when no non-technical person ever actually reads the scenarios means the Gherkin layer becomes a maintenance tax with no benefit. The pitfall to call out is BDD as cargo cult: adopted because it sounds professional in a meeting, but no business stakeholder writes or reads scenarios in practice, and every test now requires twice the files to maintain for no gain. Honest calibration earns points; tribal preference for either side loses them.*

Key points to hit

(1) BDD wins: real cross-functional collab, regulated industries, heavy step reuse. **(2)** Playwright wins: speed, debugging clarity, simpler tooling. **(3)** Combo: playwright-bdd or @cucumber/cucumber + Playwright for both worlds. **(4)** Rule: adopt BDD only when the collaboration it enables is real. **(5)** Follow-up: playwright-bdd works but is rarely the right default. **(6)** Pitfall: BDD as cargo cult, nobody reads the Gherkin, all maintenance cost.

CODE

```
// PLAYWRIGHT direct (most teams): fast, clear, less to maintain
test('user can complete checkout', async ({ page }) => {
await page.goto('/checkout');
await page.getByLabel('Card number').fill('4242 4242 4242 4242');
await page.getByRole('button', { name: 'Place order' }).click();
await expect(page.getByText('Order confirmed')).toBeVisible();
});

// PLAYWRIGHT-BDD (when stakeholders truly co-author scenarios)
// features/checkout.feature
// Feature: Checkout
// Scenario: User completes checkout
// Given the user is on the checkout page
// When they submit valid payment details
// Then they see the order-confirmation page

// steps/checkout.steps.ts
import { createBdd } from 'playwright-bdd';
const { Given, When, Then } = createBdd();
Given('the user is on the checkout page', async ({ page }) => {
await page.goto('/checkout');
});
When('they submit valid payment details', async ({ page }) => {
await page.getByLabel('Card number').fill('4242 4242 4242 4242');
await page.getByRole('button', { name: 'Place order' }).click();
});
Then('they see the order-confirmation page', async ({ page }) => {
await expect(page.getByText('Order confirmed')).toBeVisible();
});

// Twice the files for the same test. Worth it only if business actually reads .feature
files.
```

Q 290 LEVEL · Junior to Mid

TypeScript vs JavaScript vs Python/Java bindings for Playwright, what's the same across them and why do most teams pick TypeScript?

CONCEPT

Playwright ships official bindings for TypeScript/JavaScript, Python, Java and C#, and the first thing to understand is what stays constant: the browser engine and the protocol are identical across all of them. Every binding drives the same Chromium, Firefox and WebKit builds over the same WebSocket/CDP channel, so auto-waiting, the web-first assertions, network interception, tracing and the locator model behave the same way regardless of language. The API surface is deliberately mirrored: page.getByRole(…).click() reads almost identically in TypeScript and Python. What differs is at the edges: the JS/TS binding gets new features first and has the richest test runner (@playwright/test with fixtures, projects, parallelism); Python and Java are first-class but lean on the host ecosystem's runner (pytest, JUnit/TestNG) and trail slightly on the newest APIs. JavaScript vs TypeScript is not a different binding at all, same package, but TypeScript adds **static typing over the same API**. Most teams pick TypeScript for three honest reasons: Playwright is written in TypeScript and the docs, types and examples are TypeScript-first; the type system catches locator

and config mistakes at author time rather than at 2 a.m. in CI; and the test runner is most mature on this binding. The calibrated point: Python or Java is the right choice when that is your team's language and existing ecosystem, the capability is the same; you are choosing the binding that fits your people, not a weaker tool.

INTERVIEW STRATEGY

They need to hear that the bindings are not different products. Lead with what is constant, same engine, same protocol, same auto-waiting and locators, because that is the depth signal, then name where they diverge: JS/TS gets features first and owns the richest runner. Be clear that JavaScript vs TypeScript is the same binding plus types, not a separate one. Justify TypeScript on real reasons (TS-first docs, compile-time safety, mature runner), and close with the calibration line: pick Python or Java when it fits your team, the capability is identical. The trap is implying the non-JS bindings are second-class. A common follow-up is 'so is the Python binding worse?', answer no, it's first-class and trails only on bleeding-edge features and runner polish.

How to phrase it

> *The first thing I would say is that these are not different products, they are bindings over the same engine, and understanding that is what separates a senior answer from a junior one. Playwright ships official bindings for TypeScript/JavaScript, Python, Java and C-Sharp, and every one of them drives the exact same Chromium, Firefox and WebKit builds over the same WebSocket and Chrome DevTools Protocol channel. So auto-waiting, the web-first assertions, network interception, tracing and the locator model behave identically no matter which language you write in, page.getByRole(...).click() reads almost the same in TypeScript and in Python. The API is deliberately mirrored. Where they differ is at the edges. The JavaScript and TypeScript binding gets new features first and has the richest test runner, @playwright/test, with fixtures, projects and parallelism built in. Python and Java are genuinely first-class, but they lean on the host ecosystem's runner, pytest, JUnit or TestNG, and they trail slightly on the very newest APIs. And to clear up a common confusion, JavaScript versus TypeScript is not a different binding at all; it is the same package, with TypeScript adding static typing over the same API. Now, why do most teams pick TypeScript? Three honest reasons. Playwright itself is written in TypeScript, so the docs, the type definitions and the examples are all TypeScript-first, you are swimming with the current. The type system catches locator and config mistakes at author time, in the editor, rather than at two in the morning in a CI run. And the test runner is most mature on this binding. That said, and this is the follow-up about whether the Python binding is worse, it is not. If my team is a Python shop with pytest and a Python data stack, or a Java shop with TestNG, the right choice is that binding, because the capability is the same and fighting the team's language is the easiest way to kill adoption. You are choosing the binding that fits your people, not a weaker tool. The error to avoid is implying the non-JS bindings are second-class; they share the same engine and the same guarantees.*

Key points to hit

(1) Same engine, same WebSocket/CDP protocol across TS/JS/Python/Java/C#. **(2)** Auto-waiting, web-first assertions, locators, tracing behave identically. **(3)** JS/TS gets new features first and owns the richest runner (@playwright/test). **(4)** JavaScript vs TypeScript = same binding; TS just adds static types. **(5)** TS chosen for: TS-first docs, compile-time safety, most mature runner. **(6)** Calibration: pick Python/Java when it fits the team, capability is identical. **(7)** The common miss:

calling non-JS bindings second-class, they share the same engine.

CODE

```
// Same API, different binding: the click reads the same everywhere

// TypeScript / JavaScript (@playwright/test, richest runner)
import { test, expect } from '@playwright/test';
test('login', async ({ page }) => {
await page.goto('/login');
await page.getByLabel('Email').fill('qa@example.com');
await page.getByRole('button', { name: 'Sign in' }).click();
await expect(page.getByText('Welcome')).toBeVisible();
});

# Python (pytest-playwright): same engine, same protocol
# def test_login(page):
# page.goto('/login')
# page.get_by_label('Email').fill('qa@example.com')
# page.get_by_role('button', name='Sign in').click()
# expect(page.get_by_text('Welcome')).to_be_visible()

// Java (JUnit/TestNG): first-class, host-ecosystem runner
// page.navigate("/login");
// page.getByLabel("Email").fill("qa@example.com");
// page.getByRole(AriaRole.BUTTON,
// new Page.GetByRoleOptions().setName("Sign in")).click();
//
// Constant across all bindings: Chromium/Firefox/WebKit, CDP/WebSocket,
// auto-waiting, web-first assertions, locators, tracing.
// Differs: JS/TS features-first + best runner; JS vs TS = same pkg + types.
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. When is Selenium the better choice over Playwright?

A) Never: Playwright wins everything
B) Only for testing in Internet Explorer
C) Significant existing investment, real Safari via SafariDriver, or Java mandate
D) Whenever the team prefers it

2. Which architectural fact explains most Cypress vs Playwright differences?

A) Cypress was created first
B) Playwright doesn't support TypeScript
C) Cypress uses Selenium internally
D) Cypress runs in-process with the browser; Playwright runs out-of-process via WebSocket

3. When is WebdriverIO the right choice?

A) When web + native iOS + native Android must live in one framework (Appium maturity)
B) Web-only testing on TypeScript teams
C) Only for legacy Selenium migration
D) Never: always pick Playwright

4. How should Puppeteer be positioned vs Playwright?

A) Puppeteer is obsolete
B) Different purposes: Puppeteer for scripting (scrape/PDF/screenshots), Playwright for testing
C) They are interchangeable
D) Puppeteer supports all 3 browsers too

5. What is the single most reliable framework-selection step?

A) Read benchmarks
B) Ask Twitter
C) Run a POC with ~10 representative tests; measure setup time, runtime, flakiness, LOC
D) Pick the newest tool

Section 26 · Answer key

#	Answer	Why and where to revisit
1	C) Significant existing investment, real Safari via SafariDriver, or Java mandate	Both are production-grade; Selenium wins where migration cost exceeds benefit or real Safari is required. *Revisit Section 26.*
2	D) Cypress runs in-process with the browser; Playwright runs out-of-process via WebSocket	In-process gives Cypress its DX wins but limits multi-tab/multi-origin/multi-browser, things Playwright handles natively. *Revisit Section 26.*
3	A) When web + native iOS + native Android must live in one framework (Appium maturity)	WDIO's first-class Appium integration is more mature than any Playwright-based mobile solution. *Revisit Section 26.*
4	B) Different purposes: Puppeteer for scripting (scrape/PDF/screenshots), Playwright for testing	Same team, different jobs: Puppeteer is a lower-level library for scripts; Playwright is a test framework with runner/fixtures. *Revisit Section 26.*
5	C) Run a POC with ~10 representative tests; measure setup time, runtime, flakiness, LOC	A POC reveals real developer experience better than any benchmark or vendor pitch; numbers beat advocacy. *Revisit Section 26.*

SECTION 27

AI, MCP and Agentic Testing

AI, MCP and agentic testing is THE 2026 differentiator in SDET interviews, the topic where strong candidates pull ahead and weak ones reveal they have only read about it. These ten questions go beyond general AI awareness into TypeScript-specific patterns: prompt engineering that exploits type context, the compiler as a hallucination filter, MCP servers built with typed tool schemas, autonomous agents using browser_snapshot accessibility trees, and deterministic typed-output patterns that make AI safe to put in CI. Every answer leans on TypeScript interfaces and union types as the rigour that keeps AI usage production-grade.

In this section

- Prompt engineering with TypeScript context produces typed, compilable tests.
- TypeScript + tsc --noEmit is the free hallucination filter, JS teams lack it.
- MCP is the open protocol connecting AI to tools; Streamable HTTP replaces deprecated SSE.
- Playwright MCP Server gives AI agents browser control with no code written.
- Typed MCP servers and self-healing agents constrain LLM output to valid shapes.

Q 291 LEVEL · Mid to Senior

What is prompt engineering for TypeScript test generation, and how do you do it well?

CONCEPT

Prompt engineering for TypeScript test generation produces accurate, typed, usable Playwright tests, and the quality of generated tests is **entirely determined by prompt quality**. Vague prompts produce untyped happy-path-only tests; strong prompts with TypeScript context produce comprehensive, properly typed coverage. The TypeScript-specific additions that lift quality dramatically: a role framing (senior SDET, 10 years Playwright), application context, the **typed signatures of available page objects** (navigate(): Promise<void>, login(email: string, password: string): Promise<void>), the fixtures import path, a strict-mode note, and explicit acceptance criteria. Close the prompt with constraints: use test.extend fixtures, getByRole locators, Promise<void> return types, no any. The TypeScript advantage JS teams don't have: run tsc --noEmit on the generated file before running it, the compiler is the first hallucination filter.

INTERVIEW STRATEGY

The point of this one is to see disciplined prompting that exploits TypeScript, not paste-and-pray. Lead with the principle, prompt quality determines test quality, then make typed page-object signatures in the prompt the senior technique because that is what gets the AI to return properly typed code. The tsc --noEmit compile-filter is the TypeScript-specific punchline JS teams literally cannot do. The thing to avoid is asking the AI vaguely for a test and shipping what it returns. The follow-up is often 'what makes TypeScript prompts different from JavaScript prompts?', answer you can include typed signatures and the compiler validates the output.

How to phrase it

> *Prompt engineering for TypeScript test generation produces accurate, typed, usable Playwright tests, and the principle I lead with is that the quality of generated tests is entirely determined by prompt quality, vague prompts give untyped happy-path-only tests, strong prompts with TypeScript context give comprehensive typed coverage. The TypeScript-specific additions that dramatically lift quality, and my answer to the follow-up about what makes TypeScript prompts different, are these: a role framing such as senior SDET with ten years of Playwright, application context, the typed signatures of available page objects, navigate returns Promise of void, login takes email string and password string and returns Promise of void, the fixtures import path, an explicit strict-mode note, and clear acceptance criteria. I then close with constraints: use test.extend fixtures from at-fixtures, use getByRole locators, Promise of void return types, no any types, return only the test file with no explanation. With that level of context the model returns code that compiles and uses our patterns rather than generic CSS-selector boilerplate. And here is the TypeScript advantage JavaScript teams literally cannot have: I run tsc --noEmit on the generated file before I even open the test in a browser. If the file does not compile, the generated code has errors, hallucinated methods, wrong types, missing awaits, and I know in seconds rather than after a five-minute test run. TypeScript is my first hallucination filter, and it is free. The trap is asking vaguely and shipping what comes back.*

Key points to hit

(1) Prompt quality decides everything; TypeScript context dramatically lifts it. **(2)** Include typed page-object signatures, fixtures path, strict-mode note. **(3)** Constraints: test.extend fixtures, getByRole, no any, return only the file. **(4)** tsc --noEmit on the generated file = first hallucination filter (JS can't). **(5)** Expect next: typed signatures + compiler validation are the TS-only advantages. **(6)** Avoid: vague prompts producing untyped, happy-path-only tests.

CODE

```
// Strong TypeScript test-generation prompt
const prompt = `
You are a senior SDET with 10 years of Playwright TypeScript experience.
Framework: Playwright + TypeScript, POM, fixtures in src/fixtures/index.ts. strict mode
on.
Page Objects (typed):
LoginPage: navigate(): Promise<void>; login(email: string, password: string):
Promise<void>; getErrorMessage(): Promise<string>
DashboardPage: welcomeHeading: Locator; getStatValue(label: string): Promise<string>
Imports: import { test, expect } from '@fixtures/index';

User Story: Registered user logs in with email and password.
Acceptance Criteria:
- Valid credentials -> redirect /dashboard, welcome heading visible
- Invalid password -> 'Invalid email or password' alert
- Empty email -> 'Email is required'
- 5 failed attempts -> 'Account locked'

Constraints: test.extend fixtures from @fixtures/index. getByRole locators.
Promise<void> return types. No any. test.describe + test.step.
Return only the typed test file. No explanation.`;

// First filter: free, instant, JS teams can't do this
tsc --noEmit src/tests/generated-login.spec.ts
```

Q 292 LEVEL · Mid to Senior

What is LLM hallucination and how does TypeScript protect Playwright test quality?

CONCEPT

Hallucination is when an LLM generates plausible-sounding but incorrect output, a non-existent Playwright method, a hallucinated matcher, wrong types. In TypeScript, hallucinated methods fail the compiler, which makes TypeScript a first-line defence against AI-generated bugs. A three-step filter chain stops most issues before a browser launches: (1) Compile filter, **tsc --noEmit** catches hallucinated Playwright methods immediately with the exact property name and the correct type expected. (2) ESLint filter: **@typescript-eslint/no-explicit-any** catches sneaked-in any types, **playwright/prefer-web-first-assertions** catches one-shot isVisible() in place of retrying toBeVisible(). (3) Visual verify: run in headed mode and confirm the test executes the intended user journey before adding to the suite. JavaScript teams discover hallucinations at runtime; TypeScript teams discover them at compile time, free.

INTERVIEW STRATEGY

You are being asked to show TypeScript framed as a defence layer, not just documentation. Lead with the three-step filter chain, compile, ESLint, visual, because it shows you have a system, not a hope. The compile-time-vs-runtime contrast is the TS-team advantage that lands. The pitfall is treating AI output as trustworthy because it looks right. Expect the follow-up 'what makes TypeScript better than JavaScript here?', answer the compiler catches hallucinated methods immediately; JS finds them at runtime.

How to phrase it

> *Hallucination is when an LLM generates plausible-sounding but incorrect output, a non-existent Playwright method, a hallucinated matcher, the wrong types on a return value. The framing I lead with is that TypeScript is the strongest protection against this in Playwright, and that hallucinated methods fail the compiler, which makes TypeScript a first-line defence rather than an aspirational guard. My process is a three-step filter chain that stops most issues before a browser launches. Step one, compile filter. I run tsc --noEmit on every AI-generated file and the compiler immediately flags any hallucinated Playwright method with the exact property name and the correct type expected. Step two, ESLint filter. no-explicit-any catches the model sneaking in any types to make code compile, and prefer-web-first-assertions catches isVisible point-in-time checks in place of retrying toBeVisible. Step three, visual verify. I run the test in headed mode and confirm it executes the intended user journey before adding it to the suite. That answers the follow-up about what makes TypeScript better than JavaScript here, which is the senior point. in TypeScript I find these errors at compile time, instantly, with no test run; in JavaScript you find them at runtime, after a slow test fail, with a less specific error. I describe TypeScript to interviewers as the free hallucination filter, it costs me nothing to enable and it catches an entire class of AI bugs before they cost me anything else. Where this goes wrong is trusting AI output because it looks plausible; the filter chain is non-negotiable.*

Key points to hit

(1) Hallucination = plausible-sounding but incorrect AI output. **(2)** TypeScript is first-line defence: hallucinated methods fail the compiler. **(3)** Three-step filter: compile (tsc --noEmit), ESLint, visual verify. **(4)** no-explicit-any + prefer-web-first-assertions catch sneaked anys and isVisible. **(5)** They usually ask next: TS catches at compile time; JS only at runtime. **(6)** The easy mistake: trusting AI output because it looks right, the filter chain is mandatory.

CODE

```
// Three-step hallucination filter chain

// 1. Compile filter: catches hallucinated methods
tsc --noEmit src/tests/ai-generated.spec.ts
// error TS2339: Property 'waitForNetworkIdle' does not exist on type 'Page'

// 2. ESLint filter: enforce AI hygiene
// .eslintrc.json
{
"rules": {
"@typescript-eslint/no-explicit-any": "error", // no sneaked anys
"playwright/prefer-web-first-assertions": "error", // toBeVisible() not isVisible()
"@typescript-eslint/no-floating-promises": "error" // every action awaited
}
}

// 3. Visual verify: run headed once before adding to the suite
npx playwright test src/tests/ai-generated.spec.ts --headed
```

Q 293 LEVEL · Mid to Senior

What is MCP (Model Context Protocol) and why does it matter for TypeScript SDETs?

CONCEPT

MCP, Model Context Protocol, is an open standard initiated by Anthropic and now governed by the open **modelcontextprotocol.io specification**, with broad industry adoption from Microsoft, Google and others. It defines how AI models connect to external tools, data sources, and systems: read files, execute code, query databases, control browsers, call APIs. For TypeScript SDETs, MCP is transformative: an AI agent can control a real browser via Playwright, read HTML reports, query CI failure logs, generate typed test data, and run the suite, all through a standardised typed protocol. The 2026 depth detail worth knowing: the transport for remote servers is **Streamable HTTP**; SSE was deprecated in March 2025. Knowing that shift signals you're tracking the protocol actively rather than reading 2024 tutorials, a quiet credibility marker.

INTERVIEW STRATEGY

What scores here is showing whether you understand MCP as infrastructure or just as a buzzword. Frame it as the open standard for AI-to-tool connection, with the Anthropic origin and current open governance. The killer detail is the Streamable-HTTP-replaced-SSE-in-March-2025 fact, it shows you actually track the spec. The classic error is describing MCP as an Anthropic-only thing. They usually follow with ‘what transport does MCP use?’, answer Streamable HTTP; SSE is deprecated.

How to phrase it

> *MCP, Model Context Protocol, is an open standard initiated by Anthropic and now governed by the open modelcontextprotocol.io specification, with broad industry adoption from Microsoft, Google and others. The key idea is that it defines how AI models connect to external tools, data sources, and systems, read files, execute code, query databases, control browsers, call APIs, through one standardised protocol. For TypeScript SDETs that is genuinely transformative, and this is the framing I would emphasise: before MCP, connecting an AI to a test framework required custom API wrappers per tool, every integration bespoke. MCP standardises this. Any MCP-compatible AI connects to any MCP server using the same protocol. The analogy I use in interviews is that MCP is to AI agents what WebDriver is to browser automation: the protocol layer that made an ecosystem possible. The TypeScript MCP SDK means I can build typed MCP servers and clients with full IDE support, treating tool interfaces as TypeScript interfaces. And here is the depth detail that signals I track the spec actively, and my answer to the follow-up about transport: for remote MCP servers the transport is Streamable HTTP, not SSE. SSE was deprecated in March 2025. Mentioning that shift quietly signals I'm reading the spec, not 2024 tutorials, which lands well in 2026 interviews. The classic error is describing MCP as Anthropic-only; it is open and broadly adopted, and that breadth is what makes it durable infrastructure rather than a single vendor's feature.*

Key points to hit

(1) Open standard at modelcontextprotocol.io; broad industry adoption. **(2)** Defines how AI connects to tools/data/systems: files, code, DBs, browsers, APIs. **(3)** For SDETs: agents control browsers via Playwright, query CI, run suites. **(4)** Analogy: MCP is to AI agents what WebDriver is to browser automation. **(5)** The next question: transport is Streamable HTTP; SSE deprecated March 2025. **(6)** Trap: calling MCP Anthropic-only, it is open and broadly governed.

CODE

```
// MCP interaction flow: typical TS SDET use
// 1. AI agent receives: "Test login on staging, report issues"
// 2. AI connects to Playwright MCP server (TypeScript MCP client)
// 3. AI calls browser_navigate({ url: 'https://staging.example.com/login' })
// 4. MCP executes in a real Chromium browser via Playwright
// 5. AI reads browser_snapshot (accessibility tree: no vision model)
// 6. AI calls browser_type({ element: 'Email field', target: 'e2', text:
'test@example.com' })
// 7. AI decides next action autonomously
// 8. AI reports: "Sign-in button stays disabled with empty email, regression"
//
// Transport (2026): Streamable HTTP (SSE deprecated March 2025)
```

Q 294 LEVEL · Mid to Senior

What is the Playwright MCP Server and how do you use it in a TypeScript workflow?

CONCEPT

The **Playwright MCP Server** is an official MCP server that exposes Playwright browser automation as tools an AI agent can call, navigate, click, type, screenshot, snapshot, without writing any test code. You configure it in Claude Desktop or call it programmatically from TypeScript via the MCP SDK. The detail worth knowing in interviews: **browser_snapshot returns the structured accessibility tree** (roles, labels, states), **not pixels**, so the agent reasons semantically over structure rather than needing a vision model on a screenshot. That makes it faster, cheaper, and more reliable than browser_take_screenshot for agentic testing. A real example: ask Claude to check if checkout works for a new user on mobile, it navigates, fills, switches viewport, finds the payment button is hidden, a regression a desktop suite would miss for sprints.

INTERVIEW STRATEGY

This question is there to see hands-on knowledge, not theory. Lead with the practical setup (npx @playwright/mcp@latest, configure in Claude Desktop) and a real story, the mobile checkout regression Claude found. The killer detail is browser_snapshot vs browser_take_screenshot: structured accessibility tree vs pixels, semantic vs vision. The classic error is treating MCP browser tools as overhyped, they genuinely work in 2026. The next question is normally 'why use browser_snapshot instead of screenshot?', answer it returns a structured tree the agent can reason over without a vision model.

How to phrase it

> *The Playwright MCP server is an official MCP server that exposes Playwright browser automation as tools an AI agent can call, browser_navigate, browser_click, browser_type, browser_take_screenshot, browser_snapshot, without writing any test code. I configure it in Claude Desktop with the npx at-playwright-slash-mcp at latest invocation, or I call it programmatically from TypeScript using the MCP SDK with a StdioClientTransport. The detail I always emphasise, and my answer to the follow-up about why I use browser_snapshot instead of browser_take_screenshot, is that snapshot returns the structured accessibility tree, roles, labels, states, not pixels. So the agent reasons semantically over the page structure rather than needing a vision model on a screenshot, which is faster, cheaper, and more reliable for agentic testing. Screenshot is for the cases where I genuinely need visual evidence; snapshot is for the cases where I need the agent to make decisions. A real example I share to make this concrete: I asked Claude to check if checkout works for a new user on mobile. Claude navigated, filled the forms, switched to an iPhone viewport, found the payment button was hidden behind the bottom nav, a regression our desktop-only regression suite had missed for two sprints. Setup was about ten minutes. The mistake is dismissing this as overhyped; in 2026 it genuinely works for exploratory testing and bug hunting, and it complements rather than replaces the deterministic regression suite.*

Key points to hit

(1) Official MCP server exposing Playwright browser tools to AI agents. **(2)** Configure in Claude Desktop OR call programmatically via the TS MCP SDK. **(3)** browser_snapshot returns accessibility tree (semantic), not pixels. **(4)** Snapshot for agent reasoning; screenshot for visual evidence only. **(5)** Asked next: snapshot is faster/cheaper/more reliable, no vision model needed. **(6)** Real win: AI found mobile checkout regression missed for 2 sprints.

CODE

```
// Configure in Claude Desktop (claude_desktop_config.json)
// {
// "mcpServers": {
// "playwright": {
// "command": "npx",
// "args": ["@playwright/mcp@latest", "--browser", "chromium"]
// }
// }
// }

// Programmatic TS usage
import { Client } from '@modelcontextprotocol/sdk/client/index.js';
import { StdioClientTransport } from '@modelcontextprotocol/sdk/client/stdio.js';

const transport = new StdioClientTransport({
command: 'npx', args: ['@playwright/mcp@latest'],
});
const client = new Client({ name: 'ts-test-agent', version: '1.0.0' }, { capabilities:
{} });
await client.connect(transport);

await client.callTool({ name: 'browser_navigate', arguments: { url:
'https://staging.example.com/login' } });
await client.callTool({ name: 'browser_type', arguments: { element: 'Email field',
target: 'e2', text: 'admin@test.com' } });
await client.callTool({ name: 'browser_click', arguments: { element: 'Sign in button',
target: 'e5' } });

// browser_snapshot: accessibility tree (semantic), not a pixel screenshot
const snapshot = await client.callTool({ name: 'browser_snapshot' });
```

Q 295 LEVEL · Senior

How do you build a custom MCP server in TypeScript that exposes your Playwright framework?

CONCEPT

A custom TypeScript MCP server exposes your specific test capabilities, running Playwright tests, reading reports, generating typed test data, checking CI status, as tools an AI agent can call with full type safety. The server uses the MCP SDK's Server class, registers tools via **setRequestHandler(ListToolsRequestSchema, ...)**, and handles calls via **setRequestHandler(CallToolRequestSchema, ...)**. Every tool argument should have a TypeScript interface (RunTestsArgs, CreateUserArgs) and every result a typed shape (TestSummary). Concrete value: a developer asks Claude in Slack 'are smoke tests passing on staging?', Claude calls the MCP server, runs the suite, returns a typed summary with pass/fail counts and the first failure messages. Cuts the deploy go/no-go discussion from ten minutes to thirty seconds. For remote deployment, use **StreamableHTTPServerTransport**, not the deprecated SSE transport.

INTERVIEW STRATEGY

They want to see actual MCP-server construction, not theory. Walk through the SDK's Server class, ListTools and CallTool handlers, and typed argument interfaces. Make typed argument schemas

the senior point because they constrain LLM output. Quantify the value (10-minute CI check to 30-second AI query) so it lands. The 2026 detail, StreamableHTTPServerTransport, not SSE, is the credibility marker. The pitfall is describing MCP servers without ever building one. A common follow-up is 'how do remote agents connect?', answer Streamable HTTP transport; SSE is deprecated.

How to phrase it

> *A custom TypeScript MCP server exposes my specific test capabilities, running Playwright tests, reading reports, generating typed test data, checking CI status, as tools an AI agent can call with full type safety. The server uses the MCP SDK's Server class. I register tools via setRequestHandler with ListToolsRequestSchema, returning each tool's name, description, and inputSchema, and I handle calls via setRequestHandler with CallToolRequestSchema. The detail that makes this production-grade, and what I would emphasise, is typed interfaces on every argument and result, a RunTestsArgs interface, a CreateUserArgs interface, a TestSummary result interface. Those make the server self-documenting, refactor-safe, and constrain what the LLM can validly call. The concrete value I present, because it lands with leadership: a developer in Slack asks Claude 'are smoke tests passing on staging?', Claude calls our MCP server, runs the tests, and responds with a typed summary, passed and failed counts plus the first five failure titles and error messages. No CI login required. This cut our deployment go-no-go discussion from a ten-minute CI check to a thirty-second AI query, which is concrete ROI on the work. For remote deployment of the MCP server, and my answer to the follow-up about how remote agents connect, I use StreamableHTTPServerTransport, not the deprecated SSE transport from the 2024 examples. That is the production-grade pattern for 2026. Building a typed MCP server like this is what signals both MCP knowledge and TypeScript maturity to an interviewer.*

Key points to hit

(1) Custom server exposes test capabilities as AI-callable tools. **(2)** MCP SDK Server class; ListTools + CallTool handlers. **(3)** Typed arg interfaces (RunTestsArgs, CreateUserArgs) + typed results (TestSummary). **(4)** Concrete win: 10-min CI check -> 30-sec Slack-to-Claude query. **(5)** Follow-up: remote = StreamableHTTPServerTransport, not deprecated SSE. **(6)** Where this goes wrong: describing MCP servers theoretically without ever building one.

CODE

```
import { Server } from '@modelcontextprotocol/sdk/server/index.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { CallToolRequestSchema, ListToolsRequestSchema, Tool } from
'@modelcontextprotocol/sdk/types.js';
import { execSync } from 'child_process';

interface RunTestsArgs { grep?: string; project?: 'chromium'|'firefox'|'webkit'; }
interface TestFailure { title: string; error: string; file: string; }
interface TestSummary { passed: number; failed: number; durationMs: number; failures:
TestFailure[]; }

const server = new Server({ name: 'playwright-ts', version: '1.0.0' }, { capabilities: {
tools: {} } });

server.setRequestHandler(ListToolsRequestSchema, async (): Promise<{ tools: Tool[] }> =>
({
tools: [{
name: 'run_playwright_tests',
description: 'Run Playwright tests and return a typed summary',
inputSchema: { type: 'object', properties: { grep: { type: 'string' }, project: { type:
'string', enum: ['chromium','firefox','webkit'] } } },
}],
}));

server.setRequestHandler(CallToolRequestSchema, async (req) => {
const args = req.params.arguments as RunTestsArgs;
const out = execSync(`npx playwright test ${args.grep ? `--grep "${args.grep}"` : ''}
--reporter=json`, { encoding: 'utf8' });
const raw = JSON.parse(out);
const summary: TestSummary = { passed: raw.stats?.expected ?? 0, failed:
raw.stats?.unexpected ?? 0, durationMs: raw.stats?.duration ?? 0, failures: [] };
return { content: [{ type: 'text', text: JSON.stringify(summary, null, 2) }] };
});

await server.connect(new StdioServerTransport());
// Remote deployment: use StreamableHTTPServerTransport (SSE deprecated 2025-03)
```

Q 296 LEVEL · Senior

What are AI agents and how do you implement them in TypeScript for test automation?

CONCEPT

An AI agent autonomously perceives its environment, decides on actions, executes them, and evaluates outcomes, looping until a goal is achieved. The four-stage loop: **Perceive**, read the accessibility snapshot from Playwright MCP (structured tree, no vision model required); **Decide**, select the next browser tool to call based on current state and goal; Act, MCP executes the tool call in a real Chromium browser via Playwright; **Evaluate**, observe the result, decide to continue or conclude with findings. TypeScript's type system makes agent implementation more reliable by typing the tool-call message structures and result shapes. The right use case in 2026: exploratory testing and bug hunting, not regression, because agents are non-deterministic by design and regression needs to be repeatable.

INTERVIEW STRATEGY

Your job here is to show structured agent thinking, not buzzword name-dropping. Walk the four-stage loop, Perceive, Decide, Act, Evaluate, with browser_snapshot as the Perceive step (no vision model). What separates a senior answer is calling out the right use case (exploratory, not regression) because it shows you know agents are non-deterministic. The thing to avoid is using an agent for the regression suite. The follow-up is often 'would you run an agent in CI for regression?', answer no; agents for exploration, deterministic Playwright tests for regression.

How to phrase it

> *An AI agent autonomously perceives its environment, decides on actions, executes them, and evaluates outcomes, looping until a goal is achieved. The four-stage loop I would walk through is perceive, decide, act, evaluate. Perceive: Claude reads the accessibility snapshot from the Playwright MCP server via browser_snapshot, which gives a structured tree of roles, labels and states rather than pixels, so no vision model is required. Decide: Claude selects the next browser tool to call based on the current state and the goal. Act: MCP executes the tool call in a real Chromium browser via Playwright. Evaluate: Claude observes the result and decides whether to continue the loop or conclude with findings. TypeScript's type system makes this more reliable by typing the tool-call message structures and result shapes, so agent behaviour stays disciplined. But the framing that matters most, and my answer to the follow-up about running an agent in CI for regression, is that agents belong to exploratory testing and bug hunting, not regression. The reason is that agents are non-deterministic by design, they make different decisions on different runs, which is exactly the opposite of what a regression suite needs. A regression test must do the same thing every time and assert the same outcome. An exploratory agent finds bugs we did not know to write tests for, which is where it pays off. So agents for exploration; deterministic Playwright tests for regression. Where this goes wrong is dropping an agent into the regression suite and watching it fail flakily for reasons that are not bugs.*

Key points to hit

(1) Four-stage loop: Perceive, Decide, Act, Evaluate. **(2)** Perceive via browser_snapshot (semantic tree, no vision model). **(3)** TS types make agents reliable: typed tool-call messages and results. **(4)** Use for exploratory/bug-hunting; agents are non-deterministic by design. **(5)** Expect next: NO agents in CI for regression, keep that deterministic. **(6)** What loses the point: using an agent where deterministic regression is needed.

CODE

```
// Four-stage agent loop: typed
interface AgentStep {
perception: { snapshot: unknown }; // 1. Perceive
decision: { tool: string; args: unknown };// 2. Decide
action: { result: unknown }; // 3. Act
evaluation: { complete: boolean; finding?: string }; // 4. Evaluate
}

async function exploreCheckout(client: MCPClient): Promise<string[]> {
const findings: string[] = [];
while (true) {
const snapshot = await client.callTool({ name: 'browser_snapshot' }); // Perceive
const next = await claudeDecide({ goal: 'verify checkout works on mobile', snapshot });
if (next.done) { findings.push(...next.findings); return findings; }
await client.callTool({ name: next.tool, arguments: next.args }); // Act
// Evaluate happens inside claudeDecide on the next loop iteration
}
}
// Use for exploratory testing. NEVER for regression: non-deterministic by design.
```

Q 297 LEVEL · Mid to Senior

How do you test AI-powered features using Playwright TypeScript?

CONCEPT

Testing AI features, LLM chatbots, recommendation engines, content generation, requires different strategies because outputs are **non-deterministic by design**. Three rules that hold up: Assert structure, not exact text, never assert an exact generated string, assert the category, schema validity, and presence of relevance keywords. **Use TypeScript interfaces for response schemas**, cast the response to a ChatApiResponse interface and validate fields with type guards; if the AI service changes its schema, TypeScript flags every assertion that touches a changed field, giving you structural contract testing at the type level. **Allow generous timeouts**: AI responses take longer than standard API calls; use 15-second timeouts on visibility assertions for AI-powered UI elements. Together these turn a non-deterministic feature into something you can reliably test.

INTERVIEW STRATEGY

Show you handle non-determinism without giving up rigour. Lead with the three rules, structure not exact text, typed schema, generous timeouts, and stress the typed-interface idea as structural contract testing. The trap is asserting on exact generated wording, which makes the test permanently flaky. Expect the follow-up ‘how do you make a non-deterministic test reliable?’, answer assert category, schema and relevance keywords, never exact text.

How to phrase it

> *Testing AI features, chatbots, recommendation engines, content generation, requires different strategies because the outputs are non-deterministic by design. I follow three rules that hold up, and they answer the follow-up about making a non-deterministic test reliable. First, assert structure, not exact text. I never assert that the AI returned a specific string, because that test will flake the moment the model changes wording. Instead I assert the category is correct, the schema is valid, and the response contains relevance keywords, for a billing query I check the category is billing and the reply contains some subset of refund, return, policy, contact. Second, use TypeScript interfaces for response schemas, I define ChatApiResponse with reply, category, confidence and processingTimeMs, cast the response to it, and TypeScript validates every field access. The point worth emphasising is that this is structural contract testing at the type level: if the AI service changes its response shape, adds a field, renames one, changes a type, TypeScript flags every assertion that touches the changed field, before any test runs. JavaScript teams find out at runtime. Third, allow generous timeouts. AI responses are noticeably slower than ordinary API calls, so I use about fifteen-second timeouts on visibility assertions for AI-powered UI elements rather than the default. Together these three rules turn a non-deterministic feature into something I can reliably test. The mistake is asserting on exact generated wording, the test will flake the next time the model nudges its phrasing, and the test does not actually verify the behaviour I care about.*

Key points to hit

(1) Assert structure, not exact text: category + schema + relevance keywords. **(2)** Use TS interfaces for AI response schemas: structural contract at type level. **(3)** TS catches schema-change drift at compile time; JS teams catch it at runtime. **(4)** Generous timeouts (~15s) for AI-powered visibility assertions. **(5)** They usually ask next: assert category/schema/keywords, never exact text. **(6)** The common miss: asserting exact generated wording, test flakes on every prompt nudge.

CODE

```
test('AI chatbot handles billing query', async ({ request }) => {
const res = await request.post('/api/chat', {
data: { message: 'How do I get a refund?', sessionId: `t-${Date.now()}` },
});
expect(res.status()).toBe(200);

interface ChatResponse {
reply: string; category: string; confidence: number; processingTimeMs: number;
}
const body = await res.json() as ChatResponse;

// Assert structure, not exact text
expect(body.category).toBe('billing');
const relevant = ['refund', 'return', 'policy', 'contact', 'days'];
expect(relevant.some(kw => body.reply.toLowerCase().includes(kw))).toBe(true);

expect(body.reply.length).toBeGreaterThan(50);
expect(body.reply.length).toBeLessThan(2000);
expect(body.confidence).toBeGreaterThanOrEqual(0);
expect(body.confidence).toBeLessThanOrEqual(1);
expect(body.processingTimeMs).toBeLessThan(5000);
});
```

Q 298 LEVEL · Senior

What is model drift and how do you detect it with typed Playwright TypeScript tests?

CONCEPT

Model drift is when an AI model's behaviour **changes over time**, the same input that returned a billing-category response last month now returns something different, or the response time slowly creeps up, or the wording shifts away from your expected concepts. Detection is a typed golden dataset: define a DriftScenario interface (input, expectedCategory, mustContain keywords, mustNotContain keywords, maxResponseTimeMs), declare the dataset **as const** so each string becomes a literal type and the array becomes readonly, and run the same suite nightly on a cron. Any test failure indicates drift: the model changed enough to fail the structural assertions. Alert the team the next morning, before users notice unexpected AI behaviour. **as const** is the TypeScript detail worth stating: it converts mustContain to readonly-tuple-of-literal-strings, so accidental mutation is a compile error.

INTERVIEW STRATEGY

They are looking to see proactive monitoring of non-deterministic systems. Lead with what drift actually is, then make the typed golden dataset the centrepiece, DriftScenario interface, readonly array as const, nightly cron. The as const detail is a TypeScript signal that lands with depth interviewers. The pitfall is detecting drift only when users complain. They usually follow with 'what does as const buy you?', answer literal types and immutable arrays, the dataset cannot be accidentally mutated.

How to phrase it

> *Model drift is when an AI model's behaviour changes over time, the same input that returned a billing-category response last month now returns something different, or the response time slowly creeps up, or the wording shifts away from concepts I expect. I detect it with a typed golden dataset that runs on a schedule. I define a DriftScenario interface with input, expectedCategory, mustContain keywords, mustNotContain keywords, and maxResponseTimeMs. I declare the dataset itself as a readonly array of DriftScenario and add as const, and then I parameterise tests across it. The TypeScript detail worth calling out, and my answer to the follow-up about what as const buys you, is that it makes every string in the dataset a literal type and the array itself readonly. So mustContain becomes readonly tuple of 'cancel' and 'subscription', not just string array, the dataset cannot be accidentally mutated, and any drift in the expected schema is a compile error. The suite runs nightly on a cron: GitHub Actions schedule at two a.m., and any failure indicates drift, because the model changed enough to fail the structural assertions. I alert the team in the morning, before users notice that the AI is behaving differently. The typed ChatApiResponse interface means that if the AI service changes its response shape, so behaviour and actual schema, TypeScript flags every drift test that touches the changed field, so I find that out at compile time, not at three a.m. during the cron run. Where this goes wrong is detecting drift only when users complain, by which point trust is already lost; the typed nightly suite catches it the day it happens.*

Key points to hit

(1) Drift = AI behaviour changing over time (category, wording, timing). **(2)** DriftScenario interface: input, expected category, must/must-not contain, timeout. **(3)** Dataset 'as const' = literal types + readonly: no accidental mutation. **(4)** Run nightly on cron; any failure alerts the team before users notice. **(5)** The next question: 'as const' gives literal types and immutability, schema-safe. **(6)** Avoid: detecting drift only after users complain, trust is already lost.

CODE

```
import { test, expect } from '@playwright/test';

interface DriftScenario {
readonly input: string;
readonly expectedCategory: string;
readonly mustContain: readonly string[];
readonly mustNotContain: readonly string[];
readonly maxResponseTimeMs: number;
}
interface ChatApiResponse { reply: string; category: string; processingTimeMs: number; }

const DATASET: readonly DriftScenario[] = [
{ input: 'How do I cancel my subscription?', expectedCategory: 'billing',
mustContain: ['cancel','subscription'], mustNotContain: ['error'], maxResponseTimeMs:
5000 },
{ input: "My order hasn't arrived after 10 days", expectedCategory: 'shipping',
mustContain: ['order','delivery','track'], mustNotContain: ['error'], maxResponseTimeMs:
5000 },
] as const;

for (const s of DATASET) {
test(`Drift: "${s.input.slice(0, 40)}…"`, async ({ request }) => {
const t0 = Date.now();
const res = await request.post('/api/chat', { data: { message: s.input } });
const body = await res.json() as ChatApiResponse;
expect(body.category).toBe(s.expectedCategory);
for (const kw of s.mustContain) expect(body.reply.toLowerCase()).toContain(kw);
for (const kw of s.mustNotContain) expect(body.reply.toLowerCase()).not.toContain(kw);
expect(Date.now() - t0).toBeLessThan(s.maxResponseTimeMs);
});
}
// schedule: cron: "0 2 * * *" # 2 AM daily
```

Q 299 LEVEL · Senior

How do you build a TypeScript self-healing agent for Playwright test maintenance?

CONCEPT

A self-healing agent uses TypeScript types to structure the healing suggestion, integrates with the Playwright fixture system, and uses the Anthropic SDK for diagnosis. The pattern: a HealingSuggestion interface with a **locatorType union type**, 'getByRole' | 'getByLabel' | 'getByTestId' | 'locator', a locatorValue string, a confidence level, and an explanation. Include this interface

definition in the prompt: telling the LLM to return JSON matching the TypeScript interface constrains it to the four valid locator types. A **type-guard function** (isHealingSuggestion) validates the parsed JSON at runtime, rejecting any hallucinated shape. The policy that distinguishes production use: **high-confidence getByRole suggestions can be auto-applied**; low-confidence locator() suggestions always route to human review. Auto-apply is for the boring fixes; humans stay in the loop on the ambiguous ones.

INTERVIEW STRATEGY

The point of this one is to see you combine LLM output with TypeScript discipline. Lead with the HealingSuggestion interface, especially the locatorType union, because that is the trick: declaring the union and including it in the prompt constrains the LLM to valid values. The high-confidence-auto vs low-confidence-review policy marks senior judgement because it shows judgement. Where this goes wrong is auto-applying every healing suggestion. The next question is normally 'what stops the AI returning a bad locator?', answer the union type in the prompt plus a runtime type guard on the parsed JSON.

How to phrase it

> *A self-healing agent uses TypeScript types to structure the healing suggestion clearly, integrates with the Playwright fixture system, and uses the Anthropic SDK for diagnosis. The pattern I use, and the part I would emphasise as the trick, is a HealingSuggestion interface with a locatorType union type, the literal union of getByRole, getByLabel, getByTestId, and locator, plus locatorValue, a confidence level of high or medium or low, and an explanation. The TypeScript-specific move is including that interface definition in the LLM prompt: when I tell the model to return JSON matching the interface, with the union type spelled out, the AI respects the constraint and only returns one of the four valid values. That answers the follow-up about what stops the AI returning a bad locator: the union in the prompt plus a runtime type-guard function, isHealingSuggestion, that validates the parsed JSON before I trust it. If the AI hallucinates a fifth locator type, the guard rejects it and the suggestion is discarded. The policy that distinguishes production use, and the senior judgement signal, is the auto-versus-review split. High-confidence getByRole suggestions can be auto-applied, those are the boring fixes, like a renamed button or a removed comma. Low-confidence locator suggestions, the CSS-selector ones, always route to human review because they are ambiguous and getting them wrong silently breaks tests in ways that are hard to debug. Auto-apply for the obvious; humans in the loop on the ambiguous. The pitfall is auto-applying every healing suggestion regardless of confidence, fast in the short term, painful in the long term when a wrong fix silently masks a real bug.*

Key points to hit

(1) HealingSuggestion interface with locatorType union: getByRole/Label/TestId/locator. **(2)** Include the interface in the prompt to constrain the LLM's valid outputs. **(3)** Runtime type guard (isHealingSuggestion) rejects hallucinated shapes. **(4)** Policy: high-confidence getByRole auto-applies; low-confidence routes to human. **(5)** Asked next: what stops bad locators, union in prompt + type guard. **(6)** The easy mistake: auto-applying every healing suggestion regardless of confidence.

CODE

```
import { test as base } from '@playwright/test';
import Anthropic from '@anthropic-ai/sdk';

interface HealingSuggestion {
locatorType: 'getByRole' | 'getByLabel' | 'getByTestId' | 'locator';
locatorValue: string;
confidence: 'high' | 'medium' | 'low';
explanation: string;
}
function isHealingSuggestion(o: unknown): o is HealingSuggestion {
if (typeof o !== 'object' || o === null) return false;
const s = o as Record<string, unknown>;
return ['getByRole','getByLabel','getByTestId','locator'].includes(s.locatorType as
string)
&& typeof s.locatorValue === 'string'
&& ['high','medium','low'].includes(s.confidence as string);
}

export const test = base.extend<{ healingPage: typeof base }>({
healingPage: async ({ page }, use) => {
const client = new Anthropic();
page.on('pageerror', async (err) => {
if (!err.message.includes('TimeoutError')) return;
const snapshot = await page.locator('body').ariaSnapshot();
const r = await client.messages.create({
model: 'claude-sonnet-4-5', max_tokens: 512,
messages: [{ role: 'user', content: `Failing locator. Snapshot:
${JSON.stringify(snapshot)}.
Error: ${err.message}.
Return JSON matching: { locatorType: 'getByRole'|'getByLabel'|'getByTestId'|'locator';
locatorValue: string; confidence: 'high'|'medium'|'low'; explanation: string }` }],
});
const raw: unknown = JSON.parse(r.content[0].type === 'text' ? r.content[0].text :
'{}');
if (isHealingSuggestion(raw)) {
// high-confidence getByRole -> auto-apply; everything else -> human review
}
});
await use(page as any);
},
});
```

Q 300 LEVEL · Senior

How do you use AI in CI quality gates with typed structured output?

CONCEPT

An AI-powered CI quality gate analyses test failures and makes a typed deployment recommendation, a TypeScript-first production pattern that removes manual triage from the PR process. The structure: a **FailureCategory union type** ('APPLICATION_BUG' | 'TEST_BUG' | 'ENVIRONMENT_ISSUE' | 'NEEDS_INVESTIGATION') and a **CIFailureAnalysis interface** (category, confidence 0-1, affectedTests, deployRecommendation: BLOCK/WARN/PASS, reasoning). Include the union and interface in the LLM prompt so the model can only return one of the four categories. A **runtime type-guard** (isCIFailureAnalysis) rejects malformed responses. Mapping is deterministic

and reviewable: APPLICATION_BUG -> BLOCK deployment, TEST_BUG -> WARN and route to the test author, ENVIRONMENT_ISSUE -> WARN and retry, NEEDS_INVESTIGATION -> human required. Real ROI: ~80% reduction in manual triage, the number that justifies the investment to leadership.

INTERVIEW STRATEGY

You are being asked to show AI used for a high-stakes production decision, safely. Lead with the FailureCategory union and CIFailureAnalysis interface in the prompt, that is what constrains the LLM to valid output. The deterministic category-to-action mapping (BLOCK/WARN/PASS) is what makes this reviewable. Anchor the ROI, 80% triage reduction, because that is what justifies the investment. Where this goes wrong is letting the AI decide deployment without a constrained output shape. A common follow-up is 'how do you stop the AI making a wrong call?', answer the union type + deterministic action mapping plus human review on NEEDS_INVESTIGATION.

How to phrase it

> *An AI-powered CI quality gate analyses test failures and makes a typed deployment recommendation, which is the pattern that removes manual triage from the PR process. The TypeScript structure is what makes this production-grade. I define a FailureCategory union type, APPLICATION_BUG, TEST_BUG, ENVIRONMENT_ISSUE, NEEDS_INVESTIGATION, and a CIFailureAnalysis interface with category, confidence as a number between zero and one, affectedTests, deployRecommendation as BLOCK or WARN or PASS, and reasoning. I include the union and the interface in the LLM prompt: telling the model to return JSON matching the interface constrains it to one of the four valid categories. A runtime type-guard function, isCIFailureAnalysis, rejects malformed responses before they reach the pipeline. The mapping from category to action is deterministic and reviewable, which is my answer to the follow-up about how I stop the AI making a wrong call: APPLICATION_BUG maps to BLOCK deployment, TEST_BUG maps to WARN and routes to the test author, ENVIRONMENT_ISSUE maps to WARN and retries in a clean environment, NEEDS_INVESTIGATION always requires human review. The AI categorises; the deterministic mapping decides; the human stays in the loop on ambiguity. The ROI I present to leadership, because numbers are what get this funded, is around eighty percent reduction in manual triage across the PR process. That is concrete value, with the AI boxed into a safe role by typed output. Where this goes wrong is letting the AI decide deployment without a constrained output shape, unconstrained LLM output on a deployment decision is exactly where you get hallucinated recommendations that take down production.*

Key points to hit

(1) FailureCategory union: APPLICATION_BUG / TEST_BUG / ENVIRONMENT_ISSUE / NEEDS_INVESTIGATION. **(2)** CIFailureAnalysis interface in the LLM prompt constrains valid output. **(3)** Runtime type guard rejects malformed JSON before it reaches the pipeline. **(4)** Deterministic action mapping: BLOCK / WARN / PASS, plus human on NEEDS_INVESTIGATION. **(5)** Concrete ROI: ~80% reduction in manual PR triage, justifies investment. **(6)** Trap: unconstrained LLM output making deployment decisions directly.

CODE

```
type FailureCategory = 'APPLICATION_BUG' | 'TEST_BUG' | 'ENVIRONMENT_ISSUE' |
'NEEDS_INVESTIGATION';
interface CIFailureAnalysis {
category: FailureCategory;
confidence: number;
affectedTests: string[];
deployRecommendation: 'BLOCK' | 'WARN' | 'PASS';
reasoning: string;
}
function isCIFailureAnalysis(o: unknown): o is CIFailureAnalysis {
if (typeof o !== 'object' || o === null) return false;
const a = o as Record<string, unknown>;
const cats: FailureCategory[] =
['APPLICATION_BUG','TEST_BUG','ENVIRONMENT_ISSUE','NEEDS_INVESTIGATION'];
return cats.includes(a.category as FailureCategory)
&& typeof a.confidence === 'number'
&& ['BLOCK','WARN','PASS'].includes(a.deployRecommendation as string);
}

export async function analyseFailures(failures: unknown[]): Promise<CIFailureAnalysis> {
const client = new Anthropic();
const r = await client.messages.create({
model: 'claude-sonnet-4-5', max_tokens: 1024,
messages: [{ role: 'user', content: `Analyse and categorise these failures:
${JSON.stringify(failures)}.
Return JSON matching CIFailureAnalysis with FailureCategory in
{APPLICATION_BUG,TEST_BUG,ENVIRONMENT_ISSUE,NEEDS_INVESTIGATION} and
deployRecommendation in {BLOCK,WARN,PASS}.` }],
});
const raw: unknown = JSON.parse(r.content[0].type === 'text' ? r.content[0].text :
'{}');
if (!isCIFailureAnalysis(raw)) throw new Error('LLM returned invalid
CIFailureAnalysis');
return raw;
}
// APPLICATION_BUG -> BLOCK | TEST_BUG -> WARN+route | ENVIRONMENT_ISSUE -> WARN+retry |
NEEDS_INVESTIGATION -> human
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What is the TypeScript-only advantage for AI-generated Playwright tests?

A) TS is faster than JS at runtime
B) TS removes the need for any testing
C) tsc --noEmit catches hallucinated methods at compile time, free first filter
D) Claude only works with TS

2. What is MCP, in one sentence a senior engineer would accept?

A) An open protocol (modelcontextprotocol.io) connecting AI to tools/data/systems, widely adopted
B) An Anthropic-only API for Claude
C) A replacement for WebDriver
D) A Playwright plugin

3. What transport does MCP use for remote servers in 2026?

A) Server-Sent Events (SSE)
B) gRPC
C) WebSockets only
D) Streamable HTTP: SSE was deprecated in March 2025

4. Why prefer browser_snapshot over browser_take_screenshot for an AI agent?

A) Snapshot is prettier
B) Snapshot is the only option available
C) Screenshot is broken
D) Snapshot returns a structured accessibility tree, semantic, no vision model needed; faster, cheaper

5. How do you keep AI safe in a CI deployment gate?

A) Let the AI decide deployment freely
B) Constrain output via a FailureCategory union type + deterministic action mapping (BLOCK/WARN/PASS) + human on NEEDS_INVESTIGATION
C) Disable the gate entirely
D) Trust the LLM confidence score

Section 27 · Answer key

#	Answer	Why and where to revisit
1	C) tsc --noEmit catches hallucinated methods at compile time, free first filter	The compiler flags hallucinated Playwright methods immediately; JavaScript teams discover them only at runtime. *Revisit Section 27.*
2	A) An open protocol (modelcontextprotocol.io) connecting AI to tools/data/systems, widely adopted	MCP is to AI agents what WebDriver is to browser automation, the protocol layer that enables an ecosystem. *Revisit Section 27.*
3	D) Streamable HTTP: SSE was deprecated in March 2025	Knowing Streamable HTTP replaced SSE signals you're tracking the spec actively, not reading 2024 tutorials. *Revisit Section 27.*
4	D) Snapshot returns a structured accessibility tree, semantic, no vision model needed; faster, cheaper	Agents reason over roles/labels/states (structure), not pixels, making them faster and more reliable. *Revisit Section 27.*
5	B) Constrain output via a FailureCategory union type + deterministic action mapping (BLOCK/WARN/PASS) + human on NEEDS_INVESTIGATION	Union types in the prompt + type-guard validation + deterministic action mapping bound the AI safely, ~80% triage reduction. *Revisit Section 27.*

SECTION 28

Real-Time Interview Scenarios

These are the scenario questions asked in senior Playwright interviews, design problems and debugging walk-throughs that require a full architectural answer, not a code snippet. They are the questions where the offer is genuinely won or lost, because they test whether you can reason across the whole framework: configuration, fixtures, parallelism, CI, debugging, security, and migration. Each answer is structured the way a senior would actually speak it, with the diagnosis or design phases explicit, the trade-offs named, and concrete numbers attached.

In this section

- Architectural framework design: layers, fixtures, CI, organisation.
- Local-vs-CI failure diagnosis as a structured 8-step process.
- Multi-context patterns for real-time, multi-tenant, and chat scenarios.
- Performance: turning 2-hour suites into 20-minute ones with phased optimisation.
- Migration strategy: phased Selenium-to-Playwright over 3-4 sprints, not a rewrite.

Q 301 LEVEL · Senior

Design a Playwright TypeScript framework from scratch for a fintech application with 5 user roles.

CONCEPT

A senior framework design answer covers four layers explicitly. **Configuration**: playwright.config.ts with environment-specific dotenv loading, **forbidOnly** for CI, and projects per browser including mobile. Authentication: globalSetup generates one **storageState** file per role, admin-state.json, compliance-state.json, etc, eliminating login time across 300+ tests. Framework layers: page objects in src/pages with a BasePage abstract class, component classes in src/components for reusable UI (tables, modals, nav), a single fixtures file extending the test object with all page objects and role-specific authenticated pages, and test-data factories in src/test-data using faker for synthetic data. CI + organisation: typecheck first (tsc --noEmit), then lint, then parallel execution with 4 shards of 4 workers each; traces and HTML reports as artifacts, JUnit XML for result parsing; tests organised tests/ui/, tests/api/, tests/integration/, tagged @smoke, @regression, @admin, @user for selective execution.

INTERVIEW STRATEGY

This one is checking whether you can architect, not just code. Answer in four explicit layers, configuration, auth, framework, CI/organisation, because the structure itself is the signal. Five-role storageState in globalSetup is the centerpiece of the auth answer because it's where most candidates lose time. The classic error is diving into a single layer ('I'd write a page object…') without showing you see the whole system. The follow-up is often 'how does this scale to 300+ tests?', answer storageState eliminates login overhead, fixtures compose across files, parallel sharding keeps CI fast.

How to phrase it

> *I would design this in four explicit layers, because the structure is the answer the interviewer is testing. Layer one is configuration: playwright.config.ts with environment-specific dotenv loading so dev, staging and prod each get their own env, forbidOnly set for CI so a forgotten test.only fails the build rather than silently skipping everything, and projects per browser including mobile profiles. Layer two is authentication, and for five user roles this is where most candidates lose time. I use globalSetup to generate one storageState file per role, admin-state.json, compliance-state.json, trader-state.json and so on, which eliminates login time across all three hundred-plus tests. That answers the follow-up about scaling: storageState is the lever. Layer three is the framework: page objects in src-pages with a BasePage abstract class, component classes in src-components for reusable UI like tables and modals and nav, a single fixtures file that extends the test object with all page objects and role-specific authenticated pages, and test-data factories in src-test-data using faker for synthetic data and env vars for real credentials. Layer four is CI and organisation: typecheck first with tsc --noEmit, then lint, then parallel execution with four shards of four workers each, traces and HTML reports as artifacts, JUnit XML for result parsing. Tests organised by tests-ui, tests-api, tests-integration, tagged smoke, regression, admin, user for selective execution. The classic error is diving into a single layer; showing you see the whole system is the test of seniority on this.*

Key points to hit

(1) Four layers: configuration, auth, framework, CI/organisation. **(2)** globalSetup -> one storageState per role: eliminates login overhead. **(3)** BasePage + components + single fixtures file + faker data factories. **(4)** CI: typecheck -> lint -> 4 shards x 4 workers, traces + JUnit XML. **(5)** Follow-up: scaling 300+ tests is storageState + fixtures + sharding. **(6)** Where this goes wrong: diving into one layer and missing the whole-system view.

CODE

```
// Structure
// src/
// pages/ BasePage.ts, LoginPage.ts, AdminPanelPage.ts
// components/ DataTable.ts, Modal.ts, Navigation.ts
// fixtures/ index.ts (extends test with pages + auth)
// test-data/ UserBuilder.ts, OrderBuilder.ts
// tests/
// ui/ api/ integration/
// auth/ admin-state.json, compliance-state.json, trader-state.json, …
//
// playwright.config.ts: highlights
// forbidOnly: !!process.env.CI
// workers: process.env.CI ? 4 : undefined
// fullyParallel: true
// projects: [chromium, firefox, webkit, 'Mobile Chrome', 'Mobile Safari']
// reporter: [['html'], ['junit', { outputFile: 'results.xml' }]]
```

Q 302

LEVEL · Mid to Senior

Your Playwright tests pass locally but fail in CI. Walk through your complete diagnosis process.

CONCEPT

An 8-step diagnostic process. 1. Read the exact CI error and the line it occurred on. 2. Download the trace from CI, open at trace.playwright.dev, and scrub the timeline to find the failing action and inspect DOM snapshot + network response at that point. 3. Compare browser versions: npx playwright --version locally and CI; version mismatch causes behaviour drift. 4. Headless vs headed: some CSS transitions behave differently headless; reproduce locally headless. 5. Verify env vars: BASE_URL, credentials, all required vars set in CI. **6. Resource constraints**, CI has fewer CPUs; if it passes with workers:1 but fails with workers:4, it's shared state between parallel tests. **7. Hard sleeps**: waitForTimeout is fragile because CI is slower; replace with explicit waitFor conditions. **8. Network timing**: CI has different latency; if a response arrives slower than expected, add waitForResponse.

INTERVIEW STRATEGY

What scores here is showing methodical diagnosis, not guessing. Lead with the 8-step structure because the process itself is the answer. The trace download is the single highest-leverage step, it answers most questions in 5 minutes. The trap is jumping to a guess (‘probably timing’) before looking at the trace. Expect the follow-up ‘which step catches it fastest?’, answer the trace, because it shows the actual DOM and network state at failure.

How to phrase it

> *I treat this as an eight-step diagnostic process, because structure beats guessing and the structure itself shows the interviewer I have hit this many times. Step one, read the exact CI error message and the line it occurred on, not the summary, the precise stack. Step two, and this is the highest-leverage move and my answer to the follow-up about which step catches it fastest, download the trace artifact from CI and open it at trace dot playwright dot dev. Scrub the timeline to find the exact action that failed and inspect the DOM snapshot and network response at that point, most questions are answered in five minutes. Step three, compare browser versions. Run npx playwright dash dash version locally and check the CI log; version mismatch causes real behaviour drift. Step four, headless versus headed. Some CSS transitions behave differently in headless, so I reproduce locally with --headed=false. Step five, verify environment variables: BASE_URL, credentials, all required vars set in CI configuration. Step six, resource constraints. CI machines have fewer CPUs and slower disk; if tests pass with workers one but fail with workers four, it's shared state between parallel tests. Step seven, look for hard sleeps. waitForTimeout is fragile because CI is slower than local; replace with explicit waitFor conditions. Step eight, check the trace for network timing, CI has different latency to test APIs, and if a response arrives slower than the test expects, add waitForResponse. The failure mode is guessing without looking at the trace first.*

Key points to hit

(1) 8-step structured process: the structure itself is the experience marker. **(2)** Trace download is highest-leverage: answers most questions in 5 min. **(3)** Compare browser versions, headless mode, env vars, resource constraints. **(4)** Workers count exposes shared state; CI is slower so kill waitForTimeout. **(5)** Expect next: trace is fastest, shows DOM + network at exact failure point. **(6)** What loses the point: guessing the cause before downloading and reading the trace.

CODE

```
# Step 2: the trace is your fastest diagnostic
# 1. Download from CI artifacts
# 2. open https://trace.playwright.dev
# 3. Scrub timeline -> see DOM + network at every action

# Step 6: isolate shared-state bugs by varying workers
npx playwright test --workers=1 # passes? -> shared state with parallel
npx playwright test --workers=4 # reproduce CI condition locally
```

Q 303 LEVEL · Senior

How would you test a real-time chat application?

CONCEPT

Real-time chat requires **multiple browser contexts** and WebSocket awareness. The pattern: two contexts, one for the sender, one for the receiver, each authenticated as different users via storageState. Both navigate to the same chat room, sender posts a message, receiver awaits the message bubble with a generous timeout to allow the WebSocket round-trip. For WebSocket

monitoring, page.on('websocket') hooks into the connection and lets you inspect framesent and framereceived events. For **disconnection testing**, use route() to drop WebSocket traffic and verify the UI shows a reconnecting state. The two-context pattern is also the right tool for any multi-user interaction, notifications, collaborative editing, presence indicators, anywhere user A's action must be visible to user B in real time.

INTERVIEW STRATEGY

This question is there to see you handle multi-user real-time interaction. Lead with the two-context pattern because it's the architectural insight. Generous timeout on the receiver assertion is the practical detail. The risk is using one context with two pages: they'd share cookies and break the two-user setup. They usually follow with 'how do you test disconnection?', answer route() drops the WebSocket and you verify the reconnecting UI state.

How to phrase it

> *Real-time chat requires multiple browser contexts and WebSocket awareness, and the two-context pattern is the architectural insight I would lead with. I create two browser contexts, one for the sender and one for the receiver, each authenticated as a different user via its own storageState file. Both navigate to the same chat room concurrently with Promise.all, the sender fills the message field and clicks send, and the receiver awaits the message bubble appearing, I use a generous timeout, around ten seconds, to allow the full WebSocket round-trip. The trap I would call out is using one context with two pages, because they'd share cookies and you'd actually be testing one user sending themself a message, which proves nothing. For WebSocket monitoring, page.on('websocket') hooks into the connection and lets me inspect framesent and framereceived events, which is useful for verifying specific message frames or for debugging delivery issues. That answers the follow-up about how I test disconnection, which is one of the most interesting parts of a chat system: I use route() to drop or interrupt WebSocket traffic and verify the UI shows the reconnecting state, then I allow traffic again and verify the messages queued during the outage are delivered when the connection restores. The two-context pattern is also the right tool for any multi-user interaction, notifications, collaborative editing, presence indicators, anywhere user A's action must be visible to user B in real time.*

Key points to hit

(1) Two browser contexts: separate storageState per user. **(2)** Generous timeout on receiver assertion (~10s for WS round-trip). **(3)** page.on('websocket') for frame-level monitoring (framesent/received). **(4)** Use route() to drop WS traffic and test the reconnecting UI. **(5)** Pattern scales to notifications, collaborative editing, presence. **(6)** The common miss: one context with two pages, same cookies, not two users.

CODE

```
test('message from user A appears for user B', async ({ browser }) => {
const ctxA = await browser.newContext({ storageState: 'auth/user-a.json' });
const ctxB = await browser.newContext({ storageState: 'auth/user-b.json' });
const pageA = await ctxA.newPage();
const pageB = await ctxB.newPage();
await Promise.all([pageA.goto('/chat/room/123'), pageB.goto('/chat/room/123')]);

await pageA.getByLabel('Message').fill('Hello from User A');
await pageA.getByRole('button', { name: 'Send' }).click();

await expect(pageB.locator('.message-bubble').last())
.toHaveText('Hello from User A', { timeout: 10_000 });

await Promise.all([ctxA.close(), ctxB.close()]);
});

// WebSocket frame monitoring
page.on('websocket', ws => {
ws.on('framesent', f => console.log('sent:', f.payload));
ws.on('framereceived', f => console.log('recv:', f.payload));
});
```

Q 304 LEVEL · Senior

How do you reduce a 2-hour Playwright suite to under 20 minutes?

CONCEPT

A five-phase performance optimisation. **Phase 1, Audit**: run with --reporter=json, identify the 20 slowest tests; profile whether slowness is in setup (UI login flows) or in the test itself. Phase 2, Setup optimisation: move UI-based test-data setup to API calls (UI login: 5-8s; storageState: 0s; UI user creation: 30s; API user creation: 200ms), typically a 30-40% reduction alone. **Phase 3, Parallelism**: enable fullyParallel, set workers:4 per machine, 8-shard CI execution, mathematically 120 min / 8 shards = 15 min/shard, then ÷4 workers per shard = ~4 min/shard. Phase 4, Test design: mark genuinely slow tests with test.slow() and move them to a nightly project; remove duplicate tests that cover the same scenario at different layers. **Phase 5, Infrastructure**: cache browser binaries keyed on package-lock.json; Chromium-only Docker for main suite, cross-browser only on release candidates.

INTERVIEW STRATEGY

They want to see a structured optimisation, not guesses. Walk the five phases explicitly. The biggest-impact phase is setup migration to API (30-40% alone), so anchor on it. The math on sharding lands the credibility. The failure mode is jumping to parallelism before fixing slow setup. The next question is normally ‘what's the single highest-impact change?’, answer move UI-based setup to API calls.

How to phrase it

> *I would handle this as a five-phase optimisation, and the structure is what makes it credible. Phase one is audit. I run with --reporter=json and identify the twenty slowest tests, and I profile whether the slowness is in setup, like UI login flows, or in the test itself, like waiting for slow pages. Without that audit I'd be guessing. Phase two is setup optimisation, and this is also my answer to the follow-up about the single highest-impact change: move all UI-based test data setup to API calls. A login via UI takes five to eight seconds; via storageState it takes zero. A user creation via UI takes thirty seconds; via API it takes two hundred milliseconds. This alone typically reduces suite time by thirty to forty percent before I touch parallelism. Phase three is parallelism. Enable fullyParallel true, set workers to four per machine, and implement eight-shard CI execution. The math: a hundred-twenty minutes divided by eight shards is fifteen minutes per shard, then divided by four workers per shard is approximately four minutes per shard. Phase four is test design. Mark genuinely slow tests with test.slow and move them to a separate nightly project, and remove duplicate tests that cover the same scenario at different levels of the stack. Phase five is infrastructure: cache browser binaries in CI keyed on package-lock.json, use the Chromium-only Docker image for the main suite, and run cross-browser only on release candidates. The failure mode is jumping straight to parallelism before fixing slow setup, you parallelise inefficiency.*

Key points to hit

(1) Five phases: audit, setup, parallelism, test design, infrastructure. **(2)** Audit first with --reporter=json: don't guess where time goes. **(3)** Setup migration (UI -> API/storageState) = 30-40% alone. **(4)** Sharding math: 120min ÷ 8 shards ÷ 4 workers ≈ 4min/shard. **(5)** They usually ask next: highest-impact change is moving setup to API. **(6)** Avoid: parallelising before fixing slow setup, just parallelises waste.

CODE

```
# Phase 1: audit
npx playwright test --reporter=json | tee results.json
node scripts/find-slowest.js results.json # -> top 20 slowest

// Phase 2: UI setup -> API/storageState
// before: 30s of UI user creation per test
// after: 200ms API call in a fixture, storageState for the session

// Phase 3: parallelism + sharding (CI)
// playwright.config.ts: fullyParallel: true, workers: 4
// CI matrix: shard 1/8 … 8/8

// Phase 4: quarantine genuinely slow tests
test.slow(); // marks this test as slow -> nightly project
```

Q 305

LEVEL · Senior

A critical payment flow test is flaky: fails 1 in 10 runs. How do you debug it?

CONCEPT

A structured flake-debugging process. **Step 1**: enable trace: 'on' specifically for this test via test.use() in a describe block, then run the test in a loop, 20 times, to collect both passing and failing traces. Step 2: compare failing vs passing traces side-by-side in the trace viewer, looking at iframe availability (Stripe iframes load async), network response timing for the payment API, element stability in the card-number field, and whether failure is always on retry-1 or random. **Step 3**: if retry-1-only, it's environment warmup, the first test in a worker hits a cold backend; fix with a warmup request in a worker-scoped fixture. Step 4: if random, it's a race condition. The most common payment iframe race: the iframe loads but the card fields inside are not yet interactive, fix by waiting for the field to be **editable**, not just visible.

INTERVIEW STRATEGY

Your job here is to show methodical flake debugging, not 'add a retry'. Lead with collecting 20 traces in a loop because comparison is the only reliable way to see what differs between pass and fail. The retry-1-vs-random distinction signals depth here because it splits warmup from race conditions. The thing to avoid is just bumping the timeout. A common follow-up is 'what's the most common payment iframe race?', answer the field is visible but not yet editable; wait on editable state.

How to phrase it

> *One-in-ten flake on a payment flow is not a timeout problem, it's a race condition or warmup problem, and the diagnosis needs to be structured. Step one, I enable trace on for this test specifically using test.use trace on in a describe block, then I run the test in a loop twenty times to collect both passing and failing traces. Without multiple traces I can't compare; with them I can see exactly what differs. Step two, I compare failing versus passing traces side by side in the trace viewer, looking specifically at four things: the frameLocator availability for the payment iframe, Stripe iframes load asynchronously, the network response timing for the payment API, element stability in the card-number field, and whether the failure is always on retry one or random across attempts. Step three, if it's retry-one only, the issue is environment warmup, the first test in a worker hits a cold backend, and the fix is to add a warmup request in a worker-scoped fixture so cold-start time is paid once per worker, not per test. Step four, if it's random, it's a race condition, and that's also my answer to the follow-up about the most common payment iframe race: the iframe loads, the card-number field appears visible, but it's not yet interactive, the Stripe JavaScript hasn't attached its event listeners. The fix is to wait for the field to be editable, not just visible. Waiting on visible passes; waiting on editable is what makes the test deterministic. The failure mode is just bumping the timeout, which masks the race rather than fixing it.*

Key points to hit

(1) Collect 20 traces in a loop: compare passing vs failing side by side. **(2)** Check iframe availability, network timing, element stability, retry pattern. **(3)** Retry-1-only = environment warmup; fix with worker-scoped warmup fixture. **(4)** Random = race; common one: iframe field visible but not yet editable. **(5)** The next question: wait on editable state, not just visible, for iframe fields. **(6)** The easy mistake: bumping timeout to mask the race rather than fixing it.

CODE

```
// Step 1: collect traces in a loop
test.use({ trace: 'on' });
// then: npx playwright test payment.spec.ts --repeat-each=20

// Step 4 fix: wait on editable, not just visible
const cardFrame = page.frameLocator('#stripe-card-iframe');
await cardFrame.getByLabel('Card Number').waitFor({ state: 'visible' });
await cardFrame.getByLabel('Card Number').waitFor({ state: 'editable' });
await cardFrame.getByLabel('Card Number').fill('4111111111111111');
```

Q 306 LEVEL · Senior

How do you build a Playwright framework that multiple teams can contribute to without breaking each other?

CONCEPT

Multi-team contribution needs clear ownership boundaries, enforced standards, and safe contribution paths. **Architecture**: core framework layer (fixtures, base classes, utilities) owned by the QA platform team; domain layer (page objects, component classes) owned by feature teams; test layer owned by feature teams independently. **Standards**: ESLint with playwright/recommended plus custom rules, pre-commit hooks for typecheck and lint, required QA platform team review only for core-layer changes, feature teams can add page objects and tests without that gate. Versioning: core framework as an internal npm package; feature teams pin to a version and upgrade on their schedule; breaking changes require a major bump and migration guide. **Documentation**: CONTRIBUTING.md with locator priority, fixture patterns, review checklist; Architecture Decision Records for major decisions; runbooks for common tasks.

INTERVIEW STRATEGY

Show organisational design, not just technical. Lead with ownership boundaries because it's the structural insight, platform team owns core, feature teams own domain and tests. The internal npm package with semver is the senior detail. The mistake is treating the framework as one codebase everyone edits. The follow-up is often ‘how do you prevent breaking changes?’, answer semver-versioned core, feature teams pin and upgrade on their schedule.

How to phrase it

> *Multi-team contribution needs clear ownership boundaries, enforced standards, and safe contribution paths, it's an organisational design question as much as a technical one. Architecture: I split into three layers with explicit ownership. The core framework layer, fixtures, base classes, utilities, is owned by the QA platform team. The domain layer, page objects, component classes, is owned by the relevant feature teams. The test layer is owned by feature teams independently. This means the platform team is not a bottleneck for every test addition. Standards enforcement: ESLint with playwright-slash-recommended plus custom rules, pre-commit hooks for typecheck and lint, and required QA platform team review only for changes to the core layer. Feature teams can add page objects and tests without platform-team review, which is what keeps velocity. Versioning, which is also my answer to the follow-up about preventing breaking changes: the core framework is published as an internal npm package, semver-versioned. Feature teams pin to a version and upgrade on their schedule. Breaking changes in the core require a major version bump and a migration guide. Documentation: a CONTRIBUTING dot md with locator priority, fixture patterns, and the review checklist so teams self-serve; Architecture Decision Records for major framework decisions so the why is captured; runbooks for common tasks like adding a page object or fixture. The trap is treating the framework as one codebase everyone edits, that's where conflicts and broken contracts come from. Ownership boundaries plus semver are what scale this past two teams.*

Key points to hit

(1) Three layers with ownership: core (platform team), domain + tests (feature teams). **(2)** Platform team only gates core changes; feature teams ship independently. **(3)** Core published as internal semver-versioned npm package. **(4)** Feature teams pin and upgrade on their schedule; breaking = major bump. **(5)** Asked next: prevent breaks with semver + migration guides + pinned versions. **(6)** Trap: one shared codebase everyone edits, conflicts and broken contracts.

CODE

```
// Repository layout
// /core: owned by QA platform team; published as @org/test-framework
// fixtures/ base-classes/ utilities/
// /domain: owned by feature teams
// billing/pages/ billing/components/
// checkout/pages/ checkout/components/
// /tests: owned by feature teams
// billing/ checkout/ admin/

// CODEOWNERS: enforce review boundaries
// /core/ @qa-platform-team
// /domain/billing/ @billing-team
// /tests/billing/ @billing-team

// package.json (feature team): pin to a core version
// "@org/test-framework": "^3.2.0"
```

Q 307 LEVEL · Senior

How would you test an admin panel of a multi-tenant SaaS?

CONCEPT

Multi-tenant testing requires verifying both **functionality and tenant isolation**, that tenant A cannot see or modify tenant B's data. Auth strategy: storageState files per role per tenant, admin-tenant-a.json, admin-tenant-b.json, user-tenant-a.json, with globalSetup generating all combinations. **Functional tests**: standard admin-panel flows scoped per tenant. Isolation tests are the most critical: log in as admin of tenant A and attempt to access tenant B's resources via direct URL and via API. Verify **403 or 404 responses**, never tenant B's data. The isolation tests catch the failure mode that destroys a SaaS, cross-tenant data leakage, which a functional test for tenant A's UI would never find. The pattern: storageState for tenant A, attempt URLs and API calls scoped to tenant B, expect denial. Both UI route and API request must be tested because they're enforced separately.

INTERVIEW STRATEGY

They are looking to see you recognise the security dimension. Lead with the dual-axis test matrix (roles × tenants) for storageState, then make isolation tests the centerpiece because they catch the failure mode that actually destroys a SaaS. Test both UI route and API: they enforce separately. The classic error is testing only functional happy paths per tenant. Expect the follow-up 'which isolation tests matter most?', answer direct URL access and direct API access from one tenant to another's resources, because real attackers don't go through the UI.

How to phrase it

> *Multi-tenant testing requires verifying both functionality and tenant isolation, that tenant A cannot see or modify tenant B's data, and the isolation half is what makes this a serious test problem rather than just admin-panel UI tests. Auth strategy: I create storageState files per role per tenant, admin-tenant-a, admin-tenant-b, user-tenant-a, and globalSetup generates all the combinations I need. Functional tests are the obvious half: standard admin-panel flows scoped per tenant. The critical half, and the one I would emphasise because it's where most candidates lose marks, is the isolation tests. I log in as admin of tenant A and attempt to access tenant B's resources two ways: via direct URL and via direct API call. I verify the response is a 403 or 404, never tenant B's data. That answers the follow-up about which isolation tests matter most. The direct URL and direct API tests are the ones that matter, because real attackers don't go through the UI happy path, they construct URLs and hit endpoints directly, and the isolation must hold there. The reason both UI and API must be tested separately is that they're enforced by different code paths, the UI router might block tenant B's URLs, but the API might happily return tenant B's data to an authenticated tenant-A admin if the backend doesn't enforce scoping. I have seen exactly that bug in production. The trap is testing only the functional happy paths per tenant; the isolation tests catch the failure mode that destroys a SaaS, cross-tenant data leakage, which a tenant-A UI test would never find.*

Key points to hit

(1) Generate storageState per role per tenant in globalSetup. **(2)** Test functionality AND isolation: isolation is the critical half. **(3)** Attempt cross-tenant access via direct URL AND direct API. **(4)** Expect 403/404; never the other tenant's data. **(5)** Follow-up: direct URL/API are what matter, attackers don't use the UI. **(6)** Where this goes wrong: testing only functional happy paths and missing data leakage.

CODE

```
test('tenant A admin cannot access tenant B data', async ({ browser }) => {
const ctx = await browser.newContext({ storageState: 'auth/admin-tenant-a.json' });
const page = await ctx.newPage();

// 1. Direct URL access attempt
await page.goto('/admin/tenants/tenant-b/users');
await expect(page).toHaveURL(/\/403|\/unauthorized/);

// 2. Direct API access attempt: enforced separately from the UI router
const res = await page.request.get('/api/tenants/tenant-b/users');
expect(res.status()).toBe(403);
});
```

Q 308 LEVEL · Senior

Design a visual regression test strategy for a design system.

CONCEPT

A three-layer visual-regression strategy. **Component layer**: Playwright Component Testing mounts each design-system component in isolation with all variants (size, colour, state). Snapshot each variant. Run on every PR that touches the component library. Page layer: full-page screenshots of key pages with dynamic content masked. Run on nightly builds. Failures require designer review before baseline update. **Cross-browser layer**: visual tests on Chromium, Firefox, and WebKit with separate baselines per browser, browser rendering differences are expected and shouldn't be flagged. **Tooling**: Playwright's built-in toHaveScreenshot for component and page tests; Applitools when you need cross-browser visual AI that understands semantic differences vs rendering artifacts. The layer separation matters because each runs at a different frequency with different review gates.

INTERVIEW STRATEGY

The point of this one is to see strategy across granularity and cost. Lead with three layers, component, page, cross-browser, because the granularity is the structural insight. Per-browser baselines is the detail that shows you've actually run cross-browser visual tests. The mistake is one global baseline that fails on rendering differences every browser update. They usually follow with ‘why separate baselines per browser?’, answer rendering differs by engine; one shared baseline would constantly flag them as regressions.

How to phrase it

> *I structure visual regression for a design system in three layers, because each runs at a different frequency with different review gates. Layer one is the component layer. I use Playwright Component Testing to mount each design-system component in isolation with all variants, sizes, colours, states like hover and disabled, and snapshot each variant. These tests run on every PR that touches the component library, because at the component level the feedback needs to be immediate. Layer two is the page layer. Full-page screenshots of key pages with dynamic content masked, timestamps, user avatars, anything that legitimately varies. These run on nightly builds rather than every PR, and failures require designer review before baseline update, because page-level changes are often intentional. Layer three is cross-browser. I run the visual tests on Chromium, Firefox, and WebKit with separate baselines per browser. The follow-up usually probes this, and the answer is about why separate baselines per browser: rendering differs by engine, antialiasing, font hinting, gradient interpolation, and a single shared baseline would constantly flag those as regressions when they're just expected rendering differences. Per-browser baselines isolate the real regressions from engine quirks. On tooling: Playwright's built-in toHaveScreenshot covers component and page tests well; Applitools is what I reach for when I need cross-browser visual AI that understands semantic differences versus rendering artifacts, because pixel-diff at scale on three browsers gets noisy. The thing to avoid is one global baseline across browsers, it fails constantly and nobody trusts the suite.*

Key points to hit

(1) Three layers: component (per PR), page (nightly), cross-browser. **(2)** Component layer uses Playwright CT: every variant snapshotted. **(3)** Page layer masks dynamic content; designer review before baseline update. **(4)** Per-browser baselines isolate real regressions from engine differences. **(5)** Expect next: rendering differs by engine, shared baseline = constant noise. **(6)** What loses the point: one global baseline that fails on every Chromium update.

CODE

```
// Layer 1: component (Playwright CT, every PR)
test('Button: primary variant', async ({ mount }) => {
const c = await mount(<Button variant="primary">Save</Button>);
await expect(c).toHaveScreenshot('button-primary.png');
});

// Layer 2: page (nightly, dynamic content masked)
await expect(page).toHaveScreenshot('dashboard.png', {
mask: [page.locator('[data-test="user-avatar"]'), page.locator('time')],
});

// Layer 3: per-browser baselines
// snapshots/dashboard-chromium.png
// snapshots/dashboard-firefox.png
// snapshots/dashboard-webkit.png
```

Q 309 LEVEL · Mid

How do you test Canvas or WebGL content?

CONCEPT

Canvas and WebGL content cannot be tested with DOM-based locators, the content is rendered as pixels, not HTML elements. The testing strategy depends on what needs verification. **Visual correctness**: toHaveScreenshot on the canvas element captures the rendered pixels and compares to a baseline, effective for detecting rendering regressions. **Interaction testing**: canvas interactions use mouse coordinates, page.mouse.click(x, y) and page.mouse.move(x, y) simulate interactions at specific pixel positions; coordinates are layout-dependent so use **boundingBox()** to compute positions relative to the canvas. Data verification: if the canvas application exposes its state via JavaScript (common in charting libraries like Chart.js), use page.evaluate to read the chart data directly and assert on values rather than pixels. The right combination: visual snapshot for rendering, mouse coordinates for interaction, page.evaluate for data assertions.

INTERVIEW STRATEGY

You are being asked to show you handle a non-DOM rendering surface. Lead with the fundamental: canvas is pixels, not elements. Then split by what's being tested: visual correctness (screenshot), interaction (mouse coords + boundingBox), data (page.evaluate when state is exposed). The boundingBox detail is the senior practical signal. The trap is trying to use getByRole on canvas. The next question is normally 'how do you assert on chart data?', answer use page.evaluate if the chart library exposes data on a global; otherwise visual snapshot.

How to phrase it

> *Canvas and WebGL content cannot be tested with DOM-based locators because the content is rendered as pixels, not HTML elements, and once I state that the testing strategy splits cleanly by what I'm verifying. For visual correctness I use toHaveScreenshot on the canvas element, it captures the rendered pixels and compares to a baseline, which is effective for detecting rendering regressions like a chart rendering wrong or a drawing tool's stroke colour changing. For interaction testing I use mouse coordinates: page.mouse.click x y and page.mouse.move x y simulate interactions at specific pixel positions. The detail that matters here, because absolute coordinates are layout-dependent, is using boundingBox on the canvas to compute positions relative to it. So I read the box, add my offset for the start of the line and the end, and the test stays correct even if the page layout shifts. For data verification, and this is my answer to the follow-up about asserting on chart data, if the canvas application exposes its state via JavaScript, which charting libraries like Chart.js commonly do via a global instance, I use page.evaluate to read the chart data directly and assert on the actual values, not the pixels. That's far more robust than visual diffing because legitimate styling changes don't fail the test. The combination that works: visual snapshot for rendering, mouse coordinates plus boundingBox for interaction, page.evaluate for data when exposed. Where this goes wrong is trying to use getByRole on canvas, there are no roles to find.*

Key points to hit

(1) Canvas/WebGL is pixels, not DOM: no getByRole possible. **(2)** Visual: toHaveScreenshot on the canvas element. **(3)** Interaction: page.mouse with coords; use boundingBox for layout-relative. **(4)** Data: page.evaluate when the chart library exposes state (e.g. Chart.js). **(5)** They usually ask next: assert chart values via page.evaluate, robust to styling. **(6)** The common miss: trying DOM-based locators on canvas, nothing to find.

CODE

```
test('canvas drawing tool draws a line', async ({ page }) => {
await page.goto('/drawing-tool');
const canvas = page.locator('#drawing-canvas');
const box = await canvas.boundingBox();
if (!box) throw new Error('canvas not found');

// Interaction: mouse coords relative to the canvas
await page.mouse.move(box.x + 50, box.y + 100);
await page.mouse.down();
await page.mouse.move(box.x + 200, box.y + 100);
await page.mouse.up();

// Visual: capture rendered pixels
await expect(canvas).toHaveScreenshot('line-drawn.png');
});

// Data: read chart state when exposed by the library
const datasets = await page.evaluate(() => (window as any).myChart.data.datasets);
expect(datasets[0].data[0]).toBe(42);
```

Q 310 LEVEL · Senior

How would you migrate 500 Selenium tests to Playwright?

CONCEPT

A 500-test migration is a phased approach over 3-4 sprints, not a big-bang rewrite. **Phase 1, Foundation (Sprint 1)**: set up the Playwright framework alongside the existing Selenium framework. playwright.config.ts, base fixtures, folder structure. Migrate globalSetup and authentication. Run both frameworks in CI in parallel. **Phase 2, High-value tests (Sprint 2)**: migrate the 50 most flaky Selenium tests first, because they benefit most from Playwright's auto-waiting; measure flakiness before and after to demonstrate value. **Phase 3, Systematic migration (Sprints 3-4)**: migrate by feature area; each page object rewritten as a Playwright TS class, findElement becomes a locator with getByRole priority, explicit waits removed (auto-waiting replaces them), StaleElementReferenceException handlers deleted. Phase 4, Decommission: once stable for 2 sprints, remove the Selenium framework. Track flakiness rate, average execution time, CI duration, and explicit-wait statements removed.

INTERVIEW STRATEGY

What scores here is showing migration strategy and ROI measurement, not a heroic rewrite. Lead with phased approach across 3-4 sprints. Phase 2's pick-the-flakiest-tests-first is the senior insight because it produces visible value early. Running both in parallel during transition shows operational maturity. The classic error is proposing a single big-bang migration. A common follow-up is 'how do you demonstrate value to stakeholders?', answer measure flakiness before/after migration on those first 50 tests; the number sells the rest.

How to phrase it

> *A five-hundred-test migration is a phased approach over three to four sprints, never a big-bang rewrite, that's the answer that shows operational maturity to the interviewer. Phase one, foundation, in sprint one: I set up the Playwright framework alongside the existing Selenium framework. playwright.config.ts, base fixtures, the folder structure. I migrate globalSetup and authentication. Critically I run both frameworks in CI in parallel so coverage never drops during the transition. Phase two, high-value tests, in sprint two: I migrate the fifty most flaky Selenium tests first, not the easiest, the flakiest. The reason is that those tests benefit most from Playwright's auto-waiting, and that's also my answer to the follow-up about demonstrating value to stakeholders. I measure flakiness before and after on those fifty tests and present the number: a drop from twelve percent to under one is what justifies funding the rest of the migration. Phase three, systematic migration, sprints three and four: I migrate by feature area, one team at a time. Each page object is rewritten as a Playwright TypeScript class, Selenium's findElement calls become Playwright locators with getByRole as the priority, explicit waits are removed because auto-waiting replaces them, and StaleElementReferenceException handlers are deleted because Playwright re-resolves locators automatically. Phase four, decommission: once all tests are migrated and stable for two sprints, I remove the Selenium framework, ChromeDriver management, and Selenium-specific CI configuration. Throughout I track four metrics: flakiness rate, average execution time, CI pipeline duration, and number of explicit-wait statements removed, because the migration has to be measured to be credible. The failure mode is the big-bang rewrite that loses coverage during the transition.*

Key points to hit

(1) Phased over 3-4 sprints: NEVER a big-bang rewrite. **(2)** Run both frameworks in CI in parallel during transition. **(3)** Phase 2: migrate the 50 most-flaky tests first to prove value. **(4)** Migrate by feature area; delete StaleElementReferenceException handlers. **(5)** The next question: measure flakiness before/after first 50, sells the rest. **(6)** Avoid: big-bang rewrite that loses coverage during transition.

CODE

```
// Migration: page-object translation
// Selenium (Java)
// WebElement signIn = driver.findElement(By.id("sign-in"));
// wait.until(ExpectedConditions.elementToBeClickable(signIn));
// signIn.click();

// Playwright (TypeScript): the explicit wait disappears, auto-waiting handles it
await page.getByRole('button', { name: 'Sign In' }).click();

// Metrics to track during migration
// flakiness rate (per 100 runs)
// median execution time per test
// CI pipeline duration
// explicit-wait statements removed
```

Q 311

LEVEL · Mid

How do you test infinite scroll pagination?

CONCEPT

Infinite scroll loads more content when the user reaches the bottom. The reliable test pattern: capture initial count, **scroll to the bottom** with page.evaluate, then wait for the loading indicator to appear and disappear, both transitions matter, because waiting only on visible races the load and waiting only on hidden races the initial scroll, then capture new count and assert it grew. The appear-then-disappear pattern works for any async loading indicator, not just infinite scroll. For testing edge cases, scroll to bottom repeatedly to load multiple pages and assert the eventual count matches what the backend should return, and scroll past the documented end to verify the indicator stays hidden when no more data exists.

INTERVIEW STRATEGY

What earns the tick is the appear-then-disappear pattern, not a fixed sleep. Lead with capturing initial count, explicit scrollTo, then both loading-indicator transitions, then asserting count grew. The dual-transition wait is the senior signal. Where this goes wrong is page.waitForTimeout after the scroll. The follow-up is often 'why wait for the indicator to disappear, not just appear?', answer waiting only on visible races the load completion; you need both transitions.

How to phrase it

> *Infinite scroll loads more content when the user reaches the bottom, and the reliable test pattern is appear-then-disappear on the loading indicator. I capture the initial product count, scroll to the bottom with page.evaluate and scrollTo zero document body scrollHeight, then I wait for the loading indicator to be visible and then for it to be hidden. Both transitions matter, and that's my answer to the follow-up about why both, waiting only on visible races the load completion, because the indicator could be visible but the new items not yet rendered, and waiting only on hidden races the initial scroll, because the indicator might not have appeared yet so hidden is trivially true. The dual transition gives me a deterministic signal that loading actually started and actually finished. Then I capture the new count and assert it's greater than the initial. The reason this matters as a general pattern, beyond infinite scroll, is that appear-then-disappear works for any async loading indicator, modal spinners, file uploads, search autocomplete, anywhere a transient indicator marks the lifecycle of an async operation. For edge cases I scroll repeatedly to load multiple pages and assert the eventual count matches what the backend should return for the filter, and I scroll past the documented end to verify the indicator stays hidden when there's no more data. The trap is page.waitForTimeout after the scroll, either too short and flaky on slow loads, or too long and slow on every run. The dual transition adapts to actual load time.*

Key points to hit

(1) Capture initial count; explicit scrollTo bottom; capture new count. **(2)** Wait for loading indicator: visible THEN hidden, both matter. **(3)** Visible-only races completion; hidden-only races the scroll. **(4)** Pattern generalises: any transient async indicator (spinners, uploads). **(5)** Asked next: both transitions because each alone has a race. **(6)** The easy mistake: waitForTimeout, too short flakes, too long is slow.

CODE

```
test('infinite scroll loads more items', async ({ page }) => {
await page.goto('/products');
const initial = await page.locator('.product-item').count();

await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));

// appear-then-disappear: deterministic
await page.locator('#loading-more').waitFor({ state: 'visible' });
await page.locator('#loading-more').waitFor({ state: 'hidden' });

const after = await page.locator('.product-item').count();
expect(after).toBeGreaterThan(initial);
});
```

Q 312 LEVEL · Mid

How do you test drag and drop interactions?

CONCEPT

Playwright's **dragTo** handles the most common drag-and-drop cases simply: source.dragTo(target) sends the right sequence of events for HTML5 drag-and-drop. After the drop, verify the item is in the target container (and out of the source). For **complex cases** like react-dnd or libraries with custom drag handlers, dragTo may not fire the right events; fall back to **mouse.move/down/move/up** with boundingBox positions to simulate the gesture manually. For file drag-and-drop (dropping a file from the OS into a drop zone), construct a DataTransfer with the file in page.evaluateHandle and dispatch drop events. The discipline: try dragTo first, only escalate to manual mouse events when the simple form fails.

INTERVIEW STRATEGY

This question is there to see escalation by complexity. Lead with dragTo for the common case, then mouse events for custom drag libraries, then DataTransfer for file drops. After-the-drop verification on both source and target is the senior detail. The classic error is reaching for mouse events before trying dragTo. Expect the follow-up ‘what if dragTo doesn't work?’, answer fall back to mouse.move + down/up sequence using boundingBox positions.

How to phrase it

> *I escalate by complexity. The common case is Playwright's dragTo: source-locator dragTo target-locator handles standard HTML5 drag-and-drop by sending the right event sequence. After the drop I verify the item is in the target container, and I would also verify it's no longer in the source container, because the move semantics need both checks, otherwise I'm testing that the item appeared in the target without confirming it was actually moved. The failure mode is reaching for mouse events first when dragTo would work fine; always try the simple form. That answers the follow-up about what to do when dragTo doesn't work, which happens with libraries like react-dnd or custom drag handlers that don't respond to the events dragTo fires: I fall back to a manual mouse-event sequence using boundingBox to compute positions, mouse.move to the source center, mouse.down, mouse.move to the target center in incremental steps so the library's onMouseMove handler fires repeatedly, then mouse.up at the target. The incremental steps matter for some libraries that rely on continuous mouse movement to register a drag. For file drag-and-drop, dropping a file from the OS into a drop zone, neither dragTo nor mouse events work, I construct a DataTransfer in page.evaluateHandle with the file content and dispatch drop events on the target. So the rule is: dragTo first, mouse events second for custom libraries, DataTransfer for file drops. Each is a deliberate escalation, not a default.*

Key points to hit

(1) Try dragTo first: handles standard HTML5 drag-and-drop. **(2)** Verify item IN target AND OUT of source after the drop. **(3)** Custom drag libs (react-dnd): fall back to mouse.move/down/up + boundingBox. **(4)** File drag-and-drop: DataTransfer via page.evaluateHandle + dispatch drop. **(5)** Follow-up: dragTo fails -> manual mouse sequence with incremental moves. **(6)** Trap: reaching for mouse events first when dragTo would work.

CODE

```
test('drag item to reorder list', async ({ page }) => {
await page.goto('/kanban-board');
const card = page.locator('.card').filter({ hasText: 'Task A' });
const target = page.locator('.column').filter({ hasText: 'In Progress' });
const source = page.locator('.column').filter({ hasText: 'Backlog' });

await card.dragTo(target); // simple case

await expect(target.locator('.card').filter({ hasText: 'Task A' })).toBeVisible();
await expect(source.locator('.card').filter({ hasText: 'Task A' })).toBeHidden();
});

// Fallback for react-dnd / custom drag handlers
// const s = await source.boundingBox(); const t = await target.boundingBox();
// await page.mouse.move(s!.x + s!.width/2, s!.y + s!.height/2);
// await page.mouse.down();
// await page.mouse.move(t!.x + t!.width/2, t!.y + t!.height/2, { steps: 10 });
// await page.mouse.up();
```

Q 313 LEVEL · Mid

How do you test file upload and download flows?

CONCEPT

File upload uses **setInputFiles** on the file input element, accepting either a path on disk or a Buffer with specified name and mimeType. For drag-and-drop file upload (no input element), construct a DataTransfer in page.evaluateHandle and dispatch a drop event. File download uses **waitForEvent('download')** with **Promise.all** so the listener is registered before the click that triggers the download, a common race-condition trap if you click first and listen second. After receiving the download object, validate the suggested filename, save to a temp path, and assert the saved file size or content is what you expect. For large downloads, assert size only to keep tests fast; for content-critical downloads (a CSV export, an invoice PDF), read and parse the file to assert specific values.

INTERVIEW STRATEGY

You need to give both upload and download patterns correctly. Upload: setInputFiles for standard inputs, DataTransfer for drag-drop uploads. Download: Promise.all-with-waitForEvent is the centerpiece because the listener-after-click race is where most people fail. Then size-vs-content validation per file criticality. The trap is clicking the download trigger before registering the download listener. They usually follow with 'why Promise.all?', answer it registers the listener before the click fires the download event, avoiding the race.

How to phrase it

> *I handle upload and download separately because they're different problems. For file upload, the standard case is setInputFiles on the file input element, accepting either a path on disk or a Buffer with name and mimeType specified. That works whenever the upload uses a regular file input. For drag-and-drop file upload, where there's no input element to target, I construct a DataTransfer in page.evaluateHandle and dispatch a drop event on the drop zone, because the page expects the file to arrive via the DataTransfer API. For file download, the critical pattern and the one most candidates get wrong is waitForEvent('download') wrapped in a Promise.all with the click that triggers the download. That's also my answer to the follow-up about why Promise.all: it registers the listener before the click fires the download event, avoiding the race condition where the click happens, the download event fires, and only then the test starts listening, by which point the event is gone and the test hangs waiting forever. After receiving the download object I validate the suggested filename, save to a temp path, and assert on the saved file's size or content depending on how critical the file is. For a large download I assert size only to keep the test fast, but for a content-critical download like a CSV export or an invoice PDF, I read and parse the file to assert specific values, because a download that succeeds in delivering the wrong content is the worst kind of regression. The mistake is clicking first and listening second, which is the race that Promise.all exists to prevent.*

Key points to hit

(1) Upload: setInputFiles for standard inputs; DataTransfer for drag-drop. **(2)** Download: Promise.all([waitForEvent('download'), click]), listener first. **(3)** Validate suggested filename, save to temp, assert size or content. **(4)** Large downloads: size-only; content-critical: read and parse. **(5)** Expect next: Promise.all registers listener BEFORE click fires the event. **(6)** Where this goes wrong: click first, listen second, the download event is already gone.

CODE

```
// Upload: standard input
await page.getByLabel('Choose file').setInputFiles('tests/fixtures/report.pdf');
// Or by buffer:
// await input.setInputFiles({ name: 'r.pdf', mimeType: 'application/pdf', buffer: buf
});

// Download: listener-before-click via Promise.all
const [download] = await Promise.all([
page.waitForEvent('download'),
page.getByRole('button', { name: 'Export CSV' }).click(),
]);
expect(download.suggestedFilename()).toBe('orders-export.csv');
const path = await download.path();
expect(fs.statSync(path!).size).toBeGreaterThan(0);

// Content-critical: parse and assert specific values
const csv = fs.readFileSync(path!, 'utf8');
expect(csv.split('\n')[0]).toBe('order_id,customer_email,total');
```

Q 314

LEVEL · Mid

How do you test internationalisation (i18n) across multiple locales?

CONCEPT

Apply **test.use({ locale })** at the describe-block level so every test inside runs against that locale and the browser sends the matching Accept-Language header. Pair it with **test.use({ timezoneId })** when testing locale-specific date formatting, because the browser's timezone affects date rendering independently of the locale. Assert translated strings from a locale fixture rather than hard-coding the translated text in the test, if the translation changes, you update the fixture once, not every test. Test **locale-dependent formatting** separately: dates (31/10/2026 vs 10/31/2026), numbers (1.234,56 vs 1,234.56), currencies (€1.234,56 vs $1,234.56). The discipline that matters: a locale block is self-documenting, the describe name announces which locale is under test, so a reader understands the context immediately.

INTERVIEW STRATEGY

They are after disciplined locale handling, not hard-coded translations. Lead with test.use locale at the describe level, then the locale-fixture for translated strings, then the timezone pairing for date tests. The single-edit maintainability of a fixture is the senior signal. The mistake is asserting hardcoded translated text in every test, a single translation tweak breaks dozens. The next question is normally ‘what about date and number formatting?’, answer assert formatting separately with explicit timezone, because the browser timezone affects rendering independently of locale.

How to phrase it

> *I apply test.use locale at the describe-block level so every test inside runs against that locale and the browser sends the matching Accept-Language header. The describe block naming the locale makes the suite self-documenting, anyone reading it sees immediately which locale is under test. I pair it with test.use timezoneId when testing locale-specific date formatting, because the browser's timezone affects date rendering independently of the locale, and this is also my answer to the follow-up about date and number formatting: I test those separately with an explicit timezone set, asserting that 31-slash-10-slash-2026 renders in French and 10-slash-31-slash-2026 in US English, that the number one thousand two hundred thirty-four point fifty-six uses a comma decimal in German and a period in English, and so on. The discipline I would emphasise is that I assert translated strings from a locale fixture rather than hardcoding the translated text in each test. So I have a locale fixture exporting strings dot frFR dot welcomeMessage, and tests reference that. The reason is single-edit maintainability: when marketing changes a translation, I update the fixture once and every test stays green, instead of hunting through dozens of tests to update the literal. The mistake is hardcoding translated strings everywhere; the first translation tweak breaks twenty tests and nobody understands why. Locale fixture plus test.use at describe level is what makes i18n testing maintainable at scale.*

Key points to hit

(1) test.use({ locale }) at describe-block level: self-documenting structure. **(2)** Pair with test.use({ timezoneId }) for date-formatting tests. **(3)** Assert translated strings from a locale fixture, not hardcoded literals. **(4)** Test locale-dependent formatting (dates, numbers, currencies) separately. **(5)** They usually ask next: timezone affects date rendering independently of locale. **(6)** What loses the point: hardcoded translations in tests, one tweak breaks many tests.

CODE

```
// Locale block: self-documenting
test.describe('French locale: fr-FR', () => {
test.use({ locale: 'fr-FR', timezoneId: 'Europe/Paris' });

test('welcome page renders in French', async ({ page }) => {
await page.goto('/');
await expect(page.getByRole('heading'))
.toHaveText(locales.frFR.welcomeMessage); // from fixture, not hardcoded
});

test('dates render in DD/MM/YYYY format', async ({ page }) => {
await page.goto('/orders/123');
await expect(page.locator('[data-test="order-date"]'))
.toHaveText('31/10/2026'); // French format
});
});
```

Q 315

LEVEL · Mid

How do you test dark mode and other CSS theme variants?

CONCEPT

test.use({ colorScheme: 'dark' }) tells the browser to prefer dark mode via the prefers-color-scheme media query, which is how the application detects the user's theme preference. Pair it with visual snapshot comparison using separate dark-mode baselines, toHaveScreenshot with a name like 'dashboard-dark.png', because a single shared baseline would constantly fail when colours legitimately differ. For programmatic theme verification beyond pixels, **page.evaluate** reads CSS custom properties from :root, --color-bg, --color-text, --color-primary, and asserts they match the expected dark values. The combination: colorScheme for setup, visual snapshots for full-page regression, CSS custom properties for token-level assertions. Test the **theme switcher** separately by toggling and asserting both the colorScheme media query response and the persisted preference (cookie or localStorage).

INTERVIEW STRATEGY

They want to see test.use colorScheme as the setup, then dual verification: visual baselines per theme AND CSS custom properties for token assertions. The per-theme-baseline detail is critical because a single shared baseline would fail every test. The thing to avoid is one baseline across light and dark modes. A common follow-up is 'how do you verify the theme tokens specifically?', answer page.evaluate reads :root CSS custom properties and asserts on the values.

How to phrase it

> *test.use colorScheme dark tells the browser to prefer dark mode via the prefers-color-scheme media query, which is how the application detects the user's theme preference, and that's the setup. The verification is dual-layered because dark mode testing has two distinct dimensions. First, visual snapshot comparison with separate dark-mode baselines, I name them like dashboard-dark.png and dashboard-light.png, never sharing a baseline across themes. That's the trap I would call out: a single shared baseline fails the moment colours legitimately differ, which is on every dark-mode test, so the suite becomes useless. Per-theme baselines isolate real regressions from expected theme differences. Second, and this is my answer to the follow-up about verifying theme tokens specifically, I use page.evaluate to read CSS custom properties directly from the root element, the dash-dash-color-bg, dash-dash-color-text, dash-dash-color-primary values, and assert they match the expected dark values. That's token-level verification that survives pixel-level variation, which matters because anti-aliasing differences can cause visual diffs that aren't real theme changes, but a CSS custom property value is exact. I also test the theme switcher itself separately by toggling and asserting both the colorScheme media query response updates and the preference persists, usually in a cookie or localStorage, because a theme that doesn't persist across reloads is a bug. So: colorScheme for setup, per-theme baselines for visual, page.evaluate on CSS tokens for programmatic verification.*

Key points to hit

(1) test.use({ colorScheme: 'dark' }) sets prefers-color-scheme media query. **(2)** Separate dark-mode baselines (dashboard-dark.png, never shared). **(3)** page.evaluate reads :root CSS custom properties for token assertions. **(4)** Test the theme switcher: persistence in cookie/localStorage. **(5)** The next question: verify tokens via CSS custom properties, exact, not pixel-based. **(6)** The common miss: one baseline across themes, fails on every theme test.

CODE

```
test.describe('Dark mode', () => {
test.use({ colorScheme: 'dark' });

test('dashboard renders in dark mode', async ({ page }) => {
await page.goto('/dashboard');
await expect(page).toHaveScreenshot('dashboard-dark.png'); // separate baseline
});

test('CSS custom properties match dark tokens', async ({ page }) => {
await page.goto('/');
const tokens = await page.evaluate(() => {
const s = getComputedStyle(document.documentElement);
return {
bg: s.getPropertyValue('--color-bg').trim(),
text: s.getPropertyValue('--color-text').trim(),
};
});
expect(tokens.bg).toBe('#0f172a');
expect(tokens.text).toBe('#f1f5f9');
});
});
```

Q 316

LEVEL · Mid

How do you test flows that require browser permissions (geolocation, notifications, camera)?

CONCEPT

Playwright grants permissions per browser context, not per page, via **context.grantPermissions(['geolocation', 'notifications'])**. Set the geolocation coordinates explicitly with **context.setGeolocation({ latitude, longitude })** so the test is deterministic rather than tied to the CI machine's actual location. The pattern that distinguishes a complete answer: test both the granted and denied paths. The granted path verifies the feature works when permission is given; the denied path, by not granting and clearing any previously granted permissions with context.clearPermissions, verifies the UI handles refusal gracefully with the right error message and fallback behaviour. The denial test is the one most candidates skip and it's where real bugs hide: an app that crashes when a user blocks notifications because the developer assumed permission would always be granted.

INTERVIEW STRATEGY

Your job here is to show both the happy and denied paths, not just ‘grant permission and assert’. Lead with grantPermissions plus setGeolocation for determinism, then make the denied-path test the centerpiece because it's what most candidates miss. The clearPermissions + assert fallback UI is the experience marker. The classic error is only testing the granted path. The follow-up is often ‘what about when the user denies?’, answer that's the test that catches real bugs; assert the error message and fallback behaviour.

How to phrase it

> *Playwright grants permissions per browser context, not per page, via context.grantPermissions with the permissions array. For geolocation I always pair it with context.setGeolocation, passing latitude and longitude explicitly so the test is deterministic rather than tied to whatever location the CI machine happens to be in. Without the explicit coordinates the test is flaky on a VM that returns a default location or no location at all. But the pattern that distinguishes a complete answer, and my answer to the follow-up about what happens when the user denies, is testing both the granted and denied paths. The granted path is the obvious half: grant the permission, set the coordinates, verify the feature works, say, the nearby-stores list shows the right shops for that location. The denied path is the half most candidates skip, and it's where the real bugs hide. I use context.clearPermissions to ensure no permission is granted, navigate to the page that requests it, and verify the UI handles refusal gracefully, the right error message, the right fallback behaviour, perhaps a manual location entry option. I have seen apps crash with an unhandled rejection when a user blocks notifications because the developer assumed permission would always be granted, and that bug would have been caught by a denied-path test. So the pattern is two tests per permission-gated feature: granted-path proves it works, denied-path proves it fails gracefully. The thing to avoid is only testing the granted path.*

Key points to hit

(1) context.grantPermissions per context (not per page). **(2)** Set explicit coordinates with context.setGeolocation for determinism. **(3)** Test BOTH the granted and denied paths. **(4)** Use context.clearPermissions for the denied path. **(5)** Asked next: denied-path tests catch real crashes when users block. **(6)** Avoid: testing only the happy granted path.

CODE

```
// Granted path: deterministic location
test('shows nearby stores when geolocation granted', async ({ browser }) => {
const ctx = await browser.newContext();
await ctx.grantPermissions(['geolocation']);
await ctx.setGeolocation({ latitude: 51.5074, longitude: -0.1278 }); // London
const page = await ctx.newPage();
await page.goto('/nearby');
await expect(page.locator('.store-card').first()).toContainText('London');
});

// Denied path: the test most candidates skip
test('shows manual entry when geolocation denied', async ({ browser }) => {
const ctx = await browser.newContext();
await ctx.clearPermissions(); // ensure no permission
const page = await ctx.newPage();
await page.goto('/nearby');
await expect(page.getByText('Enter your postcode')).toBeVisible();
});
```

Q 317 LEVEL · Mid to Senior

How do you test Progressive Web App (PWA) offline mode?

CONCEPT

PWA offline testing verifies the service worker serves cached content when the network is unavailable, and that the **sync queue** processes pending actions when connection restores. Simulate network loss with **context.setOffline(true)**. The full lifecycle test: load the page online so the service worker can register and cache, go offline, navigate to a previously-cached page, verify it renders from cache; then perform an action that would normally hit the network (post a comment), verify it's queued locally with a pending indicator; finally **context.setOffline(false)** and verify the queued action syncs and the pending indicator clears. The trap candidates fall into: testing offline-rendering only and skipping the sync-on-reconnect half. The sync-queue behaviour is where the hard PWA bugs live, actions lost on reconnect, duplicate submissions, ordering bugs.

INTERVIEW STRATEGY

Show the full offline lifecycle, not just 'goes offline and shows cached content'. Lead with the three-phase test: online-cache, offline-render, online-sync. The sync-on-reconnect verification is the senior signal because it's where real PWA bugs hide. The thing to avoid is testing only the offline-render half. Expect the follow-up 'what's the hardest part to test?', answer the sync queue processing on reconnect, actions lost, duplicates, ordering bugs all live there.

How to phrase it

> *PWA offline testing has two dimensions and a complete answer covers both: the service worker serves cached content when offline, and the sync queue processes pending actions when connection restores. I structure the test as a three-phase lifecycle. Phase one, load the page online so the service worker can register and cache its content. Without this, the offline test is meaningless because nothing's cached. Phase two, go offline with context.setOffline true, navigate to a previously-cached page, and verify it renders from cache. That proves the service worker is serving correctly. Phase three, and this is the half most candidates skip and my answer to the follow-up about the hardest part, is the sync-on-reconnect verification. While offline, I perform an action that would normally hit the network, post a comment, save a draft, and verify it's queued locally with a pending indicator visible in the UI. Then I call context.setOffline false to restore the network and verify two things: the queued action actually syncs to the server, and the pending indicator clears. That's where the hard PWA bugs live. Actions lost on reconnect because the sync queue cleared without successfully submitting. Duplicate submissions because the sync queue retried without idempotency. Ordering bugs because two queued actions arrived out of order. The thing to avoid is testing only the offline-render half and shipping a PWA where users lose data the moment they reconnect, which is the worst possible failure mode for an offline-first app because users trust offline mode to preserve their work.*

Key points to hit

(1) Three-phase lifecycle: online-cache -> offline-render -> online-sync. **(2)** context.setOffline(true) to simulate network loss. **(3)** Verify cached pages render AND pending actions queue with indicator. **(4)** context.setOffline(false): verify queue syncs and indicator clears. **(5)** Follow-up: sync queue is hardest, lost actions, duplicates, ordering. **(6)** The easy mistake: testing only offline-render and missing the sync-on-reconnect.

CODE

```
test('PWA syncs queued actions on reconnect', async ({ browser }) => {
const ctx = await browser.newContext();
const page = await ctx.newPage();

// 1. Online: service worker registers and caches
await page.goto('/blog');
await page.waitForFunction(() => navigator.serviceWorker.controller);

// 2. Offline: cached page renders + action queues
await ctx.setOffline(true);
await page.reload();
await expect(page.locator('h1')).toBeVisible(); // from cache
await page.getByLabel('Comment').fill('Great post');
await page.getByRole('button', { name: 'Post' }).click();
await expect(page.locator('.sync-pending')).toBeVisible();

// 3. Online: queue syncs, indicator clears
await ctx.setOffline(false);
await expect(page.locator('.sync-pending')).toBeHidden();
// and verify the comment appears server-side on reload
});
```

Q 318 LEVEL · Mid to Senior

How do you test integrations with third-party APIs (Stripe, Twilio, SendGrid)?

CONCEPT

Never call real third-party APIs in automated tests, the rule that protects you from rate limits, cost, flakiness, and PCI/compliance concerns. Use page.route to mock the third-party endpoints and serve deterministic responses for both success and failure paths. Test the **success response** (payment authorised, SMS sent, email delivered) and the **failure responses** the third party can actually return: 402 insufficient funds, 429 rate limit, 503 service unavailable, malformed payloads. The discipline that separates senior answers: include contract tests against the third party's sandbox environment, run them **nightly not per-PR**, so you catch when the third party changes its API, the production-quality bug regular mocked tests will never find.

INTERVIEW STRATEGY

They are looking to see the never-call-real-third-parties rule plus the contract-test escape valve. Lead with mocking via route() for both success and the specific failure responses (402, 429, 503). The nightly-contract-test pattern is what separates junior from senior thinking because it catches third-party API drift. The failure mode is calling real third-party APIs in tests, cost, rate limits, PCI. They usually follow with ‘how do you catch third-party API changes?’, answer nightly contract tests against their sandbox, separate from the mocked regression suite.

How to phrase it

> *The fundamental rule, and what I would lead with because it shows operational maturity, is never call real third-party APIs in automated tests. The reasons compound: rate limits, real money costs for every Stripe call, flakiness from the third party's own latency, and PCI or compliance concerns for payment data. So I use page.route to mock the third-party endpoints and serve deterministic responses. The pattern is to test both the success path, payment authorised, SMS sent, email delivered, and the specific failure responses the third party can actually return. For Stripe that's 402 insufficient funds, 429 rate limit, 503 service unavailable, plus malformed payloads that the integration code might choke on. Testing only success is the easy mistake; the failure responses are where integration bugs hide because the happy path is what developers write code for. But there's a gap in mocked tests, and this is also my answer to the follow-up about how I catch third-party API changes: mocked tests pass forever even after the third party changes their API, because the mock is frozen in time. So I add a separate set of contract tests that run against the third party's actual sandbox environment, with their test API keys, and I schedule those nightly rather than per-PR. If Stripe changes their response shape or deprecates a field, the nightly contract test catches it before it reaches production, even though every mocked regression test is still green. The trap is either extreme: calling real third parties on every test, which is expensive and flaky, or only ever mocking, which silently drifts.*

Key points to hit

(1) NEVER call real third-party APIs in regular tests (cost, rate limits, PCI). **(2)** Use page.route to mock both success and specific failure responses. **(3)** Test failure responses the API actually returns: 402, 429, 503, malformed. **(4)** Add nightly contract tests against the third party's sandbox. **(5)** Expect next: nightly contract tests catch API drift mocks would miss. **(6)** Trap: only mocking (silent drift) OR calling real APIs in tests (cost/flake).

CODE

```
// Mocked test: deterministic failure response
test('checkout shows error when Stripe returns 402', async ({ page }) => {
await page.route('**/api/payments', route => route.fulfill({
status: 402,
body: JSON.stringify({ error: 'insufficient_funds' }),
}));
await page.goto('/checkout');
await page.getByRole('button', { name: 'Place Order' }).click();
await expect(page.locator('.error')).toContainText('insufficient funds');
});

// Nightly contract test: against Stripe sandbox
// .github/workflows/contract-tests.yml schedules: "0 2 * * *"
// Hits api.stripe.com with test keys; catches API drift mocks miss.
```

Q 319 LEVEL · Mid to Senior

How do you implement automated accessibility testing in Playwright?

CONCEPT

Automated accessibility testing uses **@axe-core/playwright** to run WCAG audits programmatically against each page, inject axe and call analyze() to get a list of violations with rule IDs, impact (critical, serious, moderate, minor), and the elements affected. Pair it with **toMatchAriaSnapshot** for ARIA tree validation, which catches structural accessibility regressions (a button losing its name, a heading level changing) that axe might not flag. Test keyboard navigation explicitly with **page.keyboard.press**, Tab through interactive elements, verify focus indicators are visible, verify the tab order is logical. The honest boundary: automated tools catch about 50% of real accessibility issues; the rest, colour contrast in context, screen-reader announcements, cognitive load, needs manual testing with actual assistive technology and ideally users with disabilities.

INTERVIEW STRATEGY

The point of this one is to see automated coverage plus its honest limits. Lead with axe-core for WCAG audits, toMatchAriaSnapshot for structural, keyboard navigation for interaction. The 50%-coverage statement is the senior signal because it shows you know automation isn't enough. The classic error is claiming axe catches all issues. The next question is normally ‘what does axe miss?’, answer colour contrast in context, screen-reader announcements, cognitive load, manual testing with assistive tech is still essential.

How to phrase it

> *Automated accessibility testing is what I run on every PR, but I would lead with the honest framing because interviewers value calibration: automated tools catch roughly fifty percent of real accessibility issues, and the rest needs manual testing with actual assistive technology. That said, automating the fifty percent is high-value because it catches regressions continuously. I use at-axe-core-slash-playwright to run WCAG audits programmatically against each page, I inject axe and call analyze, which returns a list of violations with rule IDs, the impact level from critical down to minor, and the elements affected. I fail the test on any critical or serious violation. I pair it with toMatchAriaSnapshot for ARIA tree validation, because that catches structural accessibility regressions axe might not flag, a button accidentally losing its accessible name, a heading level changing from h2 to div, navigation losing its landmark role. I also test keyboard navigation explicitly with page.keyboard.press Tab, cycling through interactive elements, verifying focus indicators are actually visible, verifying the tab order is logical and matches the visual order. The follow-up about what axe misses is where the honest answer matters: colour contrast in dynamic contexts, axe checks static contrast but not, for example, text over a background image. Screen-reader announcements: what does VoiceOver actually say when this dialog opens? Cognitive load and form complexity. So I automate what I can and explicitly schedule manual accessibility review with assistive tech for major releases, ideally with users who have disabilities. The failure mode is claiming axe catches everything; calibration is the senior signal.*

Key points to hit

(1) @axe-core/playwright for programmatic WCAG audits, inject + analyze. **(2)** Fail on critical/serious violations; gate every PR. **(3)** toMatchAriaSnapshot catches structural regressions axe might miss. **(4)** page.keyboard.press for tab order, focus visibility, keyboard reachability. **(5)** They usually ask next: automation catches ~50%, manual + assistive tech still essential. **(6)** Where this goes wrong: claiming axe catches all a11y issues, it doesn't.

CODE

```
import AxeBuilder from '@axe-core/playwright';

test('checkout has no critical accessibility violations', async ({ page }) => {
await page.goto('/checkout');
const results = await new AxeBuilder({ page })
.withTags(['wcag2a', 'wcag2aa'])
.analyze();
const critical = results.violations.filter(v => ['critical','serious'].includes(v.impact
?? ''));
expect(critical).toEqual([]);
});

test('navigation ARIA tree is stable', async ({ page }) => {
await page.goto('/');
await expect(page.locator('nav')).toMatchAriaSnapshot();
});

test('keyboard navigation cycles through interactive elements', async ({ page }) => {
await page.goto('/checkout');
await page.keyboard.press('Tab');
await expect(page.getByLabel('Card number')).toBeFocused();
});
```

Q 320

LEVEL · Senior

How do you prove test coverage to stakeholders in a way they can act on?

CONCEPT

Prove coverage via a **requirement traceability matrix** built from requirement-ID tags on tests (@REQ-101, @REQ-102), a script that parses results into a **requirements coverage dashboard** showing which have passing tests, failing tests, or no test at all, and a risk matrix showing test allocation by business impact (payment processing: 50 tests; about page: 2). The principle: stakeholders need business-language coverage, not technical metrics. A product manager understands ‘REQ-103 has no automated test’; they don't understand ‘73% branch coverage’. The risk matrix is what justifies the spend asymmetry to stakeholders, you're investing where the risk is, not testing everything equally. The dashboard's most actionable column is **requirements with no test**, because that's the gap someone can decide to close or accept; everything else is informational.

INTERVIEW STRATEGY

You are being asked to show stakeholder-aware communication, not technical metrics. Lead with requirement traceability matrix via @REQ tags, then the risk matrix that justifies investment. The 'PM understands REQ-103 has no test' contrast against '73% branch coverage' is the senior framing. The classic error is reporting code coverage to product. A common follow-up is ‘what's the most actionable column?’, answer the requirements with no test, because it's the gap someone can act on.

How to phrase it

> *I prove coverage through three artefacts, each calibrated to a different stakeholder audience, because speaking the audience's language is the senior skill here. Artefact one is a requirement traceability matrix built from requirement-ID tags on tests, at-REQ dash one zero one, at-REQ dash one zero two, plus a script that parses the test results and generates a requirements coverage dashboard. The dashboard shows which requirements have passing tests, which have failing tests, and which have no test at all. That's also my answer to the follow-up about the most actionable column: the requirements with no test, because that's the gap a product manager can act on, decide to invest in coverage or accept the risk, whereas everything else is informational. Artefact two is a risk matrix showing test allocation by business impact: payment processing has fifty tests, the about page has two. The matrix is what justifies the spend asymmetry to stakeholders, I'm investing where the risk is, not testing everything equally, and the matrix makes that visible. Artefact three is the dashboard trend over time so they can see coverage improving release over release. The principle I lead with, because it lands: stakeholders need business-language coverage, not technical metrics. A product manager understands REQ-103 has no automated test; they don't understand seventy-three percent branch coverage, and even if they did, it doesn't tell them which requirement is at risk. The classic error is reporting code coverage percentages to product, the number can be high while a critical requirement sits completely untested, so it's actively misleading. Requirement language is what makes coverage actionable.*

Key points to hit

(1) Three artefacts: traceability matrix, risk matrix, trend dashboard. **(2)** Tag every test with @REQ-ID; script parses results into requirement coverage. **(3)** Risk matrix justifies asymmetric investment (payment 50 tests, about page 2). **(4)** Report in business language; PMs understand 'REQ-103 has no test'. **(5)** The next question: 'no test at all' is the actionable column. **(6)** What loses the point: reporting code-coverage percentages to product, misleading.

CODE

```
// Requirement-ID tags drive traceability
test('user logs in', { tag: ['@REQ-101', '@smoke'] }, async ({ page }) => { /* ... */
});
test('invalid password error', { tag: ['@REQ-102', '@regression'] }, async ({ page }) =>
{ /* ... */ });

// Script generates a stakeholder dashboard
// npx playwright test --reporter=json | node scripts/coverage-report.js
//
// Output:
// Requirement Status Tests Owner
// REQ-101 ✓ covered (pass) 3 Payments
// REQ-102 ✗ covered (fail) 1 Auth
// REQ-103 ⚠ NO TEST 0 Reports <- actionable gap
```

Q 321 LEVEL · Mid to Senior

When and how do you combine API setup with UI verification?

CONCEPT

The API setup -> UI verification -> API teardown pattern is the fastest and most reliable way to test most flows. Create the data via API (a user, an order, a product), drive the UI to verify the data is correctly displayed and that the UI interaction works, then clean up via API. The advantage over pure UI flows is dramatic: a user creation via UI takes 30 seconds; via API it takes 200ms. A login via UI takes 5-8 seconds; via storageState it takes 0. So a 50-test regression that previously took 30 minutes drops to 5. The judgement call: UI for what only the UI can test (rendering, interaction, navigation, validation messages); **API for everything else** (setup, teardown, preconditions, postconditions). Mixed tests verify the **integration between layers** without duplicating setup logic in the UI.

INTERVIEW STRATEGY

What scores here is showing you optimise speed without losing coverage. Lead with the API-setup -> UI-verify -> API-teardown pattern. The 30s->200ms speedup is the credibility number. The judgement rule, UI for what only UI can test, is the experience marker. The thing to avoid is doing all setup through the UI 'for realism'. The follow-up is often 'how do you decide what to test through the UI?', answer rendering, interaction, validation messages, things the API contract can't verify.

How to phrase it

> *The API setup, UI verification, API teardown pattern is the fastest and most reliable way to test most flows, and I would lead with the speed math because it sells the approach. Creating a user via UI takes around thirty seconds because it walks through registration forms and validation; via API it takes two hundred milliseconds. Logging in via UI takes five to eight seconds; via storageState it takes zero. So a fifty-test regression that takes thirty minutes with UI-based setup drops to around five minutes with API setup, and that's a concrete six-times improvement with zero loss of coverage. The judgement call that defines this pattern, and my answer to the follow-up about how I decide what to test through the UI, is: UI for what only the UI can test, rendering, interaction, navigation, validation messages, the actual user-facing behaviour, API for everything else, which is setup, teardown, preconditions, postconditions. So if I'm testing that the admin panel correctly displays a user's order history, I create the orders via API, navigate to the panel via UI, and assert the rendering. I don't need to drive the UI to create the orders, because that's the order-creation test, not the panel-display test. The trap is doing all setup through the UI for realism, which adds no real coverage, it's already covered by the order-creation test, and dramatically slows the suite. The pattern also verifies the integration between layers without duplicating setup logic. The data goes through the API contract, the UI reads from the same backend, and a mismatch between what the API returns and what the UI renders is caught.*

Key points to hit

(1) Pattern: API setup -> UI verify -> API teardown, fastest reliable form. **(2)** Speed math: 30s UI user creation -> 200ms API; 30min -> 5min for 50 tests. **(3)** UI for what only UI tests: rendering, interaction, validation messages. **(4)** API for setup/teardown/preconditions: it's already covered elsewhere. **(5)** Asked next: UI tests the user-facing behaviour, API tests the data path. **(6)** The common miss: doing all setup via UI 'for realism', slow with no coverage gain.

CODE

```
test('admin sees user order history', async ({ page, request }) => {
// Setup via API: fast and deterministic
const user = await new UserBuilder().asAdmin().create(request);
const order = await new OrderBuilder().forUser(user.id).create(request);

// Verify via UI: only the UI can test this
await page.goto(`/admin/users/${user.id}/orders`);
await expect(page.locator('.order-row').first()).toContainText(order.reference);
await expect(page.locator('.order-total')).toHaveText(`£${order.total}`);

// Teardown via API
await request.delete(`/api/orders/${order.id}`);
await request.delete(`/api/users/${user.id}`);
});
```

Q 322 LEVEL · Senior

How do you handle tests against an unstable or flaky test environment?

CONCEPT

Distinguish **environment flakiness** (the test environment itself is unstable, cold backends, intermittent network, shared databases under contention) from **test flakiness** (the test has a race condition or timing bug). Most candidates conflate them. Three defences against environment flakiness: **environment health check in globalSetup**, fail fast with a clear message if the environment is broken, instead of every test failing mysteriously; retry logic for transient failures via Playwright's retries config for whole-test or withRetry for specific operations; separating metrics so environment flakiness shows in a different dashboard from test flakiness, because they have different owners, the platform team versus the test author, and conflating them sends fixes to the wrong people. The principle: name the source of flakiness explicitly so you can fix the right thing.

INTERVIEW STRATEGY

This question is there to see operational maturity, not 'just add retries'. Lead with distinguishing environment flakiness from test flakiness because conflating them is where teams get stuck. The three defences, health check, retries, separated metrics, form the structure. The platform-team-vs-test-author ownership distinction marks senior judgement. Where this goes wrong is treating all flakiness as test bugs. Expect the follow-up 'how do you tell the two apart?', answer environment health check at start of suite, and run flaky tests in isolation against a clean environment.

How to phrase it

> *The first move is to distinguish environment flakiness from test flakiness, because conflating them is where teams get stuck for months. Environment flakiness is when the test environment itself is unstable, cold backends, intermittent network to a shared service, a database under contention because every team's tests hit it. Test flakiness is when the test has a race condition or timing bug. They have different owners and different fixes, and that distinction matters because if I treat them the same, fixes go to the wrong people and nothing improves. Three defences. First, an environment health check in globalSetup that fails fast with a clear message if the environment is broken, checks the API is responding, the database is reachable, the test users exist. Without it, every test fails mysteriously and three engineers waste a day diagnosing what is actually an infra problem. Second, retry logic for transient failures: Playwright's retries config for whole-test retries on a flaky CI environment, plus my withRetry utility for specific operations that talk to known-unstable external services. Third, and this is the senior move, I separate the metrics. Environment flakiness gets reported in a different dashboard from test flakiness, because they have different owners, the platform team for the environment, the test author for the test, and conflating them sends fixes to the wrong people. That's also my answer to the follow-up about telling the two apart: the health check at suite start plus running the flaky test in isolation against a clean environment tells me which side the problem lives on. The risk is treating all flakiness as test bugs and burning the test team out chasing infra issues.*

Key points to hit

(1) Distinguish environment flakiness from test flakiness, different owners. **(2)** globalSetup health check fails fast on broken environment with clear message. **(3)** Retries config for whole-test; withRetry utility for specific operations. **(4)** Separate dashboards: platform team owns env, test author owns test. **(5)** Follow-up: health check + isolation run distinguishes the two. **(6)** Avoid: treating all flakiness as test bugs; fixes go to wrong people.

CODE

```
// globalSetup: fail fast on broken environment
export default async function globalSetup() {
const r = await fetch(`${process.env.BASE_URL}/api/health`);
if (!r.ok) throw new Error(`ENV UNHEALTHY: /api/health returned ${r.status}. Stopping
suite.`);
// Verify test users exist, DB reachable, etc.
}

// playwright.config.ts: retries for transient infra issues
// retries: process.env.CI ? 2 : 0

// Separate metrics dashboards
// env-flakiness dashboard -> platform-team owns it
// test-flakiness dashboard -> test author owns it
```

Q 323 LEVEL · Senior

How do you test advanced WebSocket scenarios (disconnection, reconnection, frame inspection)?

CONCEPT

Advanced WebSocket testing covers three scenarios beyond basic message delivery. **Frame inspection**: page.on('websocket') hooks into every WebSocket the page opens; each ws.on('framesent') and ws.on('framereceived') lets you log or assert on specific frame payloads, useful for verifying protocol-level behaviour or debugging delivery issues. **Disconnection**: use page.route to abort or block the WebSocket URL after the initial connection, then verify the UI shows a reconnecting state and that pending messages queue or fail gracefully. **Reconnection**: restore the route and verify the connection re-establishes, any queued messages are sent, and presence/state syncs. Combined with the two-context pattern from real-time chat testing, these scenarios cover the full lifecycle of a real-time application, connection, message delivery, disconnection handling, recovery.

INTERVIEW STRATEGY

They want to see you handle the full WebSocket lifecycle, not just message delivery. Lead with the three scenarios: frame inspection (the depth signal), disconnection via route abort, reconnection via route restore. Combined with the two-context pattern is the complete picture. Where this goes wrong is testing only happy-path message delivery. They usually follow with 'how do you simulate disconnection?', answer page.route to abort the WebSocket URL, then restore.

How to phrase it

> *Advanced WebSocket testing goes beyond basic message delivery and covers three scenarios where real production bugs hide. First, frame inspection. The page dot on websocket event hooks into every WebSocket the page opens, and on the resulting ws object I can listen to framesent and framereceived events with the actual payloads. That's useful for two things: verifying protocol-level behaviour, a heartbeat fires every thirty seconds, the right subscription messages are sent on connection, and for debugging delivery issues by inspecting exactly what crossed the wire. Second, disconnection, and this is also my answer to the follow-up about simulating it: I use page.route to abort or block the WebSocket URL after the initial connection has been established, then I verify the UI shows the reconnecting state with the right visual indicator, and that any pending messages either queue for retry or fail gracefully with a clear error to the user. Third, reconnection. I restore the route by calling unroute, verify the WebSocket re-establishes the connection, that any queued messages are sent on reconnect, and that presence or state syncs, because for a chat or collaboration app, what other users did during my disconnection needs to appear when I come back. Combined with the two-context pattern from real-time chat testing, those three scenarios cover the full lifecycle of a real-time application, connection, message delivery, disconnection handling, recovery. The pitfall is testing only happy-path message delivery and shipping a chat app that breaks the moment a user's wifi blinks.*

Key points to hit

(1) Three scenarios: frame inspection, disconnection, reconnection. **(2)** page.on('websocket') + ws.on('framesent'/'framereceived') for frames. **(3)** Disconnection via page.route abort: verify reconnecting UI + message queue. **(4)** Reconnection via unroute: verify re-establish + queued send + state sync. **(5)** Expect next: simulate disconnection by aborting the WebSocket URL via route. **(6)** The easy mistake: testing only happy-path message delivery, missing recovery bugs.

CODE

```
// Frame inspection
page.on('websocket', ws => {
ws.on('framesent', f => sentFrames.push(f.payload));
ws.on('framereceived', f => receivedFrames.push(f.payload));
});
await page.goto('/chat');
expect(sentFrames.some(p => p.toString().includes('subscribe'))).toBe(true);

// Disconnection: abort the WebSocket URL after connection
await page.route('**/ws/**', route => route.abort());
await expect(page.locator('.connection-status')).toContainText('Reconnecting');

// Reconnection: restore the route
await page.unroute('**/ws/**');
await expect(page.locator('.connection-status')).toContainText('Connected');
```

Q 324

LEVEL · Mid

How do you onboard new team members to a Playwright framework so they ship quickly?

CONCEPT

Structured onboarding with **progressive complexity**: run -> read -> pair -> solo -> contribute. Day 1, clone and run the suite locally so they see it work; Day 2, read about ten existing tests to absorb the team's fixture patterns and locator priority; Day 3, pair-write a new test with an experienced engineer narrating the why; Week 2, write a test solo with detailed code review; Week 3, contribute a new page object or fixture and see the framework's architecture. Maintain **CONTRIBUTING.md** with the locator priority list, fixture patterns, and review checklist so new engineers can self-serve most questions before asking. Measure success by **time-to-first-merged-test** as a concrete metric, a good program lands new engineers around 4 days, and that number tells you whether changes to the onboarding are actually helping.

INTERVIEW STRATEGY

Your job here is to show structure and measurement. Lead with the run -> read -> pair -> solo -> contribute arc, then CONTRIBUTING.md for self-service, then time-to-first-merged-test as the measured outcome. The metric signals depth here because most teams onboard without measuring. The classic error is ad-hoc ‘ask me anything’ onboarding. The next question is normally ‘how do you know it works?’, answer time-to-first-merged-test, if it goes up, something in the program regressed.

How to phrase it

> *I structure onboarding with progressive complexity, run, read, pair, solo, contribute, so new engineers build confidence in stages rather than facing the full framework at once. Day one, they clone and run the suite locally so they see it actually work end to end and understand the project structure. Day two, they read about ten existing tests, the well-written ones, to absorb how the team uses fixtures, the locator priority order, the assertion style, the test structure. Reading good code is how engineers learn what good looks like in your codebase. Day three, they pair-write a new test with an experienced engineer who narrates the why, why getByRole, why this fixture, why this assertion pattern, because the reasoning is the thing that doesn't show up in the code itself. Week two, they write a test solo with detailed code review. Week three, they contribute a new page object or fixture and see the framework's architecture from the inside. Two amplifiers make this scale. First, a CONTRIBUTING.md documenting the locator priority list, fixture patterns, common pitfalls, and the review checklist, so a new engineer can self-serve most questions before pinging anyone. Second, and this is my answer to the follow-up about how I know the program works, I measure time-to-first-merged-test as a concrete metric. A good program lands new engineers at first merge around four days. If a change to the onboarding makes that number go up, the change regressed something and I revert. The trap is ad-hoc 'ask me anything' onboarding, it doesn't scale past two new hires, and you can't improve what you don't measure.*

Key points to hit

(1) Progressive complexity: run -> read -> pair -> solo -> contribute. **(2)** Day 1 run, Day 2 read 10 tests, Day 3 pair, Week 2 solo, Week 3 contribute. **(3)** CONTRIBUTING.md (locator priority, fixtures, review checklist) for self-serve. **(4)** Measure time-to-first-merged-test: a good program lands ~4 days. **(5)** They usually ask next: the metric tells you when an onboarding change regressed. **(6)** Trap: ad-hoc onboarding, doesn't scale, can't be improved.

CODE

```
// CONTRIBUTING.md structure
// 1. Run the suite (Day 1)
// 2. Locator priority (Day 2 reading aid)
// getByRole > getByLabel > getByTestId > locator()
// 3. Fixture patterns (Day 2 reading aid)
// 4. Pairing checklist (Day 3)
// 5. Review checklist (Week 2 solo test)
// 6. Architecture overview (Week 3 contribution)
//
// Tracked metric: time-to-first-merged-test
// Target: ~4 days from start date
// Goes up after a change -> revert and try again
```

Q 325 LEVEL · Senior

How do you test a feature flag rollout (enabled, disabled, gradual percentage)?

CONCEPT

Feature flag testing covers three scenarios: flag on, flag off, and flag boundaries. Override the flag via the mechanism the application supports, a cookie, an HTTP header, a query parameter, or a fixture that calls the flag-service API, so tests are deterministic instead of subject to the actual rollout percentage. Three test variants. **Flag on**: override to enabled, verify the new behaviour. **Flag off**: override to disabled, verify the old behaviour still works, critical because a half-implemented feature toggle that breaks the off path is a regression for everyone not in the rollout. **Percentage rollouts**: override per-user identifier so a test user reliably falls inside or outside the bucket. Centralise flag overrides in a fixture (withFeatureFlag) so a test reads withFeatureFlag('new-checkout', true) and the mechanism is one place to update when the flag system changes.

INTERVIEW STRATEGY

Show all three test variants and deterministic override. Lead with flag-on / flag-off / boundary, then the override mechanism via fixture. The flag-off testing is the experience marker because most candidates skip it. The pitfall is testing only the new behaviour with the flag on. A common follow-up is 'what's the most-missed test?', answer the flag-off path, because a broken off-path is a regression for everyone not in the rollout.

How to phrase it

> *Feature flag testing covers three scenarios, and naming all three is the structure the interviewer is looking for. Flag on, flag off, and flag boundaries for percentage rollouts. I override the flag via the mechanism the application supports, a cookie, an HTTP header, a query parameter, or a fixture that calls the flag-service API directly, because tests need to be deterministic, not subject to whatever percentage the actual rollout is on. Variant one, flag on: I override to enabled and verify the new behaviour works as specified. Variant two, flag off, and this is my answer to the follow-up about the most-missed test: I override to disabled and verify the old behaviour still works perfectly. This is the one most candidates skip, but it's critical because a half-implemented feature toggle that breaks the off path is a regression for every single user who isn't in the rollout, typically the majority. I have seen exactly that bug ship, where the new-checkout flag was on for ten percent of users, the tests verified the new flow, and ninety percent of users got a broken old flow because the developer broke a shared component while building the new path. Variant three, percentage rollouts: I override per-user identifier so a specific test user reliably falls inside or outside the bucket, letting me test both sides of the boundary deterministically rather than hoping random assignment lands the right way. I centralise the override mechanism in a fixture, withFeatureFlag, so a test reads withFeatureFlag new-checkout true and the actual override mechanism, cookie, header, API call, is one place to update when the flag system changes. The failure mode is testing only the new behaviour with the flag on and shipping a regression to the off-path users.*

Key points to hit

(1) Three scenarios: flag on, flag off, boundary (percentage rollout). **(2)** Override deterministically (cookie/header/query/API), not via rollout %. **(3)** Flag-off testing is critical: off-path break = regression for majority. **(4)** Per-user override lets you test both sides of percentage boundaries. **(5)** Centralise overrides in a withFeatureFlag fixture, one place to change. **(6)** Where this goes wrong: testing only the new behaviour with the flag on.

CODE

```
// Centralised override fixture
export const test = base.extend<{ withFeatureFlag: (name: string, on: boolean) =>
Promise<void> }>({
withFeatureFlag: async ({ context }, use) => {
await use(async (name, on) => {
await context.addCookies([{
name: `ff_${name}`, value: String(on),
domain: 'localhost', path: '/',
}]);
});
},
});

test('new checkout flow works when flag is ON', async ({ page, withFeatureFlag }) => {
await withFeatureFlag('new-checkout', true);
await page.goto('/checkout');
await expect(page.getByTestId('new-checkout-form')).toBeVisible();
});

test('old checkout flow works when flag is OFF', async ({ page, withFeatureFlag }) => {
await withFeatureFlag('new-checkout', false);
await page.goto('/checkout');
await expect(page.getByTestId('legacy-checkout-form')).toBeVisible(); // regression
guard
});
```

Quiz

5 questions. Recommended time: 8 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. What's the highest-leverage step when a Playwright test passes locally but fails in CI?

A) Add a longer timeout
B) Download the trace from CI and scrub the timeline at trace.playwright.dev
C) Disable parallelism
D) Roll back the last commit

2. Why use two browser contexts when testing a real-time chat application?

A) It runs faster
B) Chat apps require it by spec
C) Separate storageState per user: one context with two pages shares cookies and is one user
D) It avoids using WebSockets

3. What is typically the single highest-impact change when reducing a slow Playwright suite?

A) Increasing parallelism first
B) Removing all assertions
C) Moving UI-based test data setup to API calls / storageState (often 30-40% alone)
D) Switching browsers

4. Why must multi-tenant isolation tests hit the API directly, not just the UI?

A) APIs are faster
B) APIs always return 200
C) It avoids using cookies
D) UI and API enforce access separately; the UI may block tenant B's URLs while the API leaks tenant B's data

5. How should a 500-test Selenium-to-Playwright migration be approached?

A) Phased over 3-4 sprints, both frameworks running in parallel, flakiest tests migrated first
B) Big-bang rewrite over one sprint
C) Migrate alphabetically
D) Migrate only when forced

Section 28 · Answer key

#	Answer	Why and where to revisit
1	B) Download the trace from CI and scrub the timeline at trace.playwright.dev	The trace shows DOM + network at the failing action, most CI-only failures are diagnosed in ~5 minutes. *Revisit Section 28.*
2	C) Separate storageState per user: one context with two pages shares cookies and is one user	Two-user interactions need two independent sessions; one context = one set of cookies = one user. *Revisit Section 28.*
3	C) Moving UI-based test data setup to API calls / storageState (often 30-40% alone)	UI login: 5-8s; storageState: 0s. UI user creation: 30s; API: 200ms. Parallelising slow setup just parallelises waste. *Revisit Section 28.*
4	D) UI and API enforce access separately; the UI may block tenant B's URLs while the API leaks tenant B's data	Attackers don't use the UI; they construct URLs and hit endpoints directly. Both layers must be tested. *Revisit Section 28.*
5	A) Phased over 3-4 sprints, both frameworks running in parallel, flakiest tests migrated first	Phased migration with flakiest-first proves value via flakiness-before/after metrics; coverage never drops. *Revisit Section 28.*

FINAL MOCK QUIZ

30 Questions Across All 28 Sections

This mock quiz draws from every section of this kit. Time yourself: aim to complete 30 questions in 32 minutes. Check your answers and revisit any topic where you miss more than one question in a category.

Quiz

30 questions. Recommended time: 32 minutes. Mark your answers, then check the answer key on the next page. Each wrong answer points you back to the question you should revisit.

Multiple choice · pick one

1. Which tsconfig setting catches the most type bugs in test code without exhaustive per-rule configuration?

A) noImplicitAny
B) strict
C) skipLibCheck
D) esModuleInterop

2. In `function pick<T, K extends keyof T>(obj: T, key: K): T[K]`, what does `K extends keyof T` enforce?

A) K can be any string
B) K must be a key that actually exists on T
C) K must be a string literal
D) K must be readonly

3. Why prefer `interface` over `type` for API response shapes that the tests will extend frequently?

A) Interfaces support declaration merging; `type` does not
B) Interfaces are faster at runtime
C) Interfaces auto-import; `type` does not
D) Interfaces support union types; `type` does not

4. Which locator strategy should you reach for FIRST when the element is interactive?

A) getByRole
B) getByTestId
C) getByText
D) CSS selector

5. A test calls `page.getByRole('button').click()` and Playwright throws a strict mode violation. What's the right fix?

A) Add `.first()`
B) Set `strict: false` in the config
C) Switch to `page.locator('button').click()`
D) Add an accessible-name filter via `{ name: '...' }`

6. What's the difference between `locator.filter({ has })` and `locator.filter({ hasText })`?

A) `has` filters by descendant locator; `hasText` filters by text content
B) There is no difference
C) `has` is for arrays; `hasText` is for strings
D) `has` was deprecated in 2025

7. Before Playwright clicks an element, which actionability checks does it run?

A) Only that the element exists
B) Visible, stable, receives events and enabled
C) All five checks, including editable, before every action
D) Visible only; events are user-driven

8. A test uses `await page.waitForTimeout(2000)` to wait for a modal to appear. What's the right replacement?

A) Increase to 5000ms for safety
B) Use `await page.evaluate(() => new Promise(r => setTimeout(r, 2000)))`
C) Use `await page.waitForLoadState('networkidle')`
D) Replace with `expect(modal).toBeVisible()`

9. When is a Page Object NOT the right abstraction?

A) A multi-step flow used by many tests
B) Pages with dynamic loading
C) Complex form with conditional fields
D) A one-off verification of a specific button on a single test

10. Which assertion auto-retries until the condition holds or the timeout fires?

A) expect(el).toBeVisible()
B) expect(await el.isVisible()).toBe(true)
C) expect(el.count()).toBeGreaterThan(0)
D) assert(el.isVisible())

11. To assert a list is non-empty in a parallel-safe, race-free way:

A) expect(await items.count()).toBeGreaterThan(0)
B) expect(items).not.toHaveCount(0)
C) expect(await items.allInnerTexts()).not.toEqual([])
D) await items.first().waitFor()

12. Which Playwright unit provides full state isolation between tests by default?

A) BrowserContext
B) Browser
C) Page
D) Worker

13. You want to mock a specific endpoint for one test only. Which scope is right?

A) browser.route() at config level
B) page.route() inside the test
C) context.route() in beforeAll
D) Intercept via service worker

14. Why use APIRequestContext for test setup instead of driving the UI?

A) It's 10x+ faster than UI flows and doesn't depend on form/field stability
B) It's the only way to send authenticated requests
C) It shares cookies with page.route mocks
D) It bypasses Playwright's auto-waiting

15. An API SLA test fails intermittently because response times occasionally exceed the 500 ms threshold. The actual p95 is 300 ms. Best fix?

A) Lower the threshold to 300 ms
B) Retry the test 5 times
C) Disable the assertion
D) Set the threshold above the worst-case p99 (e.g. 800 ms) to absorb variance

16. On a 4-vCPU CI runner with 7 GB RAM, what's typically the right Playwright worker count?

A) 4 (= cores)
B) 6-8 (more than cores; browser tests are I/O-bound)
C) 16 (max parallelism)
D) 1 (avoid contention)

17. What's the relationship between Playwright workers and CI shards?

A) They're the same thing
B) Workers parallelise within one CI job; shards parallelise across CI jobs
C) Shards are deprecated in 2025+
D) Workers run sequentially; shards run in parallel

18. Which environment variable shows every Playwright API call with timing, useful for understanding why a CI run failed?

A) PWDEBUG=1
B) PLAYWRIGHT_TRACE=verbose
C) DEBUG=pw:protocol
D) DEBUG=pw:api

19. You need to seed a test database once per worker, not per test. Which fixture scope is right?

A) Default test scope
B) scope: 'browser'
C) scope: 'worker'
D) scope: 'global'

20. In the Meszaros test-doubles taxonomy, the distinction between a 'stub' and a 'mock' is:

A) Stubs are real; mocks are fake
B) Stubs control returned state; mocks verify behaviour (that calls happened as expected)
C) Stubs are for HTTP; mocks are for databases
D) There is no distinction; the terms are synonymous

21. Before trusting an LLM-as-judge assertion in CI, what's the minimum calibration step?

A) Run it once and check the answer looks right
B) Use a more expensive model
C) Hand-label ~50 examples and confirm 80%+ agreement between the judge and humans
D) Add 'be strict' to the prompt

22. What's the right use for `testInfo.annotations` vs `testInfo.attach`?

A) They are the same API
B) Annotations for HTML report; attach for JSON report
C) Annotations for tags/metadata; attach for files and binary diagnostic data

D) Annotations are deprecated; use attach

23. A feature has 5 browsers × 4 plans × 3 countries × 4 payment methods = 240 combinations. Pairwise testing reduces this to roughly:

A) 5 cases
B) 60 cases
C) 20 cases
D) 120 cases

24. Which of these is the strongest single quality metric to report to leadership?

A) Defect escape rate (bugs reaching production)
B) Total test count
C) Code coverage percentage
D) Number of passing tests

25. Playwright's iPhone device emulation sets viewport, user-agent, and touch flags. Which gap matters most for catching iOS-specific bugs?

A) Playwright doesn't emulate the screen brightness
B) Touch events aren't supported in emulation
C) iPhone emulation lacks 5G network simulation
D) Playwright's WebKit ≠ iOS Safari: divergent rendering, memory limits, and CSS support

26. Contract tests (Pact-style) belong in which CI tier?

A) Only nightly/weekly: too slow for PRs
B) Replace e2e entirely
C) Tier 1 (consumer-side) on every PR; Tier 2 (provider verification) on provider PRs; Tier 3 (e2e) on integration
D) Only on release branches

27. What's the LCP target for Core Web Vitals to be considered 'Good' as of 2026?

A) Under 1 second
B) There is no fixed target
C) Under 4 seconds
D) Under 2.5 seconds

28. An authorisation-test matrix for 4 roles × 6 endpoints. To catch the bug where 'viewer' can delete users, which assertion type is essential?

A) Assert admin CAN delete (positive path)
B) Assert the endpoint exists
C) Assert viewer CANNOT delete (negative path expecting 403)
D) Assert all roles can read

29. What problem do branded types like `type UserId = string & { __brand: 'UserId' }` solve?

A) Performance: branded types are faster
B) Prevent the argument-swap bug where UserId and TenantId (both strings) get passed in wrong places
C) Convert TypeScript to runtime-typed
D) Replace zod validation

30. A NoSQL test on DynamoDB writes a record, then immediately reads it and gets stale data. Which fix is correct?

A) Use ConsistentRead: true on the read, OR poll with expect.poll until the read converges
B) Add waitForTimeout(500)
C) Switch to SQL
D) The test should be deleted; it's a flaky read

Section 0 · Answer key

#	Answer	Why and where to revisit
1	B) strict	strict enables a bundle of safety flags at once (noImplicitAny, strictNullChecks, strictFunctionTypes, and more). It's the single highest-leverage setting; enabling individual flags piecemeal almost always leaves gaps.
2	B) K must be a key that actually exists on T	extends keyof T constrains K to the union of T's actual keys. This is what makes test helpers reject typo'd field names at compile time instead of at runtime.
3	A) Interfaces support declaration merging; `type` does not	Declaration merging lets multiple `interface` declarations of the same name combine. Useful for extending response shapes per test, augmenting third-party types, and avoiding alias chains. `type` is otherwise often equivalent.
4	A) getByRole	getByRole maps to the accessibility tree, which is the most stable surface for interactive elements: it survives styling changes, framework upgrades, and most refactors. getByTestId is the fallback when role/name doesn't disambiguate.
5	D) Add an accessible-name filter via `{ name: '...' }`	Multiple buttons matched. Adding `{ name: 'Save' }` narrows to the intended one and preserves the strict-mode safety. `.first()` papers over the ambiguity and the test breaks the moment button order changes.
6	A) `has` filters by descendant locator; `hasText` filters by text content	filter({ has: page.locator('.warning') }) finds rows that contain a warning element; filter({ hasText: 'Pending' }) finds rows whose text contains 'Pending'. Different criteria, both essential for narrowing collections.
7	B) Visible, stable, receives events and enabled	click runs four of the five: visible, stable, receives events and enabled. The set is per action, not one fixed list. fill runs visible, enabled and editable only, and focus, press and dispatchEvent run no checks at all. That is why explicit waits are rarely needed for a click, and why a fill still succeeds under an overlay.
8	D) Replace with `expect(modal).toBeVisible()`	Web-first assertions auto-retry until the condition holds or the timeout fires. waitForTimeout is the classic anti-pattern: too long means slow tests, too short means flake, and it never matches the actual readiness of the element.
9	D) A one-off verification of a specific button on a single test	POMs amortise across reuse. A method called once doesn't earn its place in a POM; the indirection costs more than it saves. Reach for POMs when the abstraction is shared across multiple tests.

10	A) expect(el).toBeVisible()	Web-first assertions like toBeVisible(), toHaveText(), toHaveURL() auto-retry. The first option awaits a one-shot boolean and produces flake. Web-first is the discipline that eliminates a whole class of timing bugs.
11	B) expect(items).not.toHaveCount(0)	not.toHaveCount(0) is web-first and auto-retries. The first option captures count at a single moment and races; allInnerTexts() doesn't retry; waitFor() waits but doesn't assert.
12	A) BrowserContext	BrowserContext is a fresh, isolated browser session, cookies, localStorage, sessionStorage, cache, created per test by default. Pages within a context share state; contexts don't.
13	B) page.route() inside the test	page.route() is scoped to one page within one test. context.route() applies to every page in the context (good for cross-test setup). Choose the narrowest scope that meets the need.
14	A) It's 10x+ faster than UI flows and doesn't depend on form/field stability	API setup avoids the UI's loading/render cost AND the brittleness of selectors. UI setup is the right tool for testing the UI flow itself; for arranging test data, API is dramatically faster and more reliable.
15	D) Set the threshold above the worst-case p99 (e.g. 800 ms) to absorb variance	SLA assertions should be soft upper bounds, not happy-path values. The threshold absorbs CI variance and fails only on real regression. Tight thresholds produce flake; threshold-against-p99 produces signal.
16	B) 6-8 (more than cores; browser tests are I/O-bound)	Playwright tests wait on browser/network/app most of the time, so workers > cores wins. The ceiling is memory (200-400 MB per browser), not CPU. Measure 4 vs 6 vs 8 on the actual runner; pick where wall-clock plateaus.
17	B) Workers parallelise within one CI job; shards parallelise across CI jobs	Workers = within-process parallelism on one machine. Shards = splitting the suite across multiple CI jobs (parallel machines). Use both: shard count × worker count = total parallelism.
18	D) DEBUG=pw:api	DEBUG=pw:api logs every API call with timing, the workhorse for understanding CI failures. PWDEBUG=1 is for live interactive debugging; pw:protocol is raw CDP traffic, only for framework-level bugs.
19	C) scope: 'worker'	Worker-scoped fixtures run once per worker process and persist across all tests on that worker. Test-scope would re-seed for every test (slow); 'browser' and 'global' don't exist as Playwright fixture scopes.
20	B) Stubs control returned state; mocks verify behaviour (that calls happened as expected)	Stub = canned response, no verification. Mock = expectations about calls; the verification is built in. Most teams say 'mock' for everything and lose the distinction between state stubbing and behaviour verification.

21	C) Hand-label ~50 examples and confirm 80%+ agreement between the judge and humans	Without calibration you don't know if the judge is too lenient, too strict, or inconsistent. A hand-labelled dataset gives a measured agreement bar; only then do you trust it as a CI gate.
22	C) Annotations for tags/metadata; attach for files and binary diagnostic data	Annotations carry structured metadata (requirement IDs, owner, severity) for reporters to aggregate. attach embeds files (API bodies, generated CSVs, screenshots) for human review. Different purposes; both flow through reporters.
23	C) 20 cases	Pairwise coverage, every value-pair from any two parameters appears in at least one test, reduces 240 to about 20 while catching most multi-parameter defects (NIST research: most bugs are 1- or 2-way interactions).
24	A) Defect escape rate (bugs reaching production)	Defect escape rate is the direct inverse of test effectiveness, fewer escapes means testing is working. Test count, coverage %, and passing-count are vanity metrics that measure activity, not outcomes.
25	D) Playwright's WebKit ≠ iOS Safari: divergent rendering, memory limits, and CSS support	Playwright's WebKit is a related build, not Apple's iOS Safari. They diverge in memory limits, video playback, private browsing, and certain CSS features. Real-device testing on BrowserStack/SauceLabs catches what emulation misses.
26	C) Tier 1 (consumer-side) on every PR; Tier 2 (provider verification) on provider PRs; Tier 3 (e2e) on integration	Consumer contracts are fast (seconds) and gate every PR. Provider verification gates provider PRs. End-to-end with real services gates merge to main. Most interaction bugs surface at tier 1: e2e is the slower backstop.
27	D) Under 2.5 seconds	LCP < 2.5s = Good, 2.5-4.0s = Needs Improvement, > 4.0s = Poor. INP < 200ms (replaced FID in 2024), CLS < 0.1. Measure on a representative environment (throttled network + CPU), not on the dev machine.
28	C) Assert viewer CANNOT delete (negative path expecting 403)	Positive-only testing passes when everyone can delete, including viewer. The negative assertion, viewer DELETE returns 403, is what catches broken authorisation. Cover the matrix completely; the holes are where vulnerabilities live.
29	B) Prevent the argument-swap bug where UserId and TenantId (both strings) get passed in wrong places	Branded types distinguish values that share an underlying primitive but represent different domains. The compiler refuses to assign one brand to another, catching the same-primitive argument-swap bug at compile time.
30	A) Use ConsistentRead: true on the read, OR poll with expect.poll until the read converges	DynamoDB defaults to eventual consistency. ConsistentRead: true forces a strong read; expect.poll handles the case where you're testing eventual-consistency behaviour itself. waitForTimeout is the anti-pattern from auto-waiting.

End of the Q&A; Interview Kit

325 questions across 28 sections.

Ready for the next step?
The Ultimate SDET Playwright TypeScript Interview Kit (Parts 1, 2, 3) takes you deeper into framework architecture, AI for QA, and the test-architect track.